Contents

Proven C Library Complete Manual (v0.97.0) 6
Contents 6
Copyright 6
Tutorial: the library in nine short programs 7
Why this exists 7
Table of contents 7
Lesson 1 - printing, and a build that works 7
Lesson 2 - text that knows how long it is 8
Lesson 3 - a call that can fail says so 9
Lesson 4 - the value and the error arrive together 10
Lesson 5 - who gives out the memory is an argument 11
Lesson 6 - the Chapter 0 program, read line by line 12
Lesson 7 - memory that is freed all at once 13
Lesson 8 - a container remembers its allocator 14
Lesson 9 - the outside world fails too, and says so the same way 16
Where to go next 17
Chapter 0: Start Here 18
Table of contents 18
1. Who this manual is for 18
2. Why this library exists 18
3. Your first program 21
4. Building and including 23
5. The five contracts you will meet on every page 23
6. Intent and design philosophy 25
7. Build and include model 25
8. Global contracts 26
9. Ownership and destruction matrix 27
10. Operation behavior classes 31
11. Manual chapters 32
12. Platform support and verification 34
13. Appendix B: glossary 35
14. Appendix C: public header map 39
15. Appendix D: the libc map 42
16. Where to go next 43
Chapter 1: Foundation - Errors, Types, and Memory Views 45
Table of contents 45
1. Errors are values 45
2. The types, and why they are spelled differently 50
3. Memory views: a pointer and a length, together 51
4. Size arithmetic that cannot wrap 54
5. Alignment 55
6. Panic: when there is no one left to return an error to 59
7. Version macros 60
8. Examples and misuse cases 60
Chapter 2: Allocation - Heap, Arenas, Pools, and Buffers 62
Table of contents 62
1. Why allocation is a parameter, and the heap allocator 62
2. Arena: many things, one lifetime 63
3. Pool: many things, one size 66

4. The allocator trait 68
5. Raw byte buffer 70
6. Examples and misuse cases 71
7. Catching the wrong allocator: `alloc_check.h` 78
Chapter 3: Strings, Formatting, and Scanning 82
Table of contents 82
1. U8 strings and views 82
2. U16 strings and views 90
3. Formatting 95
4. Scanning 98
5. Examples and misuse cases 101
Chapter 4: Containers and Algorithms 112
Table of contents 112
1. Dynamic array 112
2. Intrusive list 115
3. Ring buffer 118
4. Hash map 120
5. Algorithms 124
6. Hashing, by use case 125
7. Bytes to text: hex and Base64 133
8. Examples and misuse cases 138
Chapter 8: Formatting and Scanning (v0.97.0) 149
Table of contents 149
1. Design model 149
2. Formatter data model 150
3. Formatter constructors and selectors 152
4. Format string grammar 157
5. Formatting APIs 158
6. Console print helpers 162
7. Scanner data model 163
8. Scanner primitive APIs 163
9. Scan argument model 165
10. Structural scan grammar 165
11. Scan formatting APIs 166
12. Examples and misuse cases 167
13. Freestanding and build-mode notes 172
Chapter 5: Hosted Services 181
Table of contents 181
1. Filesystem API 181
2. System I/O and environment 188
3. Memory mapping 189
4. Time API 191
5. Examples and misuse cases 194
Reading a directory one entry at a time 199
Walking a tree 200
Streams: writers and readers 202
The standard streams 207
UTF-16 text in and out, and the Windows console 209
Randomness, by use case 213
Chapter 9: Networking 234

Table of contents 234
1. Three rules 234
2. Addresses 235
3. Deadlines 237
4. TCP 238
5. UDP 241
6. Many sockets, one thread 243
7. Transports 246
8. A pair, and waking a loop 248
9. A selector: thousands of sockets 250
10. What the platforms do differently 254
11. What is not here 255
Chapter 10: URLs and HTTP Messages 256
Table of contents 256
1. A codec, not a server 256
2. URLs 257
3. A path that is safe to use 260
4. Parsing a head 261
5. How long is the body 262
6. Decoding a body 263
7. Writing a message 266
8. Dates 268
9. References and forms 269
10. Ranges and multipart bodies 269
11. Authentication 271
12. Cookies 274
13. Server-sent events 274
14. What is not here 277
Chapter 11: An HTTP Client and Server 278
Table of contents 278
1. What these are, and the one thing they are not 278
2. A server 278
3. Inside a handler 281
4. What the server does without asking you 283
5. Two ways to run handlers 284
6. A client 287
7. One request, one response 288
8. Redirects, challenges and cookies 290
9. Connections and proxies 291
10. Where TLS will attach 292
11. Changing protocol 296
12. What is not here 299
Chapter 12: WebSocket 300
Table of contents 300
1. What WebSocket is, in one page 300
2. A connection 301
3. Sending and receiving 302
4. Closing, and being closed on 304
5. Where a server's connections live 305
6. The codec: the handshake 308

7. The codec: frames 309
8. The codec: the decoder 310
9. Compressed messages 313
10. What is not here 324
Chapter 13: Certificates and trust 325
Table of contents 325
1. What a certificate is, in one page 325
2. Reading one 326
3. PEM 327
4. Which names it is for 327
5. Trust anchors 327
6. Verifying a chain 328
7. Pinning 332
8. What is checked, what is accepted, what is not here 332
Chapter 14: TLS 334
Table of contents 334
1. What TLS does, in one page 334
2. A configuration 335
3. HTTPS and wss: the two lines 337
4. Any connection: the transport wrapper 340
5. The engine 341
6. Resumption 345
7. Client certificates and pins 349
8. When it fails 350
9. What is negotiated, what is held, what is not here 351
10. The legacy set: what each setting exposes you to 354
Chapter 15: The event loop and the event-driven server 359
Table of contents 359
1. Two ways to serve 359
2. A loop 360
3. Timers 361
4. Work that is done elsewhere 362
5. A socket of your own 364
6. The event-driven server 370
7. Answering 372
8. Request bodies 377
9. Limits, time and refusals 377
10. Threads, TLS and shutting down 379
11. WebSocket on the loop 380
12. The event-driven client 393
13. Several loops 403
14. What a connection holds, what was measured, and what is not here 406
Chapter 16: Compression 410
Table of contents 410
1. What DEFLATE does, in one page 410
2. A whole buffer 411
3. A stream 412
4. Levels, windows and what they cost 416
5. Data from somebody else 416
6. Over HTTP 417

7. Zstandard 426
8. Brotli 433
9. What is accepted, what is not here, and how this was tested 436
Chapter 17: HTTP/2 - frames and header compression 439
Table of contents 439
1. What HTTP/2 changes, in one page 439
2. Header compression 440
3. Frames 444
4. A header list as a message 449
5. A connection 450
6. HTTP/2 in the event-driven server 459
7. HTTP/2 in the event-driven client 465
8. What is accepted, what is not here, and how this was tested 469
9. HTTP/3 in the event-driven server 472
10. HTTP/3 in the event-driven client 475
Chapter 6: Execution, Aliases, PAL, Freestanding, and Cross Builds 477
Table of contents 477
1. Stackless coroutines 477
2. Job system 478
3. Threads, allocators, and pointer provenance 481
4. Alias layer 484
5. PAL contract 485
6. Freestanding subset 486
7. Cross compilation 486
8. Examples and misuse cases 487
Proven Freestanding Mode (v0.97.0) 492
0. Why this mode exists, and why the whole library is shaped by it 492
1. Build profile 493
2. Available source files 493
3. Module availability 494
4. Minimal static arena setup 499
5. Panic handler override 501
6. Example Cortex-M compile command 501
7. Example RISC-V ELF compile command 502
8. Formatting in freestanding mode 502
8a. Tuning the float big-integer capacity 503
9. Avoid hosted APIs 503
10. Lifetime rules still apply 504
11. Verification 504
Appendix A: `alias_xcv.h` Alias Index (Chapter 7) 506
Usage rule 506
Alias table 506

Proven C Library Complete Manual (v0.97.0)

The manual of `proven`, a C23 systems library. Every chapter is one file under `manual/`; the Korean edition mirrors it under `manual-ko/`.

Contents

- Tutorial - the library in nine short programs
- Chapter 0 - Start here
- Chapter 1 - Foundation
- Chapter 2 - Allocation
- Chapter 3 - Strings and text
- Chapter 4 - Containers and algorithms
- Chapter 8 - Formatting and scanning
- Chapter 5 - Hosted services
- Chapter 9 - Networking
- Chapter 10 - URLs and HTTP messages
- Chapter 11 - An HTTP client and server
- Chapter 12 - WebSocket
- Chapter 13 - Certificates and trust
- Chapter 14 - TLS
- Chapter 15 - The event loop and the event-driven server
- Chapter 16 - Compression
- Chapter 17 - HTTP/2: frames and header compression
- Chapter 6 - Execution and platform
- Freestanding
- Appendix A - Alias index

Copyright

- Author: rubidus - rubidus@gmail.com

- Repository: github.com/rubidus-api/proven_c_lib
- Edition: v0.97.0

- License: MIT - Copyright (c) 2026 rubidus-api. The library and this manual are under the same license; the text is in `LICENSE` at the repository root.

- Every code example in this manual is compiled, and the quoted ones are run, by the build.

Tutorial: the library in nine short programs

Part I - Start here. No prerequisites beyond one introductory C book. **After lesson 6** you can read the greeting program in Chapter 0 line by line, and the reference chapters stop looking like a wall of new words. **After lesson 9** you have used an arena, a growable container and a file - the first things most programs need once they can handle text.

Why this exists

Chapter 0 shows a working program on its first page. That program is thirty lines long and every one of them is a new idea: an allocator you pass in, a result you have to check, a view that carries its length, an append that refuses rather than truncates, a destroy paired with the allocator that created it. If you have finished one C book and no more, five new ideas arriving together is four too many.

So this tutorial takes them one at a time. Nine programs, each introducing exactly one thing. The first six lead up to the Chapter 0 program; the last three go past it, into arenas, containers and files. Every one of them is a real file under `manual/examples/` that the build compiles and runs, so what you read here is what actually ran.

Build any of them the way you build any C program - the library is source you compile with your own code, so there is nothing to install:

```
cc -std=c23 -Iinclude your_program.c src/proven/*.c platform/*.c -o your_program
```

If that line is unfamiliar, read Chapter 0 section 4 first and come back.

Table of contents

1. Lesson 1 - printing, and a build that works
2. Lesson 2 - text that knows how long it is
3. Lesson 3 - a call that can fail says so
4. Lesson 4 - the value and the error arrive together
5. Lesson 5 - who gives out the memory is an argument
6. Lesson 6 - the Chapter 0 program, read line by line
7. Lesson 7 - memory that is freed all at once
8. Lesson 8 - a container remembers its allocator
9. Lesson 9 - the outside world fails too, and says so the same way
10. Where to go next

Lesson 1 - printing, and a build that works

The one new thing: `proven_println`, and the fact that its arguments are checked.

`printf("%d", 3.5)` compiles. It is wrong, and on a bad day it prints garbage or crashes, because the format string is a *string* - nothing connects it to the arguments that follow. `proven_println` uses `{}` as the placeholder and wraps each argument in `PROVEN_ARG`, which records the argument's type. The two cannot disagree, because the type travels with the value.

```
/*
 * Lesson 1 - get one line to build and run.
 *
 * Nothing here is about the library's ideas yet. The only question this program
 * answers is "does my build command work?", and it is worth answering on its
 * own, because every later lesson assumes it.
 *
 * printf(fmt, ...) is not checked: the compiler cannot tell you that "%d" was
 * handed a double. proven_println checks, because every argument is wrapped by
 * PROVEN_ARG and carries its own type.
```

```
 */

int main(void) {
    proven_println("hello from proven");

    /* {} is the placeholder. The value goes through PROVEN_ARG, which records
     * what type it is, so the format string and the argument cannot disagree. */
    proven_println("one number: {}", PROVEN_ARG(42));
    proven_println("two of them: {} and {}", PROVEN_ARG(1), PROVEN_ARG(2));

    return EXAMPLE_OK();
}
```

Running it prints three lines. If it built and ran, your include path and your source list are right, and everything after this is about the library rather than about your build.

Common first stumble. `{}` is not `%s`. There is no letter to get wrong, and there is nothing to remember about `long` versus `int`: `PROVEN_ARG` knows.

Lesson 2 - text that knows how long it is

The one new thing: the *view* - a pointer and a size that travel together.

In the C you know, a string is a pointer, and its length is wherever the first zero byte happens to be. Every function that touches it has to walk the bytes to find out how much there is; if the zero is missing, it walks off the end. That single design decision is behind a large share of the security advisories in the language's history.

A view makes the missing half explicit. It does not own the bytes - it borrows them, and it never frees anything.

```
/*
 * Lesson 2 - text that knows how long it is.
 *
 * In the C you already know, a string is a pointer and the length is wherever
 * the first zero byte happens to be. Every function has to walk the bytes to
 * find out how much there is, and if the zero is missing it walks off the end.
 *
 * A view is the pair that C leaves implicit: a pointer AND a size, travelling
 * together. It borrows - it does not own the bytes and never frees them.
 */

int main(void) {
    /* PROVEN_LIT makes a view from a literal. The size is computed at compile
     * time, so no scan happens here - unlike strlen. */
    proven_u8str_view_t hello = PROVEN_LIT("hello");

    EXAMPLE_REQUIRE(hello.size == 5, "the view already knows its own length");
    EXAMPLE_REQUIRE(hello.ptr != NULL, "and it points at the literal's bytes");

    /* Because the length travels with the pointer, comparing is a size check
     * plus a memcmp. No walking, and no way to run off the end. */
    EXAMPLE_REQUIRE(proven_u8str_view_eq(hello, PROVEN_LIT("hello")), "same text");
    EXAMPLE_REQUIRE(!proven_u8str_view_eq(hello, PROVEN_LIT("hell")), "shorter text differs");

    /* A view can name PART of something without copying it. Here is the middle
     * of the literal - still borrowed, still no allocation. */
    proven_u8str_view_t ell = { .ptr = hello.ptr + 1, .size = 3 };
    EXAMPLE_REQUIRE(proven_u8str_view_eq(ell, PROVEN_LIT("ell")), "a window onto the same bytes");
```

```
    proven_println("whole: {} (size {})", PROVEN_ARG(hello), PROVEN_ARG(hello.size));
    proven_println("part : {} (size {})", PROVEN_ARG(ell), PROVEN_ARG(ell.size));

    return EXAMPLE_OK();
}
```

What to notice. `PROVEN_LIT` computes the size at compile time, so it is not `strlen`. And the third view in the program names the *middle* of the literal without copying it: a view can describe part of something, which is why splitting and parsing in this library allocate nothing.

Common first stumble. A view is borrowed. If the bytes it points at go away - a local buffer that leaves scope, a string you destroyed - the view is dangling, exactly like any other C pointer. The library never extends a lifetime behind your back.

Lesson 3 - a call that can fail says so

The one new thing: `proven_err_t`, and refusal instead of truncation.

C reports failure in three unrelated ways: a magic return value, a null pointer, or the global `errno` that the next call overwrites. All three are easy not to look at, and nothing complains when you do not. Here, a call that can fail returns an error *value* - and these functions are marked `[[nodiscard]]`, so throwing it away is a compiler warning rather than a habit.

```
/*
 * Lesson 3 - a call that can fail says so, in its return value.
 *
 * The C you know reports failure in three different ways: a magic return value
 * (-1), a null pointer, or a global called errno that the next call overwrites.
 * All three are easy to not look at, and nothing complains when you don't.
 *
 * Here a fallible call returns proven_err_t. It is an ordinary value: you can
 * store it, compare it, and pass it upward. And [[nodiscard]] on these
 * functions means ignoring it is a compiler warning, not a habit.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* Room for exactly 8 bytes. (Chapter 2 is about where that room comes
     * from; for now, notice only that we said how much.) */
    proven_result_u8str_t s = proven_u8str_create(alloc, 8);
    EXAMPLE_REQUIRE(proven_is_ok(s.err), "8 bytes should be available");

    /* This fits. */
    proven_err_t err = proven_u8str_append(&s.value, PROVEN_LIT("12345678"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "eight bytes into eight bytes fits exactly");

    /* This does not - and the library REFUSES it. It does not append the part
     * that would fit, because half a word is not a shorter word, it is a
     * different one. */
    proven_err_t too_much = proven_u8str_append(&s.value, PROVEN_LIT("9"));
    EXAMPLE_REQUIRE(!proven_is_ok(too_much), "one byte more than capacity must fail");
    EXAMPLE_REQUIRE(too_much == PROVEN_ERR_OUT_OF_BOUNDS, "and it says why: out of bounds");

    /* The refusal changed nothing. The string is exactly as it was, which is
     * what "failure-atomic" means: a failed call leaves no half-done state. */
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&s.value), PROVEN_LIT("12345678")),
                    "the refused append must not have written anything");

    proven_println("after the refusal, the string is still: {}",
```

```
                PROVEN_ARG(proven_u8str_as_view(&s.value)));

    proven_u8str_destroy(alloc, &s.value);
    return EXAMPLE_OK();
}
```

What to notice. The append that does not fit changes *nothing*. It does not store the part that would have fit. This is called failure atomicity, and the reason for it is that a truncated string is not a shorter message, it is a different one - a truncated path names a different file, and a truncated command is a different command.

Common first stumble. `proven_is_ok(err)` is the check; `err == PROVEN_OK` says the same thing. What you must not do is ignore it and read the value anyway.

Lesson 4 - the value and the error arrive together

The one new thing: result structs - `.err` and `.value` in one return.

When a call has nothing to give back, an error code is enough. When it does have something, the library returns both in one small struct, and the rule is one sentence: `value` **means nothing until you have looked at** `err`**.**

```
/*
 * Lesson 4 - when the call has a value to give back, the value and the error
 * arrive together.
 *
 * proven_err_t alone is enough when there is nothing to return. When there IS
 * something, you get a small struct with two fields: `err` and `value`. The
 * rule is one sentence: `value` means nothing until you have looked at `err`.
 *
 * This is the same discipline as checking malloc for NULL, except the checking
 * place is part of the type instead of a convention you have to remember.
 */

/* A function of your own can return one too. Nothing in the library is magic. */
static proven_result_size_t safe_div(proven_size_t a, proven_size_t b) {
    proven_result_size_t res = {0};
    if (b == 0) {
        res.err = PROVEN_ERR_INVALID_ARG;    /* value stays 0 - and means nothing */
        return res;
    }
    res.err = PROVEN_OK;
    res.value = a / b;
    return res;
}

int main(void) {
    proven_result_size_t ok = safe_div(10, 2);
    EXAMPLE_REQUIRE(proven_is_ok(ok.err), "dividing by 2 is fine");
    EXAMPLE_REQUIRE(ok.value == 5, "and only now may we read the value");

    proven_result_size_t bad = safe_div(10, 0);
    EXAMPLE_REQUIRE(!proven_is_ok(bad.err), "dividing by zero must fail");
    EXAMPLE_REQUIRE(bad.err == PROVEN_ERR_INVALID_ARG, "and say which rule was broken");
    /* bad.value is 0, but that is not an answer. It is the absence of one. */

    /* The library's own calls have the same shape. proven_u8str_create hands
     * back the string it made together with the error that guards it. */
    proven_allocator_t alloc = proven_heap_allocator();
    proven_result_u8str_t s = proven_u8str_create(alloc, 16);
```

```
    if (!proven_is_ok(s.err)) return 1;      /* nothing was created; nothing to destroy */

    proven_println("10 / 2 = {}", PROVEN_ARG(ok.value));

    proven_u8str_destroy(alloc, &s.value);
    return EXAMPLE_OK();
}
```

What to notice. `safe_div` is *your* function, not the library's. The pattern is a plain struct return; there is no machinery underneath, which is why you can use it in your own code the moment you have read this page.

Common first stumble. On failure the `value` field usually holds a zero. That zero is not an answer - it is the absence of one. Checking `err` is what tells them apart.

Lesson 5 - who gives out the memory is an argument

The one new thing: `proven_allocator_t`, passed in rather than assumed.

`malloc` is a decision made for you: one heap, one strategy, and nothing at the call site says so. In this library, anything that needs memory takes an allocator and uses only that one. Two things follow, and both are the point: you can always answer "who allocated this?" by reading the call, and you can hand the same code a different allocator without changing it.

```
/*
 * Lesson 5 - who gives out the memory is an argument, not a global.
 *
 * malloc is a global decision made for you: one heap, one strategy, invisible
 * at the call site. Here, anything that needs memory takes a
 * proven_allocator_t and uses only that. Two consequences follow, and both are
 * the point.
 *
 *   - You can always answer "who allocated this?" by looking at the call.
 *   - You can hand a different allocator to the same code without changing it.
 */

/* This function does not know or care where the memory comes from. */
static proven_result_u8str_t make_greeting(proven_allocator_t alloc,
                                           proven_u8str_view_t name) {
    proven_result_u8str_t out = proven_u8str_create(alloc, 64);
    if (!proven_is_ok(out.err)) return out;

    proven_err_t err = proven_u8str_append(&out.value, PROVEN_LIT("hello, "));
    if (proven_is_ok(err)) err = proven_u8str_append(&out.value, name);
    if (!proven_is_ok(err)) {
        proven_u8str_destroy(alloc, &out.value);   /* undo what we made */
        out.err = err;
        out.value = (proven_u8str_t){0};
    }
    return out;
}

int main(void) {
    /* (a) the ordinary heap - malloc and free underneath */
    proven_allocator_t heap = proven_heap_allocator();

    proven_result_u8str_t a = make_greeting(heap, PROVEN_LIT("world"));
    EXAMPLE_REQUIRE(proven_is_ok(a.err), "the heap should be able to give 64 bytes");
    proven_println("from the heap : {}", PROVEN_ARG(proven_u8str_as_view(&a.value)));
    /* Destroyed with the SAME allocator that created it. That pairing is the
```

```
     * whole ownership rule of this library. */
    proven_u8str_destroy(heap, &a.value);

    /* (b) an arena - one block of memory handed out in order, freed all at once.
     *     Note that make_greeting above did not change at all. */
    alignas(PROVEN_MAX_ALIGN) proven_byte_t backing[512];
    proven_arena_t arena = proven_arena_create((proven_mem_mut_t){
        .ptr = backing, .size = sizeof backing });
    proven_allocator_t from_arena = proven_arena_as_allocator(&arena);

    proven_result_u8str_t b = make_greeting(from_arena, PROVEN_LIT("arena"));
    EXAMPLE_REQUIRE(proven_is_ok(b.err), "the arena has room for this too");
    proven_println("from an arena : {}", PROVEN_ARG(proven_u8str_as_view(&b.value)));

    /* No destroy loop here: an arena frees everything by being reset. That is
     * lesson 7's subject; the point now is only that the caller chose. */
    proven_arena_reset(&arena);

    return EXAMPLE_OK();
}
```

What to notice. `make_greeting` is written once and runs against the heap and against an arena -- a block of memory you own, handed out by bumping a pointer, and released all at once. The function did not change, and nothing global was configured.

The ownership rule, in one line: whatever created it destroys it, with the *same* allocator.

Common first stumble. With an arena, `destroy` reclaims nothing - an arena frees by being reset. Call it anyway: the pairing is what makes the code correct when the allocator later changes.

Lesson 6 - the Chapter 0 program, read line by line

The one new thing: nothing. That is the exercise.

```
/*
 * Lesson 6 - the program from Chapter 0, read line by line.
 *
 * Nothing new is introduced here. This is the same greeting program the manual
 * opens with, and the point of the lesson is that you can now name every part
 * of it: the allocator you pass in (lesson 5), the result you must check
 * (lesson 4), the error a refusal returns (lesson 3), the view that carries its
 * own length (lesson 2), and the printing that checks its arguments (lesson 1).
 *
 * If that reads as ordinary now, the tutorial has done its job and the
 * reference chapters are open to you.
 */

int main(void) {
    /* lesson 5 - the caller decides where memory comes from */
    proven_allocator_t alloc = proven_heap_allocator();

    /* lesson 4 - the string and the error that guards it, together */
    proven_result_u8str_t greeting = proven_u8str_create(alloc, 64);
    if (!proven_is_ok(greeting.err)) return 1;

    /* lesson 2 - borrowed text that knows its own size */
    proven_u8str_view_t name = PROVEN_LIT("world");

    /* lesson 3 - each append either fits or refuses; none of them truncates */
    proven_err_t err = proven_u8str_append(&greeting.value, PROVEN_LIT("hello, "));
```

```
    if (proven_is_ok(err)) err = proven_u8str_append(&greeting.value, name);
    if (proven_is_ok(err)) err = proven_u8str_append(&greeting.value, PROVEN_LIT("!"));

    if (!proven_is_ok(err)) {
        proven_u8str_destroy(alloc, &greeting.value);
        return 1;
    }

    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&greeting.value),
                                         PROVEN_LIT("hello, world!")),
                    "the three appends should have built the whole greeting");

    /* lesson 1 - the format string and the argument cannot disagree */
    proven_println("{}", PROVEN_ARG(proven_u8str_as_view(&greeting.value)));

    /* lesson 5 again - destroyed with the allocator that created it */
    proven_u8str_destroy(alloc, &greeting.value);
    return EXAMPLE_OK();
}
```

If those comments now read as labels for things you know rather than as new information, you are ready for the reference chapters. The next three lessons take the first steps into them - arenas, containers and files - in the same shape: one new idea each.

Lesson 7 - memory that is freed all at once

The one new thing: `proven_arena_reset` - one call that gives back everything an arena handed out.

Lesson 5 passed an arena to `make_greeting` and reset it at the end without saying what the reset did. Here is the whole idea. An arena hands out pieces of a block of memory you own by moving a pointer forward. It has no way to give back one piece - `destroy` on an arena reclaims nothing. It gives back *all* of them, at once, when you reset it.

That fits a shape most programs have: a loop where each round builds some temporary text, uses it, and is finished with it. Allocate freely during the round, reset at the end, and there is nothing to track and nothing to leak.

```
/*
 * Lesson 7 - memory that is freed all at once.
 *
 * Lesson 5 handed an arena to make_greeting and then reset it, without saying
 * what the reset was. This lesson is about that one call.
 *
 * An arena bumps a pointer through a block of memory you own. There is no
 * per-object free: destroy on an arena reclaims nothing. Everything comes back
 * at once, with proven_arena_reset. That sounds like a restriction, and it is
 * the reason to use one - a loop that builds scratch text for each piece of
 * work can drop all of it with one statement, however many strings it made.
 */

int main(void) {
    /* The arena does not own this array - we do. It only hands out pieces. */
    alignas(PROVEN_MAX_ALIGN) proven_byte_t backing[256];
    proven_arena_t arena = proven_arena_create((proven_mem_mut_t){
        .ptr = backing, .size = sizeof backing });
    proven_allocator_t scratch = proven_arena_as_allocator(&arena);

    static const proven_u8str_view_t names[] = {
        PROVEN_LIT("ada"), PROVEN_LIT("grace"), PROVEN_LIT("barbara"),
    };
```

```
    for (proven_size_t i = 0; i < sizeof names / sizeof names[0]; ++i) {
        /* Two strings per round, both out of the arena. */
        proven_result_u8str_t line = proven_u8str_create(scratch, 32);
        proven_result_u8str_t note = proven_u8str_create(scratch, 32);
        EXAMPLE_REQUIRE(proven_is_ok(line.err) && proven_is_ok(note.err),
                        "each round fits easily in 256 bytes");
        if (!proven_is_ok(line.err) || !proven_is_ok(note.err)) return 1;

        proven_err_t err = proven_u8str_append(&line.value, PROVEN_LIT("hello, "));
        if (proven_is_ok(err)) err = proven_u8str_append(&line.value, names[i]);
        if (proven_is_ok(err)) err = proven_u8str_append(&note.value, PROVEN_LIT("(scratch)"));
        EXAMPLE_REQUIRE(proven_is_ok(err), "the appends fit their capacity");

        /* The first allocation of every round lands at the start of `backing`:
         * the reset at the end of the previous round gave all of it back. */
        EXAMPLE_REQUIRE((const void *)proven_u8str_as_view(&line.value).ptr == (const void *)backing,
                        "each round starts again at the beginning of the block");

        proven_println("round {}: {} {} ({} bytes in use)",
                       PROVEN_ARG(i), PROVEN_ARG(proven_u8str_as_view(&line.value)),
                       PROVEN_ARG(proven_u8str_as_view(&note.value)),
                       PROVEN_ARG(arena.offset));

        /* One statement, and both strings are gone. No destroy loop, nothing to
         * forget. `line` and `note` point at reclaimed memory from here on. */
        proven_arena_reset(&arena);
        EXAMPLE_REQUIRE(arena.offset == 0, "reset reclaims everything the round allocated");
    }

    /* An arena cannot grow. Asking for more than the block holds is an error
     * value, the same kind lesson 3 showed - not a crash, and not a silent
     * fallback to malloc. */
    proven_result_u8str_t too_big = proven_u8str_create(scratch, 1024);
    EXAMPLE_REQUIRE(too_big.err == PROVEN_ERR_NOMEM, "a 256-byte arena refuses 1 KiB");

    proven_arena_destroy(&arena);
    return EXAMPLE_OK();
}
```

What to notice. Every round reports the same number of bytes in use, and the first string of every round lands at the start of `backing` - the program checks that. The reset did not free two strings; it moved one number back to zero, which costs the same whether the round made two strings or two thousand. And when the arena is too small the answer is `PROVEN_ERR_NOMEM`, an error value like lesson 3's, not a crash.

Common first stumble. After the reset, `line` and `note` still hold their old pointers, and those pointers now point at memory the next round will overwrite. A reset leaves every pointer the arena ever gave out dangling, all at once. Keep nothing from a round that you have not copied out of the arena first.

Lesson 8 - a container remembers its allocator

The one new thing: a container takes the allocator once, at creation, and keeps it.

The string in lesson 3 had a fixed room, and you passed the allocator both to `create` and to `destroy`. A growable array cannot work like that: a push that outgrows its room must allocate again, at a moment the caller did not plan for. So the array keeps the allocator it was created with. You hand it over once; after that, push, get and destroy take only the array.

```c
/*
 * Lesson 8 - a container remembers its allocator.
 *
 * A string is created with an allocator and destroyed with the same one; you
 * pass it both times. A growable array has to allocate again later, whenever a
 * push outgrows its storage, so it keeps the allocator it was created with.
 * That is the one new thing: you hand the allocator over once, at creation,
 * and after that push, get and destroy take nothing but the array.
 *
 * The PROVEN_ARRAY_* macros take the element type as an argument and build the
 * value as that type, so a value with no conversion to it - a struct pushed
 * into an array of int - does not compile. They cannot tell whether the type
 * you name is the one the array was created with: name the same one each time.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* 2 is a starting capacity, not a limit. */
    proven_result_array_t made = PROVEN_ARRAY_INIT(alloc, int, 2);
    EXAMPLE_REQUIRE(proven_is_ok(made.err), "creating an empty array must succeed");
    if (!proven_is_ok(made.err)) return 1;
    proven_array_t squares = made.value;

    /* Ten pushes into room for two. Each push that does not fit grows the
     * storage through the allocator the array remembered - no allocator here. */
    for (int i = 1; i <= 10; ++i) {
        proven_err_t err = PROVEN_ARRAY_PUSH(&squares, int, i * i);
        EXAMPLE_REQUIRE(proven_is_ok(err), "the heap can grow a ten-int array");
        if (!proven_is_ok(err)) {
            PROVEN_ARRAY_DESTROY(&squares);
            return 1;
        }
    }
    EXAMPLE_REQUIRE(squares.len == 10, "ten pushes, ten elements");

    /* get returns a pointer to the element, or NULL when the index is past the
     * end. An index out of range is an answer you can test, not a wild read. */
    const int *third = PROVEN_ARRAY_GET(&squares, int, 2);
    EXAMPLE_REQUIRE(third && *third == 9, "element 2 is 3 * 3");
    EXAMPLE_REQUIRE(PROVEN_ARRAY_GET(&squares, int, 10) == NULL, "index 10 is past the end");

    int sum = 0;
    for (proven_size_t i = 0; i < squares.len; ++i) {
        sum += *PROVEN_ARRAY_GET(&squares, int, i);
    }
    EXAMPLE_REQUIRE(sum == 385, "1 + 4 + 9 + ... + 100");
    proven_println("{} squares, sum {}", PROVEN_ARG(squares.len), PROVEN_ARG(sum));

    /* Destroy takes only the array: it frees through the allocator it kept. */
    PROVEN_ARRAY_DESTROY(&squares);
    return EXAMPLE_OK();
}
```

What to notice. The array was created with room for two and took ten pushes, and not one of them mentions an allocator. `PROVEN_ARRAY_GET` past the end is `NULL` - an answer you can check, not a read of whatever lies beyond the storage. The macros take the element type (`int`) as an argument, so a value that cannot convert to it, such as a struct, does not compile; naming the same type the array was created with is still your job.

Common first stumble. The pointer `PROVEN_ARRAY_GET` returns points *into* the array's storage. The next push may move that storage to a bigger block, and the old pointer then dangles. Fetch it, use it, and do not keep it across a push; Chapter 4 shows the mistake in full. It is lesson 7's stumble again, for the same reason: something other than your pointer decides when that memory moves.

Lesson 9 - the outside world fails too, and says so the same way

The one new thing: failures that are not your program's fault arrive as the same error values.

Until now every failure had a cause inside the program - not enough room, a bad argument. Files bring failures from outside: a name that does not exist, a directory you may not write to, a full disk. None of them is a bug, and all of them happen to working programs. The library reports them with the same `proven_err_t` values as before, so there is no second error system to learn.

The calls are the whole-file ones: `proven_fs_write_file_atomic` writes a file in one call, and `proven_fs_read_all_u8str` reads one back as an owned string. There is no open, no read loop, and no close to forget.

```
/*
 * Lesson 9 - the outside world fails too, and says so the same way.
 *
 * Everything so far could only fail for reasons inside the program: not enough
 * room, a bad argument. A file can be missing, unreadable, or on a full disk,
 * and none of that is a bug in your code. The library reports those failures
 * with the same error values as before, so the checking you already learned
 * is all the checking there is.
 *
 * The calls here are the whole-file ones: one call writes a file, one call
 * reads it back. No open, no read loop, no close to forget.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* A name in the current directory; the program removes it before it ends. */
    proven_u8str_view_t path = PROVEN_LIT("proven_tutorial_notes.tmp");
    proven_u8str_view_t text = PROVEN_LIT("buy milk\ncall home\n");

    /* Atomic: a reader sees the old file or the new one, never half of it.
     * The allocator is scratch space the call may need for the path. */
    proven_err_t err = proven_fs_write_file_atomic(alloc, path, proven_mem_view_from_u8(text));
    EXAMPLE_REQUIRE(proven_is_ok(err), "writing a small file in the current directory should work");
    if (!proven_is_ok(err)) return 1;

    /* Reading gives back an owned string - lesson 4's shape, lesson 5's rule:
     * it was made with `alloc`, so it is destroyed with `alloc`. */
    proven_result_u8str_t back = proven_fs_read_all_u8str(alloc, path);
    EXAMPLE_REQUIRE(proven_is_ok(back.err), "the file we just wrote can be read");
    if (proven_is_ok(back.err)) {
        EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&back.value), text),
                        "the bytes come back exactly as written");
        proven_println("read {} bytes back", PROVEN_ARG(proven_u8str_as_view(&back.value).size));
        proven_u8str_destroy(alloc, &back.value);
    }

    err = proven_fs_remove(alloc, path);
    EXAMPLE_REQUIRE(proven_is_ok(err), "removing the file we created should work");

    /* Now it is gone, and reading it is a failure from outside the program.
```

```
     * It arrives as an ordinary value, and it says which failure it is. */
    proven_result_u8str_t missing = proven_fs_read_all_u8str(alloc, path);
    EXAMPLE_REQUIRE(missing.err == PROVEN_ERR_NOT_FOUND, "a removed file reads as NOT_FOUND");
    if (proven_is_ok(missing.err)) proven_u8str_destroy(alloc, &missing.value);
    proven_println("reading it again: not found, as expected");

    return EXAMPLE_OK();
}
```

What to notice. The string `proven_fs_read_all_u8str` returns is owned: made with `alloc`, destroyed with `alloc` - lesson 5's rule, unchanged. Reading the file after removing it gives `PROVEN_ERR_NOT_FOUND`, the same kind of value as lesson 3's refusal, and it names what went wrong. *Atomic* in the write's name means a reader sees the old file or the new one and never a mix; Chapter 5 says what it does not promise, such as surviving a power cut.

Common first stumble. The allocator handed to the write and to `proven_fs_remove` is scratch space for handling the path, not the owner of anything: those calls give back whatever they took before they return. Only the read, which hands you a string, gives you something to destroy.

Where to go next

If you want to	Read
the full version of everything above	Chapter 0
the types, errors and contracts in detail	Chapter 1
arenas, pools and buffers properly	Chapter 2
owned strings, splitting, encodings	Chapter 3
arrays, maps, lists, hashing	Chapter 4
files, streams, time, randomness	Chapter 5
a word you do not recognise	the glossary

Chapter 0: Start Here

Part I - Start here. No prerequisites beyond one introductory C book. **After this chapter** you can build a program against `proven`, read any other chapter, and look up a term you do not recognise.

Table of contents

1. Who this manual is for
2. Why this library exists
3. Your first program
4. Building and including
5. The five contracts you will meet on every page
6. Intent and design philosophy
7. Build and include model
8. Global contracts
9. Ownership and destruction matrix
10. Operation behavior classes
11. Manual chapters
12. Platform support and verification
13. Appendix B: glossary
14. Appendix C: public header map
15. Appendix D: the libc map
16. Where to go next

---

1. Who this manual is for

This manual assumes you have finished one introductory C book. Concretely, it assumes you are comfortable with variables and control flow, functions, arrays, `struct`, pointers and `*`/`&`, `malloc` and `free`, `printf`, `char *` strings with `strlen`, and compiling a program with `gcc main.c -o main`. If all of that is familiar, you have enough.

It does **not** assume you have met ownership as a discipline, borrowed versus owned data, arenas or pools, function-pointer tables used as an interface, C23 attributes, undefined behaviour as something a compiler actively exploits, alignment beyond "it works", atomics, or the idea that a library might *refuse* an operation instead of doing its best. Every one of those is explained where it is first used, and every one is in the glossary in section 13.

This is not a C tutorial. It will not explain what a pointer is. It will explain, at length, why this library hands you a `struct` containing an error instead of setting `errno`, because that is a design decision you are entitled to disagree with, and you cannot disagree with a decision nobody explained.

---

2. Why this library exists

C gives you almost nothing and trusts you completely. That is its great strength - there is no runtime, no hidden allocation, no cost you did not write - and it is why C is still the language of operating systems, embedded devices and everything that has to be small and predictable.

It is also why the same five bugs have been shipping for fifty years. This library is a set of answers to those five bugs. Each answer costs something, and this section says what.

The string functions do not know how big anything is

```
char buf[64];
strcpy(buf, name);            /* wrong: how long is name? strcpy never asks */
strcat(buf, ", welcome!");    /* wrong: and how much room is left now? */
```

`strcpy` receives a destination pointer and a source pointer. Nothing in that signature carries the size of the destination, so nothing can check it. The function will happily write the 200th byte into a 64-byte buffer, and what it corrupts is whatever the compiler happened to put next -- often the return address. This is not a rare mistake by careless people; it is the single most exploited class of bug in the history of the language, and the API makes it the *default* behaviour.

`strncpy` is the traditional answer and it is a trap of its own: it does not always NUL-terminate, so the "safe" version silently produces a string that is not a string.

What this library does instead. A string carries its length. `proven_u8str_view_t` is a pointer *and* a size, together, always. An append checks the destination's capacity because it knows the capacity, and when the text does not fit it **returns an error and writes nothing** -- it does not truncate, because a truncated path is a wrong path and a truncated command is a different command. See Chapter 3.

What it costs. Every string is two words instead of one, and you cannot pass a `proven` string straight to a `printf("%s")` without asking for a NUL-terminated form.

Nothing makes you check for failure

```
char *p = malloc(n);
p[0] = 'x';                  /* wrong: malloc returns NULL when it fails */
```

`malloc` reports failure by returning `NULL`, and C will not say a word if you never look. The same is true of `fopen`, of `realloc`, of every function that returns a sentinel. The error is *available*; noticing it is optional.

`errno` is worse, because it is a global that survives the call. You must check it at exactly the right moment, before any other library call has a chance to overwrite it, and you must remember that a successful call is allowed to set it to garbage.

What this library does instead. A function that can fail returns its error *as a value*, and when it also has a result, the two come back together in one `struct`:

```
proven_result_u8str_t s = proven_u8str_create(alloc, 64);
if (!proven_is_ok(s.err)) return 1;      /* s.value means nothing until you check */
```

Functions whose only job can fail are marked `[[nodiscard]]`, which means the compiler refuses to build code that throws the error away. You can still ignore it deliberately - `(void)` in front of the call - and having to type that is the point. See Chapter 1.

What it costs. More `if`s. There is no exception mechanism to jump you out of a deep failure, so error paths are visible in the shape of the code. That visibility is the feature.

`printf` believes whatever you tell it

```
printf("%d\n", 3.0);          /* wrong: %d with a double. This compiles. */
printf("%s\n", 42);           /* wrong: and this one crashes */
```

The format string is checked by nothing at runtime. Modern compilers warn about literal formats, which helps exactly until the format is a variable, and then you are back to a function that reads whatever bytes the varargs stack happens to hold, in whatever shape the string demanded.

What this library does instead. `{}` is a placeholder with no type in it, and the type comes from the argument, resolved at compile time by `_Generic`:

```
proven_println("{} is {}", PROVEN_ARG(name), PROVEN_ARG(count));
```

There is no `%d`-versus-`double` mismatch available, because you never wrote the type twice. See Chapter 3 section 3 for the tutorial and Chapter 8 for the full grammar.

What it costs. `PROVEN_ARG` around each argument, and a format language that is not the one in your fingers.

Who frees this?

```
char *s = build_message();    /* do I free this? the type does not say */
```

A `char *` returned from a function might be freshly allocated, might point into a caller's buffer, might be a string literal in read-only memory, or might be a pointer into a static buffer that the next call will overwrite. The type is identical in all four cases. The answer lives in documentation, and documentation drifts.

What this library does instead. Ownership is in the type name and in the signature. A `proven_u8str_t` is **owned** - you got it from a `_create`, and you must `_destroy` it with the same allocator. A `proven_u8str_view_t` is **borrowed** - it points at bytes someone else owns, you never destroy it, and it stops being valid when the owner does. Any function that might allocate takes the allocator as a parameter, so a signature without an allocator cannot allocate.

What it costs. Two types where C has one, and the discipline of asking "who owns this?" at every boundary - which you were paying anyway, just later and in a debugger.

The comparison function nobody can typecheck

```
qsort(a, n, sizeof *a, cmp);  /* cmp takes const void*; get it wrong and it is UB */
```

`qsort` takes a comparator through a `void *` interface, so a comparator with the wrong parameter types still compiles. The classic version of this bug is comparing the pointers instead of what they point at, and it produces a program that runs, sorts nothing correctly, and never crashes.

What this library does instead. The same `void *` shape - this is C, there is no other way -- but the library documents the contract precisely, gives you working comparators to copy, and `proven_array_sort` is an introsort with an *O(n log n)* guarantee rather than a quicksort that degrades to *O(n^2)* on the input an attacker chooses. See Chapter 4.

The bytes have a type, even when you did not choose one

You think of memory as bytes. C's abstract machine does not: it treats memory as *typed*, and reading the same bytes through pointers of two different types is undefined behaviour - which the compiler is allowed to exploit, silently, only when optimisations are on. This is called **strict aliasing**, and it is the trap under every hand-written parser that reads a byte buffer through pointers of different widths:

```
void *buf = malloc(8);
uint32_t *w = buf;      /* the same memory, seen as 32-bit - no cast, no warning */
uint16_t *h = buf;      /* the same memory, seen as 16-bit */
*w = 0xAAAAAAAAu;
*h = 0x1234;            /* change the low half */
printf("%08x\n", *w);   /* wrong to expect aaaa1234: at -O2 this prints aaaaaaaa */
```

Compiled at `-O0` it prints `aaaa1234`; at `-O2` it prints `aaaaaaaa`, because the compiler assumed a `uint16_t` write and a `uint32_t` read cannot touch the same memory and dropped the write. No warning is given, and the program passed every test you ran in a debug build. This is the class of bug the Linux kernel avoids by compiling with `-fno-strict-aliasing` - a whole flag, for one rule.

What this library does instead. Raw memory is `proven_byte_t`, an alias of `unsigned char` -- the one type the rule explicitly exempts, because the standard lets you inspect any object's bytes through it. The ordinary API never quietly reinterprets your bytes as a wider type, so the bug above cannot be written through it. (Strict aliasing has a subtler sibling, *provenance*, that the library is named after; Chapter 6 section 3 and the project README cover it.)

What this library is not

It is not a framework, and it does not want to own your `main`. It has no global state to initialise, starts no threads, registers no `atexit` handler, and allocates nothing you did not hand it an allocator

for. Every module is usable on its own. Most of it runs with no operating system at all - see freestanding mode.

The problem	What C gives you	What proven gives you	Cost
Buffer overruns	strcpy, strcat - no size anywhere	Views carry a length; writes refuse rather than truncate	Two words per string
Unchecked failure	NULL returns and errno	Errors returned as values, [[nodiscard]] on the ones you must not drop	More ifs
Format mismatch	printf trusts the format string	{} with the type taken from the argument	PROVEN_ARG at each call
Unclear ownership	char * means four different things	Owned and borrowed are different types	Two types instead of one
Hidden allocation	Anything may call malloc	Only functions taking an allocator can allocate (one bounded exception: proven_println on an over-long line)	The parameter is on the signature
Bytes with a hidden type	Reinterpreting memory through a wider pointer is UB the optimiser exploits	Raw bytes go through proven_byte_t, the type the rule exempts	-

3. Your first program

This is the whole of it. Every line is one of the contracts in section 5, and the build compiles and runs this exact file, so it cannot quietly stop being true.

```
/*
 * The first program. It is deliberately small, and every line of it is one of
 * the five contracts you will meet on every page of this manual.
 *
 * Compare it with the C you already know:
 *
 *     char buf[64];
 *     strcpy(buf, name);          <- how big is name? strcpy does not ask.
 *     strcat(buf, ", welcome!");  <- and now? strcat does not ask either.
 *     printf("%s\n", buf);
 *
 * That program is correct until the day `name` is longer than you assumed, and
 * then it is a security advisory. The version below cannot do that: every write
 * knows the size of its destination, and every operation that could fail hands
 * you back an error you are not allowed to ignore silently.
 */

int main(void) {
    /* (1) You pass the allocator in. The library never reaches for a global
     *     malloc behind your back, so you always know who allocated what. */
    proven_allocator_t alloc = proven_heap_allocator();
```

```
    /* (2) Anything that can fail returns its error WITH its value. There is no
     *     errno to remember to check, and `greeting.value` means nothing until
     *     you have looked at `greeting.err`. */
    proven_result_u8str_t greeting = proven_u8str_create(alloc, 64);
    if (!proven_is_ok(greeting.err)) return 1;

    /* (3) A view is borrowed text that knows its own length. PROVEN_LIT builds
     *     one from a literal at compile time - no strlen scan happens here. */
    proven_u8str_view_t name = PROVEN_LIT("world");

    /* (4) The append refuses rather than truncates. If "hello, " and the name
     *     did not fit in the 64 bytes asked for above, this returns
     *     PROVEN_ERR_OUT_OF_BOUNDS and writes nothing - it never quietly stores
     *     half a word and lets you carry on. */
    proven_err_t err = proven_u8str_append(&greeting.value, PROVEN_LIT("hello, "));
    if (proven_is_ok(err)) err = proven_u8str_append(&greeting.value, name);
    if (proven_is_ok(err)) err = proven_u8str_append(&greeting.value, PROVEN_LIT("!"));

    if (!proven_is_ok(err)) {
        proven_u8str_destroy(alloc, &greeting.value);
        return 1;
    }

    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&greeting.value),
                                         PROVEN_LIT("hello, world!")),
                    "the three appends should have built the whole greeting");

    proven_println("{}", PROVEN_ARG(proven_u8str_as_view(&greeting.value)));

    /* (5) You created it with `alloc`, so you destroy it with the SAME `alloc`.
     *     Owning things are destroyed exactly once; borrowed things - like
     *     `name` above - are never destroyed at all. */
    proven_u8str_destroy(alloc, &greeting.value);

    return EXAMPLE_OK();
}
```

`EXAMPLE_REQUIRE` and `EXAMPLE_OK` are not part of the library - they come from `manual/examples/example.h` and exist so the build can check that every example in this manual still does what the text says it does. Your own program would use neither.

The examples' own types carry the prefix `ex_` (`ex_app_t`, `ex_task_t`), as your program's should carry its own: POSIX reserves names ending in `_t`, and a bare `task_t` or `node_t` is a name a system or SDK header may already define (macOS's Mach headers define `task_t`). This library keeps the suffix and prefixes every type it defines with `proven_`.

Three details worth pausing on, because they recur everywhere:

- `proven_u8str_create(alloc, 64)` **asks for 64 bytes of capacity**, and that capacity includes the NUL terminator. It does not grow on its own. `proven_u8str_append` is the fixed-capacity form: it fails when the text does not fit. When you want growth, you call `proven_u8str_append_grow` and pass the allocator, and the signature tells you at a glance which one you are using.
- `PROVEN_LIT("hello, ")` **costs nothing at runtime.** It is a compile-time `sizeof` on a string literal. `proven_u8str_view_from_cstr(s)` exists for the case where the text comes from elsewhere, and that one really does scan for the NUL - the two are spelled differently because one is free and the other is *O(n)*.

- **The `destroy` takes the allocator again.** The string does not remember which allocator made it; you must pass the same one. That keeps the string small and makes the dependency visible, and it is a real hazard - see section 5.

---

4. Building and including

There is one header for everything:

```
#include "proven.h"
```

It pulls in the whole public API. If you prefer to include only what you use, every module has its own header under `include/proven/` - `#include "proven/u8str.h"`, `#include "proven/fs.h"` and so on - and section 14 below maps every header to the chapter that documents it.

Compiling by hand, which is all this library needs:

```
gcc -std=c2x -Iinclude -Iplatform your_program.c src/proven/*.c platform/*.c -o your_program
```

`src/proven/` is the portable library. `platform/` is the small layer that talks to the operating system - the **PAL**, or platform abstraction layer. Everything that makes a syscall lives there and nowhere else, which is why the rest of the library can be compiled for a target with no operating system at all.

The repository builds itself with a C program rather than a build system:

```
cc -std=c2x -o nob nob.c     # build the build driver, once
./nob build                  # compile everything and run the whole test suite
./nob release                # the same, optimised (NDEBUG: misuse checks compiled out)
./nob hardened               # optimised, misuse checks kept in
```

`./nob` with no arguments lists the other modes - sanitizers, freestanding, cross-compilation and the benchmark. There is no `make`, no CMake, and nothing to install.

A C23 compiler is required. The library uses C23 features deliberately, `[[nodiscard]]` most visibly. GCC 13+ and Clang 16+ are verified, and on Windows mingw-w64 GCC (x86-64 and i686, the full suite run natively). MSVC and clang-cl are not supported: their C23 support is incomplete, and support may come later. The build driver probes for `-std=c23` and falls back to `-std=c2x` for slightly older compilers.

---

5. The five contracts you will meet on every page

These five rules explain most of the library's shape. Each is stated once here and assumed everywhere else. section 8 below gives the formal versions; these are the plain-language ones.

#	Contract	In one line	Chapter
1	**Errors are values**	Fallible calls return `proven_err_t`, or a `{ err, value }` struct	1
2	**Views are borrowed**	A view is a pointer + a length into memory someone else owns	3
3	**Allocation is a parameter**	If a function can allocate, it takes an allocator; if it cannot, it does not	2
4	**Caller-owned state must not be copied**	Some structs point into themselves; copying them creates dangling pointers	5
5	**Refuse, never truncate**	An operation that does not fit fails and writes nothing	3

1. Errors are values
Every fallible function hands the error back. When there is nothing else to return it is a bare `proven_err_t`; when there is a value, the value and the error travel together and the value is meaningless until you have checked the error. `PROVEN_OK` is zero, and `proven_is_ok(err)` reads better than `err == 0`.

Wrong - reading the value before the error:

```
proven_result_u8str_t s = proven_u8str_create(alloc, 64);
proven_u8str_append(&s.value, text);   /* wrong: s.value is garbage if create failed */
```

The whole point of pairing them is that the value is only valid on the success path. Check first, every time.

2. Views are borrowed
`proven_u8str_view_t` is a pointer and a size. It owns nothing, allocates nothing, and is free to copy - but it stops being valid the moment the thing it points into is destroyed or moved.

Wrong - a view that outlives what it points at:

```
proven_u8str_view_t name;
{
    proven_result_u8str_t owned = proven_u8str_create(alloc, 16);
    (void)proven_u8str_append(&owned.value, PROVEN_LIT("temp"));
    name = proven_u8str_as_view(&owned.value);
    proven_u8str_destroy(alloc, &owned.value);   /* the bytes are gone */
}
use(name);                                       /* wrong: dangling view */
```

This is the same lifetime bug as a dangling pointer, and it is worth stating separately because a view *looks* like a value. It is not; it is a pointer wearing a struct.

3. Allocation is a parameter
Read the signature. `proven_u8str_append(str, data)` cannot allocate, so it fails when the text does not fit. `proven_u8str_append_grow(alloc, str, data)` takes an allocator, so it can grow. Nothing in this library calls `malloc` behind your back, which is what makes it usable in an arena, in a pool, or on a device with no heap.

The corollary is the hazard: **you must destroy with the allocator you created with.** The object does not remember.

Wrong - mismatched allocators:

```
proven_result_u8str_t s = proven_u8str_create(arena_alloc, 64);
proven_u8str_destroy(heap_alloc, &s.value);   /* wrong: heap free on arena memory */
```

Nothing checks this today. It is heap corruption that surfaces later, somewhere else.

4. Caller-owned state must not be copied
Some objects are structs you own and pass by pointer - buffered writers, line readers, directory iterators. Several of them contain a pointer to one of their own fields, so copying the struct copies a pointer that still points at the *original*.

Wrong - copying a state struct:

```
proven_writer_buf_t a = ...;
proven_writer_buf_t b = a;          /* wrong: b's internals still point into a */
```

section 9.2 below lists all twenty-eight of these types. The rule is simple: create it where it lives, pass `&it`, and do not assign it.

5. Refuse, never truncate
When a result does not fit, this library fails the operation and leaves the destination unchanged. It does not write "as much as fits". A truncated path opens the wrong file, a truncated command runs the wrong command, and a truncated number is a different number - and every one of those is worse than an error you can see.

Where truncation genuinely is what you want, there is a separate, differently named function that tells you how much it wrote:

```
proven_result_size_t r = proven_u8str_append_partial(&s, huge);
/* r.value is how many bytes were actually appended. Reading it is the point. */
```

Wrong - assuming the two behave alike:

```
(void)proven_u8str_append_partial(&s, huge);   /* wrong: the count WAS the result */
```

Everything up to here is the plain-language introduction. The rest of this chapter is the reference the other chapters lean on: the library's intent and build model stated formally, the global contracts, the ownership matrix, the behaviour classes, the reading order, platform support, and the appendices. It used to be a document of its own, the manual's front page; it lives here now so that the front page is nothing but the table of contents.

6. Intent and design philosophy

`proven` is a compact C23 systems foundation library. It is intended for C programs that want practical infrastructure without hiding memory ownership, error control flow, or platform access behind global state.

It is not a libc replacement. It provides a focused set of allocator-driven memory tools, byte views, containers, strings, formatting, scanning, hashing (FNV, SipHash, CRC-32, SHA-256), hex/Base64 encoding, OS-strength randomness, filesystem helpers, buffered streams, time helpers, memory mapping, stackless coroutine macros, and a bounded job system.

Core design principles:

- C23 first: the build driver uses `-std=c23`.
- Explicit errors: fallible functions return `proven_err_t` or `proven_result_*_t`.
- Explicit ownership: owned objects have clear destroy functions and allocator rules.
- Failure atomicity: grow/realloc-style APIs preserve the old object on allocation failure unless documented otherwise.
- Pointer provenance discipline: raw object access uses `proven_byte_t` and bounded views.
- PAL isolation: hosted OS services live under `platform/` and are called through public wrappers.
- Core containers do not add hidden locks. Shared mutation requires caller synchronization.
- The build system is a single checked-in `nob.c`; tests are plain C executables.

7. Build and include model

There is nothing to install
This library has no `configure`, no CMake, no package to fetch, and no shared object to place somewhere. It is C source: you compile it into your program alongside your own files.

That is a deliberate choice and it costs something. You do not get a system package, and updating means pulling new source rather than bumping a version constraint. What you get is that the library cannot be a different version than the one you are looking at, cannot pick up a build flag you did not choose, and cannot fail to link because a distribution built it with different options. For a library that is meant to run on hosted systems *and* on bare metal, "compile it with your program" is the only model that works in both places.

Two directories matter. `src/proven/` is the portable library - no OS calls anywhere. `platform/` is the small layer that makes syscalls, and it is the only part a new target has to replace. A freestanding build simply leaves the hosted files out; see the freestanding guide.

The build driver, `nob.c`, is a C program rather than a build system, and you compile it the same way you compile everything else here. It is checked into the repository, so there is no bootstrap step and no version of it to install either.

A C23 compiler is required - GCC 13+ or Clang 16+ (mingw-w64 GCC on Windows; MSVC is not supported). The driver probes `-std=c23` and falls back to `-std=c2x` for compilers that still use the transitional spelling.

Build and run the hosted test suite:

```
cc nob.c -o nob
./nob build
```

Common validation commands:

```
./nob release
./nob strict
./nob strict-error
./nob asan
./nob ubsan
./nob tsan
./nob regression
./nob regression-asan
./nob regression-ubsan
./nob freestanding
./nob cross -build-root build-out/proven_c_lib
```

Use the umbrella header when you want the full hosted API:

```
#include "proven.h"
```

Use smaller includes when you want a narrower translation unit:

```
#include "proven/heap.h"
#include "proven/u8str.h"
#include "proven/fmt.h"
```

A direct hosted application build can follow the same source layout as `nob.c`:

```
cc -std=c23 -D_DEFAULT_SOURCE -D_POSIX_C_SOURCE=200809L \
  -Iinclude -Iplatform \
  app.c src/proven/*.c platform/proven_sys_*.c \
  -pthread -o app
```

8. Global contracts

8.1 Result contract

A result value is usable only when `err == PROVEN_OK`.

Correct:

```
proven_result_mem_mut_t r = alloc.alloc_fn(alloc.ctx, 128, PROVEN_DEFAULT_ALIGNMENT);
if (proven_is_ok(r.err)) {
    /* only now does r.value mean anything */
    proven_mem_mut_t mem = r.value;
    (void)proven_mem_copy(mem.ptr, mem.size, proven_mem_view_from_u8(PROVEN_LIT("hi")));
    alloc.free_fn(alloc.ctx, mem.ptr);
}
```

Wrong (does not compile as shown - the value is read before the error is checked):

```
proven_result_mem_mut_t r = alloc.alloc_fn(alloc.ctx, 128, PROVEN_DEFAULT_ALIGNMENT);
use_bytes(r.value.ptr, r.value.size); /* wrong: r.err was not checked */
```

8.2 Public struct contract

Many public structs expose layout for C usability. This does not mean arbitrary field mutation is supported. Treat direct mutation of internal fields as caller misuse unless the header explicitly allows it.

Wrong:

```
arr.len = 999;      /* wrong: breaks array invariants */
str.internal.cap=0; /* wrong: breaks string invariants */
```

Use the functions and macros that maintain invariants.

8.3 Borrowed view contract

Views do not own memory. A `proven_u8str_view_t`, `proven_u16str_view_t`, `proven_mem_view_t`, or `proven_mem_mut_t` is valid only while the referenced storage remains alive and unmoved.

Wrong:

```
proven_u8str_view_t v = proven_u8str_as_view(&s);
proven_u8str_append_grow(alloc, &s, PROVEN_LIT("more"));
use_view(v); /* wrong: growth may reallocate s */
```

8.4 Allocator contract

A `proven_allocator_t` is valid only when all three function pointers are present. Reallocation must be failure-atomic: if it fails, the old allocation remains valid.

```
proven_allocator_t heap = proven_heap_allocator();
if (!proven_alloc_is_valid(heap)) {
    /* no usable allocator here: e.g. the freestanding heap stub */
    proven_panic("no heap allocator on this target");
}
```

8.5 PAL boundary contract

Code under `src/proven/` should not depend on OS headers directly. Hosted services should route through `platform/proven_sys_*.[ch]` and public wrappers such as `proven_fs_*`, `proven_sysio_*`, `proven_time_*`, and `proven_mmap_*`.

Application code should prefer public APIs. Direct PAL calls are for porting and platform integration.

9. Ownership and destruction matrix

There are **two kinds of object** in this library, and the difference decides what you must do with them.

Owning objects hold storage they allocated, and you must destroy them. They are the table immediately below.

Caller-owned state objects allocate nothing. They are scratch structs you declare - usually on the stack - and hand to a constructor, which returns a small by-value handle that points *into* them. They have **no destroy function**, because there is nothing to free. What they have instead is a rule that owning objects do not have, and it is the one that bites:

> **A caller-owned state object must not be copied or moved once a handle has been made from it.** The handle holds a pointer into the struct. Copy the struct and the handle still addresses the original; let the original go out of scope and the handle points at dead memory.

They are listed in section 4.2.

9.1 Owning objects - you must destroy these

Object	Owns storage	Stores allocator	Destroy function	Notes
`proven_arena_t`	no - caller owns the backing slice	no	`proven_arena_destroy(&arena)` (no-op)	Bump pointer over caller memory: `alloc` advances an offset, `free` is a no-op, `reset` rewinds to empty. The caller owns/frees the backing block.
`proven_pool_t`	yes - items + recycle bin	yes (`base_alloc`)	`proven_pool_destroy(&pool)`	Fixed item size. You use it **through the allocator trait** (`proven_pool_as_allocator`): the trait's `free_fn` returns a slot to the recycle bin for reuse rather than freeing it. There is no `proven_pool_free` - freeing goes through the trait, like every other allocator.
`proven_buf_t`	yes	no	`proven_buf_destroy(alloc, &buf)`	Caller must pass the matching allocator.
`proven_u8str_t`	yes	no	`proven_u8str_destroy(alloc, &str)`	Always NUL-terminated when valid.
`proven_u16str_t`	yes	no	`proven_u16str_destroy(alloc, &str)`	Tracks byte length internally; API length is in `proven_u16` units.
`proven_array_t`	yes	yes	`proven_array_destroy(&arr)` or `PROVEN_ARRAY_DESTROY(&arr)`	Pointers into elements may be invalidated by growth.
`proven_ring_t`	yes	yes	`proven_ring_destroy(&ring)` or `PROVEN_RING_DESTROY(&ring)`	Fixed capacity; no growth.

`proven_map_t`	yes	yes	`proven_map_destroy(&map)` or `PROVEN_MAP_DESTROY(&map)`	Borrowed U8 keys are not copied.
`proven_array_t` from `proven_fs_list()`	yes	yes plus owned entry names	`proven_fs_list_destroy(&list)`	Do not use plain array destroy; entry names need cleanup.
`proven_mmap_t`	OS mapping	OS handle state	`proven_mmap_destroy(&map)`	Views into the mapping die with the mapping.
`proven_job_sys_t *`	yes	internal	`proven_job_system_close(sys)` then `proven_job_system_destroy(sys)`	Destroy must not race with producers.

9.2 Caller-owned state - no destroy, do not copy

These allocate nothing and free nothing. You declare one, pass its address to a constructor, and use the small handle you get back. The struct must **outlive the handle**, and must **not be copied or moved** while the handle is alive.

State object	Constructed by	The handle it backs	Notes
`proven_sha256_t`	`proven_sha256_init`	(used directly)	A hashing context. Safe to copy *before* you start, meaningless to copy mid-stream unless you intend to fork the hash.
`proven_sha1_t`, `proven_md5_t`	`proven_sha1_init`, `proven_md5_init`	(used directly)	The legacy digest contexts; the same rules as `proven_sha256_t`.
`proven_sha512_t`	`proven_sha512_init` / `proven_sha384_init`	(used directly)	The SHA-512 and SHA-384 context; the same rules. Finish it with the family it was begun with.
`proven_hmac_t`	`proven_hmac_init`	(used directly)	A MAC in progress. **It holds key material**; `proven_hmac_final` wipes it. A copy is a second copy of the key.
`proven_xoshiro256ss_t`	`proven_xoshiro256ss_seed`	`proven_rng_t`	Copying it **clones the sequence** - deliberate and useful for a replay, a bug anywhere else.
`proven_chacha_rng_t`	`proven_chacha_rng_seed` / `_seed_from_entropy`	`proven_rng_t`	Copying it clones the keystream: two "independent" tokens

			become the same token.
`proven_writer_buf_t`	`proven_writer_from_buffer`	`proven_writer_t`	**Do not copy.**
`proven_writer_u8str_t`	`proven_writer_from_u8str`	`proven_writer_t`	**Do not copy.** The string and allocator must also outlive it.
`proven_writer_buffered_t`	`proven_writer_buffered`	`proven_writer_t`	**Do not copy.** You must `proven_writer_flush` before it or its buffer dies.
`proven_reader_view_t`	`proven_reader_from_view`	`proven_reader_t`	**Do not copy.**
`proven_reader_buffered_t`	`proven_reader_buffered`	`proven_reader_t`	**Do not copy.** Views returned by `proven_reader_read_line` point *into* its buffer.
`proven_sysio_std_t`	`proven_sysio_stdout_writer` / `_stderr_writer` / `_stdin_reader`	`proven_writer_t` / `proven_reader_t`	**Do not copy.** Holds the standard handle the writer points at.
`proven_sysio_out_t`	`proven_sysio_stdout_buffered` / `_file_buffered`	`proven_writer_t`	**Do not copy.** Must be flushed.
`proven_sysio_lines_t`	`proven_sysio_lines_open` / `_stdin_lines`	(used via `proven_sysio_read_line`)	The one exception: `proven_sysio_read_line` re-binds it on every call, so this one **may** be moved.
`proven_sysio_scanner_t`	`proven_sysio_scanner_init`	(used directly)	The exception in the other direction: this one **does** own a buffer, and you must call `proven_sysio_scanner_deinit`.
`proven_transport_stream_t`	`proven_transport_reader` / `proven_transport_writer`	`proven_reader_t` / `proven_writer_t`	**Do not copy.** The transport it wraps must outlive it too.
`proven_http_body_t`	`proven_http_body_init`	(used directly)	A body being decoded. Copying it mid-body forks the decoder; do not.
`proven_url_query_iter_t`	`proven_url_query_iter`	(used directly)	A position in a query string; a copy is an independent position.
`proven_net_waker_t`	`proven_net_waker_open`	(used directly)	**Do not copy** while open. Needs `proven_net_waker_close`.
`proven_http_cookie_jar_t`	`proven_http_cookie_jar_init`	(used directly)	**Do not copy:** it owns memory. Needs `proven_http_cookie_jar_destroy`.

`proven_sse_t`	`proven_sse_init`	(used directly)	Points into the work memory you gave it, which must outlive it.
`proven_http_client_response_t`	`proven_http_client_send` / `_get`	(used directly)	**Do not copy** while live. Needs `proven_http_client_finish`, even after a failed send.
`proven_ws_decoder_t`	`proven_ws_decoder_init`	(used directly)	A stream being decoded. Copying it mid-stream forks the decoder; do not.

Wrong - the copy looks harmless and is a use-after-free:

```
proven_sysio_out_t out;
proven_writer_t w = proven_sysio_stdout_buffered(&out, buf);

proven_sysio_out_t saved = out;   /* wrong: `w` still points into `out`, not `saved` */
use_elsewhere(&saved);            /* and if `out` goes out of scope, `w` is dangling */
```

Wrong - returning the state by value from a factory function does the same thing:

```
proven_sysio_out_t make_logger(void) {
    proven_sysio_out_t out;
    proven_writer_t w = proven_sysio_stdout_buffered(&out, buf);
    (void)w;
    return out;   /* wrong: any writer made from `out` addresses this dead frame */
}
```

Correct - the state stays put, and the handle travels:

```
proven_byte_t buf[512];
proven_sysio_out_t out;                                  /* lives as long as `w` */
proven_writer_t w = proven_sysio_stdout_buffered(&out,
    (proven_mem_mut_t){ .ptr = buf, .size = sizeof buf });

(void)proven_fprintln(w, "one syscall, not {}", PROVEN_ARG(100));
(void)proven_writer_flush(w);                            /* or the bytes never happened */
```

10. Operation behavior classes

One operation, three honest answers

"Append this text to that string" has three defensible behaviours when the text does not fit, and most libraries pick one and hide the choice. This library exposes all three, gives them different names, and puts the difference in the signature - because which one you want depends on what the text *is*, and only the caller knows that.

Consider appending to a filesystem path:

- If the path does not fit, **truncating is catastrophic.** `documents/report.pdf` becomes `documents/rep`, which is a different, possibly existing, file. The operation must fail and change nothing.
- Now consider appending to a log line. If it does not fit, **truncating is fine** - you would rather have most of the message than none of it - as long as you are told how much was written.
- And appending to a buffer you are building up, where you would simply like it to **grow**.

The three classes below are those three answers. The one you get is decided by the function name and the presence of an allocator parameter, never by a flag or a global:

- `proven_u8str_append(str, data)` - no allocator, so it cannot grow: **refuses**.
- `proven_u8str_append_partial(str, data)` - returns a count: **truncates and tells you**.
- `proven_u8str_append_grow(alloc, str, data)` - takes an allocator: **grows**.

The default across the library is the first one, and Chapter 0 section 5 explains why: a truncated path opens the wrong file, a truncated command runs the wrong command, and a truncated number is a different number.

Wrong - ignoring the count from the truncating form:

```
(void)proven_u8str_append_partial(&s, huge);   /* wrong: the count WAS the answer */
```

Several APIs intentionally expose three behavior classes:

Class	Example	Behavior on insufficient capacity
Atomic fixed-capacity	`proven_u8str_append`, `proven_u16str_append`, `proven_u8str_append_fmt`	Return an error and leave the old object unchanged.
Best-effort truncating	`proven_u8str_append_partial`, `proven_u16str_append_partial`, `proven_u8str_append_fmt_trunc`	Write as much as fits, preserve a valid object, report how much was written.
Atomic growable	`proven_u8str_append_grow`, `proven_u16str_append_grow`, `proven_u8str_append_fmt_grow`	Grow with an allocator; on allocation failure, leave the old object unchanged.
Refuse, never truncate	`proven_hex_encode`, `proven_base64_encode`, `proven_base64_decode`, `proven_reader_read_line`	Write **nothing** and return `PROVEN_ERR_OUT_OF_BOUNDS`. A half-encoded string or a shortened line is a wrong answer that looks like a right one, so these do not produce one. Size the buffer with the module's own size function (`proven_base64_encoded_size`, ...), not by eye.

Choose the class deliberately. Do not treat a truncating function as an all-or-nothing function.

Wrong - assuming the truncating and the atomic form behave alike:

```
/* `_partial` wrote what fit and told you so; the error you did not read is the
   difference between "all of it" and "some of it". */
(void)proven_u8str_append_partial(&s, huge);   /* wrong: the result was the point */
```

11. Manual chapters

Just finished an introductory C book? Start with the tutorial. It is nine short programs, each introducing exactly one idea: the sixth is the greeting program from this chapter read line by line, and the last three go on to arenas, containers and files. This chapter shows that program on its first page and it carries five new ideas at once; the tutorial hands them over one at a time.

Otherwise start with this chapter. It is the only chapter that assumes nothing: why the library exists, argued from the C bugs it is answering; a hello-world program; how to build; the five contracts the rest of the manual takes for granted; and a glossary plus a libc-to-`proven` table. Every other chapter assumes it.

The reading order

The chapters are grouped into parts, and the parts are ordered so that each one only needs the ones before it. That order is not the same as the header dependency graph, and it is not arbitrary: strings need allocators, containers need allocators, and hosted services need strings, so the sequence below is what the material itself requires.

The chapter *numbers* are stable identifiers, not a reading sequence. Two of them are out of order on purpose: the alias index is an appendix you look things up in, and Chapter 8 is a reference you read after Chapter 3 has introduced the subject.

Part	Read	Prerequisites	You can then
I - Start here	tutorial -> 0	One introductory C book	Build against the library and read anything below
II - The vocabulary every program uses	1 -> 2 -> 3	Chapter 0	Handle errors as values, own memory deliberately, hold text safely
III - Data structures	4	Part II	Arrays, maps, lists, rings, sorting, searching, hashing, encoding
IV - Text in and out	8	Chapter 3 section 3-section 4	Format and parse anything, and teach the formatter your own types
V - Talking to the operating system	5 -> 9 -> 10 -> 11 -> 12 -> 13 -> 14 -> 15	Part II	Files, directories, streams, standard I/O, time, randomness, mapping; sockets with deadlines; URLs and HTTP messages; an HTTP client and server; WebSocket; certificates and trust; TLS; an event loop and a server on it
VI - Going further	6 -> freestanding	Parts II-V	Coroutines, jobs, thread-safety, bare metal, cross builds
Appendices	A, B, C, D	-	Look things up

The chapters

- **Tutorial**: the library in nine short programs, one new idea at a time - *Part I, optional on-ramp*

1. **Start here**: why this exists, hello world, the five contracts, glossary, libc map - *Part I*
2. **Foundation**: types, errors, memory views, alignment, version, panic - *Part II*
3. **Allocation**: heap, arena, pool, byte buffers, and the allocator trait - *Part II*
4. **Strings and text**: U8, U16, and an introduction to formatting and scanning - *Part II; the tutorial half of the text material*

5. **Containers and algorithms**: array, list, ring, map, sort/search, hashing, encoding - *Part III*
6. **Hosted services**: filesystem, tree walk, streams, sysio, environment, randomness, mmap, time - *Part V*
7. **Execution and platform**: coroutines, jobs, thread-safety, aliases, PAL, cross builds - *Part VI*
8. **Appendix A - Alias index**: every `alias_xcv.h` spelling - *reference only; not reading material*
9. **Formatting and scanning**: the full `fmt.h` and `scan.h` reference - *Part IV; the reference half of the text material*
10. **Networking**: addresses, TCP, UDP, deadlines, readiness, transports - *Part V; after Chapter 5*
11. **URLs and HTTP messages**: URL parsing and safe paths, the HTTP/1.1 head parser, body framing, writers, dates, ranges, multipart, authentication, cookies, event streams - *Part V; pure text handling, freestanding-available*
12. **An HTTP client and server**: a handler and a loop with every wait bounded; a client with redirects, challenges, cookies and proxies - *Part V; after Chapters 9 and 10*
13. **WebSocket**: connections from either side, messages, closing; and the frame codec by itself - *Part V; after Chapter 11; the codec is freestanding-available*
14. **Certificates and trust**: reading an X.509 certificate, the names it is for, trust anchors, verifying a chain, pinning - *Part V; after Chapter 4; freestanding-available except the system's roots*
15. **TLS**: HTTPS and wss in two lines, the transport wrapper, the engine, resumption, client certificates and pins, what failures mean - *Part V; after Chapters 11 and 13; the engine is freestanding-available*
16. **The event loop and the event-driven server**: a loop with timers and posted work, an HTTP server in which nothing waits, answering later, backpressure, WebSocket without a thread each, a client with many requests in flight, several loops, what an idle connection holds and what was measured - *Part V; after Chapters 9 and 11; hosted-only*

Chapters 3 and 8 both cover the formatter and the scanner, and the division is deliberate. Chapter 3 introduces them alongside strings, with the everyday cases and enough to be productive. Chapter 8 is the complete reference: the full format grammar, every argument constructor, the scanner's error codes and recovery rules, and how to teach the formatter a type of your own. Read Chapter 3 first; reach for Chapter 8 when you need the exact behaviour of a specifier or a failure.

12. Platform support and verification

Primary verified hosted target:

- Linux x86_64 with GCC or Clang in C23 mode.

Compile-only cross coverage exists when the corresponding toolchains are installed:

- Linux AArch64.
- Linux ARM hard-float.
- Linux i686 through `i686-linux-gnu-gcc` or `gcc -m32` multilib.
- Windows x86_64 and i686 through MinGW/WinAPI paths.
- ARM Cortex-M freestanding.
- RISC-V ELF freestanding.

The cross matrix checks compilation, public header visibility, and target ABI assumptions. It does not replace runtime validation on the target platform.

Freestanding mode builds a reduced subset with OS-backed services removed. See `manual-freestanding.md` and Chapter 6 for details.

13. Appendix B: glossary

Terms this manual uses as if they were ordinary words. They are not ordinary C words, and every one of them is load-bearing.

Two rules the chapters follow, so that nothing here has to be guessed at:

- **No private vocabulary.** Where a widely used word exists for an idea, the manual uses that word. Where the library has invented something, it is defined here.
- **Every abbreviation is written out the first time it appears in a chapter**, and every one of them is listed below as well.

The Korean edition adds one more: an English term appears with its original spelling in parentheses the first time each chapter uses it - 뷰(view) - so a reader who learned the idea in English can find it, and a reader who did not is never handed an untranslated word.

Term	Meaning here
owned	You are responsible for destroying it. Comes from a `_create`, goes to a `_destroy`, exactly once.
borrowed	Points at memory someone else owns. Never destroyed. Valid only while the owner is.
view	A borrowed pointer + length pair, e.g. `proven_u8str_view_t`. Copyable, non-owning, no allocation.
allocator	A value carrying four things: a context pointer and three function pointers (alloc, realloc, free). Passed by value into anything that may allocate.
arena	An allocator that hands out memory by bumping a pointer through one block. Individual frees do nothing; you reset or destroy the whole arena at once. Fast, and perfect for "many small things with the same lifetime".
pool	An allocator for many objects of one fixed size, with a free list, so freeing really does recycle a slot.
trait	A struct of function pointers used as an interface - C's answer to a virtual table. `proven_allocator_t`, `proven_writer_t` and `proven_rng_t` are traits. Not a C keyword; borrowed terminology.
PAL	Platform abstraction layer: the code under `platform/` that makes actual syscalls. The only OS-dependent part.
freestanding	A build with no operating system and no libc - bare metal. `PROVEN_FREESTANDING` selects it.
failure atomicity	If an operation fails, it changes nothing. A failed grow leaves your old data intact and valid.
provenance	Which allocation a pointer came from. C's optimizer assumes pointers from different allocations never overlap; violating that is undefined behaviour, not merely surprising. Chapter 6 covers it.

UB (undefined behaviour)	Not "unpredictable output" - the standard imposes no requirement at all, and optimizers are allowed to assume it never happens. This is why UB can delete your `if`.
`[[nodiscard]]`	A C23 attribute. The compiler errors if you throw the return value away. Used on every function whose error you must not drop.
fixed-capacity	Will not grow. Fails when full. Takes no allocator.
growable	Will reallocate when full. Takes an allocator. Always spelled `_grow` in the name.
CSPRNG	Cryptographically secure pseudo-random number generator: output an attacker cannot predict even after seeing earlier output.
intrusive	The list's links live *inside* your struct rather than in separately allocated nodes. No allocation per element.
code unit	One element of an encoding: a byte in UTF-8, a 16-bit value in UTF-16. Not a character - one character can take several.
code point	One character's number in Unicode. A code point takes one to four bytes in UTF-8, and one or two code units in UTF-16 - which is why counting either one is not counting characters.
API (application programming interface)	The set of functions and types a library exposes for other programs to call. In this library, everything declared in `include/proven/`.
hosted	A build that has an operating system and a C standard library under it - the opposite of *freestanding*.
heap	The general-purpose pool of memory `malloc` hands out from. `proven_heap_allocator()` is the allocator that uses it.
slice	A borrowed pointer + length pair you may **write** through, e.g. `proven_mem_mut_t`. A *view* is the read-only form of the same idea.
result	A small struct carrying an error code and a value together, e.g. `proven_result_u8str_t`. The value means nothing until the error beside it has been checked.
panic	The deliberate stop taken when a failure has no caller to return to. `proven_set_panic_handler()` chooses what happens; the default stops the program.
reserve	Raise a container's capacity **now**, so later growth does not reallocate. Saves copying on the heap, and dead storage in an arena.
dangling	A pointer to memory that has been freed or moved. Using one is undefined behaviour; the usual cause

	here is holding a pointer across a call that may reallocate.
use-after-free	Reading or writing through a dangling pointer. The sanitizers below detect it.
sanitizer	A compiler mode that adds run-time checks: **ASan** (AddressSanitizer) finds memory errors, **UBSan** (UndefinedBehaviorSanitizer) finds undefined behaviour, **TSan** (ThreadSanitizer) finds data races. `./nob asan`, `./nob ubsan`, `./nob tsan`.
partial write / short read	A single write or read that moved *fewer* bytes than asked for. Normal, not an error - and treating a short read as end of input is the classic way to lose the tail of a file.
EOF (end of file)	There is no more input. Reported as `PROVEN_ERR_EOF`, never as a zero-byte success, so it cannot be confused with "nothing arrived yet".
flush	Push a buffered writer's accumulated bytes onward. Nothing in this library flushes on your behalf at exit.
back-pressure	Slowing a producer down because the consumer cannot keep up. `proven_writer_write_partial()` is the call that lets you notice and react.
durability	The guarantee that data survives a power cut. Reaching the operating system is not enough: `proven_fs_sync()` puts a file's bytes on the storage device, and `proven_fs_sync_dir()` does the same for a rename.
atomic rename	Replacing a file by renaming a finished temporary over it. A reader sees the whole old file or the whole new one, never a half-written mixture.
advisory lock	A lock that excludes only the processes that also ask for it (`proven_fs_lock()`). It is a convention between cooperating programs, not access control.
hard link	A second **name** for the same file. There is no original; the data lives until the last name is removed. Same filesystem only.
symbolic link	A small file that holds a path. It may cross filesystems, and it may point at nothing - following it then fails.
memory mapping	Making a file's contents appear at an address, so the processor reads it as memory (`mmap.h`). **SHARED** mappings write back to the file; **PRIVATE** ones are *copy-on-write*: the writes exist in your process and nowhere else.
copy-on-write	Sharing memory until somebody writes, at which point that writer gets a private copy. What `PROVEN_MMAP_PRIVATE` does.

cursor	The scanner's position in the text it is reading (`proven_scan_t.cursor`). Scanning advances it; `proven_scan_skip_*` moves it deliberately.
locale	The system's idea of local conventions - including whether the decimal separator is `.` or `,`. This library's number parsing is **locale-free**: a comma is never a decimal point, on any machine.
entropy	Genuinely unpredictable bits, from the operating system or from hardware. A generator is *seeded* from entropy; a clock or a counter is not entropy, however random it looks.
seed	The starting value of a generator. The same seed replays the same sequence - essential for a reproducible test, and fatal for a key.
PRNG / CSPRNG	A pseudo-random number generator computes a sequence from a seed. A **C*ryptographically** S*ecure one (CSPRNG) is additionally unpredictable to an attacker who has seen earlier output.
HashDoS	An attack that feeds a hash table keys chosen to collide, turning *O(1)* lookups into *O(n^2)*. `proven_map_create()` defends against it with a keyed hash; `proven_map_create_trusted()` opts out for keys you choose yourself.
open addressing	The map's layout: entries live in one flat bucket array and a collision moves to the next slot, rather than following a chain of separately allocated nodes.
tombstone	The marker left where a map entry was removed, so lookups keep probing past it. Tombstones count towards the load factor, which is why heavy remove traffic still triggers a rehash.
rehash	Rebuilding the bucket array at a new size. It invalidates every pointer a previous `get_mut` returned, which is why you hold the key rather than the pointer.
checksum	A short value that detects accidental corruption - `proven_crc32()`. It detects accidents, never tampering.
digest / hash	A fixed-size value computed from data. **SHA-256** is a cryptographic digest (tamper-evident); **FNV-1a** and **SipHash-2-4** are table hashes, and only SipHash is keyed.
hex / Base64 / Base64URL	Ways to write bytes as text. Hex is two characters per byte; Base64 packs three bytes into four characters with `+ / =`; Base64URL uses `- _` and no padding, so it is safe in a URL or filename.
padding	The `=` characters Base64 adds so the output length is a multiple of four. Base64URL leaves them out.

BMP (Basic Multilingual Plane)	The first 65,536 Unicode code points. A character outside it - an emoji, many rarer CJK characters - needs two UTF-16 code units.
NUL terminator	The zero byte that marks the end of a C string. A *view* does not have one, which is exactly why it carries a length instead.
shortest round trip	Printing the fewest digits that read back as exactly the same floating-point value. `PROVEN_FLOAT_FORMAT_MODE_SHORTEST` asks for it; it is what a serialiser wants.
dispatch macro	A macro built on C11 `_Generic` that picks a function from an argument's type - `PROVEN_ARG(x)` and `PROVEN_SCAN_ARG(&x)`. It chooses among the named constructors; it is not itself one.
identity constructor	`proven_arg_identity()` / `proven_scan_arg_identity()`: they take an argument that has already been built and pass it through, so macro-driven code can accept one.
scratch allocator	An allocator passed for temporary working memory only, separate from the one that owns the result - e.g. the `scratch` parameter of `proven_map_set_with_scratch()`.
stackless coroutine	A function that can suspend and resume without a stack of its own: its state lives in a struct you hold. `coro.h` implements it with a switch, so it costs no thread and no allocation.
bounded queue	A queue with a fixed capacity that refuses when full rather than growing. The job system uses one, so a producer that outruns the workers is told so instead of exhausting memory.
monotonic clock	A clock that only moves forward, for measuring how long something took. Distinct from the wall clock, which can jump when the system time is corrected - measuring a duration with the wall clock is how a negative elapsed time happens.

14. Appendix C: public header map

How to find things

There are 35 public headers and one umbrella. `#include "proven.h"` pulls in everything and is what the examples in this manual do; including individual headers is for when you care about compile time or want the dependency to be visible in the file.

Two things this table tells you that the file names do not:

- **Which chapter documents it.** Every header has exactly one chapter that explains it, and the build enforces that every public function is named somewhere under `manual/` - so if a symbol is not in the chapter you expect, it is in the manual somewhere and this map says where.
- **Whether it survives a freestanding build.** The headers assigned to Chapter 5 are the hosted ones: they need a filesystem, standard streams, a clock, virtual memory or threads. Everything

else compiles with no operating system. The freestanding guide has the authoritative per-module table.

Header	Main purpose	Chapter
`proven.h`	Umbrella include	This file
`types.h`	Fixed-width aliases, checked arithmetic, error enum	Chapter 1
`error.h`	Error predicate helpers	Chapter 1
`memory.h`	Byte views, slicing, range checks, memcmp	Chapter 1
`align.h`	Alignment constants and align-up helpers	Chapter 1
`version.h`	Version macros	Chapter 1
`panic.h`	Registerable panic handler	Chapter 1
`config.h`	Compile-time feature toggles (`PROVEN_FREESTANDING`, `PROVEN_FMT_NO_FLOAT`, `PROVEN_NO_U16STR`, ...)	Chapters 1 and 6
`allocator.h`	Allocator trait	Chapter 2
`heap.h`	PAL-backed heap allocator	Chapter 2
`alloc_check.h`	An allocator wrapper that knows its own blocks: catches a foreign free, a double free and a wrong realloc at the call (tests and debugging; off unless `PROVEN_ALLOC_CHECK`)	Chapter 2
`arena.h`	Bump allocator	Chapter 2
`pool.h`	Fixed-size recycler allocator	Chapter 2
`buffer.h`	Fixed-capacity byte buffer	Chapter 2
`u8str.h`	Owned U8 string and borrowed U8 views	Chapter 3
`u16str.h`	Owned U16 string and borrowed U16 views	Chapter 3
`utf.h`	Strict UTF-8 <-> UTF-16 transcoding: measure, convert, convert in pieces, grow	Chapter 3
`fmt.h`	Structural formatter and format arguments	Chapter 3
`scan.h`	Structural scanner and typed scan destinations	Chapter 3
`float_parse.h`	Locale-free decimal -> `double`/`float` parser (`proven_strtod`, `proven_parse_double_ascii`)	Chapter 8
`float_format.h`	`double`/`float` -> decimal formatter (fixed `%f`/`%e`, shortest)	Chapter 8
`float_config.h`	Float-engine tuning (`PROVEN_FLOAT_BIGINT_LIMBS`, precision caps)	Chapters 6 and 8
`array.h`	Generic growable vector	Chapter 4
`list.h`	Intrusive doubly-linked list	Chapter 4
`ring.h`	Fixed-capacity FIFO ring	Chapter 4
`map.h`	Open-addressing map	Chapter 4
`algorithm.h`	Array sort and search helpers	Chapter 4
`hash.h`	FNV-1a, SipHash-2-4, CRC-32, SHA-256, SHA-384, SHA-512, by use case	Chapter 4
`hmac.h`	HMAC and HKDF over the SHA-2 family	Chapter 4
`deflate.h`	DEFLATE compression and decompression, bare or as zlib or gzip; streaming, with the output bounded by the caller	Chapter 16

zstd.h	Zstandard decompression and compression (nine levels), streaming, with the output and the window bounded by the caller	Chapter 16
brotli.h	Brotli decompression, streaming, with the output and the window bounded by the caller; left out by PROVEN_NO_BROTLI	Chapter 16
hash_legacy.h	SHA-1 and MD5, for formats that fix them	Chapter 4
encode.h	Hex and Base64 (standard + URL-safe), bytes to text and back	Chapter 4
fs.h	Files, directories, metadata, links, locks, read-all, tree walk	Chapter 5
stream.h	Buffered writers, readers, and a line reader - and, through sysio.h, the standard streams (hosted-only)	Chapter 5
net.h	Addresses, TCP, UDP, Unix-domain streams, deadlines, readiness, transports (hosted-only)	Chapter 9
url.h	URL parsing, percent-coding, and a request path resolved to stay inside its root	Chapter 10
http.h	HTTP/1.1 codec: head parser, body framing and decoder, writers, dates, ranges, multipart bodies, content-coding headers	Chapter 10
hpack.h	HPACK, the header compression of HTTP/2: an encoder and a decoder with their tables, bounded by the caller	Chapter 17
h2.h	HTTP/2 frames, each type's rules, the checks that make a header list a request or a response, and a connection with streams and flow control that is given bytes and gives bytes back: the protocol, without a socket. (The socket is http_event.h.)	Chapter 17
http_auth.h	Basic and Digest authentication values	Chapter 10
http_cookie.h	A host-only cookie jar	Chapter 10
sse.h	Server-sent events parser	Chapter 10
http_client.h	HTTP/1.1 client: redirects, challenges, cookies, proxies, connection reuse, gzip responses decoded on request (hosted-only)	Chapters 11, 16
http_server.h	HTTP/1.1 server: a handler and a bounded loop, on one thread or a job system; a response compressed when its handler asks (hosted-only)	Chapters 11, 16
ws.h	WebSocket codec: handshake values and checks, frames, a message decoder, the compression extension's handshake and frame bit	Chapter 12
ws_conn.h	WebSocket connections over a transport, client and server; compressed messages when asked for (hosted-only)	Chapter 12
cert.h	X.509 certificates: a strict reader, host-name matching, PEM, a store of trust anchors, chain verification. Only reading the system's roots is hosted.	Chapter 13

`tls.h`	TLS 1.3 and a cut-down TLS 1.2, client and server: a config, an engine with no I/O, and (hosted-only) a transport wrapper and the HTTP wrap	Chapter 14
`loop.h`	An event loop: socket readiness, timers, work posted from other threads, a shared scratch buffer (hosted-only)	Chapter 15
`http_event.h`	HTTP server driven by events on a loop - HTTP/1.1, and HTTP/2 beside it when asked: callbacks, responses in pieces with backpressure, optional TLS, a response compressed when asked (hosted-only)	Chapters 15, 16, 17
`ws_event.h`	WebSocket on the event-driven server: messages in pieces, sends refused and resumed, liveness and close handled, compressed messages when asked for (hosted-only)	Chapters 15, 12
`http_event_client.h`	HTTP client driven by events on a loop: many requests in flight on one thread; HTTP/2 when asked, on one connection kept for each origin; narrow - no redirects or name resolution, no reuse for HTTP/1.1; gzip responses decoded on request (hosted-only)	Chapters 15, 16, 17
`sysio.h`	Standard streams as writers/readers, line input from stdin, buffered output, printing, scanning, environment access	Chapter 5
`random.h`	Randomness by use case: xoshiro256** (reproducible), ChaCha20 (cryptographic), the OS CSPRNG, and unbiased range/shuffle helpers. The generators work freestanding; only the OS source is hosted.	Chapter 5
`mmap.h`	Memory-mapped file regions	Chapter 5
`time.h`	Timestamp, datetime, sleep, datetime formatting	Chapter 5
`coro.h`	Stackless coroutine macros	Chapter 6
`job.h`	Bounded worker-thread job system	Chapter 6
`alias_xcv.h`	Optional short alias layer and generated spelling map	Chapters 6 and 7

15. Appendix D: the libc map

If you already write C, this is the fastest way in. Every row links to the chapter that explains the trade.

You would write	Use instead	Why it differs
`malloc` / `free`	`proven_heap_allocator()` + `_create` / `_destroy`	The allocator is a parameter, so the same code runs on an arena or a pool. Ch 2
`strcpy`, `strcat`	`proven_u8str_append`, `_append_grow`	Sizes are known, so overruns are refused instead of performed. Ch 3
`strlen`	`view.size`	The length is already there; nothing scans. Ch 3

`strcmp` for equality	`proven_u8str_view_eq`	Works on text with embedded NULs, and does not walk past the end. Ch 3
`strstr`	`proven_u8str_view_find`	Returns an index or `PROVEN_INDEX_NOT_FOUND`; the search is not naive. Ch 3
`strtok`	*(no equivalent yet)*	`strtok` mutates its input and cannot be nested. A view-based splitter is designed in `RFC-0004`.
`printf`	`proven_println("{}", PROVEN_ARG(x))`	Types come from the arguments, not from the format string. Ch 8
`sprintf`	`proven_u8str_append_fmt`	Writes into a sized destination and refuses to overrun it. Ch 8
`sscanf`	`proven_scan_*`, `proven_scan_fmt`	Reports which field failed and where the cursor stopped. Ch 8
`strtod`	`proven_parse_f64_ascii`	Correctly rounded, locale-independent, no `errno`. Ch 8
`fopen` / `fread` / `fclose`	`proven_fs_open`, `_read`, `_close`, or `proven_fs_read_all_u8str`	Explicit errors, no hidden buffering, one call for whole files. Ch 5
`fgets`	`proven_sysio_read_line`, `proven_reader_read_line`	A line that exactly fills the buffer is returned, not lost. Ch 5
`qsort`	`proven_array_sort`	Introsort: *O(n log n)* guaranteed, not quicksort's worst case. Ch 4
`bsearch`	`proven_array_binary_search`	Same shape, same comparator contract. Ch 4
`rand`	`proven_xoshiro256ss_*` or `proven_random_bytes`	Reproducible and fast, or unpredictable and secure - you pick, deliberately. Ch 5
`time` / `clock`	`proven_time_now`, `proven_time_monotonic_now`, `proven_time_breakdown`	Nanoseconds, with the wall clock and the monotonic clock as two separate calls. Ch 5
`socket` / `connect` / `send` / `recv` / `poll`	`proven_net_connect`, `_read`, `_write_all`, `proven_net_poll`	Every wait takes a deadline, and refused, reset and end-of-input are different answers. Ch 9
`assert`	`proven_panic` + a panic hook	Works in freestanding builds and is overridable. Ch 1

16. Where to go next

The chapters are ordered so that each one only needs the ones before it.

Part	Read	For

I	This chapter	The contracts and the vocabulary
II	1 -> 2 -> 3	Errors, memory, and text: what every program uses
III	4	Arrays, maps, lists, rings, sorting, hashing, encoding
IV	8	Formatting and scanning in full, once Chapter 3 has introduced them
V	5 -> 9 -> 10 -> 11 -> 12 -> 13 -> 14 -> 15	Files, streams, standard I/O, time, randomness, mapping; then sockets; then URLs and HTTP; then a client and a server; then WebSocket; then certificates; then TLS; then the event loop
VI	6 -> freestanding	Coroutines, jobs, thread-safety, bare metal, cross builds
Appendices	A: alias index, B and D above	Looking things up

If you are an experienced C programmer in a hurry, read section 15 above, then Chapter 1, then whichever chapter covers the thing you need. If you are newer, read Parts I and II in order - they are short, and everything later assumes them.

Chapter 1: Foundation - Errors, Types, and Memory Views

Part II - The vocabulary every program uses. Prerequisite: Chapter 0. After this chapter you can handle every failure this library reports, describe a region of memory without losing track of its size, and do arithmetic on sizes that cannot silently wrap.

This chapter covers `types.h`, `error.h`, `memory.h`, `align.h`, `version.h`, and `panic.h` - the pieces every other chapter is built out of. Nothing here allocates, and nothing here talks to the operating system. It is the shortest chapter that is genuinely required reading.

Table of contents

1. Errors are values
2. The types, and why they are spelled differently
3. Memory views: a pointer and a length, together
4. Size arithmetic that cannot wrap
5. Alignment
6. Panic: when there is no one left to return an error to
7. Version macros
8. Examples and misuse cases

1. Errors are values

Why this is first

Every other page of this manual assumes you know how failure is reported here, so this is the one section you cannot skip.

C has never agreed with itself about how a function should report failure. `malloc` returns `NULL`. `fopen` returns `NULL`. `read` returns `-1`. `strtol` returns `0` and *maybe* sets `errno`, but only if you cleared `errno` first, because a successful call is permitted to leave junk in it. `printf` returns a negative number that almost nobody checks. Four conventions, and the only thing they share is that **ignoring them is silent and legal**:

```
char *p = malloc(n);
p[0] = 'x';                    /* wrong: malloc returns NULL on failure */

FILE *f = fopen(path, "r");
fread(buf, 1, n, f);           /* wrong: f may be NULL */

long v = strtol(s, NULL, 10);  /* wrong: 0 could be the value or the failure */
```

The deeper problem is not that these are easy to get wrong. It is that **the type does not say anything is possible.** `char *` is the same type whether or not it can be `NULL`, so nothing -- not the compiler, not a reviewer skimming, not you at 2 a.m. - is prompted to check.

`errno` makes it worse by making the error *global and temporary*. It must be read at exactly the right moment; any intervening library call may overwrite it; and it is per-thread, so the fix for one bug class opened another.

What this library does instead

The error is a return value, and it has a type of its own.

When a function's only outcome is success or failure, it returns `proven_err_t` - a plain enum where `PROVEN_OK` is `0`:

```
proven_err_t err = proven_u8str_append(&s, PROVEN_LIT("hello"));
if (!proven_is_ok(err)) { /* handle it */ }
```

When a function has something to give back, the value and the error travel together in one struct, and **the value is meaningless until the error has been checked**:

```
proven_result_u8str_t s = proven_u8str_create(alloc, 64);
if (!proven_is_ok(s.err)) return;      /* s.value is not a string; do not touch it */
```

This is the same idea as `Result` in Rust or `std::expected` in C++23, expressed in the only way C allows - a struct returned by value. There is no allocation, no indirection, and no hidden control flow. What you give up is the ability to skip error handling silently, which is the point.

Three properties are worth stating explicitly because the rest of the library relies on them:

- `PROVEN_OK` **is zero**, so a result struct zero-initialised with `{0}` starts out meaning "success, empty".
- **Fallible functions are** `[[nodiscard]]` - a C23 attribute that makes the compiler reject code that throws the return value away. You can still ignore an error, but you have to write `(void)` in front of the call and thereby say that you meant it.
- **Failure is failure-atomic** unless a function documents otherwise: if an operation fails, it changed nothing. A failed append leaves your string exactly as it was.

Reference

```
typedef enum {
    PROVEN_OK = 0,
    PROVEN_ERR_NOMEM,
    PROVEN_ERR_OUT_OF_BOUNDS,
    PROVEN_ERR_INVALID_ENCODING,
    PROVEN_ERR_INVALID_ARG,
    PROVEN_ERR_IO,
    PROVEN_ERR_NOT_FOUND,
    PROVEN_ERR_INVALID_STATE,
    PROVEN_ERR_NEED_MORE,
    PROVEN_ERR_OVERFLOW,
    PROVEN_ERR_UNSUPPORTED,
    PROVEN_ERR_AGAIN,
    PROVEN_ERR_EOF,
    PROVEN_ERR_BUSY,
    PROVEN_ERR_PERMISSION,
    PROVEN_ERR_INVALID_FORMAT,
    PROVEN_ERR_EXISTS,
    PROVEN_ERR_TIMEOUT,
    PROVEN_ERR_REFUSED,
    PROVEN_ERR_RESET,
    PROVEN_ERR_UNREACHABLE,
    PROVEN_ERR_UNTRUSTED,
    PROVEN_ERR_EXPIRED,
    PROVEN_ERR_NOT_YET_VALID,
    PROVEN_ERR_NAME_MISMATCH,
    PROVEN_ERR_PROTOCOL,
    PROVEN_ERR_CANCELLED
} proven_err_t;

#define PROVEN_ERR_LAST         PROVEN_ERR_CANCELLED        /* moves when a code is added */
#define PROVEN_ERR_RESERVED_END 0x1000                      /* never reached, in any version */
```

Error	Typical meaning	You usually respond by
`PROVEN_OK`	Success.	Carrying on.
`PROVEN_ERR_NOMEM`	The allocator could not provide memory.	Giving up on this operation; the object is unchanged.
`PROVEN_ERR_OUT_OF_BOUNDS`	An index, size, capacity, or range is invalid - including "this does not fit".	Growing the destination, or refusing the input.

PROVEN_ERR_INVALID_ENCODING	Encoded text failed validation (hex, Base64).	Rejecting the input as malformed.
PROVEN_ERR_INVALID_ARG	A null pointer, an invalid allocator, an impossible mode, a wrong argument count.	Fixing the call - this one usually means a bug in your code, not bad data.
PROVEN_ERR_IO	The OS or device failed the operation.	Retrying, or reporting to the user.
PROVEN_ERR_NOT_FOUND	A file, key, substring, or resource does not exist.	Taking the "absent" branch; often not an error at all.
PROVEN_ERR_INVALID_STATE	The object's state does not allow this operation.	Fixing the sequence of calls.
PROVEN_ERR_NEED_MORE	The input ends inside something more input would complete - a split UTF-8 character, a number cut by a read.	Keeping what you have and reading more; for a complete text it is malformed.
PROVEN_ERR_OVERFLOW	Integer conversion or size arithmetic overflowed.	Refusing the size; see section 4.
PROVEN_ERR_UNSUPPORTED	Unavailable on this platform or build profile.	Taking a different route (e.g. a pipe cannot seek).
PROVEN_ERR_AGAIN	Not now; retry later.	Retrying, usually after waiting.
PROVEN_ERR_EOF	End of input.	Stopping the read loop - expected, not exceptional.
PROVEN_ERR_BUSY	A queue, lock, or resource is busy.	Backing off.
PROVEN_ERR_PERMISSION	Access denied.	Reporting; retrying will not help.
PROVEN_ERR_INVALID_FORMAT	A format or scan template is itself malformed.	Fixing the format string - a bug, not bad data.
PROVEN_ERR_EXISTS	A call asked to create a name found it already there: an exclusive create (PROVEN_FS_CREATE_NEW), proven_fs_mkdir, proven_fs_link.	Choosing another name, or using what is there - after checking it is what you expect.
PROVEN_ERR_TIMEOUT	A deadline you gave passed before the operation could finish.	Retrying, giving up, or closing - the object is still usable; nothing broke.
PROVEN_ERR_REFUSED	The other end answered and said no: nothing is listening at that address.	Trying another address, or reporting; an immediate retry gets the same answer.
PROVEN_ERR_RESET	The peer ended the connection abruptly, or it was aborted on the way.	Closing your end; whatever was in flight may not have arrived.
PROVEN_ERR_UNREACHABLE	No route to that network or host.	Reporting, or retrying later - the cause is outside the program.
PROVEN_ERR_UNTRUSTED	A peer's identity could not be verified: no acceptable chain from its certificate to anything you trust.	Refusing to continue.

PROVEN_ERR_EXPIRED	A certificate is past the end of its validity period.	Refusing; telling the user it expired. If everything fails this way, suspect the clock.
PROVEN_ERR_NOT_YET_VALID	A certificate's validity period has not begun.	Refusing; the clock is the usual culprit.
PROVEN_ERR_NAME_MISMATCH	A certificate is valid, and is for another name than the one you meant to reach.	Refusing: this is what a misdirected or intercepted connection looks like.
PROVEN_ERR_PROTOCOL	A peer broke the rules of a security protocol, or the two sides have nothing in common to speak. The TLS unit returns it for everything a peer does wrong that is not about its certificate.	Closing the connection.
PROVEN_ERR_CANCELLED	An operation started on an event loop was cancelled before it finished: `proven_loop_cancel`, or `proven_loop_close` on its socket (Chapter 15). It comes once, through the operation's own callback.	Freeing what the operation held - from then on the loop no longer touches it.

Room for your own codes. Every `proven_err_t` value is below `PROVEN_ERR_RESERVED_END` (0x1000), and that is a promise for every later version, not a description of this one: proven will never define a code at or above it. `PROVEN_ERR_LAST` names the highest code today and moves when one is added - compare against it, never against whichever code you happen to know is last. A program with failures of its own can therefore carry proven's codes unchanged in a wider integer and number its own from 0x1000 up: no remapping table, and a proven code it passes on still means exactly what the table above says.

```
/* An application's own codes, above everything proven will ever use. */
enum { APP_ERR_FIRST = PROVEN_ERR_RESERVED_END, APP_ERR_BAD_GLYPH = APP_ERR_FIRST, APP_ERR_TOO_DEEP };
_Static_assert(PROVEN_ERR_LAST < APP_ERR_FIRST, "proven's codes stay below ours");
int app_err = APP_ERR_TOO_DEEP;   /* a proven_err_t converts into the same int unchanged */
(void)app_err;
```

```
static inline int proven_is_ok(proven_err_t err);
#define PROVEN_IS_OK(err) proven_is_ok(err)
```

`proven_is_ok` returns non-zero when `err == PROVEN_OK`. `PROVEN_IS_OK` is the macro spelling for places where you want it to look like a macro; they do the same thing.

Correct:

```
proven_result_u8str_t s = proven_u8str_create(alloc, 32);
if (!proven_is_ok(s.err)) {
    return;   /* nothing was created, so there is nothing to destroy */
}
(void)proven_u8str_append(&s.value, PROVEN_LIT("ready"));
proven_u8str_destroy(alloc, &s.value);
```

Wrong - testing the error as a bare truth value:

```
proven_result_u8str_t s = proven_u8str_create(alloc, 32);
if (s.err) {
    /* wrong: works today only because PROVEN_OK is 0. It reads as "if there is
```

```
       an error" to someone who already knows that, and as nothing at all to
       everyone else. Say what you mean: !proven_is_ok(s.err). */
    return s.err;
}
```

Wrong - the mistake this whole design exists to prevent:

```
proven_result_u8str_t s = proven_u8str_create(alloc, 32);
(void)proven_u8str_append(&s.value, text);   /* wrong: s.value may be garbage */
proven_u8str_destroy(alloc, &s.value);       /* wrong: destroying what was never created */
```

Reading `.value` before checking `.err` is the one error this convention cannot catch for you. The compiler will not complain - the struct exists either way - so it is worth making a habit of writing the check on the line immediately after the call.

Worked example: errors as values

This program is compiled and run by the test suite, so it cannot fall out of date. It shows the two shapes a fallible call takes - a bare `proven_err_t` when there is nothing to hand back, and a `proven_result_*_t` when there is - and why the value inside a result means nothing until you have checked the error beside it.

```
/*
 * Errors are values in proven: a fallible call hands back either an error or a
 * result, and the compiler makes you look at it. There is nothing to unwind and
 * nothing global to consult.
 */

/* A fallible operation returns proven_err_t when it has no value to give back. */
static proven_err_t write_greeting(proven_u8str_t *out) {
    return proven_u8str_append(out, PROVEN_LIT("hello"));
}

/* When there IS a value, it comes wrapped with the error that guards it. The
 * value is only meaningful once you have checked `err`. */
static proven_result_size_t half(proven_size_t n) {
    proven_result_size_t res = {0};
    if (n % 2 != 0) {
        res.err = PROVEN_ERR_INVALID_ARG;   /* leave res.value at 0: it means nothing */
        return res;
    }
    res.err = PROVEN_OK;
    res.value = n / 2;
    return res;
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- checking a plain proven_err_t ------------------------------------ */
    proven_result_u8str_t s = proven_u8str_create(alloc, 32);
    EXAMPLE_REQUIRE(proven_is_ok(s.err), "creating a 32-byte string should succeed");

    proven_err_t err = write_greeting(&s.value);
    if (!proven_is_ok(err)) {
        /* Nothing was appended, and the string is still valid: proven's
         * grow-style operations are failure-atomic. */
        proven_u8str_destroy(alloc, &s.value);
        return 1;
    }
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&s.value), PROVEN_LIT("hello")),
                    "the greeting should have been appended");
```

```
    /* --- checking a result struct ------------------------------------------ */
    proven_result_size_t ok = half(10);
    EXAMPLE_REQUIRE(proven_is_ok(ok.err), "10 is even, so halving it must succeed");
    EXAMPLE_REQUIRE(ok.value == 5, "half of 10 is 5");

    proven_result_size_t bad = half(7);
    EXAMPLE_REQUIRE(bad.err == PROVEN_ERR_INVALID_ARG, "7 is odd, so halving it must fail");
    /* bad.value is NOT to be read. It is 0 here, but that is an implementation
     * detail of this function, not a promise of the result type. */

    /* --- the error is impossible to drop by accident ----------------------- */
    /* proven_u8str_append is [[nodiscard]], so this would be a compile error:
     *
     *     proven_u8str_append(&s.value, PROVEN_LIT("!"));
     *
     * If you really do want to ignore a failure, you have to say so: */
    (void)proven_u8str_append(&s.value, PROVEN_LIT("!"));

    printf("greeting: %s\n", proven_u8str_as_cstr(&s.value));

    proven_u8str_destroy(alloc, &s.value);
    return EXAMPLE_OK();
}
```

2. The types, and why they are spelled differently

Why not just use `int` and `char`?
C's built-in types are famously vague. `int` is at least 16 bits, `long` is at least 32, and `char` may be signed or unsigned depending on the compiler and the target. Code that assumes otherwise works on your machine and breaks on someone else's, and the break is usually silent.

C99 fixed the width problem with `<stdint.h>` (`int32_t`, `uint64_t` and friends). This library uses those underneath and gives them its own names, for one reason: **a single spelling that means the same thing on every target the library supports**, including the freestanding ones where `<stdint.h>` may be the only header available.

There is a second, subtler reason, and it is the one that matters in practice: `proven_u8` and `proven_byte_t` are both 8 bits, and they mean different things.

- `proven_u8` is a **number** between 0 and 255. Add it, compare it, print it.
- `proven_byte_t` is a **byte of some object's representation**. It is an alias of `unsigned char`, which is the one type C allows you to use to inspect the raw bytes of anything without invoking undefined behaviour.

Using `proven_byte_t` for raw memory is not a style preference. C's strict-aliasing rules say that reading an object through a pointer of the wrong type is undefined, with `unsigned char` as the explicit exception. Every buffer, view, and hash input in this library is `proven_byte_t` for exactly that reason.

Reference

Name	Meaning	Notes
`proven_i8`, `proven_i16`, `proven_i32`, `proven_i64`	Signed fixed-width integers	Aliases of the standard fixed-width types.
`proven_u8`, `proven_u32`, `proven_u64`	Unsigned fixed-width integers	`proven_u8` is **numeric**; use `proven_byte_t` for raw object bytes.

`proven_u16`	16-bit code unit type	Uses `char16_t` when `<uchar.h>` is available, otherwise `uint_least16_t`. The U16 string APIs use it for UTF-16 code units.
`proven_byte_t`	Byte-level object representation	Alias of `unsigned char`; the type you may legally inspect any object's bytes through.
`proven_size_t`	Size and index type	Alias of `size_t`. Unsigned - see section 4 before you subtract two of them.
`proven_ptrdiff_t`	Pointer difference, signed offset	Alias of `ptrdiff_t`.
`proven_intptr_t`, `proven_uintptr_t`	Pointer-sized integers	Only for explicit pointer-to-integer work, such as the arena's range checks.

```
typedef struct {
    proven_err_t  err;
    proven_size_t value;
} proven_result_size_t;
```

`proven_result_size_t` returns either a size or an error; `value` is valid only when `err == PROVEN_OK`. It is what you get back from partial appends, file reads and writes, and size queries - anywhere the answer is "how many", and the operation could fail.

Wrong - treating a byte as a small number's storage:

```
proven_u8 *raw = (proven_u8 *)&some_struct;   /* wrong type for raw inspection */
for (size_t i = 0; i < sizeof some_struct; i++) hash ^= raw[i];
```

It happens to work on every mainstream compiler, and it is still the wrong type to have written: `proven_byte_t` is the one with the aliasing guarantee behind it, and using it documents that you are looking at representation rather than at a number.

3. Memory views: a pointer and a length, together

The problem: pointers forget

A pointer says where something starts and nothing else. Everything C has ever done about the "where does it end" half has been a convention layered on top:

- **A NUL terminator**, for strings - which costs an *O(n)* scan every time you want the length, cannot represent text containing a zero byte, and produces a buffer overrun the moment the terminator is missing.
- **A separate length parameter**, for everything else - which works right up until someone passes the wrong one, because nothing ties the two arguments together.

```
void process(const unsigned char *data, size_t len);
process(buf, sizeof buf);        /* fine */
process(buf, sizeof(buf) - 1);   /* also compiles; now the last byte is invisible */
process(other, sizeof buf);      /* also compiles; now it reads past the end */
```

Every one of those is a legal call. The relationship between the pointer and the length exists only in the programmer's head.

What this library does instead

A **view** is a pointer and a size in one struct, passed by value. They cannot get separated, because they are one thing.

```
typedef struct { const proven_byte_t *ptr; proven_size_t size; } proven_mem_view_t;  /* read-only */
typedef struct {       proven_byte_t *ptr; proven_size_t size; } proven_mem_mut_t;   /* writable  */
typedef struct {       proven_byte_t *ptr; proven_size_t size; } proven_mem_t;       /* owned     */
```

They are two words. Copying one is free, and there is no allocation anywhere in sight. What a view does *not* do is own anything: it points at bytes that belong to someone else, and it becomes invalid the moment they do. That rule is contract 2 from Chapter 0.

The three spellings differ only in intent, and the intent is the point:

- `proven_mem_view_t` - **borrowed, read-only.** The `const` is on the pointed-to bytes, so the compiler enforces it.
- `proven_mem_mut_t` - **borrowed, writable.** You may write through it; you still do not own it.
- `proven_mem_t` - **owned.** Somebody must free this with the allocator that produced it. The type does not enforce that; it announces it.

And two result wrappers, for the operations that can fail:

```
typedef struct { proven_err_t err; proven_mem_mut_t  value; } proven_result_mem_mut_t;
typedef struct { proven_err_t err; proven_mem_view_t value; } proven_result_mem_view_t;
```

`proven_result_mem_mut_t` is what an allocator hands back. `proven_result_mem_view_t` is what a *checked* slice hands back.

Slicing, checked and unchecked

Taking a sub-range of a view is the most common thing you will do with one, and it comes in two forms on purpose:

API	Purpose	Return
`proven_mem_view_from_owned(mem)`	Read-only view over an owned block.	`proven_mem_view_t`.
`proven_mem_mut_from_owned(mem)`	Writable view over an owned block.	`proven_mem_mut_t`.
`proven_mem_view_slice_checked(view, offset, size)`	Sub-view, validated.	`proven_result_mem_view_t`.
`proven_mem_view_slice_unchecked(view, offset, size)`	Sub-view, **not** validated.	`proven_mem_view_t`.
`proven_mem_mut_slice_checked(mut, offset, size)`	Writable sub-slice, validated.	`proven_result_mem_mut_t`.
`proven_mem_mut_slice_unchecked(mut, offset, size)`	Writable sub-slice, **not** validated.	`proven_mem_mut_t`.
`proven_range_contains_ptr(base, cap, ptr, size, out_offset)`	Is this pointer range inside that allocation? Integer address comparison, no pointer arithmetic on unrelated pointers.	`_Bool`.
`proven_memcmp(s1, s2, size)`	Compare raw memory.	Zero if equal; sign by byte order (unsigned).
`proven_mem_copy(dst, dst_cap, src)`	Bounded copy of a view into `dst`.	`PROVEN_OK`; `PROVEN_ERR_OUT_OF_BOUNDS` if it would not fit - **and nothing is written**;

		`PROVEN_ERR_INVALID_ARG` on a null pointer with a non-zero size. Regions must not overlap.
`proven_mem_move(dst, dst_cap, src)`	As `proven_mem_copy`, but the regions may overlap.	As above.
`proven_mem_equal_ct(a, b)`	Whether two ranges are equal, in time that does not depend on where they differ. For comparing against a secret - a MAC, a token - where `proven_memcmp` would leak how many leading bytes matched.	`bool`; ranges of different sizes are unequal.
`proven_mem_wipe(mem)`	Overwrite with zeros in a way the compiler may not remove. For a key or password that is done with: an ordinary clearing loop before a buffer goes out of scope is deleted as dead code.	none.

The checked form's exact behaviour:

- `view.size > 0 && view.ptr == NULL` -> `PROVEN_ERR_INVALID_ARG` (a size with no memory behind it is a bug, not an empty view).
- The requested range falls outside the view -> `PROVEN_ERR_OUT_OF_BOUNDS`.
- `size == 0` -> an empty view (`ptr == NULL`, `size == 0`) with `PROVEN_OK`.

Use the checked form by default. The unchecked form exists for the case where you have already proved the range in the lines above - inside a loop whose bounds you computed, for instance - and the second check would be pure cost. That is a real case, and it is narrower than it feels.

Wrong - unchecked slicing on numbers that came from outside:

```
proven_mem_view_t part = proven_mem_view_slice_unchecked(view, user_offset, user_size);
/* wrong: user_offset and user_size were never validated. This is the buffer
   overrun from the top of section 3, reintroduced through a safer-looking API. */
```

Wrong - rejecting the empty view:

```
if (!view.ptr) return PROVEN_ERR_INVALID_ARG;   /* wrong: {NULL, 0} is legal */
```

An empty view is a normal value: a zero-length slice, a file with no content, a string that was just reset. The test for a *malformed* view is a null pointer with a non-zero size:

```
proven_mem_view_t view = {0};   /* size 0, ptr NULL: a legal empty view */
proven_err_t err = PROVEN_OK;

if (view.size > 0 && !view.ptr) {
    err = PROVEN_ERR_INVALID_ARG;   /* only a null pointer with bytes behind it is a bug */
}
(void)err;
```

Passing a view to a macro

Several entry points are function-like macros: `PROVEN_ARG`, `proven_scan_fmt`, the print and `append_fmt` families, and the `PROVEN_MAP_*_U8_*` wrappers. The preprocessor splits macro arguments at every comma that is not inside parentheses - and the comma inside a compound literal's braces is not.

Passing `(proven_u8str_view_t){ p, n }` therefore hands the macro two arguments, and the compiler's error names an internal function you never called. Pass a variable, a `PROVEN_LIT`, or wrap the compound literal in parentheses:

```
const proven_byte_t bytes[] = { '4', '2' };
proven_u8str_view_t digits = { bytes, sizeof bytes };
proven_println("{}", PROVEN_ARG(digits));                                  /* a variable */
proven_println("{}", PROVEN_ARG(((proven_u8str_view_t){ bytes, 1 })));     /* parentheses */
```

4. Size arithmetic that cannot wrap

Why a multiplication needs a function call

`proven_size_t` is unsigned, and unsigned arithmetic in C does not overflow - it *wraps*, silently and legally, modulo 2^64. That single rule is behind a large fraction of the industry's allocation bugs:

```
proven_size_t total = count * sizeof(ex_item_t);   /* wrong: wraps for large count */
void *p = malloc(total);                          /* succeeds, and is far too small */
items[count - 1] = x;                             /* writes way past the end */
```

With `count = 2^61` and a 16-byte item, `total` is `0`. `malloc(0)` succeeds. Every subsequent write is out of bounds, and nothing anywhere reported an error. The same shape appears whenever a size is computed from data that came from outside the program - a file header, a network packet, a user-supplied count.

Subtraction has the mirror problem, and it is easier to hit:

```
proven_size_t remaining = view.size - offset;   /* wrong when offset > size: a huge number */
```

What this library does instead

Three macros that do the arithmetic and tell you whether it fit.

```
#define PROVEN_CKD_ADD(res, a, b)   /* true on overflow */
#define PROVEN_CKD_SUB(res, a, b)
#define PROVEN_CKD_MUL(res, a, b)
```

Macro	Computes	Returns	Writes `*res`
`PROVEN_CKD_ADD(res, a, b)`	`a + b`	**true on overflow**, false on success	Only when it fits
`PROVEN_CKD_SUB(res, a, b)`	`a - b`	**true on overflow** (including unsigned underflow)	Only when it fits
`PROVEN_CKD_MUL(res, a, b)`	`a * b`	**true on overflow**	Only when it fits
`PROVEN_SIZE_MAX`	-	The largest value a `proven_size_t` can hold	-

- `res` is a **pointer** to where the answer goes.
- They return **true on overflow**, false on success. That reads backwards the first time; the convention comes from C23's `<stdckdint.h>`, which these use when the compiler provides it, and from the GCC/Clang `__builtin_*_overflow` family otherwise. Read the call as *"did this overflow?"* and it comes out right.
- `PROVEN_SIZE_MAX` is there for the cases where you want to compare against the limit directly rather than compute and check.

Correct:

```
typedef struct { int id; double weight; } ex_item_t;

proven_size_t count = 1024;
proven_size_t total = 0;
proven_err_t err = PROVEN_OK;

if (PROVEN_CKD_MUL(&total, count, sizeof(ex_item_t))) {
```

```
    err = PROVEN_ERR_OVERFLOW;   /* refuse to allocate a size that wrapped */
} else {
    proven_result_mem_mut_t block = alloc.alloc_fn(alloc.ctx, total, alignof(ex_item_t));
    err = block.err;
    if (proven_is_ok(err)) {
        alloc.free_fn(alloc.ctx, block.value.ptr);
    }
}
(void)err;
```

Wrong:

```
proven_size_t total = count * sizeof(ex_item_t);   /* wrong: may wrap silently */
```

You do not need these for every arithmetic expression in your program. You need them wherever a size is **computed from a value you did not choose yourself** - and that is precisely where the bugs are.

5. Alignment

What alignment is, and why the library talks about it

Every type in C has an alignment: an address it must start on for the hardware to load it efficiently, or at all. A `double` typically wants an address divisible by 8. Some architectures fault on a misaligned load; x86 merely makes it slow; and either way, C says accessing an object through a misaligned pointer is undefined behaviour.

You have not had to think about this, because `malloc` returns memory suitably aligned for anything. That guarantee is exactly what disappears the moment you hand out memory yourself - and handing out memory yourself is what Chapter 2's arenas and pools do. An arena that bumps a pointer by 13 bytes and hands you the result has just given you a misaligned `double`.

So the helpers here exist for the allocators, and for you if you write one.

Reference

Macro	Meaning
`PROVEN_DEFAULT_ALIGNMENT`	The default the library uses when you do not say otherwise. Currently 8.
`PROVEN_MAX_ALIGN`	`alignof(max_align_t)` - enough for any built-in type.

Function	Purpose	Return
`proven_is_pow2(x)`	Is `x` a non-zero power of two? Alignments must be.	`bool`.
`proven_mem_align_up(addr, align)`	Round a size or address up to the next multiple of `align`.	The aligned value, or **0** if `align` is invalid or the rounding overflowed.
`proven_uintptr_align_up(addr, align)`	The same, for a `proven_uintptr_t` address.	As above.

Note the error convention: these return **0 on failure**, because they have no room for an error code. Zero is never a valid aligned address, so testing for it is unambiguous.

Correct:

```
typedef struct { int id; double weight; } ex_item_t;

proven_size_t size = 100;
proven_size_t aligned = proven_mem_align_up(size, alignof(ex_item_t));
```

```
proven_err_t err = (aligned == 0) ? PROVEN_ERR_OVERFLOW : PROVEN_OK;
(void)err;
```

Wrong - the classic bit-twiddling version:

```
proven_size_t aligned = (size + align - 1) & ~(align - 1);   /* wrong: may overflow */
```

That line is in a great deal of production code and is correct for every input except the ones near the top of the range, where `size + align - 1` wraps to a small number and the result is an address *below* where you started.

Worked example: views, slicing, and alignment together

The three ideas in this chapter - a view that carries its own length, a slice that is checked before it is taken, and a boundary that has to be a power of two -- only show their point when a program uses them at once. This one keeps a small table of fixed-size records in a single block it owns, and then does the four things any such program has to do: hand the table to a reader that must not modify it, hand one row to code that may modify only that row, delete a row by moving the rows after it down over it, and decide whether a pointer that arrived from somewhere else even points into the table.

Two of those are the places C is most often quietly wrong. Moving overlapping bytes is not what `proven_mem_copy` promises, so it has a separate call, `proven_mem_move`. And comparing two pointers with `<` when they may belong to different objects is undefined behaviour - the compiler is entitled to assume it never happens - so "is this pointer inside my buffer?" is asked with `proven_range_contains_ptr` rather than with `>=` and `<`.

This program is compiled and run by the test suite, so it cannot fall out of date.

```
#include <string.h>

/*
 * One owned buffer, three ways of talking about it.
 *
 * The program holds a small table of fixed-size records in a single block of
 * memory it owns, and then does the four things every such program has to do:
 *
 *   1. hand the table to a reader that must not modify it (a read-only view),
 *   2. hand one row to a writer that may modify only that row (a slice),
 *   3. delete a row by moving the rows after it down over it (an overlapping
 *      copy, which plain copying is not allowed to do),
 *   4. take a pointer somebody else returned and decide whether it even points
 *      into the table before using it as a row index.
 *
 * Every step is a place where a bare `char *` loses the length and the program
 * finds out later. The types here carry the length with the pointer, so the
 * bounds question is answered where the mistake would be made.
 */

#define ROW_SIZE  8u
#define ROW_COUNT 4u

/* A reader gets a view: it can read every byte and write none of them. The
 * const in the type is not advice - assigning through it does not compile. */
static proven_size_t count_nonzero(proven_mem_view_t table) {
    proven_size_t n = 0;
    for (proven_size_t i = 0; i < table.size; ++i) {
        if (table.ptr[i] != 0) {
            ++n;
        }
    }
```

```
    return n;
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* Allocate the table. The allocator returns a read-write slice, which is
     * what an owner holds: a pointer and the size that goes with it. */
    proven_result_mem_mut_t got = alloc.alloc_fn(alloc.ctx, ROW_SIZE * ROW_COUNT, alignof(proven_u32));
    EXAMPLE_REQUIRE(proven_is_ok(got.err), "allocating the record table must succeed");
    if (!proven_is_ok(got.err)) {
        return 1;
    }

    /* proven_mem_t is the "I own this" type. Keeping the owned block in one
     * variable, and handing out views and slices derived from it, is the whole
     * ownership discipline this library asks for. */
    proven_mem_t owned = { .ptr = got.value.ptr, .size = got.value.size };
    proven_mem_mut_t table = proven_mem_mut_from_owned(owned);

    /* Fill row i with the byte value i + 1, so a moved row is recognisable. */
    for (proven_u32 row = 0; row < ROW_COUNT; ++row) {
        proven_result_mem_mut_t slot = proven_mem_mut_slice_checked(table, row * ROW_SIZE, ROW_SIZE);
        EXAMPLE_REQUIRE(proven_is_ok(slot.err), "every row of the table must be in bounds");
        if (!proven_is_ok(slot.err)) {
            alloc.free_fn(alloc.ctx, owned.ptr);
            return 1;
        }
        memset(slot.value.ptr, (int)(row + 1), slot.value.size);
    }

    /* 1. Read-only use. The reader cannot modify the table and cannot run off
     *    the end of it, because it was handed both facts at once. */
    proven_mem_view_t read_only = proven_mem_view_from_owned(owned);
    EXAMPLE_REQUIRE(count_nonzero(read_only) == ROW_SIZE * ROW_COUNT,
                    "every byte of every row was written");

    /* 2. Asking for a row that does not exist is an error you can handle, not a
     *    crash you have to debug. Row 4 of a 4-row table starts one row past
     *    the end. */
    proven_result_mem_mut_t past_end = proven_mem_mut_slice_checked(table, ROW_COUNT * ROW_SIZE, ROW_SIZE);
    EXAMPLE_REQUIRE(past_end.err == PROVEN_ERR_OUT_OF_BOUNDS,
                    "slicing past the end must report OUT_OF_BOUNDS, not return a bad slice");

    /* 3. Delete row 1 by moving rows 2 and 3 down one row. Source and
     *    destination overlap, so this is the one case plain copying is not
     *    allowed to handle: proven_mem_copy documents non-overlapping regions,
     *    proven_mem_move is the one that accepts them. */
    proven_mem_view_t tail = {
        .ptr  = table.ptr + 2 * ROW_SIZE,
        .size = 2 * ROW_SIZE
    };
    proven_err_t moved = proven_mem_move(table.ptr + 1 * ROW_SIZE, table.size - 1 * ROW_SIZE, tail);
    EXAMPLE_REQUIRE(proven_is_ok(moved), "moving the tail of the table down must succeed");
    EXAMPLE_REQUIRE(table.ptr[1 * ROW_SIZE] == 3, "row 2 moved into row 1's place");
    EXAMPLE_REQUIRE(table.ptr[2 * ROW_SIZE] == 4, "row 3 moved into row 2's place");

    /* The move is bounded like everything else: a destination too small to hold
     * the source is refused, and nothing is written. */
    proven_err_t refused = proven_mem_move(table.ptr, ROW_SIZE, tail);
```

```
    EXAMPLE_REQUIRE(refused == PROVEN_ERR_OUT_OF_BOUNDS,
                    "a move that would not fit must be refused before it writes");

    /* 4. A pointer from elsewhere. Comparing unrelated pointers with < is
     *    undefined behaviour in C - the compiler is allowed to assume it never
     *    happens - so the question "does this pointer point into my buffer?"
     *    has to be asked with a function that does the comparison correctly. */
    proven_size_t offset = 0;
    const proven_byte_t *from_elsewhere = table.ptr + 2 * ROW_SIZE;
    bool inside = proven_range_contains_ptr(table.ptr, table.size, from_elsewhere, ROW_SIZE, &offset);
    EXAMPLE_REQUIRE(inside, "a pointer to row 2 is inside the table");
    EXAMPLE_REQUIRE(offset / ROW_SIZE == 2, "and its offset recovers the row index");

    proven_u8 unrelated[ROW_SIZE] = { 0 };
    EXAMPLE_REQUIRE(!proven_range_contains_ptr(table.ptr, table.size, unrelated, ROW_SIZE, NULL),
                    "a pointer into a different object is not inside the table");

    /* A row that starts inside but ends outside is also outside: the check is
     * about the whole range, not just where it begins. */
    EXAMPLE_REQUIRE(!proven_range_contains_ptr(table.ptr, table.size,
                                               table.ptr + table.size - 1, ROW_SIZE, NULL),
                    "a range that starts inside but runs past the end is refused");

    /* 5. Once the range check has proved the row is in bounds, the unchecked
     *    slice is the right call: it does no work, and the proof is the line
     *    above it rather than a comment. Never write it without that proof. */
    proven_mem_mut_t row2 = proven_mem_mut_slice_unchecked(table, offset, ROW_SIZE);
    EXAMPLE_REQUIRE(row2.size == ROW_SIZE && row2.ptr[0] == 4,
                    "the unchecked slice names the row the checked test just proved");

    /* 6. Alignment, in the one place a caller meets it: laying two differently
     *    aligned things out inside one block. The address of the second one has
     *    to be rounded up, and rounding up is only defined for a power-of-two
     *    boundary - so the boundary is checked first. */
    proven_size_t want_align = alignof(proven_u32);
    EXAMPLE_REQUIRE(proven_is_pow2(want_align), "an alignment boundary must be a power of two");
    EXAMPLE_REQUIRE(!proven_is_pow2(24u), "24 is not a power of two, so it is not a valid boundary");

    proven_uintptr_t raw     = (proven_uintptr_t)(table.ptr + 1);   /* deliberately odd */
    proven_uintptr_t aligned = proven_uintptr_align_up(raw, want_align);
    EXAMPLE_REQUIRE(aligned >= raw, "aligning up never moves backwards");
    EXAMPLE_REQUIRE(aligned % want_align == 0, "and the result is on the boundary asked for");
    EXAMPLE_REQUIRE(aligned - raw < want_align, "the padding is never more than one boundary");

    /* A boundary that is not a power of two returns 0 rather than a plausible
     * wrong address, so the mistake stops here instead of downstream. */
    EXAMPLE_REQUIRE(proven_uintptr_align_up(raw, 24u) == 0,
                    "aligning to a non-power-of-two returns 0, not a wrong address");

    alloc.free_fn(alloc.ctx, owned.ptr);
    return EXAMPLE_OK();
}
```

Wrong - the same deletion written with the copy that forbids overlap:

```
/* rows 2..3 moved down onto row 1: source and destination overlap */
proven_mem_copy(table.ptr + ROW_SIZE, table.size - ROW_SIZE, tail);   /* wrong */
```

`proven_mem_copy` documents non-overlapping regions. It will not report this; it will simply be free to copy in whatever order is fastest, and on the day that order changes the table quietly ends up holding two copies of one row.

Wrong - the bounds question asked with plain pointer comparison:

```
if (ptr >= table.ptr && ptr + size <= table.ptr + table.size) { /* wrong */ }
```

If `ptr` came from a different allocation, this comparison is undefined behaviour, not merely a comparison that returns the wrong answer.

6. Panic: when there is no one left to return an error to

Why a library that returns errors has a panic at all
Everything in this chapter has argued that failure should be a value you can inspect. A panic is the admission that this is not always possible.

Some APIs have no error channel by construction. `proven_arena_alloc_or_panic()` returns a memory block, not a result, because it exists for the setup phase of a program where an allocation failure means the program cannot run at all and there is nothing sensible to return. In a freestanding build there may be no `abort()`, no `stderr`, and nothing to unwind to.

So the library raises a panic by calling `proven_panic()`, which dispatches to a handler you can replace. It does not call `exit`, print to `stderr`, or assume an operating system.

```
typedef void (*proven_panic_handler_t)(const char *msg);

void proven_panic(const char *msg);
void proven_set_panic_handler(proven_panic_handler_t handler);
```

API	Purpose	Notes
`proven_panic(msg)`	Raise a panic. Dispatches to the installed handler.	Called by the library's `_or_panic` APIs; you can call it too.
`proven_set_panic_handler(handler)`	Install a handler.	Pass `NULL` to restore the default. Not thread-safe - install it during start-up, before other threads exist.
`proven_panic_handler_t`	`void (*)(const char *msg)`	The handler must not return.

The default handler traps: `__builtin_trap()` on GCC and Clang, and an infinite loop on any other compiler. The dispatch goes through a function pointer rather than a weak symbol, so it links the same way on ELF and on PE/COFF (Windows, mingw-w64) toolchains.

Installing your own (a file-scope function, so this is a listing rather than something you paste inside another function):

```
static void my_panic(const char *msg) {
    log_critical(msg);            /* your logger */
    for (;;) {
        /* reset, halt, or wait for a debugger */
    }
}

proven_set_panic_handler(my_panic);   /* pass NULL to restore the default */
```

A panic handler must not return. If it does, the `_or_panic` family has nothing to give back and the validity of its result is not guaranteed.

Wrong - a handler that returns:

```
static void my_panic(const char *msg) {
    fprintf(stderr, "%s\n", msg);   /* wrong: this returns, and then the caller
                                       proceeds with a block that was never allocated */
}
```

Wrong - reaching for `_or_panic` in ordinary code:

```
proven_mem_mut_t block = proven_arena_alloc_or_panic(&arena, n);
/* wrong for anything that handles input: an arena that is merely full has just
   killed the process. Use proven_arena_alloc and read the error. */
```

7. Version macros

Compile-time identification, for diagnostics and for code that must adapt to the library version.

```
#define PROVEN_VERSION_MAJOR  0
#define PROVEN_VERSION_MINOR  97
#define PROVEN_VERSION_PATCH  0
#define PROVEN_VERSION_STRING "proven_c_lib-v0.97.0"
#define PROVEN_VERSION_ENCODE(major, minor, patch) ((major) * 1000000L + (minor) * 1000L + (patch))
#define PROVEN_VERSION_NUM    PROVEN_VERSION_ENCODE(PROVEN_VERSION_MAJOR, PROVEN_VERSION_MINOR,
PROVEN_VERSION_PATCH)
```

The version is semantic - `MAJOR.MINOR.PATCH`, from v0.0.1. A PATCH release fixes or documents and changes nothing a caller has to; a MINOR release adds without breaking an existing call; a MAJOR release may break one. While MAJOR is 0 the API may still move between minors.

`PROVEN_VERSION_STRING` is what you print in a build report or a `--version` flag. The three numbers are for code that must adapt to the library version, and `PROVEN_VERSION_NUM` folds them into one integer for `#if` - compare it with `PROVEN_VERSION_ENCODE`, never with a literal: `#if PROVEN_VERSION_NUM >= PROVEN_VERSION_ENCODE(0, 1, 0)`.

Releases before v0.0.1 were numbered by date (`v26.MM.DDx`), and `PROVEN_VERSION_NUM` was that date. The number restarted with the scheme, so a comparison against an old date value is wrong on every current release; rewrite it with `PROVEN_VERSION_ENCODE`.

The numbers and the string are checked against `include/proven/version.h` by the build, and the string is checked against the numbers, so the manual cannot drift from the header. (It did, once, and this sentence is why it will not again: the string was gated and the rest was not.)

8. Examples and misuse cases

Safe result handling

```
proven_byte_t record[16] = {0};
proven_mem_view_t view = { .ptr = record, .size = sizeof record };

proven_result_mem_view_t part = proven_mem_view_slice_checked(view, 4, 8);
if (!proven_is_ok(part.err)) {
    return;   /* the range was not inside the view */
}

proven_byte_t payload[8];
proven_err_t err = proven_mem_copy(payload, sizeof payload, part.value);
(void)err;
```

Unchecked slicing only after proof

Correct when the caller already proved the range:

```
proven_byte_t record[16] = {0};
proven_mem_view_t view = { .ptr = record, .size = sizeof record };
proven_size_t offset = 4;
proven_size_t size = 8;

if (offset <= view.size && size <= view.size - offset) {
    proven_mem_view_t part = proven_mem_view_slice_unchecked(view, offset, size);
    (void)part;   /* proved in range: safe to read */
}
```

Note the shape of that test. `size <= view.size - offset` is written that way, rather than the more natural `offset + size <= view.size`, precisely because of section 4: the sum can wrap and the difference cannot, given the `offset <= view.size` check that precedes it.

Wrong:

```
proven_mem_view_t part = proven_mem_view_slice_unchecked(view, user_offset, user_size);
/* wrong: user_offset and user_size were not validated */
```

Empty views are allowed

A zero-size view may have a null pointer. Do not reject it blindly.

Wrong:

```
if (!view.ptr) return PROVEN_ERR_INVALID_ARG; /* wrong for empty views */
```

Better:

```
proven_mem_view_t view = {0};   /* size 0, ptr NULL: a legal empty view */
proven_err_t err = PROVEN_OK;

if (view.size > 0 && !view.ptr) {
    err = PROVEN_ERR_INVALID_ARG;   /* only a null pointer with bytes behind it is a bug */
}
(void)err;
```

Where to go next

Chapter 2 is where these types start doing work: allocators produce `proven_mem_mut_t`, arenas use the alignment helpers from section 5, and every growable container in the library takes the allocator as a parameter for the reasons section 1 gave for taking the error as a return value - the cost should be visible in the signature.

Chapter 2: Allocation - Heap, Arenas, Pools, and Buffers

Part II - The vocabulary every program uses. Prerequisite: Chapter 1. After this chapter you can choose an allocation strategy deliberately instead of reaching for `malloc` by reflex, and you will know which of the three costs you are paying.

This chapter covers `heap.h`, `arena.h`, `pool.h`, `allocator.h`, and `buffer.h`. The thread-safety and pointer-provenance material that used to be section 7 here now lives in Chapter 6, with the rest of the concurrency subject - it was the hardest material in the book sitting in one of the first chapters.

Table of contents

1. Why allocation is a parameter, and the heap allocator
2. Arena: many things, one lifetime
3. Pool: many things, one size
4. The allocator trait
5. Raw byte buffer
6. Examples and misuse cases
7. Catching the wrong allocator: `alloc_check.h`

1. Why allocation is a parameter, and the heap allocator

The problem with `malloc`

`malloc` is a global. Any function may call it, nothing in a signature says whether a function does, and every call goes to the same general-purpose allocator no matter what the memory is for.

That has four consequences you have probably met:

- **You cannot tell from a signature whether a function allocates.** `char *build_message(int n)` might allocate, might return a pointer into a static buffer, might return a literal. The type is identical in all three cases, so the caller cannot know whether to free, and the answer lives in a comment that may be wrong.
- **You cannot change the strategy for one part of a program.** If a parser makes ten thousand small allocations that all die at the end of the parse, `malloc` and `free` will do ten thousand general-purpose allocations and ten thousand frees. A bump allocator would do one. There is no way to say so without rewriting every call.
- **You cannot test the failure path.** Making `malloc` fail on demand means intercepting it globally - `LD_PRELOAD`, a linker trick, a `#define malloc my_malloc` that also catches the library's internal calls. Meanwhile the branch you most want to test is the one that runs when memory runs out.
- **You cannot use it at all where there is no heap.** Firmware, a kernel, a bootloader: no `malloc`, and every library that assumes one is unusable.

`malloc` is not badly designed. It is a fixed policy, hardcoded into every call site by the fact that it takes no parameter saying otherwise.

What this library does instead

An allocator is a value, and it is a parameter. A function that can allocate takes a `proven_allocator_t`; a function that cannot, does not. That is the whole idea, and every consequence follows from it:

```
proven_err_t proven_u8str_append(proven_u8str_t *str, proven_u8str_view_t data);
proven_err_t proven_u8str_append_grow(proven_allocator_t alloc, proven_u8str_t *str, proven_u8str_view_t
data);
```

Read the signatures. The first cannot grow the string, so it fails when the text does not fit. The second can, and says so by taking the allocator. You never have to wonder which one you called.

The same code now works with three different strategies, chosen by the caller:

Allocator	Get one from	Frees individually?	Use it when
Heap	`proven_heap_allocator()`	Yes	The general case. Objects with unrelated lifetimes.
Arena	`proven_arena_create(backing)`, then `proven_arena_as_allocator(&a)`	**No** - free is a no-op; you reset or destroy the whole thing	Many allocations that all die at the same moment: one request, one frame, one parse.
Pool	`proven_pool_init(&p, base, size, align, bin_cap)`, then `proven_pool_as_allocator(&p)`	Yes, into a free list	Many objects of **one fixed size**, allocated and freed repeatedly.

Start with the heap. Reach for the other two when you have a reason, and the reason is usually a measurement.

The heap allocator

```
proven_allocator_t proven_heap_allocator(void);
```

This is `malloc`, `realloc` and `free` wearing the library's interface. It is what you should use unless you have a specific reason not to, and there is nothing clever about it - which is the point. Every example in this manual that does not have a reason to do otherwise uses it:

```
proven_allocator_t heap = proven_heap_allocator();

proven_result_u8str_t s = proven_u8str_create(heap, 64);
if (!proven_is_ok(s.err)) {
    return;   /* nothing was created, so there is nothing to destroy */
}
(void)proven_u8str_append(&s.value, PROVEN_LIT("ready"));
proven_u8str_destroy(heap, &s.value);   /* the SAME allocator */
```

The value is four words - a context pointer and three function pointers - and it is passed by value. Copying it is free, storing it in your own struct is fine, and it does not need to be destroyed.

In freestanding builds `proven_heap_allocator` does not exist. There is no `malloc` to wrap. That is not a limitation to work around; it is the reason the whole library takes allocators as parameters, and it is why an arena over a static array makes the same code run on a microcontroller. See freestanding mode.

Wrong - destroying with a different allocator than you created with:

```
proven_result_u8str_t s = proven_u8str_create(arena_alloc, 64);
proven_u8str_destroy(heap_alloc, &s.value);   /* wrong: heap free on arena memory */
```

The object does not remember which allocator produced it - that is what keeps it small. Nothing checks this today, and the failure is heap corruption that surfaces somewhere else, later. Pair them by construction: keep the allocator next to the object, or pass both together.

2. Arena: many things, one lifetime

The problem: many small allocations with the same death date

Consider parsing a configuration file. You allocate a string for each key, a string for each value, a node for each section - a few thousand small allocations. Then the parse finishes, you build your result, and every one of those allocations becomes garbage at the same instant.

With `malloc` you pay for that twice. Each allocation searches a free list, updates bookkeeping, and returns memory with a header attached; each `free` returns it and possibly coalesces neighbours. And you must not miss one, because a leak is one forgotten `free` away.

The observation an arena makes is that **all of those objects have the same lifetime**, so individual frees are pointless work. If they die together, they can be freed together.

How an arena works
An arena is a block of memory and an offset. Allocating means rounding the offset up to the alignment you asked for, returning the address, and moving the offset along. That is the entire algorithm:

```
[####used####|                  free                  ]
             ^ offset
```

- **Allocation is a few instructions.** No search, no free list, no per-allocation header.
- `free` **does nothing at all.** It is a no-op, deliberately.
- **You reclaim by resetting or destroying the whole arena**, which sets the offset back to zero. One operation frees ten thousand objects.

The memory comes from you. `proven_arena_create` takes a `proven_mem_mut_t` - a block you got from the heap, or a `static` array, or a region on the stack. The arena never allocates on its own, which is what lets it work with no heap underneath.

What you give up
An arena is not a general-purpose allocator, and the trade is real:

- **You cannot free one object.** If the lifetimes are not actually shared, an arena leaks by design - memory is only reclaimed at reset.
- **Running out is a hard limit.** The backing block is fixed. `PROVEN_ERR_NOMEM` here does not mean the machine is out of memory, it means this arena is.
- **Every pointer into it dies at reset**, all at once, with nothing to warn you. A view that outlives the reset is a dangling view; see contract 2 in Chapter 0.

Use an arena when the shared lifetime is a fact about your program, not a hope.

`proven_arena_t`

```
typedef struct {
    proven_mem_mut_t backing;
    proven_size_t offset;
} proven_arena_t;
```

Fields:

- `backing`: mutable memory range owned by the caller.
- `offset`: next allocation position within `backing`.

Arena functions and helpers

API	Intent	Parameters	Return
`proven_arena_create(backing)`	Initialize an arena over a backing slice.	`backing`: caller-owned memory.	`proven_arena_t`.
`proven_arena_reset(arena)`	Discard all arena allocations.	`arena`.	void.
`proven_arena_destroy(arena)`	Formal cleanup.	`arena`.	void; no-op for caller-backed arenas.
`proven_arena_alloc_aligned(arena, size, align)`	Allocate with explicit alignment.	`arena`, byte `size`, power-of-two `align`.	`proven_result_mem_mut_t`.
`proven_arena_realloc_aligned(arena, old_ptr, old_size, new_size, align)`	Reallocate; can extend/shrink tail allocation in place.	old allocation details and new size.	`proven_result_mem_mut_t`.

`proven_arena_alloc(arena, size)`	Allocate with `PROVEN_DEFAULT_ALIGNMENT`.	`arena`, byte `size`.	`proven_result_mem_mut_t`.
`proven_arena_alloc_aligned(arena, size, align)`	Allocate or call panic hook.	`arena`, `size`, `align`.	`proven_mem_mut_t`.
`proven_arena_alloc_or_panic(arena, size)`	Default-aligned panic allocation.	`arena`, `size`.	`proven_mem_mut_t`.
`proven_arena_as_allocator(arena)`	Expose arena as `proven_allocator_t`.	`arena`.	allocator trait, or zero allocator for null arena.

Trait adapter helpers:

- `proven_arena_alloc_trait`
- `proven_arena_realloc_trait`
- `proven_arena_free_trait`

These are exposed because the allocator trait needs function pointers. Application code normally calls `proven_arena_as_allocator()` instead.

Example:

```
alignas(max_align_t) proven_byte_t storage[4096];
proven_arena_t arena = proven_arena_create((proven_mem_mut_t){
    .ptr = storage,
    .size = sizeof storage,
});

proven_result_mem_mut_t r = proven_arena_alloc(&arena, 64);
if (!proven_is_ok(r.err)) {
    return;   /* the arena cannot grow: it reports NOMEM instead */
}

proven_arena_reset(&arena);   /* reclaims r and everything else at once */
proven_arena_destroy(&arena);
```

Arena with growable containers

Growable containers can use an arena allocator, but growth may abandon earlier blocks because arena `free` is a no-op. Reserve capacity up front when possible.

Correct:

```
alignas(max_align_t) proven_byte_t storage[4096];
proven_arena_t arena = proven_arena_create((proven_mem_mut_t){
    .ptr = storage,
    .size = sizeof storage,
});
proven_allocator_t a = proven_arena_as_allocator(&arena);

/* Ask for the capacity up front, so growth never has to abandon a block. */
proven_result_array_t ar = PROVEN_ARRAY_INIT(a, int, 128);
if (!proven_is_ok(ar.err)) {
    return;
}
(void)PROVEN_ARRAY_PUSH(&ar.value, int, 10);
PROVEN_ARRAY_DESTROY(&ar.value);   /* correct, but arena free reclaims nothing */
proven_arena_reset(&arena);        /* this is what gives the bytes back */
```

Wrong - growing into an arena from a tiny initial capacity:

```
proven_result_array_t ar = PROVEN_ARRAY_INIT(a, int, 1);
for (int i = 0; i < 10000; ++i) {
    PROVEN_ARRAY_PUSH(&ar.value, int, i); /* wrong: every regrow abandons the old block */
}
```

Each regrow asks the arena for a bigger block and then "frees" the old one - which, in an arena, does nothing. The array ends up correct and the arena ends up holding every intermediate size it ever allocated: 1, 2, 4, 8 ... 8192 elements' worth of abandoned space, on top of the one buffer you wanted. Reserve the capacity up front, as the correct version above does.

Wrong - a view that outlives the reset:

```
proven_u8str_view_t name = /* ... built in the arena ... */;
proven_arena_reset(&arena);
use(name);                 /* wrong: those bytes are now free space */
```

This is the arena's sharpest edge. Reset does not touch the memory it reclaims, so the bytes are usually still there and the bug usually does not show up in testing - right up until the next allocation writes over them.

3. Pool: many things, one size

The problem an arena does not solve
An arena assumes shared lifetimes. Plenty of workloads have the opposite shape: objects of one type, created and destroyed continuously, in no particular order. A linked list of events. Nodes in a tree that grows and shrinks. Connection records that come and go.

An arena cannot do this - it never reclaims a single object, so a long-running program would grow without bound. The heap can, and that is exactly what it costs: every allocation searches, every free updates bookkeeping, and the general-purpose allocator does that general-purpose work for a request it already made a thousand times.

The observation a pool makes is that **all these objects are the same size**. If they are the same size, a freed one fits a future request exactly, so it can be handed straight back out with no search at all.

How a pool works
A pool keeps a small stack of freed blocks - the *bin* - and does the obvious thing:

- **Allocate**: if the bin has anything in it, pop one and return it. That is a pointer read and a decrement. Only when the bin is empty does it go to the underlying allocator.
- **Free**: if the bin has room, push the block onto it for reuse. If the bin is full, hand the memory back to the underlying allocator so the pool does not park memory forever.

`bin_cap` is therefore a dial: how much memory this pool is allowed to keep in reserve.

What you give up
- **One pool serves exactly one size and alignment.** A request for anything else is refused with `PROVEN_ERR_INVALID_ARG` - not served from somewhere else. A stricter alignment than the pool was built with is refused too; a looser one is fine, because the block already satisfies it.
- **The pool does not track live objects.** `proven_pool_destroy` frees what is in the bin, not what you are still holding. Free everything you allocated first, or you have leaked it *and* it is now dangling.

Arena versus pool, in one line each: an arena is for *allocate many, free all at once*; a pool is for *allocate and free the same size, over and over, cheaply*.

proven_pool_t
```
typedef struct {
    proven_allocator_t base_alloc;
    proven_size_t item_size;
    proven_size_t item_align;
```

```
    void **bin;
    proven_size_t bin_cap;
    proven_size_t bin_len;
} proven_pool_t;
```

Fields:

- `base_alloc`: allocator used for new blocks and the recycle bin.
- `item_size`: exact object size accepted by the pool. Any other size is rejected with `PROVEN_ERR_INVALID_ARG`.
- `item_align`: the alignment every block is allocated with. A request for a *stricter* alignment is rejected with `PROVEN_ERR_INVALID_ARG`; a looser one is served, since the block already meets it.
- `bin`: cached free-list array.
- `bin_cap`: maximum cached block count.
- `bin_len`: current cached block count.

How recycling works

The pool does not own one big slab; it allocates each item individually from `base_alloc` and keeps a small **stack of freed blocks** (the `bin`, an array of up to `bin_cap` pointers):

- **Allocate.** If `bin_len > 0`, pop the top pointer (O(1), no `base_alloc` call) -- this is the whole point of the pool. Otherwise fall through to `base_alloc`.
- **Free.** If `bin_len < bin_cap`, push the pointer onto the bin for reuse; otherwise (bin full) free it straight back to `base_alloc`. So the bin caps how much memory the pool keeps parked for reuse.
- **Destroy.** `proven_pool_destroy` frees every pointer still in the bin and the bin array itself - but it does **not** track live (handed-out) items, so you must free everything you allocated before destroying the pool.

This makes the pool a churn optimizer for short-lived same-type objects (nodes, events), not a region allocator. Use an `arena` when you want "allocate many, free all at once"; use a `pool` when you want "allocate/free the same size over and over cheaply".

Counter-examples

```
/* WRONG: the pool only handles its configured size/alignment. */
proven_allocator_t a = proven_pool_as_allocator(&pool);   /* item_size == sizeof(Node) */
a.alloc_fn(a.ctx, sizeof(BigThing), alignof(BigThing));   /* not the pool's item -> rejected */

/* WRONG: destroying while items are still live leaks (and dangles) them. */
void *p = a.alloc_fn(a.ctx, sizeof(Node), alignof(Node)).value.ptr;
proven_pool_destroy(&pool);   /* `p` is NOT freed and is now dangling */
/* RIGHT: free every handed-out item first, then destroy. */
```

Pool functions

API	Intent	Return
`proven_pool_init(pool, base_alloc, item_size, item_align, bin_cap)`	Initialize a fixed-size pool.	`PROVEN_OK` or error.
`proven_pool_as_allocator(pool)`	Return allocator trait backed by the pool.	`proven_allocator_t`.
`proven_pool_destroy(pool)`	Free cached blocks and bin storage.	void.

Example:

```
typedef struct Node { int value; } Node;

proven_pool_t pool = {0};
proven_err_t e = proven_pool_init(&pool, alloc, sizeof(Node), alignof(Node), 64);
if (!proven_is_ok(e)) {
```

```
    return;
}

proven_allocator_t node_alloc = proven_pool_as_allocator(&pool);
proven_result_mem_mut_t n = node_alloc.alloc_fn(node_alloc.ctx, sizeof(Node), alignof(Node));
if (proven_is_ok(n.err)) {
    /* Hand it back before destroy: the pool does not track live items. */
    node_alloc.free_fn(node_alloc.ctx, n.value.ptr);
}

proven_pool_destroy(&pool);
```

Wrong:

```
node_alloc.alloc_fn(node_alloc.ctx, sizeof(LargerObject), alignof(LargerObject));
/* wrong: one pool is for one fixed object size and alignment */
```

4. The allocator trait

Why this section is fourth and not first

You have now used three allocators without seeing the interface they share, and that was deliberate. The interface is the least interesting part of the idea and the hardest thing to read cold: three function-pointer typedefs and an alignment contract mean very little until you have seen a heap, an arena and a pool behind them.

You need this section for two things: writing an allocator of your own, and understanding the rules every allocator in this library promises to follow. If you are only ever going to *use* the three above, `proven_heap_allocator()`, `proven_arena_as_allocator()` and `proven_pool_as_allocator()` are the whole API and you can skip ahead.

A trait, here, is a struct of function pointers used as an interface - C's version of a virtual table, written out by hand. `proven_allocator_t` is the library's most important one; `stream.h` and `random.h` use the same shape.

Function pointer types

```
typedef proven_result_mem_mut_t (*proven_alloc_fn_t)(
    void *ctx,
    proven_size_t size,
    proven_size_t align
);

typedef proven_result_mem_mut_t (*proven_realloc_fn_t)(
    void *ctx,
    void *old_ptr,
    proven_size_t old_size,
    proven_size_t new_size,
    proven_size_t align
);

typedef void (*proven_free_fn_t)(void *ctx, void *ptr);
```

Intent:

- `alloc_fn` allocates a new byte slice.
- `realloc_fn` changes allocation size and must be failure-atomic: on failure the old block is still valid and unmodified, so the caller has lost nothing.
- A block must be reallocated and freed with the **same** `align` it was allocated with. An allocator may pick a different underlying mechanism for over-aligned requests than for ordinary ones, and the heap allocator does: `align <= alignof(max_align_t)` - which is every string, buffer and byte array in this library - goes through `malloc`/`realloc` so that growth can happen in place, and

anything more strictly aligned goes through an aligned allocator. Handing a block back under a different alignment class is undefined.

- `free_fn` releases memory. If an allocator needs size metadata, it must track that internally.

proven_allocator_t

```
typedef struct {
    void *ctx;
    proven_alloc_fn_t alloc_fn;
    proven_realloc_fn_t realloc_fn;
    proven_free_fn_t free_fn;
} proven_allocator_t;
```

Fields:

- `ctx`: allocator-specific state.
- `alloc_fn`: allocation function.
- `realloc_fn`: reallocation function.
- `free_fn`: deallocation function.

Reference

Member / API	Shape	Contract
`ctx`	`void *`	The allocator's own state. Opaque to callers; passed back as the first argument.
`alloc_fn`	`proven_alloc_fn_t`	Allocate `size` bytes at `align`. Returns `proven_result_mem_mut_t`.
`realloc_fn`	`proven_realloc_fn_t`	Resize. **Failure-atomic**: on failure the old block is untouched and still valid.
`free_fn`	`proven_free_fn_t`	Release. Must be given the same `align` class the block was allocated with.
`proven_alloc_is_valid(alloc)`	`static inline bool`	True when every function pointer is non-null - i.e. the trait can be called.

The three rules that every allocator in this library obeys, and that yours must:

1. **Same allocator, whole lifetime.** Allocate, reallocate and free a block with one allocator.
2. **Same alignment class.** A block allocated at one alignment must be reallocated and freed at that alignment; allocators may use a different mechanism for over-aligned requests.
3. **Failure changes nothing.** A failed `realloc_fn` leaves the old block valid and unmodified.

```
static inline bool proven_alloc_is_valid(proven_allocator_t alloc);
```

Purpose: check that all function pointers are non-null.

Return: true if the trait can be called.

Correct:

```
proven_allocator_t heap = proven_heap_allocator();
proven_err_t err = proven_alloc_is_valid(heap) ? PROVEN_OK : PROVEN_ERR_UNSUPPORTED;
(void)err;   /* a freestanding build hands back a zero allocator, not a valid one */
```

Wrong:

```
proven_allocator_t alloc = {0};
proven_result_mem_mut_t r = alloc.alloc_fn(alloc.ctx, 64, 8); /* wrong: null call */
```

5. Raw byte buffer

What it is for

`proven_buf_t` is the plainest thing in this chapter: a pointer, a length and a capacity. It owns its bytes, it never grows, and it is what `proven_u8str_t` and `proven_u16str_t` are built out of.

Reach for it directly when you want *bytes* rather than *text* - a record you are assembling, a frame you are about to write to a socket - and you know the maximum size in advance. When the content is text, use the string types in Chapter 3 instead: they give you the same storage plus NUL-termination, views, searching and formatting.

Like everything else here it does **not** store its allocator, so `proven_buf_destroy` takes the same one `proven_buf_create` was given.

proven_buf_t

```
typedef struct {
    proven_byte_t *ptr;
    proven_size_t len;
    proven_size_t cap;
} proven_buf_t;
```

Fields:

- `ptr`: storage pointer.
- `len`: bytes currently used.
- `cap`: bytes allocated.

proven_result_buf_t

```
typedef struct {
    proven_err_t err;
    proven_buf_t value;
} proven_result_buf_t;
```

Buffer functions

API	Intent	Return
`proven_buf_create(alloc, cap)`	Allocate a non-zero-capacity buffer.	`proven_result_buf_t`; invalid allocator or zero cap returns `PROVEN_ERR_INVALID_ARG`.
`proven_buf_append(buf, data)`	Append raw bytes if they fit.	`PROVEN_OK` or error.
`proven_buf_destroy(alloc, buf)`	Free buffer storage with the matching allocator.	void.

Example:

```
proven_result_buf_t r = proven_buf_create(alloc, 64);
if (!proven_is_ok(r.err)) {
    return;
}
proven_buf_t buf = r.value;

proven_err_t e = proven_buf_append(&buf, (proven_mem_view_t){
    .ptr = (const proven_byte_t *)"abc",
    .size = 3,
});
(void)e;   /* PROVEN_ERR_OUT_OF_BOUNDS if it would not fit: the buffer never grows */

proven_buf_destroy(alloc, &buf);
```

6. Examples and misuse cases

Match allocator on destroy

Correct:

```
proven_allocator_t heap = proven_heap_allocator();

proven_result_buf_t r = proven_buf_create(heap, 128);
if (!proven_is_ok(r.err)) {
    return;
}
proven_buf_destroy(heap, &r.value);   /* the same allocator that created it */
```

Wrong:

```
proven_result_buf_t r = proven_buf_create(heap, 128);
proven_buf_destroy(other_alloc, &r.value); /* wrong: allocator mismatch */
```

Nothing reports this at the call. To catch it in a test, put the allocators behind `alloc_check.h` (section 7).

Arena free is a no-op

```
alignas(max_align_t) proven_byte_t storage[1024];
proven_arena_t arena = proven_arena_create((proven_mem_mut_t){
    .ptr = storage,
    .size = sizeof storage,
});
proven_allocator_t a = proven_arena_as_allocator(&arena);

proven_result_mem_mut_t r = proven_arena_alloc(&arena, 32);
if (proven_is_ok(r.err)) {
    /* Legal, and correct to write - but it intentionally reclaims nothing. */
    a.free_fn(a.ctx, r.value.ptr);
}
proven_arena_reset(&arena);   /* only this gives the 32 bytes back */
```

Pool size contract

Correct:

```
typedef struct Node { int value; } Node;

proven_pool_t pool = {0};
if (!proven_is_ok(proven_pool_init(&pool, alloc, sizeof(Node), alignof(Node), 8))) {
    return;
}
proven_allocator_t node_alloc = proven_pool_as_allocator(&pool);

/* The pool serves exactly the size and alignment it was configured with. */
proven_result_mem_mut_t n = node_alloc.alloc_fn(node_alloc.ctx, sizeof(Node), alignof(Node));
if (proven_is_ok(n.err)) {
    node_alloc.free_fn(node_alloc.ctx, n.value.ptr);
}

/* Anything else is refused with PROVEN_ERR_INVALID_ARG rather than served. */
proven_result_mem_mut_t wrong = node_alloc.alloc_fn(node_alloc.ctx, sizeof(Node) * 2, alignof(Node));
(void)wrong;

proven_pool_destroy(&pool);
```

Wrong:

```
node_alloc.alloc_fn(node_alloc.ctx, sizeof(Other), alignof(Other));
/* wrong: PROVEN_ERR_INVALID_ARG - a pool is for one size and one alignment */
```

Buffer append does not grow

Wrong:

```
proven_buf_append(&buf, huge_data); /* returns out-of-bounds; no automatic growth */
```

Use `proven_u8str_append_grow()` or an array if you need growth.

Buffer append tolerates overlap

Correct:

```
proven_result_buf_t r = proven_buf_create(alloc, 64);
if (!proven_is_ok(r.err)) {
    return;
}
proven_buf_t buf = r.value;

proven_err_t e = proven_buf_append(&buf, (proven_mem_view_t){
    .ptr = (const proven_byte_t *)"abcdefgh",
    .size = 8,
});
if (proven_is_ok(e)) {
    /* The source view points back into the buffer's own bytes. That is allowed:
     * append moves rather than copies, so the overlap is well defined. */
    e = proven_buf_append(&buf, (proven_mem_view_t){
        .ptr = buf.ptr + 2,
        .size = 4,
    });
}
(void)e;

proven_buf_destroy(alloc, &buf);
```

Wrong:

```
proven_sys_mem_copy(buf.ptr + buf.len, buf.ptr + 2, 4); /* copy is not the public contract here */
```

Worked example: an arena over caller-supplied memory

Compiled and run by the test suite. It shows the bump-and-drop lifetime that makes an arena worth reaching for: allocations are nearly free, nothing is individually freed, and `proven_arena_reset` reclaims the whole region at once.

```
/*
 * An arena does not own memory: it bumps a pointer through memory YOU own. That
 * is the whole trade. Allocation is an add, individual frees do not exist, and
 * you get everything back at once with a reset.
 *
 * The shape that makes it worth using is bump-then-drop: a phase allocates
 * freely, the phase ends, one reset reclaims the lot. No per-object bookkeeping
 * to get wrong, and nothing to leak - the backing storage below is a plain
 * array with automatic storage duration.
 */

int main(void) {
    /* The backing store is the caller's. Over-align it so the arena can satisfy
     * any alignment a caller asks for out of the first byte. */
    alignas(max_align_t) static proven_byte_t storage[4096];

    proven_arena_t arena = proven_arena_create((proven_mem_mut_t){
        .ptr = storage,
        .size = sizeof storage,
    });
```

```
    /* --- bump ------------------------------------------------------------ */
    proven_result_mem_mut_t a = proven_arena_alloc(&arena, 64);
    EXAMPLE_REQUIRE(proven_is_ok(a.err), "64 bytes must fit in a 4 KiB arena");
    EXAMPLE_REQUIRE(a.value.ptr == storage, "the first allocation starts at the backing store");

    /* Explicit alignment when the type demands more than PROVEN_DEFAULT_ALIGNMENT.
     * The arena pads to reach it, so the bytes it skips are simply gone until reset. */
    proven_result_mem_mut_t b = proven_arena_alloc_aligned(&arena, 32, 64);
    EXAMPLE_REQUIRE(proven_is_ok(b.err), "an over-aligned block must still fit");
    EXAMPLE_REQUIRE(((uintptr_t)b.value.ptr % 64) == 0, "the block must honour the requested alignment");

    /* --- the arena as an allocator for another API ------------------------ */
    /* Anything in proven that takes a proven_allocator_t can be driven by the
     * arena. The string below therefore lives inside `storage`. */
    proven_allocator_t arena_alloc = proven_arena_as_allocator(&arena);
    EXAMPLE_REQUIRE(proven_alloc_is_valid(arena_alloc), "the arena must expose a usable allocator");

    proven_result_u8str_t s = proven_u8str_create(arena_alloc, 32);
    EXAMPLE_REQUIRE(proven_is_ok(s.err), "the arena should be able to back a 32-byte string");

    proven_err_t err = proven_u8str_append_grow(arena_alloc, &s.value, PROVEN_LIT("scratch line"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "appending into an arena-backed string must succeed");

    /* Destroying it is still correct and still required by the ownership rules -
     * but arena free is a no-op, so it reclaims nothing. That is not a leak: the
     * bytes belong to `storage`, and the reset below is what returns them. */
    proven_u8str_destroy(arena_alloc, &s.value);

    proven_size_t used = arena.offset;
    EXAMPLE_REQUIRE(used > 64, "every allocation above came out of the same backing store");

    /* --- drop ------------------------------------------------------------ */
    /* One statement frees the 64-byte block, the aligned block and the string.
     * Reset costs the same whether ten objects were allocated or ten thousand. */
    proven_arena_reset(&arena);
    EXAMPLE_REQUIRE(arena.offset == 0, "reset must reclaim every allocation at once");

    /* Proof that the storage really is reusable: the next allocation lands back
     * at the start. Every pointer handed out before the reset is dangling now -
     * that is the price of the reset being free. */
    proven_result_mem_mut_t c = proven_arena_alloc(&arena, 64);
    EXAMPLE_REQUIRE(proven_is_ok(c.err), "allocation after reset must succeed");
    EXAMPLE_REQUIRE(c.value.ptr == storage, "after reset the arena bumps from the beginning again");

    /* --- exhaustion is an error, not a crash ----------------------------- */
    proven_result_mem_mut_t too_big = proven_arena_alloc(&arena, sizeof storage);
    EXAMPLE_REQUIRE(too_big.err == PROVEN_ERR_NOMEM, "an arena cannot grow: it reports NOMEM instead");

    printf("arena: %zu bytes used before reset, %zu in use now\n",
           (size_t)used, (size_t)arena.offset);

    /* Formal cleanup. A no-op for a caller-backed arena, but writing it keeps
     * the lifetime obvious if the backing store later becomes heap memory. */
    proven_arena_destroy(&arena);
    return EXAMPLE_OK();
}
```

Worked example: a pool that recycles fixed-size blocks

Compiled and run by the test suite. It shows a freed block coming straight back out of the recycle bin, and what the pool refuses.

```c
/*
 * A pool is a churn optimizer, not a region. It is for one type: allocate and
 * free the same fixed-size block over and over - list nodes, events, particles -
 * without paying malloc every time.
 *
 * It keeps a small stack of freed blocks (the "bin"). Freeing pushes a block
 * onto the bin instead of returning it to the base allocator; allocating pops
 * one back off. Both are O(1) and neither touches the heap. That recycling is
 * the entire point, and the check below proves it happens.
 *
 * Ownership: the pool caches freed blocks, but it does NOT track the blocks it
 * has handed out. Every block you take, you must give back before destroy - the
 * pool cannot free what it does not know about.
 */

typedef struct {
    int id;
    int score;
} ex_node_t;

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    EXAMPLE_REQUIRE(proven_alloc_is_valid(heap), "hosted builds have a heap allocator");

    /* The pool takes a base allocator for the blocks it cannot serve from the
     * bin, plus the exact size and alignment of the one type it manages. The
     * last argument caps how many freed blocks are parked for reuse. */
    proven_pool_t pool = {0};
    proven_err_t err = proven_pool_init(&pool, heap, sizeof(ex_node_t), alignof(ex_node_t), 4);
    EXAMPLE_REQUIRE(proven_is_ok(err), "initializing a pool of ex_node_t must succeed");
    if (!proven_is_ok(err)) {
        return 1;
    }

    proven_allocator_t nodes = proven_pool_as_allocator(&pool);

    /* --- first block: nothing in the bin, so it comes from the heap --------- */
    proven_result_mem_mut_t first = nodes.alloc_fn(nodes.ctx, sizeof(ex_node_t), alignof(ex_node_t));
    EXAMPLE_REQUIRE(proven_is_ok(first.err), "the pool must be able to serve its own item type");
    if (!proven_is_ok(first.err)) {
        proven_pool_destroy(&pool);
        return 1;
    }

    ex_node_t *n = (ex_node_t *)first.value.ptr;
    *n = (ex_node_t){ .id = 1, .score = 100 };
    void *first_addr = n;

    /* --- hand it back: it lands in the bin, not back on the heap ------------ */
    nodes.free_fn(nodes.ctx, n);
    EXAMPLE_REQUIRE(pool.bin_len == 1, "a freed block is cached for reuse, not returned to the heap");
    /* `n` is dangling from here on. The pool owns those bytes again. */

    /* --- second block: the freed one is handed straight back ---------------- */
    proven_result_mem_mut_t second = nodes.alloc_fn(nodes.ctx, sizeof(ex_node_t), alignof(ex_node_t));
    EXAMPLE_REQUIRE(proven_is_ok(second.err), "allocating from a non-empty bin must succeed");
    EXAMPLE_REQUIRE(second.value.ptr == first_addr, "the recycled block is the one that was freed");
    EXAMPLE_REQUIRE(pool.bin_len == 0, "taking it back out empties the bin");

    /* Recycled memory is NOT zeroed for you - it is whatever the pool left there.
```

```
     * Initialize every field, exactly as you would for a fresh malloc. */
    ex_node_t *m = (ex_node_t *)second.value.ptr;
    *m = (ex_node_t){ .id = 2, .score = 50 };
    EXAMPLE_REQUIRE(m->id == 2, "the recycled block is ours to overwrite");

    /* --- one pool serves one size and one alignment ------------------------ */
    /* A request for anything else is refused: this is not a general allocator, and it will
     * not silently hand you a block of the wrong size. The code is PROVEN_ERR_UNSUPPORTED -
     * "not my job" - and not INVALID_ARG, which would read as "you passed me garbage" and
     * send you hunting for a bug in your own code. */
    proven_result_mem_mut_t wrong = nodes.alloc_fn(nodes.ctx, sizeof(ex_node_t) * 2, alignof(ex_node_t));
    EXAMPLE_REQUIRE(wrong.err == PROVEN_ERR_UNSUPPORTED, "the pool only serves its configured item size");

    /* --- return every live block before destroying ------------------------- */
    /* proven_pool_destroy frees what is in the bin and the bin itself. `m` is
     * still handed out, so if we skipped this free it would leak. */
    nodes.free_fn(nodes.ctx, m);

    printf("pool: %zu block(s) cached for reuse at teardown\n", (size_t)pool.bin_len);

    proven_pool_destroy(&pool);
    return EXAMPLE_OK();
}
```

Worked example: start-up allocation, growing in place, and wrapping an allocator
The two examples above use an arena and a pool the way most code does. This one covers the three things you reach for once the program is real:

1. **Allocation during start-up.** Some allocations have no sensible failure path: if the program cannot get its own configuration table, there is nothing left for it to do. `proven_arena_alloc_or_panic()` and `proven_arena_alloc_aligned_or_panic()` return the block itself instead of a result, and hand the failure to the panic handler (Chapter 1 section 6). `proven_set_panic_handler()` is how you choose what happens then - the example installs one that records the message so the failure can be shown here, and puts the default back afterwards.
2. **Growing the block you took last.** `proven_arena_realloc_aligned()` extends it where it stands, because the most recent block is the one at the end of the used region. Any earlier block is copied to the end instead - still correct, no longer free.
3. **Wrapping an allocator.** An allocator is three function pointers and a context pointer, so a wrapper that counts, logs, or fails the tenth call on purpose is three forwarding functions. The arena's own three -- `proven_arena_alloc_trait()`, `proven_arena_realloc_trait()` and `proven_arena_free_trait()` - are public so your wrapper can call them instead of re-implementing the arena. Everything in the library that takes a `proven_allocator_t` then runs through your wrapper without knowing it.

```
/*
 * Three things a program that owns its own memory eventually has to do, and the
 * calls that do them:
 *
 *   - Allocate during start-up, where running out of memory is not a condition
 *     the program can carry on from. That is what the `_or_panic` calls are
 *     for, and installing a panic handler is how you decide what "cannot carry
 *     on" means for your program - a log line and an exit, rather than a trap.
 *
 *   - Grow the block you allocated last, without copying it. An arena can do
 *     that in place, because the block that was allocated last is the one
 *     sitting at the end of the used region.
 *
 *   - Instrument the allocator - count allocations, or fail the tenth one on
```

```
 *     purpose in a test - without changing the code being measured. An
 *     allocator here is three function pointers and a context pointer, so
 *     wrapping one is writing three forwarding functions. The arena's own three
 *     are public for exactly this reason: your wrapper forwards to them.
 */

/* --- what a panic handler is for ---------------------------------------- */

static int g_panics = 0;
static char g_last_panic[128];

/* A panic handler receives the message and decides the program's fate. The
 * default one traps immediately, which is right in production and useless in a
 * test - so this one records the message and returns. Returning is allowed ONLY
 * when you are deliberately testing the panic path, and the memory block the
 * panicking call returns must then not be used. */
static void record_panic(const char *msg) {
    ++g_panics;
    snprintf(g_last_panic, sizeof g_last_panic, "%s", msg);
}

/* --- an allocator that counts what passes through it ---------------------- */

typedef struct {
    proven_arena_t *arena;
    proven_size_t   live_bytes;
    proven_size_t   alloc_calls;
    proven_size_t   free_calls;
} ex_counting_ctx_t;

/* Each of the three matches one field of proven_allocator_t. Each does its own
 * bookkeeping and then forwards to the arena's public trait function, so the
 * behaviour being measured is exactly the arena's behaviour and not a
 * re-implementation of it. */
static proven_result_mem_mut_t counting_alloc(void *ctx, proven_size_t size, proven_size_t align) {
    ex_counting_ctx_t *c = (ex_counting_ctx_t *)ctx;
    proven_result_mem_mut_t r = proven_arena_alloc_trait(c->arena, size, align);
    if (proven_is_ok(r.err)) {
        c->live_bytes += size;
        ++c->alloc_calls;
    }
    return r;
}

static proven_result_mem_mut_t counting_realloc(void *ctx, void *old_ptr, proven_size_t old_size,
                                                proven_size_t new_size, proven_size_t align) {
    ex_counting_ctx_t *c = (ex_counting_ctx_t *)ctx;
    proven_result_mem_mut_t r = proven_arena_realloc_trait(c->arena, old_ptr, old_size, new_size, align);
    if (proven_is_ok(r.err)) {
        c->live_bytes = c->live_bytes - old_size + new_size;
    }
    return r;
}

static void counting_free(void *ctx, void *ptr) {
    ex_counting_ctx_t *c = (ex_counting_ctx_t *)ctx;
    ++c->free_calls;
    proven_arena_free_trait(c->arena, ptr);   /* an arena free is a no-op; the count is the point */
}
```

```
int main(void) {
    alignas(max_align_t) static proven_byte_t storage[1024];
    proven_arena_t arena = proven_arena_create((proven_mem_mut_t){ .ptr = storage, .size = sizeof storage });

    /* --- 1. start-up allocation that must not fail ------------------------ */

    proven_set_panic_handler(record_panic);

    /* No result to unwrap: these return the block directly, because there is no
     * error the caller could act on. That is the entire difference. */
    proven_mem_mut_t table = proven_arena_alloc_or_panic(&arena, 256);
    EXAMPLE_REQUIRE(table.ptr != NULL, "a 256-byte start-up allocation must succeed");
    EXAMPLE_REQUIRE(g_panics == 0, "a successful allocation must not panic");

    /* The aligned form, for a type that needs more than the default boundary -
     * a 64-byte cache line here, the usual reason. */
    proven_mem_mut_t cache_line = proven_arena_alloc_aligned_or_panic(&arena, 64, 64);
    EXAMPLE_REQUIRE(((proven_uintptr_t)cache_line.ptr % 64) == 0,
                    "the block must start on the boundary that was asked for");
    EXAMPLE_REQUIRE(g_panics == 0, "an over-aligned allocation that fits must not panic either");

    /* Now the failing case, on purpose: more than the arena could ever hold.
     * With the recording handler installed we can observe it; with the default
     * handler the program would stop here, which is what it is for. */
    proven_mem_mut_t impossible = proven_arena_alloc_or_panic(&arena, sizeof storage * 2);
    (void)impossible;   /* after a handler returns, this block means nothing */
    EXAMPLE_REQUIRE(g_panics == 1, "exhausting the arena through _or_panic must panic");
    EXAMPLE_REQUIRE(g_last_panic[0] != '\0', "the handler receives a message naming the call");
    printf("panic handler saw: %s\n", g_last_panic);

    /* Passing NULL puts the default trapping handler back. Leaving a test
     * handler installed turns a real failure into silent corruption. */
    proven_set_panic_handler(NULL);

    /* --- 2. growing the most recent block in place ------------------------ */

    /* A parser that reads a header, then discovers the body is longer than it
     * guessed, wants to extend the buffer it just took - not copy it. */
    proven_result_mem_mut_t buf = proven_arena_alloc_aligned(&arena, 32, alignof(proven_u32));
    EXAMPLE_REQUIRE(proven_is_ok(buf.err), "the initial 32-byte buffer must fit");

    proven_size_t before = arena.offset;
    proven_result_mem_mut_t grown = proven_arena_realloc_aligned(&arena, buf.value.ptr, 32, 96,
alignof(proven_u32));
    EXAMPLE_REQUIRE(proven_is_ok(grown.err), "growing the most recent block must succeed");
    EXAMPLE_REQUIRE(grown.value.ptr == buf.value.ptr,
                    "the most recent block grows in place: same address, no copy");
    EXAMPLE_REQUIRE(arena.offset == before + 64, "only the extra 64 bytes were taken");

    /* The in-place path is available only for the block allocated LAST. Take
     * another block, and the earlier one can no longer be extended where it
     * stands - the arena copies it to the end instead, and the old bytes are
     * dead until the next reset. Correct either way; just not free. */
    proven_result_mem_mut_t other = proven_arena_alloc(&arena, 16);
    EXAMPLE_REQUIRE(proven_is_ok(other.err), "a second block must fit");
    proven_result_mem_mut_t moved = proven_arena_realloc_aligned(&arena, grown.value.ptr, 96, 128,
alignof(proven_u32));
    EXAMPLE_REQUIRE(proven_is_ok(moved.err), "growing an older block must still succeed");
    EXAMPLE_REQUIRE(moved.value.ptr != grown.value.ptr, "but it is relocated, not extended");
```

```
    /* --- 3. the arena behind a counting wrapper ---------------------------- */

    proven_arena_reset(&arena);

    ex_counting_ctx_t counted = { .arena = &arena };
    proven_allocator_t alloc = {
        .ctx        = &counted,
        .alloc_fn   = counting_alloc,
        .realloc_fn = counting_realloc,
        .free_fn    = counting_free,
    };
    EXAMPLE_REQUIRE(proven_alloc_is_valid(alloc), "all three function pointers must be present");

    /* Any part of the library that takes an allocator now runs through the
     * wrapper, unchanged and unaware. */
    proven_result_u8str_t s = proven_u8str_create(alloc, 16);
    EXAMPLE_REQUIRE(proven_is_ok(s.err), "creating a string through the wrapper must succeed");

    proven_err_t err = proven_u8str_append_grow(alloc, &s.value, PROVEN_LIT("a line long enough to need more
room"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "appending past the initial capacity must succeed");

    proven_u8str_destroy(alloc, &s.value);

    EXAMPLE_REQUIRE(counted.alloc_calls >= 1, "the wrapper saw the string being created");
    EXAMPLE_REQUIRE(counted.free_calls >= 1, "and saw it being destroyed");
    printf("counting allocator: %zu alloc call(s), %zu free call(s), %zu bytes handed out\n",
           (size_t)counted.alloc_calls, (size_t)counted.free_calls, (size_t)counted.live_bytes);

    proven_arena_destroy(&arena);
    return EXAMPLE_OK();
}
```

Wrong - leaving a test panic handler installed:

```
proven_set_panic_handler(record_panic);   /* returns instead of stopping */
/* ... the rest of the program ... */     /* wrong: no restore */
```

A handler that returns turns every later out-of-memory panic into a call that hands back a null block and carries on. Install one only around the code you are testing, and restore the default with `proven_set_panic_handler(NULL)`.

Wrong - using the block after a handler that returned:

```
proven_mem_mut_t m = proven_arena_alloc_or_panic(&arena, huge);
memset(m.ptr, 0, huge);   /* wrong: m.ptr is null if the handler returned */
```

Wrong - a wrapper that forgets one of the three:

```
proven_allocator_t alloc = { .ctx = &counted, .alloc_fn = counting_alloc };  /* wrong */
```

`proven_alloc_is_valid()` returns false for this, and the first `realloc` or `destroy` through it calls a null pointer. Fill in all three, even when two of them only forward.

7. Catching the wrong allocator: `alloc_check.h`

The problem: a mistake that fails somewhere else

Every owner in this library - a string, a buffer, an array - is freed through an allocator the caller passes, and nothing records which allocator the memory came from. Destroying an arena string through the heap, freeing a block twice, or growing a block with an old size it was not allocated with does not fail where it happens. The heap is handed memory it never gave out, its bookkeeping is corrupted, and the crash comes later, in code that did nothing wrong. Storing the

allocator in every owner would double the size of every string for a check most programs never need, so the library does not (B-023). The check belongs on the other side: in the allocator, which can know exactly which blocks are its own.

What `alloc_check.h` does

`proven_alloc_check_wrap` puts a checker in front of any allocator - heap, arena, pool, or one of your own - and returns an allocator you use in its place. The checker keeps a record of every block it hands out, in memory you supply, and refuses anything else: a free or realloc of a block that is not in the record (another allocator's block, or one already freed), and a realloc whose old size or alignment is not the one the block was allocated with. A refusal is a `proven_panic` at the call that made the mistake, and the refused pointer is never passed on, so the underlying allocator is not corrupted even when a test's panic handler returns. At the end of a test, `proven_alloc_check_live` is a leak check.

It is for tests and bug hunts, not for production. Every free and realloc searches the record, linearly. Write `proven_alloc_checked` where you create the allocator and leave it there: it wraps only when `PROVEN_ALLOC_CHECK` is defined, and otherwise returns the allocator it was given, at no cost. Define the macro on the build (`-DPROVEN_ALLOC_CHECK`) for tests and debug runs. It must be defined before the first proven header a file includes - the switch is read with the header, so a `#define` after `#include "proven.h"` leaves the checks silently off.

```
proven_alloc_check_t heap_chk;
proven_alloc_check_entry_t heap_rec[256];   /* up to 256 live blocks */
proven_allocator_t checked = proven_alloc_checked(&heap_chk, proven_heap_allocator(), heap_rec, 256);
/* ... the whole program or test uses `checked` ... */
(void)checked;
if (proven_alloc_check_live(&heap_chk) != 0) { /* a leak, when the checks are on */ }
```

API	Intent	Return
`proven_alloc_check_wrap(&st, inner, entries, cap)`	Wrap `inner`, recording up to `cap` live blocks. Always checks.	`proven_allocator_t` (invalid on bad arguments)
`proven_alloc_checked(&st, inner, entries, cap)`	The same when `PROVEN_ALLOC_CHECK` is defined; otherwise `inner` unchanged.	`proven_allocator_t`
`proven_alloc_check_owns(&st, ptr)`	Whether `ptr` is a live block from this checker.	`bool`
`proven_alloc_check_live(&st)`	Live blocks: 0 at the end of a test means no leak.	`proven_size_t`

The structures you hold

```
typedef struct { void *ptr; proven_size_t size; proven_size_t align; } proven_alloc_check_entry_t;

typedef struct {
    proven_allocator_t          inner;
    proven_alloc_check_entry_t *entries;   /* your record storage */
    proven_size_t               cap, live, live_bytes, peak_live;
    proven_size_t               faults;    /* refused operations, each also a panic */
} proven_alloc_check_t;   /* caller-owned; outlives the allocator; do not copy once wrapped */
```

Cautions, and what goes wrong

It only checks the allocators you wrap. Destroying an arena string through an unwrapped heap is still unchecked: the heap is not asked. Wrap every allocator the test uses.

A full record is a panic, not a pass. A record that quietly stopped recording would report the next free of an unrecorded block as foreign. Size `cap` for the most blocks the test holds at once; `peak_live` tells you what that was. Wrong - a record too small for the test:

```
proven_alloc_check_entry_t rec[4];
proven_allocator_t a = proven_alloc_check_wrap(&st, heap, rec, 4);
/* wrong: a fifth live block panics "the record is full" - enlarge rec, do not ignore it */
```

Wrong - switching it on after the include. The switch is read with the header, so this file runs with the checks off and nothing says so:

```
#include "proven.h"
#define PROVEN_ALLOC_CHECK        /* wrong: too late - use -DPROVEN_ALLOC_CHECK */
```

Leaving it on in a hot path is a slowdown you chose. The search is linear in the live blocks.

Worked example: a string destroyed through the wrong allocator

```
#include <string.h>

/*
 * Catching the wrong allocator where the mistake is made.
 *
 * Nothing in this library remembers which allocator a block came from: the caller
 * passes one to create and one to destroy, and if they differ the allocator's
 * bookkeeping is corrupted and the program breaks later, somewhere else. The
 * alloc_check wrapper goes in front of any allocator, keeps a record of the
 * blocks it handed out, and refuses - with a panic, at the call - anything else.
 *
 * It is for tests and bug hunts. In your own code, write proven_alloc_checked()
 * and switch it with -DPROVEN_ALLOC_CHECK: without the macro it returns the
 * allocator it was given and costs nothing. (A #define after the first proven
 * #include is too late - the switch is read with the header.) This example
 * calls proven_alloc_check_wrap(), which checks unconditionally, so it shows
 * the checks whatever the build flags.
 */

/* A panic handler that returns, so this example can show a refusal and go on. A
 * real program keeps the default, which stops at the mistake. */
static int g_refused;
static void note_panic(const char *msg) {
    ++g_refused;
    printf("refused: %s\n", msg);
}

int main(void) {
    proven_set_panic_handler(note_panic);

    /* Two allocators, each behind its own checker with room to record 16 blocks. */
    static proven_byte_t arena_mem[2048];
    proven_arena_t arena = proven_arena_create((proven_mem_mut_t){ arena_mem, sizeof arena_mem });
    proven_alloc_check_t arena_chk, heap_chk;
    proven_alloc_check_entry_t arena_rec[16], heap_rec[16];
    proven_allocator_t scratch = proven_alloc_check_wrap(&arena_chk, proven_arena_as_allocator(&arena),
arena_rec, 16);
    proven_allocator_t heap = proven_alloc_check_wrap(&heap_chk, proven_heap_allocator(), heap_rec, 16);

    /* --- correct use passes straight through -------------------------------- */

    proven_result_u8str_t name = proven_u8str_create_from_view(heap, PROVEN_LIT("report"));
    EXAMPLE_REQUIRE(proven_is_ok(name.err), "a string from the checked heap");
    EXAMPLE_REQUIRE(proven_is_ok(proven_u8str_append_grow(heap, &name.value, PROVEN_LIT(".txt"))), "grown by
```

```
the same allocator");
    EXAMPLE_REQUIRE(proven_alloc_check_owns(&heap_chk, name.value.internal.ptr), "the heap checker owns its
block");

    /* --- the mistake, caught at the call ----------------------------------- */

    /* A temporary built in the scratch arena... */
    proven_result_u8str_t tmp = proven_u8str_create(scratch, 64);
    EXAMPLE_REQUIRE(proven_is_ok(tmp.err), "a temporary in the arena");
    proven_u8str_t t = tmp.value;
    /* ...and destroyed through the heap. Unchecked, the heap would free arena memory. */
    proven_u8str_destroy(heap, &t);
    EXAMPLE_REQUIRE(g_refused == 1 && heap_chk.faults == 1, "the heap refuses a block it never gave out");
    EXAMPLE_REQUIRE(proven_alloc_check_live(&arena_chk) == 1, "and the arena block is untouched");
    t = tmp.value;
    proven_u8str_destroy(scratch, &t);   /* the right one */

    /* --- a leak check at the end of a test --------------------------------- */

    proven_u8str_destroy(heap, &name.value);
    EXAMPLE_REQUIRE(proven_alloc_check_live(&heap_chk) == 0 && proven_alloc_check_live(&arena_chk) == 0,
                    "nothing is left live");
    printf("peak %zu live block(s) on the heap, %zu refusal(s)\n", (size_t)heap_chk.peak_live,
(size_t)heap_chk.faults);

    proven_set_panic_handler(NULL);
    return EXAMPLE_OK();
}
```

Chapter 3: Strings, Formatting, and Scanning

Part II - The vocabulary every program uses. Prerequisites: Chapter 1 and Chapter 2. After this chapter you can hold text without a NUL terminator deciding your program's fate, build strings that refuse to overflow, and format and parse the everyday cases.

This chapter covers `u8str.h`, `u16str.h`, `utf.h`, `fmt.h`, and `scan.h`. It is the **tutorial half** of the text material: it introduces the formatter and the scanner with the cases you meet daily. Chapter 8 is the reference half - the complete grammar, every argument constructor, and the scanner's error and recovery rules. Read this one first.

Table of contents

1. U8 strings and views
2. U16 strings and views
3. Formatting
4. Scanning
5. Examples and misuse cases

1. U8 strings and views

The problem: a C string does not know how long it is
A C string is a pointer, and where it ends is decided by a zero byte somewhere in memory. That one decision - made in 1972 to save a byte per string - is behind a remarkable amount of damage:

- **Length is a search.** `strlen` walks the string. A loop that checks `strlen(s)` on each iteration is quadratic, and it looks like ordinary code.
- **Text cannot contain a zero byte.** So a C string cannot hold a UTF-16 buffer, a protocol frame, a slice of a file, or any binary data - the string type and the byte type are different things and the language pretends otherwise.
- **A missing terminator is not detectable.** `strcpy` into a buffer with no room writes until it finds a zero somewhere in your stack frame. Nothing reports it. This is the single most exploited bug class in the language's history.
- **You cannot cheaply refer to part of a string.** "The third field of this line" means either copying those bytes out or writing a zero over the original, destroying it. `strtok` chose the second, which is why it mutates its input and cannot be nested.

`strncpy`, the traditional patch, does not always NUL-terminate - so the "safe" function can produce a string that is not a string.

What this library does instead
Two types, and the difference between them is ownership:

- `proven_u8str_view_t` **- borrowed.** A pointer and a size, together, pointing at bytes someone else owns. Copying it is free. It allocates nothing, destroys nothing, and stops being valid when its owner does. This is what you pass to functions.
- `proven_u8str_t` **- owned.** It has its own storage, a capacity, and a NUL terminator kept for you so `proven_u8str_as_cstr` can hand the bytes to a libc function that still wants one. You created it with an allocator; you destroy it with the same one.

The owned string does not remember that allocator, and nothing checks it. Every call that may allocate or free - `_create`, `_reserve`, the `_grow` family, `_destroy` - takes the allocator as an argument, and it must be the one the string was created with. Pass a different one and there is no error to return: the other allocator is handed memory it never gave out, its bookkeeping is corrupted, and the crash comes later and somewhere else. Keeping the allocator in every string would make each one

twice the size and would mean nothing for a borrowed string, so the library does not; keep the pairing in your own code - one allocator per owner, passed from the same place.

Wrong - a string made in an arena and destroyed through the heap:

```
proven_u8str_t s = proven_u8str_create(arena, 64).value;
proven_u8str_destroy(proven_heap_allocator(), &s);   /* wrong: the heap frees arena memory */
```

Both count **bytes**, not characters. "한" is three bytes in UTF-8 and one character, and this library will tell you three, because that is what it knows. Text is a sequence of bytes here; interpreting those bytes as characters is a job for a Unicode layer this library does not have.

Because a view carries its length, everything the NUL terminator made hard becomes ordinary: length is a field, text may contain zero bytes, a sub-range is a view into the same memory with no copy, and a write that would not fit is refused rather than performed.

Wrong - treating a view as a C string:

```
proven_u8str_view_t v = proven_u8str_view_slice(line, 4, 8);   /* a field inside a line */
printf("%s\n", (const char *)v.ptr);   /* wrong: no terminator, prints until it finds a zero */
```

A view is deliberately *not* NUL-terminated: it usually points into the middle of someone else's buffer, where writing a terminator would corrupt the next field. Use `proven_println("{}", PROVEN_ARG(v))`, which takes the size, or copy it into an owned string first.

Structures

```
typedef struct {
    const proven_byte_t *ptr;
    proven_size_t size;
} proven_u8str_view_t;

typedef struct {
    proven_byte_t *ptr;
    proven_size_t size;
} proven_u8str_mut_t;

typedef struct {
    proven_buf_t internal;
    bool         borrowed;
} proven_u8str_t;

typedef struct {
    proven_err_t err;
    proven_u8str_t value;
} proven_result_u8str_t;

typedef struct {
    proven_err_t err;
    const char *value;
} proven_result_cstr_t;
```

Intent:

- `proven_u8str_view_t`: borrowed read-only byte string view.
- `proven_u8str_mut_t`: borrowed mutable byte string view.
- `proven_u8str_t`: owned string with NUL termination.
- `proven_result_u8str_t`: result wrapper for owned strings.
- `proven_result_cstr_t`: result wrapper for an allocated NUL-terminated C string.

Internal layout

The views are exactly what they look like - a pointer and a byte count:

```
typedef struct { const proven_byte_t *ptr; proven_size_t size; } proven_u8str_view_t;
typedef struct { proven_byte_t *ptr;       proven_size_t size; } proven_u8str_mut_t;
```

The owned string wraps a `proven_buf_t` (the fixed-capacity byte buffer from Chapter 2) plus one flag:

```
typedef struct {
    proven_buf_t internal;   /* the bytes + length + capacity; always keeps room for a NUL */
    bool         borrowed;   /* false = allocator-owned (default); true = wraps caller memory */
} proven_u8str_t;
```

- `internal.len` is the byte length (`proven_buf_t` is `ptr` / `len` / `cap` - there is no `size` member). The byte at `internal.len` is always the NUL terminator while the string is valid, so `proven_u8str_as_cstr()` is O(1).
- `borrowed` is `false` for a zero-initialized handle, so an allocator-owned string is the safe default. It is set `true` only by `proven_u8str_borrow`, which wraps `[buf, buf+cap)` you own: growing operations then refuse to reallocate (`PROVEN_ERR_OUT_OF_BOUNDS`) and `proven_u8str_destroy` is a no-op.
- **Do not** read or write these fields directly to change the string; use the functions below. Reading `internal.len` for length is fine, but prefer `proven_u8str_as_view()`.

Counter-example - treating a borrowed string like an owned one:

```
proven_byte_t stack[8];
proven_u8str_t s = proven_u8str_borrow(stack, sizeof stack);   /* borrowed */

/* This would have to reallocate caller memory, so it refuses: the call returns
   PROVEN_ERR_OUT_OF_BOUNDS and `stack` is left exactly as it was. A borrowed
   string never silently escapes to the heap. */
proven_err_t e = proven_u8str_append_grow(alloc, &s, PROVEN_LIT("far too long for eight bytes"));
(void)e;   /* == PROVEN_ERR_OUT_OF_BOUNDS */

/* destroy is a no-op here: `stack` is yours, and the library will not free it.
   Writing it anyway is correct, and it keeps teardown code uniform. */
proven_u8str_destroy(alloc, &s);
```

Macros

Macro	Intent	Notes
`PROVEN_LIT(s)`	Make a `proven_u8str_view_t` from a string literal.	Use only with string literals.
`PROVEN_LIT_INIT(s)`	Initializer form for literal views.	Use in aggregate initialization.
`PROVEN_INDEX_NOT_FOUND`	Sentinel returned by find functions.	Equal to `(proven_size_t)-1`.

U8 functions

API	Intent	Return
`proven_u8str_create(alloc, limit)`	Create an empty owned string with capacity for `limit` bytes plus NUL.	`proven_result_u8str_t`.
`proven_u8str_create_from_view(alloc, view)`	Copy a view into a new owned string.	`proven_result_u8str_t`.
`proven_u8str_borrow(buf, cap)`	Wrap caller-owned `[buf, buf+cap)` as a fixed-capacity string (no allocation). Fixed-capacity ops and `append_fmt` work; growing ops refuse to reallocate caller memory; destroy is a no-op. `cap` includes the NUL.	`proven_u8str_t`.

`proven_u8str_reset(str)`	Truncate to empty, keeping the buffer/capacity for reuse (owned or borrowed).	`proven_err_t`.
`proven_u8str_is_valid(str)`	Validate public string invariants.	`bool`.
`proven_u8str_reserve(alloc, str, new_cap)`	Ensure at least `new_cap` bytes of internal capacity. (Borrowed strings cannot grow.)	`proven_err_t`.
`proven_u8str_append(str, data)`	Atomic fixed-capacity append.	`PROVEN_OK` or `PROVEN_ERR_OUT_OF_BOUNDS`.
`proven_u8str_append_partial(str, data)`	Truncating append.	`proven_result_size_t`; `value` is bytes written.
`proven_u8str_append_grow(alloc, str, data)`	Atomic growable append.	`proven_err_t`.
`proven_u8str_append_byte(alloc, str, b)`	Append one byte, growing if needed.	`proven_err_t`.
`proven_u8str_replace_at(str, index, old_len, data)`	Replace an exact byte range with fixed-capacity semantics.	`proven_err_t`.
`proven_u8str_replace_at_grow(alloc, str, index, old_len, data)`	Like `replace_at`, but grows the buffer (doubling) when the edit does not fit instead of failing.	`proven_err_t`; string unchanged on alloc failure.
`proven_u8str_insert(str, index, data)`	Insert bytes at `index` with fixed-capacity semantics.	`proven_err_t`.
`proven_u8str_insert_grow(alloc, str, index, data)`	Like `insert`, but grows the buffer when needed. No manual `reserve` required.	`proven_err_t`; string unchanged on alloc failure.
`proven_u8str_remove(str, index, len)`	Remove a byte range.	`proven_err_t`.
`proven_u8str_replace_first(str, start_offset, target, replacement)`	Replace first matching target at or after start.	`PROVEN_OK` even when not found.
`proven_u8str_view_find(haystack, start_offset, needle)`	Find the first occurrence of a byte substring (correctly handles any byte values; not NUL-terminated).	index or `PROVEN_INDEX_NOT_FOUND`.
`proven_u8str_view_starts_with(str, prefix)`	Prefix test.	int truth value.
`proven_u8str_view_starts_with_ascii_ci(str, prefix)`	Prefix test with ASCII letters compared without case (no locale, no Unicode folding).	int truth value.
`proven_u8str_view_ends_with(str, suffix)`	Suffix test.	int truth value.
`proven_u8str_view_slice(str, index, len)`	Return a clamped subview.	`proven_u8str_view_t`.
`proven_u8str_as_cstr(str)`	Return internal NUL-terminated pointer.	`const char *`; invalidated by growth/destroy.
`proven_mem_view_from_u8(view)`	Convert U8 view to byte view.	`proven_mem_view_t`.

`proven_u8str_view_to_cstr(view, alloc)`	Allocate a NUL-terminated C string from any view.	`proven_result_cstr_t`; caller frees with allocator.
`proven_cstr_len(s)`	Count bytes until NUL.	`proven_size_t`.
`proven_u8str_view_from_cstr(s)`	Make a view from a trusted NUL-terminated string.	Empty view for null pointer.
`proven_u8str_view_from_bytes(ptr, size)`	Make a view of `size` bytes at any object pointer, no cast needed.	`proven_u8str_view_t`; nothing copied.
`proven_u8str_view_eq(a, b)`	Byte equality test.	int truth value.
`proven_u8str_view_eq_ascii_ci(a, b)`	Equality with ASCII letters compared without case: HTTP field names, URL schemes, MIME types.	int truth value.
`proven_u8str_destroy(alloc, str)`	Free owned storage with matching allocator.	void.
`proven_u8str_as_view(str)`	Borrow current contents.	`proven_u8str_view_t`.

Basic U8 example

```
proven_result_u8str_t r = proven_u8str_create_from_view(alloc, PROVEN_LIT("log"));
if (!proven_is_ok(r.err)) {
    return;
}
proven_u8str_t s = r.value;

proven_err_t e = proven_u8str_append_grow(alloc, &s, PROVEN_LIT(": ready"));
if (!proven_is_ok(e)) {
    proven_u8str_destroy(alloc, &s);
    return;
}

/* Valid until the next growing call: as_cstr points into the string's storage. */
const char *cstr = proven_u8str_as_cstr(&s);
(void)cstr;

proven_u8str_destroy(alloc, &s);
```

Splitting, trimming, searching from the end, and ordering views

Views could be found in, sliced, and compared for equality, and that was all. Everything else a program does with a line of text - cut it into fields, trim the fields, find the last dot, sort the results - every caller wrote by hand, and the hand-written versions were wrong in the same places. Splitting is the clearest case. The loop a competent person writes first is *"keep going while a separator is found"*:

```
proven_size_t off = 0, at;
while ((at = proven_u8str_view_find(s, off, sep)) != PROVEN_INDEX_NOT_FOUND) {
    emit(proven_u8str_view_slice(s, off, at - off));   /* wrong: the tail is never emitted */
    off = at + sep.size;
}
```

It loses the text after the last separator, and that is wrong on the *common* case: `"a,b,c"` gives two fields, `"a"` gives none, `""` gives none. Getting it right means hoisting the last field out of the loop, which is exactly the step that gets skipped under a deadline. The alternative callers actually reached for - one owned string per field - measured 3.4x slower and one allocation per field (RFC-0004 section 2.2).

So the view vocabulary now has the rest of the words, all pure functions over views with no allocation. Two rules hold for every one of them. **An ill-formed view** - `ptr == NULL` with `size > 0` - **is treated as empty**, by a guard in each function. And **every empty result is** `{NULL, 0}`, the spelling `proven_u8str_view_slice` already used, so an empty result carries no position: test a view by its size, never by its pointer.

Splitting keeps a permanent contract: n separators yield n + 1 fields, counting non-overlapping occurrences from the left. `"a,"` is `"a"` and an empty field; `",a"` is an empty field and `"a"`; `"a,,b"` keeps its empty middle field, unlike `strtok`; `""` is one empty field, not zero. An empty separator yields the whole input once - never an endless run of empty fields, which is what the obvious implementation does, because a search for an empty needle matches where it starts. The iterator points into your source, never into itself, so it can be copied, and the copy continues on its own.

Trimming knows exactly six bytes: space, `\t`, `\n`, `\v`, `\f`, `\r`. Not the locale, and not Unicode - a no-break space is not whitespace here. Prefix and suffix removal leave the view unchanged when the affix is absent; that is not an error, and if you need to know whether it fired, ask `proven_u8str_view_starts_with` first.

`proven_u8str_view_find_last` **returns a position**, the start of the last occurrence, and occurrences may overlap: `find_last("aaa", "aa")` is 1. For an empty needle the position is `size` - the same answer `_find` gives - which is the one result that is not a valid byte index. It makes the same choices the forward search makes, from the same sample of the haystack: on ordinary text it finds the rarest needle byte from the end and verifies around it, and on a low-entropy haystack - long runs, a small alphabet - it switches to an algorithm that is linear whatever the input (backward Shift-Or up to 64 bytes, a reverse Two-Way beyond).

`proven_u8str_view_cmp` **orders bytes unsigned, with a prefix first**, so `"\xFF"` sorts after `"a"` and `"app"` before `"apple"`; a NUL inside a view is data. It answers by sign - not necessarily -1, 0 or 1. `proven_u8str_view_cmp_ptr` is the same comparison shaped for `proven_array_sort`, which passes pointers to the elements.

API	Intent	Return
`proven_u8str_view_split(src, sep)`	Start splitting; the fields are views into `src`.	`proven_u8str_view_split_t`
`proven_u8str_view_split_next(&it, &field)`	The next field, or `false` when there are none.	`bool`
`proven_u8str_view_trim(s)`	Drop the six whitespace bytes from both ends.	`proven_u8str_view_t`
`proven_u8str_view_trim_start(s)`, `proven_u8str_view_trim_end(s)`	One end only.	`proven_u8str_view_t`
`proven_u8str_view_remove_prefix(s, prefix)`	`s` without `prefix`, or `s` unchanged.	`proven_u8str_view_t`
`proven_u8str_view_remove_suffix(s, suffix)`	`s` without `suffix`, or `s` unchanged.	`proven_u8str_view_t`
`proven_u8str_view_find_last(haystack, needle)`	Start of the last occurrence, or `PROVEN_INDEX_NOT_FOUND`.	`proven_size_t`
`proven_u8str_view_contains(haystack, needle)`	`find` from 0 is not `NOT_FOUND`.	`bool`
`proven_u8str_view_cmp(a, b)`	Bytewise unsigned order, prefix first; sign only.	`int`
`proven_u8str_view_cmp_ptr(pa, pb)`	`cmp` for `proven_array_sort`: pointers to views.	`int`

proven_u8str_view_is_well_formed(s)	False only for {NULL, n > 0}.	bool

```
typedef struct {
    proven_u8str_view_t rest;  /* not yet yielded; points into the caller's source bytes */
    proven_u8str_view_t sep;
    bool                done;  /* set once the final field has been yielded */
} proven_u8str_view_split_t;   /* copyable: a copy continues independently */
```

Wrong - ending a loop on the field, or on `_is_well_formed`. An empty field is `{NULL, 0}`, and so is a slice past the end of the text; both are well formed. `_is_well_formed` says only that a view is safe to read. End a split on `split_next`'s return value and a search on `PROVEN_INDEX_NOT_FOUND`:

```
while (proven_u8str_view_split_next(&it, &f) && f.ptr != NULL) { ... }  /* wrong: stops at the first empty field */
while (proven_u8str_view_is_well_formed(rest)) { ... }                  /* wrong: never false for a slice */
```

Wrong - reading the byte at `find_last`'s answer without checking the needle. For an empty needle the answer is `size`:

```
proven_size_t at = proven_u8str_view_find_last(s, needle);
char c = (char)s.ptr[at];   /* wrong when needle is empty: at == s.size, one past the end */
```

The worked example splits a record with trailing and doubled separators, trims each field, finds the last dot, strips a double suffix, and sorts the names:

```
#include <string.h>

/*
 * The everyday text jobs, done on views: split a line into fields, trim them,
 * strip a known prefix or suffix, find the last dot, and sort what you got.
 * Nothing here allocates except the array the sort works on; every result
 * points into the text you started with.
 *
 * Two rules hold throughout. An empty result is {NULL, 0}, so test a view by
 * its size, never its pointer. And a split loop ends on the return value of
 * split_next - n separators always give n + 1 fields, empty ones included.
 */

static bool is(proven_u8str_view_t v, const char *want) {
    return v.size == strlen(want) && (v.size == 0 || memcmp(v.ptr, want, v.size) == 0);
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- split, then trim each field ------------------------------------- */

    /* A record with a trailing separator and an empty field: five separators,
     * so six fields - the last one empty. The loop a person writes first
     * ("while a separator is found") would give five and lose the tail. */
    proven_u8str_view_t record = PROVEN_LIT(" report.tar.gz , draft.txt,,notes.md ,\tREADME ,");
    proven_u8str_view_t fields[8];
    int n = 0;
    proven_u8str_view_split_t it = proven_u8str_view_split(record, PROVEN_LIT(","));
    proven_u8str_view_t f;
    while (n < 8 && proven_u8str_view_split_next(&it, &f)) {
        fields[n++] = proven_u8str_view_trim(f);   /* ' ', \t, \n, \v, \f, \r - nothing else */
    }
    EXAMPLE_REQUIRE(n == 6, "five separators give six fields");
    EXAMPLE_REQUIRE(is(fields[0], "report.tar.gz") && is(fields[3], "notes.md") && is(fields[4], "README"),
                    "each field is trimmed on both ends");
```

```
    EXAMPLE_REQUIRE(fields[2].size == 0 && fields[5].size == 0, "the empty fields are kept, and tested by
size");

    /* One-sided trims, for text where one end is significant. */
    EXAMPLE_REQUIRE(is(proven_u8str_view_trim_start(PROVEN_LIT("  indented  ")), "indented  "), "trim_start");
    EXAMPLE_REQUIRE(is(proven_u8str_view_trim_end(PROVEN_LIT("  indented  ")), "  indented"), "trim_end");

    /* --- prefixes, suffixes, and the last dot ------------------------------ */

    proven_u8str_view_t name = fields[0];
    /* The extension is after the LAST dot: find_last, not find. */
    proven_size_t dot = proven_u8str_view_find_last(name, PROVEN_LIT("."));
    EXAMPLE_REQUIRE(dot == 10, "the last dot in report.tar.gz is at 10");
    EXAMPLE_REQUIRE(is(proven_u8str_view_slice(name, dot + 1, name.size), "gz"), "so the extension is gz");

    /* remove_suffix leaves the view unchanged when the suffix is absent - no
     * error, so ask starts_with/ends_with first when you need to know. */
    EXAMPLE_REQUIRE(is(proven_u8str_view_remove_suffix(name, PROVEN_LIT(".tar.gz")), "report"), "the double
suffix goes");
    EXAMPLE_REQUIRE(is(proven_u8str_view_remove_suffix(name, PROVEN_LIT(".zip")), "report.tar.gz"),
                    "an absent suffix changes nothing");
    EXAMPLE_REQUIRE(is(proven_u8str_view_remove_prefix(PROVEN_LIT("# heading"), PROVEN_LIT("# ")), "heading"),
                    "a known prefix goes");
    EXAMPLE_REQUIRE(proven_u8str_view_contains(fields[1], PROVEN_LIT("draft")), "contains is find !=
NOT_FOUND");

    /* --- sort the non-empty fields ----------------------------------------- */

    proven_result_array_t ra = proven_array_create(alloc, 8, sizeof(proven_u8str_view_t),
alignof(proven_u8str_view_t));
    EXAMPLE_REQUIRE(proven_is_ok(ra.err), "creating the array must succeed");
    if (!proven_is_ok(ra.err)) return EXAMPLE_OK();
    proven_array_t names = ra.value;
    for (int i = 0; i < n; ++i) {
        if (fields[i].size > 0) (void)proven_array_push(&names, &fields[i]);
    }
    /* cmp_ptr is cmp shaped for a sort: it receives pointers to the views. Bytes
     * compare unsigned, and a prefix sorts first; "README" sorts before
     * lowercase names because 'R' is below 'd' in ASCII. */
    proven_array_sort(&names, proven_u8str_view_cmp_ptr);
    const proven_u8str_view_t *first = proven_array_get(&names, 0);
    const proven_u8str_view_t *last = proven_array_get(&names, 3);
    EXAMPLE_REQUIRE(first && is(*first, "README") && last && is(*last, "report.tar.gz"), "sorted bytewise");
    EXAMPLE_REQUIRE(proven_u8str_view_cmp(PROVEN_LIT("app"), PROVEN_LIT("apple")) < 0,
                    "cmp answers by sign - never compare it with -1");

    /* --- well-formed is not "found" or "not empty" ------------------------- */

    /* A slice past the end is {NULL, 0}, which is well formed - so this
     * predicate cannot end a loop or tell "empty" from "past the end". It only
     * says the view is safe to read. */
    proven_u8str_view_t past = proven_u8str_view_slice(name, 100, 5);
    EXAMPLE_REQUIRE(proven_u8str_view_is_well_formed(past) && past.size == 0, "past the end: empty and well
formed");
    EXAMPLE_REQUIRE(!proven_u8str_view_is_well_formed((proven_u8str_view_t){ NULL, 3 }), "only {NULL, n > 0}
is not");

    /* --- case, the ASCII way ----------------------------------------------- */

    /* HTTP field names, URL schemes and MIME types ignore the case of ASCII letters
```

```
     * and nothing else: no locale, no Unicode folding. */
    EXAMPLE_REQUIRE(proven_u8str_view_eq_ascii_ci(PROVEN_LIT("Content-Type"), PROVEN_LIT("content-TYPE")),
"field names ignore ASCII case");
    EXAMPLE_REQUIRE(!proven_u8str_view_eq_ascii_ci(PROVEN_LIT("@"), PROVEN_LIT("`")), "a byte next to the
letters is only itself");
    EXAMPLE_REQUIRE(proven_u8str_view_starts_with_ascii_ci(PROVEN_LIT("HTTPS://example.org"),
PROVEN_LIT("https:")), "and so does a prefix test");

    /* A view of any buffer, without a cast at the call site. */
    const unsigned char raw[] = { 'G', 'E', 'T' };
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_view_from_bytes(raw, sizeof raw), PROVEN_LIT("GET")), "a
view of raw bytes");

    printf("%d fields, %zu names sorted\n", n, (size_t)names.len);
    proven_array_destroy(&names);
    return EXAMPLE_OK();
}
```

2. U16 strings and views

Why a second string type exists at all

UTF-8 is the right default and this library commits to it. `proven_u16str_t` exists for one reason: **the Windows API is UTF-16.** Every "wide" entry point - `CreateFileW`, `GetEnvironmentVariableW`, the whole `W` family - takes `wchar_t *`, which on Windows is 16 bits. A library that talks to those APIs needs a type that holds their code units without pretending they are bytes.

So this is a boundary type. You use it where you touch a UTF-16 API, and you use `proven_u8str_t` everywhere else. It is deliberately small - create, destroy, append, and length - because it is not meant to be the type your program thinks in.

Two things to hold on to, because they are the source of every UTF-16 bug:

- **A code unit is not a character.** UTF-16 encodes anything outside the Basic Multilingual Plane as a *surrogate pair* - two `proven_u16` values that mean one character. An emoji is two code units. Slicing between them produces an unpaired surrogate, which is not valid UTF-16.
- `size` **here counts code units, not bytes.** `proven_u16str_view_t.size` is a count of `proven_u16` values; the `proven_buf_t` underneath tracks bytes, which is why `proven_u16str_len` divides. Mixing the two units is the most common mistake with this type.

You build a `proven_u16str_t` from a `u"..."` literal, from code units you already have, or - the usual case - by converting the UTF-8 your program holds, with `utf.h` (next subsection).

U16 APIs are excluded when `PROVEN_NO_U16STR` is defined, which is the default in freestanding builds - a bare-metal target has no Windows API to talk to.

Wrong - assuming one code unit is one character:

```
proven_u16str_view_t v = ...;                /* text containing an emoji */
proven_size_t chars = proven_u16str_len(&s); /* wrong: that is code units */
```

Structures

```
typedef struct {
    const proven_u16 *ptr;
    proven_size_t size;
} proven_u16str_view_t;

typedef struct {
    proven_buf_t internal;
} proven_u16str_t;
```

```
typedef struct {
    proven_err_t err;
    proven_u16str_t value;
} proven_result_u16str_t;
```

Macro

Macro	Intent
`PROVEN_U16_LIT(s)`	Make a `proven_u16str_view_t` from a UTF-16 literal expression `u"..."`.

U16 functions

API	Intent	Return
`proven_u16str_create(alloc, unit_limit)`	Create empty owned U16 string.	`proven_result_u16str_t`.
`proven_u16str_create_from_view(alloc, view)`	Copy U16 view into owned string.	`proven_result_u16str_t`.
`proven_u16str_destroy(alloc, str)`	Free owned U16 storage.	void.
`proven_u16str_append(str, data)`	Atomic fixed-capacity append.	`proven_err_t`.
`proven_u16str_append_partial(str, data)`	Truncating append.	`proven_result_size_t`.
`proven_u16str_append_grow(alloc, str, data)`	Atomic growable append.	`proven_err_t`.
`proven_u16str_as_ptr(str)`	Return internal `proven_u16` pointer.	`const proven_u16 *`.
`proven_u16str_len(str)`	Return length in code units.	`proven_size_t`.

Example:

```
#ifndef PROVEN_NO_U16STR
proven_result_u16str_t r =
    proven_u16str_create_from_view(alloc, PROVEN_U16_LIT("hello"));
if (!proven_is_ok(r.err)) {
    return;
}
proven_u16str_t s = r.value;

(void)proven_u16str_append_grow(alloc, &s, PROVEN_U16_LIT(" world"));

/* Length is in code units, not characters: "hello world" is 11 units. */
proven_size_t units = proven_u16str_len(&s);
(void)units;

proven_u16str_destroy(alloc, &s);
#endif
```

Note: `proven_u16` is a code unit, not necessarily one full Unicode character. UTF-16 surrogate pairs use two code units.

Converting between UTF-8 and UTF-16 (`utf.h`)
The program holds UTF-8; the Windows wide API wants UTF-16; the names and text that API hands back are UTF-16 and the rest of the program wants them as UTF-8. Every program that crosses that line needs a converter, and a converter has to decide what to do with text that is not valid: a stray continuation byte, an overlong encoding, an encoded surrogate, a lone surrogate on the UTF-16 side. The tempting answer is to repair it - swap in U+FFFD and carry on - and it is the wrong one for a library that hands file names to the operating system: a repaired name is a different name, and the program opens a file it was never asked to open.

So `utf.h` is strict everywhere. Malformed input is `PROVEN_ERR_INVALID_ENCODING`, and the forms that write into your memory write nothing when they refuse. There is no lossy mode.

Malformed and incomplete are different answers. Text read in pieces - from a pipe, a buffered writer, a console - can end in the middle of a character that the next piece completes. The partial forms stop there with `PROVEN_ERR_NEED_MORE` and report how much input they used, so you keep the tail and try again with more. The whole-text forms have no "more" to wait for, so for them a text that ends mid-character is malformed.

The three shapes are the ones the rest of the library uses: measure then convert into a buffer you own, all or nothing; grow an owned string, all or nothing; and a partial form that converts whole characters until the input, the output or the valid text runs out. A partial conversion never writes half a surrogate pair or half a UTF-8 sequence.

API	Intent	Return
`proven_utf8_to_utf16_size(src)`	Code units `src` converts to; validates.	`proven_result_size_t`
`proven_utf16_to_utf8_size(src, n)`	UTF-8 bytes `n` code units convert to; validates.	`proven_result_size_t`
`proven_utf8_to_utf16(src, out, out_cap, &written)`	All of `src` into `out`, or nothing. No terminator.	`proven_err_t`
`proven_utf16_to_utf8(src, n, out, out_cap, &written)`	The same, the other way.	`proven_err_t`
`proven_utf8_to_utf16_partial(src, out, out_cap)`	Whole characters until something stops it.	`proven_utf_step_t`
`proven_utf16_to_utf8_partial(src, n, out, out_cap)`	The same, the other way; a trailing high surrogate is `NEED_MORE`.	`proven_utf_step_t`
`proven_utf8_append_to_u16str(alloc, dst, src)`	Append the UTF-16 of `src`, growing `dst`; all or nothing.	`proven_err_t`
`proven_utf16_append_to_u8str(alloc, dst, src, n)`	Append the UTF-8 of `n` code units, growing `dst`; all or nothing.	`proven_err_t`
`proven_utf8_decode_next(s, pos)`	The one character at byte `pos`, or why not; `len` is always the step to take.	`proven_utf8_char_t`
`proven_utf8_encode_one(cp, out)`	One code point as UTF-8 into `out[0..3]`.	Its length, 1 to 4; 0 for a surrogate or above U+10FFFF.
`proven_utf8_validate(s, bad_at)`	Whether all of `s` is well-formed UTF-8 (a character cut off at the end is not).	`PROVEN_OK`, or `PROVEN_ERR_INVALID_ENCODING` and the first bad offset in `*bad_at`.

```
typedef struct {
    proven_err_t  err;       /* OK, OUT_OF_BOUNDS (output full), NEED_MORE (input ends
                                mid-character), INVALID_ENCODING (malformed at `consumed`) */
    proven_size_t consumed;  /* input units used: bytes for UTF-8, code units for UTF-16 */
    proven_size_t written;   /* output units written */
} proven_utf_step_t;

typedef enum {
    PROVEN_TEXT_UTF8, PROVEN_TEXT_UTF16LE, PROVEN_TEXT_UTF16BE,
    PROVEN_TEXT_AUTO         /* readers only: decided by a byte order mark */
} proven_text_encoding_t;    /* for the u16 writers and readers in stream.h (chapter 5) */
```

```
typedef struct {
    proven_err_t  err;       /* OK, INVALID_ENCODING, NEED_MORE, EOF, OUT_OF_BOUNDS, INVALID_ARG */
    proven_u32    cp;        /* the scalar value when err is OK; 0 otherwise */
    proven_size_t len;       /* bytes this step covers: the character, the maximal subpart of
                                malformed input, or the bytes left when the text ends mid-character */
} proven_utf8_char_t;
```

Wrong - widening each byte into a code unit. It is right for ASCII, which is why it survives testing, and it turns every other character into two or three meaningless units. The library's own `proven_time_u16_fmt` did exactly this with a caller's non-ASCII day names until `utf.h` existed:

```
for (size_t i = 0; i < text.size; ++i) {
    proven_u16 unit = (proven_u16)text.ptr[i];        /* wrong: a byte is not a code unit */
    (void)proven_u16str_append_grow(alloc, &wide, (proven_u16str_view_t){ &unit, 1 });
}
```

Wrong - converting pieces with the whole-text form. The piece that ends inside a character is refused as malformed, although the text is fine; use the partial form and carry the tail:

```
proven_utf8_to_utf16(piece, out, cap, &n);   /* wrong for pieces: a split character is INVALID */
```

One character at a time. A segmenter, a shaper or a lexer walks text a character at a time, and wants a malformed byte reported where it is rather than the whole text refused. `proven_utf8_decode_next` decodes the one character at a byte offset under the same strict rules, and its `len` is always the step to take: the character's length, or for malformed input the *maximal subpart* - the longest prefix that was still a valid start - which is where Unicode says to resynchronise. Nothing is substituted. A program that draws text puts one U+FFFD per malformed step, which is Unicode's recommended practice; one that validates stops at the first.

```
proven_u8str_view_t text = PROVEN_LIT("a\xC3\xA9\xFF" "z\xE2\x82");
proven_size_t chars = 0, malformed = 0;
for (proven_size_t pos = 0; pos < text.size;) {
    proven_utf8_char_t c = proven_utf8_decode_next(text, pos);
    if (c.err == PROVEN_OK) ++chars;                              /* c.cp: 'a', U+00E9, 'z' */
    else if (c.err == PROVEN_ERR_INVALID_ENCODING) ++malformed;  /* the FF: one step */
    /* PROVEN_ERR_NEED_MORE: E2 82 start a character this text never finished */
    pos += c.len;                                                 /* at least 1 inside the text */
}
(void)chars; (void)malformed;
```

The worked example converts a Korean file name for a wide API and back, shows the refusal that writes nothing, grows owned strings both ways, and carries a character split between two pieces:

```
#include <string.h>

/*
 * The program thinks in UTF-8; a Windows "wide" call wants UTF-16. utf.h is the
 * crossing, in both directions, and it is strict: text that is not valid UTF-8
 * or UTF-16 is refused, never repaired. A converter that quietly swaps a bad
 * byte for a question mark has changed a file name you were about to open.
 *
 * Three shapes, as everywhere in this library: measure-then-convert into a
 * buffer you own (all or nothing), grow an owned string (all or nothing), and a
 * partial form for text that arrives in pieces.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* A Korean file name ("report.txt": three Hangul syllables and ".txt"), as the rest
     * of the program holds it: UTF-8. */
```

```
    proven_u8str_view_t name = PROVEN_LIT("\xEB\xB3\xB4\xEA\xB3\xA0\xEC\x84\x9C.txt");

    /* --- measure, then convert into your own buffer ------------------------ */

    /* Ask first: the answer is in code units, and it also validates. Nine bytes
     * of Hangul are three code units here, not nine. */
    proven_result_size_t need = proven_utf8_to_utf16_size(name);
    EXAMPLE_REQUIRE(proven_is_ok(need.err) && need.value == 7, "3 syllables + \".txt\" are 7 code units");

    /* One more unit for the NUL a wide API expects; the converter writes none. */
    proven_u16 wide[16];
    proven_size_t units = 0;
    proven_err_t err = proven_utf8_to_utf16(name, wide, 15, &units);
    EXAMPLE_REQUIRE(proven_is_ok(err) && units == 7, "the name converts whole");
    wide[units] = 0;
    EXAMPLE_REQUIRE(wide[0] == 0xBCF4, "the first code unit is the syllable, not its first byte");
    /* On Windows: CreateFileW((LPCWSTR)wide, ...). Not called here, so this runs everywhere. */

    /* And back: a name a wide API returned, as UTF-8 for everything else. */
    proven_result_size_t back = proven_utf16_to_utf8_size(wide, units);
    EXAMPLE_REQUIRE(proven_is_ok(back.err) && back.value == name.size, "the reverse size is the original byte
count");
    proven_byte_t bytes[32];
    proven_size_t nbytes = 0;
    err = proven_utf16_to_utf8(wide, units, bytes, sizeof bytes, &nbytes);
    EXAMPLE_REQUIRE(proven_is_ok(err) && nbytes == name.size && memcmp(bytes, name.ptr, nbytes) == 0,
                    "the round trip gives the same bytes");

    /* --- strict: malformed text is refused, and nothing is written ------- */

    /* A lone continuation byte, as a truncated or mis-decoded name would have. */
    proven_u8str_view_t broken = PROVEN_LIT("bad\x80.txt");
    wide[0] = 0x1234;
    err = proven_utf8_to_utf16(broken, wide, 15, &units);
    EXAMPLE_REQUIRE(err == PROVEN_ERR_INVALID_ENCODING, "a stray continuation byte is refused");
    EXAMPLE_REQUIRE(units == 0 && wide[0] == 0x1234, "and the output is untouched");

    /* --- grow an owned string ---------------------------------------------- */

    proven_result_u16str_t r = proven_u16str_create(alloc, 4);
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "creating the wide path must succeed");
    proven_u16str_t path = r.value;
    err = proven_utf8_append_to_u16str(alloc, &path, PROVEN_LIT("C:\\reports\\"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the directory appends");
    err = proven_utf8_append_to_u16str(alloc, &path, name);
    EXAMPLE_REQUIRE(proven_is_ok(err) && proven_u16str_len(&path) == 11 + 7, "and the name, growing the
string");
    EXAMPLE_REQUIRE(proven_u16str_as_ptr(&path)[18] == 0, "still NUL-terminated for the system call");

    /* A failed append changes nothing: no half-converted tail. */
    err = proven_utf8_append_to_u16str(alloc, &path, broken);
    EXAMPLE_REQUIRE(err == PROVEN_ERR_INVALID_ENCODING && proven_u16str_len(&path) == 18, "a refused append
leaves the path as it was");

    proven_result_u8str_t r8 = proven_u8str_create(alloc, 8);
    EXAMPLE_REQUIRE(proven_is_ok(r8.err), "creating the UTF-8 copy must succeed");
    proven_u8str_t copy = r8.value;
    err = proven_utf16_append_to_u8str(alloc, &copy, proven_u16str_as_ptr(&path), proven_u16str_len(&path));
    EXAMPLE_REQUIRE(proven_is_ok(err) && copy.internal.len == 11 + name.size, "the whole path comes back as
UTF-8");
```

```
    /* --- partial: text that arrives in pieces ------------------------------ */

    /* A reader hands over 4 bytes at a time, and the second syllable is split after its first
     * byte. The partial form converts what is whole, stops with NEED_MORE, and
     * says how much it used; the caller keeps the rest and adds the next piece. */
    proven_u16 out[8];
    proven_utf_step_t st = proven_utf8_to_utf16_partial((proven_u8str_view_t){ name.ptr, 4 }, out, 8);
    EXAMPLE_REQUIRE(st.err == PROVEN_ERR_NEED_MORE && st.consumed == 3 && st.written == 1,
                    "one whole syllable, and the start of the next kept back");
    st = proven_utf8_to_utf16_partial((proven_u8str_view_t){ name.ptr + st.consumed, 6 }, out + 1, 7);
    EXAMPLE_REQUIRE(proven_is_ok(st.err) && st.written == 2 && out[1] == 0xACE0, "with the next piece it
completes");

    /* The same distinction in the other direction: a high surrogate at the end
     * may be half of a pair whose other half is in the next piece. */
    const proven_u16 emoji_half[] = { 'o', 'k', 0xD83D };
    proven_byte_t u8out[16];
    st = proven_utf16_to_utf8_partial(emoji_half, 3, u8out, sizeof u8out);
    EXAMPLE_REQUIRE(st.err == PROVEN_ERR_NEED_MORE && st.consumed == 2, "a trailing high surrogate waits");

    /* --- one character out, and a whole text checked ---------------------- */

    /* Encoding refuses what is not a character: a lone surrogate, or past U+10FFFF. */
    proven_byte_t one[4];
    EXAMPLE_REQUIRE(proven_utf8_encode_one(0xBCF4, one) == 3 && one[0] == 0xEB, "a Hangul syllable is three
bytes");
    EXAMPLE_REQUIRE(proven_utf8_encode_one(0xD83D, one) == 0, "a surrogate is not a character");

    /* Validation says where the text stops being UTF-8. */
    proven_size_t bad = 0;
    EXAMPLE_REQUIRE(proven_utf8_validate(name, NULL) == PROVEN_OK, "the name is valid UTF-8");
    EXAMPLE_REQUIRE(proven_utf8_validate(PROVEN_LIT("ok\xC3"), &bad) == PROVEN_ERR_INVALID_ENCODING && bad ==
2, "a cut-off character is found at its first byte");

    printf("converted a %zu-byte name to %zu code units and back\n", (size_t)name.size, (size_t)need.value);

    proven_u8str_destroy(alloc, &copy);
    proven_u16str_destroy(alloc, &path);
    return EXAMPLE_OK();
}
```

3. Formatting

The problem: `printf` is told the types twice

`printf("%d", x)` states the type of `x` twice - once in the format string and once by passing `x` - and nothing checks that the two agree. Varargs erases the type, so the function reads whatever bytes the calling convention left, in whatever shape the format demanded:

```
printf("%d\n", 3.0);      /* wrong: reads a double's bytes as an int */
printf("%s\n", 42);       /* wrong: dereferences 42 as a pointer */
printf("%d %d\n", 1);     /* wrong: reads an argument that was never passed */
```

All three compile. Modern compilers warn when the format is a literal, which helps until the format is a variable - and then you have a function that will read arbitrary stack memory on demand, which is a class of vulnerability with its own name.

The second problem is where the output goes. `sprintf` writes to a buffer whose size it does not know. `snprintf` takes a size and then **truncates**, returning the length it *would* have written - so the caller who forgets to compare gets a silently shortened path, command, or identifier.

What this library does instead

The placeholder has no type in it. `{}` says "a value goes here"; the type comes from the argument, resolved at compile time:

```
proven_println("{} scored {}", PROVEN_ARG(name), PROVEN_ARG(score));
```

`PROVEN_ARG` is a `_Generic` dispatch - the compiler picks the right constructor for the argument's static type. A mismatch between the format and the argument is not possible, because the format never states a type. What the spec after `:` controls is *presentation* - width, fill, alignment, precision, base - never interpretation.

The destination is a sized object, and the fixed-capacity form refuses rather than truncates: `proven_u8str_append_fmt` fails with `PROVEN_ERR_OUT_OF_BOUNDS` and writes nothing, while `proven_u8str_append_fmt_grow` takes an allocator and grows. Which one you called is visible in the call itself.

Your own types can join in. `PROVEN_ARG_OF(&obj, render_fn)` lets a type you defined print with `{}` like everything else - the extension point is compile-time and typed, not a registry of names. Chapter 8 section 5.1 shows how.

The cost, stated plainly: `PROVEN_ARG` around each argument is more typing than `%d`, and the format language is not the one in your fingers. What you buy is that the class of bug at the top of this section cannot be written.

The formatter writes into `proven_u8str_t` or PAL-backed streams. It uses a small structural format language with `{}` placeholders, explicit indexes like `{1}`, escaped braces `{{` and `}}`, and width/alignment specs such as `{:0>5}`, `{:*^10}`, and `{:.<10}`.

Wrong - assuming the spec chooses the type:

```
proven_println("{:d}", PROVEN_ARG(3.5));   /* wrong: the spec formats, it does not convert */
```

Structures and enums

```
typedef struct {
    proven_err_t err;
    proven_size_t written;
    proven_size_t required;
} proven_fmt_result_t;
```

Fields:

- `err`: result code.
- `written`: bytes actually written.
- `required`: bytes required for full output.

`proven_arg_type_t` variants:

- `PROVEN_ARG_NONE`
- `PROVEN_ARG_I32`
- `PROVEN_ARG_U32`
- `PROVEN_ARG_I64`
- `PROVEN_ARG_U64`
- `PROVEN_ARG_F64` unless `PROVEN_FMT_NO_FLOAT` is defined
- `PROVEN_ARG_CSTR`
- `PROVEN_ARG_STR_VIEW`
- `PROVEN_ARG_DATETIME`
- `PROVEN_ARG_PTR`
- `PROVEN_ARG_FN`

```
typedef struct {
    proven_arg_type_t type;
    union {
        proven_i32 i32;
        proven_u32 u32;
        proven_i64 i64;
        proven_u64 u64;
        double f64;
        const char *cstr;
        proven_u8str_view_t str_view;
        proven_datetime_t datetime;
        const void *ptr;
        void (*fn)(void);
    } value;
} proven_arg_t;
```

Format argument constructors

API	Intent
`proven_arg_none()`	Internal sentinel value.
`proven_arg_i32(v)`, `proven_arg_u32(v)`	Integer arguments.
`proven_arg_i64(v)`, `proven_arg_u64(v)`	Wide integer arguments.
`proven_arg_f64(v)`	Floating-point argument unless float formatting is disabled.
`proven_arg_cstr(v)`	Trusted live NUL-terminated C string.
`proven_arg_cstr_n(v, max_len)`	Bounded C-string argument; scans for NUL only up to `max_len`.
`proven_arg_str_view(v)`	Borrowed string view argument.
`proven_arg_u16(v)`	Borrowed UTF-16 view, rendered as UTF-8. Strict: an unpaired surrogate fails the format. Width counts UTF-8 bytes, as for a u8 view. `PROVEN_ARG` picks it for a `proven_u16str_view_t`.
`proven_arg_datetime(v)`	Datetime argument.
`proven_arg_ptr(v)`	Object pointer argument.
`proven_arg_fn(v)`	Function pointer argument.
`proven_arg_ucstr(v)`	Unsigned-char C string helper.
`proven_arg_identity(v)`	Pass-through for existing `proven_arg_t`.

Format macros

Macro	Intent
`PROVEN_ARG(x)`	`_Generic` selector for supported argument types.
`PROVEN_ARG_FN(f)`	Function pointer formatting helper.
`PROVEN_ARG_CSTR_N(v, max_len)`	Bounded C-string helper.
`proven_u8str_append_fmt(str, fmt, ...)`	Atomic fixed-capacity formatting.
`proven_u8str_append_fmt_trunc(str, fmt, ...)`	Best-effort truncating formatting.
`proven_u8str_append_fmt_grow(alloc, str, fmt, ...)`	Atomic growable formatting.
`proven_u8str_append_fmt_with_scratch(alloc, str, fmt, scratch, ...)`	Growable formatting with scratch allocator for temporary patch allocations.
`PROVEN_FMT_IS_OK(res)`	Check `proven_fmt_result_t`.

Formatting engine

```
proven_fmt_result_t proven_u8str_fmt_internal(
    proven_allocator_t alloc,
```

```
    proven_u8str_t *str,
    bool trunc,
    const char *fmt,
    proven_allocator_t scratch,
    const proven_arg_t *args,
    proven_size_t args_count
);
```

Normal user code should call the macros instead of this internal engine. The engine expects a leading `proven_arg_none()` sentinel at index 0.

Extra unused format arguments return `PROVEN_ERR_INVALID_ARG`.

Example:

```
proven_result_u8str_t r = proven_u8str_create(alloc, 8);
if (!proven_is_ok(r.err)) {
    return;
}
proven_u8str_t s = r.value;

/* The target is only 8 bytes; the _grow form reallocates rather than truncate. */
proven_fmt_result_t fr = proven_u8str_append_fmt_grow(
    alloc,
    &s,
    "name={} score={:0>4}",
    PROVEN_ARG(PROVEN_LIT("ada")),
    PROVEN_ARG(42)
);
if (!PROVEN_FMT_IS_OK(fr)) {
    proven_u8str_destroy(alloc, &s);
    return;
}
/* s == "name=ada score=0042" */

proven_u8str_destroy(alloc, &s);
```

4. Scanning

The problem: `scanf` will not tell you where it stopped

Parsing input is the mirror of formatting, and libc's answer is worse. `sscanf` returns *how many fields it filled* and nothing else - not which one failed, not how far it got, not why:

```
int n = sscanf(line, "%d %d %d", &a, &b, &c);
if (n != 3) { /* wrong: which field? at what offset? was it malformed or missing? */ }
```

If the third field is malformed, you know only that you got two. You cannot report the position to the user, cannot skip the bad record and resume, and cannot tell "ran out of input" from "found something that is not a number". And `%s` into a `char *` has the same unbounded-write problem as `strcpy`, with the input now coming from outside your program.

Then there is `strtol`, whose contract requires you to clear `errno` first, check it after, and compare `endptr` against the input to detect "no digits at all" - three separate things to get right for one conversion.

What this library does instead

The cursor is yours. A `proven_scan_t` holds the view being parsed and an offset into it. Each scan reads from the cursor and moves it forward on success. Because the cursor is a field you can read, you always know exactly where parsing stopped - which is the position you show the user.

Each scan returns a result. `proven_scan_i64` hands back `{err, val}`, so "not a number", "out of range" and "end of input" are different errors rather than one missing field.

Failure restores the cursor for the primitive scanners: a failed `proven_scan_i64` leaves the cursor where it was, so you can try something else at the same position. That is what makes recovery possible instead of guesswork.

One thing to know before you rely on it: the *structural* scan (`proven_scan_fmt`, the `{}` form) is **not** transactional across fields. If the third placeholder fails, the first two destinations have already been written. Chapter 8 section 11.1 covers the error codes and the recovery patterns in full.

The scanner parses from a borrowed `proven_u8str_view_t`. A cursor tracks progress.

Wrong - treating a partial structural scan as if nothing happened:

```
proven_err_t e = proven_scan_fmt(&sc, "{} {} {}", ...);
if (!proven_is_ok(e)) {
    /* wrong: destinations for the fields that DID parse have already been written */
}
```

Result structs

```
typedef struct { proven_err_t err; proven_i64 val; } proven_result_i64_t;
typedef struct { proven_err_t err; proven_u64 val; } proven_result_u64_t;
typedef struct { proven_err_t err; double val; } proven_result_f64_t;
typedef struct { proven_err_t err; proven_u8str_view_t val; } proven_result_u8str_view_t;
```

proven_scan_t

```
typedef struct {
    proven_u8str_view_t view;
    proven_size_t cursor;
} proven_scan_t;
```

Purpose: hold input and current parse position. `proven_scan_init()` normalizes invalid non-empty null views to empty views.

Scanner functions

API	Intent	Return
`proven_scan_init(view)`	Create scanner from view.	`proven_scan_t`.
`proven_scan_skip_whitespace(scan)`	Advance past whitespace.	void.
`proven_scan_i64(scan)`	Parse signed 64-bit integer.	`proven_result_i64_t`.
`proven_scan_u64(scan)`	Parse unsigned 64-bit integer.	`proven_result_u64_t`.
`proven_scan_f64(scan)`	Parse floating-point value.	`proven_result_f64_t`.
`proven_parse_double_ascii(view)`	Parse one locale-free ASCII float token without skipping whitespace.	`proven_parse_double_result_t`.
`proven_parse_f64_ascii(view)`	Compatibility alias for the same locale-free binary64 parser.	`proven_parse_f64_result_t`.
`proven_strtod(nptr, endptr)`	Parse one `strtod`-style token with whitespace skipping and `endptr` reporting.	`double`.
`proven_scan_str(scan)`	Parse a whitespace-delimited token as view into input.	`proven_result_u8str_view_t`.
`proven_scan_skip_until(scan, target)`	Move cursor to target if found.	`proven_err_t`.
`proven_scan_skip_until_number(scan)`	Move cursor to next number-looking position.	void.

Scan argument types

`proven_scan_arg_type_t` identifies the destination kind stored in a `proven_scan_arg_t`. `PROVEN_SCAN_ARG(&x)` supports pointers to:

- `short`, `unsigned short`
- `int`, `unsigned int`
- `long`, `unsigned long`
- `long long`, `unsigned long long`
- `double`
- `proven_u8str_view_t`

Native destination constructors:

- `proven_scan_arg_short`, `proven_scan_arg_ushort`
- `proven_scan_arg_int`, `proven_scan_arg_uint`
- `proven_scan_arg_long`, `proven_scan_arg_ulong`
- `proven_scan_arg_llong`, `proven_scan_arg_ullong`

Explicit fixed-width and utility helpers:

- `proven_scan_arg_i32`, `proven_scan_arg_u32`
- `proven_scan_arg_i64`, `proven_scan_arg_u64`
- `proven_scan_arg_f64`
- `proven_scan_arg_str_view`
- `proven_scan_arg_none`
- `proven_scan_arg_identity`

Long aliases:

```
#define PROVEN_SCAN_ARG_LONG(ptr)  proven_scan_arg_long(ptr)
#define PROVEN_SCAN_ARG_ULONG(ptr) proven_scan_arg_ulong(ptr)
```

Format scanning macros

Macro	Intent
`PROVEN_SCAN_ARG(x)`	`_Generic` destination selector.
`proven_scan_fmt_cursor(scan_ptr, fmt, ...)`	Scan from an existing cursor.
`proven_scan_fmt(view, fmt, ...)`	Scan from a view with a temporary cursor.

Scan engine

```
proven_err_t proven_scan_fmt_internal(
    proven_scan_t *scan,
    const char *fmt,
    const proven_scan_arg_t *args,
    proven_size_t args_count
);
```

API	Intent	Return
`proven_scan_fmt_internal(scan, fmt, args, args_count)`	Structural scanner engine over an existing cursor.	`proven_err_t`.
`proven_scan_fmt_internal_view(view, fmt, args, count)`	Convenience wrapper that creates a temporary scanner over a view.	`proven_err_t`.

Normal user code should call the macros. The engine expects a leading sentinel argument.

Important behavior: `proven_scan_fmt_internal()` may advance the cursor and write earlier destinations before returning an error if a later literal mismatch occurs. Save the cursor and destination values first if you need transaction-like parsing.

Example:

```
proven_scan_t scan = proven_scan_init(PROVEN_LIT("ID: 402 SCORE: 99.5 ada"));
int id = 0;
double score = 0.0;
proven_u8str_view_t user = {0};

proven_err_t e = proven_scan_fmt_cursor(
    &scan,
    "ID: {} SCORE: {} {}",
    PROVEN_SCAN_ARG(&id),
    PROVEN_SCAN_ARG(&score),
    PROVEN_SCAN_ARG(&user)
);
if (!proven_is_ok(e)) {
    return;
}
/* id == 402, score == 99.5, user borrows "ada" out of the input - it is not a
 * copy, so it is only valid while the scanned bytes are. */
```

Float parsing notes

- `proven_scan_f64()` and `proven_parse_double_ascii()` route through the shared decimal-to-binary64 backend.
- Finite decimal inputs are rounded to IEEE-754 binary64 with round-to-nearest, ties-to-even behavior.
- The current conversion stack is `Clinger fast path -> direct Eisel-Lemire -> staged Eisel-Lemire layer -> exact bigint fallback`, with internal counters used by tests to confirm which path took a representative input.
- The direct Eisel-Lemire layer multiplies the significand by a truncated 128-bit power of five and proves the rounding from the product's low bits; it declines anything it cannot prove (a possible tie, a subnormal or overflowing result). A significand longer than 19 digits is bounded by its first 19 digits and the same plus one, and taken when both bounds round alike.
- The staged Eisel-Lemire layer currently accepts generated-`5^q` positive exponent cases and exact negative-exponent cases where the decimal significand cleanly cancels the required `5^q`.
- On every target (it needs no 128-bit integer type), the same layer also accepts a conservative rounded negative-exponent ratio subset in normal-range cases, including wider left-shift normalization than the original narrow prototype allowed.
- The widened cached-power product path now also stages some subnormal cases, including `5e-324`, while values below the half-threshold to true-min still defer to the exact bigint fallback.
- The checked-in cached `5^q` constants are generated locally by `scripts/generate_float_decimal_tables.py`.
- `proven_parse_double_ascii()` does not skip leading whitespace.
- `proven_strtod()` skips leading ASCII whitespace, updates `endptr`, returns signed infinity on overflow, and preserves signed zero on underflow.

5. Examples and misuse cases

Views are not C strings

Wrong:

```
proven_u8str_view_t view = get_view();
printf("%s\n", (const char *)view.ptr); /* wrong: view may not be NUL-terminated */
```

Correct:

```
proven_u8str_view_t view = proven_u8str_view_slice(PROVEN_LIT("/etc/hosts"), 5, 5);
```

```
/* A view is a pointer and a length into somebody else's bytes. To hand it to a
 * C API that wants a NUL, allocate a real C string from it. */
proven_result_cstr_t c = proven_u8str_view_to_cstr(view, alloc);
if (!proven_is_ok(c.err)) {
    return;
}
/* c.value is "hosts", NUL-terminated. It is yours, so free it. */
alloc.free_fn(alloc.ctx, (void *)c.value);
```

`PROVEN_LIT` is for literals

Correct:

```
proven_u8str_view_t a = PROVEN_LIT("abc");
(void)a;
```

Wrong:

```
const char *runtime = getenv("NAME");
proven_u8str_view_t a = PROVEN_LIT(runtime); /* wrong: macro requires literal syntax */
```

Use:

```
const char *runtime = "NAME=value";   /* any trusted NUL-terminated string */
proven_u8str_view_t a = proven_u8str_view_from_cstr(runtime);
(void)a;
```

Distinguish not found from replaced

`proven_u8str_replace_first()` returns `PROVEN_OK` when the target is not found. Search first if that matters.

```
proven_result_u8str_t r = proven_u8str_create_from_view(alloc, PROVEN_LIT("the old way"));
if (!proven_is_ok(r.err)) {
    return;
}
proven_u8str_t s = r.value;

proven_size_t at = proven_u8str_view_find(proven_u8str_as_view(&s), 0, PROVEN_LIT("old"));
if (at != PROVEN_INDEX_NOT_FOUND) {
    (void)proven_u8str_replace_first(&s, 0, PROVEN_LIT("old"), PROVEN_LIT("new"));
}

proven_u8str_destroy(alloc, &s);
```

Bounded format input

Wrong:

```
char *untrusted = get_untrusted_pointer();
proven_println("{}", PROVEN_ARG(untrusted));
/* wrong: C-string formatting scans until NUL */
```

Correct:

```
char untrusted[16] = { 'n', 'o', ' ', 'n', 'u', 'l', ' ', 'h', 'e', 'r', 'e', '!', '!', '!', '!', '!' };
proven_size_t max_len = sizeof untrusted;

/* The bounded form stops looking for a NUL after max_len bytes. */
proven_println("{}", PROVEN_ARG_CSTR_N(untrusted, max_len));
```

Borrowed fixed-capacity strings

Use `proven_u8str_borrow` to format into a stack or static buffer without any allocation - useful in allocator-free code and on hot paths. Use the fixed-capacity operations (and `proven_u8str_append_fmt`), reuse with `proven_u8str_reset`, and do not call growing operations or `proven_u8str_destroy` on it (the caller owns the memory).

```c
int cur = 3;
int total = 10;

proven_byte_t line[64];
proven_u8str_t s = proven_u8str_borrow(line, sizeof line);   /* cap includes NUL */

proven_fmt_result_t fr = proven_u8str_append_fmt(&s, "L{}/{}", PROVEN_ARG(cur), PROVEN_ARG(total));
if (PROVEN_FMT_IS_OK(fr)) {
    proven_println("{}", PROVEN_ARG(proven_u8str_as_view(&s)));
}

(void)proven_u8str_reset(&s);   /* reuse next frame, no allocation */
```

Misuse: a growing call that would exceed the borrowed capacity returns `PROVEN_ERR_OUT_OF_BOUNDS` rather than reallocating caller memory.

```c
proven_byte_t small[4];
proven_u8str_t t = proven_u8str_borrow(small, sizeof small);
proven_err_t e = proven_u8str_append_grow(alloc, &t, PROVEN_LIT("toolong"));
(void)e;   /* e == PROVEN_ERR_OUT_OF_BOUNDS; small[] is untouched */
```

Self-referential formatting

Wrong:

```c
const char *inside = proven_u8str_as_cstr(&s);
proven_u8str_append_fmt_grow(alloc, &s, "{}", PROVEN_ARG(inside));
/* wrong: self-aliasing C-string arguments are rejected */
```

Correct:

```c
proven_result_u8str_t r = proven_u8str_create_from_view(alloc, PROVEN_LIT("ab"));
if (!proven_is_ok(r.err)) {
    return;
}
proven_u8str_t s = r.value;

/* A view carries a length, so the formatter knows exactly which bytes to snapshot. */
proven_u8str_view_t before = proven_u8str_as_view(&s);
proven_fmt_result_t fr = proven_u8str_append_fmt_grow(alloc, &s, "{}", PROVEN_ARG(before));
(void)fr;   /* s == "abab" */

proven_u8str_destroy(alloc, &s);
```

Transactional scanning

Wrong assumption:

```c
proven_err_t e = proven_scan_fmt_cursor(&scan, "{} suffix", PROVEN_SCAN_ARG(&x));
/* if suffix mismatches, x and scan.cursor may already have changed */
```

Correct pattern:

```c
proven_scan_t scan = proven_scan_init(PROVEN_LIT("42 prefix"));
int x = -1;

proven_size_t old_cursor = scan.cursor;
int old_x = x;

proven_err_t e = proven_scan_fmt_cursor(&scan, "{} suffix", PROVEN_SCAN_ARG(&x));
if (!proven_is_ok(e)) {
    /* The literal "suffix" did not match, but 42 was already written into x and
     * the cursor already moved. Put both back yourself. */
    scan.cursor = old_cursor;
```

```
    x = old_x;
}
```

Worked example: owned strings and borrowed strings

Compiled and run by the test suite. The distinction that matters: an owned string may reallocate and must be destroyed; a borrowed one wraps caller memory, never reallocates, and refuses to grow past the buffer you gave it rather than quietly moving it.

```
/*
 * There are two string handles here and the difference is ownership, not size:
 *
 *   proven_u8str_t      - a byte string you can edit. It either owns an
 *                         allocation (create) or borrows one of yours (borrow).
 *   proven_u8str_view_t - a pointer and a length into somebody else's bytes.
 *                         It owns nothing, it is not NUL-terminated, and it is
 *                         only valid while those bytes are.
 *
 * A view is what you pass to a function that reads. A u8str is what you keep.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- an OWNED string: the allocator's memory, yours to destroy ---------- */
    /* The capacity argument is content bytes; the NUL is extra, so as_cstr is
     * always O(1) and always safe. */
    proven_result_u8str_t r = proven_u8str_create(alloc, 16);
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "creating a 16-byte string must succeed");
    if (!proven_is_ok(r.err)) {
        return 1;
    }
    proven_u8str_t path = r.value;

    /* append is fixed-capacity: it fits or it fails, and on failure it has not
     * touched the string. It never reallocates, so it needs no allocator. */
    proven_err_t err = proven_u8str_append(&path, PROVEN_LIT("/etc/hosts"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "10 bytes fit in a 16-byte string");

    /* append_grow is the growable twin: give it the allocator the string was
     * created with and it reallocates when needed. Still failure-atomic - if the
     * allocation fails, the string is exactly as it was. */
    err = proven_u8str_append_grow(alloc, &path, PROVEN_LIT(".backup.original"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "append_grow must reallocate rather than fail");

    /* Edits in the middle. insert shifts the tail right; remove shifts it left. */
    err = proven_u8str_insert_grow(alloc, &path, 0, PROVEN_LIT("/srv"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "inserting a prefix must succeed");

    err = proven_u8str_remove(&path, proven_u8str_as_view(&path).size - 9, 9);  /* drop ".original" */
    EXAMPLE_REQUIRE(proven_is_ok(err), "removing the trailing suffix must succeed");

    /* replace_first returns PROVEN_OK when the target is absent - "nothing to do"
     * is not an error. Search first when the difference matters to you. */
    err = proven_u8str_replace_first(&path, 0, PROVEN_LIT("hosts"), PROVEN_LIT("fstab"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "replacing an existing substring must succeed");

    /* --- reading it: borrow a view, do not copy ------------------------------ */
    /* as_view is free. The view is only good until the next edit: any growing
     * call may reallocate and leave the view (and any cstr) dangling. */
    proven_u8str_view_t v = proven_u8str_as_view(&path);
```

```
    EXAMPLE_REQUIRE(proven_u8str_view_eq(v, PROVEN_LIT("/srv/etc/fstab.backup")),
                    "the edits above should have produced /srv/etc/fstab.backup");
    EXAMPLE_REQUIRE(proven_u8str_view_starts_with(v, PROVEN_LIT("/srv")),
                    "the inserted prefix is at the front");

    proven_size_t dot = proven_u8str_view_find(v, 0, PROVEN_LIT(".backup"));
    EXAMPLE_REQUIRE(dot != PROVEN_INDEX_NOT_FOUND, "the suffix must be found");

    /* A slice is a view into the SAME bytes - no allocation, no copy. */
    proven_u8str_view_t stem = proven_u8str_view_slice(v, 0, dot);
    EXAMPLE_REQUIRE(proven_u8str_view_eq(stem, PROVEN_LIT("/srv/etc/fstab")),
                    "slicing at the suffix leaves the stem");

    /* as_cstr is the escape hatch to C APIs, and it is only valid because the
     * owned string keeps a NUL past its length. Do NOT do this with a view:
     * `stem.ptr` is not NUL-terminated - it just points into `path`. */
    printf("owned:  %s\n", proven_u8str_as_cstr(&path));

    /* --- a BORROWED string: your memory, no allocation at all --------------- */
    /* Same type, same operations - but the bytes are this stack buffer. `cap`
     * includes the NUL, so this holds 31 content bytes. */
    proven_byte_t line[32];
    proven_u8str_t status = proven_u8str_borrow(line, sizeof line);

    err = proven_u8str_append(&status, PROVEN_LIT("mounted "));
    EXAMPLE_REQUIRE(proven_is_ok(err), "appending into a borrowed buffer needs no allocator");
    err = proven_u8str_append(&status, stem);
    EXAMPLE_REQUIRE(proven_is_ok(err), "a view can be appended just like a literal");

    /* The growing calls exist for a borrowed string, but they refuse to
     * reallocate memory they do not own: too much data is OUT_OF_BOUNDS, and
     * `line` is left untouched. A borrowed string cannot silently escape to the
     * heap behind your back. */
    err = proven_u8str_append_grow(alloc, &status,
                                   PROVEN_LIT(" ...and a great deal more text than fits"));
    EXAMPLE_REQUIRE(err == PROVEN_ERR_OUT_OF_BOUNDS,
                    "a borrowed string reports overflow instead of reallocating caller memory");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&status), PROVEN_LIT("mounted /srv/etc/fstab")),
                    "the failed append must have left the string unchanged");

    printf("borrowed: %s\n", proven_u8str_as_cstr(&status));

    /* reset truncates to empty and keeps the buffer, so the next frame reuses
     * the same 32 bytes with no allocation. */
    err = proven_u8str_reset(&status);
    EXAMPLE_REQUIRE(proven_is_ok(err), "reset must succeed on a borrowed string");
    EXAMPLE_REQUIRE(proven_u8str_as_view(&status).size == 0, "reset empties the string");

    /* --- destroy: the ownership rule, spelled out ---------------------------- */
    /* destroy on the borrowed string is a no-op - `line` is not the library's to
     * free. Calling it anyway is correct and costs nothing, and it means the
     * teardown code does not have to know which kind of string it holds. */
    proven_u8str_destroy(alloc, &status);

    /* destroy on the owned string frees the allocation, and it must be given the
     * allocator the string was created with. */
    proven_u8str_destroy(alloc, &path);
    return EXAMPLE_OK();
}
```

Worked example: when the buffer is fixed and the data is not

The example above grows the string whenever it runs out of room. A great deal of real code cannot do that: a record has a fixed field width, a log line has a hard limit, a string lives in an arena where every reallocation leaves the old copy behind until the next reset. Those callers need to know which of three answers a call gives when the data does not fit.

Kind of call	What it does when the data does not fit	Calls
Atomic, fixed capacity	Refuses with `PROVEN_ERR_OUT_OF_BOUNDS` and changes nothing.	`proven_u8str_append`, `proven_u8str_insert`, `proven_u8str_replace_at`
Best-effort (truncating)	Writes what fits, returns `PROVEN_ERR_OUT_OF_BOUNDS` **and** the byte count it wrote.	`proven_u8str_append_partial`
Atomic, growable	Reallocates through the allocator; on allocation failure changes nothing.	`proven_u8str_append_grow`, `proven_u8str_replace_at_grow`, `proven_u8str_append_byte`

Two supporting calls appear here as well. `proven_u8str_reserve()` raises the capacity once, up front, so later growth does not reallocate - on the heap that saves copying, and in an arena it saves the dead storage every reallocation leaves behind. `proven_u8str_is_valid()` checks that a string handle's own fields are consistent; it is worth asserting where a string arrives from other code, not after every edit.

The program below writes a bounded log line, refuses an oversized append, truncates deliberately with the best-effort call, and edits a path in the middle with `proven_u8str_replace_at()` - first shrinking (which always fits), then growing (which does not, and is refused), then the same edit again through `proven_u8str_replace_at_grow()`. It ends by checking the file extension with `proven_u8str_view_ends_with()`.

```
/*
 * The previous example grew a string whenever it ran out of room. This one is
 * about the case where growing is not allowed - a fixed-size record, a log line
 * with a hard length limit, a buffer in an arena that must not be reallocated -
 * and about the two honest answers a call can give when the data does not fit:
 *
 *   "no, and I changed nothing"      - the atomic calls: append, insert,
 *                                      replace_at. They check the capacity
 *                                      first, so a refusal leaves the string
 *                                      exactly as it was.
 *   "some of it, and here is how much" - the best-effort call:
 *                                      append_partial. It fills what it can and
 *                                      tells you the byte count it wrote.
 *
 * Both are useful; picking the wrong one silently truncates a record or
 * silently drops one. The `_grow` variants are the third answer - "yes, I found
 * more room" - and appear at the end for contrast.
 */

#define FIELD_CAP 32u

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    proven_result_u8str_t r = proven_u8str_create(alloc, FIELD_CAP);
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "creating the field buffer must succeed");
```

```
    if (!proven_is_ok(r.err)) {
        return 1;
    }
    proven_u8str_t field = r.value;

    /* is_valid checks the handle's own structure - a pointer with a capacity and
     * a length that do not contradict each other. Worth asserting once at the
     * boundary of your code when a string arrives from somewhere else; it is not
     * a check you need after every edit, because every edit maintains it. */
    EXAMPLE_REQUIRE(proven_u8str_is_valid(&field), "a freshly created string must be structurally valid");

    /* --- reserving room up front ------------------------------------------- */

    /* reserve raises the capacity now, so later growth does not reallocate. On
     * the heap that saves copies; in an arena it saves something worse, because
     * every reallocation there leaks the old block until the next reset. Ask for
     * what you expect to need, once. */
    proven_err_t err = proven_u8str_reserve(alloc, &field, 64);
    EXAMPLE_REQUIRE(proven_is_ok(err), "reserving 64 bytes must succeed");
    EXAMPLE_REQUIRE(field.internal.cap >= 64, "the capacity must actually be at least what was asked for");

    /* --- the atomic calls: fit, or change nothing -------------------------- */

    err = proven_u8str_append(&field, PROVEN_LIT("2026-01-01 level=info "));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the prefix fits in the reserved capacity");

    /* append_byte adds one byte, which is what separators, terminators and
     * escape characters are. It takes the allocator because it is a growing
     * call - one byte is exactly the case where a capacity check would fail on
     * a boundary you did not think about. */
    err = proven_u8str_append_byte(alloc, &field, (proven_u8)'[');
    EXAMPLE_REQUIRE(proven_is_ok(err), "appending a single separator byte must succeed");

    err = proven_u8str_append(&field, PROVEN_LIT("disk full"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the message fits");

    err = proven_u8str_append_byte(alloc, &field, (proven_u8)']');
    EXAMPLE_REQUIRE(proven_is_ok(err), "closing the bracket must succeed");

    /* Now ask for more than the capacity can hold. The atomic append refuses and
     * - this is the property worth relying on - the string still holds exactly
     * what it held before the call. */
    proven_size_t before = proven_u8str_as_view(&field).size;
    err = proven_u8str_append(&field, PROVEN_LIT(" and a very long trailing explanation that certainly does
not fit"));
    EXAMPLE_REQUIRE(err == PROVEN_ERR_OUT_OF_BOUNDS, "an oversized atomic append must be refused");
    EXAMPLE_REQUIRE(proven_u8str_as_view(&field).size == before, "and must leave the string untouched");

    /* --- the best-effort call: as much as fits, and the count -------------- */

    /* A fixed-width column in a report is the case for this one: write what
     * fits, and know how much was written so the caller can mark the value as
     * truncated instead of pretending it is complete. */
    proven_result_size_t part = proven_u8str_append_partial(&field, PROVEN_LIT(" ...more text than there is
room for"));
    EXAMPLE_REQUIRE(part.err == PROVEN_ERR_OUT_OF_BOUNDS, "a partial append that truncates still reports the
truncation");
    EXAMPLE_REQUIRE(part.value > 0, "but it wrote what it could");
    EXAMPLE_REQUIRE(proven_u8str_as_view(&field).size == before + part.value,
                    "and the string grew by exactly the number of bytes it reports");
```

```
    printf("partial append wrote %zu byte(s) before the buffer was full\n", (size_t)part.value);

    /* --- editing in the middle without growing ----------------------------- */

    proven_result_u8str_t r2 = proven_u8str_create(alloc, FIELD_CAP);
    EXAMPLE_REQUIRE(proven_is_ok(r2.err), "creating the second buffer must succeed");
    proven_u8str_t path = r2.value;
    err = proven_u8str_append(&path, PROVEN_LIT("var/log/service.log"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the path fits");

    /* insert shifts the tail right. Fixed-capacity: it fits or it refuses. */
    err = proven_u8str_insert(&path, 0, PROVEN_LIT("/"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "inserting a leading slash must succeed");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&path), PROVEN_LIT("/var/log/service.log")),
                    "the insert lands at index 0");

    /* replace_at replaces old_len bytes at an index with data of any length, as
     * long as the result still fits. Replacing "service" (7) with "daemon" (6)
     * shrinks the string, so this cannot fail on capacity. */
    proven_size_t at = proven_u8str_view_find(proven_u8str_as_view(&path), 0, PROVEN_LIT("service"));
    EXAMPLE_REQUIRE(at != PROVEN_SIZE_MAX, "the substring must be found before it can be replaced");
    err = proven_u8str_replace_at(&path, at, 7, PROVEN_LIT("daemon"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "a shortening replacement must succeed");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&path), PROVEN_LIT("/var/log/daemon.log")),
                    "and produce the expected path");

    /* The same edit the other way round overflows a 32-byte buffer, and the
     * fixed-capacity call refuses it rather than truncating a path - which is
     * the failure that silently writes to the wrong file. */
    before = proven_u8str_as_view(&path).size;
    err = proven_u8str_replace_at(&path, at, 6, PROVEN_LIT("a-replacement-name-far-too-long-for-this-
buffer"));
    EXAMPLE_REQUIRE(err == PROVEN_ERR_OUT_OF_BOUNDS, "a replacement that does not fit must be refused");
    EXAMPLE_REQUIRE(proven_u8str_as_view(&path).size == before, "and must leave the path unchanged");

    /* replace_at_grow is the same edit with permission to reallocate. Use it
     * when the buffer is heap-backed and the length is genuinely unbounded;
     * prefer the fixed-capacity call when the limit is part of the format. */
    err = proven_u8str_replace_at_grow(alloc, &path, at, 6, PROVEN_LIT("a-replacement-name-far-too-long-for-
this-buffer"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the growing variant makes room instead of refusing");
    EXAMPLE_REQUIRE(proven_u8str_view_ends_with(proven_u8str_as_view(&path), PROVEN_LIT(".log")),
                    "the extension is still at the end after the edit");

    /* ends_with answers the question an extension check actually asks. Doing it
     * with an index computed by hand is where the off-by-one lives; doing it
     * with strcmp on a pointer requires a NUL that a view does not have. */
    EXAMPLE_REQUIRE(!proven_u8str_view_ends_with(proven_u8str_as_view(&path), PROVEN_LIT(".txt")),
                    "and it is not a .txt file");

    /* An empty suffix is a suffix of everything, which is the answer that keeps
     * loops over a list of suffixes from needing a special case. */
    EXAMPLE_REQUIRE(proven_u8str_view_ends_with(proven_u8str_as_view(&path), PROVEN_LIT("")),
                    "every string ends with the empty suffix");

    printf("log line: %s\n", proven_u8str_as_cstr(&field));
    printf("path:     %s\n", proven_u8str_as_cstr(&path));

    proven_u8str_destroy(alloc, &path);
    proven_u8str_destroy(alloc, &field);
```

```
    return EXAMPLE_OK();
}
```

Wrong - treating the best-effort call as if it were atomic:

```
(void)proven_u8str_append_partial(&line, field);   /* wrong: ignores the count */
```

The return value is the only place the truncation is reported. Discarding it turns "the record was cut short" into "the record looked fine".

Wrong - reading the byte count as a character count:

```
if (part.value == field.size) { /* all of it was written */ }
```

That comparison is correct, and it is correct in **bytes**. A UTF-8 character can be up to four of them, so a partial append can stop in the middle of one. When the tail must stay valid UTF-8, decide the cut point yourself rather than letting the capacity decide it.

Worked example: assembling a UTF-16 string for a system call

`proven_u16str_t` earns its place only at the boundary where an operating system call demands UTF-16 - the Windows wide API being the usual reason. The pattern is always the same: assemble the code units, then hand `proven_u16str_as_ptr()` to the call.

Three points decide whether this code is right:

- **The unit is a code unit, not a byte and not a character.** A capacity of 32 is 32 code units, which is 64 bytes; a character outside the Basic Multilingual Plane (BMP) - an emoji, many rarer CJK characters - occupies two of them.
- `proven_u16str_as_ptr()` **does not copy**, and the pointer it returns is good only until the next append that grows the string.
- **The result is NUL-terminated**, including after a deliberate truncation, so it is safe to pass to a system call that expects a terminator.

```
/*
 * UTF-16 exists in this library for one reason: some operating system calls
 * take it and nothing else. The Windows "wide" API is the usual case - the file
 * name you hand to CreateFileW is a NUL-terminated run of 16-bit code units,
 * not bytes.
 *
 * So the job this type does is narrow: assemble the code units, keep the count
 * right, and produce the pointer the system call wants. Everything else in your
 * program should stay UTF-8.
 *
 * The one thing to keep straight is the unit. A capacity of 32 here means 32
 * CODE UNITS, which is 64 bytes, and a character outside the Basic Multilingual
 * Plane - an emoji, most of the rarer CJK characters - costs two of them. A
 * count of code units is not a count of characters and never has been.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* The argument is a code-unit limit, not a byte limit. */
    proven_result_u16str_t r = proven_u16str_create(alloc, 32);
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "creating a 32-code-unit string must succeed");
    if (!proven_is_ok(r.err)) {
        return 1;
    }
    proven_u16str_t name = r.value;
    EXAMPLE_REQUIRE(proven_u16str_len(&name) == 0, "a new string is empty");

    /* PROVEN_U16_LIT builds a view from a u"..." literal and computes the unit
```

```
     * count from the literal itself, so the count cannot disagree with the text. */
    proven_err_t err = proven_u16str_append(&name, PROVEN_U16_LIT("C:\\logs\\"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the directory prefix fits in 32 code units");

    err = proven_u16str_append(&name, PROVEN_U16_LIT("service.log"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the file name fits too");
    EXAMPLE_REQUIRE(proven_u16str_len(&name) == 8 + 11, "the length is a count of code units");

    /* Atomic, like its byte-string twin: too much data is refused and the string
     * is left exactly as it was, so a path is never half-written. */
    proven_size_t before = proven_u16str_len(&name);
    err = proven_u16str_append(&name, PROVEN_U16_LIT(".a-suffix-long-enough-to-overflow-the-capacity"));
    EXAMPLE_REQUIRE(err == PROVEN_ERR_OUT_OF_BOUNDS, "an oversized append must be refused");
    EXAMPLE_REQUIRE(proven_u16str_len(&name) == before, "and must not truncate the path");

    /* --- the pointer the system call wants ------------------------------- */

    /* as_ptr hands back the internal code units, NUL-terminated, without
     * copying. It is the last step before the call, and the pointer is only
     * valid until the next append: a growing append may move the storage. */
    const proven_u16 *wide = proven_u16str_as_ptr(&name);
    EXAMPLE_REQUIRE(wide != NULL, "an assembled string must yield a pointer");
    EXAMPLE_REQUIRE(wide[0] == (proven_u16)'C', "the first code unit is the drive letter");
    EXAMPLE_REQUIRE(wide[proven_u16str_len(&name)] == 0, "the sequence is NUL-terminated for the system
call");
    /* On Windows this is the whole point of the type:
     *     HANDLE h = CreateFileW((LPCWSTR)wide, ...);
     * Nothing here calls it, because this example must also run everywhere else. */

    /* --- when truncation is the correct answer --------------------------- */

    /* Some system structures have a fixed-width field - a 16-unit label, say -
     * where a name that does not fit is meant to be cut, not rejected. That is
     * what the partial append is for: it fills what it can and reports the unit
     * count it wrote, so the caller can mark the value as truncated. */
    proven_result_u16str_t r2 = proven_u16str_create(alloc, 16);
    EXAMPLE_REQUIRE(proven_is_ok(r2.err), "creating the fixed-width label must succeed");
    proven_u16str_t label = r2.value;

    proven_result_size_t wrote = proven_u16str_append_partial(&label, PROVEN_U16_LIT("a-label-that-is-longer-
than-the-field"));
    EXAMPLE_REQUIRE(wrote.err == PROVEN_ERR_OUT_OF_BOUNDS, "the truncation is reported, not hidden");
    EXAMPLE_REQUIRE(wrote.value == 16, "it filled the field exactly");
    EXAMPLE_REQUIRE(proven_u16str_len(&label) == wrote.value, "and the length matches what it says it wrote");
    EXAMPLE_REQUIRE(proven_u16str_as_ptr(&label)[wrote.value] == 0,
                    "a truncated string is still NUL-terminated, so it is still safe to pass on");

    printf("assembled %zu code unit(s); label truncated to %zu\n",
           (size_t)proven_u16str_len(&name), (size_t)wrote.value);

    proven_u16str_destroy(alloc, &label);
    proven_u16str_destroy(alloc, &name);
    return EXAMPLE_OK();
}
```

Wrong - sizing a UTF-16 buffer in bytes:

```
proven_result_u16str_t r = proven_u16str_create(alloc, sizeof(buf));   /* wrong */
```

The argument is a count of code units. Passing a byte count asks for twice the storage you meant, or - when the byte count came from a UTF-8 string - for a buffer that cannot hold the conversion at all.

Wrong - keeping the pointer across an append:

```
const proven_u16 *w = proven_u16str_as_ptr(&name);
proven_err_t e = proven_u16str_append_grow(alloc, &name, more);   /* may reallocate */
use_wide(w);   /* wrong: w may dangle */
```

Take the pointer immediately before the call that consumes it.

Chapter 4: Containers and Algorithms

Part III - Data structures. Prerequisite: Part II (1, 2, 3). After this chapter you can pick the right container for a job, sort and search with a guaranteed bound, hash for the right reason, and turn bytes into text and back.

This chapter covers `array.h`, `list.h`, `ring.h`, `map.h`, `algorithm.h`, `hash.h`, and `encode.h`. Every container here takes an allocator, which is why Chapter 2 comes first.

Table of contents

1. Dynamic array
2. Intrusive list
3. Ring buffer
4. Hash map
5. Algorithms
6. Hashing, by use case
7. Bytes to text: hex and Base64
8. Examples and misuse cases

1. Dynamic array

The problem: the array you write by hand, every time

A growable array is the data structure every C program eventually writes, and writes slightly differently:

```
int *items = malloc(cap * sizeof(int));
/* ... later ... */
if (n == cap) {
    cap *= 2;
    items = realloc(items, cap * sizeof(int));   /* wrong on two counts */
    items[n++] = x;
}
```

Two bugs in one line, and both are classics. `cap * sizeof(int)` can **wrap** for a large `cap`, producing a small allocation for a huge count - see Chapter 1 section 4. And assigning `realloc`'s result straight back to `items` **leaks the old block when it returns NULL**, because the original pointer is gone and the memory it named is still allocated.

Beyond the bugs, the hand-written version has to be rewritten for every element type, or made generic with `void *` and lose type checking.

What this library does instead

`proven_array_t` is a growable vector that keeps the element size and alignment it was created with, so it works for any type without a template and without `void *` at the call site - the `PROVEN_ARRAY_*` macros take the type and do the casting where the compiler can still check it.

- Growth uses checked arithmetic, so the overflow above is `PROVEN_ERR_OVERFLOW`, not a small allocation.
- Growth is **failure-atomic**: if it cannot grow, your existing elements are untouched and still valid. Nothing is leaked and nothing is lost.
- It **stores its allocator internally**, unlike the string types - which is why `PROVEN_ARRAY_DESTROY` takes only the array.

`proven_array_t` owns contiguous element storage, so it is also what you hand to `proven_array_sort` and the searches in section 5.

Wrong - holding a pointer across a push:

```
int *first = PROVEN_ARRAY_GET_MUT(&arr, int, 0);
(void)PROVEN_ARRAY_PUSH(&arr, int, 42);   /* may reallocate */
*first = 7;                               /* wrong: `first` may point at freed memory */
```

This is the one hazard the type cannot remove. Growth moves the storage, so **any pointer or view into the array is invalidated by any operation that can grow it**. Index instead of pointing, or re-fetch the pointer after the push.

Structures

```
typedef struct {
    proven_allocator_t alloc;
    proven_byte_t *data;
    proven_size_t len;
    proven_size_t cap;
    proven_size_t elem_size;
    proven_size_t align;
} proven_array_t;

typedef struct {
    proven_err_t err;
    proven_array_t value;
} proven_result_array_t;
```

Fields:

- `alloc`: allocator used for growth and destruction.
- `data`: byte storage for elements.
- `len`: current element count.
- `cap`: capacity in elements.
- `elem_size`: size of each element.
- `align`: alignment of each element.

Functions

API	Intent	Return
`proven_array_create(alloc, init_cap, elem_size, align)`	Create a generic array.	`proven_result_array_t`.
`proven_array_is_valid(arr)`	Validate public array invariants.	`bool`.
`proven_array_reserve(arr, new_cap)`	Ensure at least `new_cap` elements.	`proven_err_t`.
`proven_array_push(arr, element)`	Append one element by copying from pointer.	`proven_err_t`.
`proven_array_pop(arr, out_element)`	Pop last element; `out_element` may be null to discard.	`proven_err_t`.
`proven_array_get_mut(arr, index)`	Get mutable element pointer.	pointer or null.
`proven_array_get(arr, index)`	Get const element pointer.	pointer or null.
`proven_array_destroy(arr)`	Free storage and clear state.	void.
`proven_array_clear(arr)`	Remove every element; capacity stays.	void.
`proven_array_truncate(arr, new_len)`	Shorten to `new_len`.	`proven_err_t`; `OUT_OF_BOUNDS` if longer.

`proven_array_insert(arr, index, element)`	Insert at `index` (0..len), moving the rest up. `element` may point into the array.	`proven_err_t`.
`proven_array_remove_at(arr, index, out_element)`	Remove at `index`, keeping order.	`proven_err_t`.
`proven_array_swap_remove(arr, index, out_element)`	Remove at `index` in O(1) by moving the last element in; order is not kept.	`proven_err_t`.
`proven_array_extend(arr, elements, count)`	Append `count` elements, one reallocation at most; may be the array's own elements.	`proven_err_t`.

The editing operations change nothing when they fail, refuse an index past the end with `PROVEN_ERR_OUT_OF_BOUNDS`, and - for the two that can grow the array - invalidate pointers into it as `push` does.

```
/* A to-do list: insert at the front, take one out from the middle keeping order, drop
 * finished items in O(1) when order does not matter, and append a batch at once. */
proven_result_array_t todo = PROVEN_ARRAY_INIT(alloc, int, 4);
if (proven_is_ok(todo.err)) {
    const int batch[] = { 10, 20, 30, 40 };
    int first = 5, taken = 0;
    proven_err_t e = proven_array_extend(&todo.value, batch, 4);                /* 10 20 30 40 */
    if (proven_is_ok(e)) e = proven_array_insert(&todo.value, 0, &first);      /* 5 10 20 30 40 */
    if (proven_is_ok(e)) e = proven_array_remove_at(&todo.value, 2, &taken);   /* 5 10 30 40, took 20 */
    if (proven_is_ok(e)) e = proven_array_swap_remove(&todo.value, 0, NULL);   /* 40 10 30 */
    if (proven_is_ok(e)) e = proven_array_truncate(&todo.value, 2);            /* 40 10 */
    proven_println("{} items, took {}", PROVEN_ARG(todo.value.len), PROVEN_ARG(taken));
    proven_array_clear(&todo.value);                                           /* empty, capacity kept */
    proven_array_destroy(&todo.value);
}
```

Macros

Macro	Intent
`PROVEN_ARRAY_INIT(alloc, type, init_cap)`	Type-safe create using `sizeof(type)` and `alignof(type)`.
`PROVEN_ARRAY_PUSH(arr_ptr, type, value)`	Push rvalue or lvalue by temporary compound literal.
`PROVEN_ARRAY_POP(arr_ptr, type, out_ptr)`	Pop into output pointer or discard with null.
`PROVEN_ARRAY_GET(arr_ptr, type, index)`	Typed const element pointer.
`PROVEN_ARRAY_GET_MUT(arr_ptr, type, index)`	Typed mutable element pointer.
`PROVEN_ARRAY_DESTROY(arr_ptr)`	Destroy array.

Example:

```
proven_result_array_t r = PROVEN_ARRAY_INIT(alloc, int, 4);
if (!proven_is_ok(r.err)) {
    return;
}
proven_array_t nums = r.value;

(void)PROVEN_ARRAY_PUSH(&nums, int, 10);
(void)PROVEN_ARRAY_PUSH(&nums, int, 20);

/* GET points into the array's own storage, and returns NULL out of range. */
const int *first = PROVEN_ARRAY_GET(&nums, int, 0);
```

```
if (first) {
    proven_println("first={}", PROVEN_ARG(*first));
}

/* The array stores its allocator, so destroy needs nothing but the array. */
PROVEN_ARRAY_DESTROY(&nums);
```

2. Intrusive list

The problem: the list you were taught allocates a node per element

`struct node { struct node *next; void *data; };`

That is the textbook linked list, and it has two costs the textbook does not mention. Every insertion **allocates** - a thousand items means a thousand allocations that exist only to hold pointers, each one able to fail and each one needing a matching free. And `data` is a `void *`, so the list has no idea what it holds: every read back out is a cast the compiler cannot check.

What an intrusive list does instead

It turns the relationship inside out: **you own the memory, and the link lives inside it.**

```
typedef struct {
    int                id;
    proven_list_node_t  link;   /* the list's hook, inside your struct */
} ex_task_t;
```

Now inserting is writing two pointers. It allocates nothing, so it **cannot fail** - notice that `proven_list_push_back` returns `void`, which is unusual in this library and is the point. Removing is the same. The objects can live on the stack, in an array, in an arena, anywhere; the list only rearranges pointers that are already inside them.

Getting from a link back to your object is `PROVEN_LIST_ENTRY(node, ex_task_t, link)`, which subtracts the member's offset from the node's address. It is arithmetic the compiler checks the types of, not a cast you hope is right.

What you give up: **an object can be in only as many lists as it has link members**, and you decide that when you declare the struct. If an item needs to be in a queue and an index at the same time, it needs two links.

`proven_list_t` is an intrusive doubly-linked **circular** list - the head is a sentinel node, so insertion and removal never special-case the ends. It allocates no nodes.

Wrong - removing while walking with the plain iterator:

```
PROVEN_LIST_FOR_EACH(it, &queue) {
    if (should_drop(it)) proven_list_remove(it);   /* wrong: it->next is read after unlink */
}
```

`proven_list_remove` writes through the node's own pointers, so the loop then reads `next` from a node that has just been detached. `PROVEN_LIST_FOR_EACH_SAFE` takes a second variable and reads the next pointer *before* the body runs, which is exactly why it exists.

Worked example: a queue whose links live in the caller's structs

Compiled and run by the test suite. It builds a queue of stack-allocated tasks, walks it, removes one from inside a safe walk, and inserts in the middle - with no allocator anywhere in the program.

```
/*
 * An INTRUSIVE list puts the links inside your struct instead of allocating a
 * node to hold your data.
 *
 * The list you were taught looks like this:
 *
 *     struct node { struct node *next; void *data; };
```

```
 *
 * Every insertion allocates a node, so a list of a thousand items costs a
 * thousand allocations that exist only to hold pointers, each one a chance to
 * fail and a thing to free. Worse, `data` is a void* - the list has no idea what
 * it holds, so every read is a cast the compiler cannot check.
 *
 * Intrusive lists invert it: YOU own the memory, and the link lives in it.
 * Inserting allocates nothing and cannot fail. Removing allocates nothing and
 * cannot fail. And because the link is a member of a known type, getting back
 * from a link to the object is arithmetic the compiler does for you, not a cast.
 *
 * The trade is that an object can only be in as many lists as it has link
 * members, and that is a decision you make when you declare the struct.
 */

/* The link is a member. This task can be in exactly one list at a time. */
typedef struct {
    int                id;
    proven_list_node_t  link;
} ex_task_t;

int main(void) {
    /* No allocator anywhere in this program: the tasks are on the stack, and the
     * list only ever rearranges pointers that live inside them. */
    ex_task_t a = { .id = 1 };
    ex_task_t b = { .id = 2 };
    ex_task_t c = { .id = 3 };

    proven_list_t queue;
    proven_list_init(&queue);
    EXAMPLE_REQUIRE(proven_list_is_empty(&queue), "a freshly initialised list is empty");

    /* --- pushing cannot fail, because nothing is allocated ------------- */
    proven_list_push_back(&queue, &a.link);
    proven_list_push_back(&queue, &b.link);
    proven_list_push_back(&queue, &c.link);
    EXAMPLE_REQUIRE(!proven_list_is_empty(&queue), "three tasks are queued");

    /* --- walking: PROVEN_LIST_ENTRY gets the object back from the link -- */
    proven_list_node_t *it = NULL;
    int seen[3] = {0}, n = 0;
    PROVEN_LIST_FOR_EACH(it, &queue) {
        ex_task_t *t = PROVEN_LIST_ENTRY(it, ex_task_t, link);
        if (n < 3) seen[n++] = t->id;
    }
    EXAMPLE_REQUIRE(n == 3 && seen[0] == 1 && seen[1] == 2 && seen[2] == 3,
                    "the walk visits every task, in insertion order");

    /* --- removing while walking needs the SAFE form -------------------- */
    /* proven_list_remove writes through the node's own next/prev pointers, so a
     * plain FOR_EACH would read `it->next` from a node that has just been
     * unlinked. The _SAFE form reads the next pointer BEFORE the body runs. */
    proven_list_node_t *safe = NULL;
    PROVEN_LIST_FOR_EACH_SAFE(it, safe, &queue) {
        ex_task_t *t = PROVEN_LIST_ENTRY(it, ex_task_t, link);
        if (t->id == 2) proven_list_remove(it);
    }

    n = 0;
    PROVEN_LIST_FOR_EACH(it, &queue) {
```

```
        ex_task_t *t = PROVEN_LIST_ENTRY(it, ex_task_t, link);
        if (n < 3) seen[n++] = t->id;
    }
    EXAMPLE_REQUIRE(n == 2 && seen[0] == 1 && seen[1] == 3, "task 2 was unlinked");

    /* --- inserting in the middle is a pointer swap ---------------------- */
    ex_task_t d = { .id = 4 };
    proven_list_insert_after(&a.link, &d.link);

    n = 0;
    PROVEN_LIST_FOR_EACH(it, &queue) {
        ex_task_t *t = PROVEN_LIST_ENTRY(it, ex_task_t, link);
        if (n < 3) seen[n++] = t->id;
    }
    EXAMPLE_REQUIRE(n == 3 && seen[0] == 1 && seen[1] == 4 && seen[2] == 3,
                    "task 4 sits directly after task 1");

    /* There is nothing to destroy. The list never owned anything: `a`, `c` and
     * `d` are still perfectly good local variables, and their lifetime is the
     * function's, exactly as it would be without the list. */
    return EXAMPLE_OK();
}
```

Structures

```
typedef struct proven_list_node_t {
    struct proven_list_node_t *next;
    struct proven_list_node_t *prev;
} proven_list_node_t;

typedef struct {
    proven_list_node_t head;
} proven_list_t;
```

Functions and macros

API	Intent	Return
`proven_list_init(list)`	Initialize circular sentinel.	void.
`proven_list_insert_after(target, node)`	Insert `node` after `target`.	void.
`proven_list_push_back(list, node)`	Insert at tail.	void.
`proven_list_remove(node)`	Detach node and poison links to null.	void.
`proven_list_is_empty(list)`	Test empty or null list.	int truth value.
`PROVEN_CONTAINER_OF(ptr, type, member)`	Convert member pointer to parent object pointer.	pointer.
`PROVEN_LIST_FOR_EACH(iter, list)`	Iterate nodes.	loop macro.
`PROVEN_LIST_FOR_EACH_SAFE(iter, safe_next, list)`	Iterate safely while removing current node.	loop macro.
`PROVEN_LIST_ENTRY(ptr, type, member)`	Convert list node to containing object.	pointer.

Example:

```
typedef struct Item {
    int value;
    proven_list_node_t link;   /* the list lives inside the object; it allocates nothing */
} Item;

proven_list_t list;
proven_list_init(&list);
```

```
Item a = { .value = 1 };
Item b = { .value = 2 };
proven_list_push_back(&list, &a.link);
proven_list_push_back(&list, &b.link);

int total = 0;
proven_list_node_t *it = NULL;
PROVEN_LIST_FOR_EACH(it, &list) {
    /* The iterator walks nodes; ENTRY converts a node back to its owner. */
    Item *item = PROVEN_LIST_ENTRY(it, Item, link);
    total += item->value;
}
proven_println("total={}", PROVEN_ARG(total));   /* 3 */
```

3. Ring buffer

The problem: a queue that must not grow
A queue between a fast producer and a slow consumer has to answer one question: what happens when the consumer falls behind? A growable queue answers "allocate more", which turns a temporary slowdown into unbounded memory growth and, eventually, into something worse than dropped work.

The hand-written alternative is an array plus a head index plus a tail index plus modulo arithmetic - and it has one famous bug. When `head == tail`, is the buffer empty or full? Both states look identical unless you keep a separate count or deliberately waste one slot, and every generation of programmers rediscovers this.

What this library does instead
`proven_ring_t` is a fixed-capacity FIFO that keeps the count, so empty and full are distinct, and that **refuses** rather than grows:

- `proven_ring_push` on a full ring returns `PROVEN_ERR_OUT_OF_BOUNDS`. It does not overwrite the oldest entry and it does not reallocate. The caller decides - wait, drop the new item, or report backpressure - because only the caller knows which is right.
- `proven_ring_pop` on an empty ring fails rather than handing back a stale slot.

The capacity is chosen once, at creation, and that number *is* the policy: it says how far ahead the producer may get. Reach for a ring for an event queue, a log of recent items, an audio or sensor buffer - anywhere a bound is part of the design rather than a limitation.

Unlike most types in this library, `proven_ring_t` **stores its allocator internally**, which is why `PROVEN_RING_DESTROY` takes only the ring.

Wrong - treating a full ring as an error to retry immediately:

```
while (PROVEN_RING_PUSH(&ring, ex_event_t, e) != PROVEN_OK) { }   /* wrong: spins forever */
```

Nothing in that loop consumes anything, so the ring stays full. A refusal from a bounded queue is information about the *consumer*, and the loop that ignores it is a hang.

Worked example: pushing until full, and what refusal looks like
Compiled and run by the test suite.

```
/*
 * A ring buffer is a fixed-size queue that never moves its contents and never
 * grows. You give it a capacity once; push adds at the tail, pop removes from
 * the head, and when it is full, push REFUSES.
 *
 * The C you would otherwise write is an array plus two indices plus the modulo
 * arithmetic to wrap them, and the bug is always the same: "is it full or is it
```

```
 * empty?" - both states have head == tail unless you keep a count or waste a
 * slot. This ring keeps the count, so the question does not arise.
 *
 * Use one when a producer and a consumer run at different speeds and you want a
 * hard bound on how far ahead the producer may get: an event queue, a log ring,
 * an audio buffer. The refusal on a full ring is the feature - it is
 * backpressure. A growable queue would answer a burst by eating memory until
 * something worse happens.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* Capacity of 4 events. It will never be 5. */
    proven_result_ring_t r = PROVEN_RING_INIT(alloc, int, 4);
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "creating a 4-slot ring should succeed");
    proven_ring_t ring = r.value;

    /* --- push until full ------------------------------------------------ */
    for (int i = 1; i <= 4; ++i) {
        proven_err_t err = PROVEN_RING_PUSH(&ring, int, i);
        EXAMPLE_REQUIRE(proven_is_ok(err), "the first four pushes fit");
    }

    /* The fifth push is refused. It does not overwrite the oldest entry, and it
     * does not grow: a full ring is a full ring, and the caller decides what to
     * do about it - wait, drop the new event, or report backpressure. */
    proven_err_t full = PROVEN_RING_PUSH(&ring, int, 5);
    EXAMPLE_REQUIRE(full == PROVEN_ERR_OUT_OF_BOUNDS,
                    "pushing into a full ring must be refused, not silently absorbed");

    /* --- pop in the order they were pushed ------------------------------ */
    int out = 0;
    proven_err_t err = PROVEN_RING_POP(&ring, int, &out);
    EXAMPLE_REQUIRE(proven_is_ok(err) && out == 1, "pop returns the oldest entry first");

    /* Now there is room again, and the ring wraps around its storage without
     * moving anything. */
    err = PROVEN_RING_PUSH(&ring, int, 5);
    EXAMPLE_REQUIRE(proven_is_ok(err), "after a pop there is room for one more");

    int expected[] = { 2, 3, 4, 5 };
    for (int i = 0; i < 4; ++i) {
        err = PROVEN_RING_POP(&ring, int, &out);
        EXAMPLE_REQUIRE(proven_is_ok(err) && out == expected[i],
                        "entries come out in the order they went in");
    }

    /* --- empty behaves like full: it refuses, it does not invent data --- */
    err = PROVEN_RING_POP(&ring, int, &out);
    EXAMPLE_REQUIRE(!proven_is_ok(err), "popping an empty ring must fail rather than return junk");

    PROVEN_RING_DESTROY(&ring);
    return EXAMPLE_OK();
}
```

Structures

```
typedef struct {
    proven_allocator_t alloc;
    proven_mem_mut_t internal;
```

```
    proven_size_t head;
    proven_size_t tail;
    proven_size_t len;
    proven_size_t cap;
    proven_size_t elem_size;
    proven_size_t align;
} proven_ring_t;

typedef struct {
    proven_err_t err;
    proven_ring_t value;
} proven_result_ring_t;
```

Functions and macros

API	Intent	Return
`proven_ring_create(alloc, cap, elem_size, align)`	Create fixed-capacity ring.	`proven_result_ring_t`.
`proven_ring_is_valid(ring)`	Validate ring invariants.	`bool`.
`proven_ring_push(ring, element)`	Push one element; fails when full.	`proven_err_t`.
`proven_ring_pop(ring, out_element)`	Pop one element; null output discards.	`proven_err_t`.
`proven_ring_destroy(ring)`	Free storage.	void.
`PROVEN_RING_INIT(alloc, type, cap)`	Type-safe create.	`proven_result_ring_t`.
`PROVEN_RING_PUSH(ring_ptr, type, value)`	Type-safe push.	`proven_err_t`.
`PROVEN_RING_POP(ring_ptr, type, out_ptr)`	Type-safe pop.	`proven_err_t`.
`PROVEN_RING_DESTROY(ring_ptr)`	Destroy ring.	void.

Example:

```
proven_result_ring_t r = PROVEN_RING_INIT(alloc, int, 8);
if (!proven_is_ok(r.err)) {
    return;
}
proven_ring_t q = r.value;

proven_err_t e = PROVEN_RING_PUSH(&q, int, 7);
if (!proven_is_ok(e)) {
    /* The ring is fixed-capacity: a full ring is PROVEN_ERR_OUT_OF_BOUNDS,
     * never a silent grow. */
    PROVEN_RING_DESTROY(&q);
    return;
}

int out = 0;
e = PROVEN_RING_POP(&q, int, &out);   /* FIFO: out == 7. Empty ring is OUT_OF_BOUNDS. */
(void)e;

PROVEN_RING_DESTROY(&q);
```

4. Hash map

`proven_map_t` is an open-addressing hash map with tombstones. It supports integer keys and borrowed or owned U8 string keys. It stores its allocator internally.

Structures

```
typedef enum {
    PROVEN_KEY_TYPE_INT,          /* keys are proven_size_t integers */
    PROVEN_KEY_TYPE_U8_BORROWED,  /* keys are u8 views; caller keeps the bytes alive */
    PROVEN_KEY_TYPE_U8_OWNED      /* keys are u8 views; the map copies and frees the bytes */
} proven_key_type_t;

typedef union {
    proven_size_t id;             /* used when key_type == PROVEN_KEY_TYPE_INT */
    proven_u8str_view_t str;      /* used for the two U8 key modes */
} proven_map_key_t;

typedef struct {
    proven_allocator_t alloc;     /* allocator for the bucket array and owned keys */
    proven_mem_mut_t internal;    /* the single contiguous bucket array (ptr + size) */
    proven_size_t len;            /* live entries */
    proven_size_t used;           /* live entries + tombstones (drives the load factor) */
    proven_size_t cap;            /* number of buckets, always a power of two */
    proven_size_t elem_size;      /* size of one stored value */
    proven_size_t align;          /* alignment requested for the value */
    proven_size_t bucket_stride;  /* bytes per bucket: align_up(header + elem_size) */
    proven_size_t payload_offset; /* bytes from a bucket start to its value payload */
    proven_key_type_t key_type;   /* key mode chosen at create time */
} proven_map_t;

typedef struct {
    proven_err_t err;
    proven_map_t value;
} proven_result_map_t;
```

How it works internally

The map is a single flat array of `cap` buckets - there are no per-entry allocations for values (and none for keys in INT/BORROWED mode), so lookups stay cache-friendly. Each bucket is laid out as:

```
[ header: state + key ][ padding to the value's alignment ][ value payload ]
^ bucket start                                              ^ payload_offset
¦<------------------------ bucket_stride ----------------------------------->¦
```

The header's `state` is one of **EMPTY** (never used), **OCCUPIED** (holds a live key+value), or **TOMBSTONE** (a removed entry). `payload_offset` and `bucket_stride` are computed once at create time from `elem_size`/`align`, so addressing bucket `i`'s value is just `internal.ptr + i*bucket_stride + payload_offset`.

- **Hashing, and why the default is the safe one. Keys are hashed under a per-process secret** drawn once from the OS CSPRNG - because a map that hashes *untrusted* keys with a predictable function is a denial of service waiting to happen: an attacker who controls the keys computes collisions offline, floods them all into one bucket, and turns every lookup into a linear scan. Keying the hash with a secret they cannot see is what closes that, and it is the same choice Python, Rust, and the Linux kernel made for their own tables. String keys use SipHash-2-4; integer keys use SipHash-1-3 over the key's 8 bytes - an integer is no safer: user ids and record numbers from a request are chosen by whoever sent it. If your keys all come from your own program, `proven_map_create_trusted` opts into the fast unkeyed functions instead - FNV-1a for strings, a SplitMix-style bit-mix finaliser for integers (about 7 ns less per lookup on a small table); `proven_map_hash` shows you which function a given map actually uses. (On a freestanding target, which has no CSPRNG and no attacker model, keys fall back to the unkeyed functions.)
- **Probing.** Linear open addressing: the start bucket is `hash & (cap - 1)` (cheap because `cap` is a power of two), then the search walks forward one bucket at a time, wrapping around, until it finds the

key (OCCUPIED with an equal key) or an EMPTY bucket (which proves the key is absent). Linear probing keeps the walked buckets contiguous in memory.

- **Removal and tombstones.** `proven_map_remove` cannot just blank a bucket -- that would cut a probe chain and hide later keys. It marks the bucket TOMBSTONE instead. Tombstones are skipped by lookups but still consume a slot, which is why `used` (live + tombstones) - not `len` - drives growth.
- **Load factor and resize.** When `used >= cap * 3/4`, the map allocates a new bucket array of the next power of two and **rehashes** every OCCUPIED entry into it, dropping all tombstones in the process. Capacity only grows; it never shrinks. Reserve ahead with `proven_map_reserve` (especially with arena allocators, to avoid leaving dead arrays behind).

Key modes - choosing one

- `PROVEN_KEY_TYPE_INT`: keys are `proven_size_t`. No key storage.
- `PROVEN_KEY_TYPE_U8_BORROWED`: the bucket stores the *view* (pointer + length). **The map never copies the bytes**, so the caller must keep the exact bytes alive and unmoved for the whole time the entry exists. Cheapest for keys that already live somewhere stable (string literals, interned strings).
- `PROVEN_KEY_TYPE_U8_OWNED`: on insert (`proven_map_set_u8_owned` / `PROVEN_MAP_SET_U8_OWNED`) the map **duplicates** the key bytes into its own storage and frees them on remove/destroy, so the source buffer may be released or reused immediately after the call.

The `set_with_scratch` / alias case

If the `element` you pass to `proven_map_set` points *into the map's own bucket array* (for example, copying a value from one key to another via a pointer returned by `proven_map_get`), a rehash triggered by that same insert would move or free the bytes you are reading. `proven_map_set_with_scratch` (and the `*_WITH_SCRATCH_*` macros) capture the source bytes into a temporary buffer from the scratch allocator first, so the insert is alias-safe. Persistent storage still uses `map->alloc`; the scratch allocator is only for that transient copy.

Functions

API	Intent	Return
`proven_map_create(alloc, init_cap, key_type, elem_size, align)`	Create a map.	`proven_result_map_t`.
`proven_map_is_valid(map)`	Validate map invariants.	`bool`.
`proven_map_reserve(map, new_cap)`	Ensure capacity.	`proven_err_t`.
`proven_map_set_with_scratch(map, key, element, scratch)`	Insert/update using scratch for temporary alias-safe copies.	`proven_err_t`.
`proven_map_set(map, key, element)`	Insert/update.	`proven_err_t`.
`proven_map_set_u8_owned(map, key, element)`	Insert/update with map-owned U8 key storage.	`proven_err_t`.
`proven_map_get_mut(map, key)`	Lookup mutable value.	pointer or null.
`proven_map_get(map, key)`	Lookup const value.	pointer or null.
`proven_map_remove(map, key)`	Remove key if present.	`proven_err_t`.
`proven_map_destroy(map)`	Free map storage.	void.
`proven_map_len(map)`	Number of entries; 0 for NULL.	`proven_size_t`.
`proven_map_iter_init(map)`	Start a walk over every entry, in bucket order.	`proven_map_iter_t`.
`proven_map_iter_next(&it, &key, &value)`	The next entry. `PROVEN_ERR_EOF` after the last; `PROVEN_ERR_INVALID_STATE` if a new key or a reserve rehashed the map	`proven_err_t`.

	during the walk. Removing entries and updating values is allowed.	

Visiting every entry. The map has no order a caller can rely on - bucket order, and a keyed map's differs per process - but it can be walked, to list what it holds, serialise it, or release what its values own before `proven_map_destroy`. Removing the entry just returned is allowed:

```
/* Count words, then drop the rare ones while walking the map. */
proven_result_map_t words = proven_map_create(alloc, 16, PROVEN_KEY_TYPE_U8_BORROWED, sizeof(int),
_Alignof(int));
if (proven_is_ok(words.err)) {
    int one = 1, three = 3;
    (void)proven_map_set(&words.value, (proven_map_key_t){ .str = PROVEN_LIT("rare") }, &one);
    (void)proven_map_set(&words.value, (proven_map_key_t){ .str = PROVEN_LIT("common") }, &three);
    proven_map_iter_t it = proven_map_iter_init(&words.value);
    proven_map_key_t key;
    void *value;
    while (proven_map_iter_next(&it, &key, &value) == PROVEN_OK) {
        if (*(int *)value < 2) (void)proven_map_remove(&words.value, key);
    }
    proven_println("{} word(s) left", PROVEN_ARG(proven_map_len(&words.value)));
    proven_map_destroy(&words.value);
}
```

Macros

Macro	Intent
`proven_map_create_with_capacity(...)`	Alias emphasizing capacity allocation.
`PROVEN_MAP_INIT_INT(alloc, type, init_cap)`	Create integer-key typed map.
`PROVEN_MAP_INIT_U8_BORROWED(alloc, type, init_cap)`	Create borrowed-string-key typed map.
`PROVEN_MAP_INIT_U8_OWNED(alloc, type, init_cap)`	Create owned-string-key typed map.
`PROVEN_MAP_SET_INT(map_ptr, int_key, type, value)`	Set integer key.
`PROVEN_MAP_SET_WITH_SCRATCH_INT(map_ptr, int_key, type, value, scratch)`	Set integer key using scratch allocator.
`PROVEN_MAP_SET_U8_BORROWED(map_ptr, u8_view, type, value)`	Set borrowed U8 key.
`PROVEN_MAP_SET_U8_OWNED(map_ptr, u8_view, type, value)`	Set owned U8 key.
`PROVEN_MAP_SET_WITH_SCRATCH_U8_BORROWED(map_ptr, u8_view, type, value, scratch)`	Set borrowed U8 key using scratch allocator.
`PROVEN_MAP_GET_INT(map_ptr, type, int_key)`	Get const value by integer key.
`PROVEN_MAP_GET_U8_BORROWED(map_ptr, type, u8_view)`	Get const value by borrowed U8 key.
`PROVEN_MAP_GET_U8_OWNED(map_ptr, type, u8_view)`	Get const value by owned U8 key.
`PROVEN_MAP_GET_MUT_INT(map_ptr, type, int_key)`	Get mutable value by integer key.
`PROVEN_MAP_GET_MUT_U8_BORROWED(map_ptr, type, u8_view)`	Get mutable value by borrowed U8 key.
`PROVEN_MAP_GET_MUT_U8_OWNED(map_ptr, type, u8_view)`	Get mutable value by owned U8 key.
`PROVEN_MAP_REMOVE_INT(map_ptr, int_key)`	Remove integer key.
`PROVEN_MAP_REMOVE_U8_BORROWED(map_ptr, u8_view)`	Remove U8 key.
`PROVEN_MAP_REMOVE_U8_OWNED(map_ptr, u8_view)`	Remove owned U8 key.
`PROVEN_MAP_DESTROY(map_ptr)`	Destroy map.

Owned key path:

`PROVEN_KEY_TYPE_U8_OWNED` stores a duplicate of the key bytes inside the map. Use it when the source buffer may move or be freed after insertion. The owned bytes are released on remove and destroy, and they move with the entry during rehash.

`PROVEN_HARDENED` and debug validation can reject some borrowed-key pointers that fall inside the map's own internal storage. That check is defensive only; it does not make borrowed keys self-owning.

Example:

```
typedef struct UserInfo {
    int level;
    double budget;
} UserInfo;

proven_result_map_t r = PROVEN_MAP_INIT_INT(alloc, UserInfo, 8);
if (!proven_is_ok(r.err)) {
    return;
}
proven_map_t users = r.value;

UserInfo u = { .level = 3, .budget = 99.0 };
(void)PROVEN_MAP_SET_INT(&users, 404, UserInfo, u);

/* GET returns a pointer into the bucket array, or NULL when absent. Any insert
 * that rehashes invalidates it: look it up, use it, drop it. */
const UserInfo *found = PROVEN_MAP_GET_INT(&users, UserInfo, 404);
if (found) {
    proven_println("level={}", PROVEN_ARG(found->level));
}

PROVEN_MAP_DESTROY(&users);
```

5. Algorithms

Array algorithms operate on `proven_array_t` and a comparison callback.

Comparison function

```
typedef int (*proven_compare_fn_t)(const void *a, const void *b);
```

Return negative if `a < b`, zero if equal, positive if `a > b`.

Functions

API	Intent	Return
`proven_array_sort(arr, cmp)`	Sort array in place.	void.
`proven_array_binary_search(arr, key, cmp)`	Search sorted array.	pointer to element or null.
`proven_array_linear_search(arr, key, cmp)`	Search any array by scanning.	pointer to element or null.

`proven_array_sort` is an introsort: a Bentley-McIlroy three-way partition, an insertion-sort cutoff for small ranges, and a heapsort fallback once the recursion exceeds `2*log2(n)` levels.

Two properties are worth stating, because they are the ones that bite:

- **O(n log n) is a guarantee, not a typical case.** The heapsort fallback is what makes it one. A sort whose worst case can be reached by an adversarial ordering is a denial of service in any program that sorts data it did not author.

- **Duplicate keys are the fast case, not the slow one.** Elements equal to the pivot are collected into a run that is final and never recursed into, so all-equal input costs a single pass. This matters because low-cardinality keys
  - a status column, an enum, a bucket id - are what callers actually sort by, and they are exactly what a naive two-way partition degrades on.

The sort is not stable: equal elements may be reordered.

The comparator is an ordinary file-scope function, so the shape is:

```
static int cmp_int(const void *a, const void *b) {
    int x = *(const int *)a;
    int y = *(const int *)b;
    return (x > y) - (x < y);   /* not (x - y): that overflows */
}

proven_array_sort(&nums, cmp_int);
int key = 20;
int *hit = proven_array_binary_search(&nums, &key, cmp_int);
```

The worked example at the end of this chapter (`manual/examples/ex_04_array.c`) is the compiled-and-run version of exactly this.

6. Hashing, by use case

There is no single "hash". Which function is correct depends entirely on what you are doing with the result, and reaching for the wrong one gives you either a program that is needlessly slow or one that is quietly insecure. `proven/hash.h` offers exactly one primitive per job so the choice is made once you name the job:

You are…	Use	And crucially
hashing keys into **your own** table, **trusted** input	`proven_hash_bytes` (FNV-1a)	fast; a crypto hash here is ~50x slower for no gain
hashing keys from **untrusted** input into a table	`proven_hash_keyed` (SipHash-2-4)	FNV lets an attacker collide every key into one bucket and turn your O(1) table into O(n^2)
checking data was not **corrupted** in transit or on disk	`proven_crc32`	a checksum; interoperates with gzip/zlib/PNG
fingerprinting content: dedup, content-addressing, "same file?"	`proven_sha256`	the only one safe against a *deliberately* forged match

The one line to remember: **CRC-32 and FNV detect accident, not attack.** Do not use them to decide whether two things are "the same" when someone might benefit from fooling you -- that is what `proven_sha256` is for. And a keyed hash is only safe if the key is a real secret chosen once from real randomness; a fixed key is no key at all.

Every function is byte-exact and endianness-independent: the same input gives the same output on any target, because a fingerprint that changed with the machine would be a fingerprint of the machine, not the content. All four are royalty-free algorithms (FNV, CRC-32: public domain; SipHash: CC0; SHA-256: FIPS 180-4, unpatented), implemented from their specifications and checked against each one's official known-answer vectors.

Reference

API	Intent	Return

`proven_hash_bytes(view)`	FNV-1a 64. A hash-table hash for keys you chose yourself.	`proven_u64`.
`proven_hash_keyed(view, key[16])`	SipHash-2-4 under a 16-byte secret. The same job, for keys an attacker supplies.	`proven_u64`.
`proven_crc32(view)`	One-shot CRC-32 (IEEE, reflected) - the one gzip/zlib/PNG carry.	`proven_u32`.
`proven_crc32_update(crc, view)`	The same CRC over a stream of chunks. Start from `0`; the value you hold between calls is the real CRC, so you can store it, log it, and resume.	`proven_u32`.
`proven_sha256(view, out[32])`	One-shot SHA-256.	void; writes `PROVEN_SHA256_SIZE` bytes.
`proven_sha256_init/_update/_final`	The same digest over content you cannot hold in memory at once. The digest depends only on the bytes, never on how they were chunked.	void.
`proven_sha256_to_hex(digest, out[65])`	The 64-character lowercase spelling `sha256sum` and `git` print. NUL-terminated.	void.

The one structure you hold:

```
typedef struct {
    /* Opaque. A running SHA-256: the 8-word chain value, a 64-byte block being
       filled, and the message length. Declare one, init it, update it, finalise it. */
} proven_sha256_t;

#define PROVEN_SHA256_SIZE 32   /* the digest; size your output buffer with this */
```

`proven_sha256_t` allocates nothing - it is caller-owned state, so there is nothing to destroy.

Cautions, and what goes wrong

A keyed hash with a fixed key is not a keyed hash. The security of `proven_hash_keyed` rests entirely on the key being a secret drawn once from real randomness. This is why `proven_map_create` does it for you and why you should almost never call `proven_hash_keyed` yourself for a table.

Wrong:

```
proven_byte_t key[16] = { 0 };            /* wrong: a "secret" everyone knows */
proven_u64 h = proven_hash_keyed(user_input, key);
```

```
proven_byte_t key[16];
memcpy(key, &timestamp, sizeof timestamp); /* wrong: guessable, and mostly zero */
```

Correct - draw it once, at startup, from the OS:

```
proven_byte_t key[16];
if (proven_random_bytes(key, sizeof key)) {
    proven_u64 h = proven_hash_keyed(
        proven_mem_view_from_u8(PROVEN_LIT("session-token")), key);
    (void)h;
}
```

Do not use CRC-32 or FNV to decide two things are "the same" when someone might gain by fooling you. Both are trivially forgeable: producing a second input with the same CRC-32 is schoolbook arithmetic.

Wrong - a content-addressed store an attacker can poison:

```
if (proven_crc32(incoming) == stored_crc) {
    accept_as_identical(incoming);   /* wrong: a forged collision costs seconds */
}
```

Correct - a fingerprint that must not be foolable is `proven_sha256`, compared over all 32 bytes.

A digest buffer that is not `PROVEN_SHA256_SIZE` is a buffer overflow. `proven_sha256` writes exactly 32 bytes and does not know how big your array is.

```
proven_byte_t digest[16];        /* wrong: SHA-256 is 32 bytes */
proven_sha256(data, digest);     /* writes 32 into a 16-byte buffer */
```

Compiled and run by the test suite:

```
/*
 * Hashing, by use case. The module gives you exactly one function per job, so the only
 * decision is which job you have - and getting THAT wrong is the whole danger.
 */

int main(void) {
    proven_mem_view_t data = proven_mem_view_from_u8(PROVEN_LIT("the quick brown fox"));

    /* Job 1: hash a key into your own table, trusted input. Fast, non-cryptographic. */
    proven_u64 table_hash = proven_hash_bytes(data);
    EXAMPLE_REQUIRE(table_hash != 0, "FNV-1a produces a spread-out 64-bit value");

    /* Job 2: hash a key from UNTRUSTED input. Same purpose, but an attacker who picks the
     * input still cannot make everything collide, because they do not have the key. Pick
     * the key once at startup from real randomness; a fixed key defeats the point. */
    proven_byte_t key[16] = { 0 };   /* in real code: fill from a random source, once */
    proven_u64 safe_hash = proven_hash_keyed(data, key);
    EXAMPLE_REQUIRE(safe_hash != table_hash, "a keyed hash is a different function");

    /* Job 3: did these bytes get corrupted? A checksum, not a hash. Interoperates with
     * gzip/zlib/PNG, which all carry this exact CRC-32. */
    proven_u32 checksum = proven_crc32(data);
    /* The canonical CRC-32 sanity value, so you can see it is the real one: */
    EXAMPLE_REQUIRE(proven_crc32(proven_mem_view_from_u8(PROVEN_LIT("123456789"))) == 0xcbf43926u,
                    "CRC-32 of \"123456789\" is the shared check value");
    (void)checksum;

    /* Job 4: fingerprint content - dedup, content-addressing, "are these the same file",
     * answered safely even against someone trying to forge a match. This is the one you
     * reach for when the answer must not be foolable. */
    proven_byte_t digest[PROVEN_SHA256_SIZE];
    proven_sha256(data, digest);

    char hex[65];
    proven_sha256_to_hex(digest, hex);
    /* The same spelling sha256sum and git print, so it interoperates: */
    EXAMPLE_REQUIRE(hex[64] == '\0' && proven_cstr_len(hex) == 64,
                    "a SHA-256 fingerprint is 64 lowercase hex characters");

    /* SHA-256 streams, for content you cannot hold in memory at once - the digest depends
     * only on the bytes, never on how they were chunked. */
    proven_sha256_t ctx;
```

```
    proven_sha256_init(&ctx);
    proven_sha256_update(&ctx, proven_mem_view_from_u8(PROVEN_LIT("the quick ")));
    proven_sha256_update(&ctx, proven_mem_view_from_u8(PROVEN_LIT("brown fox")));
    proven_byte_t streamed[PROVEN_SHA256_SIZE];
    proven_sha256_final(&ctx, streamed);

    bool same = true;
    for (proven_size_t i = 0; i < PROVEN_SHA256_SIZE; ++i) {
        if (streamed[i] != digest[i]) same = false;
    }
    EXAMPLE_REQUIRE(same, "two updates of the halves equal one hash of the whole");

    return EXAMPLE_OK();
}
```

Legacy digests: SHA-1 and MD5

`proven/hash_legacy.h` holds two digests that are broken and that other people's formats still name. Someone who chooses the input can build two different messages with the same MD5 in seconds, and with the same SHA-1 for a known and affordable cost. Neither may decide whether two things are "the same" when someone gains by fooling you - that stays `proven_sha256`.

They are here for the cases where you do not get to choose:

The format says...	Use	Where you meet it
SHA-1	`proven_sha1`	the WebSocket accept key (RFC 6455); `sha1sum` files; Git object names
MD5	`proven_md5`	HTTP Digest authentication against old servers; `md5sum` files; `Content-MD5`

API	Intent	Return
`proven_sha1(view, out[20])`	One-shot SHA-1.	void; writes `PROVEN_SHA1_SIZE` bytes.
`proven_sha1_init/_update/_final`	The same digest over a stream of chunks.	void.
`proven_sha1_to_hex(digest, out[41])`	The 40-character lowercase spelling `sha1sum` and `git` print. NUL-terminated.	void.
`proven_md5(view, out[16])`	One-shot MD5.	void; writes `PROVEN_MD5_SIZE` bytes.
`proven_md5_init/_update/_final`	The same digest over a stream of chunks.	void.
`proven_md5_to_hex(digest, out[33])`	The 32-character lowercase spelling `md5sum` prints. NUL-terminated.	void.

`proven_sha1_t` and `proven_md5_t` are caller-owned state like `proven_sha256_t`: nothing is allocated and there is nothing to destroy. Both digests are implemented from their specifications (FIPS 180-4, RFC 1321) and checked against the vectors those documents print; the header is available in freestanding builds.

Wrong - a new design that picks a broken digest because it is shorter:

```
proven_byte_t id[PROVEN_MD5_SIZE];
proven_md5(upload, id);              /* wrong: two different uploads can share this id */
store_under(id, upload);
```

Correct - when the choice is yours it is `proven_sha256`; truncate its output if 32 bytes is too long for the place it goes.

Compiled and run by the test suite:

```
/*
 * SHA-1 and MD5. Both are broken for anything an adversary can influence, and both are still
 * written into formats nobody can change - which is the only reason to call them.
 */

int main(void) {
    /* The WebSocket handshake (RFC 6455): the server proves it read the client's key by
     * returning SHA-1(key + a fixed GUID), in Base64. The format says SHA-1, so SHA-1 it is. */
    proven_sha1_t ctx;
    proven_sha1_init(&ctx);
    proven_sha1_update(&ctx, proven_mem_view_from_u8(PROVEN_LIT("dGhlIHNhbXBsZSBub25jZQ==")));
    proven_sha1_update(&ctx, proven_mem_view_from_u8(PROVEN_LIT("258EAFA5-E914-47DA-95CA-C5AB0DC85B11")));
    proven_byte_t sha1[PROVEN_SHA1_SIZE];
    proven_sha1_final(&ctx, sha1);

    proven_byte_t accept[32];
    proven_size_t accept_len = 0;
    proven_err_t err = proven_base64_encode((proven_mem_view_t){ .ptr = sha1, .size = sizeof sha1 },
                                            accept, sizeof accept, &accept_len);
    EXAMPLE_REQUIRE(proven_is_ok(err), "20 bytes encode to 28 Base64 characters");
    EXAMPLE_REQUIRE(proven_u8str_view_eq((proven_u8str_view_t){ .ptr = accept, .size = accept_len },
                                         PROVEN_LIT("s3pPLMBiTxaQ9kYGzzhZRbK+xOo=")),
                    "the accept key printed in RFC 6455 section 1.3");

    /* One-shot, and the hex spelling sha1sum and git print. */
    proven_mem_view_t data = proven_mem_view_from_u8(PROVEN_LIT("abc"));
    proven_sha1(data, sha1);
    char sha1_hex[41];
    proven_sha1_to_hex(sha1, sha1_hex);
    EXAMPLE_REQUIRE(proven_cstr_len(sha1_hex) == 40, "a SHA-1 digest is 40 hex characters");

    /* MD5: checking a download against a checksum file published as md5sum output. That
     * catches a damaged download. It does not catch a forged one - anyone can build a second
     * file with the same MD5. */
    proven_byte_t md5[PROVEN_MD5_SIZE];
    proven_md5(data, md5);
    char md5_hex[33];
    proven_md5_to_hex(md5, md5_hex);
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_view_from_cstr(md5_hex),
                                         PROVEN_LIT("900150983cd24fb0d6963f7d28e17f72")),
                    "the digest md5sum prints for \"abc\"");

    /* MD5 streams too; the digest depends only on the bytes. */
    proven_md5_t mctx;
    proven_md5_init(&mctx);
    proven_md5_update(&mctx, proven_mem_view_from_u8(PROVEN_LIT("a")));
    proven_md5_update(&mctx, proven_mem_view_from_u8(PROVEN_LIT("bc")));
    proven_byte_t streamed[PROVEN_MD5_SIZE];
    proven_md5_final(&mctx, streamed);

    bool same = true;
    for (proven_size_t i = 0; i < PROVEN_MD5_SIZE; ++i) {
```

```
        if (streamed[i] != md5[i]) same = false;
    }
    EXAMPLE_REQUIRE(same, "two updates of the pieces equal one hash of the whole");

    return EXAMPLE_OK();
}
```

Authentication and key derivation: HMAC and HKDF

A digest tells you what the bytes are. It does not tell you who made them: anyone can hash anything, so a message that arrives with its SHA-256 beside it proves nothing about its sender. `hmac.h` has the two constructions that bring a secret key into it.

You need	Use	Because
to know a message came from someone who holds the key - a signed cookie, a webhook, a token	`proven_hmac`	only a holder of the key can compute the MAC, and changing one bit of the message changes it
several keys from one secret - an encryption key and a MAC key from a shared secret	`proven_hkdf`	each key is bound to a label; knowing one says nothing about another
to store or check a **password**	neither	both are fast, and a password is guessable: an attacker tries billions a second. That needs a deliberately slow function (PBKDF2, scrypt, Argon2), which this library does not have

API	Intent	Return
`proven_sha512(view, out[64])`, `proven_sha384(view, out[48])`	One-shot SHA-512 and SHA-384.	void.
`proven_sha512_init/_update/_final`, `proven_sha384_init/_update/_final`	The same over content in pieces. Finish a digest with the family it was begun with.	void.
`proven_hmac(hash, key, data, out)`	One-shot HMAC. `hash` is `PROVEN_HMAC_SHA256`, `_SHA384` or `_SHA512`.	`proven_err_t`: `INVALID_ARG` for an unknown hash or a null pointer with a size.
`proven_hmac_init(&h, hash, key)`, `proven_hmac_update(&h, data)`, `proven_hmac_final(&h, out)`	The same over a message in pieces. `final` wipes the key out of the state.	`init`: `proven_err_t`; the others void.
`proven_hmac_size(hash)`	Bytes in the MAC: 32, 48 or 64.	`proven_size_t`; 0 for an unknown hash.
`proven_hkdf(hash, salt, ikm, info, out)`	Derive `out.size` bytes of key from input key material, bound to `info`.	`proven_err_t`: `OUT_OF_BOUNDS` for more than 255 times the hash size; `INVALID_ARG`. Nothing is written on error.
`proven_hkdf_extract(hash, salt, ikm, prk)`, `proven_hkdf_expand(hash, prk, info, out)`	The two steps apart: extract once, expand for each key.	`proven_err_t`, as above; `INVALID_ARG` for a `prk` shorter than the hash.

A MAC buffer is `PROVEN_HMAC_MAX_SIZE` (64) bytes whatever the hash, and `proven_hmac_size(hash)` of them are written: size the buffer with the constant and compare with the size.

Compare a MAC with `proven_mem_equal_ct`, never with `memcmp`. An ordinary comparison stops at the first byte that differs, and how long it took says how many leading bytes were right. A forger who can submit many guesses and time the answers recovers the MAC one byte at a time - 256 tries a byte instead of 2^256 for the whole. `proven_mem_equal_ct` looks at every byte whatever it finds (Chapter 1).

Wrong:

```
if (memcmp(received_mac, expected_mac, 32) == 0) accept();       /* wrong: leaks how many bytes matched */
```

Correct - `proven_mem_equal_ct((proven_mem_view_t){ received, 32 }, (proven_mem_view_t){ expected, 32 })`.

The key is the whole of it. Take it from `proven_random_bytes`, at least as long as the MAC. A key that is a word, a product name, or shorter than the MAC is the weakest part, and nothing in the construction makes up for it.

HMAC says the message is genuine, not that it is new. A valid signed token is valid every time it is presented. If replay matters - and for anything that causes an action it does - put an expiry or a counter *inside* what is signed, and check it.

`info` is what keeps derived keys apart. Two calls with the same secret and the same `info` give the same key - that is the point, both sides derive it. Two purposes must therefore use two labels, or one key ends up doing two jobs, which is how protocols break.

A truncated SHA-512 is not SHA-384. They start from different values, on purpose: the first 48 bytes of a SHA-512 digest are a different number from the SHA-384 of the same input.

SHA-384, SHA-512, HMAC and HKDF are here because TLS is built from them; they are public because programs need them for their own purposes. They are implemented from FIPS 180-4, RFC 2104 and RFC 5869 and tested against those documents' vectors.

Compiled and run by the test suite:

```
#include <string.h>

/*
 * A digest says what the bytes are. A MAC says who vouched for them, and a KDF turns one
 * secret into the several keys a protocol needs. Plus the two calls that go with any secret:
 * a comparison that does not leak, and a clear that is not optimised away.
 */

static bool hex_is(const proven_byte_t *got, proven_size_t n, const char *want) {
    char text[129];
    proven_size_t len = 0;
    return proven_hex_encode((proven_mem_view_t){ got, n }, (proven_byte_t *)text, sizeof text, &len) ==
PROVEN_OK &&
           len == strlen(want) && memcmp(text, want, len) == 0;
}

int main(void) {
    // ---- SHA-512 and SHA-384 ------------------------------------------------
    /* The same shape as SHA-256: one-shot, or init / update / final for what does not fit in
     * memory. SHA-384 is SHA-512 with other starting values, cut to 48 bytes. */
    proven_byte_t d512[PROVEN_SHA512_SIZE], d384[PROVEN_SHA384_SIZE], again[PROVEN_SHA512_SIZE];
    proven_sha512(proven_mem_view_from_u8(PROVEN_LIT("abc")), d512);
    EXAMPLE_REQUIRE(hex_is(d512, 8, "ddaf35a193617aba"), "SHA-512 of abc begins as FIPS 180-4 prints it");
    proven_sha384(proven_mem_view_from_u8(PROVEN_LIT("abc")), d384);
    EXAMPLE_REQUIRE(hex_is(d384, 8, "cb00753f45a35e8b"), "and SHA-384");

    proven_sha512_t running;
    proven_sha512_init(&running);
```

```
    proven_sha512_update(&running, proven_mem_view_from_u8(PROVEN_LIT("a")));
    proven_sha512_update(&running, proven_mem_view_from_u8(PROVEN_LIT("bc")));
    proven_sha512_final(&running, again);
    EXAMPLE_REQUIRE(memcmp(d512, again, sizeof d512) == 0, "in pieces, the same digest");

    proven_sha384_t running384;
    proven_sha384_init(&running384);
    proven_sha384_update(&running384, proven_mem_view_from_u8(PROVEN_LIT("abc")));
    proven_sha384_final(&running384, again);
    EXAMPLE_REQUIRE(memcmp(d384, again, sizeof d384) == 0, "and for SHA-384");

    // ---- HMAC: a message somebody with the key vouched for --------------------
    /* A server signs a token and later checks that what comes back is what it issued. */
    proven_byte_t key[32];
    EXAMPLE_REQUIRE(proven_random_bytes(key, sizeof key), "a key from the system: 32 bytes for a 32-byte
MAC");
    proven_mem_view_t token = proven_mem_view_from_u8(PROVEN_LIT("user=ada;expires=1767225600"));
    proven_byte_t mac[PROVEN_HMAC_MAX_SIZE];
    EXAMPLE_REQUIRE(proven_hmac(PROVEN_HMAC_SHA256, (proven_mem_view_t){ key, sizeof key }, token, mac) ==
PROVEN_OK, "the token's MAC");
    EXAMPLE_REQUIRE(proven_hmac_size(PROVEN_HMAC_SHA256) == 32, "32 bytes of it; the buffer is sized for the
largest");

    /* Checking: compute it again and compare WITHOUT leaking where the two differ. memcmp
     * stops at the first wrong byte, and the time it took tells a forger how far they got. */
    proven_byte_t check[PROVEN_HMAC_MAX_SIZE];
    proven_hmac_t h;
    EXAMPLE_REQUIRE(proven_hmac_init(&h, PROVEN_HMAC_SHA256, (proven_mem_view_t){ key, sizeof key }) ==
PROVEN_OK, "the streaming form, for a message in pieces");
    proven_hmac_update(&h, proven_mem_view_from_u8(PROVEN_LIT("user=ada;")));
    proven_hmac_update(&h, proven_mem_view_from_u8(PROVEN_LIT("expires=1767225600")));
    proven_hmac_final(&h, check);                   /* also wipes the key out of `h` */
    EXAMPLE_REQUIRE(proven_mem_equal_ct((proven_mem_view_t){ mac, 32 }, (proven_mem_view_t){ check, 32 }),
"the token is genuine");

    /* Someone changes the token. They cannot make the MAC that goes with it. */
    proven_mem_view_t forged = proven_mem_view_from_u8(PROVEN_LIT("user=eve;expires=1767225600"));
    EXAMPLE_REQUIRE(proven_hmac(PROVEN_HMAC_SHA256, (proven_mem_view_t){ key, sizeof key }, forged, check) ==
PROVEN_OK &&
                    !proven_mem_equal_ct((proven_mem_view_t){ mac, 32 }, (proven_mem_view_t){ check, 32 }), "a
changed token does not carry the old MAC");

    /* A published vector, so that this is the HMAC everyone else computes: RFC 4231, case 2. */
    EXAMPLE_REQUIRE(proven_hmac(PROVEN_HMAC_SHA256, proven_mem_view_from_u8(PROVEN_LIT("Jefe")),
                                proven_mem_view_from_u8(PROVEN_LIT("what do ya want for nothing?")), check) ==
PROVEN_OK &&
                    hex_is(check, 32, "5bdcc146bf60754e6a042426089575c75a003f089d2739839dec58b964ec3843"),
"RFC 4231, test case 2");

    // ---- HKDF: one secret, several keys ---------------------------------------
    /* Two sides have agreed on a shared secret. It is not yet a key: derive one for each
     * purpose, each bound to a label, so that knowing one says nothing about another. */
    proven_byte_t shared[32];
    EXAMPLE_REQUIRE(proven_random_bytes(shared, sizeof shared), "stands for the output of a key exchange");
    proven_mem_view_t secret = { shared, sizeof shared };
    proven_mem_view_t salt = proven_mem_view_from_u8(PROVEN_LIT("example-protocol v1"));       /* not secret;
may be empty */

    proven_byte_t enc_key[32], mac_key[32];
    EXAMPLE_REQUIRE(proven_hkdf(PROVEN_HMAC_SHA256, salt, secret,
```

```
proven_mem_view_from_u8(PROVEN_LIT("encryption")), (proven_mem_mut_t){ enc_key, sizeof enc_key }) == PROVEN_OK
&&
                    proven_hkdf(PROVEN_HMAC_SHA256, salt, secret,
proven_mem_view_from_u8(PROVEN_LIT("authentication")), (proven_mem_mut_t){ mac_key, sizeof mac_key }) ==
PROVEN_OK,
                    "two keys from one secret");
    EXAMPLE_REQUIRE(!proven_mem_equal_ct((proven_mem_view_t){ enc_key, 32 }, (proven_mem_view_t){ mac_key,
32 }), "different labels, unrelated keys");

    /* The two steps apart: extract once, expand as often as needed. */
    proven_byte_t prk[PROVEN_HMAC_MAX_SIZE], enc_again[32];
    EXAMPLE_REQUIRE(proven_hkdf_extract(PROVEN_HMAC_SHA256, salt, secret, prk) == PROVEN_OK &&
                    proven_hkdf_expand(PROVEN_HMAC_SHA256, (proven_mem_view_t){ prk, 32 },
proven_mem_view_from_u8(PROVEN_LIT("encryption")),
                                       (proven_mem_mut_t){ enc_again, sizeof enc_again }) == PROVEN_OK &&
                    proven_mem_equal_ct((proven_mem_view_t){ enc_key, 32 }, (proven_mem_view_t){ enc_again,
32 }), "extract then expand is the same derivation");

    /* HKDF gives at most 255 blocks of the hash; asking for more is refused, not truncated. */
    static proven_byte_t too_much[255 * 32 + 1];
    EXAMPLE_REQUIRE(proven_hkdf_expand(PROVEN_HMAC_SHA256, (proven_mem_view_t){ prk, 32 },
proven_mem_view_from_u8(PROVEN_LIT("")),
                                       (proven_mem_mut_t){ too_much, sizeof too_much }) ==
PROVEN_ERR_OUT_OF_BOUNDS, "more than 8160 bytes from SHA-256: refused");

    // ---- When a secret is done with -----------------------------------------
    /* Clear it. An ordinary loop here would be deleted by the optimiser - nothing reads the
     * buffer afterwards - and the key would stay in memory. This write is not removed. */
    proven_mem_wipe((proven_mem_mut_t){ key, sizeof key });
    proven_mem_wipe((proven_mem_mut_t){ shared, sizeof shared });
    proven_mem_wipe((proven_mem_mut_t){ prk, sizeof prk });
    proven_mem_wipe((proven_mem_mut_t){ enc_key, sizeof enc_key });
    proven_mem_wipe((proven_mem_mut_t){ mac_key, sizeof mac_key });
    proven_byte_t zeros[32] = {0};
    EXAMPLE_REQUIRE(proven_mem_equal_ct((proven_mem_view_t){ key, 32 }, (proven_mem_view_t){ zeros, 32 }),
"the key is gone from its buffer");

    return EXAMPLE_OK();
}
```

7. Bytes to text: hex and Base64

Once you can hash a thing (above) and draw a random token (`random.h`), you need to write those bytes somewhere that only holds text - a URL, an HTTP header, a log line, a JSON string. That is `encode.h`. No cryptography, no compression; the two encodings everything already agrees on, done without hidden allocation and without the two ways they are usually got wrong.

You want	Use	Alphabet
A digest or a few bytes a human reads	`proven_hex_encode`	lowercase hex, what `sha256sum` and `git` print
Bytes in a URL, a cookie, a filename	`proven_base64url_encode`	`-` `_`, **no** padding - nothing to escape, no `=` to mangle
Bytes in an HTTP header, MIME, JSON	`proven_base64_encode`	standard `+` `/`, `=`-padded

Two refusals are the point:

- **A decoder validates its whole input before writing a byte.** Text from outside the program is not guaranteed to be valid; a stray character, a bad length, bad padding, or embedded whitespace is `PROVEN_ERR_INVALID_ENCODING` with nothing committed - not a read past the end, and not a silently short result one byte into which the caller finds the corruption. `proven_base64_decode` accepts **both** alphabets and padded-or-not, because a decoder that only takes what it emits rejects half the Base64 in the world.
- **The output size is a call, not a guess** - `proven_hex_encoded_size`, `proven_base64_encoded_size`, and their decode counterparts. A buffer one byte too small is `PROVEN_ERR_OUT_OF_BOUNDS` with nothing written, never a truncated prefix.

It is pure computation - no allocation, no OS - and available freestanding.

Reference
Every call takes the input, a caller-owned output buffer and its capacity, and an optional `written_out`. None of them allocate.

API	Intent	Return
`proven_hex_encoded_size(n)`	The characters `proven_hex_encode` will write for `n` bytes: `n * 2`, no NUL.	`proven_size_t`; `PROVEN_SIZE_MAX` when `n * 2` does not fit.
`proven_hex_decoded_size(n)`	The bytes `proven_hex_decode` will write for `n` characters: `n / 2`.	`proven_size_t`.
`proven_hex_encode(data, out, cap, &w)`	Lowercase hex.	`PROVEN_OK`; `OUT_OF_BOUNDS` if `cap` is short (**nothing written**); `OVERFLOW` if the output size does not fit at all; `INVALID_ARG` for a NULL out or a `{NULL, >0}` view.
`proven_hex_decode(text, out, cap, &w)`	Decode hex; upper and lower case both accepted.	`INVALID_ENCODING` for an odd length or any non-hex byte (**nothing committed**).
`proven_base64_encoded_size(n)`	An upper bound for both Base64 forms: `4 * ceil(n/3)`.	`proven_size_t`; `PROVEN_SIZE_MAX` when that does not fit.
`proven_base64_decoded_size(n)`	An upper bound for **padded and unpadded** text: `3 * ceil(n/4)`.	`proven_size_t`.
`proven_base64_encode(data, out, cap, &w)`	Standard alphabet (`+ /`), `=`-padded.	as above.
`proven_base64url_encode(data, out, cap, &w)`	URL-safe alphabet (`- _`), **no padding**.	as above.
`proven_base64_decode(text, out, cap, &w)`	Decode. Accepts **both** alphabets, padded or not.	`INVALID_ENCODING` for a stray byte, a bad length, or bad padding.

`written_out` may be NULL. It is set to `0` on entry and on every error path, so it is never stale.

Cautions, and what goes wrong
Size the output with the size function, not by eye. The encoders refuse a short buffer rather than truncating - which means the failure you get from a hand-computed size is an error you have to handle, not a silent corruption. That is the good outcome; the point is you will still have to handle it.

Wrong - the classic off-by-one, and it now *fails* instead of overflowing:

```
proven_byte_t out[16];                      /* wrong: 12 bytes of hex needs 24 chars */
proven_hex_encode(twelve_bytes, out, sizeof out, &w);   /* OUT_OF_BOUNDS, nothing written */
```

Correct:

```
proven_mem_view_t data = proven_mem_view_from_u8(PROVEN_LIT("twelve bytes"));
proven_byte_t out[64];
proven_size_t w = 0;
if (proven_hex_encoded_size(data.size) <= sizeof out &&
    proven_is_ok(proven_hex_encode(data, out, sizeof out, &w))) {
    /* `w` characters of lowercase hex in `out` - not NUL-terminated */
}
```

Base64URL emits no padding, and the decoder accepts that. `proven_base64_decoded_size` rounds *up* so it is a correct upper bound for unpadded text too. Do not "improve" on it with `3 * (n / 4)` - that floors away the 1-2 bytes an unpadded tail carries, and you will fail to decode this library's own URL-safe output. (It did exactly that, once, and an audit caught it.)

```
proven_size_t cap = 3 * (text_len / 4);   /* wrong: 0 for "QQ", which decodes to 1 byte */
```

Never hand-roll the decoder. The whole reason these exist is that a decode reads text from outside your program, and the two-line loop everyone writes trusts it.

Wrong - reads past the end on odd input, and accepts junk as data:

```
for (size_t i = 0; i < len; i += 2)              /* wrong: no length check */
    out[i/2] = (hexval(text[i]) << 4) | hexval(text[i+1]);   /* wrong: no validation */
```

`proven_hex_decode` validates the **whole** input before writing a single byte, so a stray character near the end cannot leave you holding a half-decoded prefix you believe is complete.

Whitespace is not skipped, on purpose. A pasted, line-wrapped Base64 blob is `INVALID_ENCODING`, not a silently different result. If you *want* to accept wrapped input, strip the whitespace yourself - deliberately, where you can see it.

Size the destination with the size functions, never by hand. Each encoding has a pair - `proven_hex_encoded_size()` / `proven_hex_decoded_size()` and `proven_base64_encoded_size()` / `proven_base64_decoded_size()` - and they differ in what they promise:

Direction	What the number is	How to use it
Encoding	**Exact** for hex and standard Base64. Base64URL omits padding, so its output can be shorter - the number is still a safe upper bound.	Allocate this many bytes; the call reports what it wrote.
Decoding	An **upper bound**, because padding and the alphabet in use are not known until the text is read.	Allocate the bound, then use the reported count, not the bound, as the length.

The example below ends with that: it allocates a destination sized by `proven_base64_encoded_size()`, encodes into it, sizes the reverse buffer with `proven_base64_decoded_size()`, decodes, and round-trips the same bytes through `proven_hex_decode()` - including the uppercase input real tools produce, and the odd-length input that cannot be a whole number of bytes.

Compiled and run by the test suite:

```
/*
 * Bytes to text, by use case. The rule is the same one hashing follows: one function per job,
 * and the danger is picking the wrong job. Hex for something a human reads; Base64URL for
 * something that goes in a URL; standard Base64 for something that goes on the wire.
 */
```

```
int main(void) {
    proven_mem_view_t data = proven_mem_view_from_u8(PROVEN_LIT("the quick brown fox"));

    /* Job 1: a digest a human will read or paste - hex, the spelling sha256sum and git use. */
    proven_byte_t hex[64];   /* proven_hex_encoded_size(19) = 38 */
    proven_size_t hn = 0;
    EXAMPLE_REQUIRE(proven_is_ok(proven_hex_encode(data, hex, sizeof hex, &hn)),
                    "hex encode into a buffer sized by proven_hex_encoded_size");
    EXAMPLE_REQUIRE(hn == proven_hex_encoded_size(data.size), "two hex chars per byte");

    /* Job 2: a token that goes in a URL - Base64URL, so nothing needs percent-escaping and
     * there is no '=' padding for a parser to trip over. */
    proven_byte_t token_bytes[16] = { 0 };   /* in real code: proven_random_bytes(token_bytes, 16) */
    proven_byte_t url[32];
    proven_size_t un = 0;
    EXAMPLE_REQUIRE(proven_is_ok(proven_base64url_encode(
                        (proven_mem_view_t){ token_bytes, sizeof token_bytes }, url, sizeof url, &un)),
                    "base64url encode a token");
    /* No '=' in a URL-safe token. */
    bool has_pad = false;
    for (proven_size_t i = 0; i < un; ++i) if (url[i] == '=') has_pad = true;
    EXAMPLE_REQUIRE(!has_pad, "the URL form emits no padding");

    /* Job 3: bytes on the wire - standard Base64, the +/= alphabet HTTP and MIME expect. */
    proven_byte_t b64[64];
    proven_size_t bn = 0;
    EXAMPLE_REQUIRE(proven_is_ok(proven_base64_encode(data, b64, sizeof b64, &bn)),
                    "standard base64 encode");

    /* And it round-trips: decode gives back exactly the bytes. A decoder that accepts both
     * alphabets and padded-or-not is deliberate - real input comes in every shape. */
    proven_byte_t back[32];
    proven_size_t dn = 0;
    EXAMPLE_REQUIRE(proven_is_ok(proven_base64_decode(
                        (proven_mem_view_t){ b64, bn }, back, sizeof back, &dn)),
                    "decode the base64 back");
    EXAMPLE_REQUIRE(dn == data.size && proven_memcmp(back, data.ptr, dn) == 0,
                    "what comes back is exactly what went in");

    /* The point of a validating decoder: junk is refused, not guessed. A caller who fed this
     * to a two-line loop would read past the end or get a silently short result. */
    proven_err_t bad = proven_base64_decode(
        proven_mem_view_from_u8(PROVEN_LIT("not valid base64!!")), back, sizeof back, &dn);
    EXAMPLE_REQUIRE(bad == PROVEN_ERR_INVALID_ENCODING,
                    "a stray character is INVALID_ENCODING, with nothing committed");

    /* And a buffer one byte too small is refused, never truncated. */
    proven_byte_t tiny[4];
    EXAMPLE_REQUIRE(proven_hex_encode(data, tiny, sizeof tiny, &hn) == PROVEN_ERR_OUT_OF_BOUNDS,
                    "a too-small output buffer is OUT_OF_BOUNDS, not a truncated prefix");

    /* --- sizing the destination buffer, instead of guessing ---------------- */

    /* Every encode and decode above wrote into a fixed array that was obviously
     * big enough. Real code allocates the destination, and then the size has to
     * be computed rather than remembered. That is what the four size functions
     * are for - one per direction, per encoding. */
    proven_allocator_t alloc = proven_heap_allocator();

    proven_size_t need = proven_base64_encoded_size(data.size);
```

```
    EXAMPLE_REQUIRE(need > 0, "an encoded size must be computed, not guessed");

    proven_result_mem_mut_t out = alloc.alloc_fn(alloc.ctx, need, 1);
    EXAMPLE_REQUIRE(proven_is_ok(out.err), "allocating exactly the encoded size must succeed");

    proven_size_t wrote = 0;
    EXAMPLE_REQUIRE(proven_is_ok(proven_base64_encode(data, out.value.ptr, out.value.size, &wrote)),
                    "encoding into a buffer sized by the size function must fit exactly");
    EXAMPLE_REQUIRE(wrote == need, "and use all of it: this size is exact for standard base64");

    /* The decode direction is an UPPER BOUND, not an exact count: base64 text
     * may carry padding, so the decoder reports how many bytes it really wrote.
     * Size the buffer with the bound; use the reported count afterwards. */
    proven_size_t bound = proven_base64_decoded_size(wrote);
    EXAMPLE_REQUIRE(bound >= data.size, "the decoded bound must cover the real output");

    proven_result_mem_mut_t plain = alloc.alloc_fn(alloc.ctx, bound, 1);
    EXAMPLE_REQUIRE(proven_is_ok(plain.err), "allocating the decode bound must succeed");

    proven_size_t got = 0;
    EXAMPLE_REQUIRE(proven_is_ok(proven_base64_decode(
                        (proven_mem_view_t){ out.value.ptr, wrote }, plain.value.ptr, plain.value.size,
&got)),
                    "decoding back must succeed");
    EXAMPLE_REQUIRE(got == data.size && proven_memcmp(plain.value.ptr, data.ptr, got) == 0,
                    "and reproduce the original bytes exactly");

    /* Hex has the same pair. Its decoded bound is exact when the input length is
     * even, which valid hex always is - an odd length is malformed input, and
     * the decoder says so rather than dropping the last digit. */
    proven_size_t hex_len = 0;
    EXAMPLE_REQUIRE(proven_is_ok(proven_hex_encode(data, hex, sizeof hex, &hex_len)),
                    "re-encode to hex, since the refused call above left its count unusable");

    proven_size_t hex_bound = proven_hex_decoded_size(hex_len);
    EXAMPLE_REQUIRE(hex_bound == data.size, "two hex characters decode to one byte");

    proven_byte_t from_hex[32];
    proven_size_t hex_got = 0;
    EXAMPLE_REQUIRE(proven_is_ok(proven_hex_decode(
                        (proven_mem_view_t){ hex, hex_len }, from_hex, sizeof from_hex, &hex_got)),
                    "hex decodes back to the original bytes");
    EXAMPLE_REQUIRE(hex_got == data.size && proven_memcmp(from_hex, data.ptr, hex_got) == 0,
                    "and the round trip is exact");

    /* Hex is case-insensitive on the way in, which matters because different
     * tools print different cases of the same digest. */
    EXAMPLE_REQUIRE(proven_is_ok(proven_hex_decode(
                        proven_mem_view_from_u8(PROVEN_LIT("DEADBEEF")), from_hex, sizeof from_hex,
&hex_got)),
                    "uppercase hex decodes too");
    EXAMPLE_REQUIRE(hex_got == 4, "and yields one byte per two characters");

    /* An odd number of characters cannot be a whole number of bytes. */
    EXAMPLE_REQUIRE(proven_hex_decode(proven_mem_view_from_u8(PROVEN_LIT("abc")),
                                      from_hex, sizeof from_hex, &hex_got) == PROVEN_ERR_INVALID_ENCODING,
                    "an odd-length hex string is malformed, not silently truncated");

    alloc.free_fn(alloc.ctx, plain.value.ptr);
    alloc.free_fn(alloc.ctx, out.value.ptr);
```

```
    (void)url; (void)un;
    return EXAMPLE_OK();
}
```

8. Examples and misuse cases

Pointers into arrays can become stale

Wrong:

```
int *p = PROVEN_ARRAY_GET_MUT(&nums, int, 0);
PROVEN_ARRAY_PUSH(&nums, int, 30);
*p = 99; /* wrong: push may have reallocated the array */
```

Correct:

```
proven_result_array_t r = PROVEN_ARRAY_INIT(alloc, int, 2);
if (!proven_is_ok(r.err)) {
    return;
}
proven_array_t nums = r.value;
(void)PROVEN_ARRAY_PUSH(&nums, int, 10);

/* Push first, then take the pointer. A pointer fetched before a push may point
 * at a block the array has already reallocated away. */
(void)PROVEN_ARRAY_PUSH(&nums, int, 30);
int *p = PROVEN_ARRAY_GET_MUT(&nums, int, 0);
if (p) {
    *p = 99;
}

PROVEN_ARRAY_DESTROY(&nums);
```

One intrusive node belongs to one list at a time

Wrong:

```
proven_list_push_back(&list_a, &item.link);
proven_list_push_back(&list_b, &item.link); /* wrong */
```

Use one embedded node per membership.

Remove while iterating with the safe macro

Correct:

```
typedef struct Item {
    int value;
    proven_list_node_t link;
} Item;

proven_list_t list;
proven_list_init(&list);
Item a = { .value = 1 };
Item b = { .value = 2 };
proven_list_push_back(&list, &a.link);
proven_list_push_back(&list, &b.link);

/* The SAFE form reads `it->next` into `next` before the body runs, so removing
 * `it` (which nulls its links) cannot strand the loop. */
proven_list_node_t *it = NULL;
proven_list_node_t *next = NULL;
PROVEN_LIST_FOR_EACH_SAFE(it, next, &list) {
    Item *item = PROVEN_LIST_ENTRY(it, Item, link);
    if (item->value % 2 == 0) {
```

```
        proven_list_remove(it);
    }
}
```

Ring push does not grow

Wrong:

```
PROVEN_RING_PUSH(&q, int, value); /* wrong if you ignore full-ring errors */
```

Correct:

```
proven_result_ring_t r = PROVEN_RING_INIT(alloc, int, 2);
if (!proven_is_ok(r.err)) {
    return;
}
proven_ring_t q = r.value;

int value = 7;
for (int i = 0; i < 3; ++i) {
    proven_err_t e = PROVEN_RING_PUSH(&q, int, value);
    if (e == PROVEN_ERR_OUT_OF_BOUNDS) {
        /* The ring is full. Drop the item, or drain one first - but decide. */
        int drop = 0;
        (void)PROVEN_RING_POP(&q, int, &drop);
        e = PROVEN_RING_PUSH(&q, int, value);
    }
    (void)e;
}

PROVEN_RING_DESTROY(&q);
```

Borrowed map keys must outlive entries

Wrong:

```
proven_u8str_t key = make_key(alloc);
PROVEN_MAP_SET_U8_BORROWED(&m, proven_u8str_as_view(&key), int, 1);
proven_u8str_destroy(alloc, &key);
/* wrong: map still points at freed key bytes */
```

Correct options:

- Use string literals or other stable storage for borrowed keys.
- Store owned key objects elsewhere and destroy them after removing map entries.
- Use `PROVEN_KEY_TYPE_U8_OWNED` and `proven_map_set_u8_owned()` when the map should own key storage.

Use scratch when inserting a value borrowed from the same map

If `element` points into map storage and insertion can rehash, use `proven_map_set_with_scratch()` or the scratch macros.

```
proven_result_map_t r = PROVEN_MAP_INIT_INT(alloc, int, 4);
if (!proven_is_ok(r.err)) {
    return;
}
proven_map_t m = r.value;
(void)PROVEN_MAP_SET_INT(&m, 1, int, 42);

/* `src` points into the map's own bucket array. Passing it straight to
 * proven_map_set would be an alias: a rehash triggered by that same insert can
 * free the bytes it is still reading from. The scratch form captures the source
 * bytes into a temporary from `scratch` first, so the insert is alias-safe. */
const int *src = PROVEN_MAP_GET_INT(&m, int, 1);
if (src) {
    (void)proven_map_set_with_scratch(&m, (proven_map_key_t){ .id = 2 }, src, scratch);
```

```
}

PROVEN_MAP_DESTROY(&m);
```

Binary search requires sorted input

Wrong:

```
int *hit = proven_array_binary_search(&nums, &key, cmp_int);
/* wrong if nums was not sorted by cmp_int */
```

Worked example: array, sort, binary search

Compiled and run by the test suite. Note the sort's behaviour on duplicate keys: they are the *fast* case, not the quadratic one.

```
/*
 * proven_array_t is a growable vector of one element type. It stores the
 * allocator you created it with, so push can grow and destroy can free without
 * you passing the allocator back in - which also means the array must be
 * destroyed with nothing but itself.
 *
 * The PROVEN_ARRAY_* macros are the type-safe face of the void* core: they
 * derive sizeof/alignof from the type and hand the element to the array by
 * pointer, so pushing an int into an array of records will not compile.
 */

/* A task queue keyed by priority. Real data is low-cardinality: three
 * priorities, many tasks. That matters for the sort - see below. */
typedef struct {
    int priority;
    int id;
} ex_task_t;

/* The comparator is the array's ordering. The same one MUST be used for the
 * sort and for the binary search - a search under a different order is a bug
 * the compiler cannot see.
 *
 * The (x > y) - (x < y) form is deliberate: subtracting the two ints would
 * overflow for large values and hand back a nonsense sign. */
static int cmp_priority(const void *a, const void *b) {
    int x = ((const ex_task_t *)a)->priority;
    int y = ((const ex_task_t *)b)->priority;
    return (x > y) - (x < y);
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- push / get / pop ------------------------------------------------- */
    /* init_cap is a hint, not a limit: push grows past it. Sizing it right just
     * avoids the reallocations. */
    proven_result_array_t r = PROVEN_ARRAY_INIT(alloc, ex_task_t, 4);
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "creating an array of ex_task_t must succeed");
    if (!proven_is_ok(r.err)) {
        return 1;
    }
    proven_array_t tasks = r.value;

    /* Deliberately duplicate-heavy, because that is what real keys look like:
     * a status column, an enum, a bucket id. */
    static const ex_task_t seed[] = {
        { .priority = 2, .id = 10 }, { .priority = 0, .id = 11 },
```

```
    { .priority = 2, .id = 12 }, { .priority = 1, .id = 13 },
    { .priority = 2, .id = 14 }, { .priority = 0, .id = 15 },
    { .priority = 1, .id = 16 }, { .priority = 2, .id = 17 },
};

for (proven_size_t i = 0; i < sizeof seed / sizeof seed[0]; ++i) {
    proven_err_t err = PROVEN_ARRAY_PUSH(&tasks, ex_task_t, seed[i]);
    EXAMPLE_REQUIRE(proven_is_ok(err), "pushing into a growable array must succeed");
    if (!proven_is_ok(err)) {
        PROVEN_ARRAY_DESTROY(&tasks);
        return 1;
    }
}
EXAMPLE_REQUIRE(tasks.len == 8, "eight pushes give eight elements");

/* get returns a pointer INTO the array's storage - it is not a copy, and the
 * next push may reallocate and leave it dangling. Fetch it after the pushes,
 * use it, do not store it. */
const ex_task_t *front = PROVEN_ARRAY_GET(&tasks, ex_task_t, 0);
EXAMPLE_REQUIRE(front && front->id == 10, "element 0 is the first one pushed");

/* Out of range is a null pointer, not a crash and not UB. */
EXAMPLE_REQUIRE(PROVEN_ARRAY_GET(&tasks, ex_task_t, 99) == NULL, "an out-of-range index yields NULL");

/* pop copies the last element out (pass NULL to just discard it). */
ex_task_t last = {0};
proven_err_t err = PROVEN_ARRAY_POP(&tasks, ex_task_t, &last);
EXAMPLE_REQUIRE(proven_is_ok(err), "popping a non-empty array must succeed");
EXAMPLE_REQUIRE(last.id == 17 && tasks.len == 7, "pop returns the last element and shrinks the array");

/* Put it back: the rest of the example wants all eight. */
err = PROVEN_ARRAY_PUSH(&tasks, ex_task_t, last);
EXAMPLE_REQUIRE(proven_is_ok(err), "pushing the popped element back must succeed");

/* --- sort ---------------------------------------------------------------- */
/* An introsort: O(n log n) is a guarantee, not an average - the heapsort
 * fallback rules out the quadratic case an adversary could otherwise steer
 * you into. And equal keys are the FAST path here: everything equal to the
 * pivot is partitioned into a run that is never recursed into, so the
 * duplicate priorities above cost less than distinct ones would, not more.
 *
 * It is not stable: task 10 and task 12 may come out in either order. */
proven_array_sort(&tasks, cmp_priority);

for (proven_size_t i = 1; i < tasks.len; ++i) {
    const ex_task_t *prev = PROVEN_ARRAY_GET(&tasks, ex_task_t, i - 1);
    const ex_task_t *cur  = PROVEN_ARRAY_GET(&tasks, ex_task_t, i);
    EXAMPLE_REQUIRE(prev && cur && prev->priority <= cur->priority,
                    "after sorting, priorities must be non-decreasing");
}

/* --- binary search ------------------------------------------------------- */
/* Only legal because the array is sorted by this exact comparator. The key
 * is a whole element, but only the fields the comparator reads matter. */
ex_task_t key = { .priority = 1, .id = 0 };
const ex_task_t *hit = (const ex_task_t *)proven_array_binary_search(&tasks, &key, cmp_priority);
EXAMPLE_REQUIRE(hit != NULL, "priority 1 is present, so the search must find it");
EXAMPLE_REQUIRE(hit && hit->priority == 1, "the hit must be an element with the searched key");
/* With duplicate keys it finds SOME element with that key, not the first one.
 * If you need the first, scan backwards from the hit. */
```

```
    ex_task_t absent = { .priority = 9, .id = 0 };
    EXAMPLE_REQUIRE(proven_array_binary_search(&tasks, &absent, cmp_priority) == NULL,
                    "a key that is not there must return NULL");

    printf("tasks: %zu sorted, found priority %d (id %d)\n",
           (size_t)tasks.len, hit ? hit->priority : -1, hit ? hit->id : -1);

    /* The array owns its element storage; destroy returns it through the
     * allocator the array was created with. Elements are plain data here - if
     * they owned anything, you would have to destroy each one first. */
    PROVEN_ARRAY_DESTROY(&tasks);
    return EXAMPLE_OK();
}
```

Worked example: a map with integer keys and with owned string keys
Compiled and run by the test suite. The owned-key half proves the point that is easy to get wrong: the map copies the key, so the buffer you built it in is yours to reuse immediately.

```
/*
 * proven_map_t is a flat open-addressing hash map. The value type is fixed at
 * create time and stored inline in the bucket array - there is no per-entry
 * allocation for values, and get hands back a pointer straight into that array.
 *
 * The interesting decision is the KEY:
 *
 *   PROVEN_KEY_TYPE_INT          - the key is a proven_size_t. Nothing to own.
 *   PROVEN_KEY_TYPE_U8_BORROWED  - the bucket stores your pointer and length.
 *                                  The map never copies the bytes, so YOU must
 *                                  keep them alive and unmoved for as long as
 *                                  the entry exists. Right for string literals.
 *   PROVEN_KEY_TYPE_U8_OWNED     - the map copies the key bytes into its own
 *                                  storage and frees them again. Right for keys
 *                                  built at runtime, which is most of them.
 *
 * The second half of this example is the reason OWNED exists.
 */

typedef struct {
    int  level;
    long score;
} ex_player_t;

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- integer keys ------------------------------------------------------ */
    proven_result_map_t r = PROVEN_MAP_INIT_INT(alloc, ex_player_t, 8);
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "creating an int-keyed map must succeed");
    if (!proven_is_ok(r.err)) {
        return 1;
    }
    proven_map_t by_id = r.value;

    proven_err_t err = PROVEN_MAP_SET_INT(&by_id, 404, ex_player_t, ((ex_player_t){ .level = 3, .score =
990 }));
    EXAMPLE_REQUIRE(proven_is_ok(err), "inserting into the map must succeed");
    err = PROVEN_MAP_SET_INT(&by_id, 7, ex_player_t, ((ex_player_t){ .level = 1, .score = 10 }));
    EXAMPLE_REQUIRE(proven_is_ok(err), "inserting a second key must succeed");
```

```c
/* set on an existing key replaces the value; it does not add an entry. */
err = PROVEN_MAP_SET_INT(&by_id, 7, ex_player_t, ((ex_player_t){ .level = 2, .score = 40 }));
EXAMPLE_REQUIRE(proven_is_ok(err), "re-setting an existing key must succeed");
EXAMPLE_REQUIRE(by_id.len == 2, "re-setting a key replaces its value rather than adding an entry");

/* get returns a pointer into the bucket array, or NULL when absent. It is
 * invalidated by any insert that rehashes - look it up, use it, drop it. */
const ex_player_t *p = PROVEN_MAP_GET_INT(&by_id, ex_player_t, 7);
EXAMPLE_REQUIRE(p && p->level == 2 && p->score == 40, "get must see the replaced value");
EXAMPLE_REQUIRE(PROVEN_MAP_GET_INT(&by_id, ex_player_t, 999) == NULL, "a missing key yields NULL");

err = PROVEN_MAP_REMOVE_INT(&by_id, 7);
EXAMPLE_REQUIRE(proven_is_ok(err), "removing a present key must succeed");
EXAMPLE_REQUIRE(PROVEN_MAP_GET_INT(&by_id, ex_player_t, 7) == NULL, "a removed key is gone");
EXAMPLE_REQUIRE(by_id.len == 1, "removal decrements the live entry count");

PROVEN_MAP_DESTROY(&by_id);

/* --- owned string keys ------------------------------------------------- */
/* Same map, keyed by a name that we build at runtime - the case where a
 * borrowed key would be a dangling pointer waiting to happen. */
proven_result_map_t rm = PROVEN_MAP_INIT_U8_OWNED(alloc, ex_player_t, 8);
EXAMPLE_REQUIRE(proven_is_ok(rm.err), "creating an owned-string-keyed map must succeed");
if (!proven_is_ok(rm.err)) {
    return 1;
}
proven_map_t by_name = rm.value;

/* A scratch buffer we intend to reuse for every key. With a BORROWED map
 * that plan is fatal: every entry would point at these same bytes. */
proven_byte_t scratch[32];
proven_u8str_t name = proven_u8str_borrow(scratch, sizeof scratch);

err = proven_u8str_append(&name, PROVEN_LIT("ada"));
EXAMPLE_REQUIRE(proven_is_ok(err), "building the first key must succeed");

/* set_u8_owned COPIES the key bytes into map storage. After it returns, the
 * map's key no longer has anything to do with `scratch`. */
err = PROVEN_MAP_SET_U8_OWNED(&by_name, proven_u8str_as_view(&name), ex_player_t,
                              ((ex_player_t){ .level = 9, .score = 5000 }));
EXAMPLE_REQUIRE(proven_is_ok(err), "inserting with an owned key must succeed");

/* So the buffer is immediately free to be reused for the next key... */
err = proven_u8str_reset(&name);
EXAMPLE_REQUIRE(proven_is_ok(err), "the key buffer is ours again the moment set returns");
err = proven_u8str_append(&name, PROVEN_LIT("grace"));
EXAMPLE_REQUIRE(proven_is_ok(err), "overwriting the buffer with the next key must succeed");

err = PROVEN_MAP_SET_U8_OWNED(&by_name, proven_u8str_as_view(&name), ex_player_t,
                              ((ex_player_t){ .level = 4, .score = 700 }));
EXAMPLE_REQUIRE(proven_is_ok(err), "inserting the second owned key must succeed");

/* ...and the first entry is untouched by that overwrite. This is the whole
 * point: the map holds its own copy of "ada", not a view of a buffer that
 * now says "grace". A BORROWED map would report two entries both keyed
 * "grace" - or worse, one keyed by freed memory. */
const ex_player_t *ada = PROVEN_MAP_GET_U8_OWNED(&by_name, ex_player_t, PROVEN_LIT("ada"));
EXAMPLE_REQUIRE(ada && ada->score == 5000, "the copied key survives the caller reusing its buffer");

const ex_player_t *grace = PROVEN_MAP_GET_U8_OWNED(&by_name, ex_player_t, PROVEN_LIT("grace"));
```

```
    EXAMPLE_REQUIRE(grace && grace->score == 700, "the second key is a separate entry");
    EXAMPLE_REQUIRE(by_name.len == 2, "two distinct keys means two entries");

    /* Remove frees the key copy the map made - you never free it yourself. */
    err = PROVEN_MAP_REMOVE_U8_OWNED(&by_name, PROVEN_LIT("ada"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "removing an owned key must succeed");
    EXAMPLE_REQUIRE(PROVEN_MAP_GET_U8_OWNED(&by_name, ex_player_t, PROVEN_LIT("ada")) == NULL,
                    "the removed entry is gone");

    printf("map: %zu name(s) left, grace at level %d\n",
           (size_t)by_name.len, grace ? grace->level : -1);

    /* destroy frees the bucket array AND every key copy still in it ("grace"
     * here). `scratch` is ours and outlives the map; the borrowed `name` handle
     * has nothing to free. */
    PROVEN_MAP_DESTROY(&by_name);
    proven_u8str_destroy(alloc, &name);
    return EXAMPLE_OK();
}
```

Worked example: sizing containers, searching unsorted data, and streaming a checksum
The worked examples above take one container at a time. This one is the program those pieces end up inside - a small event intake - and it answers the questions that only come up once several containers are in play:

- **Stop a container reallocating while it fills.** `proven_array_reserve()` and `proven_map_reserve()` set the capacity once. On the heap that saves copying; behind an arena it saves the dead storage every reallocation leaves behind until the next reset. For a map it also matters for correctness of your pointers: a rehash invalidates every pointer a previous `get_mut` returned.
- **Check a handle you did not create.** `proven_array_is_valid()`, `proven_ring_is_valid()` and `proven_map_is_valid()` ask whether the handle's own fields are consistent - the check that catches a zero-initialised or never-created container at the boundary of your code, rather than at the first push.
- **Search data that is not sorted.** The event log is kept in arrival order, because that order is the thing being recorded, so binary search may not be used on it - on unsorted input it does not return "not found", it returns a wrong answer. `proven_array_linear_search()` is the correct call, and it returns the *first* match in order.
- **Update a value already in a map, with one lookup.** `proven_map_get_mut()` returns a pointer into the map's own storage, so a counter is incremented where it lives instead of being read, changed and written back.
- **Decide which hash the keys deserve.** `proven_map_create()` hashes string keys with keyed SipHash, so an attacker who chooses the keys cannot collide them all into one bucket. `proven_map_create_trusted()` uses fast FNV-1a and is right only when your own code chooses every key. `proven_map_hash()` returns the value the map actually places a key by, so the difference between the two is observable rather than a claim.
- **Checksum data that arrives in pieces.** `proven_crc32_update()` folds each chunk into a running value; the result equals `proven_crc32()` over the whole, whatever the chunk boundaries were.

```
/*
 * The container chapters show each structure on its own. A real program uses
 * several at once, and the questions that come up then are not about any single
 * container:
 *
 *   - How do I stop a container reallocating while it fills?  reserve.
 *   - How do I check a container somebody handed me is usable?  is_valid.
 *   - How do I search when the data is NOT sorted?  linear search - and knowing
```

```
 *     why binary search would give a wrong answer here.
 *   - How do I update a value already in a map without looking it up twice?
 *     get_mut.
 *   - Do I need the attack-resistant hash, or the fast one?  it depends on who
 *     chooses the keys, and map_hash lets you see the difference.
 *   - How do I checksum data that arrives in pieces?  crc32_update.
 *
 * The program is a small event intake: events arrive, are counted per client,
 * the most recent few are kept for a diagnostic dump, and the batch is
 * checksummed as it streams past.
 */

typedef struct {
    proven_u32 client;
    proven_u32 code;      /* an event code; 0 means "connection closed" */
} ex_event_t;

/* Search by client id: this is the comparison for finding an event, not for
 * sorting them. The array below is in ARRIVAL order and stays that way. */
static int by_client(const void *a, const void *b) {
    const ex_event_t *x = (const ex_event_t *)a;
    const ex_event_t *y = (const ex_event_t *)b;
    return (x->client > y->client) - (x->client < y->client);
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- 1. reserve: decide the capacity once ----------------------------- */

    proven_result_array_t ar = PROVEN_ARRAY_INIT(alloc, ex_event_t, 4);
    EXAMPLE_REQUIRE(proven_is_ok(ar.err), "creating the event log must succeed");
    if (!proven_is_ok(ar.err)) {
        return 1;
    }
    proven_array_t events = ar.value;

    /* We know the batch size before we start, so ask for the room once. Without
     * this the array doubles as it fills, copying its contents each time; behind
     * an arena allocator each of those copies also leaves the old block behind
     * until the arena is reset. */
    proven_err_t err = proven_array_reserve(&events, 16);
    EXAMPLE_REQUIRE(proven_is_ok(err), "reserving room for the whole batch must succeed");
    EXAMPLE_REQUIRE(events.cap >= 16, "the capacity is now at least what was asked for");

    /* is_valid asks whether the handle itself is structurally sound - a pointer,
     * a length and a capacity that agree. Assert it where a container arrives
     * from other code, or after a zero-initialised handle might have escaped;
     * it is not something to repeat after every push. */
    EXAMPLE_REQUIRE(proven_array_is_valid(&events), "a created array must be structurally valid");

    proven_array_t never_created = {0};
    EXAMPLE_REQUIRE(!proven_array_is_valid(&never_created),
                    "a zero-initialised handle is not a usable array");

    static const ex_event_t batch[] = {
        { .client = 7, .code = 200 }, { .client = 3, .code = 200 },
        { .client = 7, .code = 404 }, { .client = 9, .code = 200 },
        { .client = 3, .code = 500 }, { .client = 7, .code = 0   },
    };
```

```
proven_size_t batch_len = sizeof batch / sizeof batch[0];

proven_size_t cap_before = events.cap;
for (proven_size_t i = 0; i < batch_len; ++i) {
    err = PROVEN_ARRAY_PUSH(&events, ex_event_t, batch[i]);
    EXAMPLE_REQUIRE(proven_is_ok(err), "pushing an event must succeed");
}
EXAMPLE_REQUIRE(events.cap == cap_before, "the reserve was enough: nothing reallocated while filling");

/* --- 2. searching data that is not sorted ------------------------------ */

/* The log is in arrival order, which is the order we want to keep: it is the
 * thing being recorded. Binary search would be faster and WRONG here, since
 * it may only be used on a sorted range - on unsorted data it does not
 * return "not found", it returns nonsense. Linear search is the correct
 * tool, and O(n) over a batch this size is nothing. */
ex_event_t key = { .client = 9, .code = 0 };
const ex_event_t *found = (const ex_event_t *)proven_array_linear_search(&events, &key, by_client);
EXAMPLE_REQUIRE(found != NULL, "client 9 appears in the batch");
EXAMPLE_REQUIRE(found->code == 200, "and linear search returns the FIRST match in order");

ex_event_t absent = { .client = 42, .code = 0 };
EXAMPLE_REQUIRE(proven_array_linear_search(&events, &absent, by_client) == NULL,
                "a client that never appeared is reported as not found");

/* --- 3. a ring for the most recent events ------------------------------ */

proven_result_ring_t rr = PROVEN_RING_INIT(alloc, ex_event_t, 4);
EXAMPLE_REQUIRE(proven_is_ok(rr.err), "creating the recent-events ring must succeed");
proven_ring_t recent = rr.value;
EXAMPLE_REQUIRE(proven_ring_is_valid(&recent), "a created ring must be structurally valid");

proven_ring_t unset = {0};
EXAMPLE_REQUIRE(!proven_ring_is_valid(&unset), "a zero-initialised ring handle is not usable");

/* This ring refuses when full rather than overwriting, so "keep the most
 * recent four" means dropping the oldest ourselves before pushing. */
for (proven_size_t i = 0; i < batch_len; ++i) {
    if (recent.len == recent.cap) {
        ex_event_t dropped;
        err = PROVEN_RING_POP(&recent, ex_event_t, &dropped);
        EXAMPLE_REQUIRE(proven_is_ok(err), "a full ring must yield its oldest element");
    }
    err = PROVEN_RING_PUSH(&recent, ex_event_t, batch[i]);
    EXAMPLE_REQUIRE(proven_is_ok(err), "pushing after making room must succeed");
}
EXAMPLE_REQUIRE(recent.len == 4, "the ring holds the four most recent events");

/* --- 4. counting per client, with one lookup per update ---------------- */

/* create_with_capacity is proven_map_create under a name that says why the
 * capacity argument is there: sizing it now avoids rehashing later, and a
 * rehash both copies every bucket and invalidates every pointer previously
 * returned by get_mut. */
proven_result_map_t mr = proven_map_create_with_capacity(alloc, 8, PROVEN_KEY_TYPE_INT,
                                                         sizeof(proven_u32), alignof(proven_u32));
EXAMPLE_REQUIRE(proven_is_ok(mr.err), "creating the counter map must succeed");
proven_map_t counts = mr.value;
EXAMPLE_REQUIRE(proven_map_is_valid(&counts), "a created map must be structurally valid");
```

```
err = proven_map_reserve(&counts, 32);
EXAMPLE_REQUIRE(proven_is_ok(err), "reserving map capacity must succeed");

for (proven_size_t i = 0; i < batch_len; ++i) {
    proven_map_key_t k = { .id = batch[i].client };

    /* get_mut returns a pointer INTO the map's storage, so the counter is
     * incremented where it lives: one lookup, no copy back. The pointer is
     * good only until the next insert - which is another reason the capacity
     * was reserved above. */
    proven_u32 *seen = (proven_u32 *)proven_map_get_mut(&counts, k);
    if (seen) {
        *seen += 1;
    } else {
        proven_u32 one = 1;
        err = proven_map_set(&counts, k, &one);
        EXAMPLE_REQUIRE(proven_is_ok(err), "inserting a new client must succeed");
    }
}

const proven_u32 *seven = PROVEN_MAP_GET_INT(&counts, proven_u32, 7);
EXAMPLE_REQUIRE(seven != NULL && *seven == 3, "client 7 sent three events");

/* A closing event (code 0) means the client is gone: drop its counter. */
for (proven_size_t i = 0; i < batch_len; ++i) {
    if (batch[i].code == 0) {
        err = proven_map_remove(&counts, (proven_map_key_t){ .id = batch[i].client });
        EXAMPLE_REQUIRE(proven_is_ok(err), "removing a present key must succeed");
    }
}
EXAMPLE_REQUIRE(PROVEN_MAP_GET_INT(&counts, proven_u32, 7) == NULL, "the closed client is gone");

/* --- 5. which hash, and how to see the difference ---------------------- */

/* Two string-key maps over the same keys. The default one hashes with keyed
 * SipHash, so an attacker who chooses the keys cannot force them all into
 * one bucket. The trusted one uses fast FNV-1a and is the right choice ONLY
 * when your own code chooses every key. */
proven_result_map_t untrusted = PROVEN_MAP_INIT_U8_BORROWED(alloc, proven_u32, 8);
EXAMPLE_REQUIRE(proven_is_ok(untrusted.err), "creating the default string-key map must succeed");
proven_map_t from_network = untrusted.value;

proven_result_map_t tr = proven_map_create_trusted(alloc, 8, PROVEN_KEY_TYPE_U8_BORROWED,
                                                   sizeof(proven_u32), alignof(proven_u32));
EXAMPLE_REQUIRE(proven_is_ok(tr.err), "creating the trusted-key map must succeed");
proven_map_t internal = tr.value;

EXAMPLE_REQUIRE(from_network.trusted_keys == false, "the default map defends against chosen keys");
EXAMPLE_REQUIRE(internal.trusted_keys == true, "the trusted map opts out of that defence");

/* map_hash exposes the value the map actually places a key by, so the
 * choice is observable rather than something you take on faith. */
proven_map_key_t name = { .str = PROVEN_LIT("user-agent") };
proven_u64 keyed = proven_map_hash(&from_network, name);
proven_u64 fast  = proven_map_hash(&internal, name);
EXAMPLE_REQUIRE(keyed != fast, "the same key hashes differently under the two functions");
EXAMPLE_REQUIRE(proven_map_hash(&internal, name) == fast, "and each function is deterministic");

/* Keys that live in memory the map does not own are BORROWED: the bytes must
 * outlive the map. These are string literals, so they do. When the key comes
```

```
     * from a buffer you are about to reuse, use an owned-key map instead
     * (PROVEN_MAP_INIT_U8_OWNED / proven_map_set_u8_owned), which copies. */
    proven_u32 hits = 1;
    err = proven_map_set(&from_network, name, &hits);
    EXAMPLE_REQUIRE(proven_is_ok(err), "inserting a borrowed string key must succeed");
    EXAMPLE_REQUIRE(PROVEN_MAP_GET_U8_BORROWED(&from_network, proven_u32, PROVEN_LIT("user-agent")) != NULL,
                    "and it can be looked up by an equal view, not the same pointer");

    /* --- 6. checksumming a stream in chunks ------------------------------ */

    /* The batch is checksummed as it goes past, which is what a program reading
     * a file or a socket has to do: it never holds the whole thing. Start the
     * running value at 0, feed each chunk in, and the final value is the same
     * one a single call over the concatenation would produce. */
    proven_u32 running = 0;
    for (proven_size_t i = 0; i < batch_len; ++i) {
        proven_mem_view_t chunk = { .ptr = (const proven_byte_t *)&batch[i], .size = sizeof batch[i] };
        running = proven_crc32_update(running, chunk);
    }
    proven_mem_view_t whole = { .ptr = (const proven_byte_t *)batch, .size = sizeof batch };
    EXAMPLE_REQUIRE(running == proven_crc32(whole),
                    "chunked and whole-buffer CRC-32 must agree, whatever the chunking");

    printf("events=%zu recent=%zu crc32=%08x\n",
           (size_t)events.len, (size_t)recent.len, (unsigned)running);

    proven_map_destroy(&internal);
    proven_map_destroy(&from_network);
    proven_map_destroy(&counts);
    PROVEN_RING_DESTROY(&recent);
    PROVEN_ARRAY_DESTROY(&events);
    return EXAMPLE_OK();
}
```

Wrong - binary search on the arrival-ordered log:

```
const ex_event_t *e = proven_array_binary_search(&events, &key, by_client);   /* wrong */
```

`proven_array_binary_search()` may only be used on a range sorted by the same comparison. On unsorted input it does not fail; it halves its way to whatever element happens to be there and reports it, or reports nothing while the element is present two slots away.

Wrong - holding a `get_mut` pointer across an insert:

```
proven_u32 *seen = proven_map_get_mut(&counts, k);
proven_err_t e = proven_map_set(&counts, other_key, &one);   /* may rehash */
*seen += 1;                                                  /* wrong: may dangle */
```

Hold the key, not the pointer, across anything that can insert.

Wrong - the fast hash on keys from outside the program:

```
proven_result_map_t m = proven_map_create_trusted(alloc, 64, PROVEN_KEY_TYPE_U8_BORROWED,
                                                  sizeof(session_t), alignof(session_t));
/* keys are HTTP header names taken from the request */   /* wrong */
```

That is exactly the case the keyed hash defends against. Use `proven_map_create()` whenever the keys are chosen by anyone other than you.

Chapter 8: Formatting and Scanning (v0.97.0)

Part IV - Text in and out. Prerequisite: Chapter 3 section 3-section 4. After this chapter you can format any value, teach the formatter a type of your own, parse input with the cursor under your control, and recover from a partial parse.

This chapter is the **reference half** of the text material; Chapter 3 is the tutorial half. Chapter 3 introduces the formatter and the scanner beside the string types and gives you enough to be productive. This chapter gives the exact syntax, every parameter shape, every return value, and the places where callers usually go wrong. If you are meeting `{}` for the first time, read Chapter 3 first - this one assumes you have.

Table of contents

1. Design model
2. Formatter data model
3. Formatter constructors and selectors
4. Format string grammar
5. Formatting APIs

5.1. Formatting a type of your own

1. Console print helpers
2. Scanner data model
3. Scanner primitive APIs
4. Scan argument model
5. Structural scan grammar
6. Scan formatting APIs

11.1. Scan error code guide and recovery

1. Examples and misuse cases
2. Freestanding and build-mode notes

1. Design model

Why the type is never written twice
Both halves of this chapter exist to eliminate one thing: a place where the programmer states a type that the compiler cannot check against the value.

`printf("%d", x)` states it twice - once as `%d`, once by passing `x` - and varargs erases the second, so nothing can compare them. `scanf("%d", &x)` is worse: the format decides both how to parse *and* what to write through the pointer, so a mismatch corrupts memory rather than printing nonsense. Both are the same defect from opposite directions, and both compile silently.

Here the placeholder carries **no type at all**. `{}` marks a position; the type comes from `PROVEN_ARG(x)` on the formatting side and `PROVEN_SCAN_ARG(&x)` on the scanning side, both resolved by `_Generic` at compile time against the argument's static type. The spec after `:` controls presentation only - width, fill, alignment, precision, base - never interpretation. There is no `%d`-versus-`double` to get wrong because you never wrote a type in the string.

The second design decision is that **neither side owns a buffer**. Formatting appends into a destination you supply and refuses when it does not fit; scanning reads from a view you supply and moves a cursor you can read. Nothing here allocates unless you hand it an allocator, which is what lets the whole chapter work in a freestanding build (section 13).

The formatting side and the scanning side solve opposite problems.

- Formatting takes typed values and renders text.

- Scanning takes text and writes typed values.

The project keeps both sides intentionally small:

- formatting supports a compact placeholder language, positional reuse, simple alignment, width, and hex rendering for numeric values;
- scanning supports typed destination pointers, strict placeholder counting, `{:x}` for hexadecimal integers, and literal matching with whitespace collapsing;
- neither side tries to become a full `printf` or `scanf` clone.

The practical result is that the APIs are easier to reason about than large general-purpose format engines, but the syntax is still expressive enough for common systems-code tasks.

2. Formatter data model

```
proven_fmt_result_t
typedef struct {
proven_err_t  err;
proven_size_t written;
proven_size_t required;
} proven_fmt_result_t;
```

Meaning:

- `err`: the status code for the operation.
- `written`: bytes actually written into the destination.
- `required`: total bytes needed for the full formatted output.

Use `err` first. The other fields are most useful for truncating or partially successful operations.

A successful result looks like this:

```
proven_result_u8str_t rs = proven_u8str_create(alloc, 64);
if (proven_is_ok(rs.err)) {
    proven_u8str_t s = rs.value;

    proven_fmt_result_t r = proven_u8str_append_fmt_trunc(
        &s,
        "hello {}",
        PROVEN_ARG("world")
    );
    if (proven_is_ok(r.err)) {
        proven_println("{}", PROVEN_ARG(proven_u8str_as_view(&s)));
    }

    proven_u8str_destroy(alloc, &s);
}
```

A truncating result can still tell you how much more space you would have needed. Here the destination is deliberately too small, so `written` stops short of `required` - and the string is still valid:

```
proven_result_u8str_t rs = proven_u8str_create(alloc, 8);   /* on purpose: too small */
if (proven_is_ok(rs.err)) {
    proven_u8str_t s = rs.value;

    proven_fmt_result_t r = proven_u8str_append_fmt_trunc(
        &s,
        "name={} score={}",
        PROVEN_ARG("ada"),
        PROVEN_ARG(42)
    );
    /* r.written is what fit; r.required is what the whole output needed. */
```

```
    proven_println("wrote {} of {} bytes",
                   PROVEN_ARG(r.written), PROVEN_ARG(r.required));

    proven_u8str_destroy(alloc, &s);
}
```

proven_arg_type_t

```
typedef enum {
PROVEN_ARG_NONE,
PROVEN_ARG_I32,
PROVEN_ARG_U32,
PROVEN_ARG_I64,
PROVEN_ARG_U64,
#ifndef PROVEN_FMT_NO_FLOAT
PROVEN_ARG_F64,
#endif
PROVEN_ARG_CSTR,
PROVEN_ARG_STR_VIEW,
PROVEN_ARG_DATETIME,
PROVEN_ARG_PTR,
PROVEN_ARG_FN,
} proven_arg_type_t;
```

The formatter currently recognizes these value classes:

- signed 32-bit integers
- unsigned 32-bit integers
- signed 64-bit integers
- unsigned 64-bit integers
- floating-point values, unless `PROVEN_FMT_NO_FLOAT` is defined
- trusted C strings
- borrowed U8 string views
- datetimes
- object pointers
- function pointers

proven_arg_t

```
typedef struct {
proven_arg_type_t type;
union {
proven_i32 i32;
proven_u32 u32;
proven_i64 i64;
proven_u64 u64;
double f64;
const char *cstr;
proven_u8str_view_t str_view;
proven_datetime_t datetime;
const void *ptr;
void (*fn)(void);
} value;
} proven_arg_t;
```

The union field must match the selected `type`. Do not manufacture a `proven_arg_t` by writing a mismatched union field and hoping the formatter will guess.

Wrong:

```
proven_arg_t arg = {0};
arg.type = PROVEN_ARG_I64;
arg.value.u64 = 123; /* wrong: type and union field do not match */
```

Correct:

```
proven_arg_t arg = proven_arg_i64(123);
(void)arg;   /* pass it to a formatting macro; PROVEN_ARG(arg) accepts it as-is */
```

3. Formatter constructors and selectors

Constructor summary

API	Parameters	Returns	Intent
`proven_arg_none(void)`	none	`proven_arg_t`	Internal sentinel value.
`proven_arg_i32(int v)`	signed integer	`proven_arg_t`	Render as 32-bit signed integer.
`proven_arg_u32(unsigned int v)`	unsigned integer	`proven_arg_t`	Render as 32-bit unsigned integer.
`proven_arg_i64(long long v)`	wide signed integer	`proven_arg_t`	Render as 64-bit signed integer.
`proven_arg_u64(unsigned long long v)`	wide unsigned integer	`proven_arg_t`	Render as 64-bit unsigned integer.
`proven_arg_f64(double v)`	floating-point value	`proven_arg_t`	Render as floating-point text, unless float formatting is disabled.
`proven_arg_cstr(const char *v)`	trusted live C string	`proven_arg_t`	Render a NUL-terminated C string.
`proven_arg_cstr_n(const char *v, proven_size_t max_len)`	possibly bounded C string	`proven_arg_t`	Render only up to `max_len` while searching for NUL.
`proven_arg_str_view(proven_u8str_view_t v)`	borrowed U8 view	`proven_arg_t`	Render a borrowed view without assuming NUL termination.
`proven_arg_datetime(proven_datetime_t v)`	datetime value	`proven_arg_t`	Render a datetime using the formatter's datetime rules.
`proven_arg_ptr(const void *v)`	object pointer	`proven_arg_t`	Render the pointer value.
`proven_arg_fn(void (*v)(void))`	function pointer	`proven_arg_t`	Render the raw function-pointer representation.
`proven_arg_ucstr(const unsigned char *v)`	unsigned-char string	`proven_arg_t`	Convenience wrapper around `proven_arg_cstr`.
`proven_arg_identity(proven_arg_t v)`	existing argument object	`proven_arg_t`	Pass-through helper.
`proven_arg_bool(bool v)`	boolean	`proven_arg_t`	Render `true` / `false` as words, not as `1` / `0`.
`proven_arg_char(char v)`	a character	`proven_arg_t`	Render the **character**. This is why a `char` VARIABLE renders as a character while the literal `PROVEN_ARG('Z')` still renders

			`90`: in C, `'Z'` has type `int`, and no amount of `_Generic` can tell it apart from the number 90.
`proven_arg_custom(const void *v, proven_fmt_custom_fn fn)`	any type at all	`proven_arg_t`	Render a type the library has never heard of, through a function you supply. See Formatting a user-defined type.

`PROVEN_ARG(x)`

`PROVEN_ARG(x)` is the usual entry point. It uses `_Generic` so the compiler chooses a constructor from the type of `x`.

The current mapping is:

- `_Bool`, `char`, `signed char`, `short`, `int` -> `proven_arg_i32`
- `unsigned char`, `unsigned short`, `unsigned int` -> `proven_arg_u32`
- `long`, `long long` -> `proven_arg_i64`
- `unsigned long`, `unsigned long long` -> `proven_arg_u64`
- `double`, `float` -> `proven_arg_f64`, unless `PROVEN_FMT_NO_FLOAT` is defined
- `const char *`, `char *` -> `proven_arg_cstr`
- `unsigned char *`, `const unsigned char *` -> `proven_arg_ucstr`
- `void *`, `const void *` -> `proven_arg_ptr`
- `proven_u8str_view_t` -> `proven_arg_str_view`
- `proven_datetime_t` -> `proven_arg_datetime`
- `proven_arg_t` -> `proven_arg_identity`

Important consequence:

- `PROVEN_ARG` does not select function pointers.
- Use `PROVEN_ARG_FN(f)` for function pointers.

Wrong:

```
void helper(void) {}
proven_u8str_append_fmt_grow(alloc, &s, "{}", PROVEN_ARG(helper)); /* wrong */
```

Correct:

```
void helper(void);   /* whatever function you want to print the address of */

proven_result_u8str_t rs = proven_u8str_create(alloc, 64);
if (proven_is_ok(rs.err)) {
    proven_fmt_result_t r = proven_u8str_append_fmt_grow(alloc, &rs.value, "{}",
                                                         PROVEN_ARG_FN(helper));
    (void)r;
    proven_u8str_destroy(alloc, &rs.value);
}
```

`PROVEN_ARG_FN(f)`

This macro exists so callers can pass function pointers without casting them through `void *`. It is a small safety wrapper around `proven_arg_fn`.

Example:

```
void helper(void);

proven_result_u8str_t rs = proven_u8str_create(alloc, 64);
```

```c
if (proven_is_ok(rs.err)) {
    proven_u8str_t s = rs.value;

    proven_fmt_result_t r = proven_u8str_append_fmt_grow(
        alloc,
        &s,
        "callback = {}",
        PROVEN_ARG_FN(helper)
    );
    if (!PROVEN_FMT_IS_OK(r)) {
        proven_eprintln("formatting the callback failed");
    }

    proven_u8str_destroy(alloc, &s);
}
```

PROVEN_ARG_CSTR_N(v, max_len)

This macro is the bounded-string helper. Use it when the source may not be a fully trusted C string, but you still want C-string-like input handling.

It scans for NUL only up to `max_len` and then formats the bounded prefix as a view.

Good use case - `buf` came off the wire, so nothing promises it is NUL-terminated:

```c
char buf[128];                      /* filled from an untrusted source */
(void)proven_mem_copy(buf, sizeof buf, proven_mem_view_from_u8(PROVEN_LIT("payload")));

proven_result_u8str_t rs = proven_u8str_create(alloc, 64);
if (proven_is_ok(rs.err)) {
    proven_fmt_result_t r = proven_u8str_append_fmt_grow(
        alloc,
        &rs.value,
        "payload={}",
        PROVEN_ARG_CSTR_N(buf, sizeof buf)   /* looks for NUL only within 128 bytes */
    );
    (void)r;
    proven_u8str_destroy(alloc, &rs.value);
}
```

Bad use case:

```c
const char *buf = get_network_buffer();
proven_u8str_append_fmt_grow(alloc, &s, "{}", PROVEN_ARG(buf)); /* wrong if buf is not trusted */
```

Float formatting note

If `PROVEN_FMT_NO_FLOAT` is defined, float support is removed from the generic selector and the float constructor is not available. That is a compile-time configuration choice, not a runtime toggle.

The default `{}` rendering of a `double` produces a fixed six-digit fractional form for finite values and switches to scientific notation when the magnitude is too large or too small for the compact form. Unlike earlier versions, this output is now computed by an **exact, integer-only** engine (no `double`/`long double` approximation): the digits are correctly rounded (round-half-to-even), so `{}` matches what `printf("%.6f")` / `%.6e` would print on the same value. For a shortest round-trippable form, or for a precision other than six, use the `proven_float_format_*` policy API described below.

Accuracy and limits

- The default `{}` float output uses six fractional digits, correctly rounded to nearest with ties to even (matching glibc `%.6f`/`%.6e`). It is exact at any magnitude - there is no `long double` and no precision/magnitude ceiling beyond the configurable big-integer capacity.
- For round-trip serialization use the **shortest** policy (`proven_float_format_options_shortest()`), which emits the shortest decimal string that parses back to the exact same value.

- Decimal-to-double scanning is correctly rounded to IEEE-754 binary64 with round-to-nearest, ties-to-even, matching the host `strtod` bit for bit. Values below the half-way threshold to the smallest subnormal round to signed zero with the input sign preserved.
- The parser is layered `Clinger fast path -> Eisel-Lemire -> exact big-integer fallback`; every tier is exact and the fallback is the arbiter, so the result is correctly rounded for every input. The cached power tables are generated source (`scripts/generate_float_decimal_tables.py`).
- The exact-fallback big-integer capacity is tunable with `PROVEN_FLOAT_BIGINT_LIMBS` (see `include/proven/float_config.h`) for embedded targets; the fast paths never touch the big integer.
- Validation: the formatter and parser are checked exhaustively over all 4.28 billion finite `binary32` values and over 2.56 billion random `binary64` values against the host C library, with zero mismatches. See `docs/float-correctness-and-performance.md` for algorithms, methodology, and a benchmark against glibc.

Inside the engine (conceptual)
You do not need any of this to use the API - it is here for readers who want to know why the output is trustworthy. Full detail is in `docs/float-correctness-and-performance.md`.

Parsing (decimal -> binary64), three tiers, fastest first. The result is always correctly rounded; the tiers are purely a speed staircase, each one only taken when it can guarantee the exact answer:

1. *Clinger fast path.* When the value has few significant digits and a small exponent, both the significand and `10^exp` are exactly representable as `double`, so a single rounded multiply/divide is provably correct. Covers most everyday numbers.
2. *Eisel-Lemire.* A 64x128-bit fixed-point multiply by a cached power of ten, with a check that the result is far enough from a rounding boundary to be certain. If the check is inconclusive (the value sits on a halfway tie), it falls through.
3. *Exact big-integer fallback.* Builds the value as a ratio of big integers (`significand` and `5^q`/`2^q`) and compares against the candidate `double` and its neighbor exactly - this is the arbiter that makes ties and subnormals correct. A seeded +/-16-ULP window keeps the search to a few comparisons. The big-integer capacity is bounded by `PROVEN_FLOAT_BIGINT_LIMBS`; the tier is the only one that allocates limbs (on the stack), and the fast paths never reach it.

Formatting (binary64/32 -> decimal). Two engines, no `long double` anywhere:

- *Shortest* (`proven_float_format_options_shortest()`): a **Grisu3** fast path produces the minimal round-trippable digits for almost all inputs in ~90 ns; the rare cases where Grisu3 cannot prove minimality fall back to an exact **Dragon4** (Burger-Dybvig, round-to-even) core. The result is the unique shortest decimal that parses back to the same bits.
- *Fixed `%f` / scientific `%e`* (the default `{}` and the fixed options): an exact integer engine scales the value by `10^precision` with big-integer `mul_pow5`/shift, does an integer `divmod`, and rounds half-to-even - so it matches glibc at any precision and magnitude, with no `2^64`/precision ceiling. Extreme exponents do real arbitrary-precision work and are correspondingly slower (rare in practice).

Public float parsing APIs
Three entry points, sharing one correctly-rounded backend:

- `proven_scan_f64(scan)` - parse from a `proven_scan_t` cursor; restores the cursor on failure. The native, length-bounded path (no NUL terminator needed).
- `proven_parse_double_ascii(view)` - parse one locale-free ASCII token from a `proven_u8str_view_t` and report the consumed length.
- `proven_strtod(nptr, endptr)` - a `strtod`-style convenience wrapper over a C string: skips leading ASCII whitespace and reports `endptr`.

Worked example: parsing

```
#include "proven/scan.h"
#include "proven/float_parse.h"

/* (1) Native, view-based parsing through a scanner cursor. */
proven_scan_t sc = proven_scan_init(proven_u8str_view_from_cstr("3.14159e2 rest"));
proven_result_f64_t r = proven_scan_f64(&sc);
if (r.err == PROVEN_OK) {
    /* r.val == 314.159; the cursor now sits at " rest". */
    proven_println("parsed {}", PROVEN_ARG(r.val));
}

/* (2) strtod-style wrapper for C strings. endptr reports where parsing stopped. */
char *end = NULL;
double v = proven_strtod("  -0.5\t", &end);   /* v == -0.5, *end == '\t' */
(void)v;

/* A trailing exponent marker with no digits stops like strtod: "1e" parses 1,
   leaving endptr at 'e'. Inputs with hundreds of significant digits and extreme
   exponents are still rounded correctly via the exact fallback. */
```

Worked example: formatting with the policy API

`proven_float_format_f64_policy` / `_f32_policy` write directly into a caller buffer and report the number of bytes written. They never allocate.

```
#include "proven/float_format.h"

char buf[64];
proven_size_t n = 0;

/* Shortest round-trippable form: 0.1 -> "0.1" (not "0.10000000000000001"). */
(void)proven_float_format_f64_policy(buf, sizeof buf, 0.1,
    PROVEN_FLOAT_FORMAT_POLICY_RYU,
    proven_float_format_options_shortest(), &n);
/* buf == "0.1", n == 3 */

/* Fixed precision (correctly rounded, round-half-to-even). */
proven_float_format_options_t opt = proven_float_format_options_fixed_default();
opt.precision = 2;
(void)proven_float_format_f64_policy(buf, sizeof buf, 3.14159,
    PROVEN_FLOAT_FORMAT_POLICY_DEFAULT, opt, &n);
/* buf == "3.14" */

/* Always scientific - this is what {:e} selects. Six fractional digits by default,
   a signed two-digit-minimum exponent: exactly what printf's %e prints. */
proven_float_format_options_t sci = proven_float_format_options_scientific();
sci.precision = 2;
(void)proven_float_format_f64_policy(buf, sizeof buf, 42.0,
    PROVEN_FLOAT_FORMAT_POLICY_DEFAULT, sci, &n);
/* buf == "4.20e+01" - where fixed would give "42.00" and shortest "42" */
```

- `PROVEN_FLOAT_FORMAT_POLICY_RYU` selects shortest output; `DEFAULT`/`SIMPLE` select the exact fixed-precision path (`%f`, switching to `%e` for very large or very small magnitudes).
- Returns `PROVEN_ERR_OUT_OF_BOUNDS` if the buffer is too small (the value is never truncated silently) and `PROVEN_ERR_INVALID_ARG` for an unsupported policy.
- The generic `{}` formatter (`proven_u8str_append_fmt*`) uses the `DEFAULT` policy internally, so everyday logging needs no direct call to this API.

4. Format string grammar

The formatter accepts a deliberately small grammar.

Replacement fields

Supported forms:

- `{}`: next positional argument
- `{0}`: first user argument
- `{1}`: second user argument
- `{2}`: third user argument
- and so on

The numbering is user-facing and zero-based. The implementation stores a hidden sentinel at index 0 and maps user index `0` to internal argument slot `1`.

Escaped braces

- `{{` becomes a literal `{`
- `}}` becomes a literal `}`

The layout spec

`{:[[fill]align][sign][#][0][width][.precision][type]}`

Every part is optional, and the order is the one the rest of the world uses, so a spec copied from Python or Rust means here what it means there.

Part	Values	What it does
`align`	`< > ^`	left, right, centre. Default `>`.
`fill`	any character, before an align	the pad character. Default space.
`sign`	`+` or a space	force a sign on a non-negative number, or reserve a space for one.
`#`		alternate form: `0x`, `0X`, `0o`, `0b` prefixes. Integers only.
`0`		zero-fill. `{:08}` on 42 is `00000042`.
`width`	digits, up to 10000	minimum field width.
`.precision`	`.N`, up to 60	decimals. **Floats only.**
`type`	`x X o b d` (int), `f g e` (float)	base and case; `f` fixed, `g` shortest round-trip, `e` scientific (printf `%e`).

Two things are worth knowing because they used to be false:

- **A leading `0` is zero-fill, not the first digit of the width.** `{:08}` produced `"      42"` - space-padded, no error - until v26.07.12f. An explicit fill still wins: `{:*>08}` pads with `*`.
- **Zero-padding goes between the sign and the digits.** `{:+08}` on 42 is `+0000042`, never `0000+42`. Padding is part of the number, and a number's sign comes first.

Floats

`{}` gives six decimals, correctly rounded. `{:.3}` gives three, `{:.0}` gives none, `{:e}` forces scientific notation - mantissa, `precision` fractional digits (six by default), a signed two-digit-minimum exponent, correctly rounded, exactly as printf's `%e` - which is the form `{:f}` and `{:g}` do not reach: `{:f}` never shows an exponent, and `{:g}` uses one only when it is shorter. `{:f}` forces the fixed form, and `{:g}` gives the shortest representation that round-trips.

Until v26.07.12i **none of these existed**: every float came out with exactly six decimals, forever. The exact engine could always do all of it - the `{}` grammar simply could not reach it. The visible cost

was that a float column could not be aligned, because `12.5` rendered nine characters wide and `100.0` rendered ten:

```
proven_byte_t buf[64];
proven_u8str_t line = proven_u8str_borrow(buf, sizeof buf);
(void)proven_u8str_append_fmt(&line, "{:>9.2}", PROVEN_ARG(12.5));    /* "    12.50" */
```

A spec the argument cannot honour is an error
`{:x}` on a double, `{:.2}` on an integer, `{:f}` on an integer, `{:#}` on a string: all `PROVEN_ERR_INVALID_FORMAT`.

They used to be *ignored* - `{:x}` on a double printed `3.500000` and reported success. The caller asked for something, got something else, and was told it had worked. That is the worst available outcome, and it is worse than refusing.

`char` and `bool`
`PROVEN_ARG('Z')` renders `Z`, and a `bool` renders `true` or `false`. Both used to go through the integer path, so a character printed as `90` and there was no way to emit one at all - the ASCII column of a hex dump had to be built by hand in a separate buffer and passed as a string. An uppercase hex dump is now one loop:

```
proven_byte_t hexbuf[64];
proven_u8str_t hexline = proven_u8str_borrow(hexbuf, sizeof hexbuf);
unsigned char byte = 0xde;
(void)proven_u8str_append_fmt(&hexline, "{:02X} ", PROVEN_ARG((unsigned)byte));
(void)proven_u8str_append_fmt(&hexline, "{}", PROVEN_ARG((char)'.'));
```

5. Formatting APIs

`proven_u8str_fmt_internal(...)`

```
proven_fmt_result_t proven_u8str_fmt_internal(
proven_allocator_t alloc,
proven_u8str_t *str,
bool trunc,
const char *fmt,
proven_allocator_t scratch,
const proven_arg_t *args,
proven_size_t args_count
);
```

This is the internal formatting engine. User code should normally call the public macros instead.

Parameters:

- `alloc`: allocator used when the string must grow
- `str`: destination U8 string
- `trunc`: if true, allow best-effort truncation; if false, keep atomic behavior
- `fmt`: format text
- `scratch`: allocator used for temporary alias-patching when needed
- `args`: array of format arguments, including the hidden sentinel at index 0
- `args_count`: total length of `args`, including the sentinel

Return value:

- a `proven_fmt_result_t`

Important rules:

- `args_count` must match the number of placeholders plus the hidden sentinel
- extra unused arguments are an error
- missing arguments are an error

- if the engine detects aliasing between the destination string and a borrowed view argument, it may use the scratch allocator to preserve failure atomicity

`proven_u8str_append_fmt(str, fmt, ...)`
Atomic formatting into a fixed-capacity string. If the result does not fit, the function reports failure and leaves the destination unchanged.

Use this when you want all-or-nothing behavior.

`proven_u8str_append_fmt_trunc(str, fmt, ...)`
Best-effort formatting. It writes as much as fits and reports how much was written and how much was required.

Use this when partial output is acceptable.

`proven_u8str_append_fmt_grow(alloc, str, fmt, ...)`
Growable formatting. It may reallocate the destination string through the supplied allocator. On allocation failure, the old string remains valid.

Use this when you want the output to fit without manual capacity planning.

`proven_u8str_append_fmt_with_scratch(alloc, str, fmt, scratch, ...)`
Growable formatting with a separate scratch allocator. This is useful when the argument list contains string views that may alias the destination buffer and temporary patching is needed.

Use a real allocator for both `alloc` and `scratch`. Do not pass a dead arena or a one-shot temporary buffer unless its lifetime is long enough for the call.

Float format policy seam
The public float policy header provides an explicit policy layer for float formatting. It is intentionally small and keeps the exact fixed-precision formatter as the default path.

The main entry points are:

- `proven_float_format_f64_policy(...)`
- `proven_float_format_f32_policy(...)`
- `proven_float_format_options_fixed_default()`
- `proven_float_format_options_shortest()`

Policy notes:

- `PROVEN_FLOAT_FORMAT_POLICY_DEFAULT` and `PROVEN_FLOAT_FORMAT_POLICY_SIMPLE` select the exact fixed-precision output (correctly rounded, round-half-to-even).
- `PROVEN_FLOAT_FORMAT_POLICY_RYU` is the shortest-output policy branch.
- The policy API returns `PROVEN_ERR_INVALID_ARG` for unsupported enum values.
- The policy API returns `PROVEN_ERR_OUT_OF_BOUNDS` when the caller-provided buffer is too small.

Example:

```
char buf[128];
proven_size_t written = 0;
proven_err_t err = proven_float_format_f64_policy(
    buf,
    sizeof buf,
    0.1,
    PROVEN_FLOAT_FORMAT_POLICY_RYU,
    proven_float_format_options_shortest(),
    &written
);
if (proven_is_ok(err)) {
    /* buf holds the shortest form that parses back to exactly 0.1 */
```

```
    proven_println("{}", PROVEN_ARG_CSTR_N(buf, written));
}
```

PROVEN_FMT_IS_OK(res)
A small helper macro for checking `proven_fmt_result_t`. Use it when you want the intent to stay compact.

Example:

```
proven_result_u8str_t rs = proven_u8str_create(alloc, 32);
if (proven_is_ok(rs.err)) {
    proven_u8str_t s = rs.value;

    proven_fmt_result_t r = proven_u8str_append_fmt_grow(
        alloc,
        &s,
        "name={} score={:0>4}",
        PROVEN_ARG("ada"),
        PROVEN_ARG(42)
    );
    if (!PROVEN_FMT_IS_OK(r)) {
        /* the string is untouched: grow-mode formatting is failure-atomic */
        proven_eprintln("formatting failed");
    }

    proven_u8str_destroy(alloc, &s);
}
```

Console-style helpers
The `sysio` layer provides print helpers that use the same formatter machinery:

- `proven_print(fmt, ...)`
- `proven_println(fmt, ...)`
- `proven_eprint(fmt, ...)`
- `proven_eprintln(fmt, ...)`

They are convenient when you want formatted output directly to stdout or stderr. They still return `proven_err_t`, so check the result when the output matters.

Example:

```
if (!proven_is_ok(proven_println("hello {}", PROVEN_ARG("world")))) {
    /* the write to stdout failed - a closed pipe, a full disk */
    proven_eprintln("stdout is not writable");
}
```

5.1. Formatting a type of your own

`PROVEN_ARG` is built on `_Generic`, and `_Generic` can only dispatch on types it was told about at compile time. It cannot be told about yours. So the formatter's argument set - integers, floats, strings, pointers, datetimes - was a **closed** one: a `ex_rect_t`, a `uuid_t`, a `vec3_t` could not be passed to `{}` at all.

The two ways around it were both bad. Pre-format the value into a scratch string and pass *that*: an allocation and a copy for every value, in the logging path, which is the one path that must keep working when allocation is exactly what has failed. Or print the fields one at a time and give up on ever aligning the column.

`PROVEN_ARG_OF(&obj, render)` is the door:

```
proven_err_t render(proven_fmt_sink_t out, const void *obj);
```

Three things follow from the shape of that signature:

- **The renderer gets a sink, not a buffer.** It does not need to know how much room there is, and it cannot overflow anything. It emits with `proven_fmt_put`.
- **It composes.** The renderer may call the formatter again - into a stack buffer, with no allocator - and hand the result to the sink. A type whose fields are themselves user types nests naturally.
- **Width, fill and alignment work.** The formatter runs the renderer **twice**: once against a counting sink to learn how wide the output is, then once for real, with the padding applied around it. This is why `{:>10}` lines up a column of your type exactly as it lines up a column of ints - and it is why the renderer must be **deterministic** and must not mutate `obj`. If the two passes disagree, the formatter returns `PROVEN_ERR_INVALID_ARG` rather than emit a field of the wrong width into an aligned column and let you find out later.

What the formatter will **not** do is guess. `{:x}` on a rectangle, `{:.2}` on a UUID, `{:+}` on a matrix - the library has no idea what those would mean for your type, and so it refuses them with `PROVEN_ERR_INVALID_FORMAT`. Inventing a plausible answer and reporting success is how a formatter starts lying; a type letter you did not ask for is not better than an error.

Compiled and run by the test suite:

```
/*
 * Formatting a type the library has never heard of.
 *
 * `PROVEN_ARG` is built on `_Generic`, which can only dispatch on types it was told
 * about - and it cannot be told about yours. So before `PROVEN_ARG_OF` existed, a
 * `ex_rect_t` simply could not be printed. You either pre-formatted it into a scratch
 * string and passed that (an allocation and a copy per value, in the logging path,
 * which is the one path that must not allocate), or you printed the fields one at a
 * time and gave up on ever aligning the column.
 *
 * A renderer receives a *sink*, not a buffer. That is what makes it compose: it can
 * call the formatter again, and its output is just bytes going somewhere. And because
 * the formatter measures the renderer's output before emitting it - by running it once
 * against a counting sink - width, fill and alignment work on a user type exactly as
 * they do on an int.
 */

typedef struct { int w, h; } ex_rect_t;

static proven_err_t render_rect(proven_fmt_sink_t out, const void *obj) {
    const ex_rect_t *r = (const ex_rect_t *)obj;

    /* Compose: the formatter, into a stack buffer, no allocator anywhere. */
    proven_byte_t tmp[64];
    proven_u8str_t s = proven_u8str_borrow(tmp, sizeof tmp);
    proven_fmt_result_t f = proven_u8str_append_fmt(&s, "{}x{}",
                                                    PROVEN_ARG(r->w), PROVEN_ARG(r->h));
    if (!PROVEN_FMT_IS_OK(f)) return f.err;

    return proven_fmt_put(out, proven_u8str_as_view(&s));
}

int main(void) {
    ex_rect_t a = { .w = 1920, .h = 1080 };
    ex_rect_t b = { .w = 640,  .h = 480  };

    proven_byte_t buf[128];
    proven_u8str_t line = proven_u8str_borrow(buf, sizeof buf);

    /* Just like any other argument. */
    proven_fmt_result_t r = proven_u8str_append_fmt(&line, "mode={}", PROVEN_ARG_OF(&a, render_rect));
```

```
    EXAMPLE_REQUIRE(PROVEN_FMT_IS_OK(r), "a user type should format");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&line), PROVEN_LIT("mode=1920x1080")),
                    "the renderer's bytes should be what came out");

    /* And it aligns, which is the whole reason the formatter measures it first: a
     * column of user-defined values lines up like a column of anything else. */
    (void)proven_u8str_reset(&line);
    r = proven_u8str_append_fmt(&line, "[{:>10}]\n[{:>10}]",
                                PROVEN_ARG_OF(&a, render_rect),
                                PROVEN_ARG_OF(&b, render_rect));
    EXAMPLE_REQUIRE(PROVEN_FMT_IS_OK(r), "two user types should format");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&line),
                                         PROVEN_LIT("[ 1920x1080]\n[   640x480]")),
                    "both rows should be right-aligned to the same width");

    /* A spec the library cannot interpret for your type is refused, not guessed at.
     * `{:x}` on a rectangle has no meaning, and answering it with something plausible
     * while reporting success is how a formatter starts lying to you. */
    (void)proven_u8str_reset(&line);
    r = proven_u8str_append_fmt(&line, "{:x}", PROVEN_ARG_OF(&a, render_rect));
    EXAMPLE_REQUIRE(r.err == PROVEN_ERR_INVALID_FORMAT,
                    "a type letter on a user type should be an error");

    return EXAMPLE_OK();
}
```

6. Console print helpers

What `proven_println` is, and what it costs

`proven_println("{}", PROVEN_ARG(x))` is the shortest way to get text out of a program, and it is the right tool for a diagnostic, a one-off tool, or a program whose output is a few lines.

It is the wrong tool for a loop, and the reason is worth stating because it is invisible at the call site: **each call is its own write syscall.** Ten thousand `proven_println` calls are ten thousand syscalls, which is roughly two orders of magnitude more expensive than the formatting itself. `printf` hides this behind a buffer that libc flushes for you; this library does not buffer behind your back, because a buffer you did not ask for is a buffer that surprises you at exit, on a crash, or when two writers interleave.

One allocation caveat. The line is formatted into a 512-byte stack buffer, so a typical line costs zero allocations. A line that does *not* fit falls back to the global heap for that call rather than being refused - refusing to print something because it is long would be worse. This is the one place in the library where a call with no allocator parameter can still allocate, and it is bounded to the over-long case. If you need the guarantee that nothing allocates, format into your own buffer and write that.

When output volume matters, take a buffered writer from `proven_sysio_stdout_buffered` and format into that - same argument rules, one syscall per flush instead of one per line. Chapter 5 covers the stream layer; this section is short because that is where the I/O API is documented.

Wrong - reaching for the scanning argument constructor when printing:

```
proven_println("{}", PROVEN_SCAN_ARG(&x));   /* wrong: that builds a scan destination */
```

The important point for formatter users is that the console helpers share the same argument rules as the string append APIs.

Common mistakes:

- using `PROVEN_SCAN_ARG` with `proven_println`

- assuming `PROVEN_LIT` is needed for every format string
- forgetting that output functions can still fail

7. Scanner data model

The scanner is a cursor over bytes you already have. It does not own the text, it does not copy it, and it does not read from anywhere - `proven_scan_t` is a view plus an offset, and that is the whole of it.

```
typedef struct {
    proven_u8str_view_t view;    /* the bytes being read; not owned */
    proven_size_t       cursor;  /* how far in we are */
} proven_scan_t;
```

Two consequences worth stating, because they are what make this different from `scanf`:

- **The scanner never allocates and never writes into your input.** A scanned word comes back as a `proven_u8str_view_t` pointing *into* the original bytes. It is valid exactly as long as those bytes are, and no longer. If it must outlive them, copy it with `proven_u8str_create_from_view()`.
- **The cursor is yours.** It is a plain field. You may save it, restore it, or step it by hand (section 12 does exactly that after `proven_scan_skip_until`). Nothing in the scanner is hidden from you, so nothing has to be undone for you.

`proven_scan_init()` normalizes a malformed view (`size > 0` with a null pointer) to an empty one rather than trusting it, so a scanner built from garbage reads as end-of-input instead of dereferencing.

Each primitive returns a result struct pairing the value with the error that guards it: `proven_result_i64_t`, `proven_result_u64_t`, `proven_result_f64_t`, `proven_result_u8str_view_t`. **The value is meaningless unless the error is `PROVEN_OK`** - the scanner does not use a sentinel value to mean failure, because every sentinel is also a legitimate input.

8. Scanner primitive APIs

```
void                       proven_scan_skip_whitespace(proven_scan_t *scan);
proven_result_i64_t        proven_scan_i64(proven_scan_t *scan);
proven_result_u64_t        proven_scan_u64(proven_scan_t *scan);
proven_result_i64_t        proven_scan_i64_hex(proven_scan_t *scan);
proven_result_u64_t        proven_scan_u64_hex(proven_scan_t *scan);
proven_result_f64_t        proven_scan_f64(proven_scan_t *scan);
proven_result_u8str_view_t proven_scan_str(proven_scan_t *scan);
proven_err_t               proven_scan_skip_until(proven_scan_t *scan, proven_u8str_view_t target);
void                       proven_scan_skip_until_number(proven_scan_t *scan);
```

The value-returning ones are `[[nodiscard]]`: a scan whose result you throw away is a scan you did not need to make.

Shared behaviour

- **Leading whitespace is skipped** by every value scanner. You do not need to call `proven_scan_skip_whitespace()` first; it exists for when you want to position the cursor yourself.
- **Scanning stops at the first byte that cannot belong to the value.** `"12abc"` yields `12` and leaves the cursor on the `a`. That is not an error - the scanner answered the question you asked and left the rest for whoever asks next.
- **On failure the cursor is restored**, so a failed scan is a non-event: you can turn around and parse the same position as something else. section 12 does this.

The integer scanners

Input	`proven_scan_i64`	Why
`"42"`, `"+42"`, `"-42"`	`OK` - 42, 42, -42	a sign is part of the number

`"9223372036854775808"`	`PROVEN_ERR_OVERFLOW`	one past `INT64_MAX`; it does **not** wrap
`"abc"`, `""`	`PROVEN_ERR_INVALID_ARG`	there is no number here
`"0x10"`	`OK` - **0**, cursor at 1	**decimal only**: a zero, followed by text

That last row is the one that surprises people. `proven_scan_i64` and `proven_scan_u64` read decimal: no octal, no base prefix. `0x10` is the integer zero, and `x10` is still in the input.

Hexadecimal is a separate, explicit request. `proven_scan_u64_hex` reads hex digits in either case, with an optional `0x` taken only when a digit follows it (as `strtoul` takes it, so `"0xg"` is zero followed by `"xg"`); `proven_scan_i64_hex` adds a sign. In a format string, `{:x}` (or `{:X}`) reads any integer argument the same way - the formatter's spec for writing hex, read backwards. Overflow, cursor restore and the stream signal behave as for decimal.

```
/* A UnicodeData-style range, "0041..005A", read as two hexadecimal numbers. */
proven_u32 first = 0, last = 0;
proven_err_t e = proven_scan_fmt(PROVEN_LIT("0041..005A"), "{:x}..{:x}", PROVEN_SCAN_ARG(&first),
PROVEN_SCAN_ARG(&last));
/* e == PROVEN_OK, first == 0x41, last == 0x5A */

/* The primitive, for a hand-written loop: */
proven_scan_t s = proven_scan_init(PROVEN_LIT("0x1F rest"));
proven_result_u64_t h = proven_scan_u64_hex(&s);   /* h.val == 0x1F, cursor on " rest" */
proven_scan_t t = proven_scan_init(PROVEN_LIT("-7f"));
proven_result_i64_t g = proven_scan_i64_hex(&t);   /* g.val == -0x7F: the signed form takes a sign */
(void)e; (void)h; (void)g;
```

`proven_scan_u64` means unsigned: `"-1"` is `PROVEN_ERR_INVALID_ARG`, not a wrap to `18446744073709551615`. A scanner that quietly turns a negative number into a huge positive one is how a bounds check gets defeated.

The float scanner

`proven_scan_f64` routes through the same correctly-rounded decimal engine as the rest of the library: round-to-nearest, ties-to-even, no `long double` anywhere. It accepts `nan` and `inf`.

Its two boundary behaviours are **deliberately asymmetric**, and the asymmetry is the point:

- `"1e309"` gives `PROVEN_ERR_OVERFLOW`. There is no correct finite answer, so it refuses rather than handing you an infinity you did not ask for.
- `"1e-400"` gives `PROVEN_OK` and `0.0`, with the sign preserved. Underflow to zero *is* the correctly rounded answer. Reporting it as an error would mean reporting correct arithmetic as a failure.

Words and navigation

`proven_scan_str` returns the next whitespace-delimited run as a view into the input. Nothing left but whitespace is `PROVEN_ERR_INVALID_ARG`.

On a **complete view**, "the input ran out" and "the input is wrong" are the same fact - there is no more input, so a number cut off at the end really is malformed. Over a **stream** they are opposite facts, and the difference decides whether you wait or report an error. The scanner sets `proven_scan_t::needs_more` when the parse ran off the end of what it had, and the buffered scanner uses exactly that to refill and retry: a pipe that delivers `-` and then, a moment later, `12` scans as `-12`. Before, it was a malformed number. A wrong byte that is actually *present* - a letter where a digit belongs - is still an error, and no amount of further input will change that; the scanner does not wait for it.

`proven_scan_skip_until(scan, target)` moves the cursor **to** the target, not past it - you decide how much of it to consume. If the target is not there the result is `PROVEN_ERR_NOT_FOUND` **and the cursor does not move**: the scanner does not consume input it failed to navigate.

`proven_scan_skip_until_number` stops at the first digit, or at a sign immediately followed by a digit. If there is no number it runs the cursor to the end of the input - so check `scan.cursor < scan.view.size` before assuming there is something to read.

9. Scan argument model

A scan argument is a **typed pointer to your destination**, selected by the compiler:

```
PROVEN_SCAN_ARG(&x)     /* _Generic on the pointer type */
```

This is where the scanner differs most sharply from `scanf`. There is no format letter to get wrong, because there is no format letter at all. `%d` against a `long`, or `%s` against a buffer that is too small, are not mistakes available here: the destination's type *is* the specification, and a mismatch is a compile error rather than a corrupted stack.

Supported destinations: `short`, `unsigned short`, `int`, `unsigned int`, `long`, `unsigned long`, `long long`, `unsigned long long`, `double`, and `proven_u8str_view_t`.

`PROVEN_SCAN_ARG_LONG(&x)` and `PROVEN_SCAN_ARG_ULONG(&x)` exist for callers who want to be explicit at the call site; `PROVEN_SCAN_ARG` already handles `long*` and `unsigned long*`.

Narrow destinations are range-checked. Scanning `"70000"` into a `short` is `PROVEN_ERR_OVERFLOW`, not a truncated `4464`. The value is parsed at 64 bits and checked against the destination's range before anything is stored.

The `proven_scan_arg_*` constructors are public if you need to build an argument array by hand, but the macros are what callers use.

10. Structural scan grammar

Why parsing with a format string is more dangerous than printing with one
Formatting with a bad format string produces wrong output. **Parsing with one corrupts memory**, because the format decides what to write through the pointers you passed. `sscanf("%d", &c)` where `c` is a `char` writes four bytes into a one-byte object, and nothing in the call says so.

That asymmetry shapes this section. The structural scan uses the same `{}` grammar as the formatter, but the destination type comes from `PROVEN_SCAN_ARG(&x)` - the same `_Generic` dispatch, so the width written is the width of the object, and a value too large for it is `PROVEN_ERR_OVERFLOW` rather than three neighbouring bytes.

The one property to internalise before using it: **the structural scan is not transactional across placeholders.** If the third `{}` fails, the first two destinations have already been written. That is a deliberate trade - buffering every destination until the whole line parsed would need allocation, and this scanner allocates nothing - but it means a failed `proven_scan_fmt` leaves your variables in a partly-updated state. Either treat them as garbage on failure, or scan into locals and copy out only on success. section 11.1 shows both patterns.

Literals in the pattern are the other half of the grammar and the part people underuse: anything that is not a placeholder must match the input exactly, so `"{}:{}"` on `"12-34"` fails at the literal rather than quietly returning one field.

The scan format string is the formatter's, read backwards:

- a placeholder consumes one argument, in order;
- anything else is a **literal that must match the input exactly**.

One spec exists on the scanning side: `{:x}` (or `{:X}`) reads an integer argument as hexadecimal (section 8); on a float or string argument it is `PROVEN_ERR_INVALID_FORMAT`, as in the formatter. Width, fill and alignment are formatting concerns; the scanner reads what is there.

Whitespace in the format is not special. The value scanners skip leading whitespace themselves, so a format with a space between two placeholders and one without parse `"7 8"` identically - the space in the format matches the space in the input, and had it not been there, the second scanner would have skipped it anyway.

The number of placeholders must equal the number of arguments. Too few values in the input is an error; **too many is not** (section 11.1).

11. Scan formatting APIs

```
proven_scan_fmt(view, fmt, ...)           /* scan a view from the beginning */
proven_scan_fmt_cursor(&scan, fmt, ...)   /* continue from an existing cursor */
proven_err_t proven_scan_fmt_internal(...) /* what the macros expand to */
```

Use `proven_scan_fmt` for a self-contained line. Use `proven_scan_fmt_cursor` when the scan is one step in a longer walk over the same input: it advances the cursor you own, so it mixes freely with the primitives of section 8.

11.1. Scan error code guide and recovery

Code	What actually happened	What to do
`PROVEN_OK`	Every placeholder was filled and every literal matched.	Publish the values.
`PROVEN_ERR_INVALID_ARG`	The input is not the shape you asked for - a placeholder had no value to read, or the input ran out.	The line does not match. Report it; do not retry the same shape.
`PROVEN_ERR_NEED_MORE`	**Buffered scanner only.** The token is cut in half by the read boundary: the rest of it has not arrived yet. You will not normally see this - `proven_sysio_scanner_scan` refills and retries for you - it is what the scanner says to itself.	Nothing. It is handled.
`PROVEN_ERR_NOT_FOUND`	A **literal** in the format did not match.	The line has a different shape than expected. Try another format, from a saved cursor.
`PROVEN_ERR_OVERFLOW`	A number was well-formed but does not fit the destination.	The input may be valid and your destination too narrow, or the input may be hostile. Those are very different situations - tell them apart before widening the type.
`PROVEN_ERR_INVALID_FORMAT`	The format string itself is malformed.	A bug in your code, not in the input.

The structural scanner is not transactional, and this is the one that bites.

When a literal fails to match, the placeholders *before* the mismatch have already been written through. The call returns `PROVEN_ERR_NOT_FOUND` and your destination holds a value anyway - `id` below is 7, from a call that failed:

```
int id = -1;
double ratio = -1.0;
proven_err_t err = proven_scan_fmt(line, "id={} XXX={}",
                                   PROVEN_SCAN_ARG(&id), PROVEN_SCAN_ARG(&ratio));
/* err == PROVEN_ERR_NOT_FOUND, and id == 7: it was written before the failure. */
```

So: **on failure, treat every destination as clobbered.** If you need all-or-nothing, scan into locals and publish them only once the call has succeeded - the worked example in section 12 shows the shape. Alternatively, save `scan.cursor` before the call and restore it afterwards. The cursor is a plain field, and that is deliberate.

Trailing input is not an error. Scanning one placeholder against `"7 8"` succeeds with the value 7 and leaves `8` unconsumed. The scanner matched what you asked for and stopped; it does not police what you did not ask about. If the whole line must be consumed, check the cursor yourself:

```
if (scan.cursor != scan.view.size) { /* there is unparsed input left */ }
```

12. Examples and misuse cases

Worked example: formatting a line and scanning it back
Compiled and run by the test suite. It formats into a stack-borrowed string (atomic on overflow) and into an allocator-backed one (grows), uses a bounded argument for untrusted bytes, and then parses the line back - with the float round-tripping exactly.

```
/*
 * Formatting and scanning are the two halves of the same idea: `{}` renders typed
 * values into text, and the scanner reads text back into typed destinations. Both
 * are type-checked at the call site (_Generic picks the constructor), so there is
 * no format-string/argument mismatch to get wrong at runtime.
 *
 * The choice that matters is *where the bytes go*:
 *
 *   append_fmt       - fixed capacity, atomic. Too long? Nothing is written and
 *                      you get PROVEN_ERR_OUT_OF_BOUNDS. No allocator involved,
 *                      so it works on a stack buffer.
 *   append_fmt_grow  - allocator-backed. Grows to fit; on allocation failure the
 *                      string is left exactly as it was.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- fixed capacity: no allocator, no allocation ------------------------ */
    /* borrow wraps caller memory, so this string lives entirely on the stack. `cap`
     * includes the NUL, so 32 bytes hold 31 of content. Nothing to destroy. */
    proven_byte_t stack_buf[32];
    proven_u8str_t fixed = proven_u8str_borrow(stack_buf, sizeof stack_buf);

    proven_fmt_result_t r = proven_u8str_append_fmt(&fixed, "port={}", PROVEN_ARG(8080));
    EXAMPLE_REQUIRE(PROVEN_FMT_IS_OK(r), "a short line should fit in 32 bytes");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&fixed), PROVEN_LIT("port=8080")),
                    "the fixed-capacity append should have rendered the port");

    /* Atomic means atomic: an append that does not fit changes nothing. The string
     * is still valid and still holds what it held before - no truncated tail to
     * clean up. (Use append_fmt_trunc if a truncated tail is what you want.) */
    proven_fmt_result_t too_long = proven_u8str_append_fmt(
        &fixed, " and a great deal more text than will ever fit here {}", PROVEN_ARG(1));
    EXAMPLE_REQUIRE(too_long.err == PROVEN_ERR_OUT_OF_BOUNDS, "the overlong append must fail");
    EXAMPLE_REQUIRE(too_long.required > too_long.written, "it reports what it would have needed");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&fixed), PROVEN_LIT("port=8080")),
                    "a failed atomic append must leave the string untouched");

    /* --- specs: fill, align, width, hex ------------------------------------- */
    proven_result_u8str_t created = proven_u8str_create(alloc, 8);   /* deliberately small */
    EXAMPLE_REQUIRE(proven_is_ok(created.err), "creating the output string should succeed");
```

```
if (!proven_is_ok(created.err)) return 1;
proven_u8str_t out = created.value;

/* grow reallocates as needed, so the initial capacity is a hint, not a limit.
 * `{:0>4}` = fill '0', align right, width 4. `{:x}` = lowercase hex, no 0x. */
r = proven_u8str_append_fmt_grow(alloc, &out, "id={:0>4} tag={:*^9} addr=0x{:x}",
                                 PROVEN_ARG(7),
                                 PROVEN_ARG(PROVEN_LIT("ok")),
                                 PROVEN_ARG(48879));
EXAMPLE_REQUIRE(PROVEN_FMT_IS_OK(r), "the growing append should succeed");
EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&out),
                                     PROVEN_LIT("id=0007 tag=***ok**** addr=0xbeef")),
                "fill/align/width/hex should render exactly this");
printf("%s\n", proven_u8str_as_cstr(&out));

/* --- untrusted text is bounded, never trusted to be NUL-terminated ------ */
/* PROVEN_ARG on a char* means "walk it until a NUL turns up" - fine for a
 * literal, a buffer-overread for anything that came off a socket. This buffer
 * has no NUL at all; PROVEN_ARG_CSTR_N stops at the length instead, so it reads
 * only what actually exists. Use it for anything you did not create yourself. */
const char untrusted[4] = {'a', 'b', 'c', 'd'};   /* no terminator, on purpose */
EXAMPLE_REQUIRE(proven_is_ok(proven_u8str_reset(&out)), "reset should keep the buffer");
r = proven_u8str_append_fmt_grow(alloc, &out, "payload={}",
                                 PROVEN_ARG_CSTR_N(untrusted, sizeof untrusted));
EXAMPLE_REQUIRE(PROVEN_FMT_IS_OK(r), "the bounded append should succeed");
EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&out), PROVEN_LIT("payload=abcd")),
                "the bounded argument should render its whole 4 bytes and stop");

/* --- format a record, then scan it back --------------------------------- */
proven_i64 sensor_id = 42;
double reading = 3.14159;

EXAMPLE_REQUIRE(proven_is_ok(proven_u8str_reset(&out)), "reset should keep the buffer");
r = proven_u8str_append_fmt_grow(alloc, &out, "{} {} {}",
                                 PROVEN_ARG(sensor_id),
                                 PROVEN_ARG(PROVEN_LIT("boiler")),
                                 PROVEN_ARG(reading));
EXAMPLE_REQUIRE(PROVEN_FMT_IS_OK(r), "formatting the record should succeed");
printf("record: %s\n", proven_u8str_as_cstr(&out));

/* One scanner over one view. Each call advances the cursor past what it
 * consumed, so the calls compose left to right - and each one can fail
 * independently, which is the difference between a parser and a guess. */
proven_scan_t sc = proven_scan_init(proven_u8str_as_view(&out));

proven_result_i64_t id = proven_scan_i64(&sc);
EXAMPLE_REQUIRE(proven_is_ok(id.err), "the first field should parse as an integer");
EXAMPLE_REQUIRE(id.val == sensor_id, "the integer should round-trip");

/* scan_str returns a view *into the scanned string* - it copies nothing and
 * owns nothing, so it is only valid while `out` is. */
proven_result_u8str_view_t name = proven_scan_str(&sc);
EXAMPLE_REQUIRE(proven_is_ok(name.err), "the second field should parse as a word");
EXAMPLE_REQUIRE(proven_u8str_view_eq(name.val, PROVEN_LIT("boiler")), "the name should round-trip");

proven_result_f64_t temp = proven_scan_f64(&sc);
EXAMPLE_REQUIRE(proven_is_ok(temp.err), "the third field should parse as a float");

/* Exactly equal, not approximately: the scanner is correctly rounded, so it
 * returns the nearest double to the text - and the text the formatter produced
```

```
     * (six fractional digits) names this value unambiguously. Bit-for-bit, this is
     * the same double we started with. For a value that needs more than six
     * fractional digits, format it with the shortest policy
     * (proven_float_format_options_shortest) and the same round-trip holds. */
    EXAMPLE_REQUIRE(temp.val == reading, "the float must round-trip exactly, not approximately");

    /* The input is fully consumed: nothing was silently left on the table. */
    proven_result_i64_t extra = proven_scan_i64(&sc);
    EXAMPLE_REQUIRE(!proven_is_ok(extra.err), "there should be nothing left to scan");

    proven_u8str_destroy(alloc, &out);
    return EXAMPLE_OK();
}
```

Worked example: the scanner's error codes, and recovering from them
Compiled and run by the test suite. Every code in the table above is provoked on purpose here, including the non-transactional failure - because a contract you have only read is a contract you have not learned.

```
/*
 * The scanner's error codes, and how to recover from them.
 *
 * The scanner is not scanf. It never writes through a pointer it was not given,
 * it never guesses a width, and it tells you which of several different things
 * went wrong. That last part only helps if you know what the codes mean - so
 * this program provokes each one on purpose.
 */

static proven_u8str_view_t v(const char *s) {
    return proven_u8str_view_from_cstr(s);
}

int main(void) {
    /* --- the primitives restore the cursor when they fail ----------------- */
    /* A failed scan is a non-event: the cursor is where it was, so you can try
     * to parse the same position as something else. */
    {
        proven_scan_t sc = proven_scan_init(v("abc"));
        proven_result_i64_t n = proven_scan_i64(&sc);
        EXAMPLE_REQUIRE(n.err == PROVEN_ERR_INVALID_ARG, "'abc' is not an integer");
        EXAMPLE_REQUIRE(sc.cursor == 0, "a failed integer scan leaves the cursor alone");

        /* So the same position can be read as a word instead. */
        proven_result_u8str_view_t w = proven_scan_str(&sc);
        EXAMPLE_REQUIRE(proven_is_ok(w.err) && proven_u8str_view_eq(w.val, PROVEN_LIT("abc")),
                        "the same bytes parse fine as a word");
    }

    /* --- a number that does not fit is OVERFLOW, not a wrapped value ------ */
    {
        proven_scan_t sc = proven_scan_init(v("9223372036854775808"));   /* INT64_MAX + 1 */
        proven_result_i64_t n = proven_scan_i64(&sc);
        EXAMPLE_REQUIRE(n.err == PROVEN_ERR_OVERFLOW, "one past INT64_MAX must not wrap");
        EXAMPLE_REQUIRE(sc.cursor == 0, "the cursor is restored on overflow too");
    }

    /* --- but a float that underflows is NOT an error ---------------------- */
    /* Too large is OVERFLOW; too small is zero, with the sign kept. That
     * asymmetry is deliberate - underflow to zero is the correctly rounded
     * answer, while overflow has no correct finite answer at all. */
```

```
{
    proven_scan_t big = proven_scan_init(v("1e309"));
    proven_result_f64_t b = proven_scan_f64(&big);
    EXAMPLE_REQUIRE(b.err == PROVEN_ERR_OVERFLOW, "1e309 does not fit a double");

    proven_scan_t tiny = proven_scan_init(v("-1e-400"));
    proven_result_f64_t t = proven_scan_f64(&tiny);
    EXAMPLE_REQUIRE(proven_is_ok(t.err), "1e-400 underflows, which is not an error");
    EXAMPLE_REQUIRE(t.val == 0.0, "it rounds to zero");
}

/* --- the integer scanners are decimal only ---------------------------- */
/* "0x10" is not sixteen. It is a zero, followed by text the scanner has not
 * been asked to look at. This surprises people, so it is worth knowing. */
{
    proven_scan_t sc = proven_scan_init(v("0x10"));
    proven_result_i64_t n = proven_scan_i64(&sc);
    EXAMPLE_REQUIRE(proven_is_ok(n.err) && n.val == 0, "0x10 scans as the integer 0");
    EXAMPLE_REQUIRE(sc.cursor == 1, "and the cursor stops before the 'x'");
}

/* --- scanning stops at the first byte that cannot belong to the value -- */
{
    proven_scan_t sc = proven_scan_init(v("12abc"));
    proven_result_i64_t n = proven_scan_i64(&sc);
    EXAMPLE_REQUIRE(proven_is_ok(n.err) && n.val == 12, "12abc yields 12");
    EXAMPLE_REQUIRE(sc.cursor == 2, "and leaves 'abc' for whoever asks next");
}

/* --- unsigned means unsigned ------------------------------------------ */
{
    proven_scan_t sc = proven_scan_init(v("-1"));
    proven_result_u64_t n = proven_scan_u64(&sc);
    EXAMPLE_REQUIRE(n.err == PROVEN_ERR_INVALID_ARG,
                    "-1 is rejected rather than wrapping to a huge unsigned value");
}

/* --- navigating to a value: skip_until -------------------------------- */
/* skip_until leaves the cursor ON the target, not past it, so you decide
 * how much of it to consume. */
{
    proven_scan_t sc = proven_scan_init(v("port=8080"));
    proven_err_t err = proven_scan_skip_until(&sc, PROVEN_LIT("="));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the '=' is there");
    EXAMPLE_REQUIRE(sc.cursor == 4, "the cursor sits on the '=' itself");

    ++sc.cursor;                                /* step over it */
    proven_result_i64_t port = proven_scan_i64(&sc);
    EXAMPLE_REQUIRE(proven_is_ok(port.err) && port.val == 8080, "the port parses");

    /* Not finding it is NOT_FOUND, and the cursor does not move - the
     * scanner does not consume the input it failed to navigate. */
    proven_scan_t sc2 = proven_scan_init(v("port=8080"));
    proven_err_t missing = proven_scan_skip_until(&sc2, PROVEN_LIT("#"));
    EXAMPLE_REQUIRE(missing == PROVEN_ERR_NOT_FOUND, "there is no '#'");
    EXAMPLE_REQUIRE(sc2.cursor == 0, "and the cursor stayed put");
}

/* --- the structural scanner ------------------------------------------- */
{
```

```
    int id = 0;
    double ratio = 0.0;
    proven_u8str_view_t name = {0};

    proven_err_t err = proven_scan_fmt(v("id=7 ratio=0.5 name=ada"),
                                       "id={} ratio={} name={}",
                                       PROVEN_SCAN_ARG(&id),
                                       PROVEN_SCAN_ARG(&ratio),
                                       PROVEN_SCAN_ARG(&name));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the line matches the shape");
    EXAMPLE_REQUIRE(id == 7 && ratio == 0.5, "the values land in the right places");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(name, PROVEN_LIT("ada")), "including the word");
}

/* --- the structural scanner is NOT transactional ---------------------- */
/*
 * This is the one that bites. When a literal fails to match, the scan
 * returns an error - but the placeholders BEFORE the mismatch have already
 * been written through. `id` is 7 even though the call failed.
 *
 * So: on failure, treat every destination as clobbered. If you need
 * all-or-nothing, scan into locals and only publish them once the call
 * succeeded, which is what the code below does.
 */
{
    int id = -1;
    double ratio = -1.0;
    proven_err_t err = proven_scan_fmt(v("id=7 ratio=0.5"),
                                       "id={} XXX={}",       /* the literal is wrong */
                                       PROVEN_SCAN_ARG(&id),
                                       PROVEN_SCAN_ARG(&ratio));
    EXAMPLE_REQUIRE(err == PROVEN_ERR_NOT_FOUND, "the literal 'XXX=' is not in the input");
    EXAMPLE_REQUIRE(id == 7, "and yet id was already written: the scan is not atomic");

    /* The safe shape: scan into locals, publish on success. */
    int good_id = 0;
    double good_ratio = 0.0;
    int published_id = -1;
    proven_err_t ok = proven_scan_fmt(v("id=7 ratio=0.5"), "id={} ratio={}",
                                      PROVEN_SCAN_ARG(&good_id), PROVEN_SCAN_ARG(&good_ratio));
    if (proven_is_ok(ok)) published_id = good_id;
    EXAMPLE_REQUIRE(published_id == 7, "publish only what a successful scan produced");
}

/* --- running out of input, and having input left over ----------------- */
{
    int a = 0, b = 0;
    proven_err_t short_input = proven_scan_fmt(v("5"), "{} {}",
                                               PROVEN_SCAN_ARG(&a), PROVEN_SCAN_ARG(&b));
    EXAMPLE_REQUIRE(!proven_is_ok(short_input), "two placeholders, one value: that fails");

    /* Trailing input is NOT an error. The scanner matched what you asked for
     * and stopped; it does not police what you did not ask about. If the
     * whole line must be consumed, check that yourself. */
    int only = 0;
    proven_scan_t sc = proven_scan_init(v("7 8"));
    proven_err_t err = proven_scan_fmt_cursor(&sc, "{}", PROVEN_SCAN_ARG(&only));
    EXAMPLE_REQUIRE(proven_is_ok(err) && only == 7, "the first value scans");
    EXAMPLE_REQUIRE(sc.cursor < sc.view.size, "and '8' is still sitting there, unconsumed");
}
```

```
    /* --- narrow destinations are range-checked ---------------------------- */
    {
        short small = 0;
        proven_err_t err = proven_scan_fmt(v("70000"), "{}", PROVEN_SCAN_ARG(&small));
        EXAMPLE_REQUIRE(err == PROVEN_ERR_OVERFLOW,
                        "70000 does not fit a short, and the scanner says so rather than truncating");
    }

    return EXAMPLE_OK();
}
```

Misuse: assuming `0x10` is sixteen
It is zero. The decimal scanners are decimal only, and `x10` is still in the input. If the input is hex, say so: `proven_scan_u64_hex`, or `{:x}` in the format.

Misuse: treating trailing input as an error
It is not one. One placeholder against `"7 8"` succeeds. Check the cursor if you care.

Misuse: trusting destinations after a failed scan
They are clobbered. See section 11.1.

Misuse: keeping a scanned word after its input is gone
`proven_scan_str` returns a **view into the input**, not a copy. When the buffer goes, so does the word. Copy it with `proven_u8str_create_from_view()` if it has to outlive the bytes it came from.

13. Freestanding and build-mode notes

Why float is the one thing that gets compiled out
Everything in this chapter is portable computation except one part, and that part is unusually expensive.

Formatting a `double` correctly - so that the shortest decimal that round-trips is what you get, on every input including subnormals - needs big-integer arithmetic and lookup tables. It is the largest single piece of code in the formatter, and most firmware never prints a `double` at all. So the freestanding profile sets `PROVEN_FMT_NO_FLOAT` and drops it, and a build for a microcontroller does not carry kilobytes of decimal-conversion tables it will never call.

This is a build-time decision rather than a run-time one on purpose: the code is *absent*, not merely unreachable, so the linker cannot be talked into keeping it. section 8a of the freestanding guide covers the related knob - the big-integer capacity - for builds that do want floats on a small target.

The scanner is core: it does no I/O, allocates nothing, and touches no platform layer, so it is available in a freestanding build exactly as it is in a hosted one.

The one build-mode dependency is float. The freestanding build sets `PROVEN_FMT_NO_FLOAT` (which compiles out the float *formatter*) along with `PROVEN_NO_U16STR`. `proven_scan_f64` pulls in the decimal parsing engine (`float_parse.c` and `float_decimal.c`), which is integer-only - no `long double`, no libm, no soft-float helper calls - but it is not free in code size. On a target where that matters and you do not need to read floats, simply do not call it: nothing else in the scanner references the float path.

`proven_sysio_scanner_*` (Chapter 5) is a **different** thing: a buffered scanner over a file, hosted-only, because it does I/O. The scanner described in this chapter reads bytes you already have.

Worked example: wider numbers, loose text, and floating point
The examples above scan rigid formats into `proven_i32`. Three things come up as soon as the input is real, and this example covers them together.

The destination type is the contract. `proven_scan_arg_i64()`, `proven_scan_arg_u32()` and `proven_scan_arg_u64()` name the type the value is written into, and the scanner reports an overflow rather than wrapping. A byte count that needs more than 32 bits, and a value that must never be negative, are the two cases where the wrong destination type is a bug nobody notices until the numbers get big. The generic `PROVEN_SCAN_ARG()` macro chooses these from the pointer you hand it; the named forms are what it chooses, written out.

Not every input is a rigid format. `proven_scan_skip_whitespace()` advances past spaces, tabs and newlines, and `proven_scan_skip_until_number()` runs the cursor forward to the next digit - or to a sign immediately followed by one, so a negative number keeps its sign. Neither can fail: "there was nothing to skip" is not an error, and at the end of the input the cursor stops instead of running off.

Floating point has a locale problem, and this library does not have it. `strtod()` reads `"3,5"` as `3.5` under one locale and as `3` under another, so the same program disagrees with itself across machines. `proven_parse_double_ascii()` is locale-free by construction: a comma is never a decimal point. It reports how many bytes the number used, so a caller can carry on from there, and it reports unparsable text as an error rather than as a `0` that cannot be told from a real one. `proven_parse_f64_ascii()` is the same function under its earlier name, kept so existing call sites still read correctly.

On the way out, `proven_arg_f64()` is how a floating-point value enters the formatter - both `float` and `double` go through it, so what a program prints does not depend on the width of the variable it was kept in. When the exact spelling matters, `proven_float_format_f32_policy()` and its `f64` sibling take the policy and options explicitly. Shortest mode asks for the fewest digits that read back as exactly this value, which is what a serialiser wants: an exact round trip without printing seventeen digits for numbers that do not need them.

```
#include <string.h>

/*
 * Reading numbers out of text, and writing them back.
 *
 * The text here is the sort a program actually meets: a log line with mixed
 * widths and signs, and a measurements file where the interesting number is
 * buried in prose. That means three jobs the earlier chapter-8 examples do not
 * cover:
 *
 *   - Scanning into types WIDER than int, and into unsigned ones, where the
 *     destination type is the whole question - a byte count that does not fit in
 *     32 bits is the classic overflow nobody notices until the file is 5 GB.
 *   - Positioning the cursor by hand - skipping whitespace, or skipping ahead to
 *     wherever the next number starts - when the input is not a rigid format.
 *   - Converting a decimal string to a double and back, exactly, without going
 *     through the C library's locale-dependent conversions.
 *
 * The float part matters more than it sounds. strtod reads "3,5" as 3.5 in one
 * locale and as 3 in another, and the same program then disagrees with itself
 * across machines. The parser used here is locale-free by construction: a comma
 * is never a decimal point, whatever the environment says.
 */

int main(void) {
    /* --- 1. scanning into the type the value needs ------------------------ */

    /* A log line: a request id that needs 64 bits, a byte count that is never
     * negative and can exceed 4 GB, a small status code, and a negative offset. */
    proven_scan_t scan = proven_scan_init(
        PROVEN_LIT("id=9007199254740993 bytes=5368709120 status=404 delta=-17"));
```

```
proven_i64 id = 0;
proven_u64 bytes = 0;
proven_u32 status = 0;
proven_i32 delta = 0;

/* Each argument constructor names the destination type, so the scanner
 * writes the right width and reports an overflow instead of wrapping. The
 * generic PROVEN_SCAN_ARG() macro picks these for you from the pointer's
 * type; the named forms are what it picks, and what you write when you want
 * the choice visible. */
proven_err_t err = proven_scan_fmt_cursor(&scan, "id={} bytes={} status={} delta={}",
                                  proven_scan_arg_i64(&id),
                                  proven_scan_arg_u64(&bytes),
                                  proven_scan_arg_u32(&status),
                                  proven_scan_arg_i32(&delta));
EXAMPLE_REQUIRE(proven_is_ok(err), "scanning the log line must succeed");
EXAMPLE_REQUIRE(id == 90071992547409 93LL, "a 64-bit id survives, which a 32-bit destination could not");
EXAMPLE_REQUIRE(bytes == 5368709120ULL, "and so does a byte count larger than 4 GiB");
EXAMPLE_REQUIRE(status == 404u, "the small unsigned value reads normally");
EXAMPLE_REQUIRE(delta == -17, "and a signed destination accepts the minus sign");

/* The destination type is a contract, not a hint. A negative number has no
 * unsigned representation, and the scanner refuses rather than wrapping it
 * to something enormous. */
proven_scan_t neg = proven_scan_init(PROVEN_LIT("-17"));
proven_u32 nowhere = 12345;
err = proven_scan_fmt_cursor(&neg, "{}", proven_scan_arg_u32(&nowhere));
EXAMPLE_REQUIRE(err != PROVEN_OK, "a negative value cannot be scanned into an unsigned destination");
EXAMPLE_REQUIRE(nowhere == 12345, "and the destination is left as it was");

/* --- 2. moving the cursor when the input is not rigid ----------------- */

/* Free-form text: the numbers matter, the words between them do not. */
proven_scan_t notes = proven_scan_init(
    PROVEN_LIT("   sample A measured 42 units; sample B measured -8 units"));

/* skip_whitespace advances past spaces, tabs and newlines. It is what you
 * call between fields of a format you are driving by hand - it never fails,
 * because "there was no whitespace" is not an error. */
proven_scan_skip_whitespace(&notes);
EXAMPLE_REQUIRE(notes.cursor == 3, "the three leading spaces are consumed");

/* skip_until_number runs the cursor forward to the first digit, or to a sign
 * immediately followed by one. It is the call that turns "find the number in
 * this line" from a hand-written loop into one statement. */
proven_scan_skip_until_number(&notes);
proven_i32 first = 0;
err = proven_scan_fmt_cursor(&notes, "{}", proven_scan_arg_i32(&first));
EXAMPLE_REQUIRE(proven_is_ok(err) && first == 42, "the first number in the line is 42");

proven_scan_skip_until_number(&notes);
proven_i32 second = 0;
err = proven_scan_fmt_cursor(&notes, "{}", proven_scan_arg_i32(&second));
EXAMPLE_REQUIRE(proven_is_ok(err) && second == -8,
                "and the next one keeps its sign, because the sign is part of the number");

/* At the end there is nothing left to find, and the cursor stops rather than
 * running off: the scan is over, not broken. */
proven_scan_skip_until_number(&notes);
```

```
EXAMPLE_REQUIRE(notes.cursor == notes.view.size, "with no number left, the cursor lands at the end");

/* --- 3. decimal text to double, without a locale ---------------------- */

proven_parse_double_result_t d = proven_parse_double_ascii(PROVEN_LIT("3.14159 rest"));
EXAMPLE_REQUIRE(proven_is_ok(d.err), "parsing a decimal number must succeed");
EXAMPLE_REQUIRE(d.val > 3.14158 && d.val < 3.14160, "and produce the value the digits spell");

/* consumed says where the number ended, so the caller can carry on from
 * there - which is how you parse a list without copying it into pieces. */
EXAMPLE_REQUIRE(d.consumed == 7, "consumed reports exactly the bytes the number used");

/* A comma is not a decimal point here and never will be, whatever locale the
 * program is running under. The number ends at the comma. */
proven_parse_double_result_t comma = proven_parse_double_ascii(PROVEN_LIT("3,5"));
EXAMPLE_REQUIRE(proven_is_ok(comma.err) && comma.consumed == 1 && comma.val == 3.0,
                "a comma ends the number: the parser is locale-free by construction");

/* proven_parse_f64_ascii is the same function under its earlier name, kept
 * for code written before the current one existed. New code should use
 * proven_parse_double_ascii; both are here so an old call site still reads
 * correctly. */
proven_parse_f64_result_t same = proven_parse_f64_ascii(PROVEN_LIT("3.14159 rest"));
EXAMPLE_REQUIRE(same.val == d.val && same.consumed == d.consumed,
                "the compatibility name is the same parser");

/* Text that is not a number at all is an error, not a silent zero - which is
 * what atof() would have handed back with nothing to distinguish it from a
 * genuine "0". */
proven_parse_double_result_t junk = proven_parse_double_ascii(PROVEN_LIT("not a number"));
EXAMPLE_REQUIRE(!proven_is_ok(junk.err), "unparsable text is reported, not turned into 0");
EXAMPLE_REQUIRE(junk.consumed == 0, "and nothing was consumed");

/* --- 4. writing a float back ------------------------------------------ */

/* proven_arg_f64 is how a floating-point value enters the formatter. Both
 * float and double go through it, so the digits a program prints do not
 * depend on the width of the variable it happened to be kept in. */
proven_allocator_t alloc = proven_heap_allocator();
proven_result_u8str_t line = proven_u8str_create(alloc, 64);
EXAMPLE_REQUIRE(proven_is_ok(line.err), "creating the output string must succeed");

proven_fmt_result_t out = proven_u8str_append_fmt(&line.value, "measured {} units",
                                                  proven_arg_f64(d.val));
EXAMPLE_REQUIRE(proven_is_ok(out.err), "formatting the parsed value must succeed");
EXAMPLE_REQUIRE(proven_u8str_view_starts_with(proven_u8str_as_view(&line.value),
                                              PROVEN_LIT("measured 3.14159")),
                "and spell the number it was given");

/* The policy form is the explicit one, for when the caller needs a
 * particular spelling rather than the default. SHORTEST asks for the fewest
 * digits that read back as exactly this value - which is the property a
 * serialiser wants, because it makes the round trip exact without printing
 * seventeen digits for numbers that do not need them. */
char shortest[64];
proven_size_t wrote = 0;
float measured = 0.1f;
proven_err_t ferr = proven_float_format_f32_policy(shortest, sizeof shortest, measured,
                                                   PROVEN_FLOAT_FORMAT_POLICY_RYU,
                                                   proven_float_format_options_shortest(),
```

```
                                                &wrote);
    EXAMPLE_REQUIRE(proven_is_ok(ferr), "formatting a float in shortest mode must succeed");
    EXAMPLE_REQUIRE(wrote > 0 && shortest[wrote] == '\0', "the result is written and terminated");
    EXAMPLE_REQUIRE(strcmp(shortest, "0.1") == 0,
                    "0.1f prints as 0.1: the shortest text that reads back as the same float");

    /* And it round-trips: the text reads back as the value it came from. That is
     * the guarantee shortest mode exists to provide. */
    proven_parse_double_result_t roundtrip = proven_parse_double_ascii(
        (proven_u8str_view_t){ .ptr = (const proven_byte_t *)shortest, .size = wrote });
    EXAMPLE_REQUIRE(proven_is_ok(roundtrip.err), "the shortest form parses back");
    EXAMPLE_REQUIRE((float)roundtrip.val == measured, "as exactly the float it was printed from");

    printf("scanned id=%lld bytes=%llu; formatted %s and %s\n",
           (long long)id, (unsigned long long)bytes, proven_u8str_as_cstr(&line.value), shortest);

    proven_u8str_destroy(alloc, &line.value);
    return EXAMPLE_OK();
}
```

Wrong - scanning a large value into a 32-bit destination:

```
proven_i32 bytes = 0;
proven_err_t e = proven_scan_fmt_cursor(&scan, "bytes={}", proven_scan_arg_i32(&bytes));
/* input says 5368709120 */                                    /* wrong type */
```

The value does not fit. Choosing the destination type by what is convenient, rather than by the range the field can hold, is the same mistake as declaring a file offset `int`.

Wrong - using `atof` or `strtod` for data that crosses machines:

```
double v = strtod(text, NULL);   /* wrong: the decimal point depends on the locale */
```

And `atof` cannot report failure at all: unparsable text becomes `0`, which is indistinguishable from a genuine zero in the data.

Wrong - assuming `skip_until_number` reports "not found":

```
proven_scan_skip_until_number(&scan);
proven_i32 n = 0;
proven_err_t e = proven_scan_fmt_cursor(&scan, "{}", proven_scan_arg_i32(&n));  /* may fail */
```

It moves the cursor and returns nothing. When there is no number left the cursor lands at the end of the input, and it is the scan afterwards that tells you so -- check that error rather than assuming a number was found.

Worked example: naming the argument type yourself

`PROVEN_ARG(x)` and `PROVEN_SCAN_ARG(&x)` choose an argument constructor from the type of what you hand them, and most code never needs to know which one they chose. Two situations take the choice back:

1. **The macro has no type to dispatch on that means what you mean.** A `proven_u8str_view_t`, a broken-down date, a raw address printed for a diagnostic - `proven_arg_str_view()`, `proven_arg_datetime()` and `proven_arg_ptr()` exist because those are deliberate choices, not defaults to fall into.
2. **The argument list is built at run time.** A logging helper whose parameters are already `proven_arg_t` values cannot re-wrap them - a macro that turns a value into an argument has nothing to do with something that is already one. `proven_arg_identity()` and `proven_scan_arg_identity()` are what let the same macro-driven code path accept an argument built earlier.

The example writes both sides out by name: the formatting constructors for a character, a boolean, unsigned 32- and 64-bit values, the three ways of passing text (`proven_arg_str_view()`, `proven_arg_cstr()`, `proven_arg_ucstr()`), a date and a pointer; and the scanning constructors for every plain C integer width (`short`, `int`, `long`, `long long` and their unsigned forms), a `double`, and a string view captured without copying.

The three text constructors differ in how much they trust the caller, and the order is worth remembering:

Constructor	What it is given	What can go wrong
`proven_arg_str_view`	a pointer **and** a length	nothing is scanned for a terminator, so there is none to be missing - prefer this
`proven_arg_cstr`	a NUL-terminated C string	the terminator must be there and the memory must be alive, or the formatter reads past the end
`proven_arg_ucstr`	the same, as `unsigned char *`	exists so a byte buffer does not need a cast that silences a real warning

```
/*
 * Every value handed to the formatter arrives as a proven_arg_t, and every value
 * the scanner writes into arrives as a proven_scan_arg_t. Most of the time you
 * never see either: PROVEN_ARG(x) and PROVEN_SCAN_ARG(&x) pick the right
 * constructor from the type of what you passed, and that is the whole point of
 * them.
 *
 * Two situations take the choice back off the macro, and both are ordinary:
 *
 *   1. The macro cannot see a type it recognises. A `proven_u8str_view_t`, a
 *      broken-down date, a raw address printed for a diagnostic - these need the
 *      constructor named, because there is no plain C type for the macro to
 *      dispatch on that means what you mean.
 *
 *   2. You are building the argument list at RUNTIME. A logging helper that
 *      takes "whatever the caller already assembled" receives proven_arg_t
 *      values, not ints and strings - and a macro that turns a value into an
 *      argument cannot be handed something that is already an argument. That is
 *      what the identity constructors are for: they let the same macro-driven
 *      code path accept an argument that was built earlier.
 *
 * So this example writes both sides out by name. The width and signedness of a
 * destination is not decoration - it is what decides whether 70000 arrives as
 * 70000 or as 4464.
 */

/* A logging helper that takes arguments the caller has already built. Because
 * its parameters are proven_arg_t, PROVEN_ARG would be the wrong tool inside it
 * - the value is already an argument, and proven_arg_identity is what says so. */
static proven_fmt_result_t log_pair(proven_u8str_t *out, const char *fmt,
                                    proven_arg_t a, proven_arg_t b) {
    return proven_u8str_append_fmt(out, fmt, proven_arg_identity(a), proven_arg_identity(b));
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    proven_result_u8str_t line = proven_u8str_create(alloc, 256);
    EXAMPLE_REQUIRE(proven_is_ok(line.err), "creating the output string must succeed");
```

```
    if (!proven_is_ok(line.err)) {
        return 1;
    }
    proven_u8str_t out = line.value;

    /* --- naming the formatting argument yourself -------------------------- */

    /* A single character and a flag. Written by name here so it is visible that
     * a bool prints as true/false rather than as 1/0. */
    proven_fmt_result_t r = proven_u8str_append_fmt(&out, "flag={} mark={}",
                                                    proven_arg_bool(true),
                                                    proven_arg_char('!'));
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "formatting a bool and a char must succeed");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&out), PROVEN_LIT("flag=true mark=!")),
                    "a bool renders as a word, not as a digit");

    EXAMPLE_REQUIRE(proven_is_ok(proven_u8str_reset(&out)), "clearing the line must succeed");

    /* Unsigned widths. The constructor is the declaration of what the value is:
     * an unsigned 32-bit count and an unsigned 64-bit byte total are different
     * facts, and writing them out says which one you have. */
    r = proven_u8str_append_fmt(&out, "files={} bytes={}",
                                proven_arg_u32(1200u),
                                proven_arg_u64(5368709120ULL));
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "formatting unsigned values must succeed");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&out), PROVEN_LIT("files=1200
bytes=5368709120")),
                    "a 64-bit total is printed in full, not truncated to 32 bits");

    EXAMPLE_REQUIRE(proven_is_ok(proven_u8str_reset(&out)), "clearing the line must succeed");

    /* Three ways to give the formatter text, in order of how much they trust
     * the caller:
     *
     *   proven_arg_str_view  - a pointer AND a length. Nothing is scanned for a
     *                          terminator, so there is no terminator to be
     *                          missing. Prefer this.
     *   proven_arg_cstr      - a NUL-terminated C string. The formatter walks it
     *                          looking for the terminator, so it must be there,
     *                          and the memory must still be alive.
     *   proven_arg_ucstr     - the same, for `unsigned char *`, which is what a
     *                          byte buffer is usually typed as. It exists so the
     *                          caller does not have to write a cast that
     *                          silences a real warning.
     */
    const char *name = "report.txt";
    const unsigned char *tag = (const unsigned char *)"draft";
    proven_u8str_view_t note = PROVEN_LIT("first pass");

    r = proven_u8str_append_fmt(&out, "{} [{}] {}",
                                proven_arg_cstr(name),
                                proven_arg_ucstr(tag),
                                proven_arg_str_view(note));
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "formatting the three text forms must succeed");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&out),
                                         PROVEN_LIT("report.txt [draft] first pass")),
                    "all three produce the same kind of output from different kinds of pointer");

    EXAMPLE_REQUIRE(proven_is_ok(proven_u8str_reset(&out)), "clearing the line must succeed");
```

```
    /* A date, and an address. Neither has a plain C type that means what it
     * means: a date is a struct, and an address printed for a diagnostic is a
     * deliberate act rather than something to fall into. */
    proven_datetime_t when = proven_time_breakdown(0);   /* the epoch: a fixed, checkable value */
    int local = 0;

    r = proven_u8str_append_fmt(&out, "at {} object {}",
                                proven_arg_datetime(when),
                                proven_arg_ptr(&local));
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "formatting a date and a pointer must succeed");
    EXAMPLE_REQUIRE(proven_u8str_view_starts_with(proven_u8str_as_view(&out), PROVEN_LIT("at 1970-01-01")),
                    "the epoch breaks down to the first of January 1970");
    EXAMPLE_REQUIRE(proven_u8str_view_find(proven_u8str_as_view(&out), 0, PROVEN_LIT("0x")) !=
PROVEN_SIZE_MAX,
                    "and an address is rendered in hexadecimal");

    EXAMPLE_REQUIRE(proven_is_ok(proven_u8str_reset(&out)), "clearing the line must succeed");

    /* Arguments built here, passed on, and formatted there. */
    r = log_pair(&out, "status={} retries={}", proven_arg_cstr("ok"), proven_arg_u32(3u));
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "formatting pre-built arguments must succeed");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&out), PROVEN_LIT("status=ok retries=3")),
                    "an argument built by the caller formats exactly as one built in place");

    /* --- naming the scanning destination yourself ------------------------- */

    /* On the way in, the constructor names the destination, and the destination
     * decides what "too big" means. These are the plain C types - short, int,
     * long, long long and their unsigned forms - for the very common case where
     * the variable you already have is one of them rather than a fixed-width
     * type. */
    proven_scan_t scan = proven_scan_init(
        PROVEN_LIT("h=-32000 uh=65000 i=-2000000 ui=4000000000 l=-9000000 ul=9000000 "
                   "ll=-9007199254740993 ull=18446744073709551615 f=2.5 word=alpha"));

    short h = 0;
    unsigned short uh = 0;
    int i = 0;
    unsigned int ui = 0;
    long l = 0;
    unsigned long ul = 0;
    long long ll = 0;
    unsigned long long ull = 0;
    double f = 0.0;
    proven_u8str_view_t word = {0};

    proven_err_t err = proven_scan_fmt_cursor(
        &scan, "h={} uh={} i={} ui={} l={} ul={} ll={} ull={} f={} word={}",
        proven_scan_arg_short(&h),
        proven_scan_arg_ushort(&uh),
        proven_scan_arg_int(&i),
        proven_scan_arg_uint(&ui),
        proven_scan_arg_long(&l),
        proven_scan_arg_ulong(&ul),
        proven_scan_arg_llong(&ll),
        proven_scan_arg_ullong(&ull),
        proven_scan_arg_f64(&f),
        proven_scan_arg_str_view(&word));
    EXAMPLE_REQUIRE(proven_is_ok(err), "scanning every plain integer width must succeed");
    EXAMPLE_REQUIRE(h == -32000 && uh == 65000u, "the short forms hold their values");
```

```
    EXAMPLE_REQUIRE(i == -2000000 && ui == 4000000000u, "and so do the int forms");
    EXAMPLE_REQUIRE(l == -9000000L && ul == 9000000UL, "and the long forms");
    EXAMPLE_REQUIRE(ll == -9007199254740993LL, "a value needing 64 bits arrives whole");
    EXAMPLE_REQUIRE(ull == 18446744073709551615ULL, "including the largest unsigned 64-bit value");
    EXAMPLE_REQUIRE(f > 2.4999 && f < 2.5001, "the floating-point destination reads a decimal");

    /* A scanned string view points INTO the text being scanned. Nothing was
     * copied and nothing was allocated, so it is valid exactly as long as that
     * text is - copy it into a proven_u8str_t if it must outlive the scan. */
    EXAMPLE_REQUIRE(proven_u8str_view_eq(word, PROVEN_LIT("alpha")), "the word is captured as a view");

    /* The destination's width is a promise the scanner keeps. 70000 does not fit
     * in a short, so it is refused rather than wrapped to 4464 - which is what a
     * cast would have produced, silently, in a file nobody looks at again. */
    proven_scan_t narrow = proven_scan_init(PROVEN_LIT("70000"));
    short too_small = 7;
    err = proven_scan_fmt_cursor(&narrow, "{}", proven_scan_arg_short(&too_small));
    EXAMPLE_REQUIRE(err != PROVEN_OK, "a value that does not fit the destination is refused");
    EXAMPLE_REQUIRE(too_small == 7, "and the destination keeps the value it had");

    /* The scanning identity constructor, for the same reason as the formatting
     * one: a helper that receives ready-made scan arguments cannot re-wrap them. */
    proven_scan_t again = proven_scan_init(PROVEN_LIT("41"));
    proven_i32 answer = 0;
    proven_scan_arg_t prebuilt = proven_scan_arg_i32(&answer);
    err = proven_scan_fmt_cursor(&again, "{}", proven_scan_arg_identity(prebuilt));
    EXAMPLE_REQUIRE(proven_is_ok(err) && answer == 41, "a pre-built scan argument works unchanged");

    printf("arguments: %s\n", proven_u8str_as_cstr(&out));

    proven_u8str_destroy(alloc, &out);
    return EXAMPLE_OK();
}
```

Wrong - wrapping an argument that is already an argument:

```
static proven_fmt_result_t log_one(proven_u8str_t *out, proven_arg_t a) {
    return proven_u8str_append_fmt(out, "{}", PROVEN_ARG(a));   /* wrong */
}
```

Use `proven_arg_identity(a)`. `PROVEN_ARG` is for values; `a` is already an argument.

Wrong - keeping a scanned string view after the text is gone:

```
proven_u8str_view_t word = {0};
{
    proven_u8str_t line = read_a_line(alloc);
    proven_scan_t s = proven_scan_init(proven_u8str_as_view(&line));
    proven_err_t e = proven_scan_fmt_cursor(&s, "word={}", proven_scan_arg_str_view(&word));
    proven_u8str_destroy(alloc, &line);     /* wrong: `word` pointed into `line` */
}
use(word);
```

A scanned view points into the text being scanned. Copy it into a `proven_u8str_t` if it has to outlive that text.

Chapter 5: Hosted Services

Part V - Talking to the operating system. Prerequisite: Part II (1, 2, 3). After this chapter you can read and write files without losing data on a crash, read input a line at a time, generate randomness that suits the job, and tell wall-clock time from elapsed time.

This chapter covers `fs.h`, `sysio.h`, `mmap.h`, `time.h`, `stream.h`, and `random.h`. Everything in it needs an operating system: this is the only part of the manual that does not apply to a freestanding build.

These APIs require hosted platform support and are excluded from the current freestanding subset.

Table of contents

1. Filesystem API
2. System I/O and environment
3. Memory mapping
4. Time API
5. Examples and misuse cases
6. Walking a tree
7. Streams: writers and readers
8. The standard streams
9. UTF-16 text in and out, and the Windows console
10. Randomness, by use case

1. Filesystem API

The filesystem layer wraps platform file handles, paths, directory listing, metadata, permissions, links, and locks.

Raw filesystem helpers do not sanitize untrusted paths, enforce root confinement, or defend against symlink-race TOCTOU. Callers that accept untrusted paths must validate them before using the API.

Structures and enums

```
typedef struct {
    union {
        void *ptr;
        int fd;
    } internal;
} proven_fs_handle_t;

typedef proven_fs_handle_t proven_file_t;
```

Intent: represent a platform file handle without exposing POSIX or Win32 details to higher layers.

```
typedef enum {
    PROVEN_FS_READ   = 1 << 0,
    PROVEN_FS_WRITE  = 1 << 1,
    PROVEN_FS_APPEND = 1 << 2,
    PROVEN_FS_CREATE = 1 << 3,
    PROVEN_FS_TRUNC  = 1 << 4,
    PROVEN_FS_CREATE_NEW = 1 << 5
} proven_fs_mode_t;
```

File mode flags can be combined. Examples:

- `PROVEN_FS_READ`
- `PROVEN_FS_WRITE ¦ PROVEN_FS_CREATE ¦ PROVEN_FS_TRUNC`
- `PROVEN_FS_WRITE ¦ PROVEN_FS_APPEND ¦ PROVEN_FS_CREATE`

```
typedef enum {
    PROVEN_FS_TYPE_FILE,
    PROVEN_FS_TYPE_DIR,
    PROVEN_FS_TYPE_OTHER
} proven_fs_type_t;

typedef struct {
    proven_u8str_t name;
    proven_fs_type_t type;
    proven_size_t size;
} proven_fs_entry_t;
```

Directory entries own `name`. Use `proven_fs_list_destroy()` for arrays returned by `proven_fs_list()`.

```
typedef struct {
    proven_err_t err;
    proven_file_t value;
} proven_result_file_t;

typedef enum {
    PROVEN_FS_PERM_OWNER_R = 1 << 8,
    PROVEN_FS_PERM_OWNER_W = 1 << 7,
    PROVEN_FS_PERM_OWNER_X = 1 << 6,
    PROVEN_FS_PERM_GROUP_R = 1 << 5,
    PROVEN_FS_PERM_GROUP_W = 1 << 4,
    PROVEN_FS_PERM_GROUP_X = 1 << 3,
    PROVEN_FS_PERM_OTHER_R = 1 << 2,
    PROVEN_FS_PERM_OTHER_W = 1 << 1,
    PROVEN_FS_PERM_OTHER_X = 1 << 0,
    PROVEN_FS_PERM_DEFAULT = PROVEN_FS_PERM_OWNER_R ¦ PROVEN_FS_PERM_OWNER_W ¦
                             PROVEN_FS_PERM_GROUP_R ¦ PROVEN_FS_PERM_OTHER_R
} proven_fs_perms_t;

typedef enum {
    PROVEN_FS_LOCK_SHARED,
    PROVEN_FS_LOCK_EXCLUSIVE,
    PROVEN_FS_LOCK_UNLOCK
} proven_fs_lock_type_t;

typedef struct {
    proven_size_t size;          /* size in bytes (0 for directories on some hosts) */
    proven_fs_type_t type;       /* PROVEN_FS_TYPE_FILE / _DIR - there is no symlink type */
    proven_fs_perms_t perms;     /* the nine permission bits only; the file type is in `type` */
    proven_i64 created_at;       /* always 0: no birth time is queried - see below */
    proven_i64 modified_at;      /* last-modification time, seconds since the Unix epoch */
    unsigned long long dev;      /* device id (POSIX st_dev; 0 where unavailable) */
    unsigned long long ino;      /* inode number (POSIX st_ino; 0 where unavailable) */
    unsigned long long uid;      /* owner id  (POSIX st_uid; 0 on Windows - no uid/gid) */
    unsigned long long gid;      /* group id  (POSIX st_gid; 0 on Windows) */
} proven_fs_stat_t;
```

`uid`/`gid` (added in v26.06.22a) hold the POSIX owner and group ids; both are `0` on Windows, which has no `uid`/`gid` concept. Resolve them to names with the host's `getpwuid`/`getgrgid` when displaying an `ls -l`-style owner/group column.

One field is narrower than it looks, and one has a sharp edge:

- `created_at` is **always 0**. Plain `stat()` has no portable birth time, and the PAL does not ask for one. Only `modified_at` carries a real timestamp.
- `type` is `FILE` for a regular file, `DIR` for a directory, and **`PROVEN_FS_TYPE_OTHER`** for everything else - a FIFO, a socket, a device, a dangling symlink. (Until v26.07.13a all of those stat'd as `FILE`, which told a caller it could open them and read bytes out of them. A dangling symlink cannot be opened at all.)

`type` **follows symlinks**, here and in the directory walk, which is what makes the two agree: a symlink to a regular file is `FILE`, and a symlink to a directory is `DIR`. The edge that follows is real - **a recursive walker can loop**, because a symlink pointing at an ancestor is a cycle and its type says `DIR`. Carry a depth limit, or remember `(dev, ino)` pairs and refuse to descend into one you have seen.

```
proven_fs_stat_t st;
if (proven_is_ok(proven_fs_stat(scratch, PROVEN_LIT("/etc/hosts"), &st))) {
    proven_println("size={} uid={} gid={}",
        PROVEN_ARG(st.size), PROVEN_ARG(st.uid), PROVEN_ARG(st.gid));
}
```

Macro

Macro	Intent
`PROVEN_FS_PATH_SEP`	Preferred library-level separator character, currently `'/'`.

File functions

API	Intent	Return
`proven_fs_open(scratch, path, mode)`	Open a file path. `scratch` is used for path conversion.	`proven_result_file_t`.
`proven_fs_close(file)`	Close file handle.	**Returns an error.** On a file you wrote to, the close *is* part of the write: NFS, CIFS and quota-enforcing filesystems report a failed write-back here and nowhere else. On a file you only read, `(void)`-ing it is fine.
`proven_fs_read(file, dest)`	Single read attempt into mutable slice.	`proven_result_size_t`.
`proven_fs_write(file, src)`	Single write attempt from byte view.	`proven_result_size_t`.
`proven_fs_write_all(file, src)`	Retry until all bytes are written or an error occurs.	`proven_err_t`.
`proven_fs_size(file)`	Query open file size.	`proven_result_size_t`.
`proven_fs_rename(scratch, src, dest)`	Rename or move path, replacing `dest`. `PROVEN_ERR_NOT_FOUND` when `src` is not there.	`proven_err_t`.
`proven_fs_remove(scratch, path)`	Remove a file, or an empty directory (as POSIX `remove()`, on Windows too). A directory that is not empty is `PROVEN_ERR_INVALID_STATE`.	`proven_err_t`.
`proven_fs_copy(temp_alloc, src, dest)`	Copy file using temporary buffer allocation.	`proven_err_t`.
`proven_fs_mkdir(scratch, path)`	Create one directory; its parent must exist. `PROVEN_ERR_EXISTS` when the name is taken,	`proven_err_t`.

	PROVEN_ERR_NOT_FOUND when the parent is missing.	
proven_fs_mkdir_all(scratch, path)	Create a directory and every missing directory above it. A level that is already a directory is not an error; a file in the way is PROVEN_ERR_EXISTS.	proven_err_t.
proven_fs_rmdir(scratch, path)	Remove empty directory. PROVEN_ERR_NOT_FOUND when it is not there; PROVEN_ERR_INVALID_STATE when it is not empty.	proven_err_t.
proven_fs_list(alloc, path)	List directory into proven_array_t of proven_fs_entry_t.	proven_result_array_t.
proven_fs_list_destroy(alloc, list)	Destroy directory listing and entry names.	void.
proven_fs_chmod(scratch, path, perms)	Set permissions. PROVEN_ERR_NOT_FOUND when the name is not there.	proven_err_t.
proven_fs_lock(file, type, wait)	Acquire/release file lock. With wait false, a lock someone else holds is PROVEN_ERR_BUSY.	proven_err_t.
proven_fs_stat(scratch, path, out_stat)	Fill metadata.	proven_err_t.

proven_fs_stat() reports only the nine permission bits in perms, so a stat's perms can be handed straight back to proven_fs_chmod(). It used to carry the raw POSIX st_mode, whose file-type bits chmod rejects - which made that round-trip, the obvious use of the field, fail with PROVEN_ERR_INVALID_ARG for every real file. Read the file type from type.

Position, and the difference between atomic and durable

A handle that cannot seek says so. A pipe, a FIFO or a terminal returns PROVEN_ERR_UNSUPPORTED from proven_fs_seek, not PROVEN_ERR_IO. Not being seekable is a property of the thing, not a failure of the call, and code that adapts to it - the scanner does - has to be able to tell them apart.

pread **and** pwrite **do not move the position.** That is what they are for: two readers sharing one handle cannot race on a cursor that neither of them moves.

Atomic and durable are different promises, and conflating them is how data gets lost.

- proven_fs_write_file_atomic guarantees that a *reader* never sees a half-written file. It says nothing about a power cut: the kernel may still be holding your bytes, and the rename may reach the disk before the data it points at.
- proven_fs_write_file_durable closes that window, in the only order that works: fsync the temp file, **then** rename, **then** fsync the directory. Syncing the file but not the directory leaves a crash window in which the bytes are safe and the name that points at them is not - which is exactly the corruption an atomic write exists to prevent.

The durable form waits for the storage device twice. Use it when losing the write would be worse than the wait, and the atomic form when it would not.

There used to be a `proven_sysio_flush` here that was none of this: it claimed to flush a buffer that did not exist. It is **deleted**. Pushing a buffered writer's bytes to the OS is `proven_writer_flush`; pushing the OS's bytes to the disk is `proven_fs_sync`. Those are two different operations, and one word could not honestly mean both.

Important behavior:

- `proven_fs_read()` and `proven_fs_write()` are single-operation APIs and may process fewer bytes than requested.
- **A read at end-of-file returns `PROVEN_ERR_EOF`, not a zero-byte success.** A loop written the obvious way - `if (r.value == 0) break;` - never takes that branch, and treats the end of the file as an I/O failure instead. Check for `PROVEN_ERR_EOF` explicitly; the worked example at the end of this chapter shows the shape.
- Use `proven_fs_write_all()` when all bytes must be written.
- Zero-size read/write requests should succeed with zero bytes processed without requiring a non-null buffer.
- `proven_fs_is_absolute()` recognizes POSIX absolute paths, Windows drive-root paths, UNC paths, and extended Windows path forms.
- `proven_fs_read_all()` reads to EOF; it does not read to a pre-measured size. The file's reported size only seeds the initial capacity, so a regular file is still read in one allocation and one pass. This matters because `proven_fs_size()` reports 0 for anything that is not a regular file: a FIFO, a character device, or a `/proc` entry has no size that can be known up front, and reading to EOF is the only way to get their contents. It also means a file that grows while it is being read is not silently truncated. `value.size` is always the actual byte count, and an empty source yields `{ .ptr = NULL, .size = 0 }` with `PROVEN_OK`.
- `proven_fs_read_all()` and `proven_fs_read_all_u8str()` need an allocator with a `realloc_fn` if the source outgrows its reported size; a non-growing allocator returns `PROVEN_ERR_UNSUPPORTED` in that case.
- `proven_fs_read_all()` reads until the source ends, with no bound - use it on paths you trust. For a path from outside the program, `proven_fs_read_all_bounded()` refuses a source of more than `max_bytes` bytes with `PROVEN_ERR_OUT_OF_BOUNDS`: a file whose size says so before anything is allocated, and one whose size cannot be known (a FIFO, `/proc`, `/dev/zero`) as soon as the byte after the bound arrives.

```
/* An upload path comes from a request, so it may name /dev/zero, a FIFO that never
 * ends, or a 40 GB file. proven_fs_read_all would read until the allocator gives up;
 * the bounded form stops at the limit and allocates no more than it. */
proven_result_mem_mut_t body = proven_fs_read_all_bounded(alloc, PROVEN_LIT("upload.bin"), 1u << 20);
if (body.err == PROVEN_ERR_OUT_OF_BOUNDS) {
    proven_eprintln("upload.bin is larger than 1 MiB; refused");
} else if (proven_is_ok(body.err)) {
    proven_println("read {} bytes", PROVEN_ARG(body.value.size));
    if (body.value.ptr) alloc.free_fn(alloc.ctx, body.value.ptr);
}
```

- `proven_fs_read_all_u8str()` is the whole-file read most callers want: the result is NUL-terminated, so `proven_u8str_as_view()` and `proven_u8str_as_cstr()` work on it with no second copy. The terminator slot is reserved up front, so it costs no extra allocation. Contents are not validated as UTF-8. Release it with `proven_u8str_destroy()`.
- `proven_fs_write_file()` is not atomic: a reader can observe a partially written file, and a failure mid-write leaves the file truncated. `proven_fs_write_file_atomic()` writes a sibling temp file and renames it over the target, so a concurrent reader sees either the entire old file or the entire new one. It is atomic with respect to readers, not durable across power loss: the rename may reach the disk before the data. When you need durability, ask for it explicitly with `proven_fs_write_file_durable` (or `proven_fs_sync`), described above.

- The temp file is `<path>.pvtmp` plus 13 random characters, so it cannot be predicted: leftovers and names planted by another user never block the write, and a taken name is skipped (`PROVEN_ERR_EXISTS` only after sixteen in a row). A writer killed between creating and renaming it leaves the temp file behind, and proven never deletes one it did not just create - it cannot tell an abandoned file from a write in progress. A long-running service can list the directory and remove old entries for which `proven_fs_is_staging_name()` is true.

```
/* A cleanup pass a service might run at startup: count the staging files the atomic
 * writes of an earlier, killed run left in its state directory. Removing them is the
 * caller's decision - only the caller knows no writer is still filling one. */
proven_result_dir_t dir = proven_fs_dir_open(scratch, PROVEN_LIT("."));
if (proven_is_ok(dir.err)) {
    proven_size_t leftovers = 0;
    proven_fs_dir_entry_t entry;
    proven_err_t e;
    while ((e = proven_fs_dir_next(&dir.value, &entry)) != PROVEN_ERR_EOF) {
        if (proven_is_ok(e) && proven_fs_is_staging_name(entry.name)) ++leftovers;
        else if (e != PROVEN_OK && e != PROVEN_ERR_INVALID_ENCODING) break;
    }
    proven_fs_dir_close(&dir.value);
    proven_println("staging leftovers: {}", PROVEN_ARG(leftovers));
}
```

A read-only destination is refused, by all of them. `proven_fs_write_file`, `proven_fs_write_file_atomic`, `proven_fs_write_file_durable` and `proven_fs_copy` all return `PROVEN_ERR_PERMISSION` when the destination's owner-write bit is clear, and leave the file exactly as it was. That bit is where both platforms record "do not write this file" - mode `0200` on POSIX, the READONLY attribute on Windows.

It is one rule because it used to be three answers to one question, on one platform: `write_file` refused, because it opens the destination for writing; `write_file_atomic` succeeded, because `rename` asks the *directory* for permission and never consults the file's mode; and `copy` succeeded **and left the file writable afterwards**, because it carries the source's mode across. A protection the caller had set disappeared and nothing said so. Which function a caller reaches for is not a decision about permissions.

A caller who means to replace a protected file lifts the mark first - one line, and visible. The refusal is deliberately the recoverable direction; replacing a protected file by accident is not. It is a guard against accidents, not a security boundary: the mode is read before the work and acted on after it, and anyone who can `chmod` the file can lift the mark.

`proven_fs_rename` is on that list, and has to be: it replaces the destination, and it is what the atomic write is built on - without it, a caller refused by one function got the same result from the other. `proven_fs_remove` is deliberately **not** on it: deleting a name is a directory operation and POSIX has never let the file's own mode have a say. The platforms differ there - Windows will not delete a read-only file - so `proven_fs_remove` reports that as `PROVEN_ERR_PERMISSION` rather than hiding it behind an I/O error.

A refusal now says *which* refusal it is. `proven_fs_open`, `proven_fs_rename` and `proven_fs_remove` answer `PROVEN_ERR_NOT_FOUND`, `PROVEN_ERR_PERMISSION` or `PROVEN_ERR_BUSY` where they used to answer `PROVEN_ERR_IO` for all of it. Asking the user, retrying, and giving up are three different answers, and one error code supports none of them.

The directory, mode, link and lock calls follow the same rule. `proven_fs_mkdir` and `proven_fs_link` answer `PROVEN_ERR_EXISTS` when the name they were asked to create is taken; `proven_fs_mkdir`, `proven_fs_rmdir`, `proven_fs_chmod`, `proven_fs_link` and `proven_fs_rename` answer `PROVEN_ERR_NOT_FOUND` for a name, or a parent directory, that is not there, and `PROVEN_ERR_PERMISSION` when the platform refuses the caller; `proven_fs_lock` asked not to wait answers `PROVEN_ERR_BUSY` for a lock someone else holds. Two

refusals are named with codes that already had the right meaning. A directory that is not empty answers `PROVEN_ERR_INVALID_STATE` to `proven_fs_rmdir` and to `proven_fs_remove`: nothing failed, and the fix is the order of the calls - empty it first. A hard link or a rename that would have to cross from one file system to another answers `PROVEN_ERR_UNSUPPORTED`: that route does not exist, and the caller takes another - a copy, then a remove.

`proven_fs_mkdir` creates one level and reports a name that is taken, because for one level that is information. A caller that wants a whole path to exist, and does not care who made it, calls `proven_fs_mkdir_all`: it creates every missing level, treats a level that is already a directory as done, and still refuses - with `PROVEN_ERR_EXISTS` - when a level is something other than a directory. Directories it created before a failure stay.

Wrong - reading every refusal of `proven_fs_mkdir` as a failure:

```
proven_err_t e = proven_fs_mkdir(alloc, dir);
if (!proven_is_ok(e)) return e;                    /* wrong: stops when dir is already there */
```

A program run twice fails the second time, at a directory it made itself. Either ask for what is meant - `proven_fs_mkdir_all`, "this path is a directory when I return" - or accept `PROVEN_ERR_EXISTS` on purpose.

Wrong - accepting `PROVEN_ERR_EXISTS` without asking what exists:

```
proven_err_t e = proven_fs_mkdir(alloc, dir);
if (e == PROVEN_ERR_EXISTS) e = PROVEN_OK;         /* wrong: a FILE named dir also answers this */
```

`proven_fs_mkdir` does not look at what holds the name. A file of that name gives the same answer, and the write into `dir/...` that follows fails somewhere less clear. `proven_fs_mkdir_all` asks, and returns `PROVEN_OK` only for a directory.

A file someone is reading is still replaced. When another process holds the destination open but allowed delete sharing - `proven_fs_open` does - the atomic write succeeds, and the reader keeps reading the old contents through the handle it already has. POSIX has always done this; Windows 10 1809 and later do the same through the POSIX-semantics rename. Where that rename is unknown - Windows before 1809, FAT32, exFAT, network drives - the library falls back to `MoveFileExW`, which refuses the replacement while anyone has the file open, and the answer is `PROVEN_ERR_BUSY`. A holder that did not allow delete sharing gets the same `PROVEN_ERR_BUSY` on every Windows. It is in use, not protected: a retry may succeed.

Example:

```
proven_result_file_t of = proven_fs_open(
    alloc,
    PROVEN_LIT("out.txt"),
    PROVEN_FS_WRITE | PROVEN_FS_CREATE | PROVEN_FS_TRUNC
);
if (proven_is_ok(of.err)) {
    proven_err_t e = proven_fs_write_all(
        of.value,
        proven_mem_view_from_u8(PROVEN_LIT("hello\n"))
    );
    /* The close is part of the write. Close on the failure path too - the handle is
     * ours either way - but do not throw the answer away: on a network filesystem or
     * over quota, close() is the only place the failure appears. */
    proven_err_t ce = proven_fs_close(of.value);
    if (proven_is_ok(e)) e = ce;
    if (!proven_is_ok(e)) {
        proven_eprintln("writing out.txt failed");
    }
}
```

2. System I/O and environment

Standard streams

```
proven_file_t proven_sysio_stdin(void);
proven_file_t proven_sysio_stdout(void);
proven_file_t proven_sysio_stderr(void);
```

Purpose: expose the standard streams as `proven_file_t` handles. They are also writers and readers - see The standard streams below, which is what lets you read stdin a line at a time, buffer stdout, and format straight into either.

proven_sysio_scanner_t

```
typedef struct {
    proven_file_t file;
    proven_allocator_t alloc;
    proven_u8 *buffer;
    proven_size_t capacity;
    proven_size_t cursor;
    proven_size_t length;
    bool eof;
} proven_sysio_scanner_t;
```

Purpose: buffered scanner for bounded stream input, safe for pipes and stdin as well as seekable files. When a token reaches the end of the currently loaded fragment before EOF, the scanner refills the buffer and retries; only a token that cannot fit inside the whole buffer even after a refill is rejected, with `PROVEN_ERR_OUT_OF_BOUNDS`, instead of being accepted truncated.

Sysio functions and macros

API	Intent	Return
`proven_sysio_scanner_init(scanner, file, alloc, buffer_capacity)`	Allocate scanner buffer and bind stream.	`proven_err_t`.
`proven_sysio_scanner_deinit(scanner)`	Free scanner buffer.	void.
`proven_sysio_scanner_scan_impl(scanner, fmt, args, args_count)`	Internal scanner engine.	`proven_err_t`.
`proven_sysio_scanner_scan(scanner, fmt, ...)`	Type-safe buffered scan macro.	`proven_err_t`.
`proven_sysio_print_impl(handle, fmt, args, args_count)`	Internal printing engine.	`proven_err_t`.
`proven_sysio_scan_chunk_impl(handle, fmt, args, args_count)`	One-chunk scan engine.	`proven_err_t`.
`proven_print(fmt, ...)`	Print to stdout.	`proven_err_t`.
`proven_println(fmt, ...)`	Print to stdout with newline.	`proven_err_t`.
`proven_eprint(fmt, ...)`	Print to stderr.	`proven_err_t`.
`proven_eprintln(fmt, ...)`	Print to stderr with newline.	`proven_err_t`.
`proven_scan_fmt_from_file(file, fmt, ...)`	Scan from one fixed-size chunk of file.	`proven_err_t`.
`proven_scan_fmt_from_stdin(fmt, ...)`	Scan one fixed-size chunk from stdin.	`proven_err_t`.
`proven_env_get(alloc, key)`	Read environment variable into owned U8 string.	`proven_result_u8str_t`.

`proven_sysio_scan_chunk_impl()` is intended for seekable file inputs. It reads at most one fixed-size chunk. If the handle cannot be rewound, it returns `PROVEN_ERR_UNSUPPORTED` before reading. If the

chunk fills before a complete token is available, it returns `PROVEN_ERR_OUT_OF_BOUNDS` and rewinds the file cursor to the start of the chunk. It also refuses string-view destinations with `PROVEN_ERR_UNSUPPORTED` before reading: such a view would borrow the helper's local chunk buffer and dangle on return. Use the caller-owned `proven_sysio_scanner_t` for repeated buffered scanning or borrowed string results. A string view returned by that scanner remains valid only until the next scan call or scanner deinitialization, because either operation may refill, compact, or free the shared buffer.

Example:

```
proven_println("answer={}", PROVEN_ARG(42));
proven_eprintln("warning: {}", PROVEN_ARG(PROVEN_LIT("low memory")));
```

Environment example. `proven_env_get` hands back an owned string, so it has to be destroyed. A variable set to the empty string is an empty string, not an error, and a value that is not valid text (bytes that are not UTF-8 on POSIX, a lone surrogate on Windows) is `PROVEN_ERR_INVALID_ENCODING` - nothing is substituted. The same rule holds for directory entry names:

```
proven_result_u8str_t env = proven_env_get(alloc, PROVEN_LIT("PATH"));
if (proven_is_ok(env.err)) {
    proven_u8str_view_t path = proven_u8str_as_view(&env.value);
    proven_println("PATH is {} bytes", PROVEN_ARG(path.size));
    proven_u8str_destroy(alloc, &env.value);
}
```

3. Memory mapping

The problem: reading a file you do not want to copy

`proven_fs_read_all_u8str` reads a file into memory you own. For a configuration file that is exactly right. For a 4 GB database, a memory-mapped index, or a file two processes need to see at once, it is wrong in three ways: it needs 4 GB of RAM, it copies every byte whether or not you read them, and the copy is yours alone.

A memory mapping asks the operating system to make the file's contents *appear* at an address instead. Nothing is copied up front. The pages you touch are read from disk on demand; the ones you never touch are never read. Two processes mapping the same file with `PROVEN_MMAP_SHARED` see one set of pages, so a write in one is visible in the other.

What this costs, and when not to use it

This is the section of the manual where the trade is sharpest, because the failure modes do not look like errors:

- **A read can fault.** With a file in memory, an I/O error is a return value. With a mapping, touching a page whose disk read fails delivers a **signal** (`SIGBUS`), not an error code. There is no `if` you can write around it.
- **Truncation is a landmine.** If another process shortens the file after you map it, touching a page past the new end is also `SIGBUS`. Mapping a file that anything else may truncate is unsafe in a way no API can fix for you.
- **It is not free.** Setting up a mapping is a syscall and page-table work; each first touch is a page fault. For a small file, `proven_fs_read_all_u8str` is simply faster.
- `proven_mmap_sync` **is the durability point.** Writes to a shared mapping reach the file eventually; if you need them on disk *now*, ask.

Use a mapping for large files you read sparsely, for read-only data shared between processes, and for random access into a big file. Use an ordinary read for everything else.

The mapping is caller-owned state: `proven_mmap_as_view` hands you a view **into the mapping**, so that view is dead the moment `proven_mmap_destroy` runs.

Wrong - using the view after destroying the mapping:

```
proven_u8str_view_t data = proven_mmap_as_view(m);
proven_err_t e = proven_mmap_destroy(&m);
(void)e;
parse(data);                /* wrong: those addresses are no longer mapped - SIGSEGV */
```

Wrong - treating a mapping like a buffer you can grow:

```
/* wrong: a mapping is a window onto a file of a fixed size at map time.
   Appending means changing the file and mapping it again. */
```

Structures and enums

```
typedef enum {
    PROVEN_MMAP_READ  = 0x01,
    PROVEN_MMAP_WRITE = 0x02,
    PROVEN_MMAP_EXEC  = 0x04
} proven_mmap_prot_t;

typedef enum {
    PROVEN_MMAP_PRIVATE = 0x01,
    PROVEN_MMAP_SHARED  = 0x02
} proven_mmap_flags_t;

typedef struct {
    void *ptr;
    proven_size_t size;
    proven_fs_handle_t file;
    void *internal_handle;
} proven_mmap_t;

typedef struct {
    proven_err_t err;
    proven_mmap_t value;
} proven_result_mmap_t;
```

Functions

API	Intent	Return
`proven_mmap_create(file, offset, size, prot, flags)`	Map a file region. `size == 0` maps until EOF. Offset must match the platform mapping granularity.	`proven_result_mmap_t`.
`proven_mmap_destroy(mmap)`	Unmap region and clear state.	`proven_err_t`.
`proven_mmap_sync(mmap)`	Flush shared writable changes to storage.	`proven_err_t`.
`proven_mmap_as_view(mmap)`	Borrow mapped memory as U8 view.	`proven_u8str_view_t`.

Example:

```
proven_result_file_t f = proven_fs_open(alloc, PROVEN_LIT("data.bin"), PROVEN_FS_READ);
if (proven_is_ok(f.err)) {
    proven_result_mmap_t mr = proven_mmap_create(
        f.value,
        0,
        0,
        PROVEN_MMAP_READ,
        PROVEN_MMAP_PRIVATE
    );
    if (proven_is_ok(mr.err)) {
        /* The view borrows the mapping: it dies when the mapping is destroyed. */
```

```
        proven_u8str_view_t bytes = proven_mmap_as_view(mr.value);
        proven_println("mapped {} bytes", PROVEN_ARG(bytes.size));
        (void)proven_mmap_destroy(&mr.value);
    }
    (void)proven_fs_close(f.value);
}
```

4. Time API

The problem: two different questions, one word
"Time" means two things that look alike and behave nothing alike.

A **wall clock** answers *what time is it?* - the thing a user reads. It is allowed to jump: NTP corrects it, daylight saving shifts it, an administrator sets it. Measure a duration with it and you can get a negative answer, which is a bug that appears on leap-second days and on nobody's laptop during testing.

A **monotonic** clock answers *how long since?* It only moves forward and has no relationship to any calendar. It is what you time an operation with.

libc blurs the distinction. `time()` gives whole seconds of wall clock - too coarse to measure anything. `clock()` measures **CPU** time, so a program that waits on a socket appears to take no time at all. Neither name says which question it answers, and both are routinely used for the other one.

What this library does instead
There are two clocks, and each has one job.

`proven_time_now()` is the **wall clock**: nanoseconds since the Unix epoch as a signed 64-bit value. It has a date, and it is set - by NTP, by an administrator - so it can step in either direction. Break it down or format it.

`proven_time_monotonic_now()` is the **monotonic clock**: nanoseconds from an origin fixed for the life of the process. It never goes backwards and nobody sets it, so the difference of two readings is an elapsed time. It has no date, and a reading means nothing in another process or a later run. Durations, timeouts and deadlines are measured with this one.

`proven_time_breakdown()` turns that number into `proven_datetime_t`, whose fields are the ones a human uses: `month` is **1-12**, not libc's 0-11, and `year` is the actual year, not years since

1. Those two off-by-N conventions in `struct tm` have caused enough bugs to be worth diverging

from deliberately.

Formatting uses the same `{}` placeholders as Chapter 3: the name picks the field and the spec pads it, so `{month:0>2}` is zero-filled to width two. Month and weekday names come from a `proven_time_locale_t`, so rendering another language is passing a different locale rather than setting a global.

Wrong - timing with a wall clock and trusting the sign:

```
proven_time_t t0 = proven_time_now();
do_work();
proven_time_t t1 = proven_time_now();
proven_u64 ns = (proven_u64)(t1 - t0);   /* wrong: an NTP step back makes this enormous */
```

Correct - a duration comes from the monotonic clock, which cannot step:

```
proven_time_t t0 = proven_time_monotonic_now();
do_work();
proven_i64 elapsed_ns = proven_time_monotonic_now() - t0;   /* never negative */
```

If two wall-clock readings are all you have - two timestamps from a log - keep the difference signed, and treat a negative one as "the clock moved", not as a duration.

Wrong - assuming sleep is precise:

```
proven_time_sleep(15);
/* wrong to assume exactly 15ms have passed: sleep guarantees AT LEAST that long,
   and the scheduler decides when you actually run again. */
```

Worked example: a duration, a date, and a formatted timestamp
Compiled and run by the test suite. Note what it does *not* assert - an upper bound on the sleep -- because that would be a test that fails on a busy machine.

```
/*
 * Time comes in two flavours that look identical and are not, and picking the
 * wrong one is the classic timing bug.
 *
 *   - A WALL CLOCK answers "what time is it?". It is what a user wants to see,
 *     and it is allowed to jump: NTP corrects it, daylight saving shifts it, an
 *     administrator sets it. Measuring a duration with it can produce a negative
 *     elapsed time, and did, famously, on leap-second days.
 *
 *   - A MONOTONIC clock answers "how long since?". It only moves forward, at a
 *     steady rate, and has no relationship to any calendar. It is what you time
 *     an operation with.
 *
 * libc blurs this. time() is wall clock in whole seconds - useless for
 * measurement. clock() measures CPU time, not elapsed time, so a program that
 * sleeps looks instantaneous. Neither name tells you which of the two questions
 * it is answering.
 *
 * Here each has its own call. proven_time_now() is the wall clock, nanoseconds
 * since the Unix epoch: it formats as a date. proven_time_monotonic_now() is the
 * monotonic clock, nanoseconds from an origin that means nothing outside this
 * process: two readings subtract to a duration.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- as a duration ------------------------------------------------- */
    proven_time_t start = proven_time_now();            /* the wall clock, for the date below */
    proven_time_t t0 = proven_time_monotonic_now();     /* the monotonic clock, for the duration */
    proven_time_sleep(15);                 /* milliseconds */
    proven_i64 elapsed_ns = proven_time_monotonic_now() - t0;

    EXAMPLE_REQUIRE(elapsed_ns > 0, "the monotonic clock moves forward across a sleep");
    /* Sleep guarantees AT LEAST the requested time, never at most: the scheduler
     * decides when you actually run again. Asserting an upper bound here would
     * be a test that fails on a busy machine, which is why this one does not. */
    EXAMPLE_REQUIRE(elapsed_ns >= 10 * 1000 * 1000,
                    "sleeping 15ms must take at least ~10ms");

    /* --- as a date ----------------------------------------------------- */
    proven_datetime_t dt = proven_time_breakdown(start);
    EXAMPLE_REQUIRE(dt.year >= 2020 && dt.year < 3000, "the epoch breakdown gives a plausible year");
    EXAMPLE_REQUIRE(dt.month >= 1 && dt.month <= 12, "month is 1-12, not 0-11 as in libc's tm");
    EXAMPLE_REQUIRE(dt.day >= 1 && dt.day <= 31, "day is 1-31");
    EXAMPLE_REQUIRE(dt.hour <= 23 && dt.min <= 59 && dt.sec <= 60, "sec allows 60 for leap seconds");
    EXAMPLE_REQUIRE(dt.weekday <= 6, "weekday is 0-6 with 0 = Sunday");

    /* proven_time_now_datetime() is the two calls above in one, for when you
     * only want the calendar form. */
```

```
    proven_datetime_t now = proven_time_now_datetime();
    EXAMPLE_REQUIRE(now.year == dt.year, "both routes read the same clock");

    /* --- formatting a timestamp ------------------------------------- */
    proven_result_u8str_t s = proven_u8str_create(alloc, 64);
    EXAMPLE_REQUIRE(proven_is_ok(s.err), "a 64-byte string is enough for a timestamp");

    /* The locale supplies month and weekday names; proven_time_locale_en is the
     * built-in English one. Pass your own to render other languages. */
    proven_err_t err = proven_time_u8_fmt(alloc, &s.value, dt, &proven_time_locale_en,
                                          "{year}-{month:0>2}-{day:0>2} {hour:0>2}:{min:0>2}:{sec:0>2}");
    EXAMPLE_REQUIRE(proven_is_ok(err), "formatting a datetime should succeed");

    proven_u8str_view_t out = proven_u8str_as_view(&s.value);
    EXAMPLE_REQUIRE(out.size == 19, "year-month-day hour:min:sec is exactly 19 characters");
    EXAMPLE_REQUIRE(out.ptr[4] == '-' && out.ptr[7] == '-' && out.ptr[13] == ':',
                    "the separators land where the pattern put them");

    proven_println("formatted: {}", PROVEN_ARG(out));

    proven_u8str_destroy(alloc, &s.value);
    return EXAMPLE_OK();
}
```

Types

```
typedef proven_i64 proven_time_t;
```

Nanoseconds since UNIX epoch.

```
typedef struct {
    proven_i32 year;
    proven_u8 month;
    proven_u8 day;
    proven_u8 hour;
    proven_u8 min;
    proven_u8 sec;
    proven_u32 ms;
    proven_u8 weekday;
} proven_datetime_t;
```

Field ranges:

- `month`: 1 to 12.
- `day`: 1 to 31.
- `hour`: 0 to 23.
- `min`: 0 to 59.
- `sec`: 0 to 60.
- `ms`: 0 to 999.
- `weekday`: 0 to 6, Sunday is 0.

```
typedef struct {
    const proven_u8str_view_t *month_names;
    const proven_u8str_view_t *month_short_names;
    const proven_u8str_view_t *weekday_names;
    const proven_u8str_view_t *weekday_short_names;
} proven_time_locale_t;
```

`proven_time_locale_en` is the default English locale.

Functions

API	Intent	Return

proven_time_u8_fmt(alloc, str, dt, locale, fmt)	Append formatted datetime to U8 string.	proven_err_t.
proven_time_u16_fmt(alloc, str, dt, locale, fmt)	Append formatted datetime to U16 string unless U16 is disabled.	proven_err_t.
proven_time_now()	Current timestamp in nanoseconds.	proven_time_t.
proven_time_breakdown(time_ns)	Convert epoch nanoseconds to broken-down UTC time.	proven_datetime_t.
proven_time_monotonic_now()	The monotonic clock: nanoseconds from a per-process origin. Subtract two readings for a duration; never a date.	proven_time_t.
proven_time_now_datetime()	Current local broken-down time.	proven_datetime_t.
proven_time_sleep(ms)	Sleep for milliseconds.	void.

Datetime format keys:

- {year}, {month}, {day}, {hour}, {min}, {sec}, {ms}, {wday_num}
- {Month}, {mon} using locale
- {Weekday}, {wday} using locale

Example:

```
proven_result_u8str_t r = proven_u8str_create(alloc, 32);
if (proven_is_ok(r.err)) {
    proven_u8str_t s = r.value;

    proven_datetime_t now = proven_time_now_datetime();
    proven_err_t e = proven_time_u8_fmt(
        alloc,
        &s,
        now,
        &proven_time_locale_en,
        "{year}-{month:0>2}-{day:0>2} {hour:0>2}:{min:0>2}:{sec:0>2}"
    );
    if (proven_is_ok(e)) {
        proven_println("{}", PROVEN_ARG(proven_u8str_as_view(&s)));
    }

    proven_u8str_destroy(alloc, &s);
}
```

5. Examples and misuse cases

Single writes can be partial

Wrong:

```
proven_result_size_t w = proven_fs_write(file, data);
/* wrong: one write may be partial, and w.value is never looked at */
```

Correct:

```
proven_result_file_t f = proven_fs_open(alloc, PROVEN_LIT("out.txt"),
                                        PROVEN_FS_WRITE | PROVEN_FS_CREATE | PROVEN_FS_TRUNC);
if (proven_is_ok(f.err)) {
    proven_mem_view_t data = proven_mem_view_from_u8(PROVEN_LIT("payload"));
    proven_err_t e = proven_fs_write_all(f.value, data);   /* loops until done */
    proven_err_t ce = proven_fs_close(f.value);            /* and the close can fail too */
    if (proven_is_ok(e)) e = ce;
```

```
    (void)e;
}
```

Directory listings need special destruction

Wrong:

```
proven_result_array_t r = proven_fs_list(alloc, PROVEN_LIT("."));
PROVEN_ARRAY_DESTROY(&r.value); /* wrong: entry names leak */
```

Correct:

```
proven_result_array_t r = proven_fs_list(alloc, PROVEN_LIT("."));
if (proven_is_ok(r.err)) {
    proven_fs_list_destroy(alloc, &r.value);
}
```

Do not use a mapping after destroy

Wrong:

```
proven_mmap_destroy(&map);
use_bytes(map.ptr, map.size); /* wrong: mapping has been released */
```

Use buffered sysio scanning for streams

For repeated reading from stdin or pipes, prefer:

```
proven_sysio_scanner_t scanner = {0};
proven_err_t e = proven_sysio_scanner_init(&scanner, proven_sysio_stdin(), alloc, 4096);
if (proven_is_ok(e)) {
    int value = 0;
    e = proven_sysio_scanner_scan(&scanner, "{}", PROVEN_SCAN_ARG(&value));
    proven_sysio_scanner_deinit(&scanner);
}
```

Environment values are owned strings

Wrong:

```
proven_result_u8str_t env = proven_env_get(alloc, PROVEN_LIT("PATH"));
use_path(proven_u8str_as_view(&env.value)); /* wrong if env.err was not checked, and it leaks */
```

Correct:

```
proven_result_u8str_t env = proven_env_get(alloc, PROVEN_LIT("HOME"));
if (proven_is_ok(env.err)) {
    proven_u8str_view_t home = proven_u8str_as_view(&env.value);
    proven_println("HOME={}", PROVEN_ARG(home));
    proven_u8str_destroy(alloc, &env.value);
}
```

Worked example: reading and writing whole files

Compiled and run by the test suite. This is the whole-file API most callers actually want, including the atomic rewrite that preserves the target's permissions.

```
/*
 * The whole-file API: one call in, one call out. It exists because the
 * open/read-loop/close dance is where most file-handling bugs live - a forgotten
 * close, a partial read treated as EOF, a truncated file left behind by a failed
 * write. If you are reading or writing a file in its entirety, this is the API.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* A relative path in the current directory: the example must not depend on a
     * writable /tmp, and it removes what it creates before returning. */
```

```c
proven_u8str_view_t path = PROVEN_LIT("proven_example_wholefile.tmp");
proven_u8str_view_t text = PROVEN_LIT("first line\nsecond line\n");

/* --- write it in one call ------------------------------------------- */
/* Not atomic: a concurrent reader can see this file half-written. Fine here,
 * because nobody else is looking at it yet. */
proven_err_t err = proven_fs_write_file(alloc, path, proven_mem_view_from_u8(text));
EXAMPLE_REQUIRE(proven_is_ok(err), "writing the whole file should succeed");
if (!proven_is_ok(err)) return 1;

/* --- read it back as raw bytes -------------------------------------- */
/* proven_fs_read_all reads to EOF rather than to a pre-measured size, so it
 * also works on a pipe or a /proc entry, whose size cannot be known up front. */
proven_result_mem_mut_t raw = proven_fs_read_all(alloc, path);
EXAMPLE_REQUIRE(proven_is_ok(raw.err), "reading the whole file should succeed");
if (proven_is_ok(raw.err)) {
    EXAMPLE_REQUIRE(raw.value.size == text.size, "read_all should return every byte written");
    /* The block is plain allocator memory - hand it back to the allocator that
     * produced it. There is no proven_fs_read_all_destroy. */
    alloc.free_fn(alloc.ctx, raw.value.ptr);
}

/* --- read it back as a string --------------------------------------- */
/* This is the one most callers want: the result is NUL-terminated, so it can
 * be handed to a view, to as_cstr, or to the scanner with no second copy. The
 * terminator slot is reserved up front, so it costs no extra allocation. */
proven_result_u8str_t s = proven_fs_read_all_u8str(alloc, path);
EXAMPLE_REQUIRE(proven_is_ok(s.err), "reading the whole file as a string should succeed");
if (!proven_is_ok(s.err)) {
    (void)proven_fs_remove(alloc, path);
    return 1;
}
EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&s.value), text),
                "the file's contents should come back unchanged");
printf("read back %zu bytes: %s", (size_t)proven_u8str_as_view(&s.value).size,
       proven_u8str_as_cstr(&s.value));
proven_u8str_destroy(alloc, &s.value);

/* --- stat, and the perms round-trip --------------------------------- */
proven_fs_stat_t st = {0};
err = proven_fs_stat(alloc, path, &st);
EXAMPLE_REQUIRE(proven_is_ok(err), "stat on a file we just wrote should succeed");
EXAMPLE_REQUIRE(st.type == PROVEN_FS_TYPE_FILE, "a regular file should stat as a FILE");
EXAMPLE_REQUIRE(st.size == text.size, "stat should report the size we wrote");

/* `perms` carries the nine permission bits and nothing else, so it can be fed
 * straight back to chmod. That is the whole point of the field: read a file's
 * mode, and later restore it. (It used to carry the raw POSIX st_mode, whose
 * file-type bits chmod rejects - so this obvious round-trip failed.) */
err = proven_fs_chmod(alloc, path, st.perms);
EXAMPLE_REQUIRE(proven_is_ok(err), "a stat's perms must be accepted back by chmod");

/* Now make the file owner-only, so the next check has something to prove. */
proven_fs_perms_t private_perms = PROVEN_FS_PERM_OWNER_R | PROVEN_FS_PERM_OWNER_W;
err = proven_fs_chmod(alloc, path, private_perms);
EXAMPLE_REQUIRE(proven_is_ok(err), "restricting the file to its owner should succeed");

/* What the platform keeps of that: all nine bits on POSIX, only the owner-write
 * bit (the READONLY attribute) on Windows. The rewrite must keep exactly this. */
proven_fs_stat_t st1 = {0};
```

```
    err = proven_fs_stat(alloc, path, &st1);
    EXAMPLE_REQUIRE(proven_is_ok(err), "stat after chmod should succeed");

    /* --- rewrite it atomically ------------------------------------------ */
    /* A sibling temp file plus a rename: a concurrent reader sees either the whole
     * old file or the whole new one, never a half-written mix. Atomic for readers,
     * not durable across power loss. When it must be, proven_fs_write_file_durable asks. */
    proven_u8str_view_t text2 = PROVEN_LIT("replacement\n");
    err = proven_fs_write_file_atomic(alloc, path, proven_mem_view_from_u8(text2));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the atomic rewrite should succeed");

    proven_fs_stat_t st2 = {0};
    err = proven_fs_stat(alloc, path, &st2);
    EXAMPLE_REQUIRE(proven_is_ok(err), "stat after the atomic rewrite should succeed");
    EXAMPLE_REQUIRE(st2.size == text2.size, "the file should now hold the replacement text");
    /* The rename writes a *new* inode over the old name, so the permissions would
     * be lost unless they were copied across. They are: rewriting a 0600 file does
     * not republish it as 0644. */
    EXAMPLE_REQUIRE(st2.perms == st1.perms,
                    "the atomic rewrite must preserve the target's permissions");

    /* --- clean up ------------------------------------------------------- */
    err = proven_fs_remove(alloc, path);
    EXAMPLE_REQUIRE(proven_is_ok(err), "removing the temp file should succeed");

    return EXAMPLE_OK();
}
```

Worked example: open, read, write, close

Compiled and run by the test suite. Note the read loop: a read at end-of-file returns `PROVEN_ERR_EOF`, not a zero-byte success, so a loop that only checks for zero bytes never terminates the way its author expected.

```
/*
 * The open/read/write/close path, for when the whole-file API (ex_05_fs_wholefile)
 * is not enough: you are streaming, or you want to own the buffer.
 *
 * The one thing to get right here: a single read or write moves *up to* the
 * requested number of bytes, not exactly that many. Treating one short read as
 * end-of-file is the classic way to silently lose the tail of a file.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    proven_u8str_view_t path = PROVEN_LIT("proven_example_stream.tmp");
    proven_u8str_view_t text = PROVEN_LIT("streamed bytes, read back in chunks\n");

    /* --- write ---------------------------------------------------------- */
    /* CREATE makes the file if it is absent; TRUNC empties it if it is not. The
     * allocator is only used to convert the path for the platform call. */
    proven_result_file_t out = proven_fs_open(alloc, path,
                                              PROVEN_FS_WRITE | PROVEN_FS_CREATE | PROVEN_FS_TRUNC);
    EXAMPLE_REQUIRE(proven_is_ok(out.err), "opening the file for writing should succeed");
    if (!proven_is_ok(out.err)) return 1;

    /* write_all loops for us. proven_fs_write does one attempt and may write less,
     * which is almost never what a caller means. */
    proven_err_t err = proven_fs_write_all(out.value, proven_mem_view_from_u8(text));
```

```
/* The close is part of the write, and on a network or quota-enforced filesystem it is
 * the ONLY place the failure appears: the bytes were buffered, write() said yes, and
 * close() is where the disk finally says no. Close on the failure path too - the
 * handle is ours either way - but do not throw the answer away. */
proven_err_t cerr = proven_fs_close(out.value);
if (proven_is_ok(err)) err = cerr;

EXAMPLE_REQUIRE(proven_is_ok(err), "writing the whole buffer should succeed");
if (!proven_is_ok(err)) {
    (void)proven_fs_remove(alloc, path);
    return 1;
}

/* --- read ---------------------------------------------------------------- */
proven_result_file_t in = proven_fs_open(alloc, path, PROVEN_FS_READ);
EXAMPLE_REQUIRE(proven_is_ok(in.err), "opening the file for reading should succeed");
if (!proven_is_ok(in.err)) {
    (void)proven_fs_remove(alloc, path);
    return 1;
}

/* size is a hint for sizing the buffer, not a promise about how many bytes any
 * one read will hand over - and it is 0 for anything that is not a regular
 * file (a pipe, a device, a /proc entry). The loop below does not rely on it. */
proven_result_size_t sz = proven_fs_size(in.value);
EXAMPLE_REQUIRE(proven_is_ok(sz.err), "querying the size of an open file should succeed");
EXAMPLE_REQUIRE(sz.value == text.size, "the file should be as long as what we wrote");

proven_byte_t buf[128];
proven_size_t total = 0;

/* The partial-read loop. Each pass asks for whatever is left of the buffer and
 * advances by however much actually arrived: a short read is normal, not the
 * end of the file. The end of the file is a distinct status - PROVEN_ERR_EOF
 * with zero bytes - so the loop terminates on that, and on nothing else. The
 * loop also stops if the source outgrows the buffer; noticing that is the
 * caller's business (here it cannot happen, but a growing file could). */
for (;;) {
    if (total == sizeof buf) break;   /* buffer full: caller decides what to do */

    proven_mem_mut_t dest = { .ptr = buf + total, .size = sizeof buf - total };
    proven_result_size_t r = proven_fs_read(in.value, dest);
    if (r.err == PROVEN_ERR_EOF) break;
    if (!proven_is_ok(r.err)) {
        (void)proven_fs_close(in.value);
        (void)proven_fs_remove(alloc, path);
        EXAMPLE_REQUIRE(false, "reading from the open file should not fail");
        return 1;
    }
    total += r.value;
}

(void)proven_fs_close(in.value);

EXAMPLE_REQUIRE(total == text.size, "the loop should have read every byte in the file");
proven_u8str_view_t got = { .ptr = buf, .size = total };
EXAMPLE_REQUIRE(proven_u8str_view_eq(got, text), "the bytes should come back unchanged");

printf("read %zu bytes in chunks: %.*s", (size_t)total, (int)total, (const char *)buf);
```

```
    /* --- clean up ------------------------------------------------------ */
    err = proven_fs_remove(alloc, path);
    EXAMPLE_REQUIRE(proven_is_ok(err), "removing the temp file should succeed");

    return EXAMPLE_OK();
}
```

Reading a directory one entry at a time

`proven_fs_list` reads the **whole** directory before you see any of it, and it allocates a string for every name. Measured on 50,000 entries: **189 ms, +4.2 MB resident, 50,008 allocations**, with nothing visible until the last entry was read. That is fine for a config directory and useless for a mail spool.

`proven_fs_dir_open` / `_next` / `_close` walks the same directory one entry at a time and **allocates nothing per entry**.

API	Intent	Return
`proven_fs_dir_open(scratch, path)`	Open a streaming iterator. `scratch` is for the path conversion only.	`proven_result_dir_t`.
`proven_fs_dir_next(&dir, &entry)`	The next entry. `PROVEN_ERR_EOF` when there are no more; `PROVEN_ERR_INVALID_ENCODING` for one entry whose name is not valid text, and the next call goes on.	`proven_err_t`.
`proven_fs_dir_close(&dir)`	Release the iterator.	void.

```
typedef struct {
    proven_u8str_view_t name;   /* BORROWED: points into the iterator's own storage,
                                   valid only until the next proven_fs_dir_next */
    proven_fs_type_t    type;   /* FILE / DIR / OTHER - and it follows symlinks */
} proven_fs_dir_entry_t;
```

Use it like this:

```
proven_result_dir_t d = proven_fs_dir_open(alloc, PROVEN_LIT("."));
if (proven_is_ok(d.err)) {
    proven_fs_dir_t dir = d.value;
    proven_fs_dir_entry_t entry;
    for (;;) {
        proven_err_t e = proven_fs_dir_next(&dir, &entry);
        if (e == PROVEN_ERR_EOF) break;
        if (e == PROVEN_ERR_INVALID_ENCODING) continue;   /* a name that is not text: skip it */
        if (!proven_is_ok(e)) break;              /* a real error: report it */
        proven_println("{}", PROVEN_ARG(entry.name));
    }
    proven_fs_dir_close(&dir);
}
```

The name is borrowed, and it dies at the next call. This is what makes the whole thing cost no allocations - and a dangling pointer the moment you keep it.

Wrong:

```
proven_u8str_view_t names[100];
int n = 0;
while (proven_is_ok(proven_fs_dir_next(&dir, &entry)))
    names[n++] = entry.name;    /* wrong: every entry aliases the same storage, and the
                                   next _next overwrites it. All 100 end up equal - to
                                   whatever the last entry happened to be. */
```

Correct: copy the bytes (`proven_u8str_create_from_view`) for the ones you need to keep - or use `proven_fs_list`, which does exactly that for every entry and charges you for it.

`PROVEN_ERR_EOF` **is the end; anything else is a failure.** A loop that stops on "not OK" treats a permission error as a complete listing.

```
while (proven_is_ok(proven_fs_dir_next(&dir, &entry))) { ... }
/* wrong: an I/O error ends the loop exactly like the end of the directory does,
   and you cannot tell whether you saw everything. */
```

Walking a tree

`proven_fs_dir_*` walks ONE directory. `proven_fs_walk` walks a tree - and it is worth saying exactly what it refuses to do, because those refusals are the feature:

It cannot **loop**	It never descends *through* a symlink.
It cannot **escape**	Same rule: a link out of the tree is reported, not followed.
It cannot **lie**	A directory it cannot read comes back as an *error* naming that directory, and the walk goes on. A tree walker that silently skips an unreadable subtree is how a backup misses files and reports success.
It cannot **bloat**	One open handle per *level* of the current path, plus one reused path buffer. Memory is a function of depth, not of how many files there are.

A symlinked directory is still **reported** - it exists, `type` is `DIR`, `is_symlink` is true - it is simply not entered. Hiding it would be its own kind of lie. If you *want* to follow it, you have the path: open a second walk on it, and you own the cycle question.

The structure you receive

```
typedef struct {
    proven_u8str_view_t path;   /* the whole path, from the root you passed in.
                                   BORROWED: it points into the walk's one reused
                                   buffer and is valid only until the next call. */
    proven_u8str_view_t name;   /* the last component of `path`. Same lifetime. */
    proven_fs_type_t    type;   /* FILE / DIR / OTHER - and it FOLLOWS symlinks,
                                   exactly as proven_fs_stat does. */
    proven_size_t       size;   /* bytes, for a regular file; 0 otherwise. */
    proven_size_t       depth;  /* 0 for an entry directly inside the root. */
    bool                is_symlink;  /* reached through a symlink. `type` describes
                                        the TARGET; the walk does not enter it. */
} proven_fs_walk_entry_t;
```

`entry.path` and `entry.name` are borrowed from the walk's one reused buffer and are valid **until the next call**. Copy them if you need them to outlive the step; that is the price of a walk of a million entries costing one allocation instead of a million.

Wrong - the same trap the directory iterator has, for the same reason:

```
proven_u8str_view_t found[100];
int n = 0;
while (proven_is_ok(proven_fs_walk_next(&walk, &entry)))
    found[n++] = entry.path;   /* wrong: every entry aliases ONE buffer. When the loop
                                  ends, all 100 point at whatever the last path was. */
```

Two limits, both of which say so rather than going quiet:

- `max_depth` - how far to descend. A directory *at* the limit is still reported (it is an entry); it is not entered.
- `PROVEN_FS_WALK_DEPTH_LIMIT` (256) - how deep the walk's own stack goes, ever. A directory past it comes back as `PROVEN_ERR_OUT_OF_BOUNDS`, naming the directory.

Compiled and run by the test suite:

```
/*
 * Walking a tree.
 *
 * The three things a recursive walker gets wrong, and what this one does instead:
 *
 *   It loops.       A symlink pointing at an ancestor is a cycle. This walk never descends
 *                   THROUGH a symlink - the symlinked directory is still reported, it is
 *                   simply not entered - so a cycle is impossible, and so is walking out of
 *                   the tree you asked about and into the rest of the filesystem.
 *
 *   It lies.        A directory it cannot read gets skipped, and the walk reports success.
 *                   That is how a backup misses a subtree. Here the error comes back from
 *                   proven_fs_walk_next, with the entry naming the directory, and the walk
 *                   carries on from the next sibling. You decide what to do about it.
 *
 *   It bloats.      Reading a whole directory into memory before yielding anything makes a
 *                   walk of a big tree cost a big allocation. This one holds one open handle
 *                   per LEVEL of the current path and one reused path buffer - so its memory
 *                   is a function of depth, not of how many files there are.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* A small tree to walk: a file, a directory, a file inside it. */
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_mkdir(alloc, PROVEN_LIT("ex_walk"))), "mkdir");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_mkdir(alloc, PROVEN_LIT("ex_walk/inner"))), "mkdir inner");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_write_file(alloc, PROVEN_LIT("ex_walk/top.txt"),
        proven_mem_view_from_u8(PROVEN_LIT("top")))), "write top.txt");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_write_file(alloc, PROVEN_LIT("ex_walk/inner/deep.txt"),
        proven_mem_view_from_u8(PROVEN_LIT("deep")))), "write deep.txt");

    proven_result_walk_t walk = proven_fs_walk_open(alloc, PROVEN_LIT("ex_walk"),
                                                    PROVEN_FS_WALK_UNLIMITED);
    EXAMPLE_REQUIRE(proven_is_ok(walk.err), "the walk should open");

    proven_size_t files = 0;
    proven_size_t dirs = 0;
    proven_size_t unreadable = 0;
    proven_size_t total_bytes = 0;

    for (;;) {
        proven_fs_walk_entry_t entry = {0};
        proven_err_t err = proven_fs_walk_next(&walk.value, &entry);

        if (err == PROVEN_ERR_EOF) break;

        if (!proven_is_ok(err)) {
            /* A directory that could not be read, or one deeper than the walk's stack. It is
             * REPORTED, not skipped - `entry.path` says which one - and the walk goes on. A
             * tool that copies a tree should fail here; one that reports on a tree should
             * count it and say so. What it must not do is pretend it did not happen. */
            unreadable++;
            continue;
        }

        /* `entry.path` and `entry.name` are borrowed: they point into the walk's one reused
         * buffer and are valid until the next call. Copy them if you need them longer. */
```

```
        if (entry.type == PROVEN_FS_TYPE_DIR) {
            dirs++;
        } else if (entry.type == PROVEN_FS_TYPE_FILE) {
            files++;
            total_bytes += entry.size;
        }
    }

    proven_fs_walk_close(&walk.value);

    EXAMPLE_REQUIRE(files == 2, "two files: top.txt and inner/deep.txt");
    EXAMPLE_REQUIRE(dirs == 1, "one directory: inner");
    EXAMPLE_REQUIRE(unreadable == 0, "and nothing in this tree is unreadable");
    EXAMPLE_REQUIRE(total_bytes == 7, "three bytes plus four");

    /* Depth-limited: max_depth 0 reports what is directly inside the root and descends
     * nowhere. The directory at the limit is still an entry, so it is still reported. */
    walk = proven_fs_walk_open(alloc, PROVEN_LIT("ex_walk"), 0);
    EXAMPLE_REQUIRE(proven_is_ok(walk.err), "the shallow walk should open");

    proven_size_t shallow = 0;
    for (;;) {
        proven_fs_walk_entry_t entry = {0};
        proven_err_t err = proven_fs_walk_next(&walk.value, &entry);
        if (err == PROVEN_ERR_EOF) break;
        if (proven_is_ok(err)) shallow++;
    }
    proven_fs_walk_close(&walk.value);

    EXAMPLE_REQUIRE(shallow == 2, "top.txt and inner - but nothing inside inner");

    (void)proven_fs_remove(alloc, PROVEN_LIT("ex_walk/inner/deep.txt"));
    (void)proven_fs_remove(alloc, PROVEN_LIT("ex_walk/top.txt"));
    (void)proven_fs_remove(alloc, PROVEN_LIT("ex_walk/inner"));
    (void)proven_fs_remove(alloc, PROVEN_LIT("ex_walk"));

    return EXAMPLE_OK();
}
```

Streams: writers and readers

The formatter's only sink used to be a `proven_u8str_t`. A file was a `proven_file_t`. The in-memory scanner read a view; the file scanner read something else again. **Four types, four function families, no common interface** - so you could not write one `serialize(sink, value)` that worked over both memory and a file, you could not format into a file at all, and there was no way to read a file line by line.

A **writer** is a byte sink. A **reader** is a byte source. Both are small vtables passed by value, exactly like `proven_allocator_t`, and for the same reason: the caller decides where the bytes go, and nothing is hidden.

API	Intent
`proven_writer_from_file(&file)`	Unbuffered sink over an open file.
`proven_writer_from_u8str(&state, &str, alloc)`	Appends to an owned string.
`proven_writer_from_buffer(&state)`	Fixed caller memory. Allocates nothing, ever.
`proven_writer_buffered(&state, inner, buf)`	Accumulates small writes; **you** supply the buffer.

`proven_fprint(w, fmt, ...)` / `proven_fprintln`	Format straight into a writer. No allocation.
`proven_reader_from_file(&file)` / `_from_view(&state, view)`	Byte sources.
`proven_reader_buffered(&state, inner, buf)`	Buffered source; required for line reading.
`proven_reader_read_line(&state)`	One line, without the newline.
`proven_writer_is_valid(w)` / `proven_reader_is_valid(r)`	Did the constructor succeed? A zeroed handle is invalid, and every constructor returns one on bad arguments - so this is the check, not a NULL test.
`proven_fwrite_fmt(w, scratch, fmt, ...)`	`proven_fprint` with a scratch buffer **you** size. `proven_fprint` uses a 512-byte stack buffer and returns `OUT_OF_BOUNDS` for a longer line; this is how you format one.
`proven_fmt_to_writer_impl(w, scratch, fmt, args, n)`	The function the two macros above expand to. Call it directly only if you are building your own variadic wrapper; the macros exist so you do not have to count arguments.

Four rules worth stating plainly, because each of them is a way this could have been designed badly:

- **Buffering uses memory you supply.** `proven_writer_buffered` takes a `proven_mem_mut_t`, the way `proven_arena_create` does. There is no hidden global buffer, which means there is also no destructor to flush it for you - **you must flush before the buffer goes out of scope.** In exchange, your logging path never allocates, and a program logging its way out of an out-of-memory condition can still log.
- **A full sink refuses; it does not truncate.** A fixed buffer that fills up returns `PROVEN_ERR_OUT_OF_BOUNDS` and records `overflowed`. A sink that silently drops the end of your data is worse than one that says it cannot take it.
- **A line too long for the reader's buffer is an error**, not a truncated line. A truncated line handed back as if it were whole is a corruption the caller has no way to detect. The buffer is yours; size it for the input you expect.
- **A partial write is a fact, not a failure to be papered over.** A `write_fn` returns a `proven_result_size_t`: how many bytes the sink took, *and* what went wrong. A pipe, a socket, or a filling disk really does accept 4096 of your 6000 bytes and then fail, and a trait that says "consume it all or fail" simply makes such a sink impossible to write correctly. `proven_writer_write` still means all-or-nothing (it loops); `proven_writer_write_partial` is there when you need to see how far you got. The buffered writer keeps only the tail the sink did **not** take - the first version kept the whole buffer and re-sent it, so a failing sink received the accepted prefix twice.
- **A writer that has failed stays failed.** Once a writer has lost bytes - a buffered writer whose sink died, a fixed buffer that overflowed - the stream it was producing has a hole in it that the receiver cannot see, so every later write and flush returns the error. A shorter chunk that *would* fit is refused too: writing it would put it after the hole, and the result would look like complete output. There is no `clear()`: if you have a recovery story it involves a new writer over a new sink, not pretending this one is fine. (Before this, `flush` answered `PROVEN_OK` after a failed write - the buffer was empty, so there was nothing left to fail on - and "write, write, write, check the flush", which is how almost everyone uses a buffered writer, reported success on a full disk.)

A reader's rule is the mirror image: **a read that fails is an error, never an end of file.** `proven_reader_read` returns `PROVEN_ERR_IO`, not a clean zero-byte EOF, when the source breaks - because a file cut short by a disk error and a file that simply ended are the same thing to a caller who cannot tell them apart, and only one of them is safe to act on.

What it costs, measured over 10,000 lines to stdout:

	`write()` syscalls	`malloc()`
`proven_println`	10,000	**0** (was 10,000)
buffered writer, 8 KiB of caller memory	**24**	**0**

The `malloc()` column is for lines that fit the 512-byte stack buffer, which is the ordinary case. A line longer than that falls back to the heap for that one call rather than being refused.

`proven_println` is deliberately still one syscall per line: buffering it would need hidden global state. A caller who wants the 24 builds a buffered writer and says so.

Worked example: one serializer, three destinations, and reading it back
Compiled and run by the test suite. Note that `render_row` does not know where its bytes are going - that is the entire point.

```
/*
 * Writers and readers: one interface for "where bytes go" and one for "where bytes
 * come from".
 *
 * The point is that the code below - render_row - does not know and does not care
 * whether it is writing to a string, to a fixed buffer, or to a file. That was
 * impossible before: the formatter's only sink was a proven_u8str_t.
 */

/* One serializer. It takes a sink, not a destination. */
static proven_err_t render_row(proven_writer_t w, int id, const char *name) {
    proven_fmt_result_t r = proven_fprintln(w, "{:>4} | {}", PROVEN_ARG(id), PROVEN_ARG(name));
    return r.err;
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- the same code, into a growing string ------------------------------ */
    proven_result_u8str_t s = proven_u8str_create(alloc, 16);
    EXAMPLE_REQUIRE(proven_is_ok(s.err), "string create");

    proven_writer_u8str_t s_state;
    proven_writer_t to_string = proven_writer_from_u8str(&s_state, &s.value, alloc);
    EXAMPLE_REQUIRE(proven_is_ok(render_row(to_string, 7, "ada")), "render into a string");

    printf("into a string:\n%s", proven_u8str_as_cstr(&s.value));
    proven_u8str_destroy(alloc, &s.value);

    /* --- the same code, into memory you own: zero allocations ------------- */
    proven_byte_t fixed[64];
    proven_writer_buf_t b_state = { .buf = { .ptr = fixed, .size = sizeof fixed } };
    proven_writer_t to_buffer = proven_writer_from_buffer(&b_state);
    EXAMPLE_REQUIRE(proven_is_ok(render_row(to_buffer, 8, "grace")), "render into a buffer");
    EXAMPLE_REQUIRE(b_state.len > 0, "the buffer received the row");

    /* A full buffer REFUSES; it does not truncate. A sink that silently drops the
     * end of your data is worse than one that says it cannot take it. */
```

```
proven_byte_t tiny[4];
proven_writer_buf_t t_state = { .buf = { .ptr = tiny, .size = sizeof tiny } };
proven_writer_t to_tiny = proven_writer_from_buffer(&t_state);
EXAMPLE_REQUIRE(proven_writer_write_str(to_tiny, PROVEN_LIT("far too long")) == PROVEN_ERR_OUT_OF_BOUNDS,
                "a full buffer refuses rather than truncating");
EXAMPLE_REQUIRE(t_state.overflowed, "and it records that it did");

/* --- the same code, into a file, buffered ------------------------------ */
/*
 * The buffer is memory YOU supply, exactly like an arena's. This library has no
 * hidden global state, so it cannot flush for you at exit - which is why you must
 * flush before the buffer goes out of scope. In exchange, your logging path never
 * allocates: ten thousand lines here cost 0 mallocs and a couple of dozen write
 * syscalls, where ten thousand proven_println calls cost 10,000 syscalls.
 */
proven_u8str_view_t path = PROVEN_LIT("example_stream_rows.txt");
proven_result_file_t f = proven_fs_open(alloc, path,
    (proven_fs_mode_t)(PROVEN_FS_WRITE | PROVEN_FS_CREATE | PROVEN_FS_TRUNC));
EXAMPLE_REQUIRE(proven_is_ok(f.err), "open the output file");
proven_file_t file = f.value;

proven_byte_t out_buf[4096];
proven_writer_buffered_t w_state;
proven_writer_t to_file = proven_writer_buffered(&w_state,
    proven_writer_from_file(&file),
    (proven_mem_mut_t){ .ptr = out_buf, .size = sizeof out_buf });

for (int i = 0; i < 3; ++i) {
    EXAMPLE_REQUIRE(proven_is_ok(render_row(to_file, i, "row")), "render into the file");
}
EXAMPLE_REQUIRE(proven_is_ok(proven_writer_flush(to_file)),
                "flush: nothing is written until you say so");
/* And the close, which is the last thing that can tell you the write did not land. */
EXAMPLE_REQUIRE(proven_is_ok(proven_fs_close(file)), "closing the written file");

/* --- reading it back, a line at a time --------------------------------- */
/* Reading a file line by line was simply not possible before: the only route was
 * loading the entire file into memory and splitting it by hand. */
proven_result_file_t rf = proven_fs_open(alloc, path, PROVEN_FS_READ);
EXAMPLE_REQUIRE(proven_is_ok(rf.err), "reopen for reading");
proven_file_t rfile = rf.value;

proven_byte_t in_buf[128];
proven_reader_buffered_t r_state;
(void)proven_reader_buffered(&r_state, proven_reader_from_file(&rfile),
                             (proven_mem_mut_t){ .ptr = in_buf, .size = sizeof in_buf });

int lines = 0;
for (;;) {
    proven_result_u8str_view_t line = proven_reader_read_line(&r_state);
    if (line.err == PROVEN_ERR_EOF) break;
    EXAMPLE_REQUIRE(proven_is_ok(line.err), "read a line");
    /* The view points INTO the reader's buffer, and is valid only until the next
     * call. Copy it if it has to outlive that. */
    printf("line %d: %.*s\n", lines, (int)line.val.size, (const char *)line.val.ptr);
    ++lines;
}
EXAMPLE_REQUIRE(lines == 3, "three rows in, three lines out");

(void)proven_fs_close(rfile);
```

```
    (void)proven_fs_remove(alloc, path);

    return EXAMPLE_OK();
}
```

The structures you hold
The two **handles** are passed by value and are cheap:

```
typedef struct {
    void *ctx;
    proven_result_size_t (*write_fn)(void *ctx, proven_mem_view_t chunk);
    proven_err_t (*flush_fn)(void *ctx);   /* may be NULL: this sink holds nothing back */
} proven_writer_t;

typedef struct {
    void *ctx;
    proven_result_size_t (*read_fn)(void *ctx, proven_mem_mut_t dest);
} proven_reader_t;
```

`write_fn` reports **how much went out even when it then fails**, and that is not a nicety: a write to a pipe or a full disk really does put some bytes out and then fail. A buffered writer built on the tidier "all or nothing" lie kept its whole buffer on failure and re-sent it on the next flush - a 6000-byte payload arrived as 10,096 bytes with the first 4096 **duplicated**. Losing data is bad; silently doubling it is worse, because the receiver cannot tell.

Everything else is caller-owned state -- `proven_writer_buf_t`, `proven_writer_u8str_t`, `proven_writer_buffered_t`, `proven_reader_view_t`, `proven_reader_buffered_t`. They allocate nothing, they have no destroy, and **they must not be copied or moved** while a handle points into them.

Cautions, and what goes wrong
A buffered writer that is never flushed is output that never happened. There is no hidden state, so there is no destructor to flush it for you - and nothing here registers an `atexit` handler, because a library that owns your process is a library you cannot reason about.

Wrong:

```
proven_writer_buffered_t st;
proven_writer_t w = proven_writer_buffered(&st, inner, buf);
(void)proven_fprintln(w, "the important line");
return;                       /* wrong: the buffer dies with the frame, and so does the line */
```

Correct - flush before the buffer or the inner sink goes away:

```
proven_byte_t buf[256];
proven_sysio_out_t out;
proven_writer_t w = proven_sysio_stdout_buffered(&out,
    (proven_mem_mut_t){ .ptr = buf, .size = sizeof buf });
(void)proven_fprintln(w, "the important line");
(void)proven_writer_flush(w);          /* now it has happened */
```

The line a reader hands you points into its buffer. It is valid only until the next call. That is what makes reading a million lines cost one buffer instead of a million allocations - and it is a dangling pointer the moment you keep it.

Wrong:

```
proven_u8str_view_t lines[100];
for (int i = 0; i < 100; ++i) {
    proven_result_u8str_view_t ln = proven_reader_read_line(&st);
    lines[i] = ln.val;        /* wrong: every entry aliases the SAME buffer, and the
                                 next read_line overwrites what the last one returned */
}
```

Correct: copy the bytes you need to keep (into a `proven_u8str_t`, an arena, wherever), before you call again.

A flush is not a durability barrier. `proven_writer_flush` pushes a buffered writer's bytes to the thing behind it; getting a *file's* bytes onto the disk is `proven_fs_sync`. They are different operations, and one word could not honestly mean both - which is why the old `proven_sysio_flush`, which claimed to be both and was neither, is gone.

A line longer than the buffer is refused, not truncated. `PROVEN_ERR_OUT_OF_BOUNDS` - and the reader then stays wedged on that line: there is no resync, because a line you cannot hold is not a line you can skip past without deciding what to do with the bytes. Size the buffer for the input you expect. (A line that *exactly fills* the buffer is fine: the newline does not have to fit too, and neither does a final line with no newline at all.)

The standard streams

`stream.h` has writers, readers, buffered writers and a line reader. `sysio.h` has stdin, stdout and stderr. Until they were introduced to each other, two things were simply not possible - and one call was a lie.

Reading stdin a line at a time had no route. The most common thing a program does with stdin, and the choices were the token scanner or reading the whole of a stream that may never end. The bridge fixes that: a standard handle is parked in caller-owned storage, so the line reader has something stable to point at.

```
proven_byte_t buf[4096];
proven_sysio_lines_t lines;
if (proven_is_ok(proven_sysio_stdin_lines(&lines, (proven_mem_mut_t){ .ptr = buf, .size = sizeof buf }))) {
    for (;;) {
        proven_result_u8str_view_t line = proven_sysio_read_line(&lines);
        if (line.err == PROVEN_ERR_EOF) break;
        if (!proven_is_ok(line.err)) break;   /* OUT_OF_BOUNDS: a line longer than `buf` */
        /* `line.val` points INTO `buf` and is valid only until the next call. */
        proven_println("{}", PROVEN_ARG(line.val));
    }
}
```

It inherits the line reader's properties, which is the point of not writing a second one: the view costs no allocation, `"\r\n"` is handled, a final line with no trailing newline is still returned, and a line longer than your buffer is `PROVEN_ERR_OUT_OF_BOUNDS` - never a silently truncated line.

The formatter could not be aimed at a standard stream. `proven_fprintln` takes a writer; stdout was not one. Now it is, and it can be a *buffered* one - so a thousand small lines cost one syscall instead of a thousand.

`proven_sysio_stdout_writer(&st)`	An unbuffered writer over stdout. Every write is a write syscall.
`proven_sysio_stderr_writer(&st)`	The same for stderr - which is what you want for an error: it is out before the next line of code runs.
`proven_sysio_stdin_reader(&st)`	A reader over stdin.
`proven_sysio_stdout_buffered(&out, buf)`	stdout behind a buffered writer over a buffer you own.
`proven_sysio_file_buffered(&out, file, buf)`	The same over any open file.

And `flush` means something now. `proven_sysio_flush` used to claim to flush a buffer that did not exist: a no-op on POSIX, a *disk sync* on Windows. It is **deleted**. Pushing a buffered writer's bytes to

the OS is `proven_writer_flush`; pushing the OS's bytes to the disk is `proven_fs_sync`. They are different operations and now say so.

> **You must flush a buffered writer.** Nothing reaches the terminal until the buffer fills or you flush it, and nothing in this library registers an `atexit` handler to do it behind your back - a library that owns your process is a library you cannot reason about. Buffered output that is never flushed is output that never happened. The direct calls (`proven_print`, `proven_println`, `proven_eprint`) remain unbuffered for exactly this reason: what they write is on its way out before they return.

The structures you hold

```
typedef struct {
    proven_file_t        file;
    bool                 console;   /* a Windows console: see the next section */
    proven_sysio_carry_t carry;     /* text in flight at the console edge; not yours to touch */
} proven_sysio_std_t;
    /* Storage for a standard handle, so a writer or reader has something stable to
       point at. proven_writer_from_file takes a proven_file_t * and the file must
       outlive the writer - so it cannot be a temporary. This is that storage. */

typedef struct {
    proven_sysio_std_t       std;
    proven_writer_buffered_t buffered;
} proven_sysio_out_t;        /* a buffered writer over a standard stream or a file */

typedef struct {
    proven_sysio_std_t       std;
    proven_reader_buffered_t buffered;
} proven_sysio_lines_t;      /* a line reader over a standard stream or a file */
```

All three are caller-owned state.

Cautions, and what goes wrong

These state structs contain a pointer to themselves. The writer you get back addresses `&st->std.file` *inside* the struct you passed. Copy the struct, or return it by value, and the writer still points at the original - which may be a dead frame.

Wrong - and an audit reproduced exactly this as a heap-use-after-free:

```
proven_sysio_out_t out;
proven_writer_t w = proven_sysio_stdout_buffered(&out, buf);

proven_sysio_out_t copy = out;      /* wrong: `w` still points into `out` */
/* ... `out` goes out of scope ... */
(void)proven_writer_write_str(w, PROVEN_LIT("boom"));   /* writes through dead storage */
```

The one exception is `proven_sysio_lines_t`: `proven_sysio_read_line` re-binds it on every call, so a line reader **may** be moved. That is a deliberate courtesy, because it takes its state by pointer - the shape that says "relocatable" - and the library should not lay a trap in the shape of a promise.

A zero-initialised `proven_file_t` is not an invalid handle - on POSIX it is fd 0, which is stdin. The library cannot tell a handle you forgot to fill in from one that legitimately refers to fd 0.

```
proven_sysio_out_t out;
proven_file_t f = {0};                                  /* wrong: this is stdin */
proven_writer_t w = proven_sysio_file_buffered(&out, f, buf);   /* writes to fd 0 */
```

Buffered stdout and unbuffered stderr do not interleave in the order you wrote them. Anything you buffer sits in your buffer while stderr goes straight out. Flush before you print an error that is supposed to appear after your output.

UTF-16 text in and out, and the Windows console

Two problems share this section because they share a cause: text is not bytes everywhere.

UTF-16 text had no way in or out. A `proven_u16str_t` - the names a Windows wide API hands back, text from a UTF-16 file - could not be printed, written to a file, or read a line at a time. Now u16 text goes through every channel u8 text does. Output goes through the formatter, so `PROVEN_ARG` on a `proven_u16str_view_t` works in `proven_println`, `proven_eprintln`, `proven_fprintln` into a file or any other writer, and `proven_u8str_append_fmt`, and writes UTF-8. When the consumer wants UTF-16 *bytes* - a Windows tool that reads UTF-16LE files, say - `proven_writer_write_u16` writes them in the byte order you name, and a byte order mark only when you ask with `proven_writer_write_bom`.

Input is `proven_u16_reader_t`: a reader over any byte source that decodes UTF-8, UTF-16LE or UTF-16BE into your buffer of code units, a line or a chunk at a time. `PROVEN_TEXT_AUTO` lets a leading byte order mark decide and consumes it; with no mark the text is UTF-8. It carries a character split between two reads of the source, and it keeps the byte line reader's rules: `"\r\n"` loses its `'\r'`, a final line needs no newline, a line longer than the buffer is `PROVEN_ERR_OUT_OF_BOUNDS`. It is strict, like `utf.h`: malformed input stops it with `PROVEN_ERR_INVALID_ENCODING` after the valid text before it has been delivered, and so does a source that ends inside a character.

A Windows console is not a file. It decodes the bytes `WriteFile` gives it in its code page - 949 on Korean Windows - so UTF-8 written to it was mojibake unless someone had run `chcp 65001`, and `ReadFile` on it returned code-page bytes, not UTF-8. Changing the console's code page from inside a library would change it for every other program sharing that console, after this one exits. So the sysio layer does the other thing: when a standard handle is a console (`GetConsoleMode` accepts it), its writers and readers talk to it in UTF-16 through `WriteConsoleW` and `ReadConsoleW`, converting at the edge. That covers `proven_print` and `proven_eprint`, the stdout and stderr writers, the buffered writers, the u8 and u16 line readers and the token scanner. A UTF-8 character split across two writes - a buffered writer flushes at its buffer size, not at a character boundary - is carried in the state struct and completed by the next write. Ctrl+Z at the start of a console line is end of input, as it is for every Windows console program. Files, pipes and redirected streams are not consoles and still receive the exact bytes, and on POSIX nothing is a console.

Reading stdin as UTF-16, a line at a time:

```
#ifndef PROVEN_NO_U16STR
proven_u16 wbuf[512];
proven_sysio_u16_lines_t in;
if (proven_is_ok(proven_sysio_stdin_u16_lines(&in, wbuf, 512))) {   /* stdin is read as UTF-8 */
    for (;;) {
        proven_result_u16str_view_t line = proven_sysio_read_u16_line(&in);
        if (line.err == PROVEN_ERR_EOF) break;
        if (!proven_is_ok(line.err)) break;   /* OUT_OF_BOUNDS, or INVALID_ENCODING */
        /* `line.val` points INTO `wbuf` until the next call - e.g. hand it to a wide API. */
        (void)proven_eprintln("{} code units", PROVEN_ARG(line.val.size));
    }
}
#endif
```

`proven_arg_u16(view)` / `PROVEN_ARG(view)`	u16 text as a formatter argument, written as UTF-8.

`proven_writer_write_u16(w, text, enc)`	u16 text to any writer as UTF-8, UTF-16LE or UTF-16BE. Validated first: malformed text writes nothing.
`proven_writer_write_bom(w, enc)`	The byte order mark for `enc`, only when you ask.
`proven_u16_reader_init(&st, reader, enc, buf, cap)`	A u16 reader over any reader, through `cap` code units of yours (at least 2).
`proven_u16_reader_read_line(&st)`	The next line as a u16 view into your buffer.
`proven_u16_reader_read(&st, dest, cap)`	Up to `cap` code units, never half a surrogate pair.
`proven_sysio_u16_lines_open(&st, file, enc, buf, cap)`	The same reader over a file or a standard stream.
`proven_sysio_stdin_u16_lines(&st, buf, cap)`	Over stdin, read as UTF-8 (a console is read as UTF-16).
`proven_sysio_read_u16_line(&st)`	The next line; the struct may be moved between calls.

The structures you hold

```
typedef struct {
    proven_reader_t inner;  proven_text_encoding_t enc;   /* AUTO resolved from the BOM */
    proven_u16 *buf;  proven_size_t cap, len, cursor;     /* your buffer, in code units */
    proven_byte_t raw[1024];  proven_size_t raw_pos, raw_len;   /* bytes read; [raw_pos, raw_len) not yet
decoded */
    proven_u16 peek[2];  proven_size_t peek_len;          /* one character of lookahead */
    bool bom_checked, eof;  proven_err_t err;             /* err is sticky */
} proven_u16_reader_t;

typedef struct {
    proven_sysio_std_t  std;
    proven_u16_reader_t reader;
} proven_sysio_u16_lines_t;   /* a u16 line reader over a standard stream or a file */
```

Both are caller-owned state. Nothing is allocated: the raw bytes are staged in the struct, the decoded text in your buffer.

Cautions, and what goes wrong

`proven_writer_from_file` on a console handle does not convert. It is a file writer and writes the bytes as they are - mojibake on a Windows console in a non-UTF-8 code page. Make console writers and readers with the `proven_sysio_*` calls, which detect the console.

```
proven_file_t out = proven_sysio_stdout();
proven_writer_t w = proven_writer_from_file(&out);   /* wrong for a console: bytes, not text */
proven_sysio_std_t st;
proven_writer_t ok = proven_sysio_stdout_writer(&st); /* right: converts on a console */
```

Strict means strict on the console too. Malformed UTF-8 sent to a console is refused with `PROVEN_ERR_INVALID_ENCODING` after the valid text before it has been shown. Nothing is replaced with a question mark. A flush is not the end of a text - a buffered writer drains at its buffer size, and you may flush between any two bytes - so the bytes of a character still open at a flush are kept and the next write completes it, or is refused if it cannot. `proven_print` and `proven_eprint` write a whole formatted text, so for them a text that ends inside a character is refused outright.

An explicit encoding does not strip a byte order mark. Opened as `PROVEN_TEXT_UTF16LE`, a file that starts with `FF FE` delivers the character U+FEFF as the first unit of its first line. Open it as `PROVEN_TEXT_AUTO` when you do not control who wrote it.

A surrogate pair is one character. `proven_writer_write_u16` refuses a text that ends with the high half of a pair even if the next call would have supplied the low half - keep a pair in one call. `proven_u16_reader_read` needs room for two units for the same reason.

Width counts UTF-8 bytes. `{:>10}` around a u16 argument pads to ten UTF-8 bytes, exactly as it does for a u8 view - not ten characters, and not ten terminal columns; a Hangul syllable is three bytes and two columns.

Worked example: u16 text to the standard streams, to a UTF-16LE file, and back

```
#include <string.h>

/*
 * UTF-16 text in and out: to the standard streams, to a file in the encoding
 * its reader expects, and back in a line at a time.
 *
 * Output goes through the formatter, so every sink that takes u8 text takes u16
 * text too: PROVEN_ARG picks the u16 renderer for a proven_u16str_view_t and
 * writes UTF-8. When a consumer wants UTF-16 bytes - a Windows tool that reads
 * UTF-16LE files, say - a writer writes them in the byte order you name.
 *
 * Input is a reader that decodes any of the three encodings into your u16
 * buffer. It carries a character split between two reads of the source, and it
 * keeps the byte line reader's rules: "\r\n" loses its '\r', the last line needs
 * no newline, and a line too long for the buffer is an error, not a cut line.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* Two Hangul syllables ("hello"), a space, and an emoji - which is a surrogate pair. */
    static const proven_u16 hello[] = { 0xC548, 0xB155, ' ', 0xD83D, 0xDE42 };
    proven_u16str_view_t text = { hello, 5 };

    /* --- to the standard streams ---------------------------------------- */

    /* UTF-8 on stdout and stderr. On a Windows console the sysio layer hands
     * it to the console as UTF-16, so it shows correctly whatever the code page. */
    proven_err_t err = proven_println("stdout: {}", PROVEN_ARG(text));
    EXAMPLE_REQUIRE(proven_is_ok(err), "u16 text prints like u8 text");
    err = proven_eprintln("stderr: {}", proven_arg_u16(text));
    EXAMPLE_REQUIRE(proven_is_ok(err), "and to stderr");

    /* A buffered writer takes it too; flush it, or it never happened. */
    proven_sysio_out_t out;
    proven_byte_t buf[256];
    proven_writer_t w = proven_sysio_stdout_buffered(&out, (proven_mem_mut_t){ buf, sizeof buf });
    EXAMPLE_REQUIRE(proven_is_ok(proven_fprintln(w, "buffered: {}", PROVEN_ARG(text)).err), "fprintln takes
u16");
    EXAMPLE_REQUIRE(proven_is_ok(proven_writer_flush(w)), "and the flush sends it");

    /* Strict: an unpaired surrogate fails the whole line instead of printing junk. */
    static const proven_u16 torn_units[] = { 'o', 'k', 0xD83D };
    proven_u16str_view_t torn = { torn_units, 3 };
    err = proven_println("{}", PROVEN_ARG(torn));
    EXAMPLE_REQUIRE(err == PROVEN_ERR_INVALID_ENCODING, "half a surrogate pair is refused");
```

```
    /* --- to a file, as UTF-16LE with a BOM --------------------------------- */

    proven_u8str_view_t path = PROVEN_LIT("proven_example_u16.txt");
    proven_result_file_t f = proven_fs_open(alloc, path, PROVEN_FS_WRITE ¦ PROVEN_FS_CREATE ¦
PROVEN_FS_TRUNC);
    EXAMPLE_REQUIRE(proven_is_ok(f.err), "creating the file must succeed");
    if (!proven_is_ok(f.err)) return EXAMPLE_OK();

    proven_sysio_out_t fout;
    proven_writer_t fw = proven_sysio_file_buffered(&fout, f.value, (proven_mem_mut_t){ buf, sizeof buf });
    /* The BOM is written only because it is asked for. */
    err = proven_writer_write_bom(fw, PROVEN_TEXT_UTF16LE);
    for (int i = 0; i < 2 && proven_is_ok(err); ++i) {
        err = proven_writer_write_u16(fw, text, PROVEN_TEXT_UTF16LE);
        if (proven_is_ok(err)) err = proven_writer_write_u16(fw, (proven_u16str_view_t){ u"\r\n", 2 },
PROVEN_TEXT_UTF16LE);
    }
    if (proven_is_ok(err)) err = proven_writer_flush(fw);
    EXAMPLE_REQUIRE(proven_is_ok(err), "two lines of UTF-16LE are written");
    (void)proven_fs_close(f.value);

    /* --- back in, a line at a time ----------------------------------------- */

    f = proven_fs_open(alloc, path, PROVEN_FS_READ);
    EXAMPLE_REQUIRE(proven_is_ok(f.err), "opening the file again must succeed");
    if (proven_is_ok(f.err)) {
        /* AUTO: the BOM says UTF-16LE, and is consumed. The buffer is in code
         * units and must hold the longest line. */
        proven_u16 line_buf[64];
        proven_sysio_u16_lines_t lines;
        err = proven_sysio_u16_lines_open(&lines, f.value, PROVEN_TEXT_AUTO, line_buf, 64);
        EXAMPLE_REQUIRE(proven_is_ok(err), "the line reader opens");
        int n = 0;
        for (;;) {
            proven_result_u16str_view_t line = proven_sysio_read_u16_line(&lines);
            if (line.err == PROVEN_ERR_EOF) break;
            EXAMPLE_REQUIRE(proven_is_ok(line.err), "each line reads");
            if (!proven_is_ok(line.err)) break;
            /* The view points into line_buf until the next call. */
            EXAMPLE_REQUIRE(line.val.size == 5 && memcmp(line.val.ptr, hello, sizeof hello) == 0,
                            "each line is the text, without its CR LF");
            ++n;
        }
        EXAMPLE_REQUIRE(n == 2 && lines.reader.enc == PROVEN_TEXT_UTF16LE, "two lines, read as UTF-16LE");
        (void)proven_fs_close(f.value);
    }
    (void)proven_fs_remove(alloc, path);

    /* --- any reader, any encoding ------------------------------------------ */

    /* The same reader over bytes already in memory, here UTF-8. read() hands out
     * code units whatever lines they belong to, and never half a pair. */
    proven_reader_view_t src;
    proven_u16 rbuf[16];
    proven_u16_reader_t rd;
    err = proven_u16_reader_init(&rd, proven_reader_from_view(&src, PROVEN_LIT("A\xF0\x9F\x99\x82\nB")),
                                 PROVEN_TEXT_UTF8, rbuf, 16);
    EXAMPLE_REQUIRE(proven_is_ok(err), "a u16 reader over UTF-8 bytes");
    proven_result_u16str_view_t first = proven_u16_reader_read_line(&rd);
```

```
    EXAMPLE_REQUIRE(proven_is_ok(first.err) && first.val.size == 3, "'A' and the emoji's two units");
    proven_u16 rest[4];
    proven_result_size_t got = proven_u16_reader_read(&rd, rest, 4);
    EXAMPLE_REQUIRE(proven_is_ok(got.err) && got.value == 1 && rest[0] == 'B', "the last line, without a
newline");
    EXAMPLE_REQUIRE(proven_u16_reader_read(&rd, rest, 4).err == PROVEN_ERR_EOF, "then the end");

    return EXAMPLE_OK();
}
```

The console path cannot be shown by a program that also runs on Linux. It was checked on Windows 11 by `b039-console-check.c`, which makes its own console in code page 949, writes through each sysio path and reads the screen buffer back, and places key events in the console's input buffer to read them through the line readers and the scanner.

Randomness, by use case

There is no single "random". There are two jobs that look identical and are not:

Your job	Use	Why
A key, a token, a nonce - anything an attacker must not guess.	`proven_random_bytes`, or a `proven_chacha_rng_t` seeded from it.	Only a cryptographic source is unguessable.
The same, on a target with no OS.	`proven_chacha_rng_t`, seeded from the board's own entropy.	ChaCha20 is pure arithmetic; it needs no OS. It is only as unguessable as its seed.
A simulation, a test, a game, a sample.	`proven_xoshiro256ss_t`.	Fast, and **reproducible**: the same seed replays the same run, which is what makes a failing test debuggable.
A number in a range, a shuffle, a float in [0,1).	`proven_rng_below`, `proven_rng_range`, `proven_rng_f64`, `proven_rng_shuffle` - over any source.	`% n` is biased, and everyone writes it anyway. These are not.

The two requirements are in direct opposition. Reproducible means predictable, and predictable is exactly what a token must not be: a few outputs of `proven_xoshiro256ss_t` reveal its entire state and therefore every number it will ever produce. That is a feature for a simulation you need to replay and a catastrophe for a session token - so the two carry names that cannot be confused, and the choice is visible at the call site rather than buried in how something was seeded.

The trait is infallible. `proven_rng_t` is a source of random bytes, and drawing from a valid one cannot fail. That is not a simplification; it is where the failure went. Asking an operating system for entropy *can* fail, so that failure is confined to exactly one place -- seeding - which you check once, at startup. Every draw downstream is total.

`proven_random_bytes(buf, len)`	The OS CSPRNG. Returns `false` on failure; do not use `buf` then. `len == 0` is a successful no-op.
`proven_random_u64()`	One strong word from the OS, or `0` on failure.
`proven_random_u64_checked(&out)`	One strong word, and `false` when there is none. The form for a token or a key.
`proven_chacha_rng_seed_from_entropy(&g)`	Seed the cryptographic generator from the entropy source. **This is the call that can fail.**

`proven_random_set_source(fn, ctx)`	Install the entropy source. The OS is already installed on a hosted target; a bare-metal target installs its board's TRNG.
`proven_chacha_rng_seed(&g, seed32)`	Seed it from 32 bytes you supply - a hardware entropy source on a board. Never the clock.
`proven_xoshiro256ss_seed(&g, seed)`	Seed the reproducible generator. Even seed 0 is fine: it is expanded through SplitMix64.

Where entropy comes from

Everything above is pure arithmetic - the generators, the helpers, and `proven_random_bytes` itself. What differs by platform is the **entropy source** behind it, because that is the one thing a program cannot compute for itself.

- **Hosted:** the OS CSPRNG is installed for you - `getrandom` on Linux, `getentropy` on the BSDs and macOS, `BCryptGenRandom` on Windows, and `/dev/urandom` where none of those exist. You call nothing.
- **Bare metal:** there is no source until you install one. A board *has* real entropy - an on-chip TRNG, a ring oscillator, an ADC's noise floor - and the library cannot know where.

```
/* On a board: hand the library its hardware entropy, once, at startup.
 * (A listing, not a fragment: it defines a function, and `hardware_rng_read` is
 * whatever your SoC calls its entropy register.) */
static bool board_trng(void *ctx, void *buf, proven_size_t len) {
    (void)ctx;
    /* read the SoC's entropy register into buf; return false if it is not ready */
    return hardware_rng_read(buf, len);
}

proven_random_set_source(board_trng, NULL);

/* From here everything above works unchanged - including the one call that turns a few
 * hundred bytes of hardware entropy into an endless cryptographic stream. */
proven_chacha_rng_t g;
if (!proven_chacha_rng_seed_from_entropy(&g)) {
    /* the TRNG was not ready. The generator is INERT - it yields zeros and an invalid
     * trait - so ignoring this does not get you plausible-looking bytes. */
}
```

With no source installed, `proven_random_bytes` returns **false**. It does not fall back to a clock-seeded PRNG, because that looks like success and is a security hole nothing reports - a refusal is a fact a caller can act on.

There is deliberately **no built-in `RDRAND` / `RNDR` backend.** On a hosted target the OS already mixes the CPU's instruction into its own pool, so calling it directly buys nothing and costs you that mixing; and a raw hardware instruction used as the *sole* source is exactly the arrangement people have argued about for a decade. If you want it, it is four lines behind this hook - and then the choice is visibly yours.

The structures you hold

All three are caller-owned state: they allocate nothing, there is nothing to destroy, and **copying one clones its sequence**.

```
typedef struct { const proven_rng_vtable_t *vt; void *ctx; } proven_rng_t;
    /* The trait: two pointers, held by value. `ctx` points at one of the generators
       below, which must outlive it. Drawing from a VALID one cannot fail - that is
       the whole design: the failure lives in seeding, not in drawing. */

typedef struct { proven_u64 s[4]; } proven_xoshiro256ss_t;
    /* 256 bits of state. Reproducible, and NOT secret-grade. */
```

```
typedef struct {
    proven_u32    state[16];   /* the ChaCha state: constants, key, counter, nonce */
    proven_byte_t block[64];   /* the keystream block currently being handed out */
    proven_size_t used;        /* how much of it is spent */
    proven_u32    seeded;      /* set only by seeding. A zero-initialised struct is
                                  the shape of "never seeded", and must stay inert. */
} proven_chacha_rng_t;
```

Reference

API	Intent	Return
`proven_random_bytes(buf, len)`	Fill from the entropy source (the OS by default). **The one call that can fail.**	`bool`. On `false`, `buf` is unspecified and must not be used. `len == 0` succeeds.
`proven_random_u64()`	One strong word from the same source.	`proven_u64`, or `0` on failure - which is also a valid draw, so use `proven_random_bytes` when you must tell them apart.
`proven_random_u64_checked(&out)`	One strong word, for a secret.	`bool`. On `false`, `out` is 0 and must not be used: a token of 0 is the predictable secret this exists to prevent.
`proven_random_set_source(fn, ctx)`	Install the entropy source. Not needed on a hosted target; this is how a board hands over its TRNG.	void.
`proven_xoshiro256ss_seed(&g, seed)`	Seed the reproducible generator. Any seed is fine - even 0; it is expanded through SplitMix64.	void.
`proven_xoshiro256ss_next(&g)`	The next word. The hot path: call it directly, not through the trait.	`proven_u64`.
`proven_xoshiro256ss_rng(&g)`	View it as a `proven_rng_t`, for the helpers.	`proven_rng_t`.
`proven_chacha_rng_seed(&g, seed32)`	Seed the cryptographic generator from 32 bytes of **real entropy** you supply.	void.
`proven_chacha_rng_seed_from_entropy(&g)`	Seed it from the installed source. **Check this.**	`bool`. On `false` the generator is left INERT - it yields zeros and an invalid trait.
`proven_chacha_rng_next/_fill`	Draw. Cannot fail once seeded.	`proven_u64` / void.
`proven_chacha_rng(&g)`	View it as a `proven_rng_t`.	`proven_rng_t` - **invalid** if the generator was never successfully seeded.
`proven_rng_u64(rng)` / `proven_rng_fill(rng, buf, len)`	Draw through the trait, from whichever generator.	`proven_u64` / void. `0` / no-op for an invalid source.
`proven_rng_below(rng, bound)`	Uniform in `[0, bound)`, **unbiased**.	`proven_u64`; `0` when `bound == 0`.

`proven_rng_range(rng, lo, hi)`	Uniform in `[lo, hi]`, inclusive. The full `INT64_MIN..INT64_MAX` span does not overflow.	`proven_i64`; `lo` if `hi < lo`.
`proven_rng_f64(rng)`	Uniform in `[0, 1)`. 53 bits; never returns `1.0`.	`double`.
`proven_rng_shuffle(rng, base, count, elem_size)`	An unbiased Fisher-Yates permutation, in place.	void.

```
/* A session token must not be guessable, so it may not be the 0 that proven_random_u64
 * returns when the entropy source fails. The checked form says so instead. */
proven_u64 token;
if (!proven_random_u64_checked(&token)) {
    proven_eprintln("no entropy source: refusing to issue a session token");
} else {
    proven_println("token {:x}", PROVEN_ARG(token));
}
```

Cautions, and what goes wrong

Never generate a secret with `proven_xoshiro256ss_t`. It is fast *because* it is predictable: a handful of its outputs reveal its entire 256-bit state, and from the state every number it will ever produce. The two generators carry names that cannot be confused for exactly this reason.

Wrong - a session token an attacker can compute after watching a few:

```
proven_xoshiro256ss_t g;
proven_xoshiro256ss_seed(&g, 12345);
proven_u64 session_token = proven_xoshiro256ss_next(&g);   /* wrong: predictable */
```

Never seed the cryptographic generator from the clock, a counter, or a serial number. ChaCha20 is exactly as unguessable as its seed. A clock-derived seed produces a stream that looks perfectly random and is not - which is worse than an obvious failure, because nothing reports it.

```
proven_byte_t seed[32] = { 0 };
memcpy(seed, &now_ns, sizeof now_ns);      /* wrong: ~20 bits of real entropy, and guessable */
proven_chacha_rng_seed(&g, seed);
```

Check the seeding. It is the only thing here that can fail, which is precisely why ignoring it is tempting. If you do, the generator is inert and hands you zeros - a visibly dead value, by design, rather than a plausible one.

Wrong:

```
proven_chacha_rng_t g;
proven_chacha_rng_seed_from_entropy(&g);   /* wrong: the bool was the point */
proven_chacha_rng_fill(&g, key, 32);       /* key is now 32 zero bytes */
```

Correct:

```
proven_chacha_rng_t g;
if (!proven_chacha_rng_seed_from_entropy(&g)) {
    /* No entropy. There is nothing safe to do here except refuse to continue. */
} else {
    proven_byte_t key[32];
    proven_chacha_rng_fill(&g, key, sizeof key);   /* cannot fail: it is seeded */
}
```

`% n` is biased, and everyone writes it anyway. Unless `n` divides 2^64 the low values come up more often - invisible in a spot check, real in a shuffle or a sample.

```
proven_u64 die = proven_rng_u64(rng) % 6 + 1;   /* wrong: 1 and 2 are slightly likelier */
```

Correct: `proven_rng_below(rng, 6) + 1`.

Do not copy a seeded generator unless you mean to clone its stream. Two "independent" generators copied from one produce identical output - which is a feature for replaying a simulation and a catastrophe for issuing two tokens.

Compiled and run by the test suite:

```
/*
 * Randomness, by use case. There is no single "random": there are two jobs that look
 * identical and are not, and picking the wrong one is the whole danger.
 *
 *   A key, a token, a nonce - anything an attacker must not guess - needs a CRYPTOGRAPHIC
 *   source. A simulation, a test, a game needs a REPRODUCIBLE one, because a failing run you
 *   cannot replay is a failing run you cannot debug. The two requirements are in direct
 *   opposition: reproducible means predictable, and predictable is exactly what a token must
 *   not be. So the library gives them different names, and the choice is visible here at the
 *   call site rather than buried in how something was seeded.
 */

int main(void) {
    /* ---- Job 1: a secret. The OS CSPRNG - and the one place randomness can fail. ---- */
    proven_byte_t key[32];
    EXAMPLE_REQUIRE(proven_random_bytes(key, sizeof key),
                    "the OS must give us strong bytes on a hosted platform");

    /* ---- Job 2: lots of cryptographic bytes, or any at all on a board with no OS.
     * ChaCha20 is pure arithmetic: seed it once from real entropy and it needs nothing from
     * the operating system afterwards - no syscall per draw, and it works on bare metal.
     * Seeding is the ONLY step that can fail, so it is the only one you have to check. ---- */
    proven_chacha_rng_t crypto;
    EXAMPLE_REQUIRE(proven_chacha_rng_seed_from_entropy(&crypto), "seed the CSPRNG from the OS, once");

    proven_byte_t token[16];
    proven_chacha_rng_fill(&crypto, token, sizeof token);   /* cannot fail: it is seeded */

    /* ---- Job 3: a REPRODUCIBLE run. xoshiro256** is fast and replays exactly from its seed,
     * which is what makes a failing simulation debuggable. It is NOT secret-grade: a few of
     * its outputs reveal its whole state. Never hand it a token to generate. ---- */
    proven_xoshiro256ss_t sim;
    proven_xoshiro256ss_seed(&sim, 12345);

    proven_xoshiro256ss_t replay;
    proven_xoshiro256ss_seed(&replay, 12345);
    EXAMPLE_REQUIRE(proven_xoshiro256ss_next(&sim) == proven_xoshiro256ss_next(&replay),
                    "the same seed replays the same run - that is the whole point");

    /* ---- The helpers work over ANY source, through the proven_rng_t trait. ---- */
    proven_rng_t rng = proven_xoshiro256ss_rng(&sim);

    /* A number in a range. `rng_u64() % 6` is what everyone writes, and it is BIASED unless
     * the bound divides 2^64 - the low values come up more often. This one is not. */
    for (int i = 0; i < 100; ++i) {
        proven_u64 die = proven_rng_below(rng, 6) + 1;
        EXAMPLE_REQUIRE(die >= 1 && die <= 6, "a die roll is 1..6, uniformly");
    }

    proven_i64 temperature = proven_rng_range(rng, -40, 85);
    EXAMPLE_REQUIRE(temperature >= -40 && temperature <= 85, "an inclusive range, both ends");

    double p = proven_rng_f64(rng);
    EXAMPLE_REQUIRE(p >= 0.0 && p < 1.0, "a double in [0, 1) - never 1.0");
```

```
    /* An unbiased shuffle: Fisher-Yates over the unbiased index above. The `% n` version of
     * this loop measurably favours some orderings. */
    int deck[10];
    for (int i = 0; i < 10; ++i) deck[i] = i;
    proven_rng_shuffle(rng, deck, 10, sizeof deck[0]);

    int sum = 0;
    for (int i = 0; i < 10; ++i) sum += deck[i];
    EXAMPLE_REQUIRE(sum == 45, "a shuffle is a permutation: every card is still there, once");

    /* The cryptographic generator satisfies the same trait, so the same helpers work over it
     * when the choice must be unguessable rather than merely uniform. */
    proven_rng_t secure = proven_chacha_rng(&crypto);
    proven_u64 unguessable_index = proven_rng_below(secure, 1000);
    EXAMPLE_REQUIRE(unguessable_index < 1000, "the helpers do not care which source they draw from");

    (void)token;
    (void)key;
    return EXAMPLE_OK();
}
```

Worked example: replacing a file so a power cut cannot corrupt it

Writing over a file in place has a failure mode that testing never shows and production eventually does: the machine loses power halfway through, and the file that remains is neither the old one nor the new one. The fix is a four-step recipe, and every step of it is load-bearing:

1. Write the new contents to a **temporary file** beside the real one.
2. `proven_fs_sync()` - the new file's bytes are now on the storage device, not merely in the operating system's cache.
3. `proven_fs_rename()` - the name flips to the new file in one indivisible step. A reader sees the whole old file or the whole new one, never a mixture.
4. `proven_fs_sync_dir()` - the rename itself is now on the device.

Skip step 2 and the rename can publish a file whose contents never arrived. Skip step 4 and the contents are safe under a name that is not.

The same example covers the record-level calls that go with it:

Call	What it is for
`proven_fs_pread` / `proven_fs_pwrite`	Read or write at an absolute offset **without moving the file position** - which is what makes one handle safe to share between threads.
`proven_fs_seek` / `proven_fs_tell`	Move the position, and ask where it is. Seeking from the end with a negative offset finds the last record without knowing the length.
`proven_fs_truncate`	Set the length directly. One call, and the filesystem adjusts a number; the alternative is copying the part you keep.
`proven_fs_lock`	An **advisory** lock: it excludes other processes that also ask for one, and does not affect a program that never asks.
`proven_fs_copy`	Duplicate the bytes into a second, independent file.
`proven_fs_link`	A second **name** for the same file (a hard link). No original: the data lives until the last name goes. Same filesystem only.

`proven_fs_symlink`	A small file holding a path (a symbolic link). May cross filesystems, and may point at nothing.
`proven_fs_is_absolute`	Does this path start from the root? The rule differs per platform, which is why it is a call.
`proven_fs_mkdir` / `proven_fs_mkdir_all`	Create one directory, or a directory with every missing level above it. The first says `PROVEN_ERR_EXISTS` for a name that is taken; the second accepts a level that is already a directory.
`proven_fs_rmdir`	Remove an **empty** directory. A non-empty one is refused with `PROVEN_ERR_INVALID_STATE`, so a recursive delete stays an explicit decision.

```
/*
 * Updating a file so that a power cut cannot leave it half-written.
 *
 * The recipe is old and every part of it is load-bearing:
 *
 *   1. write the new contents to a TEMPORARY file beside the real one,
 *   2. proven_fs_sync   - the new file's bytes are now on the device,
 *   3. proven_fs_rename - the name flips to the new file in one step; a reader
 *                         sees either the whole old file or the whole new one,
 *   4. proven_fs_sync_dir - the rename itself is now on the device.
 *
 * Skip step 2 and the rename can publish a file whose contents never arrived.
 * Skip step 4 and the contents are safe under a name that may not be. Neither
 * failure shows up in testing; both show up in production, once.
 *
 * The same program also shows the record-level calls - seek/tell, pread/pwrite,
 * truncate - and the advisory lock that stops two copies of the program doing
 * all this at the same time.
 */

typedef struct {
    proven_u32 id;
    proven_u32 score;
} ex_record_t;

static proven_err_t write_records(proven_file_t f, const ex_record_t *recs, proven_size_t n) {
    proven_mem_view_t view = { .ptr = (const proven_byte_t *)recs, .size = n * sizeof recs[0] };
    return proven_fs_write_all(f, view);
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    proven_u8str_view_t live = PROVEN_LIT("proven_example_durable.dat");
    proven_u8str_view_t temp = PROVEN_LIT("proven_example_durable.dat.new");
    proven_u8str_view_t here = PROVEN_LIT(".");

    /* is_absolute answers a question worth asking before you join paths or
     * resolve one against a base directory: does this path already start from
     * the root? The rule differs per platform - a leading '/' here, a drive
     * letter or a UNC prefix on Windows - which is exactly why it is a call and
     * not a comparison against '/'. */
    EXAMPLE_REQUIRE(!proven_fs_is_absolute(live), "the working paths in this example are relative");
    EXAMPLE_REQUIRE(proven_fs_is_absolute(PROVEN_LIT("/etc/hosts")), "a leading slash is absolute on POSIX");

    static const ex_record_t initial[] = {
```

```
        { .id = 1, .score = 10 }, { .id = 2, .score = 20 },
        { .id = 3, .score = 30 }, { .id = 4, .score = 40 },
    };

    /* --- an ordinary first write ------------------------------------------ */

    proven_result_file_t f = proven_fs_open(alloc, live, PROVEN_FS_WRITE | PROVEN_FS_CREATE |
PROVEN_FS_TRUNC);
    EXAMPLE_REQUIRE(proven_is_ok(f.err), "creating the data file must succeed");
    if (!proven_is_ok(f.err)) return 1;

    proven_err_t err = write_records(f.value, initial, 4);
    EXAMPLE_REQUIRE(proven_is_ok(err), "writing the initial records must succeed");
    err = proven_fs_close(f.value);
    EXAMPLE_REQUIRE(proven_is_ok(err), "closing after a write must succeed");

    /* --- an advisory lock, so two copies do not interleave ---------------- */

    proven_result_file_t rw = proven_fs_open(alloc, live, PROVEN_FS_READ | PROVEN_FS_WRITE);
    EXAMPLE_REQUIRE(proven_is_ok(rw.err), "reopening for update must succeed");
    if (!proven_is_ok(rw.err)) return 1;

    /* An EXCLUSIVE lock keeps every other process that also asks for one out.
     * "Advisory" means exactly that: it stops cooperating programs, and a
     * program that never asks for the lock is not affected. `wait = false`
     * returns immediately rather than blocking - the right choice when you have
     * something else to do, and the only safe choice when the other holder
     * might be waiting on you. */
    err = proven_fs_lock(rw.value, PROVEN_FS_LOCK_EXCLUSIVE, false);
    EXAMPLE_REQUIRE(proven_is_ok(err), "taking the exclusive lock must succeed when nobody holds it");

    /* --- reading and writing one record, by offset ------------------------ */

    /* pread reads at an absolute offset and does NOT move the file position.
     * That is what makes it safe to use from two threads sharing one handle:
     * there is no shared cursor for them to race on. */
    ex_record_t third = {0};
    proven_mem_mut_t into = { .ptr = (proven_byte_t *)&third, .size = sizeof third };
    proven_result_size_t got = proven_fs_pread(rw.value, into, 2 * sizeof(ex_record_t));
    EXAMPLE_REQUIRE(proven_is_ok(got.err) && got.value == sizeof third, "reading record 2 must succeed");
    EXAMPLE_REQUIRE(third.id == 3 && third.score == 30, "and yield the record that was written there");

    /* tell reports the position; after a pread it has not moved. */
    proven_result_u64_t pos = proven_fs_tell(rw.value);
    EXAMPLE_REQUIRE(proven_is_ok(pos.err) && pos.val == 0, "pread must not move the file position");

    /* pwrite updates that record in place, again without touching the cursor. */
    third.score = 99;
    proven_mem_view_t out_view = { .ptr = (const proven_byte_t *)&third, .size = sizeof third };
    proven_result_size_t put = proven_fs_pwrite(rw.value, out_view, 2 * sizeof(ex_record_t));
    EXAMPLE_REQUIRE(proven_is_ok(put.err) && put.value == sizeof third, "writing record 2 back must succeed");

    /* seek is the cursor-moving alternative, and it returns the position it
     * arrived at. Seeking from the END with a negative offset is how you find
     * the last record without knowing the file length first. */
    proven_result_u64_t last = proven_fs_seek(rw.value, -(proven_i64)sizeof(ex_record_t), PROVEN_FS_SEEK_END);
    EXAMPLE_REQUIRE(proven_is_ok(last.err), "seeking to the last record must succeed");
    EXAMPLE_REQUIRE(last.val == 3 * sizeof(ex_record_t), "which is three records in");

    pos = proven_fs_tell(rw.value);
```

```
EXAMPLE_REQUIRE(proven_is_ok(pos.err) && pos.val == last.val, "tell agrees with the seek result");

/* truncate sets the length directly. Dropping the last record is one call
 * and O(1); the old way - read everything, write back the part you keep -
 * was an O(n) copy for an operation the filesystem does by adjusting a
 * number. */
err = proven_fs_truncate(rw.value, 3 * sizeof(ex_record_t));
EXAMPLE_REQUIRE(proven_is_ok(err), "truncating to three records must succeed");
proven_result_size_t size = proven_fs_size(rw.value);
EXAMPLE_REQUIRE(proven_is_ok(size.err) && size.value == 3 * sizeof(ex_record_t),
                "the file is now exactly three records long");

/* Release the lock explicitly. Closing the handle would also drop it, but
 * saying so keeps the critical section visible in the code. */
err = proven_fs_lock(rw.value, PROVEN_FS_LOCK_UNLOCK, false);
EXAMPLE_REQUIRE(proven_is_ok(err), "releasing the lock must succeed");
err = proven_fs_close(rw.value);
EXAMPLE_REQUIRE(proven_is_ok(err), "closing the update handle must succeed");

/* --- the durable replace ------------------------------------------- */

static const ex_record_t replacement[] = {
    { .id = 1, .score = 11 }, { .id = 2, .score = 22 },
};

/* 1. Write the new contents beside the old file. CREATE_NEW refuses if the
 *    temporary name already exists, which is how a leftover from a crashed
 *    run is noticed instead of silently reused. */
proven_result_file_t tmp = proven_fs_open(alloc, temp,
                                          PROVEN_FS_WRITE | PROVEN_FS_CREATE | PROVEN_FS_TRUNC);
EXAMPLE_REQUIRE(proven_is_ok(tmp.err), "creating the temporary file must succeed");
if (!proven_is_ok(tmp.err)) return 1;

err = write_records(tmp.value, replacement, 2);
EXAMPLE_REQUIRE(proven_is_ok(err), "writing the new contents must succeed");

/* 2. Push those bytes all the way to the storage device. This is expensive
 *    and meant to be: you are buying the guarantee that the data exists
 *    after a power cut, and the price is a real trip to the device. */
err = proven_fs_sync(tmp.value);
EXAMPLE_REQUIRE(proven_is_ok(err), "syncing the new file's data must succeed");

err = proven_fs_close(tmp.value);
EXAMPLE_REQUIRE(proven_is_ok(err), "closing the temporary file must succeed");

/* 3. Flip the name. A rename within one directory is atomic: any reader
 *    sees the old file or the new one, never a partial write. */
err = proven_fs_rename(alloc, temp, live);
EXAMPLE_REQUIRE(proven_is_ok(err), "renaming the temporary file over the live one must succeed");

/* 4. Make the rename itself durable. Until the directory reaches the device,
 *    the new contents are safe under a name that might not be. */
err = proven_fs_sync_dir(alloc, here);
/* Windows has no directory handle to flush, and says so: UNSUPPORTED, not a
 * false OK. There the rename's durability is the file system's to decide. */
EXAMPLE_REQUIRE(proven_is_ok(err) || err == PROVEN_ERR_UNSUPPORTED,
                "syncing the directory must succeed, or say it cannot");

proven_result_file_t check = proven_fs_open(alloc, live, PROVEN_FS_READ);
EXAMPLE_REQUIRE(proven_is_ok(check.err), "the live path must now open");
```

```
size = proven_fs_size(check.value);
EXAMPLE_REQUIRE(proven_is_ok(size.err) && size.value == 2 * sizeof(ex_record_t),
                "and hold exactly the replacement records");
err = proven_fs_close(check.value);
EXAMPLE_REQUIRE(proven_is_ok(err), "closing the verification handle must succeed");

/* --- copies, and the two kinds of link ------------------------------- */

/* copy duplicates the bytes: two independent files from here on. The
 * allocator is for the temporary buffer the copy moves data through. */
proven_u8str_view_t backup = PROVEN_LIT("proven_example_durable.bak");
err = proven_fs_copy(alloc, live, backup);
EXAMPLE_REQUIRE(proven_is_ok(err), "copying the file must succeed");

/* A HARD link is a second name for the same file. There is no original: the
 * data lives until the last name is removed. Both names must be on the same
 * filesystem, because a name and its data cannot span two. */
proven_u8str_view_t hard = PROVEN_LIT("proven_example_durable.hard");
err = proven_fs_link(alloc, live, hard);
EXAMPLE_REQUIRE(proven_is_ok(err), "creating a hard link must succeed");

proven_fs_stat_t st_live = {0}, st_hard = {0};
EXAMPLE_REQUIRE(proven_is_ok(proven_fs_stat(alloc, live, &st_live)), "stat of the live name");
EXAMPLE_REQUIRE(proven_is_ok(proven_fs_stat(alloc, hard, &st_hard)), "stat of the hard link");
EXAMPLE_REQUIRE(st_live.ino == st_hard.ino && st_live.dev == st_hard.dev,
                "both names refer to the same file, which is what a hard link means");

/* A SYMBOLIC link is a small file holding a path. It may point at something
 * on another filesystem, and it may point at nothing at all - following it
 * then fails, which a hard link can never do. */
proven_u8str_view_t soft = PROVEN_LIT("proven_example_durable.link");
err = proven_fs_symlink(alloc, live, soft);
EXAMPLE_REQUIRE(proven_is_ok(err), "creating a symbolic link must succeed");

proven_fs_stat_t st_soft = {0};
EXAMPLE_REQUIRE(proven_is_ok(proven_fs_stat(alloc, soft, &st_soft)),
                "stat follows the symbolic link to its target");
EXAMPLE_REQUIRE(st_soft.size == st_live.size, "so it reports the target's size");

/* --- directories, and cleaning up ------------------------------------ */

proven_u8str_view_t dir = PROVEN_LIT("proven_example_durable_dir");
err = proven_fs_mkdir(alloc, dir);
EXAMPLE_REQUIRE(proven_is_ok(err), "creating a directory must succeed");

/* Asking for a directory that is already there is refused, and the refusal says so:
 * PROVEN_ERR_EXISTS, where a missing parent is PROVEN_ERR_NOT_FOUND. A caller can tell
 * "nothing to do" from "something is wrong" without a second look at the disk. */
err = proven_fs_mkdir(alloc, dir);
EXAMPLE_REQUIRE(err == PROVEN_ERR_EXISTS, "a second mkdir of the same name must say it exists");

/* proven_fs_mkdir_all makes every missing level of a path, and a level that is already
 * a directory is not an error - so it can simply be called before writing into a tree.
 * A FILE in the way is still PROVEN_ERR_EXISTS. */
proven_u8str_view_t deep = PROVEN_LIT("proven_example_durable_dir/a/b");
err = proven_fs_mkdir_all(alloc, deep);
EXAMPLE_REQUIRE(proven_is_ok(err), "creating two missing levels at once must succeed");
err = proven_fs_mkdir_all(alloc, deep);
EXAMPLE_REQUIRE(proven_is_ok(err), "and asking again must succeed too");
EXAMPLE_REQUIRE(proven_is_ok(proven_fs_rmdir(alloc, deep)), "removing the inner level must succeed");
```

```
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_rmdir(alloc, PROVEN_LIT("proven_example_durable_dir/a"))),
                    "removing the level above it must succeed");

    /* rmdir removes an EMPTY directory only. That refusal is a feature: a
     * recursive delete is a decision the caller should have to make explicitly,
     * not something a stray path argument can trigger. The refusal has its own code,
     * PROVEN_ERR_INVALID_STATE: nothing failed, the directory has to be emptied first. */
    proven_u8str_view_t inside = PROVEN_LIT("proven_example_durable_dir/file.txt");
    proven_result_file_t child = proven_fs_open(alloc, inside, PROVEN_FS_WRITE | PROVEN_FS_CREATE);
    EXAMPLE_REQUIRE(proven_is_ok(child.err), "creating a file inside it must succeed");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_close(child.value)), "closing it must succeed");

    err = proven_fs_rmdir(alloc, dir);
    EXAMPLE_REQUIRE(err == PROVEN_ERR_INVALID_STATE, "removing a non-empty directory must be refused");

    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_remove(alloc, inside)), "removing the file must succeed");
    err = proven_fs_rmdir(alloc, dir);
    EXAMPLE_REQUIRE(proven_is_ok(err), "and then the empty directory can be removed");

    printf("durable replace complete: %zu byte(s) live\n", (size_t)size.value);

    (void)proven_fs_remove(alloc, soft);
    (void)proven_fs_remove(alloc, hard);
    (void)proven_fs_remove(alloc, backup);
    (void)proven_fs_remove(alloc, live);
    return EXAMPLE_OK();
}
```

Wrong - the in-place rewrite the recipe exists to replace:

```
proven_result_file_t f = proven_fs_open(alloc, live, PROVEN_FS_WRITE | PROVEN_FS_TRUNC);
proven_err_t e = proven_fs_write_all(f.value, new_contents);   /* wrong */
```

Between the truncate and the last byte of the write, the file on disk is incomplete. A crash there does not lose the update; it loses the data that was already there.

Wrong - renaming without syncing first:

```
proven_err_t e = proven_fs_close(tmp.value);        /* no proven_fs_sync */
e = proven_fs_rename(alloc, temp, live);            /* wrong */
```

Closing a file does not put its bytes on the device. The rename can be durable while the contents it points at are not.

Wrong - assuming the lock stops everyone:

```
proven_err_t e = proven_fs_lock(f.value, PROVEN_FS_LOCK_EXCLUSIVE, true);
/* another program writes the file without ever calling proven_fs_lock */
```

An advisory lock is a convention between programs that all use it. It is not permission control.

Worked example: readers, writers, and the standard streams

A **writer** is "somewhere bytes go" and a **reader** is "somewhere bytes come from". Each is two pointers: a small table of functions, and the state those functions work on. The whole value of the arrangement is that code written against a writer does not know whether the bytes end up in a file, in a string, on the terminal, or in a test buffer - and code written against a reader can be tested against a string in memory instead of a file on disk.

The example puts the pieces together:

- `proven_reader_from_view()` makes a reader over bytes you already have, which is how a parser gets tested without touching the filesystem. `proven_reader_read()` fills up to the buffer size and reports what it got; a short read is normal, and end of input is `PROVEN_ERR_EOF`, not a zero-byte success.

- `proven_sysio_stdout_writer()` and `proven_sysio_stderr_writer()` are the unbuffered standard streams, and `proven_sysio_stdin_reader()` the standard input. `proven_writer_write()` means "all of it, or an error"; `proven_writer_write_partial()` moves what it can and reports the count, for callers doing their own retry or back-pressure.
- `proven_reader_is_valid()` and `proven_writer_is_valid()` catch a handle that was never built, at the boundary rather than at the first read or write.
- `proven_sysio_file_buffered()` wraps an open file in a buffered writer over a buffer **you** supply, so the memory cost of buffering is a number you chose. It must be flushed: nothing here flushes on your behalf at exit.
- `proven_sysio_lines_open()` reads that file back one line at a time, through the same kind of caller-supplied buffer. Size it for the longest line you expect - a longer one is `PROVEN_ERR_OUT_OF_BOUNDS`, never a silently cut line.
- `proven_scan_fmt_from_file()` (which calls `proven_sysio_scan_chunk_impl()`) pulls typed values straight out of a file handle when the input has a known shape rather than being free text.

```
/*
 * A writer is "somewhere bytes go" and a reader is "somewhere bytes come from",
 * each of them two pointers: a small table of functions, and the state those
 * functions work on. That is all. The value of the arrangement is that code
 * written against a writer does not know or care whether the bytes end up in a
 * file, in a string, on the terminal, or in a test buffer - and code written
 * against a reader can be tested against a string in memory instead of a file
 * on disk.
 *
 * This program shows both, and the standard-stream and file plumbing that hangs
 * off them:
 *
 *   - a reader over a view (an in-memory string), which is how you test a
 *     parser without touching the filesystem,
 *   - the unbuffered stdout and stderr writers, and the difference between
 *     "write all of this" and "write what you can",
 *   - a buffered writer over a file, which turns many small writes into few
 *     large ones,
 *   - a line reader over that same file,
 *   - reading formatted values straight out of a file handle.
 *
 * A note that costs people an afternoon: the state structs below must stay
 * where they are declared. A writer holds a pointer INTO its state struct, so
 * copying the struct leaves the copy inert and the original addressed.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- 1. a reader over bytes you already have -------------------------- */

    /* The parser below does not know this is a string. It would work the same
     * over a file, a pipe or a socket - which is exactly why the test can use
     * the cheap one. */
    proven_reader_view_t src_state;
    proven_reader_t src = proven_reader_from_view(&src_state, PROVEN_LIT("id=41\nid=42\n"));

    /* is_valid asks whether the reader was actually built - a zero-initialised
     * handle is not a reader, and this is the check that says so at the
     * boundary instead of at the first read. */
    EXAMPLE_REQUIRE(proven_reader_is_valid(src), "a reader made from a view must be usable");
    proven_reader_t never_made = {0};
    EXAMPLE_REQUIRE(!proven_reader_is_valid(never_made), "a zero-initialised reader handle is not");
```

```
    /* read fills up to dest.size bytes and reports how many it actually got.
     * A short read is normal - it is not end of file, and treating it as one is
     * the classic way to lose the tail of an input. End of file has its own
     * code, PROVEN_ERR_EOF. */
    proven_byte_t chunk[5];
    proven_mem_mut_t into = { .ptr = chunk, .size = sizeof chunk };
    proven_result_size_t got = proven_reader_read(src, into);
    EXAMPLE_REQUIRE(proven_is_ok(got.err), "the first read must succeed");
    EXAMPLE_REQUIRE(got.value == 5, "and fill the buffer from the view");

    proven_size_t total = got.value;
    for (;;) {
        got = proven_reader_read(src, into);
        if (got.err == PROVEN_ERR_EOF) {
            break;
        }
        EXAMPLE_REQUIRE(proven_is_ok(got.err), "reads before end of file must succeed");
        total += got.value;
    }
    EXAMPLE_REQUIRE(total == 12, "the loop must consume the whole view, however it was chunked");

    /* --- 2. the standard streams ------------------------------------------ */

    /* The state struct holds the handle the writer points at, so it has to
     * outlive the writer. Declaring the two next to each other is the habit
     * that keeps that true. */
    proven_sysio_std_t out_state;
    proven_writer_t out = proven_sysio_stdout_writer(&out_state);
    EXAMPLE_REQUIRE(proven_writer_is_valid(out), "the stdout writer must be usable");

    /* write means "all of it, or an error". It loops internally, because a
     * single system-level write may move fewer bytes than asked. */
    proven_err_t err = proven_writer_write(out, proven_mem_view_from_u8(PROVEN_LIT("stream example:
start\n")));
    EXAMPLE_REQUIRE(proven_is_ok(err), "writing a whole line to stdout must succeed");

    /* write_partial is the honest low-level twin: it moves what it can and
     * reports the count. Use it when you are managing your own retry or
     * back-pressure; use write when you just want the bytes out. */
    proven_result_size_t part = proven_writer_write_partial(out, proven_mem_view_from_u8(PROVEN_LIT("partial
write\n")));
    EXAMPLE_REQUIRE(proven_is_ok(part.err), "a partial write to a terminal or pipe must succeed");
    EXAMPLE_REQUIRE(part.value > 0, "and report how many bytes it moved");

    /* stderr is unbuffered on purpose: a diagnostic is out before the next line
     * of code runs, which is what you need when the next line is the one that
     * crashes. */
    proven_sysio_std_t err_state;
    proven_writer_t diag = proven_sysio_stderr_writer(&err_state);
    EXAMPLE_REQUIRE(proven_writer_is_valid(diag), "the stderr writer must be usable");
    err = proven_writer_write(diag, proven_mem_view_from_u8(PROVEN_LIT("stream example: diagnostics go
here\n")));
    EXAMPLE_REQUIRE(proven_is_ok(err), "writing to stderr must succeed");

    /* A reader over stdin is built the same way. This example does not read
     * from it - a test run has no one typing - but building it shows the shape,
     * and it is the handle a filter program would loop over. */
    proven_sysio_std_t in_state;
    proven_reader_t stdin_reader = proven_sysio_stdin_reader(&in_state);
```

```
    EXAMPLE_REQUIRE(proven_reader_is_valid(stdin_reader), "the stdin reader must be usable");

    /* --- 3. buffered output to a file ---------------------------------- */

    proven_u8str_view_t path = PROVEN_LIT("proven_example_streams.txt");
    proven_result_file_t f = proven_fs_open(alloc, path, PROVEN_FS_WRITE | PROVEN_FS_CREATE |
PROVEN_FS_TRUNC);
    EXAMPLE_REQUIRE(proven_is_ok(f.err), "creating the output file must succeed");
    if (!proven_is_ok(f.err)) return 1;

    /* The buffer is yours: the library does not allocate one behind your back,
     * so the memory cost of buffering is a number you chose and can see. Sixty
     * lines through a 256-byte buffer is a handful of writes instead of sixty. */
    proven_byte_t outbuf[256];
    proven_sysio_out_t file_out;
    proven_writer_t w = proven_sysio_file_buffered(&file_out, f.value,
                                                   (proven_mem_mut_t){ .ptr = outbuf, .size = sizeof
outbuf });
    EXAMPLE_REQUIRE(proven_writer_is_valid(w), "the buffered file writer must be usable");

    for (int i = 0; i < 20; ++i) {
        proven_fmt_result_t line_out = proven_fprintln(w, "reading {} = {}",
                                                       proven_arg_i32(i), proven_arg_i32(i * i));
        EXAMPLE_REQUIRE(proven_is_ok(line_out.err), "writing a formatted line must succeed");
    }

    /* Buffered output that is never flushed is output that never happened.
     * Nothing here flushes at exit on your behalf. */
    err = proven_writer_flush(w);
    EXAMPLE_REQUIRE(proven_is_ok(err), "the flush is what actually writes the file");
    err = proven_fs_close(f.value);
    EXAMPLE_REQUIRE(proven_is_ok(err), "closing the file must succeed");

    /* --- 4. reading it back a line at a time --------------------------- */

    proven_result_file_t rf = proven_fs_open(alloc, path, PROVEN_FS_READ);
    EXAMPLE_REQUIRE(proven_is_ok(rf.err), "reopening for reading must succeed");
    if (!proven_is_ok(rf.err)) return 1;

    /* Size the buffer for the longest LINE you expect. A longer line is
     * reported as OUT_OF_BOUNDS - never silently cut in half, which is the
     * failure that turns one bad record into two plausible ones. */
    proven_byte_t linebuf[128];
    proven_sysio_lines_t lines;
    err = proven_sysio_lines_open(&lines, rf.value, (proven_mem_mut_t){ .ptr = linebuf, .size = sizeof
linebuf });
    EXAMPLE_REQUIRE(proven_is_ok(err), "opening a line reader over the file must succeed");

    proven_size_t count = 0;
    for (;;) {
        proven_result_u8str_view_t line = proven_sysio_read_line(&lines);
        if (line.err == PROVEN_ERR_EOF) {
            break;
        }
        EXAMPLE_REQUIRE(proven_is_ok(line.err), "reading a line must succeed until end of file");
        /* The view points into linebuf and is good only until the next call.
         * That is what makes a million lines cost one buffer instead of a
         * million allocations - and why you copy a line you want to keep. */
        if (count == 0) {
            EXAMPLE_REQUIRE(proven_u8str_view_eq(line.val, PROVEN_LIT("reading 0 = 0")),
```

```
                        "the first line reads back exactly as it was written");
        }
        ++count;
    }
    EXAMPLE_REQUIRE(count == 20, "every line written must be read back");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_close(rf.value)), "closing the read handle must succeed");

    /* --- 5. reading values, not lines, from a file ------------------------ */

    /* When the input is a known shape rather than free text, scanning it
     * directly saves writing the split-and-convert loop by hand. This reads one
     * chunk from the file handle and pulls the two numbers out of it. */
    proven_result_file_t sf = proven_fs_open(alloc, path, PROVEN_FS_READ);
    EXAMPLE_REQUIRE(proven_is_ok(sf.err), "opening the file for scanning must succeed");

    proven_i32 index = -1, square = -1;
    err = proven_scan_fmt_from_file(sf.value, "reading {} = {}",
                                    proven_scan_arg_i32(&index), proven_scan_arg_i32(&square));
    EXAMPLE_REQUIRE(proven_is_ok(err), "scanning the first record must succeed");
    EXAMPLE_REQUIRE(index == 0 && square == 0, "and produce the values that were written");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_close(sf.value)), "closing the scan handle must succeed");

    printf("streams: %zu byte(s) read from the view, %zu line(s) round-tripped\n",
           (size_t)total, (size_t)count);

    (void)proven_fs_remove(alloc, path);
    return EXAMPLE_OK();
}
```

Wrong - copying a state struct after making a writer from it:

```
proven_sysio_out_t state;
proven_writer_t w = proven_sysio_file_buffered(&state, file, buf);
proven_sysio_out_t moved = state;    /* wrong: w still points into `state` */
```

The writer holds a pointer **into** the struct. Leave the state where you declared it, for as long as the writer lives.

Wrong - forgetting the flush:

```
proven_fmt_result_t r = proven_fprintln(w, "done");
return 0;                            /* wrong: the buffer never reached the file */
```

Worked example: mapping a file into memory, and making the change durable
A memory-mapped file is a file the processor reads as memory: the operating system arranges for its pages to appear at an address, and a read happens without a read call. It suits random access into a large file, especially one several processes share; it does not suit streaming, where a buffered reader is simpler and does not tie the file's size to your address space.

The distinction that decides whether your writes survive:

Mapping	Writes go	`proven_mmap_sync()`
`PROVEN_MMAP_SHARED`	to the file	pushes them to the storage device
`PROVEN_MMAP_PRIVATE`	to a private copy (copy-on-write) - this process only	returns `PROVEN_ERR_UNSUPPORTED`, because there is nothing to write back

That refusal is deliberate. A caller who believed the mapping was shared finds out at the sync, rather than when the data turns out to be missing.

The example ends with the calendar formatter, because the record it writes carries a date: `proven_time_u8_fmt()` for the UTF-8 form, and `proven_time_u16_fmt()` for the UTF-16 form - the latter being right in exactly one situation, handing text to a system call that takes wide strings.

```
#include <string.h>

/*
 * A memory-mapped file is a file the processor reads as memory: the operating
 * system arranges for the pages to appear at an address, and reads happen
 * without a read call. It is the right tool for one shape of problem - random
 * access into a large file that several processes look at - and the wrong tool
 * for streaming, where a buffered reader is simpler and does not tie a file's
 * size to your address space.
 *
 * The part that is easy to get wrong is durability, and it is the reason this
 * example exists:
 *
 *   PROVEN_MMAP_SHARED  - writes go to the file. proven_mmap_sync pushes them
 *                         to the storage device.
 *   PROVEN_MMAP_PRIVATE - writes are copy-on-write: they exist in this process
 *                         and nowhere else. There is nothing to write back, and
 *                         asking to sync one says so instead of quietly doing
 *                         nothing.
 *
 * This example also shows the calendar formatter alongside it, because the
 * record it writes carries a timestamp - and a timestamp written for a Windows
 * API is the one place the UTF-16 formatter is the right call.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();
    proven_u8str_view_t path = PROVEN_LIT("proven_example_mmap.dat");

    /* A mapping cannot extend a file, so the file has to be the size you intend
     * to map before you map it. */
    static const char initial[] = "record 0: pending    \n";
    proven_result_file_t create = proven_fs_open(alloc, path,
                                                 PROVEN_FS_WRITE | PROVEN_FS_CREATE | PROVEN_FS_TRUNC);
    EXAMPLE_REQUIRE(proven_is_ok(create.err), "creating the backing file must succeed");
    if (!proven_is_ok(create.err)) return 1;

    proven_mem_view_t seed = { .ptr = (const proven_byte_t *)initial, .size = sizeof initial - 1 };
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_write_all(create.value, seed)), "writing the record must succeed");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_close(create.value)), "closing it must succeed");

    /* --- a shared mapping: writes reach the file -------------------------- */

    proven_result_file_t f = proven_fs_open(alloc, path, PROVEN_FS_READ | PROVEN_FS_WRITE);
    EXAMPLE_REQUIRE(proven_is_ok(f.err), "opening the file for mapping must succeed");
    if (!proven_is_ok(f.err)) return 1;

    /* size 0 means "to the end of the file". */
    proven_result_mmap_t m = proven_mmap_create(f.value, 0, 0,
                                                PROVEN_MMAP_READ | PROVEN_MMAP_WRITE,
                                                PROVEN_MMAP_SHARED);
    EXAMPLE_REQUIRE(proven_is_ok(m.err), "mapping the file must succeed");
    if (!proven_is_ok(m.err)) {
        (void)proven_fs_close(f.value);
        return 1;
    }
```

```
    proven_mmap_t map = m.value;

    /* Reading is just reading memory - no call, no copy. as_view hands back the
     * whole mapping as a byte view, so the ordinary view helpers apply. */
    proven_u8str_view_t contents = proven_mmap_as_view(map);
    EXAMPLE_REQUIRE(contents.size == sizeof initial - 1, "the mapping covers the whole file");
    EXAMPLE_REQUIRE(proven_u8str_view_starts_with(contents, PROVEN_LIT("record 0:")),
                    "and shows the bytes that were written");

    /* Writing is writing memory. The status field is a fixed width on purpose:
     * a mapping cannot make the file longer, so an in-place edit has to fit the
     * space that is already there. */
    proven_size_t at = proven_u8str_view_find(contents, 0, PROVEN_LIT("pending"));
    EXAMPLE_REQUIRE(at != PROVEN_SIZE_MAX, "the status field must be found");
    memcpy((proven_byte_t *)map.ptr + at, "done   ", 7);

    /* sync is the durability step: without it the change is in the page cache,
     * where it survives this program exiting but not the machine losing power. */
    EXAMPLE_REQUIRE(proven_is_ok(proven_mmap_sync(&map)), "syncing a shared mapping must succeed");
    EXAMPLE_REQUIRE(proven_is_ok(proven_mmap_destroy(&map)), "unmapping must succeed");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_close(f.value)), "closing the mapped file must succeed");

    /* The edit is in the file, not merely in this process's memory. */
    proven_result_u8str_t back = proven_fs_read_all_u8str(alloc, path);
    EXAMPLE_REQUIRE(proven_is_ok(back.err), "reading the file back must succeed");
    EXAMPLE_REQUIRE(proven_u8str_view_find(proven_u8str_as_view(&back.value), 0, PROVEN_LIT("done")) !=
PROVEN_SIZE_MAX,
                    "the mapped write reached the file");
    proven_u8str_destroy(alloc, &back.value);

    /* --- a private mapping: writes go nowhere ------------------------------ */

    proven_result_file_t pf = proven_fs_open(alloc, path, PROVEN_FS_READ);
    EXAMPLE_REQUIRE(proven_is_ok(pf.err), "reopening for a private mapping must succeed");

    proven_result_mmap_t pm = proven_mmap_create(pf.value, 0, 0, PROVEN_MMAP_READ, PROVEN_MMAP_PRIVATE);
    EXAMPLE_REQUIRE(proven_is_ok(pm.err), "a private read mapping must succeed");
    proven_mmap_t priv = pm.value;

    /* Asking to sync a private mapping is refused rather than accepted and
     * ignored. The refusal is the useful behaviour: a caller who believed the
     * mapping was shared finds out here, not when the data is missing. */
    EXAMPLE_REQUIRE(proven_mmap_sync(&priv) == PROVEN_ERR_UNSUPPORTED,
                    "a private mapping has nothing to write back, and says so");

    EXAMPLE_REQUIRE(proven_is_ok(proven_mmap_destroy(&priv)), "unmapping the private mapping must succeed");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_close(pf.value)), "closing it must succeed");

    /* --- the timestamp, in both string types ------------------------------- */

    proven_datetime_t now = proven_time_now_datetime();

    proven_result_u8str_t stamp = proven_u8str_create(alloc, 64);
    EXAMPLE_REQUIRE(proven_is_ok(stamp.err), "creating the timestamp string must succeed");
    EXAMPLE_REQUIRE(proven_is_ok(proven_time_u8_fmt(alloc, &stamp.value, now, &proven_time_locale_en,
                                                    "{year}-{month:0>2}-{day:0>2}")),
                    "formatting the date as UTF-8 must succeed");
    EXAMPLE_REQUIRE(proven_u8str_as_view(&stamp.value).size == 10, "a YYYY-MM-DD date is ten characters");

    /* The UTF-16 form of the same call, for the one place it is the right one:
```

```
     * handing the text straight to a system call that takes wide strings. */
    proven_result_u16str_t wide = proven_u16str_create(alloc, 64);
    EXAMPLE_REQUIRE(proven_is_ok(wide.err), "creating the wide timestamp string must succeed");
    EXAMPLE_REQUIRE(proven_is_ok(proven_time_u16_fmt(alloc, &wide.value, now, &proven_time_locale_en,
                                                     "{year}-{month:0>2}-{day:0>2}")),
                    "formatting the same date as UTF-16 must succeed");
    EXAMPLE_REQUIRE(proven_u16str_len(&wide.value) == 10, "ten code units, one per character here");
    EXAMPLE_REQUIRE(proven_u16str_as_ptr(&wide.value)[4] == (proven_u16)'-',
                    "and the same layout as the UTF-8 form");

    printf("mapped record updated; stamped %s\n", proven_u8str_as_cstr(&stamp.value));

    proven_u16str_destroy(alloc, &wide.value);
    proven_u8str_destroy(alloc, &stamp.value);
    (void)proven_fs_remove(alloc, path);
    return EXAMPLE_OK();
}
```

Wrong - expecting a mapping to extend the file:

```
proven_result_mmap_t m = proven_mmap_create(f.value, 0, 1 << 20, ...);  /* file is 22 bytes */
memcpy((char *)m.value.ptr + 4096, data, n);                            /* wrong */
```

Set the file's length first - `proven_fs_truncate()` does it in one call - and map what exists.

Worked example: where randomness comes from

The example above picks the right generator. This one is the layer underneath: the source of the bytes, and how to write code that does not care which source it got.

- `proven_rng_t` **is the interface.** A function that takes one works with the operating system's generator, with ChaCha20, with xoshiro, and with a fake you wrote for a test, unchanged. `proven_rng_is_valid()` says whether a source was actually built; drawing from one that was not returns 0 rather than inventing a number, so check once at the boundary instead of trusting every draw. `proven_rng_u64()` draws one 64-bit word and `proven_rng_fill()` fills a whole buffer in one call.
- **A fixed seed makes a test reproducible.** `proven_chacha_rng_seed()` takes the seed bytes directly, so `proven_chacha_rng_next()` walks a known sequence -- which is what turns "fails once a week" into a failure you can replay. In production the seed must come from real entropy, because anyone who learns it knows every byte that follows.
- `proven_random_u64()` draws a single strong word straight from the entropy source. Right for a one-off - a hash key at start-up, an identifier - and wrong in a loop, where each call costs a trip to the operating system. It returns 0 when the source fails, so for a token or a key use `proven_random_u64_checked()`, which says so.
- `proven_random_set_source()` installs the entropy source itself. A hosted program already has the operating system's and should leave it alone; a bare-metal program has none, and this is where its hardware source goes.

```
#include <string.h>

/*
 * The other example shows which generator to pick. This one is about the layer
 * underneath: where the randomness comes FROM, and how to write code that does
 * not care.
 *
 *   proven_rng_t is a source of random bytes as a pair of pointers - a small
 *   table of functions and the generator state they work on. Code that takes a
 *   proven_rng_t works with the OS generator, with ChaCha20, with xoshiro, and
 *   with a fake you wrote for a test, without a line of change.
 *
```

```
 *   proven_random_set_source is the layer below THAT: where the raw entropy a
 *   generator is seeded from comes from. A hosted program already has one - the
 *   operating system's - and should leave it alone. A bare-metal program has
 *   none, and this is the hook where its hardware source is installed.
 *
 * The fixed-seed part matters more than it looks: a cryptographic generator
 * seeded from a KNOWN seed produces a known sequence, which is what makes a
 * test that involves randomness reproducible instead of "fails once a week".
 */

/* A source of "entropy" that is not random at all: it counts. Nothing like this
 * belongs in a real program - see the counter-example in the chapter - but it
 * is exactly the right shape for showing how the hook works, and for a test
 * that must produce the same bytes every run. */
static bool counting_entropy(void *ctx, void *buf, proven_size_t len) {
    proven_u8 *next = (proven_u8 *)ctx;
    proven_u8 *out = (proven_u8 *)buf;
    for (proven_size_t i = 0; i < len; ++i) {
        out[i] = (*next)++;
    }
    return true;
}

/* A function written against the trait. It never learns which generator it got. */
static proven_u64 roll_total(proven_rng_t rng, int rolls) {
    proven_u64 sum = 0;
    for (int i = 0; i < rolls; ++i) {
        sum += proven_rng_below(rng, 6) + 1;
    }
    return sum;
}

int main(void) {
    /* --- 1. a source you can check before you use it ---------------------- */

    proven_rng_t nothing = {0};
    EXAMPLE_REQUIRE(!proven_rng_is_valid(nothing), "a zero-initialised source is not a generator");

    /* Drawing from an invalid source does not crash and does not invent a
     * number: it returns 0. That is a defined, boring answer - but a stream of
     * zeros is not randomness, so check the source once when you receive it
     * rather than trusting every draw. */
    EXAMPLE_REQUIRE(proven_rng_u64(nothing) == 0, "an invalid source yields 0, not a fabricated value");

    /* --- 2. a cryptographic generator from a KNOWN seed ------------------- */

    /* proven_chacha_rng_seed takes the seed bytes directly, so the sequence is
     * reproducible. That is what you want in a test and never in production:
     * anyone who learns the seed knows every byte the generator will produce. */
    proven_byte_t seed[PROVEN_CHACHA_SEED_SIZE];
    memset(seed, 0xA5, sizeof seed);

    proven_chacha_rng_t a, b;
    proven_chacha_rng_seed(&a, seed);
    proven_chacha_rng_seed(&b, seed);

    /* next returns one 64-bit word at a time. Two generators given the same
     * seed walk the same sequence - which is the property the test relies on. */
    proven_u64 first = proven_chacha_rng_next(&a);
    EXAMPLE_REQUIRE(first == proven_chacha_rng_next(&b), "the same seed replays the same sequence");
```

```
EXAMPLE_REQUIRE(proven_chacha_rng_next(&a) == proven_chacha_rng_next(&b), "and keeps replaying it");

/* --- 3. using it through the trait ---------------------------------- */

proven_rng_t rng = proven_chacha_rng(&a);
EXAMPLE_REQUIRE(proven_rng_is_valid(rng), "a seeded generator makes a valid source");

proven_u64 word = proven_rng_u64(rng);
(void)word;   /* any 64-bit value is a legal answer; there is nothing to assert about it */

/* fill is the bulk form: one call for a whole buffer, rather than a loop
 * over 64-bit words that has to deal with the remainder itself. */
proven_byte_t nonce[12] = {0};
proven_rng_fill(rng, nonce, sizeof nonce);

bool all_zero = true;
for (proven_size_t i = 0; i < sizeof nonce; ++i) {
    if (nonce[i] != 0) all_zero = false;
}
EXAMPLE_REQUIRE(!all_zero, "filling from a seeded generator must produce something");

/* The same function, driven by two different generators. This is the only
 * reason the trait exists. */
proven_chacha_rng_t c;
proven_chacha_rng_seed(&c, seed);
proven_u64 crypto_total = roll_total(proven_chacha_rng(&c), 50);

proven_xoshiro256ss_t fast;
proven_xoshiro256ss_seed(&fast, 7);
proven_u64 fast_total = roll_total(proven_xoshiro256ss_rng(&fast), 50);

EXAMPLE_REQUIRE(crypto_total >= 50 && crypto_total <= 300, "50 dice must total between 50 and 300");
EXAMPLE_REQUIRE(fast_total >= 50 && fast_total <= 300, "whichever generator produced them");

/* --- 4. one strong word, without holding a generator ---------------- */

/* proven_random_u64 draws straight from the entropy source. Convenient for
 * a one-off - a table's hash key at startup, a request id - and the wrong
 * tool for a loop, because each call costs a trip to the operating system.
 * For bulk output, seed a generator once and draw from that. */
proven_u64 one_off = proven_random_u64();
proven_u64 another = proven_random_u64();
EXAMPLE_REQUIRE(one_off != another || one_off != 0,
                "two draws from the OS source are essentially never the same value");

/* --- 5. installing an entropy source -------------------------------- */

/* On a hosted target the operating system's source is already installed and
 * you should leave it there. This hook exists for the bare-metal case,
 * where the library cannot know that the board's entropy lives in a
 * particular hardware register. Here it is installed with a deliberately
 * fake source, purely to show the mechanism and to prove the switch took
 * effect - a real one must be genuine hardware entropy. */
proven_u8 counter = 0;
proven_random_set_source(counting_entropy, &counter);

proven_byte_t drawn[4] = {0};
EXAMPLE_REQUIRE(proven_random_bytes(drawn, sizeof drawn), "the installed source must answer");
EXAMPLE_REQUIRE(drawn[0] == 0 && drawn[1] == 1 && drawn[2] == 2 && drawn[3] == 3,
                "and it is the source we installed that answered");
```

```
    /* Put the platform default back. Leaving a test source installed is how a
     * program ends up generating predictable keys in production. */
    proven_random_set_source(NULL, NULL);

    proven_byte_t real[8] = {0};
    EXAMPLE_REQUIRE(proven_random_bytes(real, sizeof real), "the OS source is back and working");

    printf("random: first word %llu, dice totals %llu and %llu\n",
           (unsigned long long)first, (unsigned long long)crypto_total, (unsigned long long)fast_total);

    return EXAMPLE_OK();
}
```

Wrong - installing something that merely looks random:

```
static bool clock_entropy(void *ctx, void *buf, proven_size_t len) {
    proven_time_t t = proven_time_now();       /* wrong: predictable */
    memcpy(buf, &t, len < sizeof t ? len : sizeof t);
    return true;
}
proven_random_set_source(clock_entropy, NULL);
```

A clock, a serial number, an uninitialised buffer or a pseudo-random generator all produce something that passes a glance and is guessable. If a board has no real entropy, install nothing: a refusal is a fact the caller can act on, and silent predictability is not.

Wrong - leaving a test source installed:

```
proven_random_set_source(counting_entropy, &counter);
/* ... the rest of the program, now generating predictable keys ... */
```

Restore the platform default with `proven_random_set_source(NULL, NULL)` as soon as the test is over.

Chapter 9: Networking

Part V - Talking to the operating system. Prerequisites: Part II (1, 2, 3) and Chapter 5 section 4 (the two clocks) and its section on streams. After this chapter you can open a TCP or UDP socket, put a time limit on everything that waits, tell a refused connection from a lost one, serve many sockets from one thread, and write a protocol that does not care what carries it.

This chapter covers `net.h`. Like Chapter 5 it needs an operating system and is not part of a freestanding build; a hosted build can leave it out with `PROVEN_NO_NET`.

Table of contents

1. Three rules
2. Addresses
3. Deadlines
4. TCP
5. UDP
6. Many sockets, one thread
7. Transports
8. A pair, and waking a loop
9. A selector: thousands of sockets
10. What the platforms do differently
11. What is not here

1. Three rules

BSD sockets are forty years old and every one of their sharp edges has a name. `net.h` is a thin layer whose whole job is to take three of them away.

Every call that can wait takes a deadline. `recv` on a plain socket waits as long as the peer keeps it waiting. A program built that way works on a desk and stops in production the first time a client opens a connection and says nothing - and one such client is enough to stop a server that handles one connection at a time. Here there is no way to write that program: a read, a write, an accept and a connect each take a deadline argument, and you must say something.

A failure says which failure. "The call failed" is not enough to decide what to do next. Whether the other side said no, went away, or finished properly are three different situations with three different responses:

What happened	Code	Sensible response
The peer finished sending	`PROVEN_ERR_EOF`	Stop reading; this is the normal end
The deadline passed	`PROVEN_ERR_TIMEOUT`	Retry, give up, or close - nothing is broken
Nothing listens at that address	`PROVEN_ERR_REFUSED`	Try another address, or report; a retry gets the same answer
The peer vanished, or the connection was aborted	`PROVEN_ERR_RESET`	Close; what was in flight may be lost
No route to the host or network	`PROVEN_ERR_UNREACHABLE`	Report, or retry later
The address to bind is taken, or the system is out of sockets	`PROVEN_ERR_BUSY`	Back off, or choose another port
The address may not be bound	`PROVEN_ERR_PERMISSION`	Report; a retry will not help

A host name has no address; a Unix-domain path is missing	`PROVEN_ERR_NOT_FOUND`	Report
A datagram did not fit the buffer, or is too large to send	`PROVEN_ERR_OUT_OF_BOUNDS`	Use a larger buffer; send less
The socket is not open	`PROVEN_ERR_INVALID_STATE`	Fix the sequence of calls

`PROVEN_ERR_IO` is what is left when none of these fits.

Nothing is hidden. No call in this header allocates. A socket is a small value you own and must close. A reader or writer over a socket uses state you declare. The one piece of process-wide state - starting Winsock on Windows - is done once, on first use, and is not yours to manage.

2. Addresses

A `proven_net_addr_t` says where a socket is or where it should go. It is a plain value: copy it, compare it, store it in a table.

```
typedef struct {
    proven_net_family_t family;   /* IPV4, IPV6 or UNIX; NONE when unset */
    proven_u16 port;              /* host byte order: the number you would write down */
    proven_u32 scope_id;          /* IPv6 zone for a link-local address; 0 otherwise */
    proven_byte_t ip[16];         /* network order: 4 bytes for IPv4, 16 for IPv6 */
    proven_u8 path_len;           /* Unix-domain only */
    char path[104];               /* Unix-domain only */
} proven_net_addr_t;
```

Reference

API	Intent	Return
`proven_net_addr_ipv4(a, b, c, d, port)`	An IPv4 address from its four numbers.	`proven_net_addr_t`.
`proven_net_addr_ipv6(ip[16], port, scope_id)`	An IPv6 address from sixteen bytes in network order.	`proven_net_addr_t`.
`proven_net_addr_loopback(family, port)`	This machine only: `127.0.0.1` or `::1`.	`proven_net_addr_t`; family `NONE` if `family` has no loopback.
`proven_net_addr_any(family, port)`	Every interface: `0.0.0.0` or `::`.	`proven_net_addr_t`.
`proven_net_addr_unix(path, &out)`	A Unix-domain address from a filesystem path.	`proven_err_t`: `INVALID_ARG` for an empty path or one with a NUL, `OUT_OF_BOUNDS` past `PROVEN_NET_UNIX_PATH_MAX`.
`proven_net_addr_parse(text, port, &out)`	Parse a literal: `192.0.2.1`, `2001:db8::1`, `[2001:db8::1]`, `fe80::1%3`. Never touches the network.	`proven_err_t`: `INVALID_FORMAT` when `text` is not an address; `out` is then untouched.
`proven_net_addr_format(&addr, out, &written)`	Write `192.0.2.1:80`, `[2001:db8::1]:80`, or the path. No NUL. `PROVEN_NET_ADDR_TEXT_MAX` bytes are always enough.	`proven_err_t`: `OUT_OF_BOUNDS` when `out` is too small.
`proven_net_addr_eq(&a, &b)`	Same family, address, port and zone - or the same path.	`bool`.

`proven_net_resolve(host, port, out, cap, &count)`	Look a name up through the system resolver; a literal is answered without a query.	`proven_err_t`: `NOT_FOUND`, `TIMEOUT` (the name servers did not answer), `INVALID_ARG` for an empty or over-long name.

Two decisions in the parser are deliberate. An IPv4 literal is exactly four decimal numbers: `127.1` and `2130706433` are refused although older parsers accept both, and so is `010.0.0.1`, which those parsers read as octal *eight*. And IPv6 has many spellings going in but one coming out

- lowercase, no leading zeros, the longest run of zero groups as `::` (RFC 5952) - so two equal

addresses print as equal text and the text can be a map key or a log field you search for.

Cautions, and what goes wrong

`proven_net_resolve` **blocks and has no deadline.** It is the system resolver, which offers none, and on a broken network it can take tens of seconds. It is the one call in this header that waits without a limit, and the header says so rather than pretend.

Wrong - resolving inside the loop that serves every other connection:

```
for (;;) {
    wait_for_events();
    if (client_wants_upstream)
        proven_net_resolve(name, 443, addrs, 4, &n);   /* wrong: every client waits on DNS */
}
```

Correct - resolve where waiting is acceptable: at startup, or on a job (Chapter 6), and hand the address to the loop.

Binding to "any" publishes the service. `proven_net_addr_any` listens on every interface the machine has, including the ones that face a network. A tool meant for your own machine binds `proven_net_addr_loopback`.

Compiled and run by the test suite:

```
/*
 * Addresses are plain values: build one, parse one from text, print it, compare two.
 * Nothing in this program opens a socket or asks the network anything.
 */

static bool prints_as(const proven_net_addr_t *addr, proven_u8str_view_t want) {
    proven_byte_t text[PROVEN_NET_ADDR_TEXT_MAX];
    proven_size_t n = 0;
    if (proven_net_addr_format(addr, (proven_mem_mut_t){ text, sizeof text }, &n) != PROVEN_OK) return false;
    return proven_u8str_view_eq((proven_u8str_view_t){ text, n }, want);
}

int main(void) {
    /* Built from parts. The port is the number you would write down - host byte order. */
    proven_net_addr_t web = proven_net_addr_ipv4(192, 0, 2, 10, 80);
    EXAMPLE_REQUIRE(prints_as(&web, PROVEN_LIT("192.0.2.10:80")), "an IPv4 address prints as address:port");

    /* The two fixed addresses a program needs most: this machine only, and every interface. */
    proven_net_addr_t local = proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 8080);
    proven_net_addr_t everywhere = proven_net_addr_any(PROVEN_NET_FAMILY_IPV6, 8080);
    EXAMPLE_REQUIRE(prints_as(&local, PROVEN_LIT("127.0.0.1:8080")), "loopback is 127.0.0.1");
    EXAMPLE_REQUIRE(prints_as(&everywhere, PROVEN_LIT("[::]:8080")), "any is ::, and IPv6 is bracketed before
a port");

    /* Parsed from text. Many spellings of one IPv6 address parse to the same bytes, and there
     * is exactly one spelling on the way out, so the text can be compared and used as a key. */
```

```
    proven_net_addr_t a, b;
    EXAMPLE_REQUIRE(proven_net_addr_parse(PROVEN_LIT("2001:DB8:0:0:0:0:0:1"), 443, &a) == PROVEN_OK, "the long
form parses");
    EXAMPLE_REQUIRE(proven_net_addr_parse(PROVEN_LIT("[2001:db8::1]"), 443, &b) == PROVEN_OK, "the short,
bracketed form parses");
    EXAMPLE_REQUIRE(proven_net_addr_eq(&a, &b), "and they are the same address");
    EXAMPLE_REQUIRE(prints_as(&a, PROVEN_LIT("[2001:db8::1]:443")), "printed in the one canonical form");

    /* What is not an address is refused - including what other parsers quietly accept. */
    proven_net_addr_t bad;
    EXAMPLE_REQUIRE(proven_net_addr_parse(PROVEN_LIT("010.0.0.1"), 80, &bad) == PROVEN_ERR_INVALID_FORMAT,
                    "a leading zero is refused, not read as octal");
    EXAMPLE_REQUIRE(proven_net_addr_parse(PROVEN_LIT("example.com"), 80, &bad) == PROVEN_ERR_INVALID_FORMAT,
                    "a name is not a literal: that is proven_net_resolve's job");

    /* Sixteen raw bytes, in network order, when the address comes from a packet or a file. */
    proven_byte_t raw[16] = { 0xfe, 0x80, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1 };
    proven_net_addr_t link_local = proven_net_addr_ipv6(raw, 22, 3);
    EXAMPLE_REQUIRE(prints_as(&link_local, PROVEN_LIT("[fe80::1%3]:22")), "a link-local address carries its
zone");

    /* A Unix-domain address is a filesystem path. */
    proven_net_addr_t sock;
    EXAMPLE_REQUIRE(proven_net_addr_unix(PROVEN_LIT("/run/app.sock"), &sock) == PROVEN_OK, "a path makes an
address");
    EXAMPLE_REQUIRE(prints_as(&sock, PROVEN_LIT("/run/app.sock")), "and prints as the path");

    /* Resolving a literal is answered without a query, so this line works with no network.
     * A real name goes to the system resolver, blocks, and has no deadline: call it where
     * that is acceptable. */
    proven_net_addr_t found[4];
    proven_size_t count = 0;
    EXAMPLE_REQUIRE(proven_net_resolve(PROVEN_LIT("192.0.2.10"), 80, found, 4, &count) == PROVEN_OK && count
== 1,
                    "a literal resolves to itself");
    EXAMPLE_REQUIRE(proven_net_addr_eq(&found[0], &web), "the same address as the one built by hand");

    return EXAMPLE_OK();
}
```

3. Deadlines

A deadline is a moment, not a duration: a reading of the monotonic clock (Chapter 5 section 4) at which a call must give up.

	Meaning
`proven_net_deadline_in(ms)`	The deadline `ms` milliseconds from now.
`PROVEN_NET_NO_DEADLINE`	Wait as long as it takes. A silent peer then waits with you.
`PROVEN_NET_DONT_WAIT`	Do what can be done right now; anything else is `PROVEN_ERR_TIMEOUT`.

It is absolute so that one deadline can bound a whole exchange. "This request gets five seconds" is one value passed to connect, to each write and to each read; with a per-call timeout the same five seconds would have to be recomputed after every step, and usually is not.

It is on the monotonic clock so that setting the system time does not move it.

And a deadline that passes breaks nothing. `PROVEN_ERR_TIMEOUT` means "not yet"; the socket is exactly as usable as it was. Whether to try again, give up, or close is yours to decide.

Wrong - a timeout treated as a dead connection:

```
proven_result_size_t r = proven_net_read(&conn, buf, proven_net_deadline_in(100));
if (r.err != PROVEN_OK) proven_net_close(&conn);   /* wrong: TIMEOUT is not a failure of the connection */
```

Correct - decide what silence means for this protocol:

```
if (r.err == PROVEN_ERR_TIMEOUT) { send_keepalive(&conn); continue; }
if (r.err == PROVEN_ERR_EOF)     { finish(&conn); break; }
if (r.err != PROVEN_OK)          { proven_net_close(&conn); break; }
```

4. TCP

A TCP connection is a stream of bytes in each direction. It has no message boundaries: what one side sends in a single write the other may receive in three reads, or two writes in one read.

Reference

API	Intent	Return
`proven_net_listen(at, backlog, &listener, &bound)`	Listen at `at`. Port 0 lets the OS choose; `bound` (optional) is the address actually bound.	`proven_err_t`: `BUSY` (taken), `PERMISSION`, `INVALID_ARG` (not this machine's address), `UNSUPPORTED`.
`proven_net_accept(&listener, until, &conn, &peer)`	Take the next connection. `peer` is optional.	`proven_err_t`: `TIMEOUT` when none arrived.
`proven_net_listener_addr(&listener, &out)`	Where a listener is bound.	`proven_err_t`.
`proven_net_listener_close(&listener)`	Stop listening. Accepted connections are unaffected.	`proven_err_t`.
`proven_net_connect(to, until, &conn)`	Connect. On failure there is nothing to close.	`proven_err_t`: `REFUSED`, `TIMEOUT`, `UNREACHABLE`, `NOT_FOUND` (Unix-domain path).
`proven_net_connect_start(to, &conn)`	Begin connecting and return at once, for a program that waits for many sockets together (section 9, or the loop of Chapter 15).	`proven_err_t`: `PROVEN_OK` when already connected; `AGAIN` when under way - `conn` can then be watched and closed and nothing else, until it is reported writable; otherwise the failure, with nothing open.
`proven_net_connect_finish(&conn)`	How a connect that was under way ended, once its socket was reported writable or failed.	`proven_err_t`: `PROVEN_OK`, connected; `AGAIN`, wait again; otherwise the failure, and `conn` has been closed. `INVALID_STATE` when it is not open.
`proven_net_read(&conn, dest, until)`	Read what has arrived, up to `dest.size`.	`proven_result_size_t`: `EOF` at the end - never zero bytes of success; `TIMEOUT`; `RESET`.
`proven_net_write(&conn, src, until)`	Write what fits now. `.value` is how much went, also on failure.	`proven_result_size_t`.
`proven_net_write_all(&conn, src, until)`	Write all of `src`, or fail saying how far it got.	`proven_result_size_t`: `.value == src.size` exactly when `.err` is OK.

`proven_net_shutdown_write(&conn)`	"Nothing more to send." The peer reads `EOF`; this side can still read.	`proven_err_t`.
`proven_net_close(&conn)`	Close. Safe on a connection that is not open.	`proven_err_t`.
`proven_net_conn_local_addr(&conn, &out)`, `proven_net_conn_peer_addr(&conn, &out)`	The two ends of a connection.	`proven_err_t`.
`proven_net_conn_set_nodelay(&conn, on)`	Send small writes at once instead of gathering them.	`proven_err_t`: `UNSUPPORTED` on a Unix-domain connection.
`proven_net_listener_is_open(&l)`, `proven_net_conn_is_open(&c)`	Whether the value holds an open socket.	`bool`.

`proven_net_listener_t` and `proven_net_conn_t` are small values you own. Close each one you opened, and do not copy one that is open: two copies are two owners of one socket.

A connect in two halves. `proven_net_connect` waits. A program that is waiting for many sockets at once cannot afford to, so the same connect is offered as two calls:

```
proven_err_t proven_net_connect_start(proven_net_addr_t to, proven_net_conn_t *out);
proven_err_t proven_net_connect_finish(proven_net_conn_t *conn);
```

`proven_net_connect_start` returns at once: `PROVEN_OK` when the connection is already made (on the loopback interface it often is), or `PROVEN_ERR_AGAIN` when it is under way. In the second case the socket can be watched and closed and nothing else: wait until it is reported writable - with `proven_net_poll`, a selector, or the loop of Chapter 15 - and ask `proven_net_connect_finish` how it ended. A failure, from either call, leaves nothing open. The event-driven client of Chapter 15 is built on this pair.

The same calls serve **Unix-domain** stream sockets - pass an address made by `proven_net_addr_unix`. Listening creates the path; closing the listener does not remove it, so a server removes a leftover path before it listens.

Cautions, and what goes wrong

One read is not one message. A stream delivers bytes, in order, in whatever pieces the network made of them.

Wrong - assuming the request arrives in one read:

```
proven_result_size_t r = proven_net_read(&conn, buf, until);
handle_request(buf, r.value);     /* wrong: this may be half a request, or one and a half */
```

Correct - read until the protocol says the message is complete: a length you were told, a delimiter you were promised, or `PROVEN_ERR_EOF`. The worked example below reads to the end.

One write may not send everything. `proven_net_write` reports how much went. Ignoring that count silently truncates the message whenever the buffer happens to be full - which is to say, under load, in production. `proven_net_write_all` is the call that keeps going; and when it fails, its count is what tells you whether resending would duplicate data.

A reset is not an end. `PROVEN_ERR_EOF` means the peer said it was finished and everything it sent has been read. `PROVEN_ERR_RESET` means it went away without saying so, and some of what was in flight - in either direction - may be gone. A file transfer that ends in `EOF` is complete; one that ends in `RESET` is not, however many bytes arrived.

Closing is not the same as finishing. To be sure the peer received everything, say you are done with `proven_net_shutdown_write`, then read until `EOF`, then close. Closing a connection that still has unread data in it is how the *other* side gets `PROVEN_ERR_RESET`.

Compiled and run by the test suite:

```
/*
 * A TCP exchange, start to finish, with a deadline on everything that can wait.
 *
 * Both ends live in this one program, which works because the kernel completes a connection as
 * soon as the listener's backlog takes it - before accept is called. A real client and server
 * are these same calls in two processes.
 */

int main(void) {
    /* Port 0: the OS picks a free one, and `at` says which. Loopback: this machine only. */
    proven_net_listener_t listener;
    proven_net_addr_t at;
    proven_err_t err = proven_net_listen(proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0), 16, &listener,
&at);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK && proven_net_listener_is_open(&listener), "the listener opens");
    EXAMPLE_REQUIRE(at.port != 0, "and reports the port it was given");

    /* One deadline for the whole exchange: five seconds from now, however many calls it takes. */
    proven_net_deadline_t until = proven_net_deadline_in(5000);

    proven_net_conn_t client, server;
    proven_net_addr_t peer;
    EXAMPLE_REQUIRE(proven_net_connect(at, until, &client) == PROVEN_OK, "the client connects");
    EXAMPLE_REQUIRE(proven_net_accept(&listener, until, &server, &peer) == PROVEN_OK, "the server accepts");
    EXAMPLE_REQUIRE(proven_net_conn_is_open(&client) && proven_net_conn_is_open(&server), "two open ends");

    /* Each end knows both addresses. The accepted peer is the client's own local address. */
    proven_net_addr_t client_local, client_peer, listening;
    EXAMPLE_REQUIRE(proven_net_conn_local_addr(&client, &client_local) == PROVEN_OK &&
                    proven_net_conn_peer_addr(&client, &client_peer) == PROVEN_OK, "the client's two
addresses");
    EXAMPLE_REQUIRE(proven_net_addr_eq(&client_local, &peer), "the server saw the client's address");
    EXAMPLE_REQUIRE(proven_net_listener_addr(&listener, &listening) == PROVEN_OK &&
                    proven_net_addr_eq(&client_peer, &listening), "the client's peer is where the server
listens");

    /* A request written in two small pieces: without this they may wait to be sent together. */
    EXAMPLE_REQUIRE(proven_net_conn_set_nodelay(&client, true) == PROVEN_OK, "small writes go at once");

    /* write_all sends everything or says how far it got. */
    proven_mem_view_t request = proven_mem_view_from_u8(PROVEN_LIT("what time is it?"));
    proven_result_size_t sent = proven_net_write_all(&client, request, until);
    EXAMPLE_REQUIRE(sent.err == PROVEN_OK && sent.value == request.size, "the whole request went");

    /* "I have nothing more to send." The server will read the request, then end of input. */
    EXAMPLE_REQUIRE(proven_net_shutdown_write(&client) == PROVEN_OK, "the client half-closes");

    /* The server reads until the end. A read returns what has arrived - a stream has no message
     * boundaries - so a loop is the only correct way to read "all of it". */
    proven_byte_t buf[64];
```

```
    proven_size_t got = 0;
    for (;;) {
        proven_result_size_t r = proven_net_read(&server, (proven_mem_mut_t){ buf + got, sizeof buf - got },
until);
        if (r.err == PROVEN_ERR_EOF) break;                 /* the client finished */
        EXAMPLE_REQUIRE(r.err == PROVEN_OK, "a read that is not the end succeeds");
        if (r.err != PROVEN_OK) break;
        got += r.value;
    }
    EXAMPLE_REQUIRE(got == request.size, "the server received the whole request");

    /* A single write may send only part; the count says how much. Here it all fits. */
    proven_mem_view_t reply = proven_mem_view_from_u8(PROVEN_LIT("time to close"));
    proven_result_size_t w = proven_net_write(&server, reply, until);
    EXAMPLE_REQUIRE(w.err == PROVEN_OK && w.value == reply.size, "a short reply goes in one write");

    proven_result_size_t r = proven_net_read(&client, (proven_mem_mut_t){ buf, sizeof buf }, until);
    EXAMPLE_REQUIRE(r.err == PROVEN_OK && r.value == reply.size, "the client reads the reply after its half-
close");

    /* Nothing more is coming, and the client will not wait for ever to find that out. A timeout
     * is not damage: the connection is exactly as usable as before. */
    r = proven_net_read(&client, (proven_mem_mut_t){ buf, sizeof buf }, proven_net_deadline_in(50));
    EXAMPLE_REQUIRE(r.err == PROVEN_ERR_TIMEOUT, "silence is a timeout, not a hang");

    /* Every socket is closed by its owner. */
    EXAMPLE_REQUIRE(proven_net_close(&server) == PROVEN_OK, "the server closes its end");
    r = proven_net_read(&client, (proven_mem_mut_t){ buf, sizeof buf }, until);
    EXAMPLE_REQUIRE(r.err == PROVEN_ERR_EOF, "which the client reads as the end");
    EXAMPLE_REQUIRE(proven_net_close(&client) == PROVEN_OK, "the client closes");
    EXAMPLE_REQUIRE(proven_net_listener_close(&listener) == PROVEN_OK, "the listener closes");

    /* With the listener gone, the same address refuses - a different answer from a timeout. */
    proven_net_conn_t nobody;
    EXAMPLE_REQUIRE(proven_net_connect(at, proven_net_deadline_in(15000), &nobody) == PROVEN_ERR_REFUSED,
                    "nothing listening is PROVEN_ERR_REFUSED");

    return EXAMPLE_OK();
}
```

5. UDP

UDP sends messages - datagrams - one at a time, each on its own. A datagram arrives whole or not at all; datagrams may arrive out of order, twice, or never, and nothing tells the sender which.

Reference

API	Intent	Return
`proven_net_udp_open(at, &udp, &bound)`	Open a socket bound to `at`. A client binds `proven_net_addr_any(family, 0)`.	`proven_err_t`: `BUSY`, `PERMISSION`, `UNSUPPORTED` (a Unix-domain address).
`proven_net_udp_send_to(&udp, to, data, until)`	Send one datagram: all of `data` or nothing.	`proven_err_t`: `OUT_OF_BOUNDS` when `data` is larger than a datagram can be.
`proven_net_udp_recv_from(&udp, dest, &from, until)`	Receive one datagram; `from` is optional.	`proven_result_size_t`: `OUT_OF_BOUNDS` with `.value == dest.size` when it was cut; `TIMEOUT`.

`proven_net_udp_addr(&udp, &out)`	Where the socket is bound.	`proven_err_t`.
`proven_net_udp_close(&udp)`	Close. Safe on a socket that is not open.	`proven_err_t`.
`proven_net_udp_is_open(&udp)`	Whether the value holds an open socket.	`bool`.

Cautions, and what goes wrong

"Sent" means it left, not that it arrived. A successful `proven_net_udp_send_to` has handed the datagram to the network. A protocol that needs an answer waits for one with a deadline and sends again when it does not come.

A zero-length datagram is a message. On a stream, zero bytes would mean the end, which is why `proven_net_read` never returns it. A datagram can really be empty, so `proven_net_udp_recv_from` returning success with `.value` 0 is not an end and not an error.

A buffer that is too small loses the rest. The part of a datagram that did not fit is not kept for the next receive - it is gone.

Wrong - a buffer sized by guess, and the error ignored:

```
proven_byte_t buf[64];
proven_result_size_t r = proven_net_udp_recv_from(&udp, buf_mem, &from, until);
parse(buf, r.value);      /* wrong: on OUT_OF_BOUNDS this is the first 64 bytes of something longer */
```

Correct - size the buffer for the largest message the protocol allows, and treat `PROVEN_ERR_OUT_OF_BOUNDS` as a malformed packet.

Compiled and run by the test suite:

```
/*
 * UDP: messages, not a stream. Each send is one datagram and each receive is one datagram,
 * with the sender's address attached - and no promise that it arrives.
 */

int main(void) {
    proven_net_udp_t server, client;
    proven_net_addr_t server_at, client_at;
    proven_err_t err = proven_net_udp_open(proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0), &server,
&server_at);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK && proven_net_udp_is_open(&server), "the server's socket opens");
    EXAMPLE_REQUIRE(proven_net_udp_open(proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0), &client, NULL) ==
PROVEN_OK,
                    "and the client's");
    EXAMPLE_REQUIRE(proven_net_udp_addr(&client, &client_at) == PROVEN_OK && client_at.port != 0,
                    "a socket can be asked where it is bound");

    proven_net_deadline_t until = proven_net_deadline_in(2000);

    /* Two sends are two datagrams. Success means each left this machine. */
    EXAMPLE_REQUIRE(proven_net_udp_send_to(&client, server_at, proven_mem_view_from_u8(PROVEN_LIT("first")),
until) == PROVEN_OK &&
                    proven_net_udp_send_to(&client, server_at, proven_mem_view_from_u8(PROVEN_LIT("second
message")), until) == PROVEN_OK,
                    "two datagrams leave");
```

```
    /* Each receive is one whole datagram and says who sent it. */
    proven_byte_t buf[32];
    proven_net_addr_t from;
    proven_result_size_t r = proven_net_udp_recv_from(&server, (proven_mem_mut_t){ buf, sizeof buf }, &from,
until);
    EXAMPLE_REQUIRE(r.err == PROVEN_OK && r.value == 5, "the first datagram, alone: five bytes");
    EXAMPLE_REQUIRE(proven_net_addr_eq(&from, &client_at), "from the client's address");

    /* A buffer that is too small cuts the datagram and says so. The rest of it is gone - the
     * next receive is the next datagram - so size the buffer for the largest message. */
    proven_byte_t small[6];
    r = proven_net_udp_recv_from(&server, (proven_mem_mut_t){ small, sizeof small }, &from, until);
    EXAMPLE_REQUIRE(r.err == PROVEN_ERR_OUT_OF_BOUNDS && r.value == sizeof small,
                    "fourteen bytes into six: PROVEN_ERR_OUT_OF_BOUNDS, six kept");

    /* The reply goes to the address the datagram came from. */
    EXAMPLE_REQUIRE(proven_net_udp_send_to(&server, from, proven_mem_view_from_u8(PROVEN_LIT("ack")), until)
== PROVEN_OK,
                    "the server answers the sender");
    r = proven_net_udp_recv_from(&client, (proven_mem_mut_t){ buf, sizeof buf }, NULL, until);
    EXAMPLE_REQUIRE(r.err == PROVEN_OK && r.value == 3, "and the client receives it");

    /* Nothing else is coming. A lost datagram looks exactly like this, which is why every
     * receive has a deadline and every UDP protocol has a retry. */
    r = proven_net_udp_recv_from(&client, (proven_mem_mut_t){ buf, sizeof buf }, NULL,
proven_net_deadline_in(50));
    EXAMPLE_REQUIRE(r.err == PROVEN_ERR_TIMEOUT, "silence is a timeout");

    EXAMPLE_REQUIRE(proven_net_udp_close(&client) == PROVEN_OK && proven_net_udp_close(&server) == PROVEN_OK,
                    "both sockets are closed by their owner");
    return EXAMPLE_OK();
}
```

6. Many sockets, one thread

A deadline bounds a wait on one socket. A server waits on all of its sockets at once and serves whichever needs it: that is `proven_net_poll`.

Reference

API	Intent	Return
`proven_net_listener_handle(&l)`, `proven_net_conn_handle(&c)`, `proven_net_udp_handle(&u)`	A socket as the poll sees it, whatever its kind.	`proven_net_handle_t`; not `valid` for a socket that is not open.
`proven_net_poll(items, count, until, &ready)`	Wait until at least one item is ready. Up to `PROVEN_NET_POLL_INLINE_MAX` (64) items.	`proven_err_t`: `TIMEOUT` when none was (`ready` is 0); `OUT_OF_BOUNDS` for too many items; `INVALID_ARG` for an invalid handle.
`proven_net_poll_with(scratch, items, count, until, &ready)`	The same for any number of items, using memory you supply.	`proven_err_t`, and `OUT_OF_BOUNDS` when `scratch` is too small.
`proven_net_poll_scratch_size(count)`	How much scratch `count` items need.	`proven_size_t`; `SIZE_MAX` when it cannot be represented.

```
typedef struct {
    proven_net_handle_t handle;
    proven_u8 want;   /* PROVEN_NET_READABLE and/or PROVEN_NET_WRITABLE */
```

```
    proven_u8 got;    /* filled in: zero, or what is ready - which may include PROVEN_NET_FAILED */
} proven_net_poll_item_t;
```

Ready means "will not wait", not "will succeed". A readable connection may hold data, or the end, or an error; a readable listener has a connection to accept. So the call that follows a ready report takes `PROVEN_NET_DONT_WAIT`, and its result is what you act on. An item reported `PROVEN_NET_FAILED` is in error or hung up; the next read or write on it says which.

Ask for writability only when you have something to write. An idle connection is almost always writable, so a loop that always asks is woken at once, every time, and spins a processor doing nothing.

Wrong:

```
items[i].want = PROVEN_NET_READABLE | PROVEN_NET_WRITABLE;   /* wrong when nothing is queued: never sleeps */
```

Correct - `PROVEN_NET_WRITABLE` only for a connection whose last write returned less than you asked or timed out, and only until that data has gone.

`proven_net_poll_with` exists because the operating system wants an array of its own and this library does not allocate one behind your back: you give it working memory, sized by `proven_net_poll_scratch_size`, and reuse it for the next call. Readiness here is `poll` on POSIX and `WSAPoll` on Windows - the right tool for dozens or hundreds of sockets. For thousands, of which few are busy at a time, use the selector of section 9.

Compiled and run by the test suite:

```
/*
 * One thread, many sockets: wait until any of them needs attention, then serve the ones
 * that do. This is the loop inside every single-threaded server.
 *
 * "Ready" means the next call of that kind will not wait - so each socket reported ready is
 * handled with PROVEN_NET_DONT_WAIT, and the loop never blocks anywhere but in the poll.
 */

int main(void) {
    proven_net_listener_t listener;
    proven_net_addr_t at;
    proven_err_t err = proven_net_listen(proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0), 16, &listener,
&at);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK, "the listener opens");

    /* Two clients connect and each sends one message. */
    proven_net_conn_t clients[2];
    for (int i = 0; i < 2; ++i) {
        EXAMPLE_REQUIRE(proven_net_connect(at, proven_net_deadline_in(5000), &clients[i]) == PROVEN_OK, "a
client connects");
    }
    EXAMPLE_REQUIRE(proven_net_write_all(&clients[0], proven_mem_view_from_u8(PROVEN_LIT("from the first")),
proven_net_deadline_in(5000)).err == PROVEN_OK &&
                    proven_net_write_all(&clients[1], proven_mem_view_from_u8(PROVEN_LIT("from the second")),
proven_net_deadline_in(5000)).err == PROVEN_OK,
                    "each sends a message");

    /* The server: a listener and up to two connections, in one list. */
    proven_net_conn_t served[2] = {0};
    proven_size_t served_count = 0;
```

```
    proven_size_t bytes_seen = 0;

    proven_net_deadline_t until = proven_net_deadline_in(5000);
    while (bytes_seen < 14 + 15) {
        proven_net_poll_item_t items[3];
        proven_size_t count = 0;
        items[count++] = (proven_net_poll_item_t){ .handle = proven_net_listener_handle(&listener), .want =
PROVEN_NET_READABLE };
        for (proven_size_t i = 0; i < served_count; ++i) {
            items[count++] = (proven_net_poll_item_t){ .handle = proven_net_conn_handle(&served[i]), .want =
PROVEN_NET_READABLE };
        }

        proven_size_t ready = 0;
        err = proven_net_poll(items, count, until, &ready);
        EXAMPLE_REQUIRE(err == PROVEN_OK && ready > 0, "something becomes ready before the deadline");
        if (err != PROVEN_OK) break;

        /* A readable listener has a connection waiting. */
        if (items[0].got & PROVEN_NET_READABLE) {
            EXAMPLE_REQUIRE(served_count < 2 &&
                            proven_net_accept(&listener, PROVEN_NET_DONT_WAIT, &served[served_count], NULL) ==
PROVEN_OK,
                            "the accept that poll predicted does not wait");
            served_count++;
        }
        /* A readable connection has data, or its end. `items[1 + i]` was built from `served[i]`. */
        for (proven_size_t i = 1; i < count; ++i) {
            if (!items[i].got) continue;
            proven_byte_t buf[64];
            proven_result_size_t r = proven_net_read(&served[i - 1], (proven_mem_mut_t){ buf, sizeof buf },
PROVEN_NET_DONT_WAIT);
            EXAMPLE_REQUIRE(r.err == PROVEN_OK, "the read that poll predicted does not wait");
            bytes_seen += r.value;
        }
    }
    EXAMPLE_REQUIRE(served_count == 2 && bytes_seen == 29, "both clients were accepted and both messages read,
in one thread");

    /* proven_net_poll takes up to PROVEN_NET_POLL_INLINE_MAX sockets. A server with more passes
     * its own working memory: the OS wants an array, and this library does not allocate one. */
    proven_net_poll_item_t one = { .handle = proven_net_conn_handle(&served[0]), .want =
PROVEN_NET_WRITABLE };
    proven_byte_t scratch[256];
    proven_size_t ready = 0;
    EXAMPLE_REQUIRE(proven_net_poll_scratch_size(1) <= sizeof scratch, "the scratch for one item is small");
    EXAMPLE_REQUIRE(proven_net_poll_with((proven_mem_mut_t){ scratch, sizeof scratch }, &one, 1,
PROVEN_NET_DONT_WAIT, &ready) == PROVEN_OK &&
                    (one.got & PROVEN_NET_WRITABLE), "an idle connection is writable at once");

    /* A UDP socket waits in the same list as everything else. */
    proven_net_udp_t udp;
    EXAMPLE_REQUIRE(proven_net_udp_open(proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0), &udp, NULL) ==
PROVEN_OK, "a UDP socket");
    proven_net_poll_item_t quiet = { .handle = proven_net_udp_handle(&udp), .want = PROVEN_NET_READABLE };
    EXAMPLE_REQUIRE(proven_net_poll(&quiet, 1, proven_net_deadline_in(30), &ready) == PROVEN_ERR_TIMEOUT &&
ready == 0,
                    "nothing arrives on it: PROVEN_ERR_TIMEOUT, the idle tick of an event loop");
    (void)proven_net_udp_close(&udp);
```

```
    for (int i = 0; i < 2; ++i) {
        (void)proven_net_close(&clients[i]);
        (void)proven_net_close(&served[i]);
    }
    (void)proven_net_listener_close(&listener);
    return EXAMPLE_OK();
}
```

7. Transports

A protocol - a line-based command set, an HTTP exchange - should be written once and not know what carries its bytes. Today that is a TCP connection. Under TLS it is an encrypted session over one. In a test it is two memory buffers and no network at all. `proven_transport_t` is that seam.

```
typedef struct {
    void *ctx;
    proven_result_size_t (*read_fn)(void *ctx, proven_mem_mut_t dest, proven_net_deadline_t until);
    proven_result_size_t (*write_fn)(void *ctx, proven_mem_view_t src, proven_net_deadline_t until);
    proven_err_t (*shutdown_fn)(void *ctx);   /* may be NULL */
    proven_err_t (*close_fn)(void *ctx);      /* may be NULL */
} proven_transport_t;
```

It is a small vtable passed by value, the same shape as `proven_allocator_t` (Chapter 2) and `proven_writer_t`, and for the same reason: the caller decides, and nothing is hidden. To supply your own, fill in the struct. The contract is that of the TCP calls: a read never succeeds with zero bytes, and a write reports the bytes that went even when it then fails.

Reference

API	Intent	Return
`proven_net_conn_transport(&conn)`	A connection as a transport. Closing the transport closes the connection.	`proven_transport_t`.
`proven_transport_is_valid(t)`	Whether it has a read and a write function.	`bool`.
`proven_transport_read(t, dest, until)`, `proven_transport_write(t, src, until)`	As `proven_net_read` and `proven_net_write`.	`proven_result_size_t`; `INVALID_ARG` for an invalid transport.
`proven_transport_write_all(t, src, until)`	Write everything, or fail with the count. A transport that takes nothing and reports no error is `PROVEN_ERR_IO`, not an endless loop.	`proven_result_size_t`.
`proven_transport_shutdown(t)`, `proven_transport_close(t)`	End the sending side; close. `PROVEN_OK` when the transport has nothing to do.	`proven_err_t`.
`proven_transport_reader(&state, t, timeout_ms)`	A `proven_reader_t` over a transport, for the line reader and anything else that reads a stream.	`proven_reader_t`; not valid if `state` is NULL or `t` is invalid.
`proven_transport_writer(&state, t, timeout_ms)`	A `proven_writer_t` over a transport, so `proven_fprint` can send.	`proven_writer_t`.

The reader and writer adapters connect a socket to everything in Chapter 5's stream section. A reader has nowhere to pass a deadline, so each read or write through an adapter gets its own: `timeout_ms` from the moment it starts, or no deadline when that is 0. The `proven_transport_stream_t` state is caller-owned: it must outlive the reader or writer and must not move.

The writer sends each piece as it is given. Wrap it in `proven_writer_buffered` when a message is formatted in many small pieces, so that it leaves in few packets - and flush before you wait for the reply.

Compiled and run by the test suite:

```
/*
 * A protocol written once, against proven_transport_t, and run over a TCP connection.
 *
 * The function below does not know what carries its bytes. Today it is a socket; under TLS it
 * will be an encrypted session over a socket; in a test it can be two memory buffers. That is
 * what the interface is for.
 */

/* Send one request line and read one reply line, each with a five-second limit per step. */
static proven_err_t ask(proven_transport_t t, const char *question, proven_u8str_view_t expected) {
    if (!proven_transport_is_valid(t)) return PROVEN_ERR_INVALID_ARG;

    /* The formatter writes straight to the transport. */
    proven_transport_stream_t out_state;
    proven_writer_t out = proven_transport_writer(&out_state, t, 5000);
    proven_err_t err = proven_fprintln(out, "ASK {}", PROVEN_ARG(question)).err;
    if (err != PROVEN_OK) return err;

    /* The line reader reads from it, into a buffer this function owns. */
    proven_transport_stream_t in_state;
    proven_byte_t line_buf[128];
    proven_reader_buffered_t lines;
    (void)proven_reader_buffered(&lines, proven_transport_reader(&in_state, t, 5000),
                                 (proven_mem_mut_t){ line_buf, sizeof line_buf });
    proven_result_u8str_view_t reply = proven_reader_read_line(&lines);
    if (reply.err != PROVEN_OK) return reply.err;
    return proven_u8str_view_eq(reply.val, expected) ? PROVEN_OK : PROVEN_ERR_INVALID_FORMAT;
}

int main(void) {
    proven_net_listener_t listener;
    proven_net_addr_t at;
    proven_err_t err = proven_net_listen(proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0), 4, &listener,
&at);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK, "the listener opens");

    proven_net_conn_t client_conn, server_conn;
    EXAMPLE_REQUIRE(proven_net_connect(at, proven_net_deadline_in(5000), &client_conn) == PROVEN_OK &&
                    proven_net_accept(&listener, proven_net_deadline_in(5000), &server_conn, NULL) ==
PROVEN_OK,
                    "a connected pair");

    /* A connection as a transport. From here on, neither side names a socket. */
    proven_transport_t client = proven_net_conn_transport(&client_conn);
    proven_transport_t server = proven_net_conn_transport(&server_conn);
```

```
    proven_net_deadline_t until = proven_net_deadline_in(5000);

    /* The server's half, done by hand: the reply is written before the client asks, which is
     * fine in one thread because the bytes simply wait in the connection. */
    proven_result_size_t w = proven_transport_write_all(server, proven_mem_view_from_u8(PROVEN_LIT("ANSWER
42\n")), until);
    EXAMPLE_REQUIRE(w.err == PROVEN_OK && w.value == 10, "the server's reply is on its way");

    EXAMPLE_REQUIRE(ask(client, "the question", PROVEN_LIT("ANSWER 42")) == PROVEN_OK,
                    "the protocol function sends its line and reads the reply");

    /* And the server reads what the client wrote, with the raw calls. */
    proven_byte_t buf[64];
    proven_result_size_t r = proven_transport_read(server, (proven_mem_mut_t){ buf, sizeof buf }, until);
    EXAMPLE_REQUIRE(r.err == PROVEN_OK &&
                    proven_u8str_view_eq((proven_u8str_view_t){ buf, r.value }, PROVEN_LIT("ASK the
question\n")),
                    "the request line arrived as written");

    /* A single write reports how much went; here, all of it. */
    w = proven_transport_write(server, proven_mem_view_from_u8(PROVEN_LIT("BYE\n")), until);
    EXAMPLE_REQUIRE(w.err == PROVEN_OK && w.value == 4, "one more line");

    /* Ending and closing go through the same interface. */
    EXAMPLE_REQUIRE(proven_transport_shutdown(server) == PROVEN_OK, "the server ends its sending side");
    r = proven_transport_read(client, (proven_mem_mut_t){ buf, sizeof buf }, until);
    EXAMPLE_REQUIRE(r.err == PROVEN_OK && r.value == 4, "the client reads the last line");
    r = proven_transport_read(client, (proven_mem_mut_t){ buf, sizeof buf }, until);
    EXAMPLE_REQUIRE(r.err == PROVEN_ERR_EOF, "and then the end");

    EXAMPLE_REQUIRE(proven_transport_close(client) == PROVEN_OK && proven_transport_close(server) ==
PROVEN_OK,
                    "closing a transport closes what carries it");
    EXAMPLE_REQUIRE(!proven_net_conn_is_open(&client_conn), "the connection underneath is closed");
    (void)proven_net_listener_close(&listener);
    return EXAMPLE_OK();
}
```

8. A pair, and waking a loop

A loop blocked in `proven_net_poll` sees sockets and nothing else. A thread that has finished some work for it, or wants it to stop, cannot reach it through a variable: the loop is asleep in the operating system. What it needs is a socket that becomes readable when another thread says so. That is a waker.

Reference

API	Intent	Return
`proven_net_pair(&a, &b)`	Two connections joined to each other, with no address: what is written to one is read from the other.	`proven_err_t`: `BUSY` when the system is out of sockets. Close both.
`proven_net_waker_open(&waker)`	Open a waker.	`proven_err_t`: `BUSY`.
`proven_net_waker_handle(&waker)`	The handle to put in a poll list, wanting `PROVEN_NET_READABLE`.	`proven_net_handle_t`.

`proven_net_waker_wake(&waker)`	Make it readable. Safe from any thread, and from several at once.	none; it cannot fail in a way the caller could act on.
`proven_net_waker_drain(&waker)`	Read away the wakes so far. Called by the polling thread once it has woken.	none.
`proven_net_waker_close(&waker)`	Close it.	none.

Drain after you wake. A waker stays readable until it is drained. A loop that forgets polls, returns at once, polls, returns at once - and spins a processor.

Wakes merge. Ten calls to `proven_net_waker_wake` before one drain are one wake. A waker says "look", not how many times, so what the loop should look at lives somewhere else - a queue, a flag - and the loop checks it every time it wakes.

Set the flag, then wake - and look after draining, not before. In the other order a wake can land between the look and the drain and be drained away unseen.

Wrong:

```
if (work_pending()) do_work();        /* wrong: work queued after this line... */
proven_net_waker_drain(&waker);       /* ...has its wake drained here, and waits for the next poll to time out
*/
```

Correct:

```
proven_net_waker_drain(&waker);
if (work_pending()) do_work();
```

Close it last. `proven_net_waker_close` is not safe while another thread can still call `proven_net_waker_wake`: stop or join those threads first.

`proven_net_pair` is what a waker is built from, and is useful alone wherever a test or a program wants a connection without a network. It is a Unix-domain pair on POSIX and a loopback TCP connection on Windows, where nothing else is available on every supported version.

Compiled and run by the test suite:

```
/*
 * A loop blocked in proven_net_poll sees only sockets. To tell it something from another
 * thread - "there is work for you", "stop" - you need a socket that becomes readable when
 * you say so. That is a waker.
 *
 * proven_net_pair is what a waker is made of, and is useful by itself: two connected ends
 * with no address, for handing bytes between two parts of a program.
 */

static void finish_work(void *arg) {
    proven_net_waker_t *waker = arg;
    proven_time_sleep(20);              /* stands for some work */
    proven_net_waker_wake(waker);       /* any thread may call this */
}

int main(void) {
    /* A pair: what goes into one end comes out of the other. */
    proven_net_conn_t a, b;
    proven_err_t err = proven_net_pair(&a, &b);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        return EXAMPLE_OK();
    }
```

```
    EXAMPLE_REQUIRE(err == PROVEN_OK, "a pair opens");
    EXAMPLE_REQUIRE(proven_net_write_all(&a, proven_mem_view_from_u8(PROVEN_LIT("across")),
proven_net_deadline_in(1000)).err == PROVEN_OK, "write to one end");
    proven_byte_t buf[16];
    proven_result_size_t got = proven_net_read(&b, (proven_mem_mut_t){ buf, sizeof buf },
proven_net_deadline_in(1000));
    EXAMPLE_REQUIRE(got.err == PROVEN_OK && got.value == 6, "read from the other");
    (void)proven_net_close(&a);
    (void)proven_net_close(&b);

    /* A waker in a poll list. Nothing is readable, so the poll would wait its full second... */
    proven_net_waker_t waker;
    EXAMPLE_REQUIRE(proven_net_waker_open(&waker) == PROVEN_OK, "a waker opens");
    proven_job_sys_t *jobs = NULL;
    EXAMPLE_REQUIRE(proven_job_system_init(proven_heap_allocator(), 1, 4, &jobs) == PROVEN_OK, "a worker
thread");
    EXAMPLE_REQUIRE(proven_job_submit_ex(jobs, finish_work, &waker) == PROVEN_OK, "work is handed to it");

    proven_net_poll_item_t item = { .handle = proven_net_waker_handle(&waker), .want = PROVEN_NET_READABLE };
    proven_size_t ready = 0;
    proven_time_t start = proven_time_monotonic_now();
    err = proven_net_poll(&item, 1, proven_net_deadline_in(1000), &ready);
    proven_i64 waited_ms = (proven_time_monotonic_now() - start) / 1000000;
    /* ...but the worker woke it. */
    EXAMPLE_REQUIRE(err == PROVEN_OK && ready == 1 && waited_ms < 900, "the poll returns when the worker says
so, not at its deadline");

    /* Drain it, or it stays readable and the next poll returns at once for nothing. */
    proven_net_waker_drain(&waker);
    EXAMPLE_REQUIRE(proven_net_poll(&item, 1, proven_net_deadline_in(20), &ready) == PROVEN_ERR_TIMEOUT,
"drained: quiet again");

    /* Several wakes before a drain are one wake. */
    proven_net_waker_wake(&waker);
    proven_net_waker_wake(&waker);
    proven_net_waker_drain(&waker);
    EXAMPLE_REQUIRE(proven_net_poll(&item, 1, proven_net_deadline_in(20), &ready) == PROVEN_ERR_TIMEOUT, "two
wakes, one drain: quiet");

    /* Close the waker only when no thread can still wake it: stop the workers first. */
    proven_job_system_close(jobs);
    proven_job_system_destroy(jobs);
    proven_net_waker_close(&waker);
    return EXAMPLE_OK();
}
```

9. A selector: thousands of sockets

`proven_net_poll` is handed its whole list on every call, and the system examines every entry on every call. With a few dozen sockets that is nothing. With ten thousand connections of which five have something to say, it is ten thousand entries copied and examined to find five - on every turn of the loop.

A selector asks the same question the other way round. You register a socket once; the system remembers it; a wait returns the sockets that are ready and does not look at the others.

Reference

API	Intent	Return

`proven_net_selector_create(alloc, &selector)`	Make a selector.	`proven_err_t`: `INVALID_ARG`; `NOMEM`; `BUSY` when out of descriptors.
`proven_net_selector_create_poll(alloc, &selector)`	Make one of the portable kind, whatever the system offers.	the same.
`proven_net_selector_kind(selector)`	Which facility it is built on.	`PROVEN_NET_SELECTOR_EPOLL`, `_KQUEUE`, `_IOCP` (Windows) or `_POLL`.
`proven_net_selector_add(selector, handle, want, tag)`	Register a socket: what to watch it for, and a pointer that comes back with its events.	`proven_err_t`: `EXISTS` when it is already registered; `INVALID_ARG`; `NOMEM`; `BUSY`.
`proven_net_selector_modify(selector, handle, want, tag)`	Change what is asked of it, and its tag.	`proven_err_t`: `NOT_FOUND`.
`proven_net_selector_remove(selector, handle)`	Take it out.	`proven_err_t`: `NOT_FOUND`.
`proven_net_selector_wait(selector, events, cap, until, &count)`	Wait until something is ready or until `until`; report up to `cap`.	`proven_err_t`: `TIMEOUT` when nothing was (`count` is 0).
`proven_net_selector_count(selector)`	How many sockets are registered.	`proven_size_t`.
`proven_net_selector_destroy(selector)`	Free it. The sockets are not closed.	none.

```
typedef struct {
    void *tag;        /* what you registered with the socket */
    proven_u8 got;    /* PROVEN_NET_READABLE, PROVEN_NET_WRITABLE and/or PROVEN_NET_FAILED */
} proven_net_ready_t;
```

A socket of which nothing is asked is still told when its connection is lost - once. The system reports an error or a hang-up whether you asked about the socket or not, and goes on reporting it. Registered with `want` of 0 - a connection you have paused - the socket is reported with `PROVEN_NET_FAILED` (or as readable and writable, for a hang-up) one time, and then left alone until you change what is asked of it with `proven_net_selector_modify`; then it is reported again. Before v0.38.0 it was reported on every wait, so a loop with one such socket never slept. (So on Linux and with the portable kind. The `kqueue` kind registers nothing for a socket of which nothing is asked, and reports nothing for it until something is; that kind has not been run.)

What you get depends on the system, and the selector says which. On Linux it is `epoll`; on the BSDs and macOS it is `kqueue`. There a wait costs in proportion to the sockets that are ready. On Windows it is an I/O completion port (`PROVEN_NET_SELECTOR_IOCP`): a connection is watched for reading by a receive of zero bytes, and room to write, a connection waiting to be accepted, a datagram, or a connection lost while nothing is asked of it, by a notice of the system's thread pool - about 500 bytes and two handles per socket while it is asked, so that cost falls on listeners, datagram sockets, connections waiting to write and those of which nothing is asked, not on connections waiting to read. Each wait checks what was so reported with the call `proven_net_poll` uses, so it reports what the portable kind would. A connection it has watched for reading stays bound to its port while it is open. Anywhere else it is the portable kind: the same interface kept over the call `proven_net_poll` uses, which still examines every socket on every wait. It behaves the same and it does not scale the same. `proven_net_selector_kind` tells you; a server that means to hold tens of thousands of connections should look.

Remove a socket before you close it. A closed socket's number is given to the next socket opened. A selector that still holds the old registration then reports events for a socket that is not the one you registered - with the old tag - or, in the portable kind, reports a failure on every wait.

Wrong:

```
proven_net_close(&conn);                                  /* wrong: the selector still holds its number */
proven_net_selector_remove(selector, proven_net_conn_handle(&conn));   /* too late: the handle is no longer
valid */
```

Correct - `proven_net_selector_remove` first, then `proven_net_close`.

Readiness is level-triggered. A socket is reported for as long as it is ready, exactly as with `proven_net_poll`: an event you do not act on comes back on the next wait. So the rules of section 6 hold unchanged - follow an event with a call that takes `PROVEN_NET_DONT_WAIT`, and ask for writability only while you have something to write.

The tag is yours, and so is its lifetime. The selector returns the pointer you gave it and never looks through it. If the record it points at is freed while events for it may already have been taken from the selector, the next loop iteration reads freed memory. The pattern that avoids this: when you close a connection in the middle of handling a batch of events, mark its record and free it after the batch.

One thread. A selector is not safe to use from several threads. To make a waiting loop act on something another thread did, register a waker (section 8) and wake it.

A socket ready both ways may be one event or two. `kqueue` reports readability and writability separately; handle each event for what it says rather than expecting one per socket.

Compiled and run by the test suite:

```
/*
 * A selector: register each socket once, and be told only about the ones that are ready.
 *
 * The loop is the one from the poll example - wait, then serve what is ready without waiting -
 * but nothing is rebuilt between waits, and a thousand idle connections cost a wait nothing.
 */

typedef struct {
    proven_net_conn_t conn;
    int number;
    bool open;
} ex_client_t;

int main(void) {
    proven_net_listener_t listener;
    proven_net_addr_t at;
    proven_err_t err = proven_net_listen(proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0), 16, &listener,
&at);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK, "the listener opens");

    proven_net_selector_t *selector = NULL;
    EXAMPLE_REQUIRE(proven_net_selector_create(proven_heap_allocator(), &selector) == PROVEN_OK, "a
selector");
    /* epoll on Linux, kqueue on the BSDs and macOS; elsewhere the same interface over poll. */
    proven_net_selector_kind_t kind = proven_net_selector_kind(selector);
    printf("selector kind: %s\n", kind == PROVEN_NET_SELECTOR_EPOLL ? "epoll" : kind ==
PROVEN_NET_SELECTOR_KQUEUE ? "kqueue" : kind == PROVEN_NET_SELECTOR_IOCP ? "iocp" : "poll");

    /* The tag is what comes back with an event: a pointer to your own record of the socket.
     * The listener's tag here is the listener itself. */
```

```
    EXAMPLE_REQUIRE(proven_net_selector_add(selector, proven_net_listener_handle(&listener),
PROVEN_NET_READABLE, &listener) == PROVEN_OK,
                    "the listener is registered, once");

    /* Three clients connect; the second and third say something. */
    proven_net_conn_t far[3];
    for (int i = 0; i < 3; ++i) EXAMPLE_REQUIRE(proven_net_connect(at, proven_net_deadline_in(5000), &far[i])
== PROVEN_OK, "a client connects");
    EXAMPLE_REQUIRE(proven_net_write_all(&far[1], proven_mem_view_from_u8(PROVEN_LIT("from one")),
proven_net_deadline_in(5000)).err == PROVEN_OK &&
                    proven_net_write_all(&far[2], proven_mem_view_from_u8(PROVEN_LIT("from two")),
proven_net_deadline_in(5000)).err == PROVEN_OK, "two of them send");

    ex_client_t clients[3] = {0};
    int accepted = 0, messages = 0;
    for (int round = 0; round < 20 && messages < 2; ++round) {
        proven_net_ready_t ready[8];
        proven_size_t count = 0;
        err = proven_net_selector_wait(selector, ready, 8, proven_net_deadline_in(1000), &count);
        if (err == PROVEN_ERR_TIMEOUT) continue;        /* the idle tick of a loop */
        EXAMPLE_REQUIRE(err == PROVEN_OK, "something is ready");
        for (proven_size_t i = 0; i < count; ++i) {
            if (ready[i].tag == &listener) {
                /* Ready means "will not wait": take every pending connection, without waiting. */
                while (accepted < 3 && proven_net_accept(&listener, PROVEN_NET_DONT_WAIT,
&clients[accepted].conn, NULL) == PROVEN_OK) {
                    ex_client_t *c = &clients[accepted];
                    c->number = accepted++;
                    c->open = true;
                    EXAMPLE_REQUIRE(proven_net_selector_add(selector, proven_net_conn_handle(&c->conn),
PROVEN_NET_READABLE, c) == PROVEN_OK,
                                    "each accepted connection is registered, with its record as the tag");
                }
            } else {
                ex_client_t *c = ready[i].tag;
                proven_byte_t buf[32];
                proven_result_size_t got = proven_net_read(&c->conn, (proven_mem_mut_t){ buf, sizeof buf },
PROVEN_NET_DONT_WAIT);
                if (got.err == PROVEN_OK) messages++;
            }
        }
    }
    EXAMPLE_REQUIRE(accepted == 3 && messages == 2, "three connections accepted, two messages read");
    EXAMPLE_REQUIRE(proven_net_selector_count(selector) == 4, "four sockets registered: the listener and three
connections");

    /* The first connection never spoke, so it was never reported: nothing to do for it. Now ask
     * a different question of it - may I write? - and it is reported at once. */
    proven_net_ready_t one[4];
    proven_size_t count = 0;
    EXAMPLE_REQUIRE(proven_net_selector_wait(selector, one, 4, proven_net_deadline_in(30), &count) ==
PROVEN_ERR_TIMEOUT, "all quiet: nothing is reported");
    EXAMPLE_REQUIRE(proven_net_selector_modify(selector, proven_net_conn_handle(&clients[0].conn),
PROVEN_NET_WRITABLE, &clients[0]) == PROVEN_OK, "ask about writing instead");
    EXAMPLE_REQUIRE(proven_net_selector_wait(selector, one, 4, proven_net_deadline_in(1000), &count) ==
PROVEN_OK && count == 1 &&
                    one[0].tag == &clients[0] && (one[0].got & PROVEN_NET_WRITABLE), "an idle connection is
writable");
    /* Ask for writability only while you have something to write, or every wait returns at once. */
    EXAMPLE_REQUIRE(proven_net_selector_modify(selector, proven_net_conn_handle(&clients[0].conn),
```

```
PROVEN_NET_READABLE, &clients[0]) == PROVEN_OK, "back to reading");

    /* Remove a socket BEFORE closing it. */
    for (int i = 0; i < 3; ++i) {
        EXAMPLE_REQUIRE(proven_net_selector_remove(selector, proven_net_conn_handle(&clients[i].conn)) ==
PROVEN_OK, "removed");
        (void)proven_net_close(&clients[i].conn);
        (void)proven_net_close(&far[i]);
    }
    EXAMPLE_REQUIRE(proven_net_selector_remove(selector, proven_net_listener_handle(&listener)) == PROVEN_OK
&&
                    proven_net_selector_count(selector) == 0, "and the listener: the selector is empty");
    (void)proven_net_listener_close(&listener);

    /* The portable kind can be asked for by name - to compare the two, or to run the code path
     * of a system that offers nothing better. */
    proven_net_selector_t *portable = NULL;
    EXAMPLE_REQUIRE(proven_net_selector_create_poll(proven_heap_allocator(), &portable) == PROVEN_OK &&
                    proven_net_selector_kind(portable) == PROVEN_NET_SELECTOR_POLL, "a selector of the poll
kind");
    proven_net_selector_destroy(portable);
    proven_net_selector_destroy(selector);
    return EXAMPLE_OK();
}
```

10. What the platforms do differently

The calls behave the same on Linux, the BSDs, macOS and Windows. These are the differences the library absorbs, written down because they are the ones that bite when sockets are used directly:

The trap	What `net.h` does
A write to a closed connection raises `SIGPIPE` and kills the process (POSIX).	Every send suppresses the signal; the write returns `PROVEN_ERR_RESET`. No signal handler is installed for you.
Winsock must be started before any call, name resolution included.	Started once, on first use.
On Windows `SO_REUSEADDR` lets another process bind your port and take your connections.	A listener uses `SO_REUSEADDR` on POSIX and `SO_EXCLUSIVEADDRUSE` on Windows: a second listener on one address is `PROVEN_ERR_BUSY` everywhere.
An IPv6 listener also accepts IPv4 on some systems and not on others.	IPv6 sockets are IPv6-only everywhere. A dual-stack server opens two listeners.
A non-blocking Windows send accepts a buffer of any size and queues it all.	Sends are offered in bounded pieces, so a full buffer pushes back - and a deadline can fire - as it does elsewhere.
Windows reports a peer's orderly close to `WSAPoll` as a hang-up, not as readable.	Reported as readable, as on POSIX: the read that follows returns `PROVEN_ERR_EOF`.
A UDP send to a closed port makes a *later* receive on the same Windows socket fail.	That report is turned off; a receive answers for the datagram it received.
A child process inherits open sockets.	Every socket is created not inheritable (close-on-exec).
Winsock answers a Unix-domain connect to a missing path as "refused".	`PROVEN_ERR_NOT_FOUND`, as on POSIX.

Two things differ and stay different. Windows waits about two seconds before it reports a refused loopback connection, where POSIX answers at once - allow for it in the deadline. And Unix-domain sockets need Windows 10 version 1803 or later; where the family is missing, `proven_net_listen` and `proven_net_connect` return `PROVEN_ERR_UNSUPPORTED`.

11. What is not here

- **Encryption.** A connection made here is not encrypted. TLS is a layer over the transport interface: Chapter 14.
- **A deadline on name resolution.** See section 2.
- **Windows at scale, beyond one machine.** The selector and the event loop of Chapter 15 are built on I/O completion ports there. Measured once on a Windows 11 virtual machine, over loopback: with 10,000 connections held and ten making requests, the blocking server of Chapter 11 (on this selector) answered about 52,000 requests a second with a median round trip of 0.15 ms - as with 1,000 held; the event loop's figures are in Chapter 15, section 14. Not measured there: a client on another machine, datagram sockets on the selector, and more than about 16,000 connections (the machine's ports ran out with both ends on it).
- **Unix-domain datagrams, raw sockets, multicast, socket options beyond `TCP_NODELAY`.**
- **HTTP.** The message codec is Chapter 10 the client and the server that drive it over these sockets are Chapter 11.

Chapter 10: URLs and HTTP Messages

Part V - Talking to the operating system. Prerequisites: Part II (1, 2, 3). Chapter 9 is where the bytes come from, but nothing here needs it. After this chapter you can take a URL apart, turn a request path into one that is safe to open, parse an HTTP/1.1 request or response as it arrives, tell how long its body is, decode that body, and write a message that nobody else's input can corrupt - and you have the pieces that sit around a message: a redirect resolved, a form, a byte range, a file upload, an authentication answer, a cookie jar and an event stream.

This chapter covers `url.h`, `http.h`, `http_auth.h`, `http_cookie.h` and `sse.h`. All of it is text handling with no socket and no file; only the cookie jar allocates, from the allocator you give it. Unlike Chapter 9 these headers are available in a freestanding build, and `PROVEN_NO_NET` does not remove them. The client and the server that put them to work over sockets are Chapter 11.

Table of contents

1. A codec, not a server
2. URLs
3. A path that is safe to use
4. Parsing a head
5. How long is the body
6. Decoding a body
7. Writing a message
8. Dates
9. References and forms
10. Ranges and multipart bodies
11. Authentication
12. Cookies
13. Server-sent events
14. What is not here

1. A codec, not a server

`http.h` does not read from anything and does not write to anything. You hand it bytes you already have and it tells you what they say; you hand it memory and it writes a message into it. What carries those bytes - a blocking socket, an event loop, a TLS session, a test - is not its business, and that is the point: the part of HTTP that is hard to get right is the same in all of them, and here it is written once.

Two properties follow, and the rest of the chapter leans on both.

A prefix is never an error. Bytes arrive in whatever pieces the network makes. If what has arrived so far is the beginning of a valid message, the parser answers `PROVEN_ERR_NEED_MORE` - never a failure, never a premature success. So the loop around it is always the same: parse; if it needs more, read more; otherwise act.

Ambiguity is refused, not resolved. An HTTP message is routinely read by two programs - a proxy in front and a server behind. Wherever the two read the same bytes differently, one can be shown a request the other never sees. That is *request smuggling*, and it lives entirely in the places where a lenient parser quietly picks an interpretation. This one does not pick:

What is refused	Why it matters
A line ending that is a bare LF, or a bare CR	One parser sees one line where another sees two

Whitespace between a header name and its colon	`Content-Length : 5` is a header to one parser and not to another
A header line starting with a space or tab (obsolete folding)	A continuation to one parser, a new field to another
More than one `Content-Length`, or one that is not plain digits	Two lengths means two opinions about where the body ends
`Transfer-Encoding` together with `Content-Length`	The classic: each parser believes a different one
Any transfer coding but a single `chunked`	A parser that skips an unknown coding frames the body differently
A chunk size with whitespace, a sign, a `0x`, or more than 16 digits	Chunk-size parsing is the other classic

Each row is something deployed software accepts and published attacks use. A refusal is an ordinary error value - answer 400 and close.

2. URLs

`proven_url_parse` splits an absolute URL into views of the text you gave it:

```
https://user:pw@example.com:8443/docs/a%20b?q=1#top
\___/   \_____/ \_________/ \__/\_________/ \_/ \_/
scheme  userinfo    host    port   path    query fragment
```

```
typedef struct {
    proven_u8str_view_t scheme, userinfo, host, path, query, fragment;
    proven_u16 port;
    bool has_userinfo, has_port, has_query, has_fragment;
    bool host_is_ipv6;      /* written in brackets; `host` is without them */
} proven_url_t;
```

Parsing does not decode. Every component comes back as written, percent signs included. Decoding is a separate step taken on the one piece you are about to use - because a URL decoded as a whole cannot be taken apart again. A `%2F` that was data inside a path segment becomes a `/` that separates segments; a `%26` inside a query value becomes an `&` that ends it.

Reference

API	Intent	Return
`proven_url_parse(text, &url)`	Split `scheme://authority/path?query#fragment`. ASCII only, no spaces, every `%` followed by two hex digits.	`proven_err_t`: `INVALID_FORMAT` for anything else; `url` is then untouched.
`proven_url_default_port(scheme)`	80 for `http`/`ws`, 443 for `https`/`wss`, 21 for `ftp`.	`proven_u16`; 0 for an unknown scheme.
`proven_url_effective_port(&url)`	The port to connect to: the one written, else the scheme's.	`proven_u16`.
`proven_url_scheme_is(&url, scheme)`	Compare the scheme without case.	`bool`.
`proven_url_split_target(target, &path, &query, &has_query)`	Split an HTTP request target: `/a?b`, `http://h/a?b`, or `*`.	`proven_err_t`: `INVALID_FORMAT` otherwise.

`proven_url_query_iter(query)`, `proven_url_query_next(&it, &name, &value)`	Walk `name=value&...`, yielding encoded views. Empty pairs are skipped.	iterator; `bool` (false at the end).
`proven_url_percent_decode(in, out, &written)`	`%XX` to bytes. `out` may be `in`. `+` stays `+`.	`proven_err_t`: `INVALID_FORMAT` for a bad escape; `OUT_OF_BOUNDS`.
`proven_url_form_decode(in, out, &written)`	The same, and `+` is a space: for query names and values.	`proven_err_t`.
`proven_url_encode_component(in, out, &written)`	Encode for ONE component: all but letters, digits and `-._~` become `%XX`.	`proven_err_t`: `OUT_OF_BOUNDS`; `OVERFLOW`.
`proven_url_encode_path(in, out, &written)`	The same, keeping `/`.	`proven_err_t`.
`proven_url_form_encode(in, out, &written)`	The same, writing a space as `+`.	`proven_err_t`.
`proven_url_encoded_size_max(n)`	Bytes always enough to encode `n`.	`proven_size_t`; `SIZE_MAX` if unrepresentable.

This is RFC 3986, not the URL algorithm browsers implement. A browser repairs what it is given - backslashes become slashes, spaces are encoded, a missing scheme is guessed. Nothing here is repaired: text that would need it is `PROVEN_ERR_INVALID_FORMAT`. A program that accepts URLs typed by people should say so to them rather than guess.

Cautions, and what goes wrong
Decode a component, never the whole URL.

Wrong:

```
proven_url_percent_decode(whole_url, buf, &n);      /* wrong: "a%2Fb" and "a/b" are now the same path */
proven_url_parse(decoded, &url);
```

Correct - parse first, then decode the one component you need, with the decoder that matches it: `proven_url_form_decode` for a query name or value, `proven_url_path_resolve` for a path.

Decoded text is bytes. `proven_url_percent_decode` returns whatever was encoded: a NUL, a newline, bytes that are not UTF-8. It is the sender's data. Validate it for what you are about to do with it.

The userinfo is not the host. In `http://example.com@evil.test/` the host is `evil.test`. The parser gets this right - the host begins after the *last* `@` - but a program that shows a URL to a person, or checks it against an allow-list, must compare `url.host`, never a prefix of the text.

Compiled and run by the test suite:

```
/*
 * URLs: take one apart, decode only the piece you are about to use, and never let a path
 * from the network reach the filesystem without resolving it first.
 */

static bool view_is(proven_u8str_view_t v, const char *text) {
    return proven_u8str_view_eq(v, proven_u8str_view_from_cstr(text));
}

int main(void) {
    /* Parsing gives views of what was written. Nothing is decoded and nothing is copied. */
    proven_url_t url;
    proven_err_t err = proven_url_parse(PROVEN_LIT("https://example.com:8443/docs/a%20b?q=caf%C3%A9+au+lait&
page=2#top"), &url);
    EXAMPLE_REQUIRE(err == PROVEN_OK, "an absolute URL parses");
```

```
    EXAMPLE_REQUIRE(view_is(url.host, "example.com") && url.has_port && url.port == 8443, "host and port");
    EXAMPLE_REQUIRE(view_is(url.path, "/docs/a%20b"), "the path is still encoded");
    EXAMPLE_REQUIRE(url.has_query && url.has_fragment && view_is(url.fragment, "top"), "query and fragment are
there");

    /* Scheme names compare without case; a URL with no port gets the scheme's own. */
    proven_url_t plain;
    EXAMPLE_REQUIRE(proven_url_parse(PROVEN_LIT("HTTP://example.com/"), &plain) == PROVEN_OK, "another URL");
    EXAMPLE_REQUIRE(proven_url_scheme_is(&plain, PROVEN_LIT("http")), "HTTP is http");
    EXAMPLE_REQUIRE(proven_url_effective_port(&plain) == 80 && proven_url_effective_port(&url) == 8443,
                    "the port to connect to: the default, or the one written");
    EXAMPLE_REQUIRE(proven_url_default_port(PROVEN_LIT("wss")) == 443, "wss is 443");

    /* What is not an absolute URL is refused, not repaired. */
    proven_url_t bad;
    EXAMPLE_REQUIRE(proven_url_parse(PROVEN_LIT("example.com/path"), &bad) == PROVEN_ERR_INVALID_FORMAT, "no
scheme: refused");
    EXAMPLE_REQUIRE(proven_url_parse(PROVEN_LIT("http://example.com/a b"), &bad) == PROVEN_ERR_INVALID_FORMAT,
"a raw space: refused");

    /* A query is walked pair by pair; each name and value is decoded on its own, as form data,
     * where '+' means a space. Decoding the whole query first would turn an encoded '&' inside
     * a value into a separator. */
    proven_url_query_iter_t it = proven_url_query_iter(url.query);
    proven_u8str_view_t name, value;
    proven_byte_t text[64];
    proven_size_t n = 0;
    EXAMPLE_REQUIRE(proven_url_query_next(&it, &name, &value) && view_is(name, "q"), "the first pair is q");
    EXAMPLE_REQUIRE(proven_url_form_decode(value, (proven_mem_mut_t){ text, sizeof text }, &n) == PROVEN_OK &&
                    view_is((proven_u8str_view_t){ text, n }, "caf\xc3\xa9 au lait"), "its value, decoded");
    EXAMPLE_REQUIRE(proven_url_query_next(&it, &name, &value) && view_is(name, "page") && view_is(value, "2"),
"the second pair");
    EXAMPLE_REQUIRE(!proven_url_query_next(&it, &name, &value), "and no third");

    /* Plain percent-decoding leaves '+' alone, and yields BYTES - any bytes. */
    EXAMPLE_REQUIRE(proven_url_percent_decode(PROVEN_LIT("a%2Fb+c"), (proven_mem_mut_t){ text, sizeof text },
&n) == PROVEN_OK &&
                    view_is((proven_u8str_view_t){ text, n }, "a/b+c"), "%2F becomes a slash; '+' stays");

    /* Going the other way: encode a value so that it cannot end the component it goes into. */
    proven_mem_view_t file_name = proven_mem_view_from_u8(PROVEN_LIT("report 100%/final?.txt"));
    proven_byte_t enc[96];
    EXAMPLE_REQUIRE(proven_url_encoded_size_max(file_name.size) <= sizeof enc, "three bytes per byte is always
enough");
    EXAMPLE_REQUIRE(proven_url_encode_component(file_name, (proven_mem_mut_t){ enc, sizeof enc }, &n) ==
PROVEN_OK &&
                    view_is((proven_u8str_view_t){ enc, n }, "report%20100%25%2Ffinal%3F.txt"), "one path
segment: even the slash is encoded");
    EXAMPLE_REQUIRE(proven_url_encode_path(proven_mem_view_from_u8(PROVEN_LIT("/my docs/a b.txt")),
(proven_mem_mut_t){ enc, sizeof enc }, &n) == PROVEN_OK &&
                    view_is((proven_u8str_view_t){ enc, n }, "/my%20docs/a%20b.txt"), "a whole path: the
slashes stay");
    EXAMPLE_REQUIRE(proven_url_form_encode(proven_mem_view_from_u8(PROVEN_LIT("a b&c")), (proven_mem_mut_t)
{ enc, sizeof enc }, &n) == PROVEN_OK &&
                    view_is((proven_u8str_view_t){ enc, n }, "a+b%26c"), "form data: a space is '+'");

    /* A server's side of it. The request target is split first... */
    proven_u8str_view_t path, query;
    bool has_query = false;
    EXAMPLE_REQUIRE(proven_url_split_target(PROVEN_LIT("/static/css/../img/logo%20v2.png?v=3"), &path, &query,
```

```
&has_query) == PROVEN_OK &&
                    has_query && view_is(query, "v=3"), "target into path and query");

    /* ...and the path is resolved before anything else looks at it: decoded once, dot segments
     * removed, and refused if it would leave the root. */
    proven_byte_t clean[128];
    EXAMPLE_REQUIRE(proven_url_path_resolve(path, (proven_mem_mut_t){ clean, sizeof clean }, &n) == PROVEN_OK
&&
                    view_is((proven_u8str_view_t){ clean, n }, "/static/img/logo v2.png"), "a clean path under
the root");

    /* The requests a file server exists to refuse. */
    EXAMPLE_REQUIRE(proven_url_path_resolve(PROVEN_LIT("/../etc/passwd"), (proven_mem_mut_t){ clean, sizeof
clean }, &n) == PROVEN_ERR_PERMISSION,
                    "climbing above the root");
    EXAMPLE_REQUIRE(proven_url_path_resolve(PROVEN_LIT("/%2e%2e/etc/passwd"), (proven_mem_mut_t){ clean,
sizeof clean }, &n) == PROVEN_ERR_PERMISSION,
                    "the same climb with the dots encoded");
    EXAMPLE_REQUIRE(proven_url_path_resolve(PROVEN_LIT("/..%2f..%2fetc/passwd"), (proven_mem_mut_t){ clean,
sizeof clean }, &n) == PROVEN_ERR_INVALID_FORMAT,
                    "an encoded slash");
    EXAMPLE_REQUIRE(proven_url_path_resolve(PROVEN_LIT("/file%00.png"), (proven_mem_mut_t){ clean, sizeof
clean }, &n) == PROVEN_ERR_INVALID_FORMAT,
                    "an encoded NUL");
    EXAMPLE_REQUIRE(proven_url_path_resolve(PROVEN_LIT("/%c0%ae%c0%ae/"), (proven_mem_mut_t){ clean, sizeof
clean }, &n) == PROVEN_ERR_INVALID_FORMAT,
                    "an overlong encoding of a dot");

    return EXAMPLE_OK();
}
```

3. A path that is safe to use

A request path names a file, and the sender chooses it. `GET /../../etc/passwd` is the oldest attack on a file server and it has many spellings. `proven_url_path_resolve` is the one function to put between a request path and anything that opens files:

API	Intent	Return
`proven_url_path_resolve(raw_path, out, &written)`	Decode once, refuse what has no place in a file name, resolve `.` and `..`. The result starts with `/`, has no dot or empty segments, and keeps a trailing `/`. `out` needs at most `raw_path.size` bytes.	`proven_err_t`: `PERMISSION` when a `..` would climb above the root; `INVALID_FORMAT` for the refusals below; `OUT_OF_BOUNDS` when `out` is too small.

What it does, in order - and the order is the security property:

1. **Decodes once.** `%2e%2e` becomes `..` and is then treated as `..`. And `%252e%252e` becomes the text `%2e%2e`, which is a strange file name and nothing more - decoding again would turn it into a traversal, which is the *double decoding* bug.
2. **Refuses** an encoded slash (`%2F` - it would split one segment into two after the fact), a backslash written or encoded (a separator on Windows), any control character including NUL (`file%00.png` truncates the name in C), and anything that is not valid UTF-8 once decoded - which is what catches `%c0%ae`, an *overlong* encoding of a dot that some decoders accept.
3. **Resolves** `.` and `..` on the decoded text. A `..` with nothing left to remove is `PROVEN_ERR_PERMISSION`: the path is well formed and asks for what it may not have.

Wrong - checking for `..` before decoding:

```
if (contains(raw_path, "..")) return 400;         /* wrong: "%2e%2e" sails through */
proven_url_percent_decode(raw_path, buf, &n);
open_under_root(buf, n);
```

Wrong - decoding twice "to be safe":

```
proven_url_percent_decode(raw, a, &n);
proven_url_percent_decode(a_view, b, &m);          /* wrong: "%252e%252e" has just become ".." */
```

Correct:

```
proven_err_t err = proven_url_path_resolve(raw_path, clean_mem, &n);
if (err == PROVEN_ERR_PERMISSION)     return 403;   /* or 400 */
if (err != PROVEN_OK)                 return 400;
open_under_root(clean, n);
```

What it does not do. It stops traversal by path *syntax*. It does not know your filesystem: a symbolic link inside the root can still point outside it, and Windows gives some file names a meaning of their own (`CON`, a trailing dot, `name:stream`). Check those where the file is opened. The example above shows the resolver; the file-opening half belongs to Chapter 5.

4. Parsing a head

A *head* is everything before the body: the request line or status line, the header fields, and the empty line that ends them.

```
typedef struct { proven_u8str_view_t name, value; } proven_http_header_t;

typedef struct {
    proven_http_method_t method;       /* GET, POST, ...; OTHER for a token the enum has no name for */
    proven_u8str_view_t method_text;
    proven_u8str_view_t target;        /* "/a?b", "http://h/a", "*", or "host:port" */
    proven_u8 version_minor;           /* 1 or 0 */
    proven_http_header_t *headers;     /* the array you supplied */
    proven_size_t header_count;
} proven_http_request_t;

typedef struct {
    proven_u16 status;
    proven_u8str_view_t reason;        /* may be empty; means nothing to a program */
    proven_u8 version_minor;
    proven_http_header_t *headers;
    proven_size_t header_count;
} proven_http_response_t;
```

Every view points into the buffer you parsed. Nothing is copied, so the buffer must outlive the parsed head and must not move - and the array for the fields is yours, too.

Reference

API	Intent	Return
`proven_http_parse_request(data, headers, cap, max_head, &req, &head_size)`	Parse a request head from the front of `data`. `head_size` is where the body starts. `max_head` 0 means `PROVEN_HTTP_DEFAULT_MAX_HEAD` (16 KiB).	`proven_err_t`: `NEED_MORE` (read more); `OUT_OF_BOUNDS` (head over the limit, or more fields than `cap`: 431); `INVALID_FORMAT` (400); `UNSUPPORTED` (not HTTP/1.0 or 1.1: 505).

`proven_http_parse_response(data, headers, cap, max_head, &res, &head_size)`	The same for a response.	as above.
`proven_http_header_find(headers, count, name, &value)`	The first field of that name, without case. `value` may be NULL.	`bool`.
`proven_http_header_count(headers, count, name)`	How many fields have that name.	`proven_size_t`.
`proven_http_header_has_token(headers, count, name, token)`	Whether any such field lists `token` in a comma-separated value: `Connection: keep-alive, Upgrade` lists `upgrade`.	`bool`.
`proven_http_method_from_text(text)`, `proven_http_method_text(method)`	Between a method token and the enum. Case-sensitive: `get` is not `GET`.	enum; view (empty for `OTHER`).
`proven_http_reason_phrase(status)`	The standard phrase: `Not Found`.	view; `Unknown` for a code with none.
`proven_http_request_keep_alive(&req)`, `proven_http_response_keep_alive(&res)`	Whether the connection may carry another message: HTTP/1.1 unless `Connection: close`; HTTP/1.0 only with `Connection: keep-alive`.	`bool`.

The limit is checked while the head is still incomplete. If `max_head` bytes have arrived and hold no end of head, the answer is `PROVEN_ERR_OUT_OF_BOUNDS` at once - not `NEED_MORE`. A client that sends a header line for ever costs you `max_head` bytes and no more.

Empty lines before a request line are skipped (clients have long sent a stray CRLF after a body) and counted in `head_size`.

Cautions, and what goes wrong

`NEED_MORE` is not a failure.

Wrong:

```
if (proven_http_parse_request(data, h, 32, 0, &req, &n) != PROVEN_OK) return send_400();   /* wrong: a slow client gets a 400 */
```

Correct - three outcomes, three responses: `NEED_MORE` reads more (with a deadline, from Chapter 9); `OUT_OF_BOUNDS`, `INVALID_FORMAT` and `UNSUPPORTED` answer 431, 400 and 505 and close; `PROVEN_OK` goes on.

A field can repeat. `proven_http_header_find` returns the first. For a field where repetition changes the meaning, ask `proven_http_header_count` - which is exactly how this library refuses a second `Content-Length`.

Views die with the buffer. If you read the next request into the same buffer, every view from the previous head now points at something else. Copy what you need to keep.

5. How long is the body

After the head, the single most important question is where the message ends, because the next message starts there. The head answers it:

```
typedef struct {
    proven_http_body_kind_t kind;   /* NONE, LENGTH, CHUNKED, or UNTIL_CLOSE (responses only) */
```

```
    proven_u64 length;              /* for LENGTH */
} proven_http_framing_t;
```

API	Intent	Return
`proven_http_request_framing(&req, &framing)`	Chunked, a length, or no body.	`proven_err_t`: `INVALID_FORMAT` (400) for the ambiguous cases of section 1; `UNSUPPORTED` (501) for a transfer coding other than `chunked`.
`proven_http_response_framing(&res, request_method, &framing)`	The same, knowing what was asked: a response to `HEAD`, a 1xx, 204 or 304, and a successful `CONNECT` have no body. With neither header the body runs until the peer closes.	as above.

A response needs the request's method because the same headers mean different things: a response to `HEAD` carries the `Content-Length` the `GET` would have had, and no body at all.

6. Decoding a body

```
proven_http_body_t body;                        /* caller-owned; nothing to destroy */
proven_http_body_init(&body, framing, max_body_bytes);
proven_http_body_feed(&body, in, &consumed, &payload, &done);
```

API	Intent	Return
`proven_http_body_init(&body, framing, max_body)`	Begin. `max_body` is how much you will accept - there is no default.	`proven_err_t`: `OUT_OF_BOUNDS` (413) when a `Content-Length` already exceeds it.
`proven_http_body_feed(&body, in, &consumed, &payload, &done)`	Consume a prefix of `in`; `payload` is the body bytes in it, as a view INTO `in`.	`proven_err_t`: `INVALID_FORMAT` (bad chunk framing); `OUT_OF_BOUNDS` (over the limit); `INVALID_STATE` after an error.
`proven_http_body_end(&body)`	The peer closed: was the body complete?	`PROVEN_OK`, or `PROVEN_ERR_NEED_MORE` when the message was cut short.
`proven_http_body_received(&body)`	Body bytes delivered so far.	`proven_u64`.

`feed` copies nothing. For a chunked body it steps over the chunk framing and hands back the data between it, one contiguous piece per call - so the loop is: feed what you have; use `payload`; advance by `consumed`; when everything is consumed and `done` is still false, read more. When `done` turns true, bytes of `in` past `consumed` belong to the **next** message.

State your limit. `max_body` is the answer to "how much may a stranger make me hold". A `Content-Length` over it is refused before a byte of body is read; a chunk whose size would cross it is refused when the size is read, not after its data has arrived.

A close is not always an end. A body that runs until close ends, properly, when the peer closes. Any other body that is interrupted by a close is *truncated* - `proven_http_body_end` returns `PROVEN_ERR_NEED_MORE` - and must not be treated as whole. Half a file that looks like a whole file is the worst possible outcome of a download.

After an error the connection is finished. Once framing is wrong, where the next message begins is unknown. Answer if you can, and close.

Compiled and run by the test suite - a request arriving in four awkward pieces:

```
/*
 * A server's view of one request: bytes arrive in pieces, the head is parsed when it is all
 * there, the framing says how long the body is, and the body is decoded as it comes.
 *
 * No socket appears here. The codec takes bytes you have; where they came from is your affair,
 * which is why this same code runs under a blocking read, an event loop, or a test like this.
 */

/* The request, as the network might deliver it: split in awkward places. */
static const char *const pieces[] = {
    "POST /upload/notes.txt?overwrite=1 HTT",
    "P/1.1\r\nHost: example.com\r\nTransfer-Encoding: chun",
    "ked\r\nConnection: keep-alive, TE\r\nX-Tag: a\r\nX-Tag: b\r\n\r\n5\r\nhel",
    "lo\r\n7;note=x\r\n, world\r\n0\r\n\r\nGET /next HTTP/1.1\r\n\r\n",
};

int main(void) {
    proven_byte_t buf[512];                 /* the read buffer: the head must fit in it */
    proven_size_t have = 0;
    proven_size_t next_piece = 0;

    proven_http_header_t fields[16];
    proven_http_request_t req;
    proven_size_t head_size = 0;

    /* Read until the head is complete. NEED_MORE is not an error: it means exactly that. */
    for (;;) {
        proven_err_t err = proven_http_parse_request((proven_mem_view_t){ buf, have }, fields, 16, sizeof buf,
&req, &head_size);
        if (err == PROVEN_OK) break;
        EXAMPLE_REQUIRE(err == PROVEN_ERR_NEED_MORE, "an incomplete head asks for more; anything else is a
400, 431 or 505");
        if (err != PROVEN_ERR_NEED_MORE) return EXAMPLE_OK();
        /* "read": append the next piece. */
        proven_size_t len = proven_cstr_len(pieces[next_piece]);
        for (proven_size_t i = 0; i < len; ++i) buf[have + i] = (proven_byte_t)pieces[next_piece][i];
        have += len;
        next_piece++;
    }

    /* The head, as views into `buf`. */
    EXAMPLE_REQUIRE(req.method == PROVEN_HTTP_POST && req.version_minor == 1, "POST, HTTP/1.1");
    EXAMPLE_REQUIRE(proven_http_method_from_text(req.method_text) == PROVEN_HTTP_POST &&
                    proven_u8str_view_eq(proven_http_method_text(req.method), PROVEN_LIT("POST")), "the
method, both ways");

    proven_u8str_view_t host;
    EXAMPLE_REQUIRE(proven_http_header_find(fields, req.header_count, PROVEN_LIT("host"), &host) &&
                    proven_u8str_view_eq(host, PROVEN_LIT("example.com")), "header names compare without
case");
    EXAMPLE_REQUIRE(proven_http_header_count(fields, req.header_count, PROVEN_LIT("X-Tag")) == 2, "a field may
repeat");
    EXAMPLE_REQUIRE(proven_http_header_has_token(fields, req.header_count, PROVEN_LIT("Connection"),
PROVEN_LIT("te")),
                    "a token inside a comma-separated list");
```

```
    EXAMPLE_REQUIRE(proven_http_request_keep_alive(&req), "the connection stays open for another request");

    /* Where the request points. Split, then resolve - before the path is used for anything. */
    proven_u8str_view_t path, query;
    bool has_query = false;
    proven_byte_t clean[128];
    proven_size_t clean_len = 0;
    EXAMPLE_REQUIRE(proven_url_split_target(req.target, &path, &query, &has_query) == PROVEN_OK &&
                    proven_url_path_resolve(path, (proven_mem_mut_t){ clean, sizeof clean }, &clean_len) ==
PROVEN_OK &&
                    proven_u8str_view_eq((proven_u8str_view_t){ clean, clean_len }, PROVEN_LIT("/upload/
notes.txt")),
                    "the target's path, resolved");

    /* How long is the body? The head decides, and an ambiguous head is an error, not a guess. */
    proven_http_framing_t framing;
    EXAMPLE_REQUIRE(proven_http_request_framing(&req, &framing) == PROVEN_OK && framing.kind ==
PROVEN_HTTP_BODY_CHUNKED,
                    "this body is chunked");

    /* Decode it. The limit is the caller's statement of how much a peer may send. */
    proven_http_body_t body;
    EXAMPLE_REQUIRE(proven_http_body_init(&body, framing, 1024) == PROVEN_OK, "at most 1 KiB of body");

    proven_byte_t content[64];
    proven_size_t content_len = 0;
    proven_size_t pos = head_size;          /* the body starts where the head ended */
    bool done = false;
    while (!done) {
        if (pos == have) {
            /* Everything read so far is used up: "read" more. A real server would stop here
             * with proven_http_body_end if the peer had closed instead. */
            EXAMPLE_REQUIRE(next_piece < sizeof pieces / sizeof pieces[0], "more input exists");
            proven_size_t len = proven_cstr_len(pieces[next_piece]);
            for (proven_size_t i = 0; i < len; ++i) buf[have + i] = (proven_byte_t)pieces[next_piece][i];
            have += len;
            next_piece++;
        }
        proven_size_t used = 0;
        proven_mem_view_t payload;
        proven_err_t err = proven_http_body_feed(&body, (proven_mem_view_t){ buf + pos, have - pos }, &used,
&payload, &done);
        EXAMPLE_REQUIRE(err == PROVEN_OK, "the chunk framing is well formed");
        if (err != PROVEN_OK) break;
        /* `payload` is a view into `buf`: body bytes with the chunk framing stepped over. */
        for (proven_size_t i = 0; i < payload.size; ++i) content[content_len + i] = payload.ptr[i];
        content_len += payload.size;
        pos += used;
    }
    EXAMPLE_REQUIRE(proven_u8str_view_eq((proven_u8str_view_t){ content, content_len }, PROVEN_LIT("hello,
world")),
                    "two chunks, joined");
    EXAMPLE_REQUIRE(proven_http_body_received(&body) == 12 && proven_http_body_end(&body) == PROVEN_OK,
"twelve bytes, complete");

    /* What is left in the buffer is the next request, untouched. */
    proven_http_request_t next;
    EXAMPLE_REQUIRE(proven_http_parse_request((proven_mem_view_t){ buf + pos, have - pos }, fields, 16, 0,
&next, &head_size) == PROVEN_OK &&
                    proven_u8str_view_eq(next.target, PROVEN_LIT("/next")), "the decoder stopped exactly where
```

```
the next request begins");

    return EXAMPLE_OK();
}
```

7. Writing a message

The writers append to memory you supply. Each takes the buffer and a length, writes at that length, and advances it; on failure the length does not move and nothing before it is touched.

API	Appends
`proven_http_write_request_line(out, &len, method, target)`	`GET /path HTTP/1.1` CRLF
`proven_http_write_status_line(out, &len, status, reason)`	`HTTP/1.1 404 Not Found` CRLF; an empty `reason` sends the standard phrase
`proven_http_write_header(out, &len, name, value)`	`Name: value` CRLF
`proven_http_write_header_u64(out, &len, name, number)`	`Content-Length: 1234` CRLF
`proven_http_write_head_end(out, &len)`	the empty line that ends the head
`proven_http_write_chunk_begin(out, &len, size)`	the line that starts a chunk of `size` bytes (not 0)
`proven_http_write_chunk_end(out, &len)`	the CRLF after a chunk's data
`proven_http_write_last_chunk(out, &len)`	the zero chunk and empty trailers that end a chunked body

All return `proven_err_t`: `PROVEN_ERR_OUT_OF_BOUNDS` when it does not fit, and `PROVEN_ERR_INVALID_ARG` when what you asked it to write would not be the thing it is named for.

That second error is the important one. A header value almost always comes from somewhere else - a file name, a redirect target, something the client sent. If it contains a CR and an LF, writing it verbatim ends the field and starts a new one that the *sender* chose:

```
Location: /home
Set-Cookie: session=attacker        <- this line came from inside the "value"
```

That is *response splitting*, the writing twin of request smuggling. `proven_http_write_header` refuses any value containing CR, LF, NUL or another control character, and any name that is not a token; `proven_http_write_request_line` refuses a target with a space or a line break. The check is not optional and has no "raw" variant.

Wrong - one check at the end:

```
proven_http_write_status_line(out, &len, 200, PROVEN_LIT(""));
proven_http_write_header(out, &len, name, user_value);      /* refused: len did not move */
err = proven_http_write_head_end(out, &len);                /* OK - and the header is silently missing */
```

Correct - check each call, or chain them so the first failure stops the rest, as the example does. The writers are `[[nodiscard]]`, so the compiler objects to the first form.

A body of known length is `Content-Length` and then the bytes, which you write yourself. A body of unknown length is `Transfer-Encoding: chunked` and then, for each piece, `chunk_begin`, the data, `chunk_end` - and `last_chunk` at the end.

Compiled and run by the test suite:

```
/*
 * Writing a response and reading one back.
 *
 * Each writer appends at `len` and moves it. A value that could break the message - a line
 * break in a header - is refused instead of written, so a string taken from a request cannot
 * be used to add a header of the sender's choosing.
```

```
 */

int main(void) {
    proven_byte_t msg[512];
    proven_mem_mut_t out = { msg, sizeof msg };
    proven_size_t len = 0;

    /* The date header: the wall clock, in the one form HTTP writes. */
    proven_byte_t date[PROVEN_HTTP_DATE_SIZE];
    proven_time_t now = proven_time_now();
    EXAMPLE_REQUIRE(proven_http_date_format(now, date) == PROVEN_OK, "now, as an HTTP date");

    /* A response whose length is not known in advance goes out in chunks. */
    proven_err_t err = proven_http_write_status_line(out, &len, 200, PROVEN_LIT(""));     /* "" : the standard
phrase */
    if (err == PROVEN_OK) err = proven_http_write_header(out, &len, PROVEN_LIT("Date"), (proven_u8str_view_t)
{ date, sizeof date });
    if (err == PROVEN_OK) err = proven_http_write_header(out, &len, PROVEN_LIT("Content-Type"),
PROVEN_LIT("text/plain; charset=utf-8"));
    if (err == PROVEN_OK) err = proven_http_write_header(out, &len, PROVEN_LIT("Transfer-Encoding"),
PROVEN_LIT("chunked"));
    if (err == PROVEN_OK) err = proven_http_write_header_u64(out, &len, PROVEN_LIT("X-Request-Id"), 4711);
    if (err == PROVEN_OK) err = proven_http_write_head_end(out, &len);
    EXAMPLE_REQUIRE(err == PROVEN_OK, "the head is written");
    proven_size_t head_len = len;

    static const char *const parts[] = { "written in ", "two chunks" };
    for (proven_size_t i = 0; i < 2 && err == PROVEN_OK; ++i) {
        proven_size_t n = proven_cstr_len(parts[i]);
        err = proven_http_write_chunk_begin(out, &len, n);
        if (err != PROVEN_OK || n > sizeof msg - len) { err = PROVEN_ERR_OUT_OF_BOUNDS; break; }
        for (proven_size_t k = 0; k < n; ++k) msg[len + k] = (proven_byte_t)parts[i][k];   /* the chunk's data
*/
        len += n;
        err = proven_http_write_chunk_end(out, &len);
    }
    if (err == PROVEN_OK) err = proven_http_write_last_chunk(out, &len);
    EXAMPLE_REQUIRE(err == PROVEN_OK, "the body is written");

    /* A header value with a line break in it is not written at all. */
    proven_size_t before = len;
    EXAMPLE_REQUIRE(proven_http_write_header(out, &len, PROVEN_LIT("Location"), PROVEN_LIT("/home\r\nSet-
Cookie: session=stolen")) == PROVEN_ERR_INVALID_ARG &&
                    len == before, "response splitting is refused, and nothing was appended");

    /* --- the client's side: the same bytes, read back ------------------------------ */

    proven_http_header_t fields[16];
    proven_http_response_t res;
    proven_size_t head_size = 0;
    EXAMPLE_REQUIRE(proven_http_parse_response((proven_mem_view_t){ msg, len }, fields, 16, 0, &res,
&head_size) == PROVEN_OK &&
                    head_size == head_len, "the head parses, and ends where it was written to end");
    EXAMPLE_REQUIRE(res.status == 200 && proven_u8str_view_eq(res.reason, proven_http_reason_phrase(200)),
"200 OK");
    EXAMPLE_REQUIRE(proven_http_response_keep_alive(&res), "an HTTP/1.1 response leaves the connection open");

    /* The body's framing depends on the request it answers: a response to HEAD has none. */
    proven_http_framing_t framing;
    EXAMPLE_REQUIRE(proven_http_response_framing(&res, PROVEN_HTTP_HEAD, &framing) == PROVEN_OK &&
```

```
framing.kind == PROVEN_HTTP_BODY_NONE,
                    "had this answered HEAD, no body would follow");
    EXAMPLE_REQUIRE(proven_http_response_framing(&res, PROVEN_HTTP_GET, &framing) == PROVEN_OK && framing.kind
== PROVEN_HTTP_BODY_CHUNKED,
                    "it answers GET, and the body is chunked");

    proven_http_body_t body;
    proven_byte_t text[64];
    proven_size_t text_len = 0;
    proven_size_t pos = head_size;
    bool done = false;
    EXAMPLE_REQUIRE(proven_http_body_init(&body, framing, sizeof text) == PROVEN_OK, "accept at most what the
buffer holds");
    while (!done && pos < len) {
        proven_size_t used = 0;
        proven_mem_view_t payload;
        if (proven_http_body_feed(&body, (proven_mem_view_t){ msg + pos, len - pos }, &used, &payload,
&done) != PROVEN_OK) break;
        for (proven_size_t i = 0; i < payload.size; ++i) text[text_len + i] = payload.ptr[i];
        text_len += payload.size;
        pos += used;
    }
    EXAMPLE_REQUIRE(done && pos == len && proven_u8str_view_eq((proven_u8str_view_t){ text, text_len },
PROVEN_LIT("written in two chunks")),
                    "the body, decoded");

    /* The date comes back as the time it was made from, to the second. */
    proven_u8str_view_t date_text;
    proven_time_t parsed = 0;
    EXAMPLE_REQUIRE(proven_http_header_find(fields, res.header_count, PROVEN_LIT("Date"), &date_text) &&
                    proven_http_date_parse(date_text, now, &parsed) == PROVEN_OK && parsed == now - now %
1000000000,
                    "the Date header parses back to the same second");

    /* A request is written the same way. */
    len = 0;
    err = proven_http_write_request_line(out, &len, PROVEN_LIT("GET"), PROVEN_LIT("/search?q=a%20b"));
    if (err == PROVEN_OK) err = proven_http_write_header(out, &len, PROVEN_LIT("Host"),
PROVEN_LIT("example.com"));
    if (err == PROVEN_OK) err = proven_http_write_head_end(out, &len);
    EXAMPLE_REQUIRE(err == PROVEN_OK && proven_u8str_view_eq((proven_u8str_view_t){ msg, len },
                    PROVEN_LIT("GET /search?q=a%20b HTTP/1.1\r\nHost: example.com\r\n\r\n")), "a request head,
byte for byte");

    return EXAMPLE_OK();
}
```

8. Dates

HTTP writes one date format and must read three.

API	Intent	Return
`proven_http_date_format(wall_ns, out[29])`	`Sun, 06 Nov 1994 08:49:37 GMT`. Always 29 bytes (`PROVEN_HTTP_DATE_SIZE`), always GMT, no NUL.	`proven_err_t`: `INVALID_ARG` before 1970.
`proven_http_date_parse(text, now_ns, &wall_ns)`	Read that form, the obsolete `Sunday, 06-Nov-94 08:49:37 GMT`,	`proven_err_t`: `INVALID_FORMAT`; `OVERFLOW`.

	and asctime's `Sun Nov  6 08:49:37 1994`.	

Pass the **wall clock** (`proven_time_now`), never the monotonic one, which has no date (Chapter 5 section 4).

The parser checks that the date exists - 30 February is refused, and so is a weekday that does not match its date. `now_ns` is used for one rule only: a two-digit year, which the obsolete form has, is read as the most recent year with those digits that is not more than 50 years in the future.

`PROVEN_ERR_OVERFLOW` is a valid date that `proven_time_t` cannot hold: its nanoseconds end in April 2262. Servers send such dates on purpose - `Expires` in the year 9999 means "never" - so treat that answer as "far in the future", not as a malformed header.

9. References and forms

A `Location` header, a link in a page, a form's `action`: each is a *reference*, and where it leads depends on the URL it was found at. `proven_url_resolve` is that step, as RFC 3986 section 5.2 lays it out.

Reference, found at `http://example.com/docs/guide/intro.html?v=2`	**Resolves to**
`chapter2.html`	`http://example.com/docs/guide/chapter2.html`
`../img/logo.png`	`http://example.com/docs/img/logo.png`
`/login?next=%2F`	`http://example.com/login?next=%2F`
`?v=3`	`http://example.com/docs/guide/intro.html?v=3`
`//cdn.example.net/x`	`http://cdn.example.net/x`
`https://other.example/`	`https://other.example/`

API	Intent	Return
`proven_url_resolve(base, reference, out, &written)`	Resolve `reference` against the absolute URL `base`. Dot segments are removed; a reference that is itself absolute is returned as it is.	`proven_err_t`: `INVALID_FORMAT` when `base` is not an absolute URL or the result would have no host; `OUT_OF_BOUNDS`.
`proven_url_form_append(out, &len, name, value)`	Append one `name=value` pair of a form body, encoded, with the `&` before it when it is not the first.	`proven_err_t`: `OUT_OF_BOUNDS`, and then `len` and the bytes before it are unchanged.

A resolved URL is somebody else's choice of destination. The server that sent the `Location` chose it. Resolving it tells you where it leads; whether to go there - to another host, from `https` down to `http`, to an address inside your own network - is a decision the resolver does not make. The client of Chapter 11 makes the first two; the third is yours.

`proven_url_form_append` writes `application/x-www-form-urlencoded`, the format of a query string and of a simple form body: a space becomes `+`, and everything but letters, digits and `-._~` becomes `%XX`. Names and values are bytes, so they are encoded as given.

10. Ranges and multipart bodies

A range request asks for part of a resource - to resume a download, or to read the end of a log. The request names bytes; the answer is a `206` that says which bytes it carries.

API	Intent	Return

`proven_http_write_range(out, &len, first, last)`	Append `Range: bytes=first-last`; pass `PROVEN_HTTP_RANGE_TO_END` as `last` for "to the end".	`proven_err_t`: `INVALID_ARG` when `last < first`.
`proven_http_range_parse(value, size, &first, &last)`	A server's side: read a `Range` value against a resource of `size` bytes. Takes `bytes=a-b`, `bytes=a-` and `bytes=-n`, clamped to the resource.	`proven_err_t`: `OUT_OF_BOUNDS` - answer 416; `UNSUPPORTED` for several ranges or another unit - send the whole resource; `INVALID_FORMAT` - ignore the header.
`proven_http_write_content_range(out, &len, first, last, total)`	Append `Content-Range: bytes first-last/total` for the 206.	`proven_err_t`: `INVALID_ARG` unless `first <= last < total`.
`proven_http_content_range_parse(value, &first, &last, &total, &has_total)`	A client's side: read it back. `has_total` is false for `/*`.	`proven_err_t`: `INVALID_FORMAT`.

The three errors of `proven_http_range_parse` are three different answers, and RFC 9110 names each: a range that cannot be satisfied is a `416` with `Content-Range: bytes */size`; a range this server does not implement, or a header it cannot read, is *ignored* - the response is the ordinary `200` with the whole resource. Answering `400` to a `Range` header is wrong.

A client that resumes must check what came back. A server may ignore `Range`. If you asked for bytes 5000 onward and the answer is a `200`, the body starts at byte 0: appending it to the 5000 bytes you have produces a file that is silently corrupt. Append only to a `206` whose `Content-Range` starts where you asked.

A `multipart/form-data` body is how a form uploads a file: each field is a part, and the parts are separated by a boundary line that must not occur inside any of them.

API	Intent	Return
`proven_http_multipart_boundary(random, out)`	Make a boundary - `PROVEN_HTTP_BOUNDARY_SIZE` (40) characters - from 16 random bytes you supply.	none.
`proven_http_multipart_write_content_type(out, &len, boundary)`	Append `Content-Type: multipart/form-data; boundary=...`.	`proven_err_t`: `INVALID_ARG` for a boundary that is empty, longer than 70, or needs quoting.
`proven_http_multipart_write_part(out, &len, boundary, name, filename, content_type)`	Append the opening of a part. `filename` empty: an ordinary field. `content_type` empty: none written. The part's bytes follow, written by you.	`proven_err_t`: `INVALID_ARG` for an empty name or a control character in `content_type`.
`proven_http_multipart_write_part_end(out, &len)`	Append the line break that ends a part's bytes.	`proven_err_t`.
`proven_http_multipart_write_end(out, &len, boundary)`	Append the closing boundary.	`proven_err_t`.

The boundary must be unpredictable. If the bytes of an uploaded file can contain the boundary line, that file can end its own part and begin another - a field the form never had. Sixteen bytes from `proven_random_bytes` make that as unlikely as guessing a key. A boundary made from a counter or a clock does not.

A double quote or a line break in a field name or a file name is written percent-encoded, as browsers do, so a file called `a".png` cannot close the quoted string it is written in.

These functions build the framing only. The parts' bytes are written by you between `_write_part` and `_write_part_end`, so a large file can be sent in pieces without ever being held whole - and nothing here parses a multipart body that someone sent to you.

11. Authentication

HTTP's own authentication is a challenge and an answer: a `401` response carries `WWW-Authenticate`, and the request is sent again with `Authorization`.

API	Intent	Return
`proven_http_basic_auth(user, password, out, &written)`	Write the value `Basic <base64(user:password)>`.	`proven_err_t`: `INVALID_ARG` for a colon in `user` or a control character in either; `OUT_OF_BOUNDS`.
`proven_http_auth_offers(header_value, scheme)`	Whether a `WWW-Authenticate` value offers `scheme` (compared without case).	`bool`.
`proven_http_digest_challenge_parse(header_value, &challenge)`	Find the Digest challenge in a `WWW-Authenticate` value; if several, the strongest algorithm this library has.	`proven_err_t`: `NOT_FOUND` - no Digest challenge; `UNSUPPORTED` - only algorithms it lacks, or no `qop=auth`; `INVALID_FORMAT`.
`proven_http_digest_auth(&challenge, user, password, method, uri, nonce_count, cnonce, out, &written)`	Write the `Authorization` value that answers it. `uri` is the request target exactly as sent; `cnonce` is random text you supply; `nonce_count` starts at 1 for each new nonce.	`proven_err_t`: `INVALID_ARG`; `OUT_OF_BOUNDS`.

Basic is the password, encoded so it survives a header - not hidden. Anyone who can read the request can read the password. Over a connection that is not encrypted, that is everyone on the path. Use it over TLS or not at all.

Digest does not send the password, and that is all it does. The answer is a hash over the password, the server's nonce and the request line, so an eavesdropper learns nothing to log in with later. The request and the response still cross the network readable, and anyone who captures one exchange can guess passwords against it at their leisure. It is better than Basic over plain HTTP; it is not a substitute for TLS.

`MD5`, `MD5-sess`, `SHA-256` and `SHA-256-sess` are implemented, with `qop=auth`. MD5 is there because deployed servers still ask for it - see Chapter 4 on the legacy digests - and a challenge that offers both is answered with SHA-256. `qop=auth-int`, `SHA-512-256` and the `userhash` option are not.

Do not answer a challenge you did not expect. A client that sends credentials to any server that asks will send them to the wrong one - after a redirect, for instance. Decide which host the credentials are for, and answer only that host.

Compiled and run by the test suite:

```
#include <string.h>

/*
 * The small pieces a client and a server need around the message codec: resolving a
 * redirect, building a form, asking for part of a resource, uploading a file, and answering
```

```
 * an authentication challenge. All of it is text in, text out - no socket, no allocation.
 */

static bool view_is(proven_u8str_view_t v, const char *text) {
    return proven_u8str_view_eq(v, proven_u8str_view_from_cstr(text));
}

int main(void) {
    proven_byte_t buf[512];
    proven_mem_mut_t out = (proven_mem_mut_t){ buf, sizeof buf };
    proven_size_t len = 0;

    /* A Location header is a reference; where it leads depends on where you are. */
    proven_u8str_view_t base = PROVEN_LIT("http://example.com/docs/guide/intro.html?v=2");
    EXAMPLE_REQUIRE(proven_url_resolve(base, PROVEN_LIT("../img/logo.png"), out, &len) == PROVEN_OK &&
                    view_is((proven_u8str_view_t){ buf, len }, "http://example.com/docs/img/logo.png"), "a
relative reference");
    EXAMPLE_REQUIRE(proven_url_resolve(base, PROVEN_LIT("/login?next=%2F"), out, &len) == PROVEN_OK &&
                    view_is((proven_u8str_view_t){ buf, len }, "http://example.com/login?next=%2F"), "an
absolute path");
    EXAMPLE_REQUIRE(proven_url_resolve(base, PROVEN_LIT("//cdn.example.net/x"), out, &len) == PROVEN_OK &&
                    view_is((proven_u8str_view_t){ buf, len }, "http://cdn.example.net/x"), "another host,
same scheme");

    /* A form body, one pair at a time; the separators and the encoding are done for you. */
    len = 0;
    EXAMPLE_REQUIRE(proven_url_form_append(out, &len, proven_mem_view_from_u8(PROVEN_LIT("q")),
proven_mem_view_from_u8(PROVEN_LIT("tea & cake"))) == PROVEN_OK &&
                    proven_url_form_append(out, &len, proven_mem_view_from_u8(PROVEN_LIT("page")),
proven_mem_view_from_u8(PROVEN_LIT("2"))) == PROVEN_OK,
                    "two fields");
    EXAMPLE_REQUIRE(view_is((proven_u8str_view_t){ buf, len }, "q=tea+%26+cake&page=2"), "application/x-www-
form-urlencoded");

    /* Ranges. A client asks; a server reads the request against the size of what it has. */
    len = 0;
    EXAMPLE_REQUIRE(proven_http_write_range(out, &len, 100, PROVEN_HTTP_RANGE_TO_END) == PROVEN_OK &&
                    view_is((proven_u8str_view_t){ buf, len }, "Range: bytes=100-\r\n"), "from byte 100 to the
end");
    proven_u64 first = 0, last = 0, total = 0;
    EXAMPLE_REQUIRE(proven_http_range_parse(PROVEN_LIT("bytes=100-"), 1000, &first, &last) == PROVEN_OK &&
first == 100 && last == 999, "of 1000 bytes: 100 through 999");
    EXAMPLE_REQUIRE(proven_http_range_parse(PROVEN_LIT("bytes=-50"), 1000, &first, &last) == PROVEN_OK &&
first == 950 && last == 999, "the last fifty");
    EXAMPLE_REQUIRE(proven_http_range_parse(PROVEN_LIT("bytes=2000-"), 1000, &first, &last) ==
PROVEN_ERR_OUT_OF_BOUNDS, "past the end: answer 416");
    EXAMPLE_REQUIRE(proven_http_range_parse(PROVEN_LIT("bytes=0-9,20-29"), 1000, &first, &last) ==
PROVEN_ERR_UNSUPPORTED, "several ranges: send the whole thing");
    len = 0;
    EXAMPLE_REQUIRE(proven_http_write_content_range(out, &len, 950, 999, 1000) == PROVEN_OK &&
                    view_is((proven_u8str_view_t){ buf, len }, "Content-Range: bytes 950-999/1000\r\n"), "the
206 says which part it is");
    bool has_total = false;
    EXAMPLE_REQUIRE(proven_http_content_range_parse(PROVEN_LIT("bytes 950-999/1000"), &first, &last, &total,
&has_total) == PROVEN_OK &&
                    first == 950 && last == 999 && has_total && total == 1000, "and the client reads it
back");

    /* multipart/form-data: a text field and a file. The boundary comes from random bytes
     * you supply, so it cannot be predicted and placed inside a part. */
```

```
    proven_byte_t random[16];
    EXAMPLE_REQUIRE(proven_random_bytes(random, sizeof random), "sixteen random bytes");
    proven_byte_t boundary_bytes[PROVEN_HTTP_BOUNDARY_SIZE];
    proven_http_multipart_boundary(random, boundary_bytes);
    proven_u8str_view_t boundary = (proven_u8str_view_t){ boundary_bytes, sizeof boundary_bytes };

    proven_byte_t head[128];
    proven_size_t head_len = 0;
    EXAMPLE_REQUIRE(proven_http_multipart_write_content_type((proven_mem_mut_t){ head, sizeof head },
&head_len, boundary) == PROVEN_OK, "the header that announces it");

    len = 0;
    proven_err_t err = proven_http_multipart_write_part(out, &len, boundary, PROVEN_LIT("title"),
PROVEN_LIT(""), PROVEN_LIT(""));
    if (err == PROVEN_OK) { memcpy(buf + len, "Holiday", 7); len += 7; }      /* the part's bytes are yours to
write */
    if (err == PROVEN_OK) err = proven_http_multipart_write_part_end(out, &len);
    if (err == PROVEN_OK) err = proven_http_multipart_write_part(out, &len, boundary, PROVEN_LIT("photo"),
PROVEN_LIT("a \"b\".png"), PROVEN_LIT("image/png"));
    if (err == PROVEN_OK) { memcpy(buf + len, "\x89PNG", 4); len += 4; }
    if (err == PROVEN_OK) err = proven_http_multipart_write_part_end(out, &len);
    if (err == PROVEN_OK) err = proven_http_multipart_write_end(out, &len, boundary);
    EXAMPLE_REQUIRE(err == PROVEN_OK, "a body of two parts");
    buf[len] = '\0';
    EXAMPLE_REQUIRE(strstr((const char *)buf, "filename=\"a %22b%22.png\"") != NULL, "a quote in a file name
cannot end the quoted string");

    /* Authentication. Basic is the two strings, encoded - not encrypted. */
    proven_size_t n = 0;
    EXAMPLE_REQUIRE(proven_http_basic_auth(PROVEN_LIT("Aladdin"), PROVEN_LIT("open sesame"), out, &n) ==
PROVEN_OK &&
                    view_is((proven_u8str_view_t){ buf, n }, "Basic QWxhZGRpbjpvcGVuIHNlc2FtZQ=="), "the value
of an Authorization header");

    /* Digest answers a challenge, so the password itself is never sent. */
    proven_u8str_view_t www = PROVEN_LIT("Digest realm=\"http-auth@example.org\", qop=\"auth, auth-int\",
algorithm=SHA-256, "
                                         "nonce=\"7ypf/xlj9XXwfDPEoM4URrv/xwf94BcCAzFZH4GiTo0v\",
opaque=\"FQhe/qaU925kfnzjCev0ciny7QMkPqMAFRtzCUYo5tdS\"");
    EXAMPLE_REQUIRE(proven_http_auth_offers(www, PROVEN_LIT("digest")) && !proven_http_auth_offers(www,
PROVEN_LIT("Basic")), "what the server offers");
    proven_http_digest_challenge_t challenge;
    EXAMPLE_REQUIRE(proven_http_digest_challenge_parse(www, &challenge) == PROVEN_OK && challenge.algorithm ==
PROVEN_HTTP_DIGEST_SHA256 && challenge.qop_auth, "the challenge");
    EXAMPLE_REQUIRE(proven_http_digest_auth(&challenge, PROVEN_LIT("Mufasa"), PROVEN_LIT("Circle of Life"),
PROVEN_LIT("GET"), PROVEN_LIT("/dir/index.html"),
                                            1, PROVEN_LIT("f2/wE4q74E6zIJEtWaHKaf5wv/H5QzzpXusqGemxURZJ"),
out, &n) == PROVEN_OK, "the answer");
    buf[n] = '\0';
    /* The response RFC 7616 section 3.9.1 works out for exactly these inputs. */
    EXAMPLE_REQUIRE(strstr((const char *)buf,
"response=\"753927fa0e85d155564e2e272a28d1802ca10daf4496794697cf8db5856cb6c1\"") != NULL, "is the one in the
RFC");

    return EXAMPLE_OK();
}
```

12. Cookies

A cookie is a name and a value that a server asks to be sent back. `proven_http_cookie_jar_t` keeps them between requests: it stores what `Set-Cookie` headers set, and writes the `Cookie` header for a request.

API	Intent	Return
`proven_http_cookie_jar_init(&jar, alloc, max_cookies)`	Make a jar that holds up to `max_cookies`.	`proven_err_t`: `INVALID_ARG` for a limit of 0; `NOMEM`.
`proven_http_cookie_jar_store(&jar, host, path, secure, set_cookie, now)`	Act on one `Set-Cookie` value from a response to `host` and `path`. `now` is `proven_time_now()`.	`proven_err_t`: `INVALID_FORMAT` - not a cookie, ignore it; `PERMISSION` - not this host's to set; `OUT_OF_BOUNDS` - larger than `PROVEN_HTTP_COOKIE_MAX_SIZE`; `NOMEM`. The jar is unchanged in each case.
`proven_http_cookie_jar_header(&jar, host, path, secure, now, out, &written)`	Write the `Cookie` value for a request: `a=1; b=2`. `written` is 0 when there is nothing to send.	`proven_err_t`: `OUT_OF_BOUNDS`.
`proven_http_cookie_jar_count(&jar)`	How many cookies it holds.	`proven_size_t`.
`proven_http_cookie_jar_clear(&jar)`	Forget them all.	none.
`proven_http_cookie_jar_destroy(&jar)`	Free it.	none.

This jar is host-only, on purpose. A browser lets `www.example.com` set a cookie for all of `example.com`, and to do that safely it needs the public suffix list - the table that says `co.uk` is not a site but `example.co.uk` is. This library has no such table, and a jar that honours `Domain` without one will let `evil.co.uk` set a cookie for every site under `co.uk`. So a `Domain` attribute is checked - one that does not cover the host that sent it is `PROVEN_ERR_PERMISSION` - and then set aside: every cookie goes back to the exact host that set it and to no other. A site that relies on a cookie shared between its subdomains will not work through this jar; nothing that it should not receive will reach it either.

What the jar does implement of RFC 6265: `Path` and its matching, `Max-Age` and `Expires` (`Max-Age` wins), `Secure` - never sent, and refused when set, over a connection that is not encrypted - replacement of a cookie with the same name and path, deletion by expiry, and eviction of the oldest when the jar is full. `HttpOnly` and `SameSite` describe what a browser's scripts and pages may do; a program has neither, and they are accepted and ignored.

The jar is not thread-safe, and it lives in memory only: when it is destroyed the cookies are gone.

13. Server-sent events

An event stream is a response that does not end: `Content-Type: text/event-stream`, and then lines, for as long as the connection lasts.

```
id: 41
event: price
data: {"sym":"X","p":12.5}

data: a second event, of the default type
```

A blank line ends an event. `proven_sse_t` takes the stream in whatever pieces it arrives and hands back events.

API	Intent	Return
`proven_sse_init(&sse, work)`	Begin. `work` is memory for the line and the event being	`proven_err_t`: `INVALID_ARG`.

	assembled - half each - so it bounds the largest event accepted. At least 64 bytes.	
`proven_sse_feed(&sse, in, &consumed, &event, &have_event)`	Consume bytes of `in` until an event is complete or `in` is used up. With `have_event` true, use `event` and call again with the rest.	`proven_err_t`: `OUT_OF_BOUNDS` when a line or an event outgrows `work`; after that, `INVALID_STATE`.
`proven_sse_last_id(&sse)`	The last `id` seen, to send as `Last-Event-ID` when reconnecting.	`proven_u8str_view_t`; empty when there was none.

```
typedef struct {
    proven_u8str_view_t event;   /* the type; empty means the default, "message" */
    proven_u8str_view_t data;    /* the data lines joined with LF */
    proven_u8str_view_t id;      /* the last id so far, from this event or an earlier one */
    proven_u32 retry_ms;         /* valid when has_retry */
    bool has_retry;
} proven_sse_event_t;
```

An event's views are good until the next feed. They point into `work`. Copy what you keep.

A stream that ends mid-event has not delivered that event. The standard says an event is dispatched by its blank line; one cut off before it is dropped, not delivered half. When the connection ends, reconnect with `Last-Event-ID` and the server can send it again.

The parser follows the WHATWG rules - the three line endings, comment lines, a leading byte-order mark, a field with no colon, the single space after a colon - and does not reconnect, which is the client's job: this is the parser, and the connection is Chapter 11's.

Compiled and run by the test suite:

```
/*
 * Two pieces of state a client keeps between messages: the cookies a server set, and its
 * place in a stream of server-sent events. Neither touches a socket.
 */

static bool view_is(proven_u8str_view_t v, const char *text) {
    return proven_u8str_view_eq(v, proven_u8str_view_from_cstr(text));
}

int main(void) {
    // ---- A cookie jar -------------------------------------------------------
    proven_http_cookie_jar_t jar;
    EXAMPLE_REQUIRE(proven_http_cookie_jar_init(&jar, proven_heap_allocator(), 32) == PROVEN_OK, "a jar for up
to 32 cookies");
    proven_time_t now = proven_time_now();

    /* Each Set-Cookie header of a response is stored with where it came from. */
    proven_u8str_view_t host = PROVEN_LIT("shop.example.com");
    EXAMPLE_REQUIRE(proven_http_cookie_jar_store(&jar, host, PROVEN_LIT("/"), true, PROVEN_LIT("sid=abc123;
Path=/; Secure; HttpOnly"), now) == PROVEN_OK, "a session cookie");
    EXAMPLE_REQUIRE(proven_http_cookie_jar_store(&jar, host, PROVEN_LIT("/cart/view"), true,
PROVEN_LIT("items=3; Max-Age=3600"), now) == PROVEN_OK,
                    "one with no Path: it belongs to /cart");
    /* A cookie for somebody else's domain is refused. This jar goes further than a browser
     * and keeps every cookie for the exact host that set it. */
    EXAMPLE_REQUIRE(proven_http_cookie_jar_store(&jar, host, PROVEN_LIT("/"), true, PROVEN_LIT("track=1;
```

```
Domain=ads.example.net"), now) == PROVEN_ERR_PERMISSION,
                    "not this host's to set");
    EXAMPLE_REQUIRE(proven_http_cookie_jar_count(&jar) == 2, "two cookies");

    /* For a request, the jar writes the value of the Cookie header. */
    proven_byte_t buf[256];
    proven_mem_mut_t out = { buf, sizeof buf };
    proven_size_t n = 0;
    EXAMPLE_REQUIRE(proven_http_cookie_jar_header(&jar, host, PROVEN_LIT("/cart/checkout"), true, now, out,
&n) == PROVEN_OK &&
                    view_is((proven_u8str_view_t){ buf, n }, "items=3; sid=abc123"), "under /cart: both, the
longer path first");
    EXAMPLE_REQUIRE(proven_http_cookie_jar_header(&jar, host, PROVEN_LIT("/account"), true, now, out, &n) ==
PROVEN_OK &&
                    view_is((proven_u8str_view_t){ buf, n }, "sid=abc123"), "elsewhere: only the one for /");
    EXAMPLE_REQUIRE(proven_http_cookie_jar_header(&jar, host, PROVEN_LIT("/account"), false, now, out, &n) ==
PROVEN_OK && n == 0,
                    "over plain HTTP: nothing - sid is Secure. n == 0 means send no header");
    EXAMPLE_REQUIRE(proven_http_cookie_jar_header(&jar, PROVEN_LIT("example.com"), PROVEN_LIT("/"), true, now,
out, &n) == PROVEN_OK && n == 0,
                    "another host: nothing");

    /* Time passes; Max-Age runs out. And a server deletes a cookie by expiring it. */
    proven_time_t later = now + (proven_time_t)7200 * 1000000000;
    EXAMPLE_REQUIRE(proven_http_cookie_jar_header(&jar, host, PROVEN_LIT("/cart"), true, later, out, &n) ==
PROVEN_OK &&
                    view_is((proven_u8str_view_t){ buf, n }, "sid=abc123"), "two hours on, items has
expired");
    EXAMPLE_REQUIRE(proven_http_cookie_jar_store(&jar, host, PROVEN_LIT("/"), true, PROVEN_LIT("sid=; Path=/;
Max-Age=0"), later) == PROVEN_OK &&
                    proven_http_cookie_jar_count(&jar) == 0, "logged out: the jar is empty");

    proven_http_cookie_jar_clear(&jar);       /* or empty it yourself */
    proven_http_cookie_jar_destroy(&jar);

    // ---- Server-sent events ------------------------------------------------
    /* The parser is fed whatever arrived, in whatever pieces, and hands back whole events. */
    proven_byte_t work[512];
    proven_sse_t sse;
    EXAMPLE_REQUIRE(proven_sse_init(&sse, (proven_mem_mut_t){ work, sizeof work }) == PROVEN_OK, "a parser
with room for 256-byte events");

    static const char *const arrived[] = {
        ": keep-alive\n\n",                          /* a comment: no event */
        "id: 41\nevent: price\ndata: {\"sym\":\"X\",",   /* an event, cut in the middle */
        "\ndata:  \"p\":12.5}\n\n",                  /* ...and its end */
        "data: plain\n\n",
    };
    int events = 0;
    for (proven_size_t i = 0; i < 4; ++i) {
        proven_mem_view_t in = proven_mem_view_from_u8(proven_u8str_view_from_cstr(arrived[i]));
        while (in.size > 0) {
            proven_size_t used = 0;
            proven_sse_event_t ev;
            bool have = false;
            EXAMPLE_REQUIRE(proven_sse_feed(&sse, in, &used, &ev, &have) == PROVEN_OK, "the stream parses");
            in.ptr += used;
            in.size -= used;
            if (!have) continue;          /* everything consumed, no event yet */
            events++;
```

```
            if (events == 1) {
                EXAMPLE_REQUIRE(view_is(ev.event, "price") && view_is(ev.id, "41"), "a typed event with an
id");
                EXAMPLE_REQUIRE(view_is(ev.data, "{\"sym\":\"X\",\n \"p\":12.5}"), "two data lines, joined by
a line feed");
            } else {
                EXAMPLE_REQUIRE(ev.event.size == 0 && view_is(ev.data, "plain"), "no type given: a
\"message\"");
                EXAMPLE_REQUIRE(view_is(ev.id, "41"), "the id stays until another replaces it");
            }
        }
    }
    EXAMPLE_REQUIRE(events == 2, "two events from four pieces");
    /* After a disconnect, send this as Last-Event-ID and the server can resume from there. */
    EXAMPLE_REQUIRE(view_is(proven_sse_last_id(&sse), "41"), "where to resume");

    return EXAMPLE_OK();
}
```

14. What is not here

- **A client or a server.** They are Chapter 11.
- **HTTP/2 and HTTP/3.** A head that says `HTTP/2.0` is `PROVEN_ERR_UNSUPPORTED`.
- **Content codings in the codec.** The parser and the body decoder hand you the body as it was sent. Two helpers read the headers involved (`proven_http_accepts_coding`, `proven_http_content_coding`), and the servers and clients built on this codec can compress and decode `gzip`: Chapter 16, section 6.
- **Trailers.** After a chunked body they are checked and skipped, not returned.
- **Parsing a multipart body, and multi-range (`multipart/byteranges`) responses.**
- **A public suffix list**, and so cookies shared across subdomains - see section 12.
- **`Negotiate`, `NTLM` and bearer-token flows.** A bearer token is a header you write yourself.

Chapter 11: An HTTP Client and Server

Part V - Talking to the operating system. Prerequisites: Chapter 9 (addresses, deadlines, the transport) and Chapter 10 (requests, responses, headers); Chapter 6 section 1 if you will run handlers on a job system. After this chapter you can serve HTTP from a function of your own, with every wait bounded and every malformed request refused before it reaches you; and fetch a URL - through redirects, a login, a proxy - reading the body at your own pace.

This chapter covers `http_server.h` and `http_client.h`. Both need sockets: they are not part of a freestanding build, and `PROVEN_NO_NET` leaves them out.

Table of contents

1. What these are, and the one thing they are not
2. A server
3. Inside a handler
4. What the server does without asking you
5. Two ways to run handlers
6. A client
7. One request, one response
8. Redirects, challenges and cookies
9. Connections and proxies
10. Where TLS will attach
11. Changing protocol
12. What is not here

1. What these are, and the one thing they are not

Chapter 10 is a codec: it turns bytes into messages and back, and never touches a socket. Chapter 9 is sockets, and knows nothing of HTTP. These two headers are the part in between - the loop that accepts, reads, parses, times out and keeps connections open, and its mirror image on the client side.

They are drivers and nothing above that. The server has no router, no static-file handler, no sessions and no templates: it calls one function with one request. The client has no retry policy, no cache and no JSON: it sends one request and gives you one response. What a program builds on them is the program's.

TLS is a choice you make, and the default is none. A server speaks plain HTTP unless its config is given a TLS configuration, and a client refuses an `https` URL with `PROVEN_ERR_UNSUPPORTED` - it does not quietly fetch it over `http` instead - unless it is given the TLS wrap. Both are one field each; Chapter 14 is how. Without them, everything these two send and receive can be read, and changed, by anyone on the path:

- Give the server a TLS configuration, or put it behind something that terminates TLS, or keep it on the loopback interface or a network you trust.
- Do not send a password, a token or a session cookie through a client over `http://` to another machine.

That is a statement of what the code does, not a recommendation to be weighed.

2. A server

```
static void handle(void *ctx, proven_http_exchange_t *x) {
    (void)proven_http_exchange_respond(x, 200, NULL, 0, body);
}

proven_http_server_config_t config = {0};
```

```
config.alloc = proven_heap_allocator();
config.handler = handle;
proven_http_server_create(&config, &server);
proven_http_server_listen(server, proven_net_addr_any(PROVEN_NET_FAMILY_IPV4, 8080), NULL);
proven_http_server_run(server);          /* until proven_http_server_stop */
proven_http_server_destroy(server);
```

Reference

API	Intent	Return
`proven_http_server_create(&config, &server)`	Make a server. It does not listen yet.	`proven_err_t`: `INVALID_ARG` without an allocator or a handler; `NOMEM`; `BUSY` when out of sockets.
`proven_http_server_listen(server, at, &bound)`	Listen at an address. Up to four - an IPv4 and an IPv6 one, say. `bound` (optional) receives the address actually bound, so port 0 can be used.	the errors of `proven_net_listen`; `OUT_OF_BOUNDS` for a fifth.
`proven_http_server_run(server)`	Serve until stopped, or until a shutdown is over.	`proven_err_t`: `PROVEN_OK` after a stop, or a shutdown that finished everything; `TIMEOUT` after a shutdown whose deadline cut something.
`proven_http_server_poll(server, until)`	One round: wait until something is ready or until `until`, and handle it. For a program with a loop of its own.	`proven_err_t`: `PROVEN_OK` when something was handled, `TIMEOUT` when nothing was.
`proven_http_server_stop(server)`	Make `run` return. Safe from any thread and from a handler.	none.
`proven_http_server_shutdown(server, grace_ms)`	End gracefully (below). Safe from any thread and from a handler; returns at once.	`proven_err_t`: `INVALID_STATE` when already asked.
`proven_http_server_shutting_down(server)`	Whether a shutdown was asked. Any thread: for a handler that keeps an upgraded connection.	`bool`.
`proven_http_server_drained(server, &why)`	For a program that calls `poll`: whether the shutdown is over, and how.	`bool`; `why` as `run` returns.
`proven_http_server_destroy(server)`	Close every connection and listener and free the server. Not from a handler.	none.
`proven_http_server_connection_count(server)`	Connections open now. For the loop's thread.	`proven_size_t`.

```
typedef struct {
    proven_allocator_t alloc;          /* required */
    proven_http_handler_fn handler;    /* required: void (*)(void *ctx, proven_http_exchange_t *) */
    void *handler_ctx;
    proven_job_sys_t *jobs;            /* NULL: handlers run on the loop's thread (section 5) */
```

```
    proven_size_t max_connections;     /* 0: 64 */
    proven_size_t max_head_bytes;      /* 0: 16 KiB - request head, and response head */
    proven_size_t max_headers;         /* 0: 64 fields */
    proven_u64 max_body_bytes;         /* 0: 1 MiB */
    proven_u32 head_timeout_ms;        /* 0: 10 s */
    proven_u32 body_timeout_ms;        /* 0: 30 s */
    proven_u32 write_timeout_ms;       /* 0: 30 s */
    proven_u32 idle_timeout_ms;        /* 0: 60 s */
    proven_u32 linger_ms;              /* 0: 1 s */
    const proven_tls_config_t *tls;    /* NULL: plain HTTP */
} proven_http_server_config_t;
```

Zero the struct and set what you need. Every limit has a default, and none can be turned off: there is no "unlimited".

What each limit is for

Limit	What it stops
`head_timeout_ms`	A client that opens a connection and sends a request one byte a minute. It has this long from the first byte to the end of the head, then it gets a `408` and is closed. A new connection that sends nothing at all gets the same time.
`body_timeout_ms`	The same trick played with the body: each read of the body may wait this long.
`write_timeout_ms`	A client that asks for a large response and never reads it. Each write may wait this long.
`idle_timeout_ms`	Connections kept open and unused. Closed without a word, as HTTP allows.
`linger_ms`	Not a limit on the client but on the server's patience with one it is closing. After an answer that ends the connection - a refusal in the middle of an upload, a `Connection: close` - the server has said all it will, and still reads and discards what the client sends for this long, because closing at once would turn the answer into a reset on the client's side. A connection in that state counts against `max_connections`: where many requests are refused at once, a shorter time gives the places back sooner, and a longer one suits clients on slow links.
`tls`	A TLS configuration with a certificate (Chapter 14): every connection is then HTTPS. It must outlive the server. The handshake shares `head_timeout_ms` with the first request's head.
`max_head_bytes`, `max_headers`	A head that never ends, or has ten thousand fields: `431`.
`max_body_bytes`	A body larger than you are prepared to take: `413`, before it is read when it was announced, while it is read when it was chunked.
`max_connections`	More connections than you have memory for. The next client waits in the listen backlog; it is not refused.

Memory per connection is fixed when it is accepted: `2 * max_head_bytes` plus 4 KiB plus the header array - about 38 KiB with the defaults, so 64 connections are about 2.4 MiB. Nothing a client sends makes a connection cost more.

Cautions, and what goes wrong

`proven_net_addr_any` **is every interface.** A server listening there is reachable from wherever the machine is, in the clear. For a service meant for the same machine, listen on `proven_net_addr_loopback`.

An IPv6 listener does not take IPv4. As Chapter 9 says, IPv6 sockets here are IPv6-only on every platform. For both, call `proven_http_server_listen` twice.

`destroy` is not `stop`. `stop` makes `run` return, and the server stays stopped: `run` called again returns at once. `destroy` frees it. From a handler, call `stop`, never `destroy`.

Ending gracefully: `proven_http_server_shutdown(server, grace_ms)`. `stop` leaves every connection as it is, for `destroy` to cut. A shutdown ends them in order. From the next round the listeners are closed - a new connection is refused - and a connection with nothing of a request on it is closed; one where a request has begun to arrive, or is with its handler, finishes it, and the response says `Connection: close`. `run` returns `PROVEN_OK` when nothing is left, or `PROVEN_ERR_TIMEOUT` when `grace_ms` has passed - and then closes what the loop holds. **The deadline bounds connections, not handlers**: a handler still running on a worker keeps its connection until it returns, and `destroy` waits for it, as always. A connection handed over by an upgrade is the program's: a handler that keeps one looks at `proven_http_server_shutting_down` and closes it (a WebSocket with 1001, "going away"). A client may send a request just as its idle connection is closed; it sends it again if it can, and this library's clients do so once. The example below ends that way.

What an open connection costs the loop. The loop waits on a selector (Chapter 9, section 9) and keeps its timeouts on a timer wheel, so a round of the loop costs in proportion to the connections that have something to do, not to the connections that are open: with twenty thousand idle keep-alive connections held, a request on one more takes as long as with none (measured on Linux, where the selector is `epoll`). On Windows the selector is built on an I/O completion port; on a Windows 11 virtual machine, ten connections making requests among 10,000 held were answered about 52,000 times a second, a median of 0.15 ms, as among 1,000 (Chapter 15, section 14).

That removes one limit and leaves two. Each connection still holds its buffers from the moment it is accepted (below), and each request still occupies a thread while its handler runs (section 5). A server for tens of thousands of *busy* connections needs neither of those to be true, and this one does not provide that yet. Timeouts are kept to about a sixtieth of a second: a connection may live that much longer than its limit, never shorter.

3. Inside a handler

The handler is called with a request whose head has been read, parsed and checked. The body has not been read: that is the handler's choice.

Reference

API	Intent	Return
`proven_http_exchange_request(x)`	The request: method, target, headers (Chapter 10's `proven_http_request_t`).	pointer; views are good until the handler returns.
`proven_http_exchange_peer(x)`	The client's address.	`proven_net_addr_t`.
`proven_http_exchange_early(x)`	The request arrived as TLS early data (0-RTT), before the handshake was over - only with a `tls` configuration that has `early_data`, and only for GET, HEAD or OPTIONS without a body: the server answers anything else that arrives early with 425 itself (Chapter 14, section 6). A	`bool`.

	handler whose GET changes something answers 425 to an early one; the client sends it again.	
`proven_http_exchange_read(x, dest)`	Read more of the request body.	`proven_result_size_t`: `.err` is `EOF` when the body is complete; `TIMEOUT`; `OUT_OF_BOUNDS` past `max_body_bytes`; `INVALID_FORMAT` for bad chunk framing; `RESET` when the client left.
`proven_http_exchange_respond(x, status, headers, count, body)`	Send a whole response, with a body held in memory.	`proven_err_t`: `INVALID_STATE` when a response was already begun; `INVALID_ARG` for a header the server owns or a value the writers refuse; `OUT_OF_BOUNDS` when the head does not fit; `TIMEOUT`, `RESET`.
`proven_http_exchange_begin(x, status, headers, count, content_length)`	Begin a response whose body follows in pieces. `PROVEN_HTTP_LENGTH_UNKNOWN` when you do not know the length: the body is then sent chunked.	as `respond`.
`proven_http_exchange_write(x, data)`	Write more of that body.	`proven_err_t`: `OUT_OF_BOUNDS` past the length announced; `INVALID_STATE`; `TIMEOUT`, `RESET`.
`proven_http_exchange_end(x)`	Finish it. Optional: returning from the handler does the same.	`proven_err_t`.
`proven_http_exchange_flush(x)`	Hand over what a compressed response holds back (Chapter 16, section 5); nothing for any other.	`proven_err_t`: `INVALID_STATE` before `begin` or after `end`; otherwise as a write.
`proven_http_exchange_compress(x)`	Ask that this response be compressed, if the client accepts gzip. Before the response begins. Chapter 16, section 6 - read its cautions before using it.	none.

Cautions, and what goes wrong

The target is the client's text. `request->target` is what was sent - `/a/b?x=1`, or an absolute URL, undecoded. Split it with `proven_url_split_target` before comparing, and never build a file name from it without `proven_url_path_resolve` (Chapter 10, section 3).

Wrong:

```
snprintf(file, sizeof file, "/var/www%.*s", (int)req->target.size, req->target.ptr);   /* wrong: "/../../etc/passwd" */
```

Correct - `proven_url_split_target` for the path, `proven_url_path_resolve` on it, and only a `PROVEN_OK` result joined to the root.

Do not write `Content-Length`, `Transfer-Encoding`, `Connection` or `Date` yourself. The server writes them, from what you actually send. Giving one is `PROVEN_ERR_INVALID_ARG`, and nothing is sent - two `Content-Length` fields that disagree are exactly the ambiguity that request smuggling is made of, and here it cannot be produced.

A header value taken from a request cannot split your response. A value with a line break in it is refused by the writers, so `Location:` followed by text a client chose cannot add a header or a second response. After that refusal nothing has been sent, and you may send another response - an error page - instead.

Check the result of a write and stop when it fails. `PROVEN_ERR_RESET` and `PROVEN_ERR_TIMEOUT` mean the client is gone or not reading. A handler that carries on generating a large body for nobody holds a connection - and in the single-thread model the whole server - for nothing.

If you return without sending anything, the client gets a `500`. That is the server keeping its promise that every request gets an answer; it is not a way to send errors. When reading the body failed, the status is the one that failure calls for instead - `408`, `413` or `400`.

Everything from the exchange ends when the handler returns - the request, its views, the exchange itself. A handler that hands the exchange to another thread and returns has handed over a dangling pointer. To answer later, do not return: wait in the handler (with handlers on a job system, section 5, that costs one worker).

Announce a length only if you will write that many bytes. Writing more is refused. Writing fewer cannot be repaired - the client is waiting for the rest - so the connection is closed at that point and the client sees a truncated response, which is the truth.

4. What the server does without asking you

A request that is malformed, ambiguous or too large never reaches the handler:

The request	The answer
Not parseable as HTTP/1.x: bare line feeds, a space before a colon, control bytes, a folded header	`400`
HTTP/1.1 without exactly one `Host`	`400`
Both `Content-Length` and `Transfer-Encoding`; two lengths that differ; a length that is not a number	`400`
A head larger than `max_head_bytes`, or more than `max_headers` fields	`431`
Another HTTP version	`505`
A transfer coding other than `chunked`	`501`
A body announced larger than `max_body_bytes`	`413`
A head not finished within `head_timeout_ms`	`408`
The job system cannot take another request (section 5)	`503`

Each of these closes the connection, in the way the end of this section describes. They are deliberately strict: where two servers read one request differently, an attacker puts one request inside another, and the cure is to refuse what is ambiguous rather than pick a reading.

And around a request that does reach the handler:

- **Keep-alive.** The connection stays open for the next request unless the client asked to close it, the request was HTTP/1.0 without `keep-alive`, or the response could not be framed.
- **Pipelining.** Requests sent back to back are answered in order.
- **An unread body.** If the handler did not read the request body, the server reads and discards it - up to 64 KiB - so the next request starts in the right place. Past that it closes.

- `Expect: 100-continue`. The `100 Continue` is sent when the handler first reads the body. A handler that refuses by the headers alone never invites the body, and the connection is then closed, since whether a body follows is the client's secret.
- `HEAD`. The handler answers as for `GET`; the server sends the head - `Content-Length` included - and none of the body.
- `204` and `304` are sent without a body and without a length.
- **HTTP/1.0.** A response of unknown length cannot be chunked for a 1.0 client; it is sent until the connection closes.
- `Date` on every response.
- **Closing without destroying the answer.** When the server closes a connection it first says so - it stops sending - and then keeps reading and discarding what the client sends, for up to a second, until the client closes too. A socket closed with unread input answers with a reset, and a reset makes the client's system discard the response it had not yet read: a client told `413` in the middle of an upload would see a broken connection instead. The second is fixed and is not a setting.

5. Two ways to run handlers

`jobs` in the configuration chooses.

`jobs == NULL`**: on the loop's thread.** One thread does everything. While a handler runs - or waits for a slow client to send its body, or to read its response - no other connection is served. That is fine for handlers that compute and answer; it is simple, and nothing needs a lock. It is wrong for a handler that waits on something slow.

`jobs` **set: on a job system's workers.** Each request is handed to the job system of Chapter 6, and the loop goes straight back to accepting and reading. The handler reads the body and writes the response from its worker; the connection belongs to it until it returns. Handlers run at the same time, and so:

- whatever they share needs the protection shared data always needs;
- `alloc` in the configuration is called from several threads and must be thread-safe - the heap allocator is; an arena is not;
- when the job queue is full the request is answered `503` and the connection closed. Size the queue for the load you mean to carry, and treat `503` as the server saying it has enough.

The handler is written the same way for both. What it may not do in either: block for ever. A handler that never returns holds a connection for ever, and with it a worker - or the server.

Slow clients cost a worker, not the loop - but they do cost a worker. With `jobs` set, a client that sends its body slowly occupies the worker running that handler until `body_timeout_ms`. Enough such clients occupy every worker. The timeouts bound it; they do not make it free. If that matters, read the request body in the handler only when you must, and keep the timeouts short.

`proven_http_server_destroy` waits for handlers that are running on workers. Close the job system after destroying the server, not before.

Compiled and run by the test suite:

```
/*
 * A complete HTTP server: one function that answers requests, and a loop that does the rest.
 *
 * The handler sees a request that has already been read, checked and limited. It reads the
 * body if it wants it, and sends one response - whole, or in pieces.
 */

typedef struct {
    proven_http_server_t *server;
    int served;
```

```
    proven_err_t ended;                 /* what run returned */
} ex_app_t;

static void handle(void *ctx, proven_http_exchange_t *x) {
    ex_app_t *app = ctx;
    const proven_http_request_t *req = proven_http_exchange_request(x);
    app->served++;

    /* The target is a path and perhaps a query; split it before comparing. */
    proven_u8str_view_t path, query;
    bool has_query;
    if (proven_url_split_target(req->target, &path, &query, &has_query) != PROVEN_OK) {
        (void)proven_http_exchange_respond(x, 400, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("bad
target\n")));
        return;
    }

    if (req->method == PROVEN_HTTP_GET && proven_u8str_view_eq(path, PROVEN_LIT("/"))) {
        /* The simplest answer: a status, your headers, a body in memory. Content-Length,
         * Date and Connection are written for you. */
        proven_http_header_t type = { PROVEN_LIT("Content-Type"), PROVEN_LIT("text/plain; charset=utf-8") };
        (void)proven_http_exchange_respond(x, 200, &type, 1, proven_mem_view_from_u8(PROVEN_LIT("hello\n")));

    } else if (req->method == PROVEN_HTTP_POST && proven_u8str_view_eq(path, PROVEN_LIT("/shout"))) {
        /* Read the body as it comes and answer in pieces, without knowing the length first:
         * the response goes out chunked. Nothing here holds the whole body. */
        if (proven_http_exchange_begin(x, 200, NULL, 0, PROVEN_HTTP_LENGTH_UNKNOWN) != PROVEN_OK) return;
        proven_byte_t piece[256];
        for (;;) {
            proven_result_size_t got = proven_http_exchange_read(x, (proven_mem_mut_t){ piece, sizeof
piece });
            if (got.err != PROVEN_OK) break;          /* PROVEN_ERR_EOF: the body is complete */
            for (proven_size_t i = 0; i < got.value; ++i) {
                if (piece[i] >= 'a' && piece[i] <= 'z') piece[i] = (proven_byte_t)(piece[i] - 32);
            }
            if (proven_http_exchange_write(x, (proven_mem_view_t){ piece, got.value }) != PROVEN_OK)
return;   /* the client left */
        }
        (void)proven_http_exchange_end(x);

    } else if (proven_u8str_view_eq(path, PROVEN_LIT("/whoami"))) {
        proven_net_addr_t peer = proven_http_exchange_peer(x);
        proven_byte_t text[PROVEN_NET_ADDR_TEXT_MAX];
        proven_size_t n = 0;
        if (proven_net_addr_format(&peer, (proven_mem_mut_t){ text, sizeof text }, &n) != PROVEN_OK)
return;   /* returning with nothing sent: a 500 */
        (void)proven_http_exchange_respond(x, 200, NULL, 0, (proven_mem_view_t){ text, n });

    } else if (proven_u8str_view_eq(path, PROVEN_LIT("/quit"))) {
        (void)proven_http_exchange_respond(x, 200, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("bye\n")));
        /* End gracefully: what is open finishes - this answer too - then run returns. */
        if (proven_http_server_shutdown(app->server, 5000) != PROVEN_OK) proven_http_server_stop(app->server);

    } else {
        (void)proven_http_exchange_respond(x, 404, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("no such
page\n")));
    }
}

static void serve(void *arg) {
```

```
    ex_app_t *app = arg;
    app->ended = proven_http_server_run(app->server);   /* until the shutdown is over */
}

/* GET or POST `path` and compare the whole body. */
static bool fetch_is(proven_http_client_t *client, proven_u16 port, const char *method, const char *path,
const char *body, proven_u16 status, const char *expect) {
    char url[128];
    int n = snprintf(url, sizeof url, "http://127.0.0.1:%u%s", (unsigned)port, path);
    proven_http_client_request_t req = {
        .method = proven_u8str_view_from_cstr(method),
        .url = { (const proven_byte_t *)url, (proven_size_t)n },
        .body = proven_mem_view_from_u8(proven_u8str_view_from_cstr(body)),
    };
    proven_http_client_response_t resp;
    bool ok = proven_http_client_send(client, &req, &resp) == PROVEN_OK && resp.status == status;
    proven_u8str_t text = {0};
    ok = ok && proven_http_client_read_all(&resp, proven_heap_allocator(), &text, 4096) == PROVEN_OK;
    ok = ok && (expect == NULL || proven_u8str_view_eq(proven_u8str_as_view(&text),
proven_u8str_view_from_cstr(expect)));
    proven_u8str_destroy(proven_heap_allocator(), &text);
    proven_http_client_finish(&resp);
    return ok;
}

int main(void) {
    static ex_app_t app;
    proven_allocator_t heap = proven_heap_allocator();

    /* Zero the configuration and set what you need; every zero field has a default. */
    proven_http_server_config_t config = {0};
    config.alloc = heap;
    config.handler = handle;
    config.handler_ctx = &app;
    config.max_body_bytes = 64 * 1024;        /* a larger request body is answered 413 */
    config.head_timeout_ms = 5000;            /* a client has this long to send its request head */
    proven_err_t err = proven_http_server_create(&config, &app.server);
    EXAMPLE_REQUIRE(err == PROVEN_OK, "a server");

    /* Port 0 lets the system choose; `at` says what it chose. */
    proven_net_addr_t at;
    err = proven_http_server_listen(app.server, proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0), &at);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        proven_http_server_destroy(app.server);
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK, "it listens");

    /* A program with a loop of its own calls poll: one round, bounded by a deadline. */
    EXAMPLE_REQUIRE(proven_http_server_poll(app.server, proven_net_deadline_in(10)) == PROVEN_ERR_TIMEOUT,
"nobody has connected yet");

    /* Most programs call run. Here it runs on another thread, so this one can be the client. */
    proven_job_sys_t *threads = NULL;
    proven_job_group_t running;
    proven_job_group_init(&running);
    EXAMPLE_REQUIRE(proven_job_system_init(heap, 1, 4, &threads) == PROVEN_OK &&
                    proven_job_group_submit(threads, &running, serve, &app) == PROVEN_OK, "the server loop is
started");
```

```
    proven_http_client_config_t client_config = {0};
    client_config.alloc = heap;
    client_config.max_idle_connections = 2;
    proven_http_client_t *client = NULL;
    EXAMPLE_REQUIRE(proven_http_client_create(&client_config, &client) == PROVEN_OK, "a client");

    EXAMPLE_REQUIRE(fetch_is(client, at.port, "GET", "/", "", 200, "hello\n"), "GET /");
    EXAMPLE_REQUIRE(fetch_is(client, at.port, "POST", "/shout", "a body, read and answered in pieces", 200, "A
BODY, READ AND ANSWERED IN PIECES"), "POST /shout");
    EXAMPLE_REQUIRE(fetch_is(client, at.port, "GET", "/whoami?verbose=1", "", 200, NULL), "GET /whoami");
    EXAMPLE_REQUIRE(fetch_is(client, at.port, "GET", "/nothing-here", "", 404, "no such page\n"), "a 404 is
the handler's to send");
    EXAMPLE_REQUIRE(fetch_is(client, at.port, "GET", "/quit", "", 200, "bye\n"), "GET /quit");

    /* The handler asked for a shutdown: run returned when nothing was left open. */
    proven_job_group_wait(threads, &running);
    EXAMPLE_REQUIRE(app.served == 5, "five requests reached the handler");
    EXAMPLE_REQUIRE(app.ended == PROVEN_OK && proven_http_server_connection_count(app.server) == 0, "run
returned PROVEN_OK: everything open had finished");
    /* A program that calls poll itself, not run, asks the same of proven_http_server_drained. */
    proven_err_t how = PROVEN_ERR_INVALID_STATE;
    EXAMPLE_REQUIRE(proven_http_server_shutting_down(app.server) && proven_http_server_drained(app.server,
&how) && how == PROVEN_OK, "the shutdown is over, and nothing was cut");

    proven_http_client_destroy(client);
    proven_http_server_destroy(app.server);   /* closes what is left */
    proven_job_system_close(threads);
    proven_job_system_destroy(threads);
    return EXAMPLE_OK();
}
```

6. A client

```
proven_http_client_config_t config = {0};
config.alloc = proven_heap_allocator();
proven_http_client_create(&config, &client);

proven_http_client_response_t resp;
if (proven_http_client_get(client, PROVEN_LIT("http://example.com/"), &resp) == PROVEN_OK) {
    /* resp.status, resp.headers; then proven_http_client_read until PROVEN_ERR_EOF */
}
proven_http_client_finish(&resp);
proven_http_client_destroy(client);
```

Reference

API	Intent	Return
`proven_http_client_create(&config, &client)`	Make a client. The configuration's strings are copied.	`proven_err_t`: `INVALID_ARG` for an invalid allocator, a proxy that is not `http` or `socks5`, a control character in a credential, or `http3` without a `tls` that offers `h3`; `UNSUPPORTED` for `http3` in a build without it; `NOMEM`.
`proven_http_client_destroy(client)`	Close idle connections and free it. Finish every response first.	none.

`proven_http_client_send(client, &request, &response)`	Send a request and read the head of the response.	`proven_err_t`; see section 7.
`proven_http_client_get(client, url, &response)`	`send` with only a URL.	the same.
`proven_http_client_read(&response, dest)`	Read more of the body.	`proven_result_size_t`: `.err` is `EOF` at the end; `RESET` for a body cut short; `TIMEOUT`; `INVALID_FORMAT`.
`proven_http_client_read_all(&response, alloc, &out, max_bytes)`	Read the rest of the body, appending to a string.	`proven_err_t`: `OUT_OF_BOUNDS` when the body is longer than `max_bytes` - `out` then holds that many.
`proven_http_client_finish(&response)`	Release a response; the connection is kept or closed. Safe after a failed send.	none.

```
typedef struct {
    proven_allocator_t alloc;               /* required */
    proven_u32 connect_timeout_ms;          /* 0: 30 s */
    proven_u32 io_timeout_ms;               /* 0: 30 s, for each read and each write */
    proven_size_t max_head_bytes;           /* 0: 16 KiB */
    proven_u32 max_redirects;               /* 0: none are followed */
    proven_size_t max_idle_connections;     /* 0: every connection is closed after use */
    proven_u8str_view_t proxy;              /* "http://host:port" or "socks5://host:port" */
    proven_u8str_view_t no_proxy;           /* hosts reached without it: "localhost, .corp.example" */
    proven_u8str_view_t username, password; /* offered when a server answers 401 */
    proven_u8str_view_t user_agent;         /* empty: none is sent */
    proven_http_cookie_jar_t *cookies;      /* NULL: cookies are ignored */
    proven_http_tls_wrap_fn tls_wrap;       /* NULL: an https URL is PROVEN_ERR_UNSUPPORTED */
    void *tls_ctx;
    proven_u32 expect_timeout_ms;           /* 0: 1 s - see expect_continue in section 7 */
    bool http3;                             /* a request that sets http3 goes over HTTP/3 - section 10 */
    const proven_tls_config_t *tls;         /* with http3: the QUIC connections' TLS, offering h3 */
    bool http3_fallback;                    /* with http3: TCP when QUIC does not come up */
} proven_http_client_config_t;
```

A zeroed configuration does the least. With only `alloc` set, a client follows no redirect, keeps no connection, offers no credentials, stores no cookie and sends no `User-Agent`. Each of those is something you turn on, knowing you did.

One client, one thread. A client is not safe to use from several threads at once. Give each thread its own; they are cheap.

7. One request, one response

```
typedef struct {
    proven_u8str_view_t method;             /* empty: GET */
    proven_u8str_view_t url;                /* absolute: http://host[:port]/path?query */
    const proven_http_header_t *headers;    /* extra header fields */
    proven_size_t header_count;
    proven_mem_view_t body;                 /* a body in memory: sent with Content-Length */
    proven_reader_t body_stream;            /* or a stream: sent chunked */
    bool expect_continue;                   /* ask the server before sending the body */
} proven_http_client_request_t;

typedef struct {
    proven_u16 status;
    proven_u8str_view_t reason;
```

```
    const proven_http_header_t *headers;
    proven_size_t header_count;
    proven_u8str_view_t url;               /* where this response came from, after redirects */
    proven_u32 redirects;
    void *internal;
} proven_http_client_response_t;
```

`proven_http_client_send` returns when the status and headers have arrived. The body is still on the connection, and you read it in pieces into a buffer of your own - so a response of any size costs the memory you chose, and a download can go straight to a file.

An error status is not an error. A `404` or a `500` is a response: `send` returns `PROVEN_OK` and you look at `status`. `send` fails when there is *no* response to look at:

`send` **returns**	**Because**
`PROVEN_ERR_INVALID_FORMAT`	The URL is not absolute, or the response was not HTTP - or was more than eight interim (`1xx`) responses with no final one.
`PROVEN_ERR_UNSUPPORTED`	A scheme other than `http` - or `https` without a `tls_wrap`.
`PROVEN_ERR_INVALID_ARG`	A header the client writes itself was given: `Host`, `Content-Length`, `Transfer-Encoding`, `Connection`. Or both a body and a stream.
`PROVEN_ERR_NOT_FOUND`	The host name does not resolve.
`PROVEN_ERR_REFUSED`, `PROVEN_ERR_UNREACHABLE`, `PROVEN_ERR_TIMEOUT`, `PROVEN_ERR_RESET`	The connection: Chapter 9's meanings.
`PROVEN_ERR_OUT_OF_BOUNDS`	The request or the response head is larger than `max_head_bytes`.
`PROVEN_ERR_PERMISSION`	A proxy refused.
`PROVEN_ERR_UNTRUSTED`, `PROVEN_ERR_EXPIRED`, `PROVEN_ERR_NOT_YET_VALID`, `PROVEN_ERR_NAME_MISMATCH`, `PROVEN_ERR_PROTOCOL`	From `tls_wrap`: the server could not be verified, or the TLS handshake failed (Chapter 14, section 8). Nothing of the request was sent.

Cautions, and what goes wrong

Always call `finish`. A response holds a connection and memory until it is finished. `finish` is safe on a response whose `send` failed, so the simple shape is the right one: `send`, use it if that succeeded, `finish` either way.

`PROVEN_ERR_EOF` is the end of the body; `PROVEN_ERR_RESET` is not. A body that stops before its announced length - the server crashed, the network dropped - is reported as `PROVEN_ERR_RESET`, never as `EOF`. Treat only `EOF` as a complete body.

Wrong:

```
while (proven_http_client_read(&resp, buf).err == PROVEN_OK) save(buf);   /* wrong: stops on RESET and calls the file done */
commit_download();
```

Correct - keep the result, and commit only when it is `PROVEN_ERR_EOF`.

`read_all` needs a limit, and you choose it. A server decides how much it sends. `max_bytes` is the most you will hold; a longer body is `PROVEN_ERR_OUT_OF_BOUNDS`, not an allocation that grows until the machine has none left.

The response's views die with it. `headers`, `reason` and `url` point into memory that `finish` frees. Copy what you keep.

A stream body is sent once. A body read from `body_stream` cannot be read again, so a request that has one is not repeated - not for a redirect that needs the body, not for a challenge, not when a kept connection turns out to be dead. You get the response that would have caused the repeat, and decide.

Asking before a large body. With `expect_continue` the client sends the head with `Expect: 100-continue` and holds the body back until the server has spoken:

The server	The client
answers `100 Continue`	sends the body, and the call returns the response that follows.
answers anything final - `401`, `413`, `417`	sends none of the body; that answer is the response. The connection is not kept afterwards, whatever the answer says. A stream body has not been read from.
says nothing for `expect_timeout_ms` (1 s unless set)	sends the body anyway: a server that does not know the rule is waiting for it.

It costs a round trip, so it is for a body worth not sending: an upload that a server may refuse for its size or for missing credentials. It does nothing for a request without a body. Set the flag and do not write the field yourself: an `Expect` among your own headers is sent as it is, and the body follows it at once. The servers of this library answer it (section 4).

An answer in the middle of an upload. Without `expect_continue` a server can still answer before it has the whole body, and stop reading. The client's write then fails - and the client reads what the server said before reporting anything: a `413` that arrived is the result of the call, not `PROVEN_ERR_RESET`. Only when no answer is there is the failed write the error. Two limits, both the system's and not this library's: a write that *timed out* is reported at once, without a second wait; and on Windows a reset discards what had arrived and not been read, so there the answer can be lost - `expect_continue` is what does not depend on it.

`io_timeout_ms` **bounds each wait, not the whole request.** A server that sends one byte every ten seconds never trips a thirty-second timeout. For a limit on the whole exchange, note the time before `send` and stop reading when you have had enough.

Name resolution has no deadline, as in Chapter 9: `connect_timeout_ms` starts once the name has resolved.

8. Redirects, challenges and cookies

Redirects (`301`, `302`, `303`, `307`, `308`) are followed up to `max_redirects`, each `Location` resolved against the URL that sent it. `response.url` and `response.redirects` say where you ended up and how.

Status	The next request
`303`	`GET`, without the body - whatever the method was
`301`, `302`	`GET` without the body if the method was `POST`; otherwise the same request again
`307`, `308`	The same request again, body included

A redirect is *not* followed, and its `3xx` response is handed to you instead, when:

- the limit is reached;
- it leads from `https` to `http` - the client does not step down from an encrypted connection because a server said to;
- it leads to a scheme other than `http` and `https`;
- the request would have to be sent again but its body was a stream.

Credentials stay with the host they were given for. When a redirect leads to another host, port or scheme, the `Authorization`, `Cookie` and `Proxy-Authorization` headers you supplied are left behind, and a `401` from the new place is not answered with the configured user name and password. A server you trust with a token must not be able to send that token to a server of its choosing.

What the client does not check is where a redirect leads. A server can redirect to `http://127.0.0.1/` or to an address inside your network. If the URLs come from people you do not trust - a "fetch this link" feature - the destination of every hop is yours to check: set `max_redirects` to 0, read `Location` yourself, and decide.

Challenges. With `username` set, a `401` carrying a Digest or Basic challenge is answered once: Digest if it is offered, Basic otherwise. A Digest nonce reported `stale` is answered once more. A wrong password therefore costs two requests, not a loop. Credentials are never sent before a server asks for them - and when it does ask over plain HTTP with Basic, they are sent readable. Section 1 applies.

Cookies. With a jar in `cookies`, every response's `Set-Cookie` headers are stored and every request carries the cookies that match. A `Cookie` header of your own in a request replaces the jar's for that request. The jar is the host-only one of Chapter 10, section 12, with all that implies.

9. Connections and proxies

Reuse. With `max_idle_connections` above zero, a connection whose response was read to its end goes back to the client and carries the next request to the same scheme, host and port. A response finished *before* its body was read cannot be reused - the rest of the body is still on it - and is closed. To keep the connection, read to `PROVEN_ERR_EOF` before `finish`.

A kept connection can be dead. Servers close idle connections without notice. If a request on a reused connection fails before any byte of an answer arrives, the client sends it again on a new connection, once. That is safe exactly because nothing came back - and it is not done for a stream body, which cannot be sent twice.

Proxies. `proxy` is `http://[user:password@]host:port` or `socks5://[user:password@]host:port`.

Proxy	For an `http` URL	For an `https` URL
HTTP	The proxy is sent the whole URL and fetches it.	The proxy is asked to `CONNECT` to `host:port`; the request then travels through that tunnel.
SOCKS5	A tunnel to `host:port`, then an ordinary request.	The same, with TLS inside it.

- The target's name is given to the proxy, not resolved here - with SOCKS5 as a name, which is what `socks5h` means elsewhere. The machine running the client does not look the name up.
- Credentials in the proxy URL are sent to the proxy - `Proxy-Authorization: Basic` for an HTTP proxy, the user/password method of RFC 1929 for SOCKS5 - and to nobody else: they do not travel inside a tunnel to the origin. Both forms cross the network to the proxy readable.
- A proxy that refuses (`407`, `403`, a SOCKS5 "not allowed") is `PROVEN_ERR_PERMISSION`.
- An HTTP proxy sees, and can change, everything in a plain `http` exchange. That is what a proxy is.

Hosts reached without the proxy. `no_proxy` is a list separated by commas:

Entry	Excuses
`example.org` or `.example.org`	That name and every name under it - `www.example.org`, not `notexample.org`. Letter case does not matter.

`10.0.0.5`, `::1` or `[::1]`	That address, written the same way as in the URL. An address is matched whole: there are no ranges and no `10.0.0.0/8`.
`*`	Every host: the proxy is not used.

The port is not part of an entry. A host in the list is connected to directly - and so its name is looked up on this machine, which is the one thing the proxy otherwise prevents.

Proxy settings are not read from the environment. `HTTP_PROXY` and its relatives are a convention with a history of surprises; if your program honours them, it reads them and sets `proxy` and `no_proxy` itself.

10. Where TLS will attach

```
typedef proven_err_t (*proven_http_tls_wrap_fn)(void *ctx, proven_transport_t plain,
                                                proven_u8str_view_t host,
                                                proven_net_deadline_t until,
                                                proven_transport_t *out);
```

For an `https` URL the client connects - through the proxy tunnel if there is one - and then calls `tls_wrap` with the connected transport and the host name. The function performs the handshake and returns a transport that encrypts; the client speaks HTTP over that and never learns the difference. This is Chapter 9's transport interface doing the job it was shaped for.

The library supplies one: `proven_tls_http_wrap`, with a `proven_tls_config_t *` as its context (Chapter 14, section 3). The seam stays an interface so that the client's behaviour around it - no downgrade on redirect, credentials kept to their origin, the tunnel before the handshake - does not depend on which TLS is behind it, and so that a program with another TLS implementation can attach that instead.

Early data. With `early_data` in that configuration the wrap resumes the newest session of the server's name and, when its ticket allows, returns without sending anything: the request head this client writes next goes with the ClientHello as early data, and is answered without a round trip of its own. Only a GET, HEAD or OPTIONS without a body goes early - for anything else the client completes the handshake first - and a server that answers 425 Too Early gets the request once more, on a new connection, not early (Chapter 14, section 6).

If you write one, it must verify the server. `host` is passed so that the certificate can be checked against it. A wrap that encrypts without verifying gives a connection that is private from everyone except whoever is in the middle of it - and returns `PROVEN_OK`, so nothing downstream can tell. Return `PROVEN_ERR_UNTRUSTED` when the peer cannot be verified; the client passes it up and sends nothing.

Compiled and run by the test suite:

```
/*
 * An HTTP client: from a URL to the bytes, with what lies between - a redirect, a login, a
 * cookie - handled by the client and visible in the response.
 *
 * The server it talks to is in this same program, on another thread, so the example needs no
 * network. Its handler is the first half of the file; the client is in main.
 */

static proven_http_server_t *g_server;

static void site(void *ctx, proven_http_exchange_t *x) {
    (void)ctx;
    const proven_http_request_t *req = proven_http_exchange_request(x);
    proven_u8str_view_t value;
```

```
    if (proven_u8str_view_eq(req->target, PROVEN_LIT("/old-home"))) {
        proven_http_header_t h = { PROVEN_LIT("Location"), PROVEN_LIT("/home") };
        (void)proven_http_exchange_respond(x, 301, &h, 1, (proven_mem_view_t){0});
    } else if (proven_u8str_view_eq(req->target, PROVEN_LIT("/home"))) {
        proven_http_header_t h = { PROVEN_LIT("Set-Cookie"), PROVEN_LIT("visited=yes; Path=/") };
        (void)proven_http_exchange_respond(x, 200, &h, 1, proven_mem_view_from_u8(PROVEN_LIT("welcome
home")));
    } else if (proven_u8str_view_eq(req->target, PROVEN_LIT("/cookie"))) {
        bool has = proven_http_header_find(req->headers, req->header_count, PROVEN_LIT("Cookie"), &value);
        (void)proven_http_exchange_respond(x, 200, NULL, 0, proven_mem_view_from_u8(has ? value :
PROVEN_LIT("none")));
    } else if (proven_u8str_view_eq(req->target, PROVEN_LIT("/private"))) {
        /* "Basic" + base64("ada:lovelace") */
        if (proven_http_header_find(req->headers, req->header_count, PROVEN_LIT("Authorization"), &value) &&
            proven_u8str_view_eq(value, PROVEN_LIT("Basic YWRhOmxvdmVsYWNl"))) {
            (void)proven_http_exchange_respond(x, 200, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("the
private page")));
        } else {
            proven_http_header_t h = { PROVEN_LIT("WWW-Authenticate"), PROVEN_LIT("Basic
realm=\"example\"") };
            (void)proven_http_exchange_respond(x, 401, &h, 1, proven_mem_view_from_u8(PROVEN_LIT("who are
you?")));
        }
    } else if (proven_u8str_view_eq(req->target, PROVEN_LIT("/count"))) {
        /* Answers with how many bytes the request body had. */
        proven_byte_t piece[512];
        proven_u64 total = 0;
        for (;;) {
            proven_result_size_t got = proven_http_exchange_read(x, (proven_mem_mut_t){ piece, sizeof
piece });
            if (got.err != PROVEN_OK) break;
            total += got.value;
        }
        char text[32];
        int n = snprintf(text, sizeof text, "%llu bytes", (unsigned long long)total);
        (void)proven_http_exchange_respond(x, 200, NULL, 0, (proven_mem_view_t){ (const proven_byte_t *)text,
(proven_size_t)n });
    } else {
        (void)proven_http_exchange_respond(x, 404, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("not found")));
    }
}

static void serve(void *arg) {
    (void)arg;
    (void)proven_http_server_run(g_server);
}

static char g_url[128];
static proven_u8str_view_t url_for(proven_u16 port, const char *path) {
    int n = snprintf(g_url, sizeof g_url, "http://127.0.0.1:%u%s", (unsigned)port, path);
    return (proven_u8str_view_t){ (const proven_byte_t *)g_url, (proven_size_t)n };
}

static bool view_is(proven_u8str_view_t v, const char *text) {
    return proven_u8str_view_eq(v, proven_u8str_view_from_cstr(text));
}

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
```

```
    // ---- the server, to have something to talk to ---------------------------
    proven_http_server_config_t server_config = {0};
    server_config.alloc = heap;
    server_config.handler = site;
    EXAMPLE_REQUIRE(proven_http_server_create(&server_config, &g_server) == PROVEN_OK, "a server");
    proven_net_addr_t at;
    proven_err_t err = proven_http_server_listen(g_server, proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4,
0), &at);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        proven_http_server_destroy(g_server);
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK, "it listens");
    proven_job_sys_t *threads = NULL;
    proven_job_group_t running;
    proven_job_group_init(&running);
    EXAMPLE_REQUIRE(proven_job_system_init(heap, 1, 4, &threads) == PROVEN_OK &&
                    proven_job_group_submit(threads, &running, serve, NULL) == PROVEN_OK, "its loop runs on
another thread");

    // ---- the client ---------------------------------------------------------
    proven_http_cookie_jar_t jar;
    EXAMPLE_REQUIRE(proven_http_cookie_jar_init(&jar, heap, 32) == PROVEN_OK, "a cookie jar");

    /* Zero it and set what you need. With every field zero but the allocator, a client
     * follows no redirect, keeps no connection, sends no credentials and holds no cookies. */
    proven_http_client_config_t config = {0};
    config.alloc = heap;
    config.max_redirects = 5;
    config.max_idle_connections = 4;          /* keep connections for the next request */
    config.io_timeout_ms = 5000;              /* no read or write waits longer */
    config.user_agent = PROVEN_LIT("manual-example/1.0");
    config.username = PROVEN_LIT("ada");      /* offered only when a server asks */
    config.password = PROVEN_LIT("lovelace");
    config.cookies = &jar;
    proven_http_client_t *client = NULL;
    EXAMPLE_REQUIRE(proven_http_client_create(&config, &client) == PROVEN_OK, "a client");

    /* GET. The call returns when the status and headers have arrived; the body is still on
     * the wire, and you read it at your own pace into your own buffer. */
    proven_http_client_response_t resp;
    err = proven_http_client_get(client, url_for(at.port, "/old-home"), &resp);
    EXAMPLE_REQUIRE(err == PROVEN_OK && resp.status == 200, "the 301 was followed to a 200");
    EXAMPLE_REQUIRE(resp.redirects == 1 && resp.url.size > 5 && view_is((proven_u8str_view_t){ resp.url.ptr +
resp.url.size - 5, 5 }, "/home"),
                    "and the response says where it ended up");
    proven_byte_t buf[64];
    proven_size_t have = 0;
    for (;;) {
        proven_result_size_t got = proven_http_client_read(&resp, (proven_mem_mut_t){ buf + have, sizeof buf -
have });
        if (got.err == PROVEN_ERR_EOF) break;             /* the whole body has been read */
        EXAMPLE_REQUIRE(got.err == PROVEN_OK, "a piece of the body");
        if (got.err != PROVEN_OK) break;                  /* PROVEN_ERR_RESET: it was cut short */
        have += got.value;
    }
    EXAMPLE_REQUIRE(view_is((proven_u8str_view_t){ buf, have }, "welcome home"), "the body");
    /* Always finish. Read to the end, the connection goes back to the client for reuse. */
    proven_http_client_finish(&resp);
```

```
    /* That response set a cookie; the jar has it and the next request sends it. */
    EXAMPLE_REQUIRE(proven_http_cookie_jar_count(&jar) == 1, "one cookie in the jar");
    proven_u8str_t text = {0};
    err = proven_http_client_get(client, url_for(at.port, "/cookie"), &resp);
    EXAMPLE_REQUIRE(err == PROVEN_OK && proven_http_client_read_all(&resp, heap, &text, 1024) == PROVEN_OK,
"read_all: the whole body, up to a limit you name");
    EXAMPLE_REQUIRE(view_is(proven_u8str_as_view(&text), "visited=yes"), "the server saw the cookie");
    proven_http_client_finish(&resp);

    /* A page behind a login. The first answer is a 401 with a challenge; the client answers
     * it with the configured credentials and you see only the result. */
    EXAMPLE_REQUIRE(proven_u8str_reset(&text) == PROVEN_OK, "reuse the string");
    err = proven_http_client_get(client, url_for(at.port, "/private"), &resp);
    EXAMPLE_REQUIRE(err == PROVEN_OK && resp.status == 200 && proven_http_client_read_all(&resp, heap, &text,
1024) == PROVEN_OK, "let in");
    EXAMPLE_REQUIRE(view_is(proven_u8str_as_view(&text), "the private page"), "the page");
    proven_http_client_finish(&resp);

    /* A request with everything set: method, extra headers, and a body from a stream, which
     * is sent chunked - so a file or a pipe can be uploaded without knowing its size. */
    proven_reader_view_t source;
    proven_http_header_t headers[1] = { { PROVEN_LIT("Content-Type"), PROVEN_LIT("text/plain") } };
    proven_http_client_request_t req = {
        .method = PROVEN_LIT("POST"),
        .url = url_for(at.port, "/count"),
        .headers = headers,
        .header_count = 1,
        .body_stream = proven_reader_from_view(&source, PROVEN_LIT("nineteen bytes here")),
    };
    EXAMPLE_REQUIRE(proven_u8str_reset(&text) == PROVEN_OK, "reuse the string");
    err = proven_http_client_send(client, &req, &resp);
    EXAMPLE_REQUIRE(err == PROVEN_OK && proven_http_client_read_all(&resp, heap, &text, 1024) == PROVEN_OK, "a
POST from a stream");
    EXAMPLE_REQUIRE(view_is(proven_u8str_as_view(&text), "19 bytes"), "the server counted them");
    proven_http_client_finish(&resp);

    /* An error status is a response, not an error: the call succeeds and you look at status. */
    err = proven_http_client_get(client, url_for(at.port, "/missing"), &resp);
    EXAMPLE_REQUIRE(err == PROVEN_OK && resp.status == 404, "a 404 arrives as a response");
    proven_http_client_finish(&resp);

    /* An error is when there is no response to look at. */
    err = proven_http_client_get(client, PROVEN_LIT("https://127.0.0.1/"), &resp);
    EXAMPLE_REQUIRE(err == PROVEN_ERR_UNSUPPORTED, "https without a tls_wrap: the client says so rather than
sending in the clear");
    proven_http_client_finish(&resp);         /* harmless after a failure */

    proven_u8str_destroy(heap, &text);
    proven_http_client_destroy(client);
    proven_http_cookie_jar_destroy(&jar);

    proven_http_server_stop(g_server);
    proven_job_group_wait(threads, &running);
    proven_http_server_destroy(g_server);
    proven_job_system_close(threads);
    proven_job_system_destroy(threads);
    return EXAMPLE_OK();
}
```

HTTP/3, when a request asks for it

Since v0.97.0 (RFC-0034) a client made with `http3`, and a `tls` configuration that offers `h3`, sends a request that sets `http3` in `proven_http_client_request_t` over HTTP/3 - an `https` URL, over QUIC. Nothing else changes for the caller: `send` returns at the head, `read` gives the body, `finish` releases it. HTTP/3 cannot go through `tls_wrap` - QUIC is TLS inside UDP packets, not TLS over a connected socket - so underneath is the event-driven client of Chapter 15 on a loop of this client's own. The calling thread runs that loop while it waits: in `send` until the head (bounded by `connect_timeout_ms` and `io_timeout_ms` together, as a connection may have to be made first), in `read` until body bytes (`io_timeout_ms`). No thread is made, and between calls nothing runs it.

- **The same rules above the exchange.** Redirects, challenges, cookies, `User-Agent` and decoding are decided as for HTTP/1.1. A redirect keeps HTTP/3 within the origin the caller named; to another origin it is an ordinary request - HTTP/1.1 through `tls_wrap`, or the 3xx is returned when there is none. The client never picks HTTP/3 by itself.
- **The connection is kept** between calls in the event client, one QUIC connection per origin; a server that let it go meanwhile (nobody answered its packets) is reached on a new one.
- **Each resolved address in turn** while one gives no QUIC answer, as the TCP connect tries each.
- **Not over HTTP/3:** a proxy (there is no `CONNECT-UDP`) and `upgrade`, each `PROVEN_ERR_UNSUPPORTED`; `expect_continue` - the body goes at once. HTTP/2 is not offered here.
- `http3_fallback` sends such a request over TCP when its QUIC connection does not come up, as Chapter 17 section 10 describes for the event client.

11. Changing protocol

A request may ask that the connection stop carrying HTTP: `Connection: Upgrade` and `Upgrade: <protocol>`. If the server agrees it answers `101 Switching Protocols`, and every byte after that belongs to the other protocol. Both drivers can hand the connection over at that point.

API	Intent	Return
`proven_http_exchange_upgrade(x, protocol, headers, count, &transport, &early)`	In a handler: answer `101` with `Upgrade: protocol` and `Connection: Upgrade`, and take the connection out of the server.	`proven_err_t`: `INVALID_STATE` when a response was begun, the request has an unread body, or it is HTTP/1.0; `INVALID_ARG` for a header the server owns (`Upgrade` included); `NOMEM`; `OUT_OF_BOUNDS`; `TIMEOUT`, `RESET`.
`request.upgrade`	In a client request: the protocol to ask for. The client writes `Connection: Upgrade` and `Upgrade` itself.	-
`proven_http_client_upgrade(&response, &transport, &early)`	After a `101`: take the connection out of the client.	`proven_err_t`: `INVALID_STATE` when the response is not a `101` or was already taken.

Either way you receive a `proven_transport_t` that **owns** the connection. It is not the server's or the client's any more: it does not count against `max_connections`, none of their timeouts apply to it, it will not be reused, and it stays open until you call `proven_transport_close`. A server handler may keep it after it returns.

`early` **is the start of the other protocol.** A peer need not wait for the handshake to finish before sending, so the first bytes of the new protocol can arrive in the same read as the HTTP head. They are in `early` - a view that dies with the exchange, or at `proven_http_client_finish` - and a program that ignores them loses the beginning of the conversation, on some connections and not others.

Check before you agree. `proven_http_exchange_upgrade` writes the `101`; it does not judge whether the request was a well-formed request for the protocol you mean. For WebSocket that check, and the reply header it needs, are `proven_ws_check_request` and `proven_ws_accept_key`

- or use Chapter 12's `proven_ws_conn_accept`, which does all of it.

After the hand-over, the limits are yours. The server's promise that every wait is bounded ends at the `101`. A read on the transport with `PROVEN_NET_NO_DEADLINE` waits for ever.

Compiled and run by the test suite:

```
/*
 * Changing protocol: a request asks to stop speaking HTTP on this connection, the server
 * agrees with a 101, and from then on the same connection carries something else.
 *
 * The "something else" here is a toy - lines of text, sent back in capitals - because the
 * point is the hand-over. WebSocket is the protocol people usually mean, and chapter 12's
 * connection does these steps for you.
 */

static proven_http_server_t *g_server;

static void handle(void *ctx, proven_http_exchange_t *x) {
    (void)ctx;
    const proven_http_request_t *request = proven_http_exchange_request(x);
    /* Is this a request to change to OUR protocol? Checking is the handler's job. */
    if (!proven_http_header_has_token(request->headers, request->header_count, PROVEN_LIT("Upgrade"),
PROVEN_LIT("shout")) ||
        !proven_http_header_has_token(request->headers, request->header_count, PROVEN_LIT("Connection"),
PROVEN_LIT("Upgrade"))) {
        (void)proven_http_exchange_respond(x, 426, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("this address
speaks only shout\n")));
        return;
    }

    /* The server writes the 101, and the connection is ours: a transport that owns the socket. */
    proven_transport_t t;
    proven_mem_view_t early;                /* bytes the client sent without waiting for the 101 */
    if (proven_http_exchange_upgrade(x, PROVEN_LIT("shout"), NULL, 0, &t, &early) != PROVEN_OK) return;

    /* From here on this is not HTTP and none of the server's limits apply: every wait needs
     * a deadline of our own. `early` dies when the handler returns, so use it first. */
    proven_byte_t line[64];
    proven_size_t n = early.size < sizeof line ? early.size : sizeof line;
    for (proven_size_t i = 0; i < n; ++i) line[i] = early.ptr[i];
    while (n < sizeof line && (n == 0 || line[n - 1] != '\n')) {
        proven_result_size_t got = proven_transport_read(t, (proven_mem_mut_t){ line + n, sizeof line - n },
proven_net_deadline_in(5000));
        if (got.err != PROVEN_OK) break;
        n += got.value;
    }
    for (proven_size_t i = 0; i < n; ++i) if (line[i] >= 'a' && line[i] <= 'z') line[i] = (proven_byte_t)
(line[i] - 32);
    (void)proven_transport_write_all(t, (proven_mem_view_t){ line, n }, proven_net_deadline_in(5000));
    /* It could also be kept and used after the handler returns. Either way, closing it is ours. */
    (void)proven_transport_close(t);
}

static void serve(void *arg) {
    (void)arg;
    (void)proven_http_server_run(g_server);
```

```
}

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    proven_http_server_config_t server_config = {0};
    server_config.alloc = heap;
    server_config.handler = handle;
    EXAMPLE_REQUIRE(proven_http_server_create(&server_config, &g_server) == PROVEN_OK, "a server");
    proven_net_addr_t at;
    proven_err_t err = proven_http_server_listen(g_server, proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4,
0), &at);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        proven_http_server_destroy(g_server);
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK, "it listens");
    proven_job_sys_t *threads = NULL;
    proven_job_group_t running;
    proven_job_group_init(&running);
    EXAMPLE_REQUIRE(proven_job_system_init(heap, 1, 4, &threads) == PROVEN_OK &&
                    proven_job_group_submit(threads, &running, serve, NULL) == PROVEN_OK, "its loop runs on
another thread");

    proven_http_client_config_t client_config = { .alloc = heap, .max_idle_connections = 2 };
    proven_http_client_t *client = NULL;
    EXAMPLE_REQUIRE(proven_http_client_create(&client_config, &client) == PROVEN_OK, "a client");
    char url[64];
    int url_len = snprintf(url, sizeof url, "http://127.0.0.1:%u/", (unsigned)at.port);

    /* `upgrade` names the protocol; the client writes `Connection: Upgrade` and `Upgrade`. */
    proven_http_client_request_t request = { .url = { (const proven_byte_t *)url,
(proven_size_t)url_len }, .upgrade = PROVEN_LIT("shout") };
    proven_http_client_response_t response;
    err = proven_http_client_send(client, &request, &response);
    EXAMPLE_REQUIRE(err == PROVEN_OK && response.status == 101, "the server agreed: 101 Switching Protocols");

    /* A 101 is a response like any other until you take its connection. */
    proven_transport_t t;
    proven_mem_view_t early;
    EXAMPLE_REQUIRE(proven_http_client_upgrade(&response, &t, &early) == PROVEN_OK, "the connection is ours
now");
    proven_http_client_finish(&response);       /* still required; it no longer touches the connection */

    proven_byte_t reply[16];
    proven_size_t have = 0;
    EXAMPLE_REQUIRE(proven_transport_write_all(t, proven_mem_view_from_u8(PROVEN_LIT("hello\n")),
proven_net_deadline_in(5000)).err == PROVEN_OK, "a line, in the new protocol");
    while (have < 6) {
        proven_result_size_t got = proven_transport_read(t, (proven_mem_mut_t){ reply + have, sizeof reply -
have }, proven_net_deadline_in(5000));
        if (got.err != PROVEN_OK) break;
        have += got.value;
    }
    EXAMPLE_REQUIRE(have == 6 && proven_u8str_view_eq((proven_u8str_view_t){ reply, 6 },
PROVEN_LIT("HELLO\n")), "and the answer, in capitals");
    (void)proven_transport_close(t);            /* closes the socket and frees the transport */

    /* Without `upgrade` the same address answers as HTTP, and there is nothing to take. */
    err = proven_http_client_get(client, (proven_u8str_view_t){ (const proven_byte_t *)url,
```

```
(proven_size_t)url_len }, &response);
    EXAMPLE_REQUIRE(err == PROVEN_OK && response.status == 426, "an ordinary request is told what this address
speaks");
    EXAMPLE_REQUIRE(proven_http_client_upgrade(&response, &t, &early) == PROVEN_ERR_INVALID_STATE, "a response
that is not a 101 has no connection to give");
    proven_http_client_finish(&response);

    proven_http_client_destroy(client);
    proven_http_server_stop(g_server);
    proven_job_group_wait(threads, &running);
    proven_http_server_destroy(g_server);
    proven_job_system_close(threads);
    proven_job_system_destroy(threads);
    return EXAMPLE_OK();
}
```

12. What is not here

- **TLS**, in both directions. Section 1.
- **HTTP/2**, and **HTTP/3 except for a request that asks** (section 10).
- **Compression, unless asked for.** With nothing set, no `Accept-Encoding` is sent, no content coding is decoded, and no response is compressed. Setting `decompress` in the client's configuration, and calling `proven_http_exchange_compress` in a handler, turn on gzip in each direction (and Zstandard with `decompress_zstd` in the client and `compress_zstd` in the server; Brotli, in the client only, with `decompress_brotli`): Chapter 16, section 6.
- **WebSocket** is not in these two headers: it is Chapter 12, built on section 11.
- **A router, static files, sessions, middleware** on the server; **retries, caching, a connection limit per host** on the client.
- **Trailers**, in either direction.
- `Expect: 100-continue` **in the event-driven client.** `expect_continue` is the blocking client's; the client of Chapter 15 sends a body without asking.
- **Proxy auto-configuration, address ranges in** `no_proxy`**, and Digest authentication to a proxy.**

Chapter 12: WebSocket

Part V - Talking to the operating system. Prerequisites: Chapter 11 (the HTTP client and server, and its section on changing protocol); Chapter 10 for the codec half alone. After this chapter you can open a WebSocket connection from either side, exchange text and binary messages, close it properly, and say what your program does when the other side breaks the rules - or you can take the frames apart yourself, with no socket in sight.

This chapter covers `ws.h` and `ws_conn.h`, and - in section 9 - compressed messages in both of them and in the event-driven server of Chapter 15. `ws.h` is pure text-and-bytes handling, available in a freestanding build and untouched by `PROVEN_NO_NET`. `ws_conn.h` needs sockets: it is hosted-only and `PROVEN_NO_NET` leaves it out.

Table of contents

1. What WebSocket is, in one page
2. A connection
3. Sending and receiving
4. Closing, and being closed on
5. Where a server's connections live
6. The codec: the handshake
7. The codec: frames
8. The codec: the decoder
9. Compressed messages
10. What is not here

1. What WebSocket is, in one page

HTTP is one question and one answer. WebSocket (RFC 6455) is what a connection becomes when both sides want to speak whenever they have something to say: after one HTTP request and a `101` response, the same TCP connection carries *messages* in both directions until one side closes it.

```
client                                          server
  ¦ GET /chat HTTP/1.1                            ¦
  ¦ Upgrade: websocket                            ¦
  ¦ Sec-WebSocket-Key: <16 random bytes>          ¦
  ¦-----------------------------------------------¦
  ¦            HTTP/1.1 101 Switching Protocols   ¦
  ¦            Sec-WebSocket-Accept: <the answer> ¦
  ¦<----------------------------------------------¦
  ¦  message  --->         <---  message          ¦      any number, either way, any time
  ¦  ping     --->         <---  pong             ¦
  ¦  close    --->         <---  close            ¦      then the connection ends
```

A message is **text** (UTF-8, and checked) or **binary** (any bytes). On the wire a message is one or more *frames*; a long or streamed message is split into fragments, and the receiver joins them. Three small *control* frames travel between them: **ping** and **pong**, to find out whether the other end is still there, and **close**, which carries a code and a reason.

Two rules look odd until you know why. A client **masks** every frame - XORs the payload with a random four-byte key - and a server never does. The mask hides nothing; it stops a script in a web page from choosing the exact bytes that cross the network, which was once enough to poison a caching proxy. And the handshake's key and answer are not authentication: they only prove that the server really speaks WebSocket and is not an HTTP server echoing things back.

`ws://` is not protected. A `ws://` connection is readable and changeable by anyone on the path, exactly as Chapter 11 says of `http://`. `wss://` is the same protocol over TLS: give the HTTP client the

TLS wrap, or the HTTP server a TLS configuration (Chapter 14), and nothing else in this chapter changes.

2. A connection

```
/* a client */
proven_ws_conn_config_t config = { .alloc = proven_heap_allocator() };
proven_ws_conn_t *ws;
if (proven_ws_conn_connect(http_client, PROVEN_LIT("ws://example.com/feed"), NULL, 0,
                           PROVEN_LIT(""), &config, &ws, NULL) == PROVEN_OK) {
    proven_ws_conn_send_text(ws, PROVEN_LIT("hello"));
    ...
    proven_ws_conn_close(ws, PROVEN_WS_CLOSE_NORMAL, PROVEN_LIT(""));
    proven_ws_conn_destroy(ws);
}
```

Reference

API	Intent	Return
`proven_ws_conn_connect(client, url, headers, count, protocols, &config, &ws, &http_status)`	Connect as a client, through an HTTP client - so its proxy, timeouts and `tls_wrap` apply. `url` is `ws://` or `wss://`. `http_status` (optional) receives what the server answered.	`proven_err_t`: `REFUSED` when the answer was not `101` (see `http_status`); `INVALID_FORMAT` when the `101` did not answer this handshake; `UNSUPPORTED` for `wss` without `tls_wrap`; `IO` when no random bytes could be drawn; the errors of `proven_http_client_send`.
`proven_ws_conn_accept(x, protocol, &config, &ws)`	Accept as a server, inside an HTTP handler: check the request, answer `101`, take the connection. `protocol` is the subprotocol to select, or empty.	`proven_err_t`: `NOT_FOUND` - not an upgrade, nothing sent, answer it as HTTP; `UNSUPPORTED` - another version, a `426` was sent; `INVALID_FORMAT` - a malformed upgrade, a `400` was sent; `INVALID_ARG` - a `protocol` the client did not offer, nothing sent.
`proven_ws_conn_open(transport, is_server, early, &config, &ws)`	Make a connection over a transport already past its handshake. On success it owns the transport.	`proven_err_t`: `INVALID_ARG`; `NOMEM`; `OUT_OF_BOUNDS` when `early` exceeds 16 KiB; `IO`.
`proven_ws_conn_destroy(ws)`	Close the transport and free the connection.	none.
`proven_ws_conn_protocol(ws)`	The subprotocol agreed; empty when none.	`proven_u8str_view_t`.

```
typedef struct {
    proven_allocator_t alloc;           /* required */
    proven_size_t max_message_bytes;    /* 0: 1 MiB - the largest message accepted */
    proven_u32 io_timeout_ms;           /* 0: 30 s - for each write */
    proven_u32 close_timeout_ms;        /* 0: 5 s - how long close waits for the peer */
} proven_ws_conn_config_t;
```

A connection costs about 21 KiB when it is made - a 16 KiB read buffer and a 4 KiB send buffer

- plus the largest message it has had to assemble, which is kept for the next one and is never

more than `max_message_bytes`.

Cautions, and what goes wrong

A connection is for one thread at a time. Not even "one thread sends while another receives": receiving answers pings, which is a send. Two threads on one connection will interleave the bytes of two frames, and the peer will close with a protocol error that points nowhere near the cause. Give the connection to one thread. One lock around every call keeps the bytes apart, but `proven_ws_conn_receive` holds it for as long as it waits, and every send waits with it - so a program in which one thread waits for messages while another sends (a chat client) wants the loop's connection instead: Chapter 15, section 11, "Two threads, one connection", has the program.

`max_message_bytes` is how much a peer can make you hold. A message is assembled in memory before you see it. The limit is the most one connection will allocate for that, whatever is sent; a message past it ends the connection with code 1009. Set it for the messages you expect, not for the largest you can imagine.

Subprotocols are an agreement, not a decoration. The client offers names; the server picks one of them or none. `proven_ws_conn_accept` refuses a name the client did not offer, and `proven_ws_conn_connect` refuses a server that picks one. If your protocol has versions, this is where the two sides find out which one they share - check `proven_ws_conn_protocol` and do not assume.

The `Origin` header is the server's to check. A browser sends `Origin` with every WebSocket handshake and does *not* apply the same-origin policy to it: any web page can open a WebSocket to your server, and the browser will attach the user's cookies. If the server is reachable from browsers and uses cookies, compare `Origin` with the sites you expect before accepting, or any page the user visits can act as them.

`proven_ws_check_origin(&request, allowed, count)` is that comparison. An origin is written as a browser writes it - `https://app.example`, the port only when it is not the scheme's own, no path, no final slash - and is compared whole, letter case aside.

Result	Meaning	What a server does
`PROVEN_OK`	One `Origin`, and it is in the list.	Accepts.
`PROVEN_ERR_NOT_FOUND`	No `Origin`: the client is not a browser.	Decides - a program that is not a browser carries no user's cookies, and could have written any `Origin` it liked.
`PROVEN_ERR_PERMISSION`	An origin that is not in the list (`null` among them), or two `Origin` fields.	Answers `403`.

It works on the parsed request, so it is the same call in a handler of the blocking server, in `on_request` of the event-driven one, and with the codec alone.

3. Sending and receiving

API	Intent	Return
`proven_ws_conn_send_text(ws, text)`	Send a text message.	`proven_err_t`: `INVALID_ENCODING` when it is not UTF-8 - nothing is sent; `INVALID_STATE` after a close or inside a message begun with `_send_part`; `TIMEOUT`, `RESET`.

`proven_ws_conn_send_binary(ws, data)`	Send a binary message.	the same, without the encoding check.
`proven_ws_conn_send_part(ws, text, data, last)`	Send one fragment of a message; `last` ends it. `text` is read on the first call.	the same. Text sent this way is **not** checked.
`proven_ws_conn_ping(ws, data)`	Send a ping, with at most 125 bytes.	`proven_err_t`: `INVALID_ARG` for more.
`proven_ws_conn_receive(ws, until, &message)`	Wait for the next whole message, or until `until`.	`proven_err_t`: `TIMEOUT` - nothing lost, call again; `EOF` - the peer closed; `RESET` - the connection dropped; `INVALID_FORMAT`, `INVALID_ENCODING`, `OUT_OF_BOUNDS` - the peer broke a rule and has been told.
`proven_ws_conn_pong_count(ws)`	How many pongs have arrived.	`proven_u64`.

```
typedef struct {
    bool text;                  /* text (UTF-8, checked) rather than binary */
    proven_mem_view_t data;     /* owned by the connection; good until the next receive */
} proven_ws_message_t;
```

`proven_ws_conn_receive` does the protocol's housekeeping while it waits: a **ping is answered** with its pong and you never see it; a **pong is counted**; **fragments are joined**, with control frames between them handled where they arrive. What it returns is always a whole message.

Cautions, and what goes wrong

A refusal takes up to a second. When the peer breaks a rule the connection sends its close and then reads away what the peer is still sending, for at most a second, before `receive` returns - otherwise closing the socket would reset the connection and destroy the close before the peer read it.

`PROVEN_ERR_TIMEOUT` **is the only error you can receive after.** It means no complete message arrived yet; whatever part of one did arrive is kept, and the next call continues. Every other result ends the connection: the same error comes back from every later call, and what is left to do is `proven_ws_conn_destroy`.

A message's bytes are good until the next receive. `message.data` points into memory the connection reuses. Copy what you keep.

Wrong:

```
proven_ws_conn_receive(ws, until, &first);
proven_ws_conn_receive(ws, until, &second);
use(first.data);                    /* wrong: first.data now holds the second message, or freed memory */
```

Correct - use or copy `first.data` before receiving again.

Pings are only answered while you are receiving. The connection has no thread of its own. A program that sends for a long time without calling `proven_ws_conn_receive` answers no pings, and a peer that uses pings to detect dead connections will conclude this one is dead. If you send a great deal, receive now and then - with `PROVEN_NET_DONT_WAIT` if there may be nothing.

Nothing sends pings for you either. To find a silent connection that has died - a laptop that closed, a NAT that forgot - send `proven_ws_conn_ping`, receive for as long as you are prepared to wait,

and see whether `proven_ws_conn_pong_count` moved. A TCP connection whose other end vanished without a word can otherwise look open for hours.

`_send_part` **does not check text.** `proven_ws_conn_send_text` refuses text that is not UTF-8, because the peer must drop the connection when it receives such text. Fragments cannot be checked one by one - a character may be cut between two of them - so there the pieces *together* must be UTF-8, and making sure is yours.

A write that times out ends the connection. `io_timeout_ms` bounds each write. When it expires part of a frame may have gone, and there is no way to continue from the middle of a frame: the connection is finished.

4. Closing, and being closed on

API	Intent	Return
`proven_ws_conn_close(ws, code, reason)`	Send a close frame and wait for the peer's. Messages that arrive meanwhile are discarded.	`proven_err_t`: `PROVEN_OK` when both sides have closed; `TIMEOUT` when the peer did not answer in `close_timeout_ms`; `INVALID_ARG` for a code or reason that may not be sent.
`proven_ws_conn_close_code(ws)`	The peer's close code; 1006 when the connection ended without one; 0 while open.	`proven_u16`.
`proven_ws_conn_close_reason(ws)`	The reason that came with it.	`proven_u8str_view_t`.

Closing is a small handshake: one side sends a close frame, the other answers with one, and then the TCP connection ends. Skipping it - destroying a connection that was never closed - works, and the peer sees code 1006, "abnormal closure", which is also what it sees when the network fails. A peer that logs or retries on 1006 will do so. Close first.

Code	Meaning	Sent by
1000	Normal: the purpose is fulfilled.	you
1001	Going away: the server is shutting down, the page is being left.	you
1002	Protocol error.	the connection, for a frame the protocol forbids
1003	Unsupported data: text where binary was wanted, or the reverse.	you
1007	Invalid data: text that is not UTF-8.	the connection
1008	Policy violation.	you
1009	Message too big.	the connection, past `max_message_bytes`
1011	Internal error.	you
3000-4999	Yours and your application's to define.	you
1005, 1006	Never sent: "no code was given" and "no close frame came".	nobody; you only read them

When the peer closes, `receive` **returns** `PROVEN_ERR_EOF` **and the close has been answered.** Read the code and the reason if you want them, then destroy. Nothing more can be sent.

The reason is for a person reading a log. At most 123 bytes of UTF-8. Do not put anything in it that a program must parse, and remember it crosses the network readable.

A close does not flush the other direction. Messages the peer sent before it saw your close are discarded by `proven_ws_conn_close`. If you need them, stop sending and keep receiving until the peer closes, rather than closing first.

5. Where a server's connections live

`proven_ws_conn_accept` takes the connection out of the HTTP server (Chapter 11, section 11). From that moment the server's loop, its connection limit and its timeouts have nothing to do with it, and the question is which thread will call `proven_ws_conn_receive`.

The handler hands it off and returns. This is the shape that works with either way of running handlers, and the one the example uses: give the connection to a thread or a job of your own, and return. The server goes on serving.

The handler stays and talks. Simple to write. With handlers on a job system it occupies one worker for as long as the connection lasts - so the number of workers is the number of WebSocket connections you can have, and the next client's handshake waits in the queue until one closes. With handlers on the loop's thread it stops the whole server for that time, which is almost never what was meant.

Either way **a WebSocket connection costs a thread here.** `proven_ws_conn_receive` waits on one connection; there is no call that waits on many. A hundred connections are a hundred threads blocked in receive. That is a workable design for tens or hundreds of connections and the wrong one for tens of thousands - for which you would drive the codec of sections 6-8 from a readiness loop of your own (`proven_net_poll`, Chapter 9), one decoder per connection.

No limit is applied for you. The HTTP server's `max_connections` stops counting a connection when it is upgraded. If clients can open WebSockets without bound, count them yourself and answer the handshake with a `503` when you have enough.

Compiled and run by the test suite:

```
/*
 * A WebSocket server and a client, in one program.
 *
 * The server is an ordinary HTTP server whose handler accepts the upgrade. The handler does not
 * stay to talk: it hands the connection to a worker and returns, so the server's loop is free
 * for the next request. The client connects through an HTTP client and from then on the two
 * ends are the same kind of thing.
 */

static proven_http_server_t *g_server;
static proven_job_sys_t *g_workers;

/* One connection's life, on a worker: echo each message until the peer closes. */
static void talk(void *arg) {
    proven_ws_conn_t *ws = arg;
    for (;;) {
        proven_ws_message_t msg;
        /* Pings are answered inside receive; a peer silent for 30 s is given up on. */
        proven_err_t err = proven_ws_conn_receive(ws, proven_net_deadline_in(30000), &msg);
        if (err != PROVEN_OK) break;        /* PROVEN_ERR_EOF: the peer closed, and was answered */
        err = msg.text ? proven_ws_conn_send_text(ws, (proven_u8str_view_t){ msg.data.ptr, msg.data.size })
                       : proven_ws_conn_send_binary(ws, msg.data);
        if (err != PROVEN_OK) break;
    }
    proven_ws_conn_destroy(ws);             /* always: it owns the socket now */
```

```
}

static void handle(void *ctx, proven_http_exchange_t *x) {
    (void)ctx;
    proven_ws_conn_config_t config = { .alloc = proven_heap_allocator(), .max_message_bytes = 64 * 1024 };
    proven_ws_conn_t *ws = NULL;
    /* Select a subprotocol only if the client offered it. */
    const proven_http_request_t *request = proven_http_exchange_request(x);
    proven_u8str_view_t protocol = proven_ws_request_offers(request, PROVEN_LIT("echo.v1")) ?
PROVEN_LIT("echo.v1") : PROVEN_LIT("");

    proven_err_t err = proven_ws_conn_accept(x, protocol, &config, &ws);
    if (err == PROVEN_ERR_NOT_FOUND) {
        /* Not an upgrade at all - an ordinary request to this address. Nothing was sent. */
        (void)proven_http_exchange_respond(x, 200, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("this is a
WebSocket endpoint\n")));
        return;
    }
    if (err != PROVEN_OK) return;           /* a bad handshake was already answered 400 or 426 */

    /* The connection no longer belongs to the server. Give it a thread and return. */
    if (proven_job_submit_ex(g_workers, talk, ws) != PROVEN_OK) proven_ws_conn_destroy(ws);
}

static void serve(void *arg) {
    (void)arg;
    (void)proven_http_server_run(g_server);
}

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();

    // ---- the server ----------------------------------------------------------
    proven_http_server_config_t server_config = {0};
    server_config.alloc = heap;
    server_config.handler = handle;
    EXAMPLE_REQUIRE(proven_http_server_create(&server_config, &g_server) == PROVEN_OK, "a server");
    proven_net_addr_t at;
    proven_err_t err = proven_http_server_listen(g_server, proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4,
0), &at);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        proven_http_server_destroy(g_server);
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK, "it listens");
    /* One thread for the server's loop, and workers for connections: each open WebSocket
     * occupies one for as long as it lasts. */
    proven_job_sys_t *loop_thread = NULL;
    proven_job_group_t running;
    proven_job_group_init(&running);
    EXAMPLE_REQUIRE(proven_job_system_init(heap, 1, 4, &loop_thread) == PROVEN_OK &&
                    proven_job_system_init(heap, 4, 16, &g_workers) == PROVEN_OK &&
                    proven_job_group_submit(loop_thread, &running, serve, NULL) == PROVEN_OK, "the loop and
the workers are started");

    // ---- the client ----------------------------------------------------------
    proven_http_client_config_t http_config = { .alloc = heap };
    proven_http_client_t *http = NULL;
    EXAMPLE_REQUIRE(proven_http_client_create(&http_config, &http) == PROVEN_OK, "an HTTP client to connect
```

```
through");

    char url[64];
    int n = snprintf(url, sizeof url, "ws://127.0.0.1:%u/talk", (unsigned)at.port);
    proven_ws_conn_config_t config = { .alloc = heap };
    proven_ws_conn_t *ws = NULL;
    proven_u16 status = 0;
    err = proven_ws_conn_connect(http, (proven_u8str_view_t){ (const proven_byte_t *)url, (proven_size_t)n },
NULL, 0,
                                 PROVEN_LIT("echo.v1, echo.v0"), &config, &ws, &status);
    EXAMPLE_REQUIRE(err == PROVEN_OK && status == 101, "connected: the server answered 101 and the answer
checked out");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_ws_conn_protocol(ws), PROVEN_LIT("echo.v1")), "the subprotocol
the server chose");
    /* The connection is its own thing now; the HTTP client is not needed to use it. */
    proven_http_client_destroy(http);

    proven_ws_message_t msg;
    EXAMPLE_REQUIRE(proven_ws_conn_send_text(ws, PROVEN_LIT("hello")) == PROVEN_OK, "a text message");
    EXAMPLE_REQUIRE(proven_ws_conn_receive(ws, proven_net_deadline_in(5000), &msg) == PROVEN_OK && msg.text &&
                    proven_u8str_view_eq((proven_u8str_view_t){ msg.data.ptr, msg.data.size },
PROVEN_LIT("hello")), "comes back, whole");

    proven_byte_t bytes[4] = { 0x00, 0xff, 0x10, 0x80 };
    EXAMPLE_REQUIRE(proven_ws_conn_send_binary(ws, (proven_mem_view_t){ bytes, sizeof bytes }) == PROVEN_OK &&
                    proven_ws_conn_receive(ws, proven_net_deadline_in(5000), &msg) == PROVEN_OK && !msg.text
&& msg.data.size == 4, "binary is any bytes");

    /* Text is checked before it is sent: the peer would have to drop the connection over this. */
    EXAMPLE_REQUIRE(proven_ws_conn_send_text(ws, PROVEN_LIT("not \xff text")) == PROVEN_ERR_INVALID_ENCODING,
"not UTF-8: refused here, nothing sent");

    /* A message produced in pieces goes out in fragments; the other end still sees one message. */
    EXAMPLE_REQUIRE(proven_ws_conn_send_part(ws, true, proven_mem_view_from_u8(PROVEN_LIT("one, ")), false) ==
PROVEN_OK &&
                    proven_ws_conn_send_part(ws, true, proven_mem_view_from_u8(PROVEN_LIT("two, ")), false) ==
PROVEN_OK &&
                    proven_ws_conn_send_part(ws, true, proven_mem_view_from_u8(PROVEN_LIT("three")), true) ==
PROVEN_OK, "three fragments");
    EXAMPLE_REQUIRE(proven_ws_conn_receive(ws, proven_net_deadline_in(5000), &msg) == PROVEN_OK &&
msg.data.size == 15, "one message of fifteen bytes");

    /* Is the other end alive? Ping, then receive for a while: the pong is counted on the way. */
    EXAMPLE_REQUIRE(proven_ws_conn_ping(ws, proven_mem_view_from_u8(PROVEN_LIT("still there?"))) == PROVEN_OK,
"a ping");
    err = proven_ws_conn_receive(ws, proven_net_deadline_in(200), &msg);
    EXAMPLE_REQUIRE(err == PROVEN_ERR_TIMEOUT && proven_ws_conn_pong_count(ws) == 1, "no message came, but the
pong did: the peer is alive");

    /* Closing is a handshake: our close, then the peer's. */
    EXAMPLE_REQUIRE(proven_ws_conn_close(ws, PROVEN_WS_CLOSE_NORMAL, PROVEN_LIT("done")) == PROVEN_OK,
"closed, both ways");
    EXAMPLE_REQUIRE(proven_ws_conn_close_code(ws) == PROVEN_WS_CLOSE_NORMAL &&
proven_ws_conn_close_reason(ws).size == 0, "the peer echoed the code, with no reason of its own");
    proven_ws_conn_destroy(ws);

    // ---- a connection over any transport -----------------------------------
    /* The handshake is HTTP's; what follows needs only a transport. Here: a socket pair. */
    proven_net_conn_t a, b;
    proven_ws_conn_t *left = NULL, *right = NULL;
```

```
    EXAMPLE_REQUIRE(proven_net_pair(&a, &b) == PROVEN_OK, "two joined sockets");
    EXAMPLE_REQUIRE(proven_ws_conn_open(proven_net_conn_transport(&a), false, (proven_mem_view_t){0}, &config,
&left) == PROVEN_OK &&
                    proven_ws_conn_open(proven_net_conn_transport(&b), true, (proven_mem_view_t){0}, &config,
&right) == PROVEN_OK, "a client end and a server end");
    EXAMPLE_REQUIRE(proven_ws_conn_send_text(left, PROVEN_LIT("no handshake needed")) == PROVEN_OK &&
                    proven_ws_conn_receive(right, proven_net_deadline_in(1000), &msg) == PROVEN_OK &&
msg.data.size == 19, "a message across");
    proven_ws_conn_destroy(left);
    proven_ws_conn_destroy(right);

    proven_http_server_stop(g_server);
    proven_job_group_wait(loop_thread, &running);
    proven_http_server_destroy(g_server);
    proven_job_system_close(g_workers);         /* waits for the worker that is finishing the conversation */
    proven_job_system_destroy(g_workers);
    proven_job_system_close(loop_thread);
    proven_job_system_destroy(loop_thread);
    return EXAMPLE_OK();
}
```

6. The codec: the handshake

`ws.h` is the protocol without a connection: you give it bytes and it tells you what they mean. Use it when the transport is not a socket this library opened, when you serve many connections from one loop, or on a target with no sockets at all.

API	Intent	Return
`proven_ws_make_key(random, out)`	A client's `Sec-WebSocket-Key` (24 characters) from 16 random bytes you supply.	none.
`proven_ws_accept_key(key, out)`	The `Sec-WebSocket-Accept` value (28 characters) that answers a key.	`proven_err_t`: `INVALID_FORMAT` unless the key is 24 characters of Base64 for 16 bytes.
`proven_ws_check_request(&request, &key)`	A server's check of a parsed request - also an HTTP/2 extended CONNECT as the event-driven server presents it (Chapter 15, section 11), whose `key` is empty.	`proven_err_t`: `PROVEN_OK` - answer 101 (over HTTP/2, 200: `proven_ws_event_accept` does either); `NOT_FOUND` - not a WebSocket upgrade; `UNSUPPORTED` - another version, answer 426 with `Sec-WebSocket-Version: 13`; `INVALID_FORMAT` - answer 400.
`proven_ws_request_offers(&request, name)`	Whether the request offers a subprotocol, compared exactly.	`bool`.
`proven_ws_check_origin(&request, allowed, count)`	Whether the request's `Origin` is one of the origins served (section 2).	`proven_err_t`: `PROVEN_OK`; `NOT_FOUND` - no `Origin`; `PERMISSION` - another origin, or two fields; `INVALID_ARG`.
`proven_ws_check_response(&response, key, offered, &protocol)`	A client's check of the answer: 101, the right accept value, a subprotocol that was offered, no extension.	`proven_err_t`: `INVALID_FORMAT` otherwise.

The accept value is Base64 of the SHA-1 of the key followed by a fixed string from the RFC. SHA-1 is broken as a signature and this use does not care: nothing here is a secret, and the value only has to be something an HTTP server that is *not* a WebSocket server would not produce by reflex. That is also all it proves. It does not authenticate the server and it does not authenticate the client; anyone can compute it.

A response that names an extension is refused by `proven_ws_check_response`: a client that offered none is being told of frames it did not ask for. A client that offered compression checks with `proven_ws_check_response_deflate` instead (section 9), which accepts that one extension in the form it was offered and nothing else.

7. The codec: frames

```
typedef struct {
    bool fin;                   /* the last frame of its message */
    proven_u8 opcode;           /* PROVEN_WS_TEXT, _BINARY, _CONTINUATION, _CLOSE, _PING, _PONG */
    bool masked;
    proven_byte_t mask[4];
    proven_u64 length;          /* payload bytes that follow the header */
} proven_ws_frame_t;
```

API	Intent	Return
`proven_ws_frame_parse(data, &frame, &header_size)`	Parse a frame header - 2 to 14 bytes - from the front of `data`.	`proven_err_t`: `NEED_MORE`; `INVALID_FORMAT` for a reserved bit (the first is reported in `rsv1` where a message begins: section 9), an undefined opcode, a length not in its shortest form, or a control frame that is fragmented or over 125 bytes.
`proven_ws_frame_write(out, &len, &frame)`	Append a frame header. The payload is yours to write after it.	`proven_err_t`: `INVALID_ARG` for a frame the parser would refuse; `OUT_OF_BOUNDS`.
`proven_ws_mask(data, mask, offset)`	Mask or unmask payload bytes in place. `offset` is where `data` starts within the payload.	none.
`proven_ws_close_write(out, &len, code, reason)`	Append a close frame's payload.	`proven_err_t`: `INVALID_ARG` for a code that may not be sent or a reason over 123 bytes; `INVALID_ENCODING`; `OUT_OF_BOUNDS`.
`proven_ws_close_parse(payload, &code, &reason)`	Read one. An empty payload is code 1005.	`proven_err_t`: `INVALID_FORMAT`; `INVALID_ENCODING`.
`proven_ws_close_code_is_valid(code)`	Whether a code may appear in a close frame.	`bool`.
`proven_ws_close_code_for(err)`	The close code that reports a decoder error: 1002, 1007, 1009, or 1011.	`proven_u16`.

A length has one spelling. Up to 125 it is in the second byte; up to 65535 in two more; beyond that in eight. `proven_ws_frame_write` always uses the shortest, and `proven_ws_frame_parse` refuses any other -

two encodings of one frame are how a filter and the program behind it come to disagree about where a frame ends.

A client must mask with a key the application cannot predict. Take the four bytes from `proven_random_bytes` or a generator seeded from it - a counter or a constant defeats the purpose. A server sets `masked` to false and writes the payload as it is.

The codec has no random source. Keys and masks are arguments, as the multipart boundary is in Chapter 10, so the same code runs where there is no operating system to ask.

8. The codec: the decoder

Parsing headers is the small part. What a receiver has to get right is the stream: frames arriving in arbitrary pieces, fragments to join, control frames between fragments, text to check across fragment boundaries, and a list of things to refuse. `proven_ws_decoder_t` is that.

API	Intent	Return
`proven_ws_decoder_init(&decoder, from_client, max_message_bytes)`	Begin. `from_client` true: every frame must be masked (a server's decoder). False: none may be.	none.
`proven_ws_decoder_feed(&decoder, in, &consumed, &event)`	Consume bytes of `in` until there is something to report or `in` is used up.	`proven_err_t`: `INVALID_FORMAT` (close 1002); `INVALID_ENCODING` (1007); `OUT_OF_BOUNDS` (1009); `EOF` for bytes after a close frame.

```
typedef struct {
    proven_ws_event_kind_t kind;   /* PROVEN_WS_EVENT_NONE, _DATA, _PING, _PONG, _CLOSE */
    bool text, first, last;        /* DATA: text or binary; the ends of the message */
    proven_mem_view_t data;        /* DATA: a piece, unmasked, inside `in`. PING, PONG: the payload. CLOSE:
the reason. */
    proven_u16 close_code;         /* CLOSE */
} proven_ws_event_t;
```

The loop is the one from Chapter 10's body decoder: feed what you read; act on the event; feed the rest; when the event is `NONE`, read more.

A message passes through without being held. A `DATA` event is a *piece* of a message, in the buffer you fed, unmasked where it lay. `first` and `last` mark the message's ends. A gigabyte message is so many events and no allocation - and joining the pieces, if you want them joined, is yours, with a limit of your choosing. (`ws_conn.h` joins them for you, up to `max_message_bytes`.)

`in` is written to. Unmasking happens in place, which is why `in` is not `const`. Do not feed memory you need unchanged.

What the decoder refuses, each with the error that names its close code:

The stream	Error
A frame from a client that is not masked, or one from a server that is	`INVALID_FORMAT`
A reserved bit (the first only where it may not be, or when compression was not agreed: section 9), an undefined opcode, a length not in its shortest form	`INVALID_FORMAT`
A control frame over 125 bytes, or fragmented	`INVALID_FORMAT`
A continuation with no message open; a new message inside an open one	`INVALID_FORMAT`
A close frame with a one-byte payload, or a code that may not be sent	`INVALID_FORMAT`
Text that is not UTF-8, checked across fragments; a close reason that is not	`INVALID_ENCODING`

A message, fragments together, past the limit	OUT_OF_BOUNDS

After any of these the decoder repeats the error. That is deliberate: once a frame is wrong, where the next one starts is not known, and guessing is how one bad frame becomes a different conversation.

Answering is yours. The decoder reports a ping; it does not send the pong. It reports a close; it does not send the answering close. It has no way to send anything - which is what lets it run anywhere.

Compiled and run by the test suite:

```
#include <string.h>

/*
 * The WebSocket codec by itself: the handshake's two values, a frame written and read, and a
 * decoder fed a stream in awkward pieces. No socket anywhere - the "network" is an array.
 */

int main(void) {
    // ---- The opening handshake ------------------------------------------------
    /* A client makes a key from sixteen random bytes; the server answers with a value only
     * something that speaks WebSocket would compute. These are the values of RFC 6455. */
    proven_byte_t accept[PROVEN_WS_ACCEPT_SIZE];
    EXAMPLE_REQUIRE(proven_ws_accept_key(PROVEN_LIT("dGhlIHNhbXBsZSBub25jZQ=="), accept) == PROVEN_OK &&
                    memcmp(accept, "s3pPLMBiTxaQ9kYGzzhZRbK+xOo=", sizeof accept) == 0, "the RFC's key and its
answer");

    proven_byte_t random[16], key[PROVEN_WS_KEY_SIZE];
    EXAMPLE_REQUIRE(proven_random_bytes(random, sizeof random), "sixteen random bytes");
    proven_ws_make_key(random, key);
    EXAMPLE_REQUIRE(proven_ws_accept_key((proven_u8str_view_t){ key, sizeof key }, accept) == PROVEN_OK, "a
key of our own, and its answer");

    /* A server looks at a parsed request and learns which of four things it is. */
    static const char upgrade[] = "GET /chat HTTP/1.1\r\nHost: example.com\r\nUpgrade:
websocket\r\nConnection: Upgrade\r\n"
                                  "Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==\r\nSec-WebSocket-Protocol:
chat, superchat\r\n"
                                  "Origin: http://example.com\r\nSec-WebSocket-Version: 13\r\n\r\n";
    proven_http_header_t fields[16];
    proven_http_request_t request;
    proven_size_t head = 0;
    proven_u8str_view_t client_key;
    EXAMPLE_REQUIRE(proven_http_parse_request((proven_mem_view_t){ (const proven_byte_t *)upgrade, sizeof
upgrade - 1 }, fields, 16, 0, &request, &head) == PROVEN_OK, "the request parses");
    EXAMPLE_REQUIRE(proven_ws_check_request(&request, &client_key) == PROVEN_OK, "it is a WebSocket upgrade:
answer 101");
    EXAMPLE_REQUIRE(proven_ws_request_offers(&request, PROVEN_LIT("superchat")) && !
proven_ws_request_offers(&request, PROVEN_LIT("mqtt")), "and it offers these subprotocols");
    /* A server that knows its users by a cookie checks where the page came from. */
    const proven_u8str_view_t served[] = { PROVEN_LIT("https://app.example"), PROVEN_LIT("http://example.
com") };
    EXAMPLE_REQUIRE(proven_ws_check_origin(&request, served, 2) == PROVEN_OK &&
proven_ws_check_origin(&request, served, 1) == PROVEN_ERR_PERMISSION,
                    "its Origin is one the server serves; were it not, the answer would be 403");

    /* The client checks the answer: the right accept value, a subprotocol it offered, nothing else. */
    static const char answer[] = "HTTP/1.1 101 Switching Protocols\r\nUpgrade: websocket\r\nConnection:
Upgrade\r\n"
```

```
                                "Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=\r\nSec-WebSocket-
Protocol: chat\r\n\r\n";
    proven_http_response_t response;
    proven_u8str_view_t chosen;
    EXAMPLE_REQUIRE(proven_http_parse_response((proven_mem_view_t){ (const proven_byte_t *)answer, sizeof
answer - 1 }, fields, 16, 0, &response, &head) == PROVEN_OK, "the response parses");
    EXAMPLE_REQUIRE(proven_ws_check_response(&response, client_key, PROVEN_LIT("chat, superchat"), &chosen) ==
PROVEN_OK &&
                    proven_u8str_view_eq(chosen, PROVEN_LIT("chat")), "it answers this handshake, and chose
chat");

    // ---- One frame ---------------------------------------------------------
    /* A client's text frame: a header, then the payload XORed with a four-byte key. */
    proven_byte_t wire[256];
    proven_mem_mut_t out = { wire, sizeof wire };
    proven_size_t len = 0;
    proven_ws_frame_t frame = { .fin = true, .opcode = PROVEN_WS_TEXT, .masked = true, .mask = { 0x37, 0xfa,
0x21, 0x3d }, .length = 5 };
    EXAMPLE_REQUIRE(proven_ws_frame_write(out, &len, &frame) == PROVEN_OK && len == 6, "a six-byte header");
    memcpy(wire + len, "Hello", 5);
    proven_ws_mask((proven_mem_mut_t){ wire + len, 5 }, frame.mask, 0);
    len += 5;
    EXAMPLE_REQUIRE(memcmp(wire, "\x81\x85\x37\xfa\x21\x3d\x7f\x9f\x4d\x51\x58", 11) == 0, "byte for byte the
masked Hello of RFC 6455");

    proven_ws_frame_t parsed;
    proven_size_t header_size = 0;
    EXAMPLE_REQUIRE(proven_ws_frame_parse((proven_mem_view_t){ wire, 3 }, &parsed, &header_size) ==
PROVEN_ERR_NEED_MORE, "half a header: read more");
    EXAMPLE_REQUIRE(proven_ws_frame_parse((proven_mem_view_t){ wire, len }, &parsed, &header_size) ==
PROVEN_OK &&
                    parsed.opcode == PROVEN_WS_TEXT && parsed.masked && parsed.length == 5 && header_size ==
6, "the header, read back");

    // ---- A stream, decoded -------------------------------------------------
    /* A text message in two fragments with a ping between them, then a close. */
    len = 0;
    frame = (proven_ws_frame_t){ .fin = false, .opcode = PROVEN_WS_TEXT, .length = 3 };
    EXAMPLE_REQUIRE(proven_ws_frame_write(out, &len, &frame) == PROVEN_OK, "first fragment");
    memcpy(wire + len, "Hel", 3); len += 3;
    frame = (proven_ws_frame_t){ .fin = true, .opcode = PROVEN_WS_PING, .length = 0 };
    EXAMPLE_REQUIRE(proven_ws_frame_write(out, &len, &frame) == PROVEN_OK, "a ping, in the middle of the
message");
    frame = (proven_ws_frame_t){ .fin = true, .opcode = PROVEN_WS_CONTINUATION, .length = 2 };
    EXAMPLE_REQUIRE(proven_ws_frame_write(out, &len, &frame) == PROVEN_OK, "last fragment");
    memcpy(wire + len, "lo", 2); len += 2;
    proven_byte_t close_payload[PROVEN_WS_MAX_CONTROL];
    proven_size_t close_len = 0;
    EXAMPLE_REQUIRE(proven_ws_close_write((proven_mem_mut_t){ close_payload, sizeof close_payload },
&close_len, PROVEN_WS_CLOSE_NORMAL, PROVEN_LIT("bye")) == PROVEN_OK, "a close payload");
    frame = (proven_ws_frame_t){ .fin = true, .opcode = PROVEN_WS_CLOSE, .length = close_len };
    EXAMPLE_REQUIRE(proven_ws_frame_write(out, &len, &frame) == PROVEN_OK, "the close frame");
    memcpy(wire + len, close_payload, close_len); len += close_len;

    /* A client decodes what a server sent: `false` - these frames must NOT be masked. The
     * stream arrives four bytes at a time; the decoder does not care. */
    proven_ws_decoder_t decoder;
    proven_ws_decoder_init(&decoder, false, 1024);
    char message[32];
    proven_size_t message_len = 0;
```

```
    int pings = 0;
    proven_u16 close_code = 0;
    for (proven_size_t pos = 0; pos < len;) {
        proven_size_t n = len - pos < 4 ? len - pos : 4;
        proven_mem_mut_t piece = { wire + pos, n };
        while (piece.size > 0) {
            proven_size_t used = 0;
            proven_ws_event_t ev;
            EXAMPLE_REQUIRE(proven_ws_decoder_feed(&decoder, piece, &used, &ev) == PROVEN_OK, "the stream
decodes");
            piece.ptr += used;
            piece.size -= used;
            if (ev.kind == PROVEN_WS_EVENT_DATA) {
                memcpy(message + message_len, ev.data.ptr, ev.data.size);      /* a piece of the message */
                message_len += ev.data.size;
            } else if (ev.kind == PROVEN_WS_EVENT_PING) {
                pings++;                                                       /* a real program answers with
a pong */
            } else if (ev.kind == PROVEN_WS_EVENT_CLOSE) {
                close_code = ev.close_code;
            }
        }
        pos += n;
    }
    EXAMPLE_REQUIRE(message_len == 5 && memcmp(message, "Hello", 5) == 0, "two fragments, one message");
    EXAMPLE_REQUIRE(pings == 1 && close_code == PROVEN_WS_CLOSE_NORMAL, "the ping between them, and the
close");

    // ---- What a receiver refuses -------------------------------------------
    /* A masked frame from a server breaks the protocol; the error names the close code to send. */
    proven_byte_t bad[] = { 0x81, 0x81, 1, 2, 3, 4, 'x' ^ 1 };
    proven_ws_event_t ev;
    proven_size_t used = 0;
    proven_ws_decoder_init(&decoder, false, 1024);
    proven_err_t err = proven_ws_decoder_feed(&decoder, (proven_mem_mut_t){ bad, sizeof bad }, &used, &ev);
    EXAMPLE_REQUIRE(err == PROVEN_ERR_INVALID_FORMAT && proven_ws_close_code_for(err) ==
PROVEN_WS_CLOSE_PROTOCOL_ERROR, "refused: close with 1002");

    /* A close frame's payload, read by itself. Code 1005 means "no code was given" and may
     * not itself be sent. */
    proven_u16 code = 0;
    proven_u8str_view_t reason;
    EXAMPLE_REQUIRE(proven_ws_close_parse((proven_mem_view_t){ close_payload, close_len }, &code, &reason) ==
PROVEN_OK &&
                    code == 1000 && proven_u8str_view_eq(reason, PROVEN_LIT("bye")), "code and reason");
    EXAMPLE_REQUIRE(proven_ws_close_code_is_valid(4000) && !
proven_ws_close_code_is_valid(PROVEN_WS_CLOSE_NO_STATUS), "which codes may travel");

    return EXAMPLE_OK();
}
```

9. Compressed messages

```
bool proven_ws_conn_compressed(const proven_ws_conn_t *conn);            /* ws_conn.h */
bool proven_ws_stream_compressed(const proven_ws_stream_t *ws);          /* ws_event.h, Chapter 15 */

#define PROVEN_WS_DEFLATE_OFFER "permessage-deflate; client_no_context_takeover; server_no_context_takeover"
proven_err_t proven_ws_deflate_accept(const proven_http_request_t *request, proven_mem_mut_t out,
proven_size_t *len, int *window_bits);
proven_err_t proven_ws_check_response_deflate(const proven_http_response_t *response, proven_u8str_view_t key,
```

```
                                                proven_u8str_view_t offered, proven_u8str_view_t *protocol, bool
*deflate);

#define PROVEN_WS_DEFLATE_OFFER_TAKEOVER "permessage-deflate; client_max_window_bits"
/* proven_ws_deflate_terms_t: bool server_takeover, client_takeover; int server_window_bits,
client_window_bits; */
proven_err_t proven_ws_deflate_accept_takeover(const proven_http_request_t *request, proven_mem_mut_t out,
proven_size_t *len,
                                               proven_ws_deflate_terms_t *terms);
proven_err_t proven_ws_check_response_takeover(const proven_http_response_t *response, proven_u8str_view_t
key,
                                               proven_u8str_view_t offered, proven_u8str_view_t *protocol,
bool *deflate,
                                               proven_ws_deflate_terms_t *terms);

#define PROVEN_WS_DEFLATE_OFFER_PEER_NO_TAKEOVER "permessage-deflate; client_max_window_bits;
server_no_context_takeover"
proven_err_t proven_ws_deflate_accept_peer_no_takeover(const proven_http_request_t *request, proven_mem_mut_t
out, proven_size_t *len,
                                                       proven_ws_deflate_terms_t *terms);
proven_err_t proven_ws_check_response_peer_no_takeover(const proven_http_response_t *response,
proven_u8str_view_t key,
                                                       proven_u8str_view_t offered, proven_u8str_view_t
*protocol, bool *deflate,
                                                       proven_ws_deflate_terms_t *terms);
void proven_inflate_restart(proven_inflate_t *z);                       /* deflate.h, Chapter 16 */
void proven_ws_decoder_allow_compressed(proven_ws_decoder_t *decoder);
```

A WebSocket connection can agree, in its handshake, to compress its messages: the extension is called `permessage-deflate` (RFC 7692), every browser offers it, and it is the DEFLATE of Chapter 16. A page of JSON or a batch of log lines shrinks to a third or less, and so - because each side keeps its window from one message to the next - do short messages that resemble the ones before them.

It is off until you ask. Set `compress` in the connection's configuration - in a `proven_ws_conn_config_t`, or in the `proven_ws_event_config_t` of Chapter 15. A client then offers the extension and a server accepts it when offered; `proven_ws_conn_compressed` and `proven_ws_stream_compressed` say whether the other side agreed. **Nothing else changes**: the same calls send and receive, and a connection whose peer did not agree simply goes on uncompressed.

	With compression agreed
A message you send	Compressed when it is 8 bytes or more (64 with `compress_no_takeover`); shorter ones go as they are, and stay out of the window
A message you receive	Inflated before you see it: whole from `proven_ws_conn_receive`, in pieces of at most 16 KiB from the event-driven server
`max_message_bytes`	Counts the **inflated** size. A few hundred bytes that unfold past it end the connection with 1009, as an oversized plain message does
Text	Checked to be UTF-8 after inflating; 1007 if it is not
Bytes that do not inflate	1007
A frame marked compressed on a connection that did not agree	1002

Each side keeps its window from one message to the next ("context takeover"): a message refers back to the messages before it. That is where small, similar messages gain - measured with this library's compressor over 2,000 messages each:

Messages	Each compressed on its own	With takeover (window 11, the default)	With takeover, window 15
Chat JSON, about 140 bytes	80.7% of their size	22.1%	17.3%
Quotes, about 60 bytes	100% (too short to compress)	39.3%	35.5%
Pings, about 25 bytes	100% (too short to compress)	26.1%	26.1%
Game state, about 750 bytes	30.4%	17.4%	16.7%

The price is memory **for the life of the connection**, from its first compressed message: a compressor (about 28 KiB at the default window of 11 bits; 324 KiB at 15) and a decompressor (about 35 KiB) - about 64 KiB for a connection that has sent and received one, measured (`test_unit_ws_event`). Time per message is about the same for short ones; a 750-byte one took 38 us against 24 compressed on its own.

`compress_no_takeover` in the configuration asks for the other form ("no context takeover", in both directions), which was the only one before v0.80.0: a compressor exists for the length of one send, a decompressor from the first frame of a compressed message to its last, and **a connection that is idle holds no compression state at all** - ten thousand quiet connections cost what they cost without compression. Either side may ask for it: a server with takeover still answers an offer that forbids it, and a client with takeover accepts an answer that forbids it. A peer that limits this side's window (`server_max_window_bits` in an offer, `client_max_window_bits` in an answer) is obeyed; one that limits it to 256 bytes, smaller than this compressor's smallest, gets this side's messages uncompressed.

`compress_peer_no_takeover` is the form between the two: **this side keeps its window, the other side must not**. Its messages to the other side keep their gain; it needs no decompressor between messages, so an idle connection that has sent and received a compressed message holds about 30 KiB where with takeover both ways it holds about 64 (measured, `test_unit_ws_event`). The other side's short, similar messages then shrink no more than they do on their own. A server answers `client_no_context_takeover` whether or not the offer had it, which RFC 7692 allows; a client offers `server_no_context_takeover`, and a server that will not honour it declines the extension, so the connection goes on uncompressed. A client that takes over anyway is closed with 1007: its messages do not inflate on their own.

`proven_ws_conn_open` makes no handshake: both ends must say the same - takeover, or `compress_no_takeover` (`compress_peer_no_takeover` is refused there with `PROVEN_ERR_INVALID_ARG`: it is an agreement only a handshake makes). An end built before v0.80.0 always means no takeover; give the other end `compress_no_takeover`, or its second compressed message is refused with 1007.

Example: one field, and what it changes

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/*
```

```
 * Compressed WebSocket messages, the way a program uses them: one field in a configuration.
 *
 * The server here is the event-driven one, the client the blocking one. Each sets `compress`;
 * the handshake agrees; after that every call is used exactly as before, and what crosses
 * the network is a fraction of what was said.
 */

typedef struct {
    proven_loop_t *loop;
    bool compressed;                    /* what the server found agreed */
    proven_size_t received, largest, pieces;
} ex_app_t;

/* A compressed message arrives decoded, in pieces of at most 16 KiB. Nothing holds the whole
 * of it unless you do. This handler counts, and answers when the message is complete. */
static void on_message(void *ctx, proven_ws_stream_t *ws, bool text, proven_mem_view_t piece, bool first, bool
last) {
    ex_app_t *app = ctx;
    (void)text;
    if (first) { app->received = 0; app->pieces = 0; app->largest = 0; }
    app->received += piece.size;
    app->pieces++;
    if (piece.size > app->largest) app->largest = piece.size;
    if (!last) return;
    char answer[64];
    int n = snprintf(answer, sizeof answer, "%u bytes", (unsigned)app->received);
    /* Sent like any message. Short ones go as they are: under 8 bytes there is nothing to gain
     * (under 64 with compress_no_takeover, where a message has only itself to refer to). */
    (void)proven_ws_stream_send(ws, true, (proven_mem_view_t){ (const proven_byte_t *)answer,
(proven_size_t)n });
}

static void on_request(void *ctx, proven_http_stream_t *stream, const proven_http_request_t *head) {
    ex_app_t *app = ctx;
    proven_ws_event_config_t config = {
        .on = { .on_message = on_message },
        .ctx = app,
        .compress = true,               /* accept the extension when the client offers it */
        .max_message_bytes = 256 * 1024,    /* the limit on a message AS DECODED: a few bytes cannot become a
gigabyte */
    };
    proven_ws_stream_t *ws = NULL;
    if (proven_ws_event_accept(stream, head, &config, &ws) != PROVEN_OK) {
        (void)proven_http_stream_respond(stream, 400, NULL, 0, (proven_mem_view_t){ 0 });
        return;
    }
    /* Whether it was agreed depends on the client too. Nothing else has to be done either way. */
    app->compressed = proven_ws_stream_compressed(ws);
    /* The same switch on this side, for a server whose answers must not be compressed:
     * proven_ws_stream_set_compress(ws, false). Here they may be. */
    proven_ws_stream_set_compress(ws, true);
}

static void run(void *arg) { (void)proven_loop_run(((ex_app_t *)arg)->loop); }

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    ex_app_t app = { 0 };
    EXAMPLE_REQUIRE(proven_loop_create(heap, &app.loop) == PROVEN_OK, "a loop");
    proven_http_event_server_config_t server_config = { .on = { .on_request = on_request }, .ctx = &app };
```

```
    proven_http_event_server_t *server = NULL;
    proven_net_addr_t at;
    EXAMPLE_REQUIRE(proven_http_event_server_create(app.loop, &server_config, &server) == PROVEN_OK, "a
server");
    proven_err_t err = proven_http_event_server_listen(server,
proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0), &at);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        proven_http_event_server_destroy(server);
        proven_loop_destroy(app.loop);
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK, "it listens");
    proven_job_sys_t *threads = NULL;
    proven_job_group_t running;
    proven_job_group_init(&running);
    EXAMPLE_REQUIRE(proven_job_system_init(heap, 1, 4, &threads) == PROVEN_OK &&
                    proven_job_group_submit(threads, &running, run, &app) == PROVEN_OK, "the loop runs on
another thread");

    // ---- a client that asks for compression ------------------------------------
    proven_http_client_config_t client_config = { .alloc = heap };
    proven_http_client_t *client = NULL;
    EXAMPLE_REQUIRE(proven_http_client_create(&client_config, &client) == PROVEN_OK, "an HTTP client");
    char url[64];
    int n = snprintf(url, sizeof url, "ws://127.0.0.1:%u/", (unsigned)at.port);
    proven_ws_conn_config_t ws_config = { .alloc = heap, .compress = true };       /* offer the extension */
    proven_ws_conn_t *conn = NULL;
    EXAMPLE_REQUIRE(proven_ws_conn_connect(client, (proven_u8str_view_t){ (const proven_byte_t *)url,
(proven_size_t)n }, NULL, 0, PROVEN_LIT(""),
                                           &ws_config, &conn, NULL) == PROVEN_OK, "connected");
    EXAMPLE_REQUIRE(proven_ws_conn_compressed(conn), "and the server agreed to compress");

    /* A hundred thousand bytes of a report. On the wire it is a few hundred. */
    static char report[100000];
    for (proven_size_t i = 0; i < sizeof report; ++i) report[i] = "temperature 21.5, humidity 40\n"[i % 30];
    proven_ws_message_t msg;
    EXAMPLE_REQUIRE(proven_ws_conn_send_text(conn, (proven_u8str_view_t){ (const proven_byte_t *)report,
sizeof report }) == PROVEN_OK &&
                    proven_ws_conn_receive(conn, proven_net_deadline_in(5000), &msg) == PROVEN_OK &&
                    msg.data.size == 12 && memcmp(msg.data.ptr, "100000 bytes", 12) == 0, "the server received
all of it");

    /* The limit is on the decoded size. Three hundred thousand of one letter are a few hundred
     * bytes compressed - and still too much: the server ends the connection with 1009. */
    /* One message kept out of the compressor - what a message that carries a secret beside
     * text the other side chose needs. The switch is read when a message begins. */
    proven_ws_conn_set_compress(conn, false);
    EXAMPLE_REQUIRE(proven_ws_conn_send_text(conn, (proven_u8str_view_t){ (const proven_byte_t *)report,
1000 }) == PROVEN_OK &&
                    proven_ws_conn_receive(conn, proven_net_deadline_in(5000), &msg) == PROVEN_OK &&
                    msg.data.size == 10 && memcmp(msg.data.ptr, "1000 bytes", 10) == 0, "a thousand bytes,
sent as they are");
    proven_ws_conn_set_compress(conn, true);

    static char flood[300000];
    memset(flood, 'x', sizeof flood);
    EXAMPLE_REQUIRE(proven_ws_conn_send_text(conn, (proven_u8str_view_t){ (const proven_byte_t *)flood, sizeof
flood }) == PROVEN_OK, "a message that inflates past the server's limit");
    EXAMPLE_REQUIRE(proven_ws_conn_receive(conn, proven_net_deadline_in(5000), &msg) == PROVEN_ERR_EOF &&
```

```
proven_ws_conn_close_code(conn) == 1009, "is answered with 1009, however small it was on the wire");
    proven_ws_conn_destroy(conn);
    proven_http_client_destroy(client);

    proven_loop_stop(app.loop);
    proven_job_group_wait(threads, &running);
    EXAMPLE_REQUIRE(app.compressed, "the server found the extension agreed as well");
    proven_http_event_server_destroy(server);
    proven_loop_destroy(app.loop);
    proven_job_system_close(threads);
    proven_job_system_destroy(threads);
    return EXAMPLE_OK();
}
```

Example: the extension with the codec alone

For a program that drives the codec itself (sections 6 to 8): the handshake functions, the bit in the frame, and the four bytes.

```
#include <stdio.h>
#include <string.h>

/*
 * Compressed WebSocket messages with the codec alone: no socket, no driver.
 *
 * Three things make up the extension called permessage-deflate. The handshake agrees on it.
 * One bit in a frame's header says "this message is compressed". And the payload is DEFLATE
 * with its last four bytes left off. The codec does the first two; the third is deflate.h.
 *
 * A program that uses ws_conn.h or ws_event.h sets one field and sees none of this.
 */

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();

    // ---- the handshake ------------------------------------------------------------
    /* A client asks with one header. PROVEN_WS_DEFLATE_OFFER is the simpler of the two forms
     * this library speaks: each message compressed on its own, in both directions. The other,
     * with context takeover, is at the end. */
    proven_http_header_t asked[] = {
        { PROVEN_LIT("Sec-WebSocket-Extensions"), PROVEN_LIT(PROVEN_WS_DEFLATE_OFFER) },
    };
    proven_http_request_t request = { .headers = asked, .header_count = 1 };

    /* A server looks at what was offered and writes its answer - or finds nothing it can
     * meet (PROVEN_ERR_NOT_FOUND), and the connection goes on uncompressed. */
    proven_byte_t answer[PROVEN_WS_DEFLATE_ANSWER_MAX];
    proven_size_t answer_len = 0;
    int window_bits = 0;
    EXAMPLE_REQUIRE(proven_ws_deflate_accept(&request, (proven_mem_mut_t){ answer, sizeof answer },
&answer_len, &window_bits) == PROVEN_OK, "the offer can be met");
    printf("the server answers: %.*s\n", (int)answer_len, (const char *)answer);
    EXAMPLE_REQUIRE(window_bits == 15, "and may compress with the full window");

    /* A browser's offer looks different, and is met as well. */
    proven_http_header_t browser[] = { { PROVEN_LIT("Sec-WebSocket-Extensions"), PROVEN_LIT("permessage-
deflate; client_max_window_bits") } };
    proven_http_request_t from_browser = { .headers = browser, .header_count = 1 };
    proven_size_t len2 = 0;
    EXAMPLE_REQUIRE(proven_ws_deflate_accept(&from_browser, (proven_mem_mut_t){ answer, sizeof answer },
&len2, &window_bits) == PROVEN_OK, "a browser's offer too");
```

```
    /* The client checks the whole answer: the usual things, and that the extension came back
     * in the form it asked for. (The key and the accept value are the pair in RFC 6455.) */
    proven_http_header_t answered[] = {
        { PROVEN_LIT("Upgrade"), PROVEN_LIT("websocket") },
        { PROVEN_LIT("Connection"), PROVEN_LIT("Upgrade") },
        { PROVEN_LIT("Sec-WebSocket-Accept"), PROVEN_LIT("s3pPLMBiTxaQ9kYGzzhZRbK+xOo=") },
        { PROVEN_LIT("Sec-WebSocket-Extensions"), { answer, answer_len } },
    };
    proven_http_response_t response = { .status = 101, .headers = answered, .header_count = 4 };
    proven_u8str_view_t protocol;
    bool compressed = false;
    EXAMPLE_REQUIRE(proven_ws_check_response_deflate(&response, PROVEN_LIT("dGhlIHNhbXBsZSBub25jZQ=="),
PROVEN_LIT(""), &protocol, &compressed) == PROVEN_OK && compressed,
                    "the client finds the extension agreed");

    // ---- a compressed message arrives ------------------------------------------------
    /* This frame is in RFC 7692: the text "Hello", compressed, from a server. Its first
     * byte is c1 where an uncompressed text frame has 81: the extra bit is RSV1. */
    proven_byte_t wire[] = { 0xc1, 0x07, 0xf2, 0x48, 0xcd, 0xc9, 0xc9, 0x07, 0x00 };

    proven_ws_decoder_t decoder;
    proven_ws_event_t event;
    proven_size_t used = 0;
    /* A decoder that was not told about the extension refuses the bit, as it must. */
    proven_byte_t copy[sizeof wire];
    memcpy(copy, wire, sizeof wire);
    proven_ws_decoder_init(&decoder, false, 0);
    EXAMPLE_REQUIRE(proven_ws_decoder_feed(&decoder, (proven_mem_mut_t){ copy, sizeof copy }, &used, &event)
== PROVEN_ERR_INVALID_FORMAT, "without the extension, the bit is an error");

    proven_ws_decoder_init(&decoder, false, 0);
    proven_ws_decoder_allow_compressed(&decoder);
    EXAMPLE_REQUIRE(proven_ws_decoder_feed(&decoder, (proven_mem_mut_t){ wire, sizeof wire }, &used, &event)
== PROVEN_OK &&
                    event.kind == PROVEN_WS_EVENT_DATA && event.compressed && event.last, "with it, the
message is handed over, marked compressed");

    /* The decoder does not inflate: it allocates nothing. The bytes it hands over are raw
     * DEFLATE whose last four bytes - always 00 00 ff ff - the sender left off. Put them
     * back and inflate. The limit on what comes out is the size of your buffer. */
    proven_inflate_t *z = NULL;
    EXAMPLE_REQUIRE(proven_inflate_create(heap, PROVEN_DEFLATE_RAW, &z) == PROVEN_OK, "a decompressor for raw
DEFLATE");
    static const proven_byte_t tail[4] = { 0x00, 0x00, 0xff, 0xff };
    proven_byte_t text[64];
    proven_size_t taken = 0, made = 0, total = 0;
    bool ended = false;
    EXAMPLE_REQUIRE(proven_inflate(z, event.data, &taken, (proven_mem_mut_t){ text, sizeof text }, &made,
&ended) == PROVEN_OK && taken == event.data.size, "the payload");
    total = made;
    EXAMPLE_REQUIRE(proven_inflate(z, (proven_mem_view_t){ tail, 4 }, &taken, (proven_mem_mut_t){ text +
total, sizeof text - total }, &made, &ended) == PROVEN_OK && taken == 4, "then the four bytes");
    total += made;
    EXAMPLE_REQUIRE(total == 5 && memcmp(text, "Hello", 5) == 0, "and the message is Hello");
    printf("the message: %.*s\n", (int)total, (const char *)text);
    /* Now - not before - it is text, and yours to check: the decoder could not. */
    proven_inflate_destroy(z);

    // ---- a compressed message is sent ------------------------------------------------
```

```
    /* The other way: compress with a sync flush, which ends in those same four bytes, and
     * leave them off. The frame that carries it has `rsv1` set. */
    static const char said[] = "to be, or not to be, that is the question: to be, or not to be";
    proven_deflate_options_t options = { .alloc = heap, .format = PROVEN_DEFLATE_RAW };
    proven_deflate_t *c = NULL;
    EXAMPLE_REQUIRE(proven_deflate_create(&options, &c) == PROVEN_OK, "a compressor");
    proven_byte_t packed[128];
    EXAMPLE_REQUIRE(proven_deflate(c, (proven_mem_view_t){ (const proven_byte_t *)said, sizeof said - 1 },
&taken, (proven_mem_mut_t){ packed, sizeof packed }, &made,
                                   PROVEN_DEFLATE_FLUSH_SYNC, &ended) == PROVEN_OK && taken == sizeof said - 1
&& made > 4, "compressed, with a sync flush");
    EXAMPLE_REQUIRE(memcmp(packed + made - 4, tail, 4) == 0, "which ends in 00 00 ff ff");
    proven_size_t payload = made - 4;
    proven_deflate_destroy(c);

    proven_ws_frame_t frame = { .fin = true, .opcode = PROVEN_WS_TEXT, .rsv1 = true, .length = payload };
    proven_byte_t header[PROVEN_WS_MAX_FRAME_HEADER];
    proven_size_t header_len = 0;
    EXAMPLE_REQUIRE(proven_ws_frame_write((proven_mem_mut_t){ header, sizeof header }, &header_len, &frame) ==
PROVEN_OK && header[0] == 0xc1, "a frame header with the bit");
    printf("%u bytes of text go out as %u\n", (unsigned)(sizeof said - 1), (unsigned)(header_len + payload));
    EXAMPLE_REQUIRE(header_len + payload < sizeof said - 1, "fewer than were said");

    // ---- with context takeover ------------------------------------------------------
    /* The other form: each side keeps its window from one message to the next, so a message
     * refers back to the ones before it. A client offers PROVEN_WS_DEFLATE_OFFER_TAKEOVER; a
     * server answers with proven_ws_deflate_accept_takeover, which forbids takeover only
     * where the offer asked, and says what was agreed. */
    proven_http_header_t asked_take[] = { { PROVEN_LIT("Sec-WebSocket-Extensions"),
PROVEN_LIT(PROVEN_WS_DEFLATE_OFFER_TAKEOVER) } };
    proven_http_request_t request_take = { .headers = asked_take, .header_count = 1 };
    proven_ws_deflate_terms_t terms;
    answer_len = 0;
    EXAMPLE_REQUIRE(proven_ws_deflate_accept_takeover(&request_take, (proven_mem_mut_t){ answer, sizeof
answer }, &answer_len, &terms) == PROVEN_OK &&
                    terms.server_takeover && terms.client_takeover, "takeover is agreed both ways");
    printf("the server answers: %.*s\n", (int)answer_len, (const char *)answer);
    answered[3].value = (proven_u8str_view_t){ answer, answer_len };
    proven_ws_deflate_terms_t seen;
    EXAMPLE_REQUIRE(proven_ws_check_response_takeover(&response, PROVEN_LIT("dGhlIHNhbXBsZSBub25jZQ=="),
PROVEN_LIT(""), &protocol, &compressed, &seen) == PROVEN_OK &&
                    compressed && seen.server_takeover && seen.client_takeover, "and the client sees the same
terms");

    /* A side may keep its own window and forbid the other's: it then needs no decompressor
     * between messages. A server does it with proven_ws_deflate_accept_peer_no_takeover; a
     * client offers PROVEN_WS_DEFLATE_OFFER_PEER_NO_TAKEOVER and checks the answer with
     * proven_ws_check_response_peer_no_takeover. The server forbids here, then the client. */
    answer_len = 0;
    EXAMPLE_REQUIRE(proven_ws_deflate_accept_peer_no_takeover(&request_take, (proven_mem_mut_t){ answer,
sizeof answer }, &answer_len, &terms) == PROVEN_OK &&
                    terms.server_takeover && !terms.client_takeover, "the server keeps its window and forbids
the client's");
    printf("a server that forbids the client's: %.*s\n", (int)answer_len, (const char *)answer);
    proven_http_header_t asked_own[] = { { PROVEN_LIT("Sec-WebSocket-Extensions"),
PROVEN_LIT(PROVEN_WS_DEFLATE_OFFER_PEER_NO_TAKEOVER) } };
    proven_http_request_t request_own = { .headers = asked_own, .header_count = 1 };
    answer_len = 0;
    EXAMPLE_REQUIRE(proven_ws_deflate_accept_takeover(&request_own, (proven_mem_mut_t){ answer, sizeof
answer }, &answer_len, &terms) == PROVEN_OK &&
```

```
                    !terms.server_takeover && terms.client_takeover, "a server honours a client that forbids
its takeover");
    answered[3].value = (proven_u8str_view_t){ answer, answer_len };
    EXAMPLE_REQUIRE(proven_ws_check_response_peer_no_takeover(&response,
PROVEN_LIT("dGhlIHNhbXBsZSBub25jZQ=="), PROVEN_LIT(""), &protocol, &compressed, &seen) == PROVEN_OK &&
                    compressed && !seen.server_takeover && seen.client_takeover, "and the client checks that
it did");

    /* A side that takes over keeps one compressor for the connection and ends every message
     * with a sync flush; the side that reads keeps one decompressor. The same line twice: the
     * second goes out as a reference to the first. */
    EXAMPLE_REQUIRE(proven_deflate_create(&options, &c) == PROVEN_OK && proven_inflate_create(heap,
PROVEN_DEFLATE_RAW, &z) == PROVEN_OK, "one of each, kept");
    for (int round = 0; round < 2; ++round) {
        EXAMPLE_REQUIRE(proven_deflate(c, (proven_mem_view_t){ (const proven_byte_t *)said, sizeof said - 1 },
&taken, (proven_mem_mut_t){ packed, sizeof packed }, &made,
                                       PROVEN_DEFLATE_FLUSH_SYNC, &ended) == PROVEN_OK && made > 4, "a message
through the kept compressor");
        printf("message %d goes out as %u bytes\n", round + 1, (unsigned)(made - 4));
        if (round == 1) EXAMPLE_REQUIRE(made - 4 < 10, "the second is a few bytes: a reference into the
first");
        /* The reader: the payload as it arrived - the four bytes left off, then put back. */
        proven_byte_t back[128];
        proven_size_t got = 0;
        EXAMPLE_REQUIRE(proven_inflate(z, (proven_mem_view_t){ packed, made }, &taken, (proven_mem_mut_t)
{ back, sizeof back }, &got, &ended) == PROVEN_OK &&
                        got == sizeof said - 1 && memcmp(back, said, got) == 0, "and the kept decompressor
reads it");
        /* A sender may end a message with a final block instead (RFC 7692, 7.2.3.4): the
         * decompressor then reports `ended`, and the reader goes on with
         * proven_inflate_restart, which keeps the window - not reset, which forgets it. */
        if (ended) proven_inflate_restart(z);
    }
    proven_deflate_destroy(c);
    proven_inflate_destroy(z);
    return EXAMPLE_OK();
}
```

Reference

API	Intent	Return
`compress` (both connection configurations)	Offer the extension (a client) or accept it when offered (a server).	-
`compress_level`, `compress_window_bits` (both)	As in Chapter 16, section 4: 1 to 9 or -1; 9 to 15. Zero: 6, and 11 with takeover or 15 without.	`INVALID_ARG` outside those ranges.
`compress_no_takeover` (both)	Each message compressed on its own, nothing kept between messages - the form before v0.80.0.	-
`proven_ws_conn_compressed(conn)`, `proven_ws_stream_compressed(ws)`	Whether the other side agreed.	`bool`.
`proven_ws_conn_set_compress(conn, on)`,	Whether the messages this side sends from now on are	none.

`proven_ws_stream_set_compress(ws, on)`	compressed. For the one that must not be - a secret beside text the other side chose, or bytes compressed already: off, send, on. Nothing on a connection that did not agree to compression.	
`PROVEN_WS_DEFLATE_OFFER`	The `Sec-WebSocket-Extensions` value a client sends.	a string literal.
`proven_ws_deflate_accept(request, out, &len, &window_bits)`	A server's choice among the request's offers; writes the answer.	`proven_err_t`: `NOT_FOUND` when no offer can be met - go on without; `OUT_OF_BOUNDS` (`PROVEN_WS_DEFLATE_ANSWER_MAX` bytes are enough).
`proven_ws_check_response_deflate(response, key, offered, &protocol, &deflate)`	`proven_ws_check_response` for a client that offered.	the same, and `INVALID_FORMAT` for an answer in a form that was not offered.
`PROVEN_WS_DEFLATE_OFFER_TAKEOVER`	The value a client sends to ask for takeover.	a string literal.
`proven_ws_deflate_accept_takeover(request, out, &len, &terms)`	`proven_ws_deflate_accept` for a server that keeps its window: forbids takeover only where the offer asked; `terms` says what was agreed.	as `proven_ws_deflate_accept`.
`proven_ws_check_response_takeover(response, key, offered, &protocol, &deflate, &terms)`	The check for a client that sent `PROVEN_WS_DEFLATE_OFFER_TAKEOVER`: any of the four parameters, once each; `terms` says what was agreed.	the same, and `INVALID_FORMAT` for a parameter twice, unknown, or out of range.
`compress_peer_no_takeover` (both)	This side keeps its window; the other side must compress each message on its own. A server forbids it in its answer, a client asks for it in its offer.	`INVALID_ARG` from `proven_ws_conn_open`.
`PROVEN_WS_DEFLATE_OFFER_PEER_NO_TAKEOVER`	The value a client sends to keep its own window and ask the server to keep none.	a string literal.
`proven_ws_deflate_accept_peer_no_takeover(request, out, &len, &terms)`	`proven_ws_deflate_accept_takeover`, with `client_no_context_takeover` always in the answer.	as `proven_ws_deflate_accept`.
`proven_ws_check_response_peer_no_takeover(response, key, offered, &protocol, &deflate, &terms)`	The check for a client that sent `PROVEN_WS_DEFLATE_OFFER_PEER_NO_TAKEOVER`.	as `proven_ws_check_response_takeover`, and `INVALID_FORMAT` for an answer without `server_no_context_takeover`.
`proven_ws_deflate_terms_t`	Who may keep a window (`server_takeover`, `client_takeover`),	-

	and how large each side's may be (`server_window_bits`, `client_window_bits`; 8 means this side cannot compress and sends uncompressed).	
`proven_inflate_restart(z)`	After `ended`: go on keeping the window, for a peer that took over and ended a message with a final block.	none.
`proven_ws_decoder_allow_compressed(dec, 1)`	Let the decoder pass compressed messages through, marked `compressed`. It does not inflate.	none.
`rsv1` in `proven_ws_frame_t`	The bit that marks a compressed message, on its first frame.	-

Cautions, and what goes wrong

- **Do not compress messages that carry a secret together with text the other side chose.** The length of a compressed message tells an observer of the connection how much the two have in common, with or without TLS - the leak described in Chapter 16, section 5. With takeover (the default) the secret and the chosen text need not be in the same message: earlier messages change a later one's length too. With `compress_no_takeover` each message is compressed alone and one message cannot leak another; what is inside a single message still can. Compression is per connection here: a connection that carries such messages should not ask for it.
- **The limit is on the inflated size, and you should set one.** `max_message_bytes` is what stops a kilobyte from becoming a gigabyte. Its default is 1 MiB.
- **Raw DEFLATE has no checksum.** A compressed message damaged in transit can inflate to different bytes with no error. TLS prevents that; without it, check what matters.
- **A peer that offers only what cannot be met is served uncompressed, not refused.** The one such offer is a compression window of 256 bytes (`server_max_window_bits=8`), which this compressor does not have. A client's own window size (`client_max_window_bits`) is accepted and ignored: any window can be read.
- **A client with `compress_no_takeover` refuses a server that answers in another form.** One that takes that offer without `server_no_context_takeover` would compress across messages against what was asked; the connection fails with `PROVEN_ERR_INVALID_FORMAT`. Servers that follow RFC 7692 either accept the offer as made or decline it.
- **Memory.** With takeover: about 28 KiB (window 11) and 35 KiB, kept from the first compressed message to the end, and 16 KiB more in the event-driven server while a message is arriving. With `compress_no_takeover`: about 35 KiB to inflate one message (51 KiB in the event-driven server), and from 11 KiB to 325 KiB to compress one, by the window and no more than the message needs, allocated for the message and freed with it - so a busy connection allocates per message where one with takeover does not. With `compress_peer_no_takeover`: the compressor kept, and a decompressor per message received.
- **With the codec alone, text is yours to check.** The decoder cannot check bytes that are not text yet; after inflating, check them. And count what comes out: the decoder's limit counts compressed bytes.

10. What is not here

- **Any extension other than `permessage-deflate`.** Section 9.
- **A server that forbids its clients' takeover while keeping its own** (it would hold about 28 KiB instead of 64): a server takes over exactly as far as each offer allows.
- **One call that waits on many connections.** See section 5.
- **Concurrent send and receive on one connection.**
- **Automatic pings, reconnection, backoff.** Policy, and yours.
- **Messages larger than memory through `ws_conn.h`.** A message is assembled whole; to stream one, use the decoder.
- **Checking `Origin` for you.** No server here refuses an origin by itself: the list of origins is yours, and so is the call (`proven_ws_check_origin`, section 2).
- **HTTP/2 WebSockets (RFC 8441).**

Chapter 13: Certificates and trust

Part V - Talking to the operating system. Prerequisites: Chapter 4 for digests; Chapter 2 for the allocator a store takes. After this chapter you can read an X.509 certificate, say which names it is for, load the certificates you trust, and decide whether a peer's certificate chain is to be believed - and you will know exactly which checks were made and which were not.

This chapter covers `cert.h`. Reading and verifying allocate nothing and call no operating system: they are available in a freestanding build. A store of trust anchors allocates through the allocator you give it. One call, `proven_cert_store_add_system`, reads the operating system's roots and is hosted-only.

What this is. This is the certificate half of TLS, usable on its own: checking a certificate you were handed, from a file, a message or another library's handshake. The protocol itself - the handshake and the encrypted records - is Chapter 14, which uses everything here.

And a caution that will stay in this chapter whatever is added to it. This is a new implementation with no external audit. It is tested against another implementation and against published adversarial vectors (section 8 says which), and that is evidence, not proof. A program whose users depend on it against a capable adversary should have its certificates checked by something audited.

Table of contents

1. What a certificate is, in one page
2. Reading one
3. PEM
4. Which names it is for
5. Trust anchors
6. Verifying a chain
7. Pinning
8. What is checked, what is accepted, what is not here

1. What a certificate is, in one page

A public key lets you check that a message came from whoever holds the matching private key. It does not tell you *who* that is. A **certificate** is the missing sentence: "this public key belongs to `example.com`", signed by an **issuer**. You can check the issuer's signature if you have the issuer's public key - which comes in the issuer's own certificate, signed by *its* issuer. The sentences form a **chain**, and the chain has to end somewhere: at a certificate you trust not because anyone signed it but because it was already on your machine. That is a **trust anchor**, or root.

So believing a peer is four questions, and `proven_cert_verify` asks all four:

Question	What goes wrong when it is skipped
Is there a chain of valid signatures from this certificate to an anchor I hold?	Anyone can make a certificate for any name. Only the chain says someone you trust vouched for it.
Was every certificate in the chain valid at this moment?	A key that leaked years ago still signs. The dates bound how long a leak matters.
Was every issuer in the chain *allowed* to issue?	The certificate of an ordinary web site is signed by a real authority. If it could sign others, every site owner could mint certificates for any name.

Is the name in the certificate the name I meant to reach?	A perfectly valid certificate for `attacker.example` proves nothing about `bank.example`.

The formats: a certificate is **DER** - a compact binary encoding - and it usually travels as **PEM**, which is that DER in Base64 between a `-----BEGIN CERTIFICATE-----` line and an `END` line, so that it survives being pasted into a text file.

2. Reading one

```
proven_err_t proven_cert_parse(proven_mem_view_t der, proven_cert_t *out);
```

`proven_cert_parse` reads one DER certificate into a `proven_cert_t`. **It copies nothing.** Every field of the struct is a view into the bytes you passed, which must stay alive for as long as you use the struct. There is nothing to free.

The reader is strict, on purpose. A certificate is input from a stranger, and an encoding that two programs can read differently is how one of them gets fooled. So: definite lengths only, each written in its shortest form; integers without needless leading zeros; a `TRUE` that is exactly `FF`; the signature algorithm named identically inside and outside the signed part; no extension twice; and not one byte after the end. Anything else is `PROVEN_ERR_INVALID_FORMAT`.

The fields you are most likely to want:

Field	What it is
`der`, `tbs`	The whole certificate, and the part the issuer signed
`subject`, `issuer`	The two names, each as DER. Compare them as bytes; do not parse them for a host name (section 4)
`not_before`, `not_after`	The validity period, as seconds since 1970-01-01 UTC
`public_key_info`	The public key with its algorithm, as DER - what a pin hashes (section 7)
`key_kind`, `key`, `rsa_n`, `rsa_e`	The key itself: RSA, P-256, P-384 or Ed25519; `PROVEN_CERT_KEY_UNKNOWN` for anything else
`sig_kind`, `sig_hash`, `signature`	How the *issuer* signed this certificate
`is_ca`, `has_path_len`, `path_len`	Whether it may issue certificates, and how deep a chain below it may go
`key_usage`, `ext_key_usage`	What the key may be used for, as `PROVEN_CERT_KU_*` and `PROVEN_CERT_EKU_*` flags
`alt_names`	The names it is for. Walk them with `proven_cert_alt_name_next`
`unknown_critical`	It carries an extension marked "do not accept me unless you understand this", which this library does not understand

An algorithm this library does not know - a key on another curve, a signature with SHA-1 - is **not** a parse error. The certificate parses with the kind `UNKNOWN`, because a store of system roots contains a few such certificates and one of them should not cost the rest. It fails later, in verification, if a chain actually needs it.

```
bool proven_cert_alt_name_next(const proven_cert_t *cert, proven_size_t *pos,
                               proven_cert_name_kind_t *kind, proven_mem_view_t *value);
```

`proven_cert_alt_name_next` walks the subjectAltName entries. Start with `*pos = 0` and call it until it returns `false`. A `PROVEN_CERT_NAME_DNS` value is the name's ASCII text; a `PROVEN_CERT_NAME_IP` value is 4 or 16 address bytes; anything else (an email address, a URI) comes as `PROVEN_CERT_NAME_OTHER` with its raw content.

3. PEM

```
proven_err_t proven_pem_next(proven_mem_view_t pem, proven_size_t *pos, proven_u8str_view_t *label,
                             proven_mem_mut_t out, proven_size_t *written);
```

`proven_pem_next` finds the next block at or after `*pos`, decodes its Base64 into `out`, sets `label` to the words after `BEGIN` (`CERTIFICATE` for a certificate), and moves `*pos` past the block. Call it in a loop to read a file of several. Text between blocks - comments, the human-readable dump some tools add - is skipped.

Returns	When
`PROVEN_OK`	A block was decoded; `*written` bytes are in `out`
`PROVEN_ERR_NOT_FOUND`	No further block
`PROVEN_ERR_OUT_OF_BOUNDS`	`out` is too small. `*written` is the size needed and `*pos` has not moved: enlarge and call again
`PROVEN_ERR_INVALID_ENCODING`	A block with no matching `END` line, or a body that is not Base64

It is not only for certificates: a key file is PEM too, with another label. This call does not care what is inside.

4. Which names it is for

```
bool proven_cert_matches_host(const proven_cert_t *cert, proven_u8str_view_t host);
```

`proven_cert_matches_host` answers whether the certificate is for `host`, which is a DNS name or an IP address written as text. The rules are the ones browsers settled on, and each closes a hole that was once open:

- **Only subjectAltName is consulted.** The subject's "common name" is free text that certificates once put a host name in. It is not a host name, and it is not looked at.
- **A wildcard is a whole left-most label and stands for exactly one label.** `*.example.com` matches `www.example.com`. It does not match `example.com`, nor `a.b.example.com`, nor does `w*.example.com` mean anything. A pattern with fewer than two labels after the star (`*.com`) matches nothing.
- **An IP address is matched as an address, against IP entries only.** `192.0.2.1` never matches a DNS entry that happens to read `192.0.2.1`, and `2001:db8::1` matches however the address was spelled.
- **Letter case does not matter, and one trailing dot on the host is ignored.**

Names are compared as ASCII. An internationalised name must be given in its `xn--` form, which is how it appears in the certificate.

5. Trust anchors

```
proven_err_t proven_cert_store_create(proven_allocator_t alloc, proven_cert_store_t **out);
void proven_cert_store_destroy(proven_cert_store_t *store);
proven_err_t proven_cert_store_add_der(proven_cert_store_t *store, proven_mem_view_t der);
proven_err_t proven_cert_store_add_pem(proven_cert_store_t *store, proven_mem_view_t pem, proven_size_t
*added);
proven_err_t proven_cert_store_add_system(proven_cert_store_t *store, proven_size_t *added);   /* hosted */
proven_size_t proven_cert_store_count(const proven_cert_store_t *store);
```

A `proven_cert_store_t` is the set of certificates you trust. **It owns copies** of what you add, so the bytes you added from may be freed straight away; `proven_cert_store_destroy` frees the copies and the store.

The library carries no roots. A list of trusted authorities compiled into a library is stale the day it ships and wrong for half its users. You choose:

- `proven_cert_store_add_system` takes the operating system's roots - the right set for talking to the public internet. On Windows it reads the `ROOT` system store. Elsewhere it reads the first file it can among the path in the environment variable `SSL_CERT_FILE` and the usual bundle locations. It returns

`PROVEN_ERR_NOT_FOUND` when the machine has none it can read - a minimal container often has none - and that is something to report, not to work around by trusting everything. **On Windows the answer can be smaller than you expect:** Windows keeps only the roots it has needed so far and fetches others on demand when its own TLS meets them. This call reads what is installed and triggers no fetch, so a chain to a root that machine has never used is `PROVEN_ERR_UNTRUSTED` there. A program that must reach arbitrary servers from Windows should ship a bundle and use `proven_cert_store_add_pem`.

- `proven_cert_store_add_pem` takes a bundle you supply: your organisation's authority, or the single certificate of the one server your program talks to. A block that is not a certificate and a certificate that does not parse are skipped, and `*added` counts the ones taken. A bundle with no certificate at all is `PROVEN_ERR_NOT_FOUND`.
- `proven_cert_store_add_der` adds one.

A store is not changed by verification, so one store may be shared by any number of threads that only verify. Adding to it while another thread verifies is a race.

6. Verifying a chain

```
proven_err_t proven_cert_verify(const proven_mem_view_t *chain, proven_size_t count,
                                const proven_cert_verify_options_t *options,
                                proven_cert_verify_result_t *result);
```

`chain[0]` is the peer's own certificate. The rest are whatever else the peer sent, **in any order** - servers are careless about order, and some send certificates that are not needed. `proven_cert_verify` searches for a path from `chain[0]` to an anchor, using the others as material. A certificate among them that does not parse is simply not used.

The options:

- `anchors` - the store. Required.
- `host` - the name you meant to reach. Leave it empty to skip the name check, which is right only when you check identity some other way (a pin, a client certificate mapped to an account).
- `now` - the time, in seconds since 1970-01-01 UTC. **You supply it.** A library that read the clock itself could not be tested against an expired certificate without waiting, and could not run where there is no clock. On a hosted system pass the wall-clock time; on a device with no battery-backed clock, decide what time you are willing to assume and know that the dates are only as good as that.
- `use` - `PROVEN_CERT_USE_SERVER` when the peer is a server, `PROVEN_CERT_USE_CLIENT` when it is a client presenting a certificate to you.

The answer comes at two levels. The **error code** is what your program branches on:

Returns	Meaning	Reasonable response
`PROVEN_OK`	A path exists and every check passed	Proceed
`PROVEN_ERR_UNTRUSTED`	No acceptable path to an anchor	Refuse. Nothing vouches for this certificate
`PROVEN_ERR_EXPIRED`	A path exists, but a certificate on it is past its `not_after`	Refuse; tell the user it expired. If *every* site fails this way, suspect your clock
`PROVEN_ERR_NOT_YET_VALID`	A path exists, but a certificate on it is before its `not_before`	Refuse; the clock is the usual culprit

`PROVEN_ERR_NAME_MISMATCH`	The chain is good and the certificate is for another name	Refuse. This is what a misdirected or intercepted connection looks like
`PROVEN_ERR_INVALID_FORMAT`	`chain[0]` is not a certificate	Refuse
`PROVEN_ERR_INVALID_ARG`	No chain, no options, or no store	Fix the call

The **fault** in `result` (optional - pass a null pointer if you do not want it) is the precise reason, for a log line or an error message: `PROVEN_CERT_FAULT_NO_ISSUER`, `_BAD_SIGNATURE`, `_ALGORITHM`, `_NOT_A_CA`, `_PATH_LENGTH`, `_NAME_CONSTRAINT`, `_CRITICAL_EXTENSION`, `_USAGE`, `_EXPIRED`, `_NOT_YET_VALID`, `_NAME_MISMATCH`, `_TOO_DEEP`, `_MALFORMED`. `result->depth` is the number of certificates in the accepted path, anchor included.

When several things are wrong at once, the answer is the one closest to success: if a signed path to an anchor exists but is expired, you are told it expired, not that nothing was found. That is the difference between "renew your certificate" and "who are you?".

Compiled and run by the test suite:

```
#include <string.h>

/*
 * A certificate is a signed statement that a key belongs to a name. This reads one, asks which
 * names it is for, and then decides whether to believe it: a chain to a trust anchor, the dates,
 * and the name - with the error that says which of those failed.
 */

/* A root and a certificate it issued for example.test, made for this example. Ed25519, so they
 * are short. Valid from 2026 to 2036. */
static const char ROOT_PEM[] =
    "-----BEGIN CERTIFICATE-----\n"
    "MIIBAjCBtaADAgECAhQxTiZoH2JgH7bPWBktoxUyd0d3ZzAFBgMrZXAwFzEVMBMG\n"
    "A1UEAwwMRXhhbXBsZSBSb290MB4XDTI2MDEwMTAwMDAwMFoXDTM2MDEwMTAwMDAw\n"
    "MFowFzEVMBMGA1UEAwwMRXhhbXBsZSBSb290MCowBQYDK2VwAyEAJcpfTb5GYzM+\n"
    "bgKssEvuWEC6urqf1Znqz9I8CY2RdLujEzARMA8GA1UdEwEB/wQFMAMBAf8wBQYD\n"
    "K2VwA0EARcXC/GhPDj4gAU3zpeSAhJ033zALpz2P91wtEqbf2R4Z7drW+RSRQGp2\n"
    "Ky/CDeEyx//WjylXEoHVNvixwKrxAw==\n"
    "-----END CERTIFICATE-----\n"
    ;

static const char LEAF_PEM[] =
    "-----BEGIN CERTIFICATE-----\n"
    "MIIBPTCB8KADAgECAhRBU6wAvGvpDo28++Tq/BfZ/8DkNzAFBgMrZXAwFzEVMBMG\n"
    "A1UEAwwMRXhhbXBsZSBSb290MB4XDTI2MDEwMTAwMDAwMFoXDTM2MDEwMTAwMDAw\n"
    "MFowFzEVMBMGA1UEAwwMZXhhbXBsZS50ZXN0MCowBQYDK2VwAyEA3dq7G1ndiCbb\n"
    "IAS6O7cRIlxdACAdLSsPSRROKyglnTOjTjBMMAwGA1UdEwEB/wQCMAAwJwYDVR0R\n"
    "BCAwHoIMZXhhbXBsZS50ZXN0gg4qLmV4YW1wbGUudGVzdDATBgNVHSUEDDAKBggr\n"
    "BgEFBQcDATAFBgMrZXADQQBVDpl9ZAqf4j2R3NTG8JuKfPJuh+PD99QVdpjnqPAR\n"
    "dgg3sGIps+UTADlifcSOVlqJF6T9ya5lHpO5FwIpLpMI\n"
    "-----END CERTIFICATE-----\n"
    ;

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();

    // ---- PEM: text around the bytes ---------------------------------------------
    /* Certificates travel as PEM: Base64 between BEGIN and END lines. proven_pem_next finds the
     * next block and decodes it; the bytes inside are DER, which is what everything else takes. */
    proven_byte_t leaf_der[1024];
    proven_size_t pos = 0, leaf_len = 0;
```

```
    proven_u8str_view_t label;
    proven_mem_view_t leaf_pem = { (const proven_byte_t *)LEAF_PEM, sizeof LEAF_PEM - 1 };
    EXAMPLE_REQUIRE(proven_pem_next(leaf_pem, &pos, &label, (proven_mem_mut_t){ leaf_der, sizeof leaf_der },
&leaf_len) == PROVEN_OK,
                    "the block decodes");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(label, PROVEN_LIT("CERTIFICATE")), "and says what it is");
    EXAMPLE_REQUIRE(proven_pem_next(leaf_pem, &pos, &label, (proven_mem_mut_t){ leaf_der, 0 },
&(proven_size_t){ 0 }) == PROVEN_ERR_NOT_FOUND,
                    "there is no second block");

    // ---- reading one certificate ------------------------------------------------
    /* Parsing copies nothing: every field of proven_cert_t is a view into leaf_der, which must
     * stay alive as long as the struct is used. */
    proven_cert_t leaf;
    EXAMPLE_REQUIRE(proven_cert_parse((proven_mem_view_t){ leaf_der, leaf_len }, &leaf) == PROVEN_OK, "it
parses");
    EXAMPLE_REQUIRE(leaf.version == 3 && leaf.key_kind == PROVEN_CERT_KEY_ED25519 && leaf.key.size == 32 && !
leaf.is_ca,
                    "version 3, an Ed25519 key, not a CA");
    EXAMPLE_REQUIRE(leaf.not_before < 1811808000 && leaf.not_after > 1811808000, "valid in June 2027");

    proven_size_t at = 0, names = 0;
    proven_cert_name_kind_t kind;
    proven_mem_view_t name;
    while (proven_cert_alt_name_next(&leaf, &at, &kind, &name)) {
        if (kind == PROVEN_CERT_NAME_DNS) names++;
    }
    EXAMPLE_REQUIRE(names == 2, "two DNS names in subjectAltName");

    /* Which hosts is it for? Only subjectAltName counts. A wildcard stands for exactly one label. */
    EXAMPLE_REQUIRE(proven_cert_matches_host(&leaf, PROVEN_LIT("example.test")), "the name itself");
    EXAMPLE_REQUIRE(proven_cert_matches_host(&leaf, PROVEN_LIT("www.example.test")), "one label under the
wildcard");
    EXAMPLE_REQUIRE(!proven_cert_matches_host(&leaf, PROVEN_LIT("a.b.example.test")), "but not two");
    EXAMPLE_REQUIRE(!proven_cert_matches_host(&leaf, PROVEN_LIT("example.org")), "and not another name");

    // ---- trust anchors ----------------------------------------------------------
    /* A store holds the certificates you already trust. It copies what it is given, so ROOT_PEM
     * could be freed after this. The library ships no roots: you supply them. */
    proven_cert_store_t *anchors = NULL;
    proven_size_t added = 0;
    EXAMPLE_REQUIRE(proven_cert_store_create(heap, &anchors) == PROVEN_OK, "a store");
    EXAMPLE_REQUIRE(proven_cert_store_add_pem(anchors, (proven_mem_view_t){ (const proven_byte_t *)ROOT_PEM,
sizeof ROOT_PEM - 1 }, &added) == PROVEN_OK &&
                    added == 1 && proven_cert_store_count(anchors) == 1, "one anchor from the bundle");

    // ---- verifying --------------------------------------------------------------
    /* chain[0] is the peer's certificate; anything else it sent follows, in any order. The time
     * is yours to supply: a library that read the clock itself could not be tested, and could
     * not run where there is no clock. */
    proven_mem_view_t chain[1] = { { leaf_der, leaf_len } };
    proven_cert_verify_options_t options = {
        .anchors = anchors,
        .host = PROVEN_LIT("www.example.test"),
        .now = 1811808000,                      /* 2027-06-01 */
        .use = PROVEN_CERT_USE_SERVER,
    };
    proven_cert_verify_result_t result;
    EXAMPLE_REQUIRE(proven_cert_verify(chain, 1, &options, &result) == PROVEN_OK && result.depth == 2, "a path
of two: the leaf and the anchor");
```

```
    /* Each way of being wrong has its own answer. The error code is the coarse one; the fault in
     * the result is for the log line. */
    options.host = PROVEN_LIT("example.org");
    EXAMPLE_REQUIRE(proven_cert_verify(chain, 1, &options, &result) == PROVEN_ERR_NAME_MISMATCH &&
                    result.fault == PROVEN_CERT_FAULT_NAME_MISMATCH, "the chain is good and the name is not");
    options.host = PROVEN_LIT("www.example.test");
    options.now = 2127427200;                   /* 2037-06-01 */
    EXAMPLE_REQUIRE(proven_cert_verify(chain, 1, &options, &result) == PROVEN_ERR_EXPIRED, "ten years on, it
has expired");
    options.now = 1811808000;
    leaf_der[leaf_len - 1] ^= 1;
    EXAMPLE_REQUIRE(proven_cert_verify(chain, 1, &options, &result) == PROVEN_ERR_UNTRUSTED &&
                    result.fault == PROVEN_CERT_FAULT_BAD_SIGNATURE, "one changed bit, and the signature no
longer verifies");
    leaf_der[leaf_len - 1] ^= 1;

    proven_cert_store_t *empty = NULL;
    EXAMPLE_REQUIRE(proven_cert_store_create(heap, &empty) == PROVEN_OK, "a store");
    options.anchors = empty;
    EXAMPLE_REQUIRE(proven_cert_verify(chain, 1, &options, &result) == PROVEN_ERR_UNTRUSTED &&
                    result.fault == PROVEN_CERT_FAULT_NO_ISSUER, "with no anchors, nothing is trusted");

    // ---- a pin ------------------------------------------------------------------
    /* A pin is the SHA-256 of the certificate's public key, as SubjectPublicKeyInfo. A program
     * that knows which key it expects compares this, in addition to or instead of the chain. */
    proven_byte_t pin[32], again[32];
    EXAMPLE_REQUIRE(proven_cert_key_sha256(&leaf, pin) == PROVEN_OK, "the pin");
    proven_sha256(leaf.public_key_info, again);
    EXAMPLE_REQUIRE(proven_mem_equal_ct((proven_mem_view_t){ pin, 32 }, (proven_mem_view_t){ again, 32 }), "is
the hash of the key's encoding");

    // ---- the system's roots -----------------------------------------------------
    /* For talking to the public internet, the anchors are the operating system's. Whether a
     * machine has any is the machine's business, so this example accepts both answers. */
    proven_cert_store_t *system_roots = NULL;
    EXAMPLE_REQUIRE(proven_cert_store_create(heap, &system_roots) == PROVEN_OK, "a store");
    proven_err_t sys = proven_cert_store_add_system(system_roots, &added);
    EXAMPLE_REQUIRE(sys == PROVEN_OK || sys == PROVEN_ERR_NOT_FOUND, "the system store was read, or there is
none");
    EXAMPLE_REQUIRE((sys == PROVEN_OK) == (proven_cert_store_count(system_roots) > 0), "and it is not empty
when it was read");

    /* A single DER certificate can be added too. */
    proven_byte_t root_der[1024];
    proven_size_t root_len = 0;
    pos = 0;
    EXAMPLE_REQUIRE(proven_pem_next((proven_mem_view_t){ (const proven_byte_t *)ROOT_PEM, sizeof ROOT_PEM -
1 }, &pos, &label,
                                    (proven_mem_mut_t){ root_der, sizeof root_der }, &root_len) == PROVEN_OK,
"the block decodes");
    EXAMPLE_REQUIRE(proven_cert_store_add_der(empty, (proven_mem_view_t){ root_der, root_len }) == PROVEN_OK,
"the root as DER");
    EXAMPLE_REQUIRE(proven_cert_verify(chain, 1, &options, &result) == PROVEN_OK, "and now the once-empty
store trusts the leaf");

    proven_cert_store_destroy(system_roots);
    proven_cert_store_destroy(empty);
    proven_cert_store_destroy(anchors);
```

```
    return EXAMPLE_OK();
}
```

7. Pinning

```
proven_err_t proven_cert_key_sha256(const proven_cert_t *cert, proven_byte_t out[32]);
```

`proven_cert_key_sha256` gives the SHA-256 of the certificate's public key, in its SubjectPublicKeyInfo encoding. That is a **pin**: a program that knows in advance which key its server holds can compare this value and refuse everything else, whatever any authority says.

Pin the key, not the certificate. A certificate is reissued every few months and its hash changes each time; the key underneath can stay. Compare with `proven_mem_equal_ct`, and ship at least two pins - the key in use and a spare kept offline - because a pinned program whose only pinned key is lost cannot be fixed without an update.

A pin can be used **in addition to** verification (the chain must be good *and* the key must be the expected one) or **instead of** it (parse the peer's certificate, compare the pin, never build a chain). The second is right for a self-signed server you operate; it means the dates and names in the certificate are not checked at all, so the pin is then the whole of your security.

8. What is checked, what is accepted, what is not here

Checked, for every certificate on the path:

- The issuer's signature over it.
- The validity period, against the time you supplied. This includes the anchor.
- That its issuer is a CA: `basicConstraints` says so, and `keyUsage`, when present, allows signing certificates. An anchor with no extensions at all (version 1) is accepted as a CA because you put it in the store; a version 3 certificate that does not say it is a CA is not.
- The path length an issuer allows below itself.
- Name constraints an issuer imposes, for DNS names and IP addresses - the only names this library asserts. Constraints on other name forms are not applied.
- Extended key usage, when present: the leaf and every intermediate must allow the use you asked for.
- That no critical extension was left unread.

Accepted algorithms. Signatures: RSA PKCS #1 v1.5 and RSA-PSS with SHA-256, SHA-384 or SHA-512, over keys of 2048 to 8192 bits; ECDSA over P-256 and P-384 with the same three hashes; Ed25519. A signature with SHA-1 or MD5, an RSA key below 2048 bits, and any other curve are refused, with the fault `PROVEN_CERT_FAULT_ALGORITHM` or `_BAD_SIGNATURE`. The anchor's *own* signature is not checked - an anchor is trusted because it is in the store - so an old root signed with SHA-1 still anchors chains.

Bounds. A path is at most `PROVEN_CERT_MAX_DEPTH` (10) certificates. At most 16 certificates beyond the leaf are considered, and the search stops after 64 signature checks: a peer cannot make verification expensive by sending a tangle.

Not here, and you should plan around it:

- **Revocation.** No CRL, no OCSP, no check of a stapled response. A certificate that was valid and has since been revoked verifies. Short-lived certificates are the practical defence.
- **Certificate Transparency.** No check that a certificate was publicly logged.
- **Certificate policies.** The policy extensions are recognised so that a certificate carrying them is not refused, and are not enforced.
- **Internationalised names.** Compared as ASCII: supply the `xn--` form.
- **Private keys.** Nothing here reads or holds one.

How this was tested. Certificate chains built by another implementation, one for every acceptance and refusal listed above; chains saved from public servers, against the system's roots; and for the signature algorithms underneath, Project Wycheproof - a collection of inputs built to break implementations in the ways implementations are known to break. The catalog in `TEST.md` says what each registered test covers.

Chapter 14: TLS

Part V - Talking to the operating system. Prerequisites: Chapter 13 (certificates and trust anchors); Chapter 11 for HTTPS, Chapter 12 for `wss`. After this chapter you can serve and fetch over HTTPS, open a `wss` connection, wrap any connection of your own in TLS, drive the protocol yourself where there is no socket - and say what your program believes about its peer, and why.

This chapter covers `tls.h`. The engine in it calls no operating system and is available in a freestanding build; the transport wrapper and everything that touches a socket are hosted-only, and `PROVEN_NO_NET` leaves them out.

Read this first. This is a new implementation of TLS with no external audit. It is tested against the protocol's published example handshakes, against two other implementations in both roles, and against collections of adversarial inputs (section 9 says exactly what, and what was not possible). That is evidence, not proof. A program whose users depend on TLS against a capable adversary should terminate it in something audited and use this library behind it.

Table of contents

1. What TLS does, in one page
2. A configuration
3. HTTPS and wss: the two lines
4. Any connection: the transport wrapper
5. The engine
6. Resumption
7. Client certificates and pins
8. When it fails
9. What is negotiated, what is held, what is not here
10. The legacy set: what each setting exposes you to

1. What TLS does, in one page

A TCP connection gives you a pipe. Everyone between the two ends - the cafe's access point, an internet provider, whoever compromised a router - can read what goes through it and change it, and neither end can tell. TLS (Transport Layer Security; this library speaks version 1.3, RFC 8446, and a cut-down version 1.2, RFC 5246 - section 9 says which part; older parts exist only for a configuration that asks for them by name, section 10) puts three properties on top of the pipe:

Property	Meaning	What it is made from
Authentication	The other end is who you meant to reach	Its certificate, checked as Chapter 13 describes, and a signature proving it holds the certificate's key
Confidentiality	Nobody in between can read the data	Keys agreed by a key exchange that those in between can watch and still not compute
Integrity	Nobody in between can change, drop or reorder the data unnoticed	Every record carries a tag only a key holder could have made; a clean end is itself a message

It happens in two phases. The **handshake** is a few messages at the start: the two sides agree on algorithms, exchange key shares, the server (and optionally the client) presents its certificate and proves it owns the key, and both derive the same secret keys. After that **records** carry your data, encrypted and tagged.

Three consequences are worth knowing before you write a line:

- **A name is half of it.** Encryption to an unverified peer is encryption to whoever answered. The client must know which name it meant to reach, and the certificate must be for that name.
- **A clean close matters.** If the connection just stops, TLS cannot tell "the peer finished" from "someone cut the wire to hide the rest". So closing is a message, and a connection that ends without it is reported as an error, not as an end of data.
- **It costs.** A handshake is public-key arithmetic. Section 9 has this implementation's numbers, and they are not small.

2. A configuration

```
proven_err_t proven_tls_config_create(const proven_tls_options_t *options, proven_tls_config_t **out);
void proven_tls_config_destroy(proven_tls_config_t *config);
```

A `proven_tls_config_t` holds everything that is the same for every connection: whom to believe, who you are, which application protocols you speak. **It is built once and never changes**, so one config is shared by every connection and every thread. `proven_tls_config_create` copies what it is given - except the anchor store, which must outlive the config - and `proven_tls_config_destroy` wipes the private key as it frees it.

Zero-initialise a `proven_tls_options_t`, set `alloc`, and set what applies:

Field	For	Meaning
`alloc`	both	Required. Connections made from this config allocate through it too
`anchors`	whoever verifies	The trust anchors (Chapter 13). A client needs them; a server needs them only to check client certificates
`verify`	whoever verifies	`PROVEN_TLS_VERIFY_CHAIN` (the default): a chain to the anchors, valid now, for the expected name. `PROVEN_TLS_VERIFY_PIN_ONLY`: section 7
`pins`, `pin_count`	whoever verifies	Public-key pins (section 7)
`certificate_pem`, `private_key_pem`	a server; a client with a certificate	This side's certificates, its own first, and its key. The key is P-256, Ed25519, or RSA of 2048 to 4096 bits, as a PKCS #8 block (labelled `PRIVATE KEY`), an `EC PRIVATE KEY` block or an `RSA PRIVATE KEY` block, not encrypted
`client_auth`	a server	Whether to ask clients for a certificate (section 7)
`alpn`, `alpn_count`	both	Application protocol names, most preferred first (`http/1.1`). The server picks its first choice among what the client offers; if both sides have a list and nothing is common, the handshake fails

`random`, `now`	both	Where unpredictable bytes and the time come from. Leave them null on a hosted system: the operating system's source and the wall clock are used. A freestanding build must supply both
`no_resumption`, `ticket_lifetime_s`	both, a server	Section 6
`early_data`	both: HTTP/3, and HTTP/1.1 and HTTP/2 over TCP	Section 6, "Early data"
`keep_peer_certificate`	whoever verifies	Keep the peer's certificate after the handshake, for `proven_tls_peer_certificate`
`max_handshake_bytes`	both	The largest handshake message accepted. Default 65,536
`min_version`, `max_version`	both	The oldest and newest protocol version this side agrees to: `PROVEN_TLS_VERSION_1_2` or `PROVEN_TLS_VERSION_1_3`. Zero is the default - 1.2 for the minimum, 1.3 for the maximum. `min_version = PROVEN_TLS_VERSION_1_3` **is how to refuse TLS 1.2** (section 9). `PROVEN_TLS_VERSION_1_1` and `PROVEN_TLS_VERSION_1_0` are legacy (section 10)
`legacy`	both	`PROVEN_TLS_LEGACY_*` bits: parts of TLS that are left out unless named here. Zero, the default, is none of them. **Read section 10 before setting one**

`anchors`, `verify` **and** `pins` **describe how the peer is verified, in either role.** There is no setting that verifies nothing. A client that would check a chain and is given no anchors is refused when the config is made; a test that wants to skip the public authorities makes its own root, or pins a key.

Everything that can be wrong with the options is an error here, not at the first handshake, which is at three in the morning on somebody else's machine:

Returns	When
`PROVEN_ERR_INVALID_ARG`	No allocator; anchors missing where this side verifies a chain; `PIN_ONLY` with no pins; a certificate without a key, or a key without a certificate; more than 8 certificates; an empty ALPN name; a version that does not exist, or a minimum above the maximum; a minimum below TLS 1.2 without `PROVEN_TLS_LEGACY_CBC`; a `legacy` bit that does not exist
`PROVEN_ERR_INVALID_FORMAT`	PEM that does not parse, or holds no certificate, or no key; an RSA key outside 2048 to 4096 bits, or whose parts do not belong together, or that does not sign
`PROVEN_ERR_UNSUPPORTED`	A key of another kind - P-384, for one - or an encrypted key file

PROVEN_ERR_INVALID_STATE	The key is not the one in the certificate
PROVEN_ERR_NOMEM	The allocator refused

Which key to choose. All three kinds work on either side of a connection. They do not cost the same: with an RSA key, every full handshake this side signs takes longer than with P-256 - about five times as long with 2048 bits, thirty-five times with 4096 - and Ed25519 is a little cheaper than P-256 (section 9 has the figures). On a server the signature is the largest part of a handshake. Use RSA where a certificate you already have requires it; for a new one, P-256 or Ed25519. An RSA key is tried when the configuration is made - one signature, checked - so a key file that is damaged is refused there and not at the first connection.

An identity without an authority. The library reads an identity; it does not make one. A self-signed certificate and its key come from a tool - with OpenSSL, for the names you will be reached by:

```
openssl req -x509 -newkey ed25519 -nodes -days 30 -subj /CN=example.test \
    -addext subjectAltName=DNS:example.test,IP:127.0.0.1 -keyout key.pem -out cert.pem
```

`cert.pem` goes in `certificate_pem` and `key.pem` (PKCS #8) in `private_key_pem`, read from the files as both examples below do. `-newkey ec -pkeyopt ec_paramgen_curve:P-256` or `-newkey rsa:2048` make the other two kinds. Until v0.78.0 the library made such a pair itself (`proven_tls_self_signed`); the function is gone from the public API, because an identity a program invents for itself at each start is rarely what anyone should deploy.

Nobody vouches for a self-signed certificate. A peer believes it only by already having it: put the certificate in the peer's anchor store, or give the peer the key's pin and `PROVEN_TLS_VERIFY_PIN_ONLY` (section 7). That is the right arrangement between machines you run yourself, and for development and tests; it is not a way to be reached by the public, whose browsers trust authorities and nothing else. The key is yours to store safely and to wipe.

3. HTTPS and wss: the two lines

For HTTP nothing changes but the configuration.

A client gets `https` URLs by being given the wrap function this library provides and a config as its context:

```
proven_err_t proven_tls_http_wrap(void *config, proven_transport_t plain, proven_u8str_view_t host,
                                  proven_net_deadline_t until, proven_transport_t *out);
```

Set `tls_wrap = proven_tls_http_wrap` and `tls_ctx` to your `proven_tls_config_t *` in the `proven_http_client_config_t` of Chapter 11. From then on a request for an `https` URL connects, runs the handshake for the URL's host - through a proxy tunnel if there is a proxy - and proceeds; a `wss` URL given to `proven_ws_conn_connect` does the same. For the public internet the config's anchors are the system's (`proven_cert_store_add_system`). A server that cannot be verified is `PROVEN_ERR_UNTRUSTED`, `PROVEN_ERR_EXPIRED`, `PROVEN_ERR_NOT_YET_VALID` or `PROVEN_ERR_NAME_MISMATCH` from the request, **and nothing of the request was sent**.

A server speaks TLS when its `proven_http_server_config_t` has `tls` set to a config with a certificate. Every connection it accepts is then TLS; a client that sends plain HTTP to that port is dropped. Handlers do not change: by the time one runs, the request is plain text, and what it writes is encrypted on the way out. A WebSocket accepted from such a handler is `wss`.

The handshake is driven by the server's own loop as the client's bytes arrive. It never waits on one connection, so a client that connects and says nothing costs a slot until `head_timeout_ms` - which a new connection's handshake shares with its first request's head - and holds up nobody.

To serve both `http` and `https`, run two servers; a server is one or the other.

Compiled, and run by the test suite with a certificate and key it makes for it (`tests/test_docs_tls_example_https.c`):

```
#include <stdio.h>
#include <string.h>

/*
 * HTTPS, both ends, in one program: the HTTP server of chapter 11 with a TLS config, and the
 * HTTP client of chapter 11 with the TLS wrap. Nothing else about either changes.
 *
 * The server is on another thread on the loopback interface, so the example needs no network.
 */

/* There is no certificate in this file. The server's identity is read from two PEM files named
 * on the command line: a certificate for 127.0.0.1 and its key. Run with none, the program says
 * how to make a self-signed pair with OpenSSL. The client is given that certificate as the one
 * thing it believes. */

/* What to run when no files are given. */
static const char *const HOW_TO_MAKE =
    "  openssl req -x509 -newkey ed25519 -nodes -days 30 -subj /CN=example.test \\\n"
    "      -addext subjectAltName=DNS:example.test,IP:127.0.0.1 -keyout key.pem -out cert.pem\n";

typedef struct {
    proven_http_server_t *server;
} ex_app_t;

static void handle(void *ctx, proven_http_exchange_t *x) {
    ex_app_t *app = ctx;
    const proven_http_request_t *req = proven_http_exchange_request(x);
    /* The handler is the same as without TLS: by the time it runs, the request is plain text. */
    if (proven_u8str_view_eq(req->target, PROVEN_LIT("/quit"))) {
        (void)proven_http_exchange_respond(x, 200, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("bye\n")));
        proven_http_server_stop(app->server);
        return;
    }
    (void)proven_http_exchange_respond(x, 200, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("hello over
TLS\n")));
}

static void serve(void *arg) {
    (void)proven_http_server_run(((ex_app_t *)arg)->server);
}

int main(int argc, char **argv) {
    proven_allocator_t heap = proven_heap_allocator();

    // ---- an identity ------------------------------------------------------------
    /* A certificate for the address this example serves on, and its key, from files. A real
     * server is given a certificate from an authority; a pair of machines you run yourself can
     * use a self-signed one, or a pin (chapter 14, section 7). */
    if (argc < 3) {
        printf("usage: %s CERTIFICATE.pem KEY.pem\nA self-signed pair for 127.0.0.1:\n%s", argv[0],
HOW_TO_MAKE);
        return 0;
    }
    proven_result_mem_mut_t cert = proven_fs_read_all_bounded(heap, proven_u8str_view_from_cstr(argv[1]), 64 *
1024);
    proven_result_mem_mut_t key = proven_fs_read_all_bounded(heap, proven_u8str_view_from_cstr(argv[2]), 16 *
1024);
```

```
    EXAMPLE_REQUIRE(cert.err == PROVEN_OK && key.err == PROVEN_OK, "the certificate and the key are read");
    if (cert.err != PROVEN_OK || key.err != PROVEN_OK) return 1;
    const proven_byte_t *cert_pem = cert.value.ptr, *key_pem = key.value.ptr;
    const proven_size_t cert_len = cert.value.size, key_len = key.value.size;

    // ---- the server ----------------------------------------------------------
    /* One field makes a server speak TLS: its config's `tls`. The TLS config holds the
     * certificate the server presents and the key that proves it is the server's. */
    proven_tls_options_t server_options = {
        .alloc = heap,
        .certificate_pem = { cert_pem, cert_len },
        .private_key_pem = { key_pem, key_len },
    };
    proven_tls_config_t *server_tls = NULL;
    EXAMPLE_REQUIRE(proven_tls_config_create(&server_options, &server_tls) == PROVEN_OK, "the server's TLS
configuration");

    ex_app_t app = { 0 };
    proven_http_server_config_t server_config = { .alloc = heap, .handler = handle, .handler_ctx = &app, .tls
= server_tls };
    proven_net_addr_t at;
    EXAMPLE_REQUIRE(proven_http_server_create(&server_config, &app.server) == PROVEN_OK &&
                    proven_http_server_listen(app.server, proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0),
&at) == PROVEN_OK, "an HTTPS server on a free port");
    proven_job_sys_t *threads = NULL;
    proven_job_group_t running;
    proven_job_group_init(&running);
    EXAMPLE_REQUIRE(proven_job_system_init(heap, 1, 4, &threads) == PROVEN_OK &&
                    proven_job_group_submit(threads, &running, serve, &app) == PROVEN_OK, "its loop runs on
another thread");

    // ---- the client ----------------------------------------------------------
    /* The client's TLS config says whom to believe: here, the certificate just made. For the
     * public internet it would be proven_cert_store_add_system instead. */
    proven_cert_store_t *anchors = NULL;
    EXAMPLE_REQUIRE(proven_cert_store_create(heap, &anchors) == PROVEN_OK &&
                    proven_cert_store_add_pem(anchors, (proven_mem_view_t){ cert_pem, cert_len }, NULL) ==
PROVEN_OK,
                    "the anchors");
    proven_tls_options_t client_options = { .alloc = heap, .anchors = anchors };
    proven_tls_config_t *client_tls = NULL;
    EXAMPLE_REQUIRE(proven_tls_config_create(&client_options, &client_tls) == PROVEN_OK, "the client's TLS
configuration");

    /* Two fields make a client speak HTTPS: the wrap function this library provides, and the
     * TLS config as its context. */
    proven_http_client_config_t client_config = { .alloc = heap, .max_idle_connections = 1, .tls_wrap =
proven_tls_http_wrap, .tls_ctx = client_tls };
    proven_http_client_t *client = NULL;
    EXAMPLE_REQUIRE(proven_http_client_create(&client_config, &client) == PROVEN_OK, "an HTTP client");

    char url_text[64];
    int url_len = snprintf(url_text, sizeof url_text, "https://127.0.0.1:%u/", (unsigned)at.port);
    proven_u8str_view_t url = { (const proven_byte_t *)url_text, (proven_size_t)url_len };

    proven_http_client_response_t response;
    proven_u8str_t body = { 0 };
    EXAMPLE_REQUIRE(proven_http_client_get(client, url, &response) == PROVEN_OK && response.status == 200,
"GET over https: 200");
    EXAMPLE_REQUIRE(proven_http_client_read_all(&response, heap, &body, 4096) == PROVEN_OK &&
```

```
                    proven_u8str_view_eq(proven_u8str_as_view(&body), PROVEN_LIT("hello over TLS\n")), "and
the body, which crossed the wire encrypted");
    proven_u8str_destroy(heap, &body);
    proven_http_client_finish(&response);

    // ---- a client with nothing to believe ----------------------------------------
    /* There is no setting that verifies nothing. A client that would check a chain must be given
     * anchors, and one given an empty store believes no server at all. */
    proven_tls_options_t stranger_options = { .alloc = heap, .anchors = NULL };
    proven_tls_config_t *no_anchors = NULL;
    EXAMPLE_REQUIRE(proven_tls_config_create(&stranger_options, &no_anchors) == PROVEN_ERR_INVALID_ARG, "no
anchors at all is refused when the config is made");

    proven_cert_store_t *empty = NULL;
    EXAMPLE_REQUIRE(proven_cert_store_create(heap, &empty) == PROVEN_OK, "an empty store");
    stranger_options.anchors = empty;
    EXAMPLE_REQUIRE(proven_tls_config_create(&stranger_options, &no_anchors) == PROVEN_OK, "a config that
trusts nobody");
    proven_http_client_config_t wary_config = { .alloc = heap, .tls_wrap = proven_tls_http_wrap, .tls_ctx =
no_anchors };
    proven_http_client_t *wary = NULL;
    EXAMPLE_REQUIRE(proven_http_client_create(&wary_config, &wary) == PROVEN_OK, "an HTTP client");
    EXAMPLE_REQUIRE(proven_http_client_get(wary, url, &response) == PROVEN_ERR_UNTRUSTED, "the same server is
PROVEN_ERR_UNTRUSTED, and no request was sent to it");
    proven_http_client_finish(&response);
    proven_http_client_destroy(wary);

    // ---- done --------------------------------------------------------------------
    url_len = snprintf(url_text, sizeof url_text, "https://127.0.0.1:%u/quit", (unsigned)at.port);
    EXAMPLE_REQUIRE(proven_http_client_get(client, (proven_u8str_view_t){ (const proven_byte_t *)url_text,
(proven_size_t)url_len }, &response) == PROVEN_OK, "the server is asked to stop");
    proven_http_client_finish(&response);
    proven_job_group_wait(threads, &running);

    proven_http_client_destroy(client);
    proven_http_server_destroy(app.server);
    proven_job_system_close(threads);
    proven_job_system_destroy(threads);
    proven_tls_config_destroy(client_tls);
    proven_tls_config_destroy(no_anchors);
    proven_tls_config_destroy(server_tls);
    proven_cert_store_destroy(anchors);
    proven_cert_store_destroy(empty);
    proven_mem_wipe(key.value);
    heap.free_fn(heap.ctx, key.value.ptr);
    heap.free_fn(heap.ctx, cert.value.ptr);
    return EXAMPLE_OK();
}
```

4. Any connection: the transport wrapper

```
proven_err_t proven_tls_transport_client(proven_transport_t plain, const proven_tls_config_t *config,
                                         proven_u8str_view_t server_name, proven_tls_session_t *session,
                                         proven_net_deadline_t until, proven_transport_t *out);
proven_err_t proven_tls_transport_server(proven_transport_t plain, const proven_tls_config_t *config,
                                         proven_net_deadline_t until, proven_transport_t *out);
proven_tls_conn_t *proven_tls_transport_conn(proven_transport_t tls);
```

A `proven_transport_t` (Chapter 9) is a connection as an interface: read, write, shut down, close. `proven_tls_transport_client` and `proven_tls_transport_server` take one, run the handshake over it within

`until`, and return another - which is what the HTTP wrap in section 3 is made of, and what you use for a protocol of your own.

- **On success the new transport owns the old one.** Closing it says a TLS goodbye and closes the connection underneath. Do not use `plain` again.
- **On failure `plain` is untouched and still yours** to close.
- `server_name` is what the client meant to reach: it is sent to the server and checked against its certificate.
- A read that returns `PROVEN_ERR_EOF` means the peer closed properly. A connection that ends any other way is `PROVEN_ERR_RESET`: do not treat what you have as complete.
- `proven_tls_transport_conn` gives the engine inside, for the questions of section 5 (which suite, which protocol, whose key). It returns null for a transport that is not one of these.

Reads and writes take deadlines as every transport's do. A read may have to write (the protocol occasionally owes the peer a message) and a write never has to read.

5. The engine

Under the wrapper is a state machine with no I/O in it. You give it the bytes that arrived; it gives you the bytes to send. Use it directly when the transport is yours - an event loop, a serial line, a memory buffer in a test - or when there is no operating system at all.

```
proven_err_t proven_tls_client_create(const proven_tls_config_t *config, proven_u8str_view_t server_name,
                                      proven_tls_session_t *session, proven_tls_conn_t **out);
proven_err_t proven_tls_server_create(const proven_tls_config_t *config, proven_tls_conn_t **out);
void proven_tls_conn_destroy(proven_tls_conn_t *conn);

proven_err_t proven_tls_feed(proven_tls_conn_t *conn, proven_mem_view_t in, proven_size_t *consumed);
proven_mem_view_t proven_tls_pending_output(proven_tls_conn_t *conn);
void proven_tls_output_sent(proven_tls_conn_t *conn, proven_size_t n);

bool proven_tls_is_established(const proven_tls_conn_t *conn);
proven_result_size_t proven_tls_read(proven_tls_conn_t *conn, proven_mem_mut_t dest);
proven_result_size_t proven_tls_write(proven_tls_conn_t *conn, proven_mem_view_t src);
proven_err_t proven_tls_close(proven_tls_conn_t *conn);
```

The whole of using it is one loop:

1. **Whatever `proven_tls_pending_output` shows, send**, and tell it how much went with `proven_tls_output_sent`. A client has output the moment it is created.
2. **Whatever arrives, give to `proven_tls_feed`.** It need not be a whole record or only one; incomplete records are kept inside.
3. **After every feed, call `proven_tls_read` until it says `PROVEN_ERR_NEED_MORE`.**
4. Once `proven_tls_is_established`, `proven_tls_write` encrypts your data into the pending output. Go to 1.

Two rules in that loop are easy to get wrong:

- `proven_tls_feed` **may consume less than you gave it.** When it has decrypted application data that you have not read yet, it stops and sets `*consumed` to what it took. Read, then feed the rest. A caller that discards the unconsumed part loses data.
- **The view from `proven_tls_pending_output` is good only until the next call** on that connection. Send from it or copy it; do not hold it across a feed.

`proven_tls_read` returns `PROVEN_ERR_EOF` once the peer has closed properly and everything before the close was read. `proven_tls_close` puts this side's goodbye in the pending output; after it nothing more can be written, and reading continues until the peer closes too. `proven_tls_write` takes everything unless more than 64 KiB of output is already waiting to be sent, in which case it takes what fits - possibly nothing - and you send some first.

A connection is used by one thread at a time. A config, and the anchor store under it, may be used by any number.

What was agreed, once established:

```
proven_u16 proven_tls_version(const proven_tls_conn_t *conn);
proven_u16 proven_tls_cipher_suite(const proven_tls_conn_t *conn);
proven_u8str_view_t proven_tls_alpn(const proven_tls_conn_t *conn);
bool proven_tls_resumed(const proven_tls_conn_t *conn);
bool proven_tls_peer_key_sha256(const proven_tls_conn_t *conn, proven_byte_t out[32]);
proven_mem_view_t proven_tls_peer_certificate(const proven_tls_conn_t *conn);
proven_u8str_view_t proven_tls_server_name(const proven_tls_conn_t *conn);
proven_err_t proven_tls_key_update(proven_tls_conn_t *conn);
```

- `proven_tls_version` is `PROVEN_TLS_VERSION_1_3` or `PROVEN_TLS_VERSION_1_2` (or one of the two legacy versions, for a configuration that allowed them), and zero until the peer's hello has settled it. A program that must not carry something over TLS 1.2 checks here, or - better - sets `min_version` and never gets that far.
- `proven_tls_cipher_suite` is the suite's number (section 9); `proven_tls_alpn` the agreed application protocol, empty when none was.
- `proven_tls_peer_key_sha256` gives the hash of the peer's public key - the value a pin holds
  - and returns `false` when the peer presented no certificate. The certificate itself is not kept after the handshake unless the config set `keep_peer_certificate`, in which case `proven_tls_peer_certificate` returns it for `proven_cert_parse`.
- `proven_tls_server_name` is, in the server role and during the handshake only, the name the client asked for.
- `proven_tls_key_update` replaces this side's sending key with the next one. The engine does this by itself long before a key has protected too much; call it when policy wants it sooner. TLS 1.2 has no such message: there it returns `PROVEN_ERR_UNSUPPORTED`, and a connection that has sent 2^31 records refuses to write more (`PROVEN_ERR_OVERFLOW`) - open another.

Compiled, and run by the test suite with a certificate and key it makes for it (`tests/test_docs_tls_example_engine.c`):

```
#include <string.h>

/*
 * The TLS engine with no network in sight: a client and a server connection in one program,
 * with this file carrying the bytes from one to the other. That is all a network does for them.
 *
 * It is the layer under the transport wrapper. Use it directly when the transport is yours:
 * an event loop, a serial line, a test.
 */

/* There is no certificate in this file. The server's identity is read from two PEM files named
 * on the command line: a certificate for example.test and its key. Run with none, the program
 * says how to make a self-signed pair with OpenSSL. A certificate nobody vouches for is believed
 * only by a peer that already holds it, so the client is given it as its anchor. */

/* What to run when no files are given. */
static const char *const HOW_TO_MAKE =
    "  openssl req -x509 -newkey ed25519 -nodes -days 30 -subj /CN=example.test \\\n"
    "      -addext subjectAltName=DNS:example.test,IP:127.0.0.1 -keyout key.pem -out cert.pem\n";

/* Hand everything one side wants sent to the other. The receiver may take less than it was
 * given when it has application data waiting to be read; what it did not take stays pending. */
static proven_err_t carry(proven_tls_conn_t *from, proven_tls_conn_t *to) {
    proven_mem_view_t out = proven_tls_pending_output(from);
    proven_size_t at = 0;
```

```
    while (at < out.size) {
        proven_size_t used = 0;
        proven_err_t e = proven_tls_feed(to, (proven_mem_view_t){ out.ptr + at, out.size - at }, &used);
        if (e != PROVEN_OK) return e;
        if (used == 0) break;                  /* it wants its application data read first */
        at += used;
    }
    proven_tls_output_sent(from, at);
    return PROVEN_OK;
}

/* True when the bytes of `text` appear, in order, somewhere in `data`. */
static bool appears(proven_mem_view_t data, const char *text) {
    proven_size_t n = strlen(text);
    for (proven_size_t i = 0; i + n <= data.size; ++i) {
        if (memcmp(data.ptr + i, text, n) == 0) return true;
    }
    return false;
}

int main(int argc, char **argv) {
    proven_allocator_t heap = proven_heap_allocator();

    // ---- an identity -------------------------------------------------------
    /* A real server is given a certificate from an authority; a pair of machines you run
     * yourself can use a self-signed one. Either way it arrives as files. The key is the
     * server's secret: read it, use it, wipe it. */
    if (argc < 3) {
        printf("usage: %s CERTIFICATE.pem KEY.pem\nA self-signed pair for example.test:\n%s", argv[0],
HOW_TO_MAKE);
        return 0;
    }
    proven_result_mem_mut_t cert = proven_fs_read_all_bounded(heap, proven_u8str_view_from_cstr(argv[1]), 64 *
1024);
    proven_result_mem_mut_t key = proven_fs_read_all_bounded(heap, proven_u8str_view_from_cstr(argv[2]), 16 *
1024);
    EXAMPLE_REQUIRE(cert.err == PROVEN_OK && key.err == PROVEN_OK, "the certificate and the key are read");
    if (cert.err != PROVEN_OK || key.err != PROVEN_OK) return 1;
    const proven_byte_t *cert_pem = cert.value.ptr, *key_pem = key.value.ptr;
    const proven_size_t cert_len = cert.value.size, key_len = key.value.size;

    // ---- two configurations ------------------------------------------------
    /* A config says whom to believe and who you are. It is built once and shared by every
     * connection. The client believes the certificate just made; the server is that
     * certificate with its key. */
    proven_cert_store_t *anchors = NULL;
    EXAMPLE_REQUIRE(proven_cert_store_create(heap, &anchors) == PROVEN_OK &&
                    proven_cert_store_add_pem(anchors, (proven_mem_view_t){ cert_pem, cert_len }, NULL) ==
PROVEN_OK,
                    "the anchors");

    proven_u8str_view_t protocols[1] = { PROVEN_LIT("example/1") };
    /* The engine reads no clock of its own: `now` gives it one. Left out, as here, it is the
     * system's - the certificate is checked against today. */
    proven_tls_options_t client_options = { .alloc = heap, .anchors = anchors, .alpn = protocols, .alpn_count
= 1 };
    proven_tls_options_t server_options = {
        .alloc = heap, .alpn = protocols, .alpn_count = 1,
        .certificate_pem = { cert_pem, cert_len },
        .private_key_pem = { key_pem, key_len },
```

```
    };
    proven_tls_config_t *client_config = NULL, *server_config = NULL;
    EXAMPLE_REQUIRE(proven_tls_config_create(&client_options, &client_config) == PROVEN_OK, "a client
configuration");
    EXAMPLE_REQUIRE(proven_tls_config_create(&server_options, &server_config) == PROVEN_OK, "a server
configuration: its key matches its certificate");

    // ---- the handshake ----------------------------------------------------------
    /* A client connection has its first message ready the moment it exists. A server
     * connection waits. The session is where a ticket for next time will be put. */
    proven_tls_session_t session = { 0 };
    proven_tls_conn_t *client = NULL, *server = NULL;
    EXAMPLE_REQUIRE(proven_tls_client_create(client_config, PROVEN_LIT("example.test"), &session, &client) ==
PROVEN_OK, "a client");
    EXAMPLE_REQUIRE(proven_tls_server_create(server_config, &server) == PROVEN_OK, "a server");
    EXAMPLE_REQUIRE(proven_tls_pending_output(client).size > 0 && proven_tls_pending_output(server).size == 0,
"the client speaks first");

    /* Carry bytes back and forth until both are done. Two round trips do it. */
    for (int round = 0; round < 10 && !(proven_tls_is_established(client) &&
proven_tls_is_established(server)); ++round) {
        EXAMPLE_REQUIRE(carry(client, server) == PROVEN_OK, "client to server");
        EXAMPLE_REQUIRE(carry(server, client) == PROVEN_OK, "server to client");
    }
    EXAMPLE_REQUIRE(proven_tls_is_established(client) && proven_tls_is_established(server), "both sides are
established");
    EXAMPLE_REQUIRE(proven_tls_version(client) == PROVEN_TLS_VERSION_1_3 && proven_tls_version(server) ==
PROVEN_TLS_VERSION_1_3, "both spoke TLS 1.3, the newest version both sides have");
    EXAMPLE_REQUIRE(proven_tls_cipher_suite(client) == proven_tls_cipher_suite(server) &&
proven_tls_cipher_suite(client) != 0, "they agreed on a cipher suite");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_tls_alpn(client), PROVEN_LIT("example/1")), "and on the
application protocol");
    EXAMPLE_REQUIRE(!proven_tls_resumed(client), "this was a full handshake");

    // ---- application data -------------------------------------------------------
    /* Writing encrypts into the pending output; reading gives what feeding decrypted. */
    proven_result_size_t wrote = proven_tls_write(client, proven_mem_view_from_u8(PROVEN_LIT("hello,
server")));
    EXAMPLE_REQUIRE(wrote.err == PROVEN_OK && wrote.value == 13, "thirteen bytes taken");
    proven_mem_view_t wire = proven_tls_pending_output(client);
    EXAMPLE_REQUIRE(wire.size > 13 && !appears(wire, "hello, server"), "what goes on the wire is a record, and
not the text");
    EXAMPLE_REQUIRE(carry(client, server) == PROVEN_OK, "client to server");
    proven_byte_t buf[64];
    proven_result_size_t got = proven_tls_read(server, (proven_mem_mut_t){ buf, sizeof buf });
    EXAMPLE_REQUIRE(got.err == PROVEN_OK && got.value == 13 && memcmp(buf, "hello, server", 13) == 0, "the
server reads what the client wrote");
    EXAMPLE_REQUIRE(proven_tls_read(server, (proven_mem_mut_t){ buf, sizeof buf }).err ==
PROVEN_ERR_NEED_MORE, "and then there is nothing more yet");

    // ---- closing ----------------------------------------------------------------
    /* A close is a message too: it tells the peer that the end is the end and not a cut wire. */
    EXAMPLE_REQUIRE(proven_tls_close(client) == PROVEN_OK && carry(client, server) == PROVEN_OK, "the close
travels");
    EXAMPLE_REQUIRE(proven_tls_read(server, (proven_mem_mut_t){ buf, sizeof buf }).err == PROVEN_ERR_EOF, "and
the server's read reports a proper end");
    proven_tls_conn_destroy(client);
    proven_tls_conn_destroy(server);

    // ---- resumption -------------------------------------------------------------
```

```
    /* The first connection left a ticket in the session. A second connection that is given the
     * same session offers it, and the handshake skips the certificates and one signature. */
    EXAMPLE_REQUIRE(proven_tls_client_create(client_config, PROVEN_LIT("example.test"), &session, &client) ==
PROVEN_OK &&
                    proven_tls_server_create(server_config, &server) == PROVEN_OK, "a second pair");
    for (int round = 0; round < 10 && !(proven_tls_is_established(client) &&
proven_tls_is_established(server)); ++round) {
        EXAMPLE_REQUIRE(carry(client, server) == PROVEN_OK && carry(server, client) == PROVEN_OK, "client to
server");
    }
    EXAMPLE_REQUIRE(proven_tls_resumed(client) && proven_tls_resumed(server), "both sides resumed");
    proven_tls_conn_destroy(client);
    proven_tls_conn_destroy(server);

    // ---- a server that is not the one asked for -----------------------------------
    /* The same server, reached under a name its certificate does not carry. The client refuses,
     * and says precisely why. */
    proven_tls_conn_t *wrong = NULL;
    EXAMPLE_REQUIRE(proven_tls_client_create(client_config, PROVEN_LIT("elsewhere.test"), NULL, &wrong) ==
PROVEN_OK &&
                    proven_tls_server_create(server_config, &server) == PROVEN_OK, "a second pair");
    EXAMPLE_REQUIRE(carry(wrong, server) == PROVEN_OK, "client to server");
    EXAMPLE_REQUIRE(carry(server, wrong) == PROVEN_ERR_NAME_MISMATCH, "the client's feed returns
PROVEN_ERR_NAME_MISMATCH");
    EXAMPLE_REQUIRE(proven_tls_peer_fault(wrong) == PROVEN_CERT_FAULT_NAME_MISMATCH && !
proven_tls_is_established(wrong), "with the precise fault, and no connection");
    /* A failure leaves an alert in the pending output. Send it: it is how the peer learns. */
    EXAMPLE_REQUIRE(proven_tls_pending_output(wrong).size > 0 && carry(wrong, server) == PROVEN_ERR_PROTOCOL,
"the server is told");
    EXAMPLE_REQUIRE(proven_tls_alert_received(server) == 42, "bad_certificate, by its number");
    proven_tls_conn_destroy(wrong);
    proven_tls_conn_destroy(server);

    proven_mem_wipe((proven_mem_mut_t){ session.opaque, sizeof session.opaque });
    proven_mem_wipe(key.value);
    heap.free_fn(heap.ctx, key.value.ptr);
    heap.free_fn(heap.ctx, cert.value.ptr);
    proven_tls_config_destroy(client_config);
    proven_tls_config_destroy(server_config);
    proven_cert_store_destroy(anchors);
    return EXAMPLE_OK();
}
```

6. Resumption

A full handshake costs two signatures and a certificate chain. A client that has talked to a server before can skip those: the server gave it a **ticket** at the end of the first handshake, and a second connection that presents the ticket proves with it that both sides already share a secret. A fresh key exchange still happens, so the new connection's keys are its own.

On the client it is one struct:

```
typedef struct { alignas(8) proven_byte_t opaque[PROVEN_TLS_SESSION_SIZE]; } proven_tls_session_t;   /* 2,048
bytes */
```

Zero-initialise a `proven_tls_session_t`, and pass it to `proven_tls_client_create` (or to `proven_tls_transport_client`) for each connection to the same server name. The connection writes a ticket into it when the server sends one, and the next connection offers it; `proven_tls_resumed` says whether the server accepted. It is a plain block of bytes with no pointers in it - keep it wherever you like, for as long as you like - and **it holds a secret**: wipe it with `proven_mem_wipe` when you are

done. It is offered only to the name it was made for, and never past the lifetime the server gave it. One session serves one connection at a time.

On the server there is nothing to do. Tickets are issued unless `no_resumption` is set, and honoured for `ticket_lifetime_s` (two hours by default, a week at most). **The server stores nothing per client**: a ticket is the session's state sealed under a key that only this process holds. That key is made when the config is, replaced every ticket lifetime, kept one lifetime longer so that outstanding tickets still open, and never written anywhere - so tickets do not survive a restart, and two processes do not accept each other's.

What resumption does not do here:

- **Early data only where its owner asks** (next part): the HTTP servers and clients of this library, with `early_data`. A program that drives the engine itself, or the transport wrapper as a server, gets none.
- **Through `proven_tls_http_wrap` only with `early_data`.** A client configuration that has it keeps the newest session of each server name (eight names), and the transport wrapper resumes with it when it is given no session; without it the HTTP client's connections do not resume, and it reuses open connections instead.
- A resumed connection's peer is the one verified when the session was made: the certificate is not sent again, so it is not checked again, and an expiry that has passed since is not noticed. The ticket lifetime bounds how long that can matter.

Under TLS 1.2 a resumed connection is weaker than a new one, and you should know how. There is no fresh key exchange: the new connection's keys come from the first connection's master secret and two new random values. So whoever later obtains the server's ticket key can read every connection resumed from a ticket it sealed, and the first one's secret with it - which a full handshake, and TLS 1.3's resumption, do not allow. The ticket key here lives only in the process's memory and is replaced every ticket lifetime, which bounds the exposure to that; `no_resumption` removes it. A session made under one version is never offered to, or accepted by, the other: one `proven_tls_session_t` holds whichever kind the last connection left, and a handshake that ends up in the other version is simply a full one.

Early data (0-RTT)
Resumption still waits one round trip for the handshake before the first request leaves. With **early data** a client sends that request in its first packet, under a key made from the ticket, and the server can answer in its first reply, before the client's Finished: one round trip fewer for a new connection (measured: a GET over HTTP/3 with a 20 ms round trip, 43 ms to 22 ms; over TCP the saving is one round trip of three - TCP's own, the handshake's, the request's). It is there for HTTP/3 (chapter 17) and, since version 0.84.0, over TCP for HTTP/1.1 and HTTP/2 - the event-driven server and client (chapters 15 and 17) and the blocking ones (chapter 11) - only when both configurations set `early_data` (it cannot be combined with `no_resumption`):

- A server with `early_data` issues tickets that allow it - over TCP 16,384 bytes of it, in QUIC as RFC 9001 asks - and accepts early data when all of these hold: the ticket allows it and was made over the same transport; it is the first identity and its binder checks; the cipher suite and the protocol (ALPN; over TCP, none for none) are the ticket's; in QUIC, the transport limits this server promises are the ones it promised when it issued the ticket; no HelloRetryRequest happened; the age the client reports is within 10 seconds of the time since the ticket was issued; and **the ticket has not carried early data before**. Otherwise the early data is refused and the handshake goes on as a resumption. Over TCP more early data than the ticket allowed ends the connection (`unexpected_message`), and the client's `EndOfEarlyData` marks where it stops.
- A client with `early_data` sends early data when its session holds a ticket that allows it, for a protocol it still offers; a refusal costs it only the round trip it hoped to save - what it sent goes

again after the handshake. A ticket that carried early data is not offered for it again: the server would refuse it, and a newer ticket takes its place.

Early data can be replayed. Whoever sees the client's first flight can send it again, and a server that accepts it acts on the request again. Each server configuration keeps a register of the tickets it took early data on (4,096 of them, 96 KiB, made with the configuration), for as long as the freshness window could let them in, and takes each ticket's early data once; when the register is full, early data is refused. Because tickets open only in the process and configuration that sealed them (above), that one register covers every server that could take the ticket. What no register prevents: an attacker who holds the client's flight back and sends it first makes the server act on the copy; the original's early data is then refused and sent again after the handshake, so the request is acted on twice. Hence the rule the HTTP servers and clients follow: only requests that are safe to repeat - GET, HEAD and OPTIONS without a body - go early, a handler can ask `proven_http_stream_early` and answer **425 Too Early**, and anything else that arrives early is answered 425 by the server itself.

Who takes it over TCP. Only a connection whose owner asks: the HTTP servers do when their `tls` configuration has `early_data`, and they mark each request that arrived early (`proven_http_exchange_early` in the blocking server, `proven_http_stream_early` in the event-driven one), answer 425 themselves to anything that is not safe to repeat, and may answer the rest before the client's Finished. The HTTP clients send early only what is safe to repeat; the blocking client's transport keeps the copy it sent early and sends it again, first, when the server refused it, and a 425 sends the request once more on a new connection, not early. A program that drives the engine or the transport wrapper itself as a server takes none, whatever its configuration says.

```
#include <stdio.h>
#include <string.h>

/*
 * Early data (0-RTT) over HTTPS, both ends in one program: the server of chapter 11 and the
 * client of chapter 11, each with `early_data` in its TLS configuration. The first request makes
 * a full handshake and leaves a ticket; the second goes with the next connection's ClientHello,
 * and the handler can tell.
 */

static const char *const HOW_TO_MAKE =
    "  openssl req -x509 -newkey ed25519 -nodes -days 30 -subj /CN=example.test \\\n"
    "      -addext subjectAltName=DNS:example.test,IP:127.0.0.1 -keyout key.pem -out cert.pem\n";

typedef struct {
    proven_http_server_t *server;
} ex_app_t;

static void handle(void *ctx, proven_http_exchange_t *x) {
    ex_app_t *app = ctx;
    const proven_http_request_t *req = proven_http_exchange_request(x);
    if (proven_u8str_view_eq(req->target, PROVEN_LIT("/quit"))) {
        (void)proven_http_exchange_respond(x, 200, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("bye\n")));
        proven_http_server_stop(app->server);
        return;
    }
    /* A request that came as early data may be a copy someone replayed. Only GET, HEAD and
     * OPTIONS without a body get this far early - the server answers anything else 425 itself -
     * and a GET that changes something should answer 425 too: the client sends it again after
     * the handshake. */
    const bool early = proven_http_exchange_early(x);
    (void)proven_http_exchange_respond(x, 200, NULL, 0, proven_mem_view_from_u8(early ?
PROVEN_LIT("early\n") : PROVEN_LIT("after the handshake\n")));
```

```
}

static void serve(void *arg) {
    (void)proven_http_server_run(((ex_app_t *)arg)->server);
}

/* GET `url` and return whether the body is `want`. */
static bool get_is(proven_http_client_t *client, proven_allocator_t heap, proven_u8str_view_t url,
proven_u8str_view_t want) {
    proven_http_client_response_t response;
    proven_u8str_t body = { 0 };
    bool ok = proven_http_client_get(client, url, &response) == PROVEN_OK && response.status == 200 &&
              proven_http_client_read_all(&response, heap, &body, 4096) == PROVEN_OK &&
proven_u8str_view_eq(proven_u8str_as_view(&body), want);
    proven_u8str_destroy(heap, &body);
    proven_http_client_finish(&response);
    return ok;
}

int main(int argc, char **argv) {
    proven_allocator_t heap = proven_heap_allocator();
    if (argc < 3) {
        printf("usage: %s CERTIFICATE.pem KEY.pem\nA self-signed pair for 127.0.0.1:\n%s", argv[0],
HOW_TO_MAKE);
        return 0;
    }
    proven_result_mem_mut_t cert = proven_fs_read_all_bounded(heap, proven_u8str_view_from_cstr(argv[1]), 64 *
1024);
    proven_result_mem_mut_t key = proven_fs_read_all_bounded(heap, proven_u8str_view_from_cstr(argv[2]), 16 *
1024);
    EXAMPLE_REQUIRE(cert.err == PROVEN_OK && key.err == PROVEN_OK, "the certificate and the key are read");
    if (cert.err != PROVEN_OK || key.err != PROVEN_OK) return 1;

    /* The server: `early_data` makes its tickets allow early data, and it takes it - once per
     * ticket, within 10 seconds of the ticket's age. */
    proven_tls_options_t server_options = {
        .alloc = heap, .early_data = true,
        .certificate_pem = { cert.value.ptr, cert.value.size }, .private_key_pem = { key.value.ptr,
key.value.size },
    };
    proven_tls_config_t *server_tls = NULL;
    EXAMPLE_REQUIRE(proven_tls_config_create(&server_options, &server_tls) == PROVEN_OK, "the server's TLS
configuration");
    ex_app_t app = { 0 };
    proven_http_server_config_t server_config = { .alloc = heap, .handler = handle, .handler_ctx = &app, .tls
= server_tls };
    proven_net_addr_t at;
    EXAMPLE_REQUIRE(proven_http_server_create(&server_config, &app.server) == PROVEN_OK &&
                    proven_http_server_listen(app.server, proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0),
&at) == PROVEN_OK, "an HTTPS server on a free port");
    proven_job_sys_t *threads = NULL;
    proven_job_group_t running;
    proven_job_group_init(&running);
    EXAMPLE_REQUIRE(proven_job_system_init(heap, 1, 4, &threads) == PROVEN_OK &&
                    proven_job_group_submit(threads, &running, serve, &app) == PROVEN_OK, "its loop runs on
another thread");

    /* The client: `early_data` makes its configuration keep the newest session of each server,
     * and send a request that is safe to repeat with the next connection's ClientHello. */
    proven_cert_store_t *anchors = NULL;
```

```
    EXAMPLE_REQUIRE(proven_cert_store_create(heap, &anchors) == PROVEN_OK &&
                    proven_cert_store_add_pem(anchors, (proven_mem_view_t){ cert.value.ptr, cert.value.size },
NULL) == PROVEN_OK, "the anchors");
    proven_tls_options_t client_options = { .alloc = heap, .anchors = anchors, .early_data = true };
    proven_tls_config_t *client_tls = NULL;
    EXAMPLE_REQUIRE(proven_tls_config_create(&client_options, &client_tls) == PROVEN_OK, "the client's TLS
configuration");
    /* No connection kept, so that each request makes a new one. */
    proven_http_client_config_t client_config = { .alloc = heap, .tls_wrap = proven_tls_http_wrap, .tls_ctx =
client_tls };
    proven_http_client_t *client = NULL;
    EXAMPLE_REQUIRE(proven_http_client_create(&client_config, &client) == PROVEN_OK, "an HTTP client");

    char url_text[64];
    int url_len = snprintf(url_text, sizeof url_text, "https://127.0.0.1:%u/", (unsigned)at.port);
    proven_u8str_view_t url = { (const proven_byte_t *)url_text, (proven_size_t)url_len };
    EXAMPLE_REQUIRE(get_is(client, heap, url, PROVEN_LIT("after the handshake\n")), "the first connection: a
full handshake, and a ticket");
    EXAMPLE_REQUIRE(get_is(client, heap, url, PROVEN_LIT("early\n")), "the second: the GET went with its
ClientHello");

    url_len = snprintf(url_text, sizeof url_text, "https://127.0.0.1:%u/quit", (unsigned)at.port);
    url.size = (proven_size_t)url_len;
    (void)get_is(client, heap, url, PROVEN_LIT("bye\n"));
    proven_job_group_wait(threads, &running);
    proven_http_client_destroy(client);
    proven_http_server_destroy(app.server);
    proven_job_system_close(threads);
    proven_job_system_destroy(threads);
    proven_tls_config_destroy(client_tls);
    proven_tls_config_destroy(server_tls);
    proven_cert_store_destroy(anchors);
    proven_mem_wipe(key.value);
    heap.free_fn(heap.ctx, key.value.ptr);
    heap.free_fn(heap.ctx, cert.value.ptr);
    return EXAMPLE_OK();
}
```

Early data a server does not take. A client may send early data that this server will not take - a ticket it cannot open, a server whose owner does not take early data, a retried hello. The server does what RFC 8446 section 4.2.10 asks of a server that refuses it: it passes over those records - after a HelloRetryRequest, every application_data record until the second ClientHello; otherwise every record that does not open under the handshake keys, until one does - and the handshake goes on as if they had not been sent. The client learns from the server's answer that its early data was not taken, and sends it again. What is passed over is bounded: 64 KiB, each record counted with its 5-byte header. Past that the next record ends the connection, as any record that does not open would (bad_record_mac, or unexpected_message after a HelloRetryRequest).

7. Client certificates and pins

Client certificates. A server can require that the client proves who it is, too - mutual TLS, the usual choice between services. On the server set `client_auth` and give `anchors`:

`client_auth`	**A client with no certificate**	**A client with one that does not verify**
`PROVEN_TLS_CLIENT_AUTH_NONE`	(not asked)	(not asked)

`PROVEN_TLS_CLIENT_AUTH_REQUEST`	Let in. `proven_tls_peer_key_sha256` returns `false`	Refused
`PROVEN_TLS_CLIENT_AUTH_REQUIRE`	Refused	Refused

On the client set `certificate_pem` and `private_key_pem`; it presents them when asked and not otherwise. The server learns *that* the client holds a key some authority vouched for. *Which* client it is is yours to decide - from `proven_tls_peer_key_sha256`, or from the certificate (`keep_peer_certificate`) and its names - and `PROVEN_TLS_CLIENT_AUTH_REQUEST` leaves even the "whether" to you: check before acting on a request.

With an HTTP server the engine is inside the server, and a handler has no call to reach it in this version: use `REQUIRE` there, so that every request a handler sees came from a verified client.

Pins. A pin is the SHA-256 of a public key (Chapter 13, section 7). Give `pins` and the peer's key must be one of them:

- With `PROVEN_TLS_VERIFY_CHAIN`, **in addition** to the chain. A valid certificate from any authority in the world is then not enough; it has to be for the key you expect.
- With `PROVEN_TLS_VERIFY_PIN_ONLY`, **instead** of it. No chain, no dates, no name - the key is the identity, and `server_name` may be empty. This is the mode for a peer you run yourself with a self-signed certificate, and it means the pin is all of your security.

Ship two pins - the key in use and a spare - or a lost key becomes a fleet that cannot connect.

8. When it fails

A connection fails once, with one error, and stays failed.

Error	What happened	Usual cause
`PROVEN_ERR_UNTRUSTED`	The peer's certificate does not chain to an anchor, or is not pinned, or a required client certificate was not presented (the peer is told `certificate_required` in TLS 1.3 and `handshake_failure` in 1.2, which has no closer word)	A private or self-signed authority; a missing intermediate; on Windows, a root the machine has not fetched (Chapter 13)
`PROVEN_ERR_EXPIRED`, `PROVEN_ERR_NOT_YET_VALID`	A certificate on the path is outside its validity period	An unrenewed certificate - or this machine's clock
`PROVEN_ERR_NAME_MISMATCH`	The certificate is valid and is for another name	A wrong URL, a misconfigured server, or an interception
`PROVEN_ERR_PROTOCOL`	Everything else the peer did: a version or algorithm this side does not speak, a malformed or unexpected message, a record that does not authenticate - or the peer ended the handshake with an alert of its own	A peer that speaks only TLS 1.1 or older, or only the parts of TLS 1.2 this library leaves out (section 9); a TLS 1.2 peer when `min_version` is 1.3; no cipher suite or group in common; a peer that refused *this* side's certificate; corruption
`PROVEN_ERR_RESET` (the wrapper)	The connection ended without a TLS close	The peer crashed or was cut off; do not trust what was received as complete

`PROVEN_ERR_TIMEOUT` (the wrapper)	The deadline passed	A peer that stopped answering

```
proven_cert_fault_t proven_tls_peer_fault(const proven_tls_conn_t *conn);
int proven_tls_alert_received(const proven_tls_conn_t *conn);
int proven_tls_alert_sent(const proven_tls_conn_t *conn);
```

For the log line: `proven_tls_peer_fault` is the precise reason a certificate was refused (Chapter 13's list, plus `PROVEN_CERT_FAULT_PIN_MISMATCH`). `proven_tls_alert_received` is the alert the peer sent, as its number in RFC 8446 section 6 - when the answer is `PROVEN_ERR_PROTOCOL` and you want to know what the *other* side objected to, this is where it is (48 `unknown_ca` and 42 `bad_certificate` mean it did not accept your certificate; 40 `handshake_failure` that nothing was in common; 70 `protocol_version` that it wants another version). `proven_tls_alert_sent` is what this side told the peer. Both are -1 when there was none.

With the engine, the error is returned once by `proven_tls_feed`; the alert that tells the peer is then in the pending output - send it - and after that every call but the output pair and the questions returns `PROVEN_ERR_INVALID_STATE`.

The errors are deliberately few. A peer that fails authentication learns only that it failed. Do not relay `proven_tls_peer_fault` or the details of a `PROVEN_ERR_PROTOCOL` to the other side of the connection; they are for your log.

9. What is negotiated, what is held, what is not here

Negotiated. TLS 1.3, and TLS 1.2 cut down to the part of it that is still defensible. When both sides can speak 1.3, that is what they speak.

In TLS 1.3:

What	Which
Cipher suites	`TLS_AES_128_GCM_SHA256` (0x1301), `TLS_AES_256_GCM_SHA384` (0x1302), `TLS_CHACHA20_POLY1305_SHA256` (0x1303)
Their order	AES-128, AES-256, ChaCha20 on a processor with AES instructions (x86 with AES-NI - 64-bit, and 32-bit too - and AArch64 with the cryptography extension). **ChaCha20 first everywhere else**: without those instructions this library's AES is constant-time and at least thirty times slower than its ChaCha20, which on x86 runs four blocks at a time with SSE2 and eight with AVX2 (since v0.85.0; the processor is asked at run time)
Key exchange	X25519; P-256 when the peer insists (one extra round trip)
This side's key	ECDSA with P-256, Ed25519, or RSA of 2048 to 4096 bits (signing with RSA-PSS and SHA-256)
A peer's key	Those, ECDSA with P-384, and RSA (2048 to 8192 bits, RSA-PSS)
Also	ALPN, server name indication, session tickets, key update, `record_size_limit`, the middlebox-compatibility messages

In TLS 1.2:

What	Which
Cipher suites	`ECDHE-ECDSA` and `ECDHE-RSA`, each with `AES128-GCM-SHA256`, `AES256-GCM-SHA384` and `CHACHA20-POLY1305`: 0xC02B, 0xC02C, 0xCCA9, 0xC02F, 0xC030, 0xCCA8. In the same order of ciphers as above
Key exchange	X25519 or P-256, a fresh key pair for every connection - so there is always forward secrecy

This side's key	ECDSA with P-256 or Ed25519 for the `ECDHE-ECDSA` suites; RSA for `ECDHE-RSA`, signing with RSA-PSS when the peer offers it and PKCS #1 v1.5 with SHA-256 when it does not
A peer's key	The same kinds as in 1.3; its signature must use SHA-256, SHA-384 or SHA-512
Required of the peer	The extended master secret (RFC 7627). A peer without it is refused with `handshake_failure`
Also	ALPN, server name indication, session tickets (RFC 5077; section 6), client certificates, the downgrade mark of RFC 8446 section 4.1.3

What that leaves out is the point. Every attack on TLS 1.2 that has a name went through something this list does not have: the CBC cipher suites (padding oracles), RSA key exchange (Bleichenbacher's attack, and no forward secrecy), finite-field Diffie-Hellman with weak or chosen groups, renegotiation, compression, a master secret not bound to the handshake, SHA-1 and MD5 in handshake signatures. A peer that offers only those gets `handshake_failure`, and your program gets `PROVEN_ERR_PROTOCOL`. Some of them can be switched on, one by one, for a peer that has nothing else: section 10 says which, and what each one costs.

- **Renegotiation is declined, not answered.** A peer that asks for a new handshake on an established connection is sent the warning `no_renegotiation` and the connection carries on as it was. What the peer does next is its choice: some go on, some close. One that keeps asking is cut off after sixteen requests in a row.
- **A client is not fooled into 1.2.** A server that could have spoken 1.3 and answers 1.2 marks its random value; a client here that offered 1.3 and sees the mark ends the handshake with `illegal_parameter`, because somebody in between removed the better offer.
- **ChangeCipherSpec is accepted in one place per direction** - where the keys change - and nowhere else.
- **An Ed25519 key authenticates a client in TLS 1.3 only.** A client whose key is Ed25519 presents no certificate to a 1.2 server (as if it had none). A server's Ed25519 key works in both.

To speak TLS 1.3 only, set `min_version = PROVEN_TLS_VERSION_1_3`: that is exactly what versions of this library before 0.22.0 did. Consider it for a server whose clients you control, and for anything where the weaker resumption of section 6 is not acceptable. The default accepts 1.2 because a client that cannot reach a server that speaks nothing newer is not safer, only disconnected - and a great many servers and devices still speak nothing newer.

Held per connection. During the handshake, a few kilobytes that are freed when it completes. After it, **about 1.25 KiB** per connection when nothing is in flight: the keys (with the powers of the GHASH key that AES-GCM's one-pass code keeps, since v0.77.0) and the counters. (The event-driven server of Chapter 15 also gives back the keys' expanded form - about 700 bytes - once a connection has been quiet for a second, keeping only the keys it was made from, and makes it again for the next record: see Chapter 15, section 14.) Record buffers exist only while a record is being received, is waiting to be read, or is waiting to be sent, and are freed when it is done - so ten thousand idle TLS connections hold about twelve megabytes, not the three hundred that two 16 KiB buffers each would cost. That is this layer's share: the transport wrapper adds a few dozen bytes, and an HTTP server's own per-connection request buffer is Chapter 11's and is separate. (A TLS 1.2 connection holds the same.)

What a handshake costs. On one x86-64 core of the development machine, a full handshake with X25519 and a P-256 certificate is about 0.31 ms of processor time on the server and about 0.41 ms on a client that holds that certificate as its anchor; each certificate whose signature the client checks on the way to an anchor - one, for a certificate an anchor issued - adds about 0.25 ms. For a

server that is some 3,200 new connections a second per core. (Built with GCC 14. Since v0.90.0 P-256 has a field code of its own, which GCC 16 and Clang compile to about twice the speed of the general code - GCC 14 to about 1.15 times.) With an Ed25519 certificate the two figures are 0.21 ms and 0.25 ms, a further certificate 0.08 ms, and a core serves some 4,800 new connections a second. Resumption (section 6) removes the signatures; keeping connections open removes the handshake.

The public-key arithmetic here is written to be read, and to be constant-time wherever it handles a secret: a private key, a nonce, a shared secret. **Signature verification is not constant-time**, and need not be: the key, the message and the signature it is given are all public. It takes both of its multiplications in one variable-time pass, which is about twice as fast. The functions that do so are reachable from verification alone, and the build fails if anything else calls them (`scripts/check-vartime.py`). Against `openssl speed` (OpenSSL 3.5) on the same machine, Ed25519 here signs in about three quarters of OpenSSL's time and verifies in about two thirds; X25519 and the NIST curves take 1.2 to 7 times as long, P-384 verification the least and P-256 key agreement the most (P-256 2.1 to 4 times when built with GCC 16 or Clang, since v0.90.0).

Those figures are a 64-bit build's, by GCC or Clang: the numbers of P-256, RSA and Diffie-Hellman are then held in 64-bit limbs, and a number of Curve25519's field in five limbs of 51 bits, two of which multiply into the compiler's 128-bit integer. A 32-bit build, and any build with `-DPROVEN_CRYPTO_MP_LIMB_BITS=32`, holds them in limbs of 32 bits and of 26 and 25, and takes two to four times as long over the same arithmetic - 0.83 ms and 1.1 ms for the handshake with a P-256 certificate, 0.59 ms and 0.75 ms with Ed25519, and 3.4, 11 and 25 ms for the RSA signatures below. The results are the same bytes either way.

With an RSA key the signature dominates. On the same core one signature takes about 1.25 ms with a 2048-bit key, 3.7 ms with 3072 bits and 8.4 ms with 4096 (since v0.88.0, which squares in about three quarters of a multiplication's work; 1.5, 4.7 and 11 ms before): a server with a 2048-bit RSA key completes about 650 new connections a second per core, against 3,000 with P-256. On the event-driven server of Chapter 15 that time is spent on the loop's thread, where nothing else is served meanwhile. The signing is done modulo the two primes with a constant-time exponentiation, its input blinded, and its result checked with the public key before it is released; it keeps its working numbers on the stack - about 20 KiB by the compiler's own accounting at `-O2` on x86-64 - which a small target must allow for. A peer must accept `rsa_pss_rsae_sha256`, as TLS 1.3 requires of every implementation: it is the one scheme this side signs with there.

Bounds on a peer. A handshake message is at most `max_handshake_bytes`; a record at most what the protocol allows, refused on its header. Records that carry nothing - empty ones, repeated compatibility messages - are tolerated sixteen in a row. Key-update requests are answered until 64 KiB of answers are waiting unsent. Past any of these the connection ends with `PROVEN_ERR_PROTOCOL`.

Not here:

- **Unless a configuration asks (section 10):** TLS 1.1 and 1.0, CBC cipher suites, RSA and finite-field Diffie-Hellman key exchange, a peer without the extended master secret.
- **Not at all:** SSL 3.0 and earlier; RC4, 3DES, export, anonymous and DSS suites; renegotiation; compression; session IDs kept on the server; `record_size_limit` in TLS 1.2. A peer that needs one of those fails with `PROVEN_ERR_PROTOCOL`.
- **RSA keys above 4096 bits for this side**, and RSA-PSS keys (a certificate whose key is marked for PSS only).
- **Early data to a program that drives the engine itself.** Over TCP only the HTTP servers and clients of this library send and take it (section 6); a server of your own over the engine or the transport wrapper passes over what a client sends, up to 64 KiB.
- **Revocation.** No CRL, no OCSP, no stapling. See Chapter 13.
- **Choosing a certificate by the name the client asked for.** One config, one certificate.

- **Encrypted ClientHello, post-handshake client authentication, external pre-shared keys.**
- **Padding of records.** Record lengths reveal data lengths, as with most TLS in use.
- **Sharing tickets between processes**, or keeping them across a restart.

How this was tested. The registered tests replay the five example handshakes of RFC 8448 through the key schedule and the record layer, value by value, and two of them through the client, record by record; run a client and a server against each other in memory for every suite and key type, every refusal in sections 7 and 8, resumption, and a sweep of single-bit changes to the handshake; and run HTTPS and `wss` over real sockets. Outside the registered tests, both roles were run against OpenSSL and the server against GnuTLS, for every suite and group, with retries, resumption and client certificates; and the client fetched pages from public servers. For RSA keys on this side: the same two implementations accepted handshakes signed with keys of 2048, 3072 and 4096 bits that OpenSSL had made; the library's PKCS #1 v1.5 signatures were the same bytes as OpenSSL's for those keys and its PSS signatures were accepted by it. For TLS 1.2: its key derivation is checked against the two published test vectors for the PRF and against values computed outside this library (there is no published trace of a whole 1.2 handshake as there is for 1.3, so for the handshake the other implementations are the evidence); both roles were run against OpenSSL for all six suites and all three kinds of key, with tickets, client certificates, PKCS #1 and PSS signatures, a peer without the extended master secret (refused), a peer offering only what is left out (refused), and a peer asking to renegotiate (declined); and GnuTLS's client was run against the server. For the legacy set: its pieces are checked against values computed outside the library and against the published ones that exist (section 10 lists them); both roles were run against OpenSSL, and the server against GnuTLS's client, in TLS 1.0, 1.1 and 1.2 with each added key exchange and cipher, with and without encrypt_then_mac, with client certificates, and against peers without the extended master secret; and the two places where a wrong step gives a key away - opening a CBC record and recovering an RSA premaster (while the server's half of RSA key exchange existed) - were run under the same checker as below, with what is decrypted marked secret, and reported nothing. The signing path, and the key derivation and record protection of both versions, were run under a checker that reports any branch or memory index that depends on a secret - the primes, the private exponents, the blinding values, the shared key and everything derived from it - and reported none, at two optimisation levels on one compiler and one processor family. **Not done:** no coverage-guided fuzzing, no protocol-level fuzzing suite (tlsfuzzer, BoGo), no timing measurement on hardware, no external review. The primitives underneath are Chapter 13's, with the limits stated there.

10. The legacy set: what each setting exposes you to

Everything in this section is **off unless a configuration names it**, and none of it is what you want. It exists because a program sometimes has to talk to a machine that has nothing better - an appliance nobody can update, a server on somebody else's network - and the alternative is no connection or a second TLS library. RFC 8996 retired TLS 1.0 and 1.1 and RFC 9325 says not to negotiate the rest. If you control both ends, change the other end.

Each bit is one decision. Set `legacy` in the options to the ones you mean:

Bit	What becomes possible	What you give up
`PROVEN_TLS_LEGACY_CBC`	Cipher suites that encrypt with AES in CBC mode and authenticate with HMAC-SHA1, HMAC-SHA256 or HMAC-SHA384. Needed for	A construction with a twenty-year record of padding attacks

	any version below 1.2, which has nothing else	
`PROVEN_TLS_LEGACY_RSA_KEY_EXCHANGE`	The client encrypts the session's secret under the server's RSA key instead of both sides agreeing on one	**Forward secrecy**: whoever obtains the server's key later reads every recorded connection. And the key itself would be put within reach of a server-side mistake (below), so a server refuses it
`PROVEN_TLS_LEGACY_DHE`	Key exchange by Diffie-Hellman in a finite field, with an RSA certificate	Nothing in principle; in practice time - it is several times slower than the elliptic-curve exchange - and it exists mostly on peers that are old in other ways too
`PROVEN_TLS_LEGACY_NO_EXTENDED_MASTER_SECRET`	A connection to a peer that does not have RFC 7627 goes ahead	The binding of the session's keys to the handshake that made them

and `min_version` to `PROVEN_TLS_VERSION_1_1` or `PROVEN_TLS_VERSION_1_0` to allow those versions - which also needs the CBC bit, and is refused without it. Nothing changes for a configuration that sets none of this: it sends the same hello, gives the same answers and holds the same memory as before these existed.

They are a last resort in the handshake as well. A legacy suite is offered after, and chosen after, every suite of section 9; an older version is agreed only when the peer has no newer one. Two sides that both allow everything still end up on TLS 1.3. And the downgrade checks of section 9 cover the old versions: a server that could have spoken 1.2 and is made to answer 1.0 says so in a way the client checks.

CBC suites

`ECDHE-ECDSA-` and `ECDHE-RSA-` with `AES256-SHA384`, `AES128-SHA256`, `AES128-SHA` and `AES256-SHA`; and, with the bits below, `AES128-SHA256`, `AES256-SHA256`, `AES128-SHA` and `AES256-SHA` after `DHE-RSA-` and after plain RSA. The suites whose MAC is SHA-1 are the only ones TLS 1.0 and 1.1 have.

The weakness is the order of operations. The sender authenticates the plaintext, pads it, then encrypts; the receiver must decrypt, strip the padding and only then check the MAC. A receiver that is a little faster or a little different for one kind of bad record than for another tells an attacker, one byte at a time, what the plaintext was (Vaudenay 2002, POODLE, Lucky Thirteen). What this library does about it:

- **It asks for the fix first.** `encrypt_then_mac` (RFC 7366) authenticates the ciphertext instead, so nothing is decrypted that was not sent by the peer. It is offered whenever CBC is and used whenever the peer agrees. Most peers that still need CBC do not have it.
- **Without it, opening a record does the same work whatever is in it.** Everything after decryption - finding the padding, checking it, finding the MAC, computing it over a length that depends on the padding, comparing - is done with masks, no branch and no memory index depending on a decrypted byte, and the number of hash blocks computed depends only on the record's length. Every failure is the one alert, `bad_record_mac`.
- **Decryption does not take a time that depends on the data either way**: with the processor's AES instructions on x86 - a CBC connection then receives at about 100 MiB/s on the development

machine, as fast as it sends - and with the portable, bitsliced cipher everywhere else, at about 6 MiB/s.
- **In TLS 1.0** each record's IV is the previous record's last block, so an attacker who can make you send chosen data knows the IV in advance (BEAST). Application data is therefore sent as one byte, then the rest - two records for every write.

Key exchange by RSA
`AES128-GCM-SHA256` and `AES256-GCM-SHA384`, and with the CBC bit `AES128-SHA256`, `AES256-SHA256`, `AES128-SHA` and `AES256-SHA`. The server's key must be RSA.

A client's only, from v0.78.0. `proven_tls_server_create` refuses a configuration with this bit (`PROVEN_ERR_INVALID_ARG`), and so does every server built on it - the event-driven server when it is made. A client may use it to reach an old server or device; no risk below is the client's, since the client decrypts nothing. Why a server may not:

- **No forward secrecy.** The secret every key is derived from travels encrypted under the server's long-term key. A recording of the connection is as safe as that key, for ever.
- **The server becomes a decryption oracle if it makes one mistake.** It decrypts a value the client chose. If anything it does afterwards - an alert, a delay - depends on whether that value decrypted to something well-formed, an attacker can use it, a few thousand connections at a time, to decrypt or *sign* anything with the server's key (Bleichenbacher 1998; ROBOT, 2017). That is not limited to this key exchange: a signature forged this way is as good in TLS 1.3. **A server with this key exchange puts its key at that risk for every protocol the key is used in**, and the clients it would serve - those with neither ECDHE nor DHE - are very old. Before v0.78.0 a server could enable it. The code that did was kept for a while, reached only by this library's own tests as the peer its client was tested against, and has since been removed: the library holds no code that decrypts an RSA key exchange. The client's side is run against other implementations' servers, and its encryption is checked in the registered tests by opening it with the private key.

Finite-field Diffie-Hellman
`DHE-RSA-` with `AES128-GCM-SHA256`, `AES256-GCM-SHA384` and `CHACHA20-POLY1305`, and with the CBC bit `AES128-SHA256`, `AES256-SHA256`, `AES128-SHA` and `AES256-SHA`.

What went wrong with DHE was the group, which the server chooses and the client cannot examine in the time a handshake takes: too small (Logjam), not a prime, or built to leak. So no group is examined here at all:

- **A client accepts a prime only if it is, byte for byte, one of six** - the 2048-, 3072- and 4096-bit groups of RFC 7919 and of RFC 3526 - with generator 2. Anything else ends the handshake with `insufficient_security`: a server with a group of its own making cannot be reached, however good the group is.
- **A server offers one**: the 2048-bit group of RFC 7919.
- A value from the peer of 0, 1 or the prime less one is refused. Every connection uses a fresh exponent, which is what keeps a known weakness of the key derivation (Raccoon, 2020) out of reach.
- It costs about 7 ms of processor time a side on the development machine, on top of the signature: a server with a 2048-bit RSA key completes some sixty-five such handshakes a second per core. And it keeps its working numbers on the stack: about 16 KiB at the deepest point of a handshake, by the compiler's own accounting at `-O2` on x86-64.

TLS 1.0 and 1.1
`min_version = PROVEN_TLS_VERSION_1_1` or `PROVEN_TLS_VERSION_1_0`, with the CBC bit.

- The handshake's integrity rests on MD5 and SHA-1 together, and signatures are made over them: an RSA key signs both hashes, an ECDSA key signs the SHA-1. Neither hash would be accepted for this anywhere else in the library.
- There is no AEAD cipher: every suite is CBC with HMAC-SHA1, with everything said above.
- **An Ed25519 key cannot serve these versions** - they have no way to carry its signature. A server whose key is Ed25519 refuses a client that speaks nothing newer.
- Sessions work as in 1.2 (section 6): a ticket, resumed **in the version it was made in** and no other - a session made in TLS 1.0 is not resumed by a handshake that agrees on 1.2.
- Client certificates work, with P-256 and RSA keys.

A peer without the extended master secret
RFC 7627 (2015) binds a session's keys to the handshake that made them. Without it, somebody in the middle can arrange for two different connections to share a master secret, and then pass a client's certificate proof from one to the other (the triple handshake attack). A default configuration refuses a peer that lacks it. Most peers that speak only TLS 1.0 or 1.1 lack it.

With `PROVEN_TLS_LEGACY_NO_EXTENDED_MASTER_SECRET` such a peer is talked to, and on that connection the things the attack needs are not done: **no ticket is issued or accepted**, a client **shows no certificate**, and a server **does not ask for one** - a server that *requires* client certificates refuses such a client outright. A peer that does have the extension still uses it; the bit only allows its absence.

What this costs a program that uses none of it
Nothing: a connection that agrees on no CBC suite holds what it held before any of this existed. An idle CBC connection holds about 1.4 KiB instead of about 1.25 KiB.

How this was checked, and what was not
The registered tests check every piece against values computed outside the library - AES-CBC against SP 800-38A, HMAC against RFC 2202 and RFC 4231, and the rest (the old PRF, CBC records in every version and both MAC orders, Diffie-Hellman) against an independent implementation - and run every suite in every version it exists in, both roles, with each refusal described above. The six Diffie-Hellman primes were compared with two other implementations when they were added. Outside the registered tests, both roles were run against OpenSSL and the server against GnuTLS; and opening a CBC record and - while the server's half existed - recovering an RSA premaster were each run under a checker with the decrypted bytes marked secret, together with a deliberately wrong version of each that the checker must catch (it did).

And the time was measured, on the development machine's processor, the way such leaks are looked for: the same operation run hundreds of thousands of times on inputs of two kinds, in random order, each run timed with the cycle counter, and the two sets of times compared statistically. For a CBC record - each of the three MACs, with the processor's AES and with the portable one - a record with no padding against one with 255 bytes of it, a wrong MAC after each amount of padding, a wrong MAC against wrong padding, and against an impossible padding length; for the RSA premaster, while that code existed, a proper one against noise, against a wrong version and against a wrong length. Twice over. **No pair showed a difference that repeated.** Over all runs as measured the test statistic stayed below 3 for every pair, where 4.5 is the usual line; with the slowest tenth of runs set aside, as is customary to remove interruptions, it rose to between 4.5 and 7.7 for ten pairs in one run or the other and for no pair in both - which is what noise does, and what a real difference does not. A deliberately naive record check, measured the same way as a control, gave a statistic in the thousands. Whether a record opened at all *is* visible in the time, as it is in the answer.

What that does not show. One processor, one compiler, a machine that was doing other work; 400,000 runs a pair for records and 6,000 for RSA, which would miss a difference of a few cycles. It

is a measurement, not a proof, and a quieter machine or another processor may see what this one did not. No fuzzing of these paths; no external review.

Chapter 15: The event loop and the event-driven server

Part V - Talking to the operating system. Prerequisites: Chapter 9 (sockets, deadlines, the selector); Chapter 11 (the HTTP server this one is the other form of); Chapter 14 if it is to speak TLS. After this chapter you can run a loop with timers and work that comes back from other threads, serve HTTP from it without a call that waits, answer a request later, stream a response to a client slower than your program, keep WebSocket connections open without a thread each, make many HTTP requests at once from one thread, spread a server over several processor cores, and say what a connection costs while it does nothing - and what was measured.

This chapter covers `loop.h`, `http_event.h`, `ws_event.h` and `http_event_client.h`. All are hosted-only, and `PROVEN_NO_NET` leaves them out.

Table of contents

1. Two ways to serve
2. A loop
3. Timers
4. Work that is done elsewhere
5. A socket of your own
6. The event-driven server
7. Answering
8. Request bodies
9. Limits, time and refusals
10. Threads, TLS and shutting down
11. WebSocket on the loop
12. The event-driven client
13. Several loops
14. What a connection holds, what was measured, and what is not here

1. Two ways to serve

The server of Chapter 11 hands your handler a request and lets it read and write with calls that wait. Nothing is simpler to write: the handler is a straight line from the request to the response. The price is that a request being handled occupies a thread for as long as it takes - including all the time its client is slow.

The server of this chapter turns that around. **Nothing in it waits.** It sits on an event loop, one thread that asks the system which sockets have something to say, deals with those, and asks again. Your functions are told what happened - a request arrived, a piece of its body arrived, there is room to write more - and return at once. A connection that is doing nothing is then a small struct and nothing else: no thread, no stack, no buffer.

What you take on in exchange is real, and it is the whole of this chapter:

- **Your functions must not wait** - not on a socket, a file, or a lock someone may hold for long. Work that takes time goes to another thread and its result is posted back (section 4).
- **Writing can be refused.** When the client is not taking what was sent, the server stops accepting more from you and says when to continue (section 7).
- **What you are shown is short-lived.** The request head and the pieces of a body are views good only until your function returns. Copy what you keep.

Which to choose: the blocking server while the number of connections open at once is something a pool of threads carries comfortably and the handlers are easier to write as straight lines; this one when most connections are idle most of the time and there are many of them - long polling, event

streams, slow clients, a great many kept connections. Both parse HTTP with the same code of Chapter 10 and refuse the same malformed requests.

2. A loop

```
proven_err_t proven_loop_create(proven_allocator_t alloc, proven_loop_t **out);
proven_err_t proven_loop_create_ex(proven_allocator_t alloc, const proven_loop_options_t *options,
proven_loop_t **out);
proven_loop_backend_t proven_loop_backend(const proven_loop_t *loop);
void proven_loop_destroy(proven_loop_t *loop);
proven_err_t proven_loop_run(proven_loop_t *loop);
proven_err_t proven_loop_poll(proven_loop_t *loop, proven_net_deadline_t until);
void proven_loop_stop(proven_loop_t *loop);
```

A `proven_loop_t` is a set of operations on sockets that complete (section 5), a set of timers, a queue that other threads can post to, and a scratch buffer.

Call	What it does	Returns
`proven_loop_create(alloc, &loop)`	Make a loop.	`proven_err_t`: `INVALID_ARG`; `NOMEM`; what opening the selector returned.
`proven_loop_create_ex(alloc, &options, &loop)`	Make a loop with the backend `options.backend` names (below). `options` may be null: then it is `proven_loop_create`.	`proven_err_t`: as `proven_loop_create`; `UNSUPPORTED` for a backend this system does not have.
`proven_loop_backend(loop)`	Which backend a loop was made with.	`proven_loop_backend_t`, never `AUTO`.
`proven_loop_run(loop)`	Wait, call what is ready, fire what is due, repeat - until `proven_loop_stop`.	`PROVEN_OK` after a stop; an error if waiting itself failed.
`proven_loop_poll(loop, until)`	One round: wait until something is ready or `until` passes, and handle it. `PROVEN_NET_DONT_WAIT` handles only what is ready now.	`proven_err_t`.
`proven_loop_stop(loop)`	Make `proven_loop_run` return. Safe from any thread and from a callback.	nothing.
`proven_loop_destroy(loop)`	Free the loop. What has already ended gets its last answer now - `closed` of a socket given to `proven_loop_close`, the `PROVEN_ERR_CANCELLED` of a cancelled accept - so that its memory can go; those functions only free. Anything else still attached, in flight or set is forgotten and nothing is called; the bytes waiting to be sent are freed. Null is accepted.	nothing.

`proven_loop_run` is for a thread that does nothing else. `proven_loop_poll` is for a program that already has a main loop of its own and gives this one a turn in it.

Backends. How a loop does its work is chosen when it is made, and nothing else about it changes with the choice: the same calls, the same answers in the same order rules.

`proven_loop_backend_t`	**Where**	**How**
`PROVEN_LOOP_BACKEND_AUTO`	everywhere	The completion port on Windows; io_uring on Linux 6.0 or later where the system allows it, epoll where it does not (since v0.94.0); the selector elsewhere - what `proven_loop_create` gives.
`PROVEN_LOOP_BACKEND_SELECTOR`	POSIX	epoll, kqueue or poll (Chapter 9): the loop is told a socket is ready and makes the call itself.
`PROVEN_LOOP_BACKEND_URING`	Linux 5.19 or later	io_uring: the receives, sends, accepts and connects are given to the kernel, which reports them done. Refused (`UNSUPPORTED`) on an older kernel, and where the system turns io_uring off - a container's seccomp filter, `io_uring_disabled`. Datagrams are sent with a plain call, as on the other backends.
`PROVEN_LOOP_BACKEND_IOCP`	Windows	The I/O completion port.

io_uring is the default on Linux 6.0 or later since v0.94.0, where the system lets the process use it; elsewhere on Linux, and where it is refused, `AUTO` is epoll, as it was before. In an emulated machine an HTTP/1.1 server on it made 0.071 system calls a request against epoll's 2.07, and answered 5 to 7 % more requests a second; no real machine has measured it here yet. `PROVEN_LOOP_BACKEND_SELECTOR` asks for epoll whatever the kernel. A loop made with io_uring frees what the kernel still holds the way Windows does - cancelled and waited for, up to two seconds, when the loop is destroyed. On Linux 6.0 or later a connection read with `proven_loop_read` (section 5) is received for by the kernel into a pool of the loop's - by default 64 buffers of 16 KiB, made at the first such read; `ring_buffers` and `ring_buffer_bytes` in `proven_loop_options_t` change it - so that the read after a receive of zero bytes is a copy, not a system call. A buffer is taken only when data arrives: an idle connection still holds none. A connection whose data nobody reads holds four at most; then the kernel's own window holds the peer, as on the other backends. Sends too go through the ring: a send is copied and handed to the kernel in the loop's next round, together with the receives, so that the reads and sends of every connection share the loop's one entry into the kernel; `proven_loop_send_queued` counts the bytes until the kernel has taken them, and a close writes to the socket what had not been handed over yet.

The thread rule. Everything in this chapter except `proven_loop_stop` and `proven_loop_post` must be called on the loop's own thread - the one inside `proven_loop_run` or `proven_loop_poll` - or before the loop starts. That covers the server and every stream function of the later sections. Nothing checks it for you.

3. Timers

```
void proven_loop_timer_set(proven_loop_t *loop, proven_loop_timer_t *timer, proven_u32 ms, proven_loop_fn fn,
void *ctx);
void proven_loop_timer_cancel(proven_loop_t *loop, proven_loop_timer_t *timer);
bool proven_loop_timer_is_set(const proven_loop_timer_t *timer);
```

A `proven_loop_timer_t` is yours: put it inside the struct it belongs to, zero-initialised, and do not copy or move it while it is set. Setting one allocates nothing.

- A timer **fires once**, `ms` milliseconds from now. It does not fire early, and on POSIX it fires within about a millisecond of that time; on Windows the wait ends at the system timer's interval, about 16 ms (until v0.70.0 every timer was rounded up to the loop's 16 ms tick). One that should repeat sets itself again in its own function.
- Setting a timer that is already set **moves** it.
- `proven_loop_timer_cancel` is safe inside any callback, another timer's included, and harmless when the timer is not set.
- Setting, cancelling and firing are constant-time, and a timer that is waiting is not looked at until its time. That is what lets every connection of a server have a timeout of its own.

Timers measure the monotonic clock of Chapter 5 setting the wall clock does not move them.

4. Work that is done elsewhere

```
proven_err_t proven_loop_post(proven_loop_t *loop, proven_loop_fn fn, void *ctx);
```

`proven_loop_post` asks the loop to call `fn(ctx)` on its own thread, soon. It is **safe from any thread**, and it is how a result comes back: the callback hands the slow part to a worker (the job system of Chapter 6), returns, and the worker posts a function that finishes the work where the loop's data may be touched.

- Functions are called in the order they were posted.
- The queue entry comes from the loop's allocator, so when other threads post, that allocator must be one that may be called from several threads. The heap allocator is.
- It returns `PROVEN_ERR_NOMEM` when that allocation fails, and `PROVEN_ERR_INVALID_ARG`.

The program below is a loop with nothing on it but timers and one job. The test suite compiles and runs it:

```
#include <stdatomic.h>

/*
 * An event loop by itself: timers, and work that is done on another thread and comes back.
 *
 * A loop's thread never waits for anything but "what is next". Whatever takes time - here a
 * worker that sleeps, standing in for a file or a database - happens elsewhere, and its result
 * is posted to the loop, where it runs between other events.
 */

typedef struct {
    proven_loop_t *loop;
    proven_loop_timer_t tick, late;
    int ticks;
    int order[8];
    int count;
    atomic_int from_worker;
} ex_app_t;

static void note(ex_app_t *app, int what) { if (app->count < 8) app->order[app->count++] = what; }

/* A timer fires once. One that should repeat sets itself again. */
static void on_tick(void *ctx) {
    ex_app_t *app = ctx;
    note(app, 1);
    if (++app->ticks < 3) proven_loop_timer_set(app->loop, &app->tick, 20, on_tick, app);
}

static void on_late(void *ctx) { note(ctx, 9); }

/* This runs on the loop's thread, although a worker thread asked for it. */
```

```
static void on_result(void *ctx) {
    ex_app_t *app = ctx;
    note(app, 5);
    proven_loop_timer_cancel(app->loop, &app->late);      /* it would have fired in five seconds; now it never
will */
    proven_loop_stop(app->loop);
}

/* This runs on a worker thread. It may take as long as it likes: the loop is not waiting. */
static void slow_work(void *ctx) {
    ex_app_t *app = ctx;
    proven_time_sleep(400);
    atomic_store(&app->from_worker, 1);
    if (proven_loop_post(app->loop, on_result, app) != PROVEN_OK) proven_loop_stop(app->loop);   /* only a
failed allocation refuses a post; then just end the run */
}

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    ex_app_t app = { 0 };

    // ---- a loop ------------------------------------------------------------------
    /* A loop is operations on sockets that complete, a set of timers, and a queue other threads can post to.
     * The options choose how it does its work; AUTO is what proven_loop_create gives. */
    EXAMPLE_REQUIRE(proven_loop_create_ex(heap, &(proven_loop_options_t){ .backend =
PROVEN_LOOP_BACKEND_AUTO }, &app.loop) == PROVEN_OK, "a loop");
    EXAMPLE_REQUIRE(proven_loop_backend(app.loop) != PROVEN_LOOP_BACKEND_AUTO, "it says which backend does the
work");

    // ---- timers ------------------------------------------------------------------
    /* The timer structs are yours: they live in your own state, and setting one allocates
     * nothing. A timer waiting to fire costs nothing either - it is not looked at until its time. */
    proven_loop_timer_set(app.loop, &app.tick, 20, on_tick, &app);
    proven_loop_timer_set(app.loop, &app.late, 5000, on_late, &app);
    EXAMPLE_REQUIRE(proven_loop_timer_is_set(&app.tick) && proven_loop_timer_is_set(&app.late), "two timers
are set");

    // ---- work elsewhere ----------------------------------------------------------
    /* The job system runs slow_work on a worker thread. When it is done it posts on_result back. */
    proven_job_sys_t *workers = NULL;
    EXAMPLE_REQUIRE(proven_job_system_init(heap, 1, 4, &workers) == PROVEN_OK && proven_job_submit(workers,
slow_work, &app), "a worker has the slow job");

    // ---- run ---------------------------------------------------------------------
    /* proven_loop_run returns when something calls proven_loop_stop - here on_result does. */
    proven_time_t began = proven_time_monotonic_now();
    EXAMPLE_REQUIRE(proven_loop_run(app.loop) == PROVEN_OK, "the loop ran and was stopped");
    proven_time_t took = proven_time_monotonic_now() - began;

    EXAMPLE_REQUIRE(app.ticks == 3 && app.count == 4 && app.order[0] == 1 && app.order[1] == 1 && app.order[2]
== 1 && app.order[3] == 5,
                    "three ticks, then the worker's result, in that order");
    EXAMPLE_REQUIRE(atomic_load(&app.from_worker) == 1 && took < 2000000000, "the result came from the worker,
and long before the five-second timer");
    EXAMPLE_REQUIRE(!proven_loop_timer_is_set(&app.late), "a cancelled timer is no longer set");

    proven_job_system_close(workers);
    proven_job_system_destroy(workers);
    proven_loop_destroy(app.loop);
```

```
    return EXAMPLE_OK();
}
```

5. A socket of your own

The server of the next section does this for you. It is here for a protocol that is not HTTP.

The loop does not tell you that a socket is ready and leave the rest to you. You start an **operation** - a receive, an accept, a connect - and its function is called when it has **finished**; you give the loop bytes to send, and it sends them. On Windows these are the I/O completion port's operations; on Linux, the BSDs and macOS the loop does the same over the selector of Chapter 9 (epoll, kqueue). Your program sees the same calls, answered in the same order, everywhere.

```
proven_err_t proven_loop_attach(proven_loop_t *loop, proven_loop_sock_t *sock, const proven_net_conn_t *conn,
                                const proven_loop_sock_fns_t *fns, void *ctx);
proven_err_t proven_loop_connect(proven_loop_t *loop, proven_loop_sock_t *sock, proven_net_addr_t to,
proven_net_conn_t *out,
                                 const proven_loop_sock_fns_t *fns, void *ctx);
proven_err_t proven_loop_recv(proven_loop_t *loop, proven_loop_sock_t *sock, proven_mem_mut_t buf);
void proven_loop_recv_cancel(proven_loop_t *loop, proven_loop_sock_t *sock);
proven_result_size_t proven_loop_read(proven_loop_t *loop, proven_loop_sock_t *sock, proven_mem_mut_t buf);
proven_result_size_t proven_loop_send(proven_loop_t *loop, proven_loop_sock_t *sock, proven_mem_view_t bytes);
proven_size_t proven_loop_send_queued(const proven_loop_sock_t *sock);
proven_err_t proven_loop_send_wait(proven_loop_t *loop, proven_loop_sock_t *sock, proven_size_t at_most);
void proven_loop_close(proven_loop_t *loop, proven_loop_sock_t *sock);
proven_err_t proven_loop_accept(proven_loop_t *loop, proven_loop_accept_t *acc, proven_net_listener_t
*listener,
                                proven_loop_accept_fn fn, void *ctx);
void proven_loop_accept_cancel(proven_loop_t *loop, proven_loop_accept_t *acc);
proven_mem_mut_t proven_loop_scratch(proven_loop_t *loop);
proven_allocator_t proven_loop_allocator(const proven_loop_t *loop);
```

A connection given to the loop is a `proven_loop_sock_t` - yours, inside your connection struct, not moved from `proven_loop_attach` until its `closed` - and a table of four functions, `proven_loop_sock_fns_t`, usually one `static const` table for every connection of a kind:

Function	Called when
`received(ctx, err, n)`	A receive has finished: data has arrived (a receive of zero bytes: `PROVEN_OK`, `n` 0), or `n` bytes are in your buffer; `PROVEN_ERR_EOF`, `PROVEN_ERR_RESET`; `PROVEN_ERR_CANCELLED`.
`writable(ctx, err)`	The bytes waiting to be sent are down to what `proven_loop_send_wait` asked for; or sending them failed (then also without a wait); or `PROVEN_ERR_CANCELLED` by a close.
`connected(ctx, err)`	A connect begun by `proven_loop_connect` has ended.
`closed(ctx)`	Last: after `proven_loop_close`, once nothing of the socket is in flight. Close the connection and free the memory here.

Call	What it does	Returns
`proven_loop_attach(loop, &sock, &conn, &fns, ctx)`	Give a connection to the loop. Nothing is started.	`proven_err_t`: `INVALID_ARG`.
`proven_loop_connect(loop, &sock, to, &conn, &fns, ctx)`	Begin connecting; the new connection is attached whatever the outcome, and `connected` says how it ended.	`proven_err_t`: a connect that failed at once, with nothing attached.

`proven_loop_recv(loop, &sock, buf)`	Receive. With `buf.size` 0, "tell me when there is something to read": nothing is taken and no buffer is held while it waits. With a buffer, what arrived is in it.	`proven_err_t`: `BUSY` with a receive in flight; `INVALID_STATE` after a close or before a connect has ended.
`proven_loop_recv_cancel(loop, &sock)`	Stop a receive in flight; it is answered with `PROVEN_ERR_CANCELLED`.	nothing.
`proven_loop_read(loop, &sock, buf)`	Read what has arrived, without waiting: what `proven_net_read` with `PROVEN_NET_DONT_WAIT` does on the connection - and on io_uring, a copy from the loop's pool (section 2). Use it for every read of an attached connection.	`proven_result_size_t`: the bytes read; `.err` `TIMEOUT` when nothing is there now, `EOF`, `RESET`.
`proven_loop_send(loop, &sock, bytes)`	Send. With nothing waiting the bytes go straight to the socket; what it does not take at once the loop copies and keeps, up to 64 KiB per connection. On io_uring every send is copied and handed to the kernel in the loop's next round (section 2). Your bytes are free when the call returns.	`proven_result_size_t`: the bytes taken; `.err` `AGAIN` when the rest did not fit; after a failed send, that failure.
`proven_loop_send_queued(&sock)`	How many bytes taken have not left yet.	`proven_size_t`.
`proven_loop_send_wait(loop, &sock, at_most)`	Have `writable` called once no more than `at_most` bytes wait (0: everything has left).	`proven_err_t`.
`proven_loop_close(loop, &sock)`	The end: what is in flight is answered `PROVEN_ERR_CANCELLED`, bytes still waiting are dropped (on io_uring, what the loop had not yet handed to the kernel is written to the socket first), then `closed`.	nothing.
`proven_loop_accept(loop, &acc, &listener, fn, ctx)`	Accept one connection: `fn(ctx, err, &conn)` once, with the connection - yours; copy it out - or the reason there is none. Arm it again from inside `fn`. `acc` is yours, zero-initialised before its first use.	`proven_err_t`: `BUSY` with an accept in flight.

`proven_loop_accept_cancel(loop, &acc)`	Stop accepting: one in flight is answered `PROVEN_ERR_CANCELLED`, and the listener may be closed.	nothing.
`proven_loop_scratch(loop)`	The loop's scratch buffer, 64 KiB.	`proven_mem_mut_t`.
`proven_loop_allocator(loop)`	The allocator the loop was made with.	`proven_allocator_t`.

The rules that hold it together:

- **Every operation started is answered exactly once**, by its function, on the loop's thread - and never from inside the call that started it, even when it could finish at once. Success, failure and cancellation all come this way, so a function of yours never runs inside your own call to the loop.
- **One receive at a time per socket, one accept at a time per acceptor.** Sends are not operations of yours: the loop takes the bytes.
- **The end is `proven_loop_close`, then `closed`.** Until `closed` the loop may still use the socket's memory; afterwards it never touches it. Close the connection itself in `closed`. For a graceful end - everything sent first - wait with `proven_loop_send_wait(loop, &sock, 0)` and close in `writable`.
- **A receive of zero bytes is how ten thousand idle connections cost no read buffers.** When it is answered, read with `proven_loop_read` into the scratch buffer, use what arrived, and keep only what cannot be used yet. Its contents mean nothing once your function returns, or calls anything that may use it too.
- **Keep what the loop holds for a slow peer bounded.** When `proven_loop_send_queued` says much is waiting, stop reading and ask `proven_loop_send_wait` to say when to go on - the example does.
- Once a connection is attached, read it with `proven_loop_read` and send through `proven_loop_send`, so that bytes keep their order: on io_uring the loop's pool may hold bytes the socket no longer has. A connection is given to one loop, once.

The program below is an echo server and its client on one loop. The test suite compiles and runs it:

```
#include <string.h>

/*
 * An echo server and its client on one loop, with nothing but the loop's operations.
 *
 * The server takes connections with an accept it arms again from inside its own function, and
 * waits for each connection's data with a receive of zero bytes: an idle connection holds no
 * read buffer. When data has arrived it reads it into the loop's scratch buffer and sends it
 * back; what the socket does not take at once, the loop keeps. The client connects through the
 * loop too, and reads with a receive into a buffer of its own, under a timer.
 */

#define MESSAGE "every operation is answered exactly once"
#define ROUNDS 3

typedef struct ex_app ex_app_t;

typedef struct {                /* one connection of the server */
    ex_app_t *app;
    proven_loop_sock_t sock;
    proven_net_conn_t conn;
    bool ending, used;
} ex_peer_t;

struct ex_app {
```

```
    proven_loop_t *loop;
    proven_net_listener_t listener;
    proven_loop_accept_t acc;
    ex_peer_t peers[4];
    int peers_closed;
    /* the client */
    proven_loop_sock_t sock;
    proven_net_conn_t conn;
    proven_loop_timer_t deadline;
    proven_byte_t buf[64];
    proven_size_t have;
    int rounds;
    bool echoed, client_closed, accepting_ended;
};

// ---- the server's connections ---------------------------------------------------

static void peer_end(ex_peer_t *p) {
    /* The client has said all it will: close once everything owed to it has left. */
    p->ending = true;
    if (proven_loop_send_wait(p->app->loop, &p->sock, 0) != PROVEN_OK) proven_loop_close(p->app->loop, &p-
>sock);
}

/* Read what has arrived and send it back. A client that sends faster than it reads would make
 * the loop hold more and more for it, so past 32 KiB waiting the server stops reading until it
 * has caught up: a send wait, and reading resumes in `writable`. */
static void peer_read(ex_peer_t *p) {
    proven_loop_t *loop = p->app->loop;
    proven_mem_mut_t scratch = proven_loop_scratch(loop);
    scratch.size = 16384;
    for (;;) {
        if (proven_loop_send_queued(&p->sock) > 32768) {
            if (proven_loop_send_wait(loop, &p->sock, 0) != PROVEN_OK) proven_loop_close(loop, &p->sock);
            return;
        }
        proven_result_size_t r = proven_loop_read(loop, &p->sock, scratch);
        if (r.err == PROVEN_ERR_TIMEOUT) break;                 /* all of it read: wait for more */
        if (r.err != PROVEN_OK) { peer_end(p); return; }        /* PROVEN_ERR_EOF: the client is done */
        proven_result_size_t s = proven_loop_send(loop, &p->sock, (proven_mem_view_t){ scratch.ptr,
r.value });
        if (s.err != PROVEN_OK) { proven_loop_close(loop, &p->sock); return; }
    }
    /* Nothing is held while this connection waits: a receive of zero bytes takes no buffer. */
    if (proven_loop_recv(loop, &p->sock, (proven_mem_mut_t){ 0 }) != PROVEN_OK) proven_loop_close(loop, &p-
>sock);
}

static void peer_received(void *ctx, proven_err_t err, proven_size_t n) {
    ex_peer_t *p = ctx;
    (void)n;                                                    /* a receive of zero bytes takes nothing */
    if (err != PROVEN_OK) return;                               /* PROVEN_ERR_CANCELLED: it is being closed */
    peer_read(p);
}

static void peer_writable(void *ctx, proven_err_t err) {
    ex_peer_t *p = ctx;
    if (err == PROVEN_ERR_CANCELLED) return;
    if (err != PROVEN_OK || p->ending) proven_loop_close(p->app->loop, &p->sock);
    else peer_read(p);
```

```
}

static void peer_closed(void *ctx) {
    ex_peer_t *p = ctx;
    (void)proven_net_close(&p->conn);                           /* the connection itself is closed here, last
*/
    p->used = false;
    /* The one client of this program has gone: stop taking connections. */
    if (++p->app->peers_closed == 1) proven_loop_accept_cancel(p->app->loop, &p->app->acc);
}

static const proven_loop_sock_fns_t peer_fns = { .received = peer_received, .writable = peer_writable, .closed
= peer_closed };

static void on_accept(void *ctx, proven_err_t err, proven_net_conn_t *conn) {
    ex_app_t *app = ctx;
    if (err != PROVEN_OK) {                                     /* PROVEN_ERR_CANCELLED: the listener may go
now */
        (void)proven_net_listener_close(&app->listener);
        app->accepting_ended = true;
        return;
    }
    ex_peer_t *p = NULL;
    for (int i = 0; i < 4 && !p; ++i) if (!app->peers[i].used) p = &app->peers[i];
    if (!p) { (void)proven_net_close(conn); }
    else {
        *p = (ex_peer_t){ .app = app, .conn = *conn, .used = true };    /* the connection is ours now */
        if (proven_loop_attach(app->loop, &p->sock, &p->conn, &peer_fns, p) != PROVEN_OK ||
            proven_loop_recv(app->loop, &p->sock, (proven_mem_mut_t){ 0 }) != PROVEN_OK)
{ (void)proven_net_close(&p->conn); p->used = false; }
    }
    /* Armed again from inside: a connection already waiting is taken without another round. */
    if (proven_loop_accept(app->loop, &app->acc, &app->listener, on_accept, app) != PROVEN_OK) app-
>accepting_ended = true;
}

// ---- the client ------------------------------------------------------------

static void client_deadline(void *ctx) {
    ex_app_t *app = ctx;
    proven_loop_recv_cancel(app->loop, &app->sock);             /* its `received` comes with
PROVEN_ERR_CANCELLED */
}

static void client_send(ex_app_t *app) {
    app->have = 0;
    proven_result_size_t s = proven_loop_send(app->loop, &app->sock, (proven_mem_view_t){ (const proven_byte_t
*)MESSAGE, strlen(MESSAGE) });
    /* A receive into the client's own buffer, which is the loop's until `received`. */
    if (s.err != PROVEN_OK || proven_loop_recv(app->loop, &app->sock, (proven_mem_mut_t){ app->buf, sizeof
app->buf }) != PROVEN_OK) {
        proven_loop_close(app->loop, &app->sock);
        return;
    }
    proven_loop_timer_set(app->loop, &app->deadline, 5000, client_deadline, app);
}

static void client_received(void *ctx, proven_err_t err, proven_size_t n) {
    ex_app_t *app = ctx;
    if (err != PROVEN_OK) { proven_loop_close(app->loop, &app->sock); return; }    /* the deadline passed, or
```

```
the server went */
    app->have += n;
    if (app->have < strlen(MESSAGE)) {                          /* a stream: the echo may come in pieces */
        if (proven_loop_recv(app->loop, &app->sock, (proven_mem_mut_t){ app->buf + app->have, sizeof app->buf
- app->have }) != PROVEN_OK)
            proven_loop_close(app->loop, &app->sock);
        return;
    }
    proven_loop_timer_cancel(app->loop, &app->deadline);
    if (memcmp(app->buf, MESSAGE, strlen(MESSAGE)) != 0) { proven_loop_close(app->loop, &app->sock); return; }
    if (++app->rounds < ROUNDS) { client_send(app); return; }
    app->echoed = true;
    proven_loop_close(app->loop, &app->sock);                   /* the server reads the end, and closes its
side */
}

static void client_connected(void *ctx, proven_err_t err) {
    ex_app_t *app = ctx;
    if (err != PROVEN_OK) { proven_loop_close(app->loop, &app->sock); return; }
    client_send(app);
}

static void client_closed(void *ctx) {
    ex_app_t *app = ctx;
    (void)proven_net_close(&app->conn);
    app->client_closed = true;
}

static const proven_loop_sock_fns_t client_fns = {
    .received = client_received, .connected = client_connected, .closed = client_closed
};

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    static ex_app_t app;
    EXAMPLE_REQUIRE(proven_loop_create(heap, &app.loop) == PROVEN_OK, "a loop");

    // ---- the server ------------------------------------------------------------
    proven_net_addr_t bound;
    EXAMPLE_REQUIRE(proven_net_listen(proven_net_addr_ipv4(127, 0, 0, 1, 0), 16, &app.listener, &bound) ==
PROVEN_OK, "a listener on a free port");
    EXAMPLE_REQUIRE(proven_loop_accept(app.loop, &app.acc, &app.listener, on_accept, &app) == PROVEN_OK,
"accepting");

    // ---- the client ------------------------------------------------------------
    EXAMPLE_REQUIRE(proven_loop_connect(app.loop, &app.sock, bound, &app.conn, &client_fns, &app) ==
PROVEN_OK, "a connect begun");

    // ---- run until everything has ended ----------------------------------------
    proven_net_deadline_t end = proven_net_deadline_in(10000);
    while (!(app.client_closed && app.accepting_ended) && proven_time_monotonic_now() < end) {
        EXAMPLE_REQUIRE(proven_loop_poll(app.loop, proven_net_deadline_in(100)) == PROVEN_OK, "a round");
    }
    EXAMPLE_REQUIRE(app.echoed && app.rounds == ROUNDS, "the message came back three times");
    EXAMPLE_REQUIRE(app.peers_closed == 1 && app.accepting_ended && !
proven_net_listener_is_open(&app.listener),
                    "the server closed its side after the client's, and stopped accepting");
    proven_loop_destroy(app.loop);
    return EXAMPLE_OK();
}
```

6. The event-driven server

```
proven_err_t proven_http_event_server_create(proven_loop_t *loop, const proven_http_event_server_config_t
*config,
                                             proven_http_event_server_t **out);
proven_err_t proven_http_event_server_listen(proven_http_event_server_t *server, proven_net_addr_t at,
proven_net_addr_t *bound);
void proven_http_event_server_destroy(proven_http_event_server_t *server);
proven_size_t proven_http_event_server_connections(const proven_http_event_server_t *server);
proven_err_t proven_http_event_server_set_max_streams(proven_http_event_server_t *server, proven_size_t n);
proven_err_t proven_http_event_server_set_window(proven_http_event_server_t *server, proven_size_t n);
void proven_http_event_server_stop_listening(proven_http_event_server_t *server);
typedef void (*proven_http_event_drained_fn)(void *ctx, proven_err_t why);
proven_err_t proven_http_event_server_shutdown(proven_http_event_server_t *server, proven_u32 grace_ms,
                                               proven_http_event_drained_fn on_drained, void *ctx);
```

A server is made on a loop, told where to listen, and does nothing until the loop runs.

Call	What it does	Returns
`proven_http_event_server_create(loop, &config, &server)`	Make a server.	`proven_err_t`: `INVALID_ARG` for no loop, no `on_request`, or a TLS configuration with no certificate; `NOMEM`.
`proven_http_event_server_listen(server, at, &bound)`	Listen at `at`; up to four addresses for one server. `bound` (optional) receives the address in use, with the port the system chose when `at` asked for port 0.	`proven_err_t`.
`proven_http_event_server_connections(server)`	Connections open now.	`proven_size_t`.
`proven_http_event_server_set_max_streams(server, n)`	Change `max_concurrent_streams` while the server runs: 1 to the configured value, for the connections open and those to come (Chapter 17, sections 6 and 9). On the loop's thread.	`proven_err_t`: `INVALID_ARG`.
`proven_http_event_server_set_window(server, n)`	A ceiling, from now on, on the request body a client may send ahead of the handler - per stream and per connection, HTTP/2 and HTTP/3 - for the connections open and those to come. What was granted stands; growing windows stop at it (Chapter 17, section 6). On the loop's thread.	`proven_err_t`: `INVALID_ARG` for 0 or more than 2^31-1.
`proven_http_event_server_stop_listening(server)`	Stop accepting; open connections go on, an HTTP/1.1 response from now on says `Connection: close`, HTTP/2 says goodbye.	nothing.

`proven_http_event_server_shutdown(server, grace_ms, on_drained, ctx)`	End gracefully (section 10); `on_drained(ctx, why)` when it is over.	`proven_err_t`: `INVALID_STATE` when already shutting down.
`proven_http_event_server_destroy(server)`	Close every connection and the listeners, and free the server. Exchanges still in progress get `on_done` with `PROVEN_ERR_RESET`. The loop is neither stopped nor freed. What the loop has not answered yet - an accept in flight, a connection's socket - keeps its memory until the loop's next round or `proven_loop_destroy` (section 5, rule 3). Null is accepted.	nothing.

Four functions. The configuration carries them in `on`, and `ctx` is passed to each:

Function	When	Notes
`on_request(ctx, stream, head)`	A request head has arrived and passed the server's checks.	Required. Answer here, or remember `stream` and answer later. `head` is good only until it returns.
`on_body(ctx, stream, piece, last)`	A piece of the request body; `last` is true with the final one, which may be empty.	Optional: without it a body is read and thrown away. `piece` is good only until it returns.
`on_writable(ctx, stream)`	Output that was held back has gone, and `proven_http_stream_write` will take more.	Optional.
`on_done(ctx, stream, why)`	The exchange is over.	Optional. Called **exactly once for every** `on_request`: `PROVEN_OK` when the whole response was handed to the connection, otherwise why it could not be.

A stream is one request and its response. A `proven_http_stream_t *` is valid from `on_request` until `on_done` returns, and not a moment longer. `on_done` is therefore the one place to free what you attached to it, and to cancel a timer or forget a job that still holds it: a function that arrives later with a stream that is over has nothing valid to call.

```
void proven_http_stream_set_user(proven_http_stream_t *stream, void *user);
void *proven_http_stream_user(const proven_http_stream_t *stream);
proven_net_addr_t proven_http_stream_peer(const proven_http_stream_t *stream);
```

`proven_http_stream_set_user` attaches a pointer of yours to the stream and `proven_http_stream_user` gives it back - a stream that was given none returns null, which an `on_done` that frees must allow for. `proven_http_stream_peer` is the client's address.

7. Answering

```
proven_err_t proven_http_stream_respond(proven_http_stream_t *stream, proven_u16 status,
                                        const proven_http_header_t *headers, proven_size_t header_count,
                                        proven_mem_view_t body);
proven_err_t proven_http_stream_begin(proven_http_stream_t *stream, proven_u16 status,
                                      const proven_http_header_t *headers, proven_size_t header_count,
                                      proven_u64 content_length);
proven_result_size_t proven_http_stream_write(proven_http_stream_t *stream, proven_mem_view_t data);
proven_err_t proven_http_stream_end(proven_http_stream_t *stream);
void proven_http_stream_abort(proven_http_stream_t *stream);
proven_size_t proven_http_stream_buffered(const proven_http_stream_t *stream);
```

All of it at once. `proven_http_stream_respond` sends a status, your headers and a body held in memory. The body is copied whatever its size, so it is for bodies that are small. `Content-Length`, `Date` and `Connection` are written for you and may not be among `headers`. It may be called inside `on_request` or at any later time on the loop's thread, while the stream is valid.

In pieces. `proven_http_stream_begin` sends the head and promises a body of `content_length` bytes, or - given `PROVEN_HTTP_EVENT_LENGTH_UNKNOWN` - a body sent chunked, whose length nobody knows yet. Then `proven_http_stream_write` offers bytes, and `proven_http_stream_end` finishes.

`proven_http_stream_write` **may take fewer bytes than you offer** - as many as fit under `max_buffered_output`, possibly none - and returns how many it took. That is not an error. It means the client has not taken what was already sent, and the rule is: **keep the rest, return, and offer it again when `on_writable` is called.** A producer written that way goes exactly as fast as its slowest client, and the server's memory for that connection stays under the limit whatever the size of the response.

Call	Returns
`proven_http_stream_respond`, `proven_http_stream_begin`	`proven_err_t`: `INVALID_STATE` when a response was already begun or the stream is over; `INVALID_ARG` for a status outside 200-999 or a header the server writes itself; `OUT_OF_BOUNDS` when the response head does not fit `max_head_bytes`; `NOMEM`.
`proven_http_stream_write`	`proven_result_size_t`: the bytes taken. `INVALID_STATE` before `begin`, after `end`, or when the stream is over; `OUT_OF_BOUNDS` for more than the `Content-Length` promised.
`proven_http_stream_flush`	`proven_err_t`: hands over what a compressed response holds back (Chapter 16, section 5), and does nothing for any other. `INVALID_STATE` before `begin`, after `end`, or when the stream is over; `RESET` when the connection ended over it.
`proven_http_stream_end`	`proven_err_t`: `INVALID_STATE` when no response was begun or it was already ended; `INVALID_FORMAT` when fewer bytes were written than promised - the connection is then closed, so that the client cannot take a short body for a whole one.
`proven_http_stream_abort`	nothing. The connection is closed at once and `on_done` is called with `PROVEN_ERR_RESET`.
`proven_http_stream_buffered`	`proven_size_t`: response bytes held for this connection that the client has not accepted.

`proven_http_stream_compress`	nothing. Asks that this response be compressed if the client accepts gzip; before the response begins. Chapter 16, section 6 - read its cautions first.

Three details that follow from the protocol and need nothing from you: a response to `HEAD`, and one with status 204 or 304, is sent without a body whatever you pass; a request that said `Expect: 100-continue` is told to go on when its body is first wanted; and after the response to a request that asked for it, or that could not be read to its end, the connection is closed rather than kept.

`on_done` may be called **inside** `proven_http_stream_respond` or `proven_http_stream_end`, when everything fits in the socket at once. After either returns, do not touch the stream or what `on_done` freed.

The program below answers in four ways - at once, later from a timer, streamed with a pump that gives way when it is refused, and after a body received in pieces. The test suite compiles and runs it:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/*
 * An HTTP server on an event loop: one thread, and no call in it that waits.
 *
 * The server tells these functions what happened, and they return at once. Four kinds of
 * answer are shown: at once; later, from a timer; streamed, giving way when the client is
 * slow; and after a request body has arrived piece by piece.
 *
 * The client is the blocking one of chapter 11, on the example's main thread.
 */

typedef struct {
    proven_loop_t *loop;
    int served;
    proven_http_event_server_t *server;
    proven_err_t ended;                /* what on_drained was told */
} ex_app_t;

/* What this program keeps for one request while it is being answered. */
typedef struct {
    ex_app_t *app;
    proven_http_stream_t *stream;
    proven_loop_timer_t timer;
    proven_size_t sent, total;         /* /count: how far the body has got */
    proven_size_t part;                /* /count: how much of the current line has gone */
    proven_size_t received;            /* /size: body bytes seen */
} ex_exchange_t;

/* Write lines until the server says "no more for now". It says so when the client has not
 * taken what was already sent; the rest waits for on_writable. */
static void count_pump(ex_exchange_t *x) {
    while (x->sent < x->total) {
        char line[32];
        int n = snprintf(line, sizeof line, "%u\n", (unsigned)x->sent);
        proven_result_size_t w = proven_http_stream_write(x->stream, (proven_mem_view_t){ (const proven_byte_t
*)line + x->part, (proven_size_t)n - x->part });
        if (w.err != PROVEN_OK) return;                  /* the client left */
        x->part += w.value;                             /* a write may take part of what it is given */
        if (x->part < (proven_size_t)n) return;          /* held back: on_writable will say when */
```

```
        x->part = 0;
        x->sent++;
    }
    (void)proven_http_stream_end(x->stream);
}

static void answer_later(void *ctx) {
    ex_exchange_t *x = ctx;
    (void)proven_http_stream_respond(x->stream, 200, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("worth the
wait\n")));
}

static void on_request(void *ctx, proven_http_stream_t *stream, const proven_http_request_t *head) {
    ex_app_t *app = ctx;
    ex_exchange_t *x = calloc(1, sizeof *x);
    if (!x) { proven_http_stream_abort(stream); return; }
    x->app = app;
    x->stream = stream;
    proven_http_stream_set_user(stream, x);

    /* `head` and everything in it are good only until this function returns. */
    if (proven_u8str_view_eq(head->target, PROVEN_LIT("/"))) {
        /* The simplest answer: all of it, now. */
        (void)proven_http_stream_respond(stream, 200, NULL, 0,
proven_mem_view_from_u8(PROVEN_LIT("hello\n")));
    } else if (proven_u8str_view_eq(head->target, PROVEN_LIT("/slow"))) {
        /* Not now: remember the stream and answer from a timer. The loop goes on meanwhile. */
        proven_loop_timer_set(app->loop, &x->timer, 50, answer_later, x);
    } else if (proven_u8str_view_eq(head->target, PROVEN_LIT("/count"))) {
        /* A body made as it goes, sent chunked because its length is not known first. */
        x->total = 20000;
        if (proven_http_stream_begin(stream, 200, NULL, 0, PROVEN_HTTP_EVENT_LENGTH_UNKNOWN) == PROVEN_OK)
count_pump(x);
    } else if (proven_u8str_view_eq(head->target, PROVEN_LIT("/size"))) {
        /* Nothing yet: the body is on its way, and on_body answers when it has all come. */
    } else {
        (void)proven_http_stream_respond(stream, 404, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("not
here\n")));
    }
}

static void on_body(void *ctx, proven_http_stream_t *stream, proven_mem_view_t piece, bool last) {
    (void)ctx;
    ex_exchange_t *x = proven_http_stream_user(stream);
    x->received += piece.size;
    if (last) {
        char text[32];
        int n = snprintf(text, sizeof text, "%u bytes\n", (unsigned)x->received);
        (void)proven_http_stream_respond(stream, 200, NULL, 0, (proven_mem_view_t){ (const proven_byte_t
*)text, (proven_size_t)n });
    }
}

/* The output that was held back has gone: carry on from where the writing stopped. */
static void on_writable(void *ctx, proven_http_stream_t *stream) {
    (void)ctx;
    ex_exchange_t *x = proven_http_stream_user(stream);
    if (x->total > 0) count_pump(x);
}
```

```
/* Called once for every on_request, whatever happened. The place to free what was attached. */
static void on_done(void *ctx, proven_http_stream_t *stream, proven_err_t why) {
    ex_app_t *app = ctx;
    ex_exchange_t *x = proven_http_stream_user(stream);
    if (!x) return;
    proven_loop_timer_cancel(app->loop, &x->timer);
    free(x);
    if (why == PROVEN_OK) app->served++;
}

/* The end, on the loop's thread: what is open finishes, and then the loop is stopped. */
static void on_drained(void *ctx, proven_err_t why) {
    ex_app_t *app = ctx;
    app->ended = why;
    proven_loop_stop(app->loop);
}
static void begin_end(void *ctx) {
    ex_app_t *app = ctx;
    if (proven_http_event_server_shutdown(app->server, 2000, on_drained, app) != PROVEN_OK)
proven_loop_stop(app->loop);
}

static void run(void *arg) { (void)proven_loop_run(((ex_app_t *)arg)->loop); }

static char g_url[64];
static proven_u8str_view_t url_for(proven_u16 port, const char *path) {
    int n = snprintf(g_url, sizeof g_url, "http://127.0.0.1:%u%s", (unsigned)port, path);
    return (proven_u8str_view_t){ (const proven_byte_t *)g_url, (proven_size_t)n };
}

static bool body_equals(proven_http_client_response_t *resp, proven_allocator_t alloc, const char *want) {
    proven_u8str_t body = { 0 };
    bool ok = proven_http_client_read_all(resp, alloc, &body, 1024 * 1024) == PROVEN_OK &&
              proven_u8str_view_eq(proven_u8str_as_view(&body), (proven_u8str_view_t){ (const proven_byte_t
*)want, strlen(want) });
    proven_u8str_destroy(alloc, &body);
    return ok;
}

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    ex_app_t app = { 0 };

    // ---- the server ----------------------------------------------------------
    /* A server is made on a loop and listens; it does nothing until the loop runs. */
    EXAMPLE_REQUIRE(proven_loop_create(heap, &app.loop) == PROVEN_OK, "a loop");
    proven_http_event_server_config_t config = {
        .on = { on_request, on_body, on_writable, on_done },
        .ctx = &app,
        .max_buffered_output = 16 * 1024,        /* hold at most this much per connection that the client has
not taken */
    };
    proven_http_event_server_t *server = NULL;
    proven_net_addr_t at;
    EXAMPLE_REQUIRE(proven_http_event_server_create(app.loop, &config, &server) == PROVEN_OK &&
                    proven_http_event_server_listen(server, proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4,
0), &at) == PROVEN_OK, "an event-driven server on a free port");
    app.server = server;

    /* This example gives the loop a thread of its own, so that its main thread can be a client. */
```

```
    proven_job_sys_t *threads = NULL;
    proven_job_group_t running;
    proven_job_group_init(&running);
    EXAMPLE_REQUIRE(proven_job_system_init(heap, 1, 4, &threads) == PROVEN_OK &&
                    proven_job_group_submit(threads, &running, run, &app) == PROVEN_OK, "the loop runs on
another thread");

    // ---- four requests ------------------------------------------------------
    proven_http_client_config_t client_config = { .alloc = heap, .max_idle_connections = 1 };
    proven_http_client_t *client = NULL;
    EXAMPLE_REQUIRE(proven_http_client_create(&client_config, &client) == PROVEN_OK, "a client");
    proven_http_client_response_t resp;

    EXAMPLE_REQUIRE(proven_http_client_get(client, url_for(at.port, "/"), &resp) == PROVEN_OK &&
body_equals(&resp, heap, "hello\n"), "answered at once");
    proven_http_client_finish(&resp);
    EXAMPLE_REQUIRE(proven_http_client_get(client, url_for(at.port, "/slow"), &resp) == PROVEN_OK &&
body_equals(&resp, heap, "worth the wait\n"), "answered 50 ms later, from a timer");
    proven_http_client_finish(&resp);

    /* Twenty thousand lines. Whenever the server holds back, the handler stops and on_writable resumes it. */
    EXAMPLE_REQUIRE(proven_http_client_get(client, url_for(at.port, "/count"), &resp) == PROVEN_OK &&
resp.status == 200, "a streamed response begins");
    proven_u8str_t counted = { 0 };
    EXAMPLE_REQUIRE(proven_http_client_read_all(&resp, heap, &counted, 1024 * 1024) == PROVEN_OK, "and is read
to its end");
    proven_u8str_view_t all = proven_u8str_as_view(&counted);
    EXAMPLE_REQUIRE(all.size == 108890 && memcmp(all.ptr, "0\n1\n2\n", 6) == 0 && memcmp(all.ptr + all.size -
6, "19999\n", 6) == 0, "every line arrived, in order");
    proven_u8str_destroy(heap, &counted);
    proven_http_client_finish(&resp);

    static proven_byte_t upload[50000];
    proven_http_client_request_t post = { .method = PROVEN_LIT("POST"), .url = url_for(at.port, "/
size"), .body = { upload, sizeof upload } };
    EXAMPLE_REQUIRE(proven_http_client_send(client, &post, &resp) == PROVEN_OK && body_equals(&resp, heap,
"50000 bytes\n"), "a body counted piece by piece");
    proven_http_client_finish(&resp);
    proven_http_client_destroy(client);

    // ---- done ---------------------------------------------------------------
    /* End gracefully, on the loop's thread; on_drained stops the loop. Then take the server
     * apart: nothing else is touching it by then. */
    app.ended = PROVEN_ERR_INVALID_STATE;
    EXAMPLE_REQUIRE(proven_loop_post(app.loop, begin_end, &app) == PROVEN_OK, "the end is asked for");
    proven_job_group_wait(threads, &running);
    EXAMPLE_REQUIRE(app.ended == PROVEN_OK && app.served == 4, "it ended with nothing cut, and four exchanges
ended with on_done(PROVEN_OK)");
    proven_http_event_server_destroy(server);
    proven_loop_destroy(app.loop);
    proven_job_system_close(threads);
    proven_job_system_destroy(threads);
    return EXAMPLE_OK();
}
```

Look at `count_pump`: it is the shape every streamed response has. It is called from `on_request` to start and from `on_writable` to go on, it keeps its position in the exchange's own state, and it returns the moment a write takes less than was offered. Whether a given run is ever refused depends on how fast the client reads; the function is correct either way.

8. Request bodies

```
void proven_http_stream_pause(proven_http_stream_t *stream);
void proven_http_stream_resume(proven_http_stream_t *stream);
```

A body arrives through `on_body` in pieces of whatever size the network delivered, with the framing - a `Content-Length` or chunks - already taken off. A request with no body gets no `on_body` call at all; one with a body gets a last call with `last` true.

A handler may answer before the body has arrived, or without reading it. The rest of the body is then read and thrown away if that is cheap, and the connection is closed after the response if it is not.

`proven_http_stream_pause` is backpressure in the other direction: no more pieces are delivered, and the connection is not read, until `proven_http_stream_resume`. A handler that writes what it receives to something slower than the network uses the pair so that the client is slowed instead of the body being piled up in memory. Pieces that were waiting may arrive inside the call to `proven_http_stream_resume`.

9. Limits, time and refusals

Zero-initialise a `proven_http_event_server_config_t`, set `on.on_request`, and set what you need. Every zero field has a default:

Field	Meaning	Default
`alloc`	The allocator for connections and buffers.	the loop's
`max_connections`	Connections open at once. Further ones wait, unaccepted, in the system's backlog.	10,000
`max_head_bytes`	Largest request head; also the largest response head.	16 KiB
`max_headers`	Most header fields in a request.	64
`max_body_bytes`	Largest request body.	1 MiB
`max_buffered_output`	Response bytes held for one connection before writes are refused.	64 KiB
`head_timeout_ms`	From a request's first byte to the end of its head. With TLS it covers the handshake too.	10 s
`body_timeout_ms`	Between pieces of a request body.	30 s
`write_timeout_ms`	Output held with none of it accepted by the client.	30 s
`idle_timeout_ms`	An open connection with no request on it.	60 s
`linger_ms`	How long a connection that is being closed after its answer is still read from, so that the answer is not lost to a reset. It counts as a connection until then.	1 s
`tls`	A TLS configuration with a certificate, or null for plain HTTP.	null

	With its `early_data` (since v0.84.0) a request that arrives as TLS early data reaches its handler before the handshake is over, with `proven_http_stream_early` true, and may be answered at once; only GET, HEAD and OPTIONS without a body get that far - the server answers anything else that arrives early with 425 itself (Chapter 14, section 6).	
`http2`	Let a connection be HTTP/2 - by ALPN over TLS, by its first bytes in the clear - beside HTTP/1.1 on the same port, for the same handler: Chapter 17, section 6.	false
`max_concurrent_streams`	HTTP/2: requests in progress on one connection.	100
`http2_window_bytes`	HTTP/2: request-body bytes a client may send on a stream, and on a connection as a whole, before the handler has taken them. What a paused request can make a connection hold; a large upload waits for room each time it has sent this much. With 0 the window starts there and grows while bodies are read - doubling while half of it is taken within two round trips, up to 16 MiB a stream and 24 MiB a connection (v0.75.0, as HTTP/3's since v0.71.0); a body not read stops the growth. A value set stays fixed.	65,535, growing
`http3`	HTTP/3 too, over QUIC on a UDP socket beside each TCP listener on the same port, for the same handler, and `Alt-Svc` on the TCP responses: Chapter 17, section 9. Needs `tls`, offering `h3`.	false
`http3_retry`	HTTP/3: answer each client's first packet with a Retry, so that no state is kept for an address that has not shown it receives - a round trip more per connection, for a server under load.	false
`http3_window_bytes`	HTTP/3: request-body bytes a client may send on a stream, and on a	256 KiB a stream and 1 MiB a connection, growing while bodies

	connection as a whole, before the handler has taken them (QUIC's flow control).	are read as fast as they come, up to 16 MiB and 24 MiB; a value set is fixed
`http3_header_table_bytes`	HTTP/3: the QPACK dynamic table a client's encoder may fill (RFC-0032): said in SETTINGS with 16 streams that may wait for its inserts; made when a client first uses it, about twice this then. 32 to 1 MiB. Since v0.96.0 also the most of the client's table the server's encoder fills, referring only to what the client acknowledged.	4,096
`http3_no_header_table`	HTTP/3: no such table; every request head arrives in full.	false

A request the server will not serve never reaches `on_request`. It is answered and the connection closed:

Status	For
400	A head that is not HTTP; an HTTP/1.1 request without exactly one `Host`; body framing that contradicts itself; a chunk that is not one.
408	A head begun and not finished within `head_timeout_ms`.
413	A body longer than `max_body_bytes`.
431	A head larger than `max_head_bytes`, or with more fields than `max_headers`.
501	A transfer coding that is not `chunked`.
505	A version that is not HTTP/1.0 or HTTP/1.1.

When one of these happens after `on_request` - a body that turns out too long, or malformed part-way - the exchange ends with `on_done` and the reason, and the client gets the status if a response had not already begun. A connection that passes `body_timeout_ms`, `write_timeout_ms` or `idle_timeout_ms` is closed; an exchange in progress on it ends with `on_done(PROVEN_ERR_TIMEOUT)`. A client that goes away ends its exchange with `PROVEN_ERR_RESET`.

Requests sent one after another on a connection without waiting - pipelined - are served in order, one stream at a time.

10. Threads, TLS and shutting down

Threads. One loop is one thread, and the server belongs to it. A handler that must compute or wait for something submits a job, returns, and the job posts the answer back:

1. `on_request` copies what it needs from the head, remembers the stream, and submits the job.
2. The job works on another thread, touching nothing of the server.
3. It calls `proven_loop_post` with a function that, on the loop's thread, calls `proven_http_stream_respond`.
4. If `on_done` came first - the client left - that function must find out and do nothing. The usual way is for `on_done` to mark the exchange's own state as finished and leave freeing it to whichever of the two runs last.

TLS. Give the configuration a `proven_tls_config_t` with a certificate (Chapter 14, section 2) and every connection is TLS; nothing else in your code changes. The handshake is carried by the loop like any other reading and writing, and is subject to `head_timeout_ms`. The configuration must outlive the

server. A handshake's public-key work is done on the loop's thread, and while it runs nothing else is served: Chapter 14, section 9, gives what one costs.

Shutting down: `proven_http_event_server_shutdown(server, grace_ms, on_drained, ctx)`. On the loop's thread. From the call nothing new is taken; an HTTP/1.1 connection with no request on it is closed; one where a request has begun to arrive, or is being answered, finishes it with `Connection: close`; an HTTP/2 connection says goodbye (GOAWAY) and finishes its open streams; a WebSocket is closed with 1001, "going away", and its peer's answer waited for. When `grace_ms` has passed, what is left is ended - an exchange gets `on_done(PROVEN_ERR_RESET)`, a WebSocket `on_closed`. `on_drained(ctx, why)` is called once, from the loop, when the last connection has gone: `why` is `PROVEN_OK`, or `PROVEN_ERR_TIMEOUT` when the deadline cut something. It may call `proven_loop_stop`; then destroy the server and the loop after the loop has returned. A shutdown is final - the server listens and adopts no more. The deadline bounds connections, not handlers: work a handler sent to another thread comes back to a stream that is gone.

`proven_http_event_server_stop_listening` alone is the first step of that, without the rest: nobody new, HTTP/1.1 responses say `Connection: close`, HTTP/2 says goodbye - and connections end when their clients let them, or at `idle_timeout_ms`. Destroy the server on the loop's thread or after the loop has stopped - not while another thread is inside `proven_loop_run`. The example of section 7 ends with a shutdown.

11. WebSocket on the loop

Chapter 12's WebSocket connection has calls that wait, and so occupies a thread for as long as it is open. `ws_event.h` is the other form: a request to the server of this chapter is turned into a WebSocket inside `on_request`, and from then on the loop tells your functions what arrived. A connection that is silent holds its state and no buffer.

```
proven_err_t proven_ws_event_accept(proven_http_stream_t *stream, const proven_http_request_t *head,
                                    const proven_ws_event_config_t *config, proven_ws_stream_t **out);
proven_err_t proven_ws_stream_send(proven_ws_stream_t *ws, bool text, proven_mem_view_t data);
proven_err_t proven_ws_stream_send_piece(proven_ws_stream_t *ws, bool text, proven_mem_view_t data, bool
first, bool last);
proven_err_t proven_ws_stream_ping(proven_ws_stream_t *ws, proven_mem_view_t data);
void proven_ws_stream_close(proven_ws_stream_t *ws, proven_u16 code, proven_u8str_view_t reason);
void proven_ws_stream_abort(proven_ws_stream_t *ws);
void proven_ws_stream_pause(proven_ws_stream_t *ws);
void proven_ws_stream_resume(proven_ws_stream_t *ws);
void proven_ws_stream_set_user(proven_ws_stream_t *ws, void *user);
void *proven_ws_stream_user(const proven_ws_stream_t *ws);
proven_net_addr_t proven_ws_stream_peer(const proven_ws_stream_t *ws);
proven_size_t proven_ws_stream_buffered(const proven_ws_stream_t *ws);
```

Accepting. `proven_ws_event_accept` is called inside `on_request`, while `head` is valid. It checks the request, answers 101, and turns the connection into a WebSocket. **The HTTP exchange ends inside that call**: `on_done(PROVEN_OK)` is called for the stream, which is then over, and `*out` is what you hold until `on_closed` returns.

It returns	Meaning	What the handler does
`PROVEN_OK`	The connection is a WebSocket.	Attach its state to `*out` and return.
`PROVEN_ERR_NOT_FOUND`	The request did not ask for a WebSocket. Nothing was sent.	Answer it as the ordinary request it is.
`PROVEN_ERR_UNSUPPORTED`	An upgrade for a version other than 13. Nothing was sent.	Answer 426 with `Sec-WebSocket-Version: 13`.

`PROVEN_ERR_INVALID_FORMAT`	An upgrade, but malformed. Nothing was sent.	Answer 400.
`PROVEN_ERR_INVALID_ARG`	No `on_message`, or a subprotocol that is not a token.	A defect in the program.
`PROVEN_ERR_INVALID_STATE`	Not inside `on_request` for this stream, or a response was already begun.	A defect in the program.
`PROVEN_ERR_NOMEM`		Answer 503, or abort.
`PROVEN_ERR_RESET`	The client vanished at that moment; `on_done` has been called.	Nothing: the stream is over.

A `proven_ws_event_config_t` is zero-initialised, given `on.on_message`, and whatever else is needed:

Field	Meaning	Default
`on.on_message(ctx, ws, text, piece, first, last)`	A piece of a message. Required.	
`on.on_writable(ctx, ws)`	A send that was refused will now be taken.	none
`on.on_closed(ctx, ws, code, why)`	The connection is over. Called exactly once.	none
`ctx`	Passed to every callback.	
`subprotocol`	Named in the answer when not empty. Name one only if the client offered it (`proven_ws_request_offers`).	none
`max_message_bytes`	Largest message accepted, its pieces together.	1 MiB
`max_buffered_output`	Output held before sends are refused.	64 KiB
`ping_interval_ms`	Silence from the client after which it is sent a ping.	30 s
`pong_timeout_ms`	Further silence after which the connection is ended.	10 s
`close_timeout_ms`	How long to wait for the answer to a close, counted from the last output the client took.	5 s
`compress`, `compress_level`, `compress_window_bits`, `compress_no_takeover`, `compress_peer_no_takeover`	Accept compressed messages when the client offers them: Chapter 12, section 9. Each side keeps its window between messages (about 64 KiB held per connection) unless `compress_no_takeover` is set or the client forbids it. A compressed message arrives inflated, in pieces of at most	off; 6; 11 with takeover, 15 without; off

	16 KiB, and `max_message_bytes` counts the inflated size.	

Messages arrive in pieces. `on_message` is given each message in the pieces the network delivered, with `first` and `last` marking its ends: a message of any size passes through without being assembled or held, and `piece` is good only until the function returns. Text is checked to be UTF-8 over the whole message, and a piece may end in the middle of a character. A handler that wants whole messages collects the pieces itself, up to a limit of its own choosing - the program below does. `max_message_bytes` is the server's limit, not a buffer: a message past it is refused with close code 1009.

A send is taken whole or not at all. `proven_ws_stream_send` queues one message when less than `max_buffered_output` is held, and otherwise takes nothing and returns `PROVEN_ERR_AGAIN`: keep the message, and send it again when `on_writable` is called. That is the difference from the HTTP body of section 7, whose write says how many bytes it took - a frame cannot be half-sent without the library choosing where to cut your message. The most a connection holds is therefore the limit and one message of your choosing; for a large message produced as it goes, `proven_ws_stream_send_piece` sends it a fragment at a time under the same rule. A handler that only echoes can stop reading while it waits, with `proven_ws_stream_pause`, so that a client that sends faster than it reads is slowed rather than buffered.

Call	Returns
`proven_ws_stream_send`, `proven_ws_stream_send_piece`	`proven_err_t`: `AGAIN` when the output limit is reached, with nothing taken; `INVALID_STATE` after a close was sent, for `first` inside a message, or for a piece that is not `first` outside one; `RESET` when the connection is gone.
`proven_ws_stream_ping`	the same; `OUT_OF_BOUNDS` for more than 125 bytes.
`proven_ws_stream_close`	nothing. Sends a close with `code` and `reason`, and ends the connection when the client answers or the close timeout passes; `on_closed` follows later. A code that may not be sent becomes 1000.
`proven_ws_stream_abort`	nothing. Ends the connection at once with no close frame; `on_closed(1006, PROVEN_ERR_RESET)` is called inside the call.
`proven_ws_stream_pause`, `proven_ws_stream_resume`	nothing. Stop and restart the delivery of messages and the reading of the connection.
`proven_ws_stream_buffered`	`proven_size_t`: bytes queued for the client and not yet accepted by it.
`proven_ws_stream_compressed`	`bool`: whether `compress` was set and the client agreed.
`proven_ws_stream_set_compress`	nothing. Whether the messages sent from now on are compressed, on a connection that agreed to it (Chapter 12, section 9).

How it ends. `on_closed` is called exactly once, and what it is given says how:

code	why	What happened

the client's code (1005 if its close had none)	`PROVEN_OK`	The client closed, and was answered.
the code this side sent	`PROVEN_OK`	This side closed, and the client answered.
the code this side sent	`PROVEN_ERR_TIMEOUT`	This side closed, and the client never answered.
1002, 1007, 1009	`PROVEN_ERR_INVALID_FORMAT`, `PROVEN_ERR_INVALID_ENCODING`, `PROVEN_ERR_OUT_OF_BOUNDS`	The client broke the protocol, sent text that is not UTF-8, or a message past the limit; it was sent that code.
1006	`PROVEN_ERR_TIMEOUT`	It answered neither data nor a ping within the two liveness limits.
1006	`PROVEN_ERR_RESET`	It vanished without a close frame; or `proven_ws_stream_abort`; or the server was destroyed.

Pings from the client are answered for you, and its pongs need nothing from you. Liveness is judged by what is received: a client that is sent data and never reads it is not detected by that alone - but nothing more is queued for it than the output limit, and once it stops answering pings it is dropped.

The test suite compiles and runs this program:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/*
 * WebSocket on the event-driven server: connections that cost no thread.
 *
 * A request becomes a WebSocket inside on_request. From then on the loop tells these functions
 * what arrived, and they return at once. This server shouts back: every text message is
 * answered in capitals.
 *
 * The client is the blocking one of chapter 12, on the example's main thread.
 */

typedef struct {
    proven_loop_t *loop;
    int opened, closed;
    proven_u16 last_code;
} ex_app_t;

/* What this program keeps for one connection: the message being collected. */
typedef struct {
    ex_app_t *app;
    char text[4096];
    proven_size_t len;
    bool waiting;                /* a reply the server would not take yet */
} ex_peer_t;

/* Send the reply. If the client is not taking what was sent before, the server refuses it:
 * keep it, stop reading from this client, and try again when on_writable says so. */
static void reply(proven_ws_stream_t *ws, ex_peer_t *peer) {
```

```
    proven_err_t err = proven_ws_stream_send(ws, true, (proven_mem_view_t){ (const proven_byte_t *)peer->text,
peer->len });
    if (err == PROVEN_ERR_AGAIN) {
        peer->waiting = true;
        proven_ws_stream_pause(ws);
        return;
    }
    peer->waiting = false;
    peer->len = 0;
}

/* A message arrives in the pieces the network delivered. This handler wants whole messages,
 * so it collects them - up to a limit of its own, which is the point of doing it by hand. */
static void on_message(void *ctx, proven_ws_stream_t *ws, bool text, proven_mem_view_t piece, bool first, bool
last) {
    (void)ctx;
    ex_peer_t *peer = proven_ws_stream_user(ws);
    if (first) peer->len = 0;
    if (!text || peer->len + piece.size > sizeof peer->text) {
        proven_ws_stream_close(ws, 1009, PROVEN_LIT("short text only"));       /* 1009: too big */
        return;
    }
    for (proven_size_t i = 0; i < piece.size; ++i) {
        proven_byte_t ch = piece.ptr[i];
        peer->text[peer->len++] = (char)(ch >= 'a' && ch <= 'z' ? ch - 32 : ch);
    }
    if (last) reply(ws, peer);
}

/* There is room again. */
static void on_writable(void *ctx, proven_ws_stream_t *ws) {
    (void)ctx;
    ex_peer_t *peer = proven_ws_stream_user(ws);
    if (!peer->waiting) return;
    reply(ws, peer);
    if (!peer->waiting) proven_ws_stream_resume(ws);
}

/* Called once, however the connection ended. The place to free what was attached. */
static void on_closed(void *ctx, proven_ws_stream_t *ws, proven_u16 code, proven_err_t why) {
    ex_app_t *app = ctx;
    (void)why;
    free(proven_ws_stream_user(ws));
    app->last_code = code;
    app->closed++;
}

static void on_request(void *ctx, proven_http_stream_t *stream, const proven_http_request_t *head) {
    ex_app_t *app = ctx;
    ex_peer_t *peer = calloc(1, sizeof *peer);
    if (!peer) { proven_http_stream_abort(stream); return; }
    peer->app = app;

    proven_ws_event_config_t config = {
        .on = { on_message, on_writable, on_closed },
        .ctx = app,
        .max_message_bytes = sizeof peer->text,
    };
    proven_ws_stream_t *ws = NULL;
    /* Check the request, answer 101, and turn the connection into a WebSocket. The HTTP
```

```
     * exchange ends inside this call (on_done is called, if there is one). */
    proven_err_t err = proven_ws_event_accept(stream, head, &config, &ws);
    if (err != PROVEN_OK) {
        free(peer);
        /* Nothing was sent: the request is still an ordinary one, and is answered as such. */
        if (err == PROVEN_ERR_UNSUPPORTED) {
            proven_http_header_t version = { PROVEN_LIT("Sec-WebSocket-Version"), PROVEN_LIT("13") };
            (void)proven_http_stream_respond(stream, 426, &version, 1, (proven_mem_view_t){ 0 });
        } else if (err != PROVEN_ERR_RESET) {
            (void)proven_http_stream_respond(stream, err == PROVEN_ERR_NOT_FOUND ? 404 : 400, NULL, 0,
(proven_mem_view_t){ 0 });
        }
        return;
    }
    /* `stream` is over now. `ws` is valid until on_closed returns. */
    proven_ws_stream_set_user(ws, peer);
    app->opened++;
}

static void run(void *arg) { (void)proven_loop_run(((ex_app_t *)arg)->loop); }

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    ex_app_t app = { 0 };

    // ---- a server on a loop, on its own thread ------------------------------------
    EXAMPLE_REQUIRE(proven_loop_create(heap, &app.loop) == PROVEN_OK, "a loop");
    /* HTTP/2 too, and WebSocket on it (RFC 8441): a client that speaks HTTP/2 opens its
     * WebSocket as a stream of that connection. on_request above is the same for both - it is
     * this program's client, over HTTP/1.1, that the run below uses. */
    proven_http_event_server_config_t config = { .on = { .on_request = on_request }, .ctx = &app, .http2 =
true, .http2_websocket = true };
    proven_http_event_server_t *server = NULL;
    proven_net_addr_t at;
    EXAMPLE_REQUIRE(proven_http_event_server_create(app.loop, &config, &server) == PROVEN_OK &&
                    proven_http_event_server_listen(server, proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4,
0), &at) == PROVEN_OK, "an event-driven server on a free port");
    proven_job_sys_t *threads = NULL;
    proven_job_group_t running;
    proven_job_group_init(&running);
    EXAMPLE_REQUIRE(proven_job_system_init(heap, 1, 4, &threads) == PROVEN_OK &&
                    proven_job_group_submit(threads, &running, run, &app) == PROVEN_OK, "the loop runs on
another thread");

    // ---- a client -----------------------------------------------------------------
    /* A blocking client is the simplest way to show the other end. */
    proven_http_client_config_t client_config = { .alloc = heap };
    proven_http_client_t *client = NULL;
    EXAMPLE_REQUIRE(proven_http_client_create(&client_config, &client) == PROVEN_OK, "an HTTP client");
    char url[64];
    int n = snprintf(url, sizeof url, "ws://127.0.0.1:%u/shout", (unsigned)at.port);
    proven_ws_conn_config_t ws_config = { .alloc = heap };
    proven_ws_conn_t *conn = NULL;
    EXAMPLE_REQUIRE(proven_ws_conn_connect(client, (proven_u8str_view_t){ (const proven_byte_t *)url,
(proven_size_t)n }, NULL, 0, PROVEN_LIT(""),
                                           &ws_config, &conn, NULL) == PROVEN_OK, "the server accepts a
WebSocket");

    proven_ws_message_t msg;
    EXAMPLE_REQUIRE(proven_ws_conn_send_text(conn, PROVEN_LIT("hello, loop")) == PROVEN_OK &&
```

```
                    proven_ws_conn_receive(conn, proven_net_deadline_in(5000), &msg) == PROVEN_OK &&
                    msg.text && msg.data.size == 11 && memcmp(msg.data.ptr, "HELLO, LOOP", 11) == 0, "a
message comes back in capitals");
    /* However the client cuts a message up, the handler sees one message. */
    EXAMPLE_REQUIRE(proven_ws_conn_send_part(conn, true, proven_mem_view_from_u8(PROVEN_LIT("in three ")),
false) == PROVEN_OK &&
                    proven_ws_conn_send_part(conn, true, proven_mem_view_from_u8(PROVEN_LIT("pieces, ")),
false) == PROVEN_OK &&
                    proven_ws_conn_send_part(conn, true, proven_mem_view_from_u8(PROVEN_LIT("one answer")),
true) == PROVEN_OK &&
                    proven_ws_conn_receive(conn, proven_net_deadline_in(5000), &msg) == PROVEN_OK &&
                    msg.data.size == 27 && memcmp(msg.data.ptr, "IN THREE PIECES, ONE ANSWER", 27) == 0,
"three fragments are one message to the handler");

    // ---- closing ----------------------------------------------------------
    EXAMPLE_REQUIRE(proven_ws_conn_close(conn, 1000, PROVEN_LIT("done")) == PROVEN_OK, "the client's close is
answered");
    proven_ws_conn_destroy(conn);
    proven_http_client_destroy(client);

    /* Stop the loop before looking at what its thread wrote. */
    proven_loop_stop(app.loop);
    proven_job_group_wait(threads, &running);
    EXAMPLE_REQUIRE(app.opened == 1 && app.closed == 1 && app.last_code == 1000, "one connection was opened,
and closed once, with the client's code");
    proven_http_event_server_destroy(server);
    proven_loop_destroy(app.loop);
    proven_job_system_close(threads);
    proven_job_system_destroy(threads);
    return EXAMPLE_OK();
}
```

Over HTTP/2 (RFC 8441)

With `http2_websocket` set (and `http2`), the server says so in its first HTTP/2 settings, and a client that knows RFC 8441 opens its WebSocket as a stream of the HTTP/2 connection it already has, instead of a connection of its own. **The handler is the same**: the request reaches `on_request` as CONNECT, with its path as the target and a `:protocol` field first, `proven_ws_check_request` recognises it (with an empty key - there is none), and `proven_ws_event_accept` answers 200 instead of 101. The program above enables it with one line; its own client is HTTP/1.1, which the same server goes on serving.

- **Each such WebSocket holds a stream** - one of `max_concurrent_streams` (100 by default) - for as long as it is open. A connection full of WebSockets refuses new requests.
- Its frames travel in DATA frames, so HTTP/2's flow control is under the WebSocket's own: `http2_window_bytes` is what a client may send before the WebSocket has taken it (with 0, a window that grows while the WebSocket reads, as for any body), and `on_writable` comes when the stream's output has moved.
- Its end is END_STREAM from each side after the close handshake - until the client's arrives the stream keeps its place; a peer that resets the stream is a WebSocket closed with 1006 and `PROVEN_ERR_RESET`. A shutdown closes it with 1001. A handler that refuses the request (any answer but 2xx) ends the stream as it would a request's.
- Without `http2_websocket` a client must not try (the setting says so), and a stream that does is refused with PROTOCOL_ERROR.
- **Not here**: this library's own WebSocket client over HTTP/2 - `proven_ws_event_connect` makes a connection of its own, as before. Tested against Node's HTTP/2 client; no browser was tried.

A connection this side makes

```
proven_err_t proven_ws_event_connect(proven_loop_t *loop, const proven_ws_event_connect_options_t *options,
proven_ws_stream_t **out);
```

A program on a loop is as often the one that asks. `proven_ws_event_connect` begins a connection to a server and returns at once; what it gives back is a `proven_ws_stream_t`, **the same type a server's connections are**. Every call of this section works on it as it does on one of those, and its messages arrive through the same three callbacks: a handler written for one side serves the other.

What differs is the beginning.

`url`	`ws://` or `wss://`. Its host goes into `Host` and, for `wss`, is the name the certificate must be for
`address`	Where to connect. Needed unless the host is an IP address: **names are not resolved on a loop** (section 12 says why, and what to do)
`tls`	For `wss`. Without it a `wss` URL is refused - never connected in the clear instead
`headers`, `protocols`	Fields of your own (`Origin`, `Authorization`), and the subprotocols to offer
`connect_timeout_ms`	To be connected, secured and accepted. 0: 10 s
`on_open`	The server accepted. It is given the subprotocol the server chose
`ws`	A `proven_ws_event_config_t` - the callbacks, limits, pings and compression of the connection, as a server gives `proven_ws_event_accept`. `compress` here offers the extension

Nothing may be sent until the server has accepted: the sending calls answer `PROVEN_ERR_INVALID_STATE`, and `on_open` says when it has (sending from inside it is fine). `proven_ws_stream_abort` ends an attempt that is still under way, and so does `proven_ws_stream_close`.

`on_closed` **is called exactly once for every connection that was begun** - also for one that never became a WebSocket. Its code is then 0, and `why` says what happened:

`why`, with code 0	What happened
`PROVEN_ERR_REFUSED`	Nothing listens there - or the server answered with anything but 101. No redirect is followed and no challenge answered
`PROVEN_ERR_TIMEOUT`	Not connected, secured and accepted within `connect_timeout_ms`
`PROVEN_ERR_INVALID_FORMAT`	The answer was not HTTP, or was a 101 that is not this handshake's: a wrong `Sec-WebSocket-Accept`, a subprotocol or an extension that was not offered
`PROVEN_ERR_RESET`, `PROVEN_ERR_UNREACHABLE`, a TLS error	The connection was lost, could not be made, or could not be secured

A client's frames are masked, as the protocol requires of them; the keys come from a generator seeded from the system when the connection is begun. Not here: a proxy, reconnection (begin again), and WebSocket over HTTP/2.

This program has both ends on one loop. It is compiled and run by the test suite:

```
#include <stdio.h>
#include <string.h>

/*
```

```
 * A WebSocket client on the loop: a connection that costs no thread on this side either.
 *
 * proven_ws_event_connect begins a connection and returns at once. What it gives back is a
 * proven_ws_stream_t - the type a server's connections are - so the calls and the callbacks
 * are the ones the server above uses. What is new is the beginning: on_open says when the
 * server has accepted, and until then nothing may be sent.
 *
 * One loop carries both ends here: the server that is asked, and the client that asks.
 */

typedef struct {
    proven_loop_t *loop;
    int opened, answers, closed, refused;
    proven_u16 code;
    char last[64];
} ex_app_t;

// ---- the server: it says what it was sent ----

static void served(void *ctx, proven_ws_stream_t *ws, bool text, proven_mem_view_t piece, bool first, bool
last) {
    (void)ctx; (void)first;
    if (!last) return;                      /* this example's messages are small: one piece each */
    char reply[48];
    int n = snprintf(reply, sizeof reply, "%s, %u bytes", text ? "text" : "binary", (unsigned)piece.size);
    (void)proven_ws_stream_send(ws, true, (proven_mem_view_t){ (const proven_byte_t *)reply,
(proven_size_t)n });
}

static void on_request(void *ctx, proven_http_stream_t *stream, const proven_http_request_t *head) {
    (void)ctx;
    proven_ws_event_config_t config = { .on = { .on_message = served } };
    proven_ws_stream_t *ws = NULL;
    if (proven_ws_event_accept(stream, head, &config, &ws) != PROVEN_OK)
(void)proven_http_stream_respond(stream, 400, NULL, 0, (proven_mem_view_t){ 0 });
}

// ---- the client ----

/* The server accepted. From here on - and from inside this call - messages may be sent. */
static void on_open(void *ctx, proven_ws_stream_t *ws, proven_u8str_view_t protocol) {
    ex_app_t *app = ctx;
    (void)protocol;
    app->opened++;
    (void)proven_ws_stream_send(ws, true, proven_mem_view_from_u8(PROVEN_LIT("hello")));
}

/* The same callback a server has. This side asks twice and then says goodbye. */
static void on_message(void *ctx, proven_ws_stream_t *ws, bool text, proven_mem_view_t piece, bool first, bool
last) {
    ex_app_t *app = ctx;
    (void)text; (void)first;
    if (!last) return;
    proven_size_t n = piece.size < sizeof app->last - 1 ? piece.size : sizeof app->last - 1;
    memcpy(app->last, piece.ptr, n);
    app->last[n] = '\0';
    printf("the server says: %s\n", app->last);
    if (++app->answers == 1) {
        static const proven_byte_t bytes[300] = { 1, 2, 3 };
        (void)proven_ws_stream_send(ws, false, (proven_mem_view_t){ bytes, sizeof bytes });
```

```
    } else {
        proven_ws_stream_close(ws, 1000, PROVEN_LIT("thank you"));
    }
}

/* Called exactly once for every connection that was begun - also for one that never opened:
 * its code is then 0, and `why` says what went wrong. */
static void on_closed(void *ctx, proven_ws_stream_t *ws, proven_u16 code, proven_err_t why) {
    ex_app_t *app = ctx;
    (void)ws;
    if (code == 0) {
        /* Never a WebSocket: nothing listened, the server answered something else, or time ran out. */
        printf("no connection: error %d\n", (int)why);
        app->refused++;
    } else {
        app->code = code;
        app->closed++;
    }
    /* Both connections of this program have ended: there is nothing left for the loop to do. */
    if (app->closed == 1 && app->refused == 1) proven_loop_stop(app->loop);
}

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    ex_app_t app = { 0 };
    EXAMPLE_REQUIRE(proven_loop_create(heap, &app.loop) == PROVEN_OK, "a loop");
    proven_http_event_server_config_t server_config = { .on = { .on_request = on_request } };
    proven_http_event_server_t *server = NULL;
    proven_net_addr_t at;
    EXAMPLE_REQUIRE(proven_http_event_server_create(app.loop, &server_config, &server) == PROVEN_OK, "a
server");
    proven_err_t err = proven_http_event_server_listen(server,
proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0), &at);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        proven_http_event_server_destroy(server);
        proven_loop_destroy(app.loop);
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK, "it listens");

    /* The host is an address, so none has to be given: names are not resolved on a loop. */
    char url[64];
    int n = snprintf(url, sizeof url, "ws://127.0.0.1:%u/chat", (unsigned)at.port);
    proven_ws_event_connect_options_t options = {
        .url = { (const proven_byte_t *)url, (proven_size_t)n },
        .on_open = on_open,
        .ws = { .on = { .on_message = on_message, .on_closed = on_closed }, .ctx = &app },
    };
    proven_ws_stream_t *ws = NULL;
    EXAMPLE_REQUIRE(proven_ws_event_connect(app.loop, &options, &ws) == PROVEN_OK && ws != NULL && app.opened
== 0, "a connection is begun; nothing has happened yet");
    /* The call has returned and the connection is being made: this is too early to speak. */
    EXAMPLE_REQUIRE(proven_ws_stream_send(ws, true, proven_mem_view_from_u8(PROVEN_LIT("too early"))) ==
PROVEN_ERR_INVALID_STATE, "nothing can be sent before the server has accepted");

    /* And one that will fail, to see how that is told: a port that was just closed. */
    proven_net_listener_t unused;
    proven_net_addr_t nobody;
    EXAMPLE_REQUIRE(proven_net_listen(proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0), 1, &unused,
```

```
&nobody) == PROVEN_OK && proven_net_listener_close(&unused) == PROVEN_OK, "a port nobody listens on");
    n = snprintf(url, sizeof url, "ws://127.0.0.1:%u/chat", (unsigned)nobody.port);
    options.url.size = (proven_size_t)n;
    proven_ws_stream_t *lost = NULL;
    err = proven_ws_event_connect(app.loop, &options, &lost);
    EXAMPLE_REQUIRE(err == PROVEN_OK || err == PROVEN_ERR_REFUSED, "an attempt that cannot succeed: refused
now, or told later");
    if (err != PROVEN_OK) app.refused++;

    /* Everything else happens here. */
    if (!(app.closed == 1 && app.refused == 1)) EXAMPLE_REQUIRE(proven_loop_run(app.loop) == PROVEN_OK, "the
loop runs until both have ended");
    EXAMPLE_REQUIRE(app.opened == 1 && app.answers == 2 && strcmp(app.last, "binary, 300 bytes") == 0, "it
opened once, and two messages were answered");
    EXAMPLE_REQUIRE(app.closed == 1 && app.code == 1000 && app.refused == 1, "it closed once, with this side's
code - and the other attempt was reported once");

    proven_http_event_server_destroy(server);
    proven_loop_destroy(app.loop);
    return EXAMPLE_OK();
}
```

Two threads, one connection. A program in which one thread waits for messages and another decides what to send - a chat client, a feed with a control channel - cannot do that on a blocking connection (Chapter 12 says why). It can on this one: the loop waits for messages without a thread of its own, and the other thread hands it what to send with `proven_loop_post`. The send runs on the loop's thread, the only one that touches the connection; what crosses is a copy of the message, in memory the posted call frees. This program is compiled and run by the test suite:

```
#include <stdatomic.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/*
 * One thread waits for messages while another decides what to send - a chat client's shape.
 *
 * A blocking connection (Chapter 12) cannot do it: it is one thread's, and a lock around
 * proven_ws_conn_receive would hold every send for as long as the receive waits. The loop's
 * connection can. It waits for messages without a thread of its own, and another thread hands
 * it what to send with proven_loop_post: the send itself happens on the loop's thread, the
 * only one that touches the connection.
 *
 * One loop carries both ends here: a server that answers each message, and the client.
 */

#define LINES 3

typedef struct {
    proven_loop_t *loop;
    proven_ws_stream_t *ws;          /* the client's connection: the loop's thread only */
    atomic_bool open;                /* what the typing thread may read */
    int answers, closed;
} ex_app_t;

// ---- the server: it answers each message ------------------------------------

static void served(void *ctx, proven_ws_stream_t *ws, bool text, proven_mem_view_t piece, bool first, bool
last) {
    (void)ctx; (void)text; (void)first;
```

```
    if (!last) return;                      /* this example's messages are small: one piece each */
    char reply[80];
    int n = snprintf(reply, sizeof reply, "got: %.*s", (int)piece.size, (const char *)piece.ptr);
    (void)proven_ws_stream_send(ws, true, (proven_mem_view_t){ (const proven_byte_t *)reply,
(proven_size_t)n });
}

static void on_request(void *ctx, proven_http_stream_t *stream, const proven_http_request_t *head) {
    (void)ctx;
    proven_ws_event_config_t config = { .on = { .on_message = served } };
    proven_ws_stream_t *ws = NULL;
    if (proven_ws_event_accept(stream, head, &config, &ws) != PROVEN_OK)
(void)proven_http_stream_respond(stream, 400, NULL, 0, (proven_mem_view_t){ 0 });
}

// ---- the client, on the loop's thread ----------------------------------------

static void on_open(void *ctx, proven_ws_stream_t *ws, proven_u8str_view_t protocol) {
    ex_app_t *app = ctx;
    (void)protocol;
    app->ws = ws;
    atomic_store(&app->open, true);
}

static void on_message(void *ctx, proven_ws_stream_t *ws, bool text, proven_mem_view_t piece, bool first, bool
last) {
    ex_app_t *app = ctx;
    (void)text; (void)first;
    if (!last) return;
    printf("the server says: %.*s\n", (int)piece.size, (const char *)piece.ptr);
    if (++app->answers == LINES) proven_ws_stream_close(ws, 1000, PROVEN_LIT("bye"));
}

static void on_closed(void *ctx, proven_ws_stream_t *ws, proven_u16 code, proven_err_t why) {
    ex_app_t *app = ctx;
    (void)ws; (void)code; (void)why;
    app->ws = NULL;
    app->closed++;
    proven_loop_stop(app->loop);
}

// ---- what crosses from the typing thread -------------------------------------

/* A line to send, in memory of its own: the typing thread has moved on by the time it is sent. */
typedef struct { ex_app_t *app; proven_size_t len; char text[64]; } ex_line_t;

/* On the loop's thread. */
static void send_here(void *ctx) {
    ex_line_t *line = ctx;
    if (line->app->ws) (void)proven_ws_stream_send(line->app->ws, true, (proven_mem_view_t){ (const
proven_byte_t *)line->text, line->len });
    free(line);
}

/* The typing thread. It never touches the connection: it posts. */
static void typist(void *arg) {
    ex_app_t *app = arg;
    static const char *const lines[LINES] = { "hello", "how are you", "goodbye" };
    while (!atomic_load(&app->open)) proven_time_sleep(1);
    for (int i = 0; i < LINES; ++i) {
```

```
        ex_line_t *line = malloc(sizeof *line);
        if (!line) return;
        line->app = app;
        line->len = strlen(lines[i]);
        memcpy(line->text, lines[i], line->len);
        if (proven_loop_post(app->loop, send_here, line) != PROVEN_OK) free(line);
        proven_time_sleep(5);               /* someone typing */
    }
}

static void run(void *arg) { (void)proven_loop_run(((ex_app_t *)arg)->loop); }

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    static ex_app_t app;
    atomic_init(&app.open, false);
    EXAMPLE_REQUIRE(proven_loop_create(heap, &app.loop) == PROVEN_OK, "a loop");
    proven_http_event_server_config_t server_config = { .on = { .on_request = on_request } };
    proven_http_event_server_t *server = NULL;
    proven_net_addr_t at;
    EXAMPLE_REQUIRE(proven_http_event_server_create(app.loop, &server_config, &server) == PROVEN_OK, "a
server");
    proven_err_t err = proven_http_event_server_listen(server,
proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0), &at);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        proven_http_event_server_destroy(server);
        proven_loop_destroy(app.loop);
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK, "it listens");

    char url[64];
    int n = snprintf(url, sizeof url, "ws://127.0.0.1:%u/chat", (unsigned)at.port);
    proven_ws_event_connect_options_t options = {
        .url = { (const proven_byte_t *)url, (proven_size_t)n },
        .on_open = on_open,
        .ws = { .on = { .on_message = on_message, .on_closed = on_closed }, .ctx = &app },
    };
    proven_ws_stream_t *ws = NULL;
    EXAMPLE_REQUIRE(proven_ws_event_connect(app.loop, &options, &ws) == PROVEN_OK, "a connection is begun");

    // ---- two threads: the loop, and someone typing -------------------------------
    proven_job_sys_t *threads = NULL;
    proven_job_group_t looping, typing;
    proven_job_group_init(&looping);
    proven_job_group_init(&typing);
    EXAMPLE_REQUIRE(proven_job_system_init(heap, 2, 4, &threads) == PROVEN_OK &&
                    proven_job_group_submit(threads, &looping, run, &app) == PROVEN_OK &&
                    proven_job_group_submit(threads, &typing, typist, &app) == PROVEN_OK, "the loop on one
thread, the typist on another");

    /* The loop stops when the connection has closed, after the third answer. */
    proven_job_group_wait(threads, &typing);
    proven_job_group_wait(threads, &looping);
    EXAMPLE_REQUIRE(app.answers == LINES && app.closed == 1, "three lines sent from the typing thread, three
answers received on the loop's");

    /* Anything posted and not yet run is run here, before the loop goes. */
    EXAMPLE_REQUIRE(proven_loop_poll(app.loop, PROVEN_NET_DONT_WAIT) == PROVEN_OK, "a last round");
```

```
    proven_job_system_close(threads);
    proven_job_system_destroy(threads);
    proven_http_event_server_destroy(server);
    proven_loop_destroy(app.loop);
    return EXAMPLE_OK();
}
```

12. The event-driven client

```
proven_err_t proven_http_event_client_create(proven_loop_t *loop, const proven_http_event_client_config_t
*config,
                                             proven_http_event_client_t **out);
void proven_http_event_client_destroy(proven_http_event_client_t *client);
proven_size_t proven_http_event_client_requests(const proven_http_event_client_t *client);
proven_err_t proven_http_event_client_start(proven_http_event_client_t *client, const
proven_http_event_request_options_t *options,
                                            proven_http_event_request_t **out);
proven_result_size_t proven_http_event_request_write(proven_http_event_request_t *request, proven_mem_view_t
data);
proven_err_t proven_http_event_request_end(proven_http_event_request_t *request);
void proven_http_event_request_abort(proven_http_event_request_t *request);
void proven_http_event_request_pause(proven_http_event_request_t *request);
void proven_http_event_request_resume(proven_http_event_request_t *request);
void proven_http_event_request_set_user(proven_http_event_request_t *request, void *user);
void *proven_http_event_request_user(const proven_http_event_request_t *request);
```

Chapter 11's client sends a request and waits for the answer. This one starts a request and returns; the response head, the pieces of its body and the end arrive through callbacks on the loop's thread, and any number of requests are in flight at once.

It is deliberately narrow, and the two limits are worth knowing before anything else.

- **A request is one exchange and nothing around it.** There are no redirects, answers to challenges, cookies or proxies. Chapter 11's client has all of them; use it wherever a thread per request in flight is affordable.
- **Names are not resolved.** Looking a name up can take seconds, and nothing on a loop may wait. A request says where to connect with `address`; only a URL whose host is an IP address needs none. For a name: call `proven_net_resolve` on a job, post the result to the loop, and start the request there, as the program at the end of this section does. The URL keeps the name - it is what goes into `Host`, and over TLS it is the name the server's certificate must be for.

A request is described by a `proven_http_event_request_options_t`. Everything it points to is copied or used before `proven_http_event_client_start` returns:

Field	Meaning
`method`	Empty means `GET`. `CONNECT` is refused.
`url`	Absolute, `http` or `https`, with no credentials in it.
`address`	Where to connect; its port is used as given. Null only when the URL's host is an IP address.
`headers`, `header_count`	Yours. `Host`, `Content-Length`, `Transfer-Encoding` and `Connection` are written for you and may not be among them.
`body`	A body held in memory, sent whole.
`body_length`	With no `body`: 0 for none; a length for a body written afterwards; `PROVEN_HTTP_EVENT_LENGTH_UNKNOWN` to write it chunked.
`on.on_response(ctx, request, head)`	The response head. Optional. Interim responses (100, 103) are passed over and not reported.

`on.on_body(ctx, request, piece, last)`	A piece of the body; `last` with the final one, which may be empty. A response with no body gets no call. Optional.
`on.on_writable(ctx, request)`	Request-body bytes that were held back have gone. Optional.
`on.on_done(ctx, request, why)`	The end. **Required**, and called exactly once for every request that was started.
`ctx`	Passed to every callback.

Connections are kept, and counted. Since v0.36.0 an HTTP/1.1 request does not make a connection of its own and close it. The client keeps the connections of each **origin** - the URL's scheme, host and port, and the address connected to - and a request takes one of them:

- **one that is idle**, if its origin has one. A connection is kept when an exchange on it ended cleanly: the response was HTTP/1.1 without `Connection: close`, its body had an end of its own (a length, or chunks) and that end was reached, the request had been sent whole, and nothing came after the response. At most `max_kept_connections` (8) are kept in the whole client, the one idle longest making room for another, and none for longer than `idle_timeout_ms` (60 s);
- **or a new one**, while the origin has fewer than `max_connections_per_origin` (6), busy and idle together;
- **or it waits**, in the order of starting, for the next connection of its origin that comes free. **This is the part to know: the seventh request in flight to one origin is not sent until one of the six has ended.** A program that keeps more than six requests open to one server on purpose - long polls, streams that do not end - must raise `max_connections_per_origin`, or its seventh request waits and, after `connect_timeout_ms`, ends with `PROVEN_ERR_TIMEOUT`: waiting is on the connecting clock, which starts with the request. At most `max_waiting_requests` (1,000) wait in a client; the next `start` is `PROVEN_ERR_BUSY`. A waiting request holds its head and its body, can be written to (up to `max_buffered_output`), and can be aborted.

A kept connection may be dead by the time it is used: servers close idle connections without asking, and the close can cross the next request on the wire. So when a request is lost on a connection that had carried one before, and nothing at all had come back on it, the client takes it that the server never saw the request, and **sends it once more, on a new connection, without telling you** - one `on_done`, for whichever sending ends it. A request whose body is written in pieces cannot be sent again, since the client does not have the pieces; it ends with `PROVEN_ERR_AGAIN`, which means what it means for HTTP/2: not acted on, and yours to start again. Once any byte of an answer has arrived, or on a connection made for the request, a loss is `PROVEN_ERR_RESET` as it always was. (The reading "never saw it" can be wrong - a server may take a request and die before it answers. It is the reading every HTTP client that reuses connections makes, the blocking client of Chapter 11 among them; `no_connection_reuse` is for a program that will not have it.)

`retry_unsent` - a number, zero by default - has the client begin a request again by itself when the server cannot have acted on it. Over HTTP/1.1 that is one case, and a narrow one: a connection that was made and then lost before any of the request had been given to it, which in practice is a server that goes away while TLS is being agreed. Such a request ends with `PROVEN_ERR_RESET` without the setting, and with it is begun on a new connection, so many times, each with its own `connect_timeout_ms`. A request of which anything was sent is not begun again, and neither is one whose body is written in pieces. (The sending-again on a kept connection, above, needs no setting and is not counted. Over HTTP/2 the setting covers more: Chapter 17.)

`retry_delay_ms` is how long the client waits before each such try, in either protocol: zero by default, and the same for every try unless `retry_delay_max_ms` is set - then the wait doubles with each try of a request, from `retry_delay_ms` up to that bound. Nothing is added at random: requests that were refused together are tried again together. While it waits the request has no connection - the one it

lost is closed, and its place may go to a request that was waiting - and `connect_timeout_ms` of the try begins when the wait ends. Aborting the request during the wait ends it as any abort does. The sending-again on a kept connection is not delayed. (With `max_connections`, below: when the wait is over the request takes a place as a new one would, waiting for one if there is none; if as many wait already as may, it ends with `PROVEN_ERR_BUSY` - the one case in which `on_done` says so, and not `proven_http_event_client_start`.)

`max_connections` bounds the connections of the whole client, of both protocols, busy and idle; zero, the default, is no bound. A request that would make one more first has an idle connection closed for it - an HTTP/1.1 one, the one idle longest, before an HTTP/2 one with nothing on it. With none idle the request waits, whichever its protocol. An HTTP/1.1 request waits in the queue described above, and has the next place that is freed (a connection that closes serves a request to its own origin first, then the one that has waited longest and whose origin has room). An HTTP/2 request whose origin has no connection waits on no connection at all, in the order of starting, and is put on one when a place can be had - or on its origin's, if another request has made one meanwhile. Either wait is the wait for a connection: `connect_timeout_ms` ends it with `PROVEN_ERR_TIMEOUT`, and no more requests wait than `max_waiting_requests` (the next start is `PROVEN_ERR_BUSY`). When both protocols wait, a place goes to an HTTP/1.1 request first. Setting `max_kept_connections` above `max_connections` is `PROVEN_ERR_INVALID_ARG`.

`no_connection_reuse` gives back the old behaviour for connections - `Connection: close` in every request, every connection closed when its response has ended - but not the old absence of a limit: six at a time to an origin still.

Starting. When `proven_http_event_client_start` returns an error, the request was not started and no callback is or will be made. When it returns `PROVEN_OK`, `on_done` will be called exactly once - **never inside that call**, so code that starts a request can finish what it was doing before anything happens to it. A request is valid from the start until `on_done` returns.

`start` **returns**	**For**
`PROVEN_ERR_INVALID_ARG`	No `on_done`; a URL that is not absolute `http` or `https`, or that carries credentials; `https` with no TLS configuration - refused, never sent in the clear; a name with no `address`; a header the client writes itself; a method that is not a token, or `CONNECT`.
`PROVEN_ERR_OUT_OF_BOUNDS`	A request head that does not fit `max_head_bytes`.
`PROVEN_ERR_NOMEM`	
`PROVEN_ERR_REFUSED`, `PROVEN_ERR_UNREACHABLE`	The connect failed at once. It may also fail later, and is then reported by `on_done`.

How it ends. `on_done` says:

`why`	**What happened**
`PROVEN_OK`	The whole response arrived - whatever its status: a 404 is a response.
`PROVEN_ERR_REFUSED`, `PROVEN_ERR_UNREACHABLE`	Nothing was listening, or there was no route.
`PROVEN_ERR_TIMEOUT`	One of the four time limits passed.
`PROVEN_ERR_RESET`	The server closed or was lost before the response was whole - a body cut short of its length is never reported as complete; or

	`proven_http_event_request_abort`; or the client was destroyed.
`PROVEN_ERR_INVALID_FORMAT`	The answer was not HTTP, its framing contradicted itself, or more than eight interim responses came in a row.
`PROVEN_ERR_OUT_OF_BOUNDS`	The head was larger than `max_head_bytes`, or the body larger than `max_body_bytes`.
`PROVEN_ERR_UNSUPPORTED`	The server answered 101: this client asks for no change of protocol.
`PROVEN_ERR_UNTRUSTED`, `PROVEN_ERR_NAME_MISMATCH`, `PROVEN_ERR_EXPIRED`, ...	TLS refused the server (Chapter 14, section 8). The request was never sent.

A request body written as it goes follows the rule of section 7. `proven_http_event_request_write` takes as many bytes as fit under `max_buffered_output`, possibly none, and says how many; the rest is offered again when `on_writable` is called. It may be called at once after `start`, before the connection exists: what is taken waits. `proven_http_event_request_end` finishes the body; ending short of a promised length ends the request with `PROVEN_ERR_INVALID_FORMAT`.

Call	Returns
`proven_http_event_client_create(loop, &config, &client)`	`proven_err_t`: `INVALID_ARG` for no loop; `NOMEM`. `config` may be null.
`proven_http_event_client_requests(client)`	`proven_size_t`: requests started and not ended - those waiting for a connection among them.
`proven_http_event_client_destroy(client)`	nothing. Every request, in flight or waiting, gets `on_done(PROVEN_ERR_RESET)`, and the kept connections are closed. Connections whose sockets the loop may still use keep their memory until the loop's next round or `proven_loop_destroy` (section 5, rule 3). Null is accepted.
`proven_http_event_request_write(request, data)`	`proven_result_size_t`: the bytes taken. `INVALID_STATE` when the request has no body to write or it was ended; `OUT_OF_BOUNDS` for more than the length promised.
`proven_http_event_request_end(request)`	`proven_err_t`: `INVALID_STATE` when there was nothing to end; `INVALID_FORMAT` when fewer bytes were written than promised - `on_done` is then called inside the call.
`proven_http_event_request_abort(request)`	nothing. `on_done(PROVEN_ERR_RESET)` is called inside the call.
`proven_http_event_request_pause`, `proven_http_event_request_resume`	nothing. Stop and restart the delivery of the response body.

The configuration's fields, each with a default when zero:

Field	Meaning	Default
`alloc`	The allocator for requests and buffers.	the loop's

`tls`	A TLS configuration that can verify servers, for `https`. Must outlive the client. A second connection to an origin resumes the TLS session the last one learned (since v0.72.0, RFC-0020): the client keeps the sessions of 64 origins for TCP and 64 for QUIC, the least recently used given up and wiped, and never offers TCP's over QUIC; a server that refuses the ticket is answered by a full handshake. The configuration's `no_resumption` turns it off - for a client that must check the certificate every time, or must not be linked from one connection to the next. A session is 2,048 bytes (since v0.73.0), so the sessions kept take at most 256 KiB. Its `early_data` (since v0.73.0 over HTTP/3, RFC-0021; since v0.84.0 over HTTP/1.1 and HTTP/2, RFC-0027) sends GET, HEAD and OPTIONS requests without a body as early data on a connection that resumes - one round trip sooner - and sends a request refused that way, or answered 425, again once after the handshake: Chapter 14, section 6.	none: `https` is refused
`max_head_bytes`	Largest response head, and request head.	16 KiB
`max_headers`	Most header fields in a response.	64
`max_body_bytes`	Largest response body.	no limit: it is delivered in pieces and never held
`max_buffered_output`	Request-body bytes held before writes are refused.	64 KiB
`connect_timeout_ms`	To be connected, the TLS handshake included - and the wait for a connection, when the origin has as many as it may. It runs from the start of the request; a request that is sent again gets it anew.	10 s
`response_timeout_ms`	From the request being sent to the end of the response head.	30 s

`body_timeout_ms`	Between pieces of the response body.	30 s
`write_timeout_ms`	Output held with none of it accepted by the server.	30 s
`decompress`	Ask for gzip and decode it: `on_body` gets decoded pieces, and `max_body_bytes` counts them. Chapter 16, section 6.	off
`http2`, `http2_cleartext`	Speak HTTP/2 to origins that agree to it (ALPN), or to every `http` origin: one connection kept for each, shared by its requests. Chapter 17, section 7.	off
`idle_timeout_ms`	How long a connection with no request on it is kept, in either protocol.	60 s
`max_waiting_requests`	Requests that may wait: for an HTTP/2 connection, on that connection; for an HTTP/1.1 connection, in the whole client. The next `start` is `PROVEN_ERR_BUSY`.	1,000
`max_connections_per_origin`	HTTP/1.1: connections to one origin at a time, busy and idle together; a request beyond that waits.	6
`max_kept_connections`	HTTP/1.1: idle connections kept in the whole client. Zero is the default, as every zero in this table is; `no_connection_reuse` is what keeps none. (It was `max_idle_connections` in v0.36.0 - the name of a field of chapter 11's client, where 0 keeps none. The two no longer share a name.)	8
`no_connection_reuse`	HTTP/1.1: `Connection: close` in every request, and no connection kept.	off
`retry_unsent`	How many times the client itself puts again a request the server did not act on (HTTP/2: `PROVEN_ERR_AGAIN`; HTTP/1.1: lost before any of it was sent), at once and without telling you. Never a body written in pieces, never after `on_response`.	0: never
`retry_delay_ms`	The wait before each try `retry_unsent` makes, in either	0: none

	protocol; `connect_timeout_ms` of the try begins when it ends.	
`retry_delay_max_ms`	When set, the wait doubles with each try of a request, from `retry_delay_ms` up to this. Below `retry_delay_ms`, or with no `retry_delay_ms`, is `INVALID_ARG`.	0: the same wait every time
`max_connections`	Connections of the whole client, both protocols, busy and idle. At the limit an idle one is closed for a new request; with none idle the request waits, in either protocol, for `connect_timeout_ms` at most.	0: no limit
`http2_window_bytes`	HTTP/2: response-body bytes a server may send on a stream, and on a connection as a whole, before `on_body` has taken them. What a paused request can make a connection hold; a large download waits for room each time the server has sent this much (sixteen responses of 1 MiB at once, measured before v0.75.0: 568 a second at the default, 1,208 with 1 MiB). With 0 the window starts there and grows while bodies are read - doubling while half of it is taken within two round trips, up to 16 MiB a stream and 24 MiB a connection (v0.75.0, as HTTP/3's since v0.71.0); a body not read stops the growth. A value set stays fixed. One download of 64 MiB over a relay with 10 ms each way: 16 Mbit/s with the window fixed at 65,535, 1,310 growing.	65,535, growing
`http3`	A request that asks for it (`http3` in its options, an `https` URL) goes over HTTP/3, on one QUIC connection kept for its origin: Chapter 17, section 10. Never one that does not ask, and never by following `Alt-Svc`. **No fallback** unless `http3_fallback`: no answer over UDP is `PROVEN_ERR_TIMEOUT` after `connect_timeout_ms`. Needs `tls`, offering `h3`.	false

`http3_window_bytes`	HTTP/3: response-body bytes a server may send on a stream, and on a connection as a whole, before `on_body` has taken them (QUIC's flow control).	256 KiB a stream and 1 MiB a connection, growing while bodies are read as fast as they come, up to 16 MiB and 24 MiB; a value set is fixed
`http3_header_table_bytes`	HTTP/3: the QPACK dynamic table a server's encoder may fill (RFC-0032): said in SETTINGS with 16 streams that may wait for its inserts; made when a server first uses it, about twice this then. 32 to 1 MiB. Since v0.96.0 also the most of the server's table the client's encoder fills, referring only to what the server acknowledged.	4,096
`http3_no_header_table`	HTTP/3: no such table; every response head arrives in full.	false
`http3_fallback`	A request asked over HTTP/3 whose QUIC connection does not come up - no handshake within `http3_fallback_ms`, an unreachable address or refused port, `h3` refused - starts again over TCP, on the object the caller holds; the origin goes over TCP from the start for five minutes after. Never for an untrusted certificate: Chapter 17, section 10. Needs `http3`.	false
`http3_fallback_ms`	With `http3_fallback`: how long a QUIC handshake may take before its requests go over TCP. At most `connect_timeout_ms`.	1 s

The test suite compiles and runs this program:

```
#include <stdio.h>
#include <string.h>

/*
 * The event-driven HTTP client: several requests in flight on one thread.
 *
 * A request is started and the call returns at once; what happens to it arrives through
 * callbacks, on the loop's thread. The server here is the event-driven one of this chapter, on
 * the same loop - this whole program is one thread and one worker for the one thing that waits.
 *
 * That one thing is looking a name up. The client does not resolve names: a lookup can take
 * seconds, and nothing on a loop may wait. So the third request shows the pattern - resolve on
 * a worker, post the answer back, start the request.
 */

typedef struct app ex_app_t;

/* What this program keeps for one request. */
```

```
typedef struct {
    ex_app_t *app;
    const char *path;
    int status;
    char body[64];
    proven_size_t len;
    proven_err_t why;
    bool done;
} ex_fetch_t;

struct app {
    proven_loop_t *loop;
    proven_http_event_client_t *client;
    proven_job_sys_t *workers;
    proven_u16 port;
    ex_fetch_t fetch[3];
    int finished;
    /* written by the worker, read on the loop after it has been posted */
    proven_net_addr_t found;
    proven_err_t lookup;
};

// ---- the server the requests go to -----------------------------------------

static void serve(void *ctx, proven_http_stream_t *stream, const proven_http_request_t *head) {
    (void)ctx;
    if (proven_u8str_view_eq(head->target, PROVEN_LIT("/a"))) (void)proven_http_stream_respond(stream, 200,
NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("first")));
    else if (proven_u8str_view_eq(head->target, PROVEN_LIT("/b"))) (void)proven_http_stream_respond(stream,
200, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("second")));
    else (void)proven_http_stream_respond(stream, 404, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("nothing
here")));
}

// ---- the client's callbacks ------------------------------------------------

/* The head: good only until this function returns, so take what is wanted from it. */
static void on_response(void *ctx, proven_http_event_request_t *request, const proven_http_response_t *head) {
    (void)request;
    ((ex_fetch_t *)ctx)->status = head->status;
}

/* The body, a piece at a time. Nothing is collected for you; collect what you want. */
static void on_body(void *ctx, proven_http_event_request_t *request, proven_mem_view_t piece, bool last) {
    (void)request; (void)last;
    ex_fetch_t *f = ctx;
    for (proven_size_t i = 0; i < piece.size && f->len < sizeof f->body - 1; ++i) f->body[f->len++] =
(char)piece.ptr[i];
}

/* Once for every request that was started, whatever happened to it. */
static void on_done(void *ctx, proven_http_event_request_t *request, proven_err_t why) {
    (void)request;
    ex_fetch_t *f = ctx;
    f->why = why;
    f->done = true;
    if (++f->app->finished == 3) proven_loop_stop(f->app->loop);
}

static proven_err_t fetch(ex_app_t *app, ex_fetch_t *f, const char *path, const proven_net_addr_t *address,
```

```
const char *host) {
    char url[96];
    int n = snprintf(url, sizeof url, "http://%s:%u%s", host, (unsigned)app->port, path);
    f->app = app;
    f->path = path;
    proven_http_event_request_options_t options = {
        .url = { (const proven_byte_t *)url, (proven_size_t)n },
        .address = address,                              /* null is allowed only when the URL's host is an IP
address */
        .on = { on_response, on_body, NULL, on_done },
        .ctx = f,
    };
    return proven_http_event_client_start(app->client, &options, NULL);     /* returns at once */
}

// ---- a name, resolved elsewhere ------------------------------------------

/* On the loop's thread again: now there is an address, and the request can start.
 * The URL keeps the name - it is what goes into Host - and the address says where to connect. */
static void resolved(void *ctx) {
    ex_app_t *app = ctx;
    if (app->lookup != PROVEN_OK || fetch(app, &app->fetch[2], "/missing", &app->found, "localhost") !=
PROVEN_OK) {
        app->fetch[2].why = PROVEN_ERR_NOT_FOUND;
        if (++app->finished == 3) proven_loop_stop(app->loop);
    }
}

/* On a worker thread: this call may wait as long as the resolver takes. */
static void resolve(void *ctx) {
    ex_app_t *app = ctx;
    proven_net_addr_t all[8];
    proven_size_t count = 0;
    app->lookup = proven_net_resolve(PROVEN_LIT("localhost"), app->port, all, 8, &count);
    /* The server here listens on IPv4 only; a real program would try each address in turn. */
    bool have = false;
    for (proven_size_t i = 0; i < count && !have; ++i) {
        if (all[i].family == PROVEN_NET_FAMILY_IPV4) { app->found = all[i]; have = true; }
    }
    if (app->lookup == PROVEN_OK && !have) app->lookup = PROVEN_ERR_NOT_FOUND;
    if (proven_loop_post(app->loop, resolved, app) != PROVEN_OK) proven_loop_stop(app->loop);
}

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    ex_app_t app = { 0 };

    // ---- one loop, a server and a client on it -----------------------------------
    /* Nothing runs yet: a loop does nothing until it is run. */
    EXAMPLE_REQUIRE(proven_loop_create(heap, &app.loop) == PROVEN_OK, "a loop");
    proven_http_event_server_config_t server_config = { .on = { .on_request = serve } };
    proven_http_event_server_t *server = NULL;
    proven_net_addr_t at;
    EXAMPLE_REQUIRE(proven_http_event_server_create(app.loop, &server_config, &server) == PROVEN_OK &&
                    proven_http_event_server_listen(server, proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4,
0), &at) == PROVEN_OK, "a server to ask");
    app.port = at.port;
    EXAMPLE_REQUIRE(proven_http_event_client_create(app.loop, NULL, &app.client) == PROVEN_OK, "an event-
driven client");
    EXAMPLE_REQUIRE(proven_job_system_init(heap, 1, 4, &app.workers) == PROVEN_OK, "one worker, for the
```

```
lookup");

    // ---- three requests ------------------------------------------------------
    /* Two by address. Both are started before either has been sent. */
    EXAMPLE_REQUIRE(fetch(&app, &app.fetch[0], "/a", NULL, "127.0.0.1") == PROVEN_OK &&
                    fetch(&app, &app.fetch[1], "/b", NULL, "127.0.0.1") == PROVEN_OK, "two requests started");
    EXAMPLE_REQUIRE(proven_http_event_client_requests(app.client) == 2 && !app.fetch[0].done && !
app.fetch[1].done, "both in flight, neither answered: nothing has run yet");

    /* One by name. Without an address it is refused - so resolve first, on the worker. */
    ex_fetch_t unresolved = { 0 };
    EXAMPLE_REQUIRE(fetch(&app, &unresolved, "/missing", NULL, "localhost") == PROVEN_ERR_INVALID_ARG, "a name
with no address is refused");
    EXAMPLE_REQUIRE(proven_job_submit(app.workers, resolve, &app), "the lookup is handed to a worker");

    // ---- run -----------------------------------------------------------------
    /* on_done stops the loop when the third request has ended. */
    EXAMPLE_REQUIRE(proven_loop_run(app.loop) == PROVEN_OK, "the loop ran until all three were done");

    EXAMPLE_REQUIRE(app.fetch[0].why == PROVEN_OK && app.fetch[0].status == 200 && strcmp(app.fetch[0].body,
"first") == 0, "the first answer");
    EXAMPLE_REQUIRE(app.fetch[1].why == PROVEN_OK && app.fetch[1].status == 200 && strcmp(app.fetch[1].body,
"second") == 0, "the second answer");
    EXAMPLE_REQUIRE(app.fetch[2].why == PROVEN_OK && app.fetch[2].status == 404 && strcmp(app.fetch[2].body,
"nothing here") == 0, "the third, by name: a 404 is a response, not an error");

    proven_job_system_close(app.workers);
    proven_job_system_destroy(app.workers);
    proven_http_event_client_destroy(app.client);
    proven_http_event_server_destroy(server);
    proven_loop_destroy(app.loop);
    return EXAMPLE_OK();
}
```

13. Several loops

One loop is one thread, and a connection belongs to one loop for its whole life: nothing about it is ever touched by another thread, which is why nothing in this chapter takes a lock. A server that needs more than one processor core is therefore **several loops, each with a server of its own** - and something to share the incoming connections among them.

```
proven_err_t proven_http_event_server_adopt(proven_http_event_server_t *server, proven_net_conn_t *conn);
```

`proven_http_event_server_adopt` gives a server a connection that was accepted somewhere else. It becomes one of the server's exactly as if the server had accepted it; with TLS, the handshake begins. So one thread listens and accepts, and deals each connection to the servers in turn. Three rules make that correct:

- `adopt` **is called on the server's loop thread**, like everything else. The accepting thread does not call it: it posts a function to that loop (`proven_loop_post`), and the function calls it.
- **The connection travels in memory of its own.** A `proven_net_conn_t` is not to be copied while it is open, so what is posted is a small block holding it, freed by the function that adopts.
- **Taken apart in order:** the accepting thread first, so that nothing more is posted; then the loops; then - after one last round of each loop, for anything posted and not yet run - the servers.

`adopt` **returns**	**Meaning**
`PROVEN_OK`	The server owns the connection; the caller's value is no longer open.

`PROVEN_ERR_BUSY`	The server is at `max_connections`. The connection is still the caller's, to close.
`PROVEN_ERR_INVALID_STATE`	The connection is not open, or the server was told `proven_http_event_server_stop_listening`. Still the caller's.
`PROVEN_ERR_INVALID_ARG`, `PROVEN_ERR_NOMEM`	Still the caller's.

The other way to the same end needs no call at all: give each loop's server a listener of its own on a different address or port, and let whatever is in front - a load balancer, or DNS - spread the clients. The load measurements of the next section were made that way.

Whether more loops help depends on what the time goes on. They multiply what handlers can compute; they do nothing for a handler that waits, which should not be on a loop at all (section 10).

The test suite compiles and runs this program:

```
#include <stdatomic.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/*
 * Several loops behind one port: one thread accepts, and deals connections to servers on
 * loops of their own.
 *
 * A connection belongs to one loop, and nothing about it is ever touched by another thread.
 * So scaling across processors is several loops, each with its own server - and something has
 * to share the incoming connections among them. Here that is a thread that accepts and deals
 * them out in turn, which works wherever the library does.
 */

enum { LOOPS = 2 };

/* One loop, its server, and what its handler counts. Only that loop's thread touches it. */
typedef struct {
    int index;
    proven_loop_t *loop;
    proven_http_event_server_t *server;
    int served;                       /* read by the main thread only after the loop has stopped */
} ex_worker_t;

static ex_worker_t g_workers[LOOPS];
static proven_net_listener_t g_listener;
static atomic_bool g_stop;

static void on_request(void *ctx, proven_http_stream_t *stream, const proven_http_request_t *head) {
    ex_worker_t *me = ctx;
    (void)head;
    char text[16];
    int n = snprintf(text, sizeof text, "loop %d", me->index);
    me->served++;
    (void)proven_http_stream_respond(stream, 200, NULL, 0, (proven_mem_view_t){ (const proven_byte_t *)text,
(proven_size_t)n });
}

/* What crosses from the accepting thread to a loop. An open connection is not to be copied,
 * so it travels in memory of its own. */
typedef struct { ex_worker_t *to; proven_net_conn_t conn; } ex_handoff_t;
```

```
/* On the loop's thread - the only place a server may be given a connection. */
static void adopt_here(void *ctx) {
    ex_handoff_t *h = ctx;
    if (proven_http_event_server_adopt(h->to->server, &h->conn) != PROVEN_OK) (void)proven_net_close(&h-
>conn);   /* refused: still ours to close */
    free(h);
}

/* The accepting thread. It touches no server: it accepts, chooses, and posts. */
static void acceptor(void *arg) {
    (void)arg;
    int next = 0;
    while (!atomic_load(&g_stop)) {
        ex_handoff_t *h = malloc(sizeof *h);
        if (!h) break;
        /* A short wait at a time, so that the thread notices when it is told to stop. */
        if (proven_net_accept(&g_listener, proven_net_deadline_in(20), &h->conn, NULL) != PROVEN_OK)
{ free(h); continue; }
        h->to = &g_workers[next];
        next = (next + 1) % LOOPS;
        if (proven_loop_post(h->to->loop, adopt_here, h) != PROVEN_OK) { (void)proven_net_close(&h->conn);
free(h); }
    }
}

static void run(void *arg) { (void)proven_loop_run(((ex_worker_t *)arg)->loop); }

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();

    // ---- two loops, each with a server that has no listener -----------------------
    /* A server with no listener serves only what it is given. */
    proven_job_sys_t *threads = NULL;
    proven_job_group_t loops, accepting;
    proven_job_group_init(&loops);
    proven_job_group_init(&accepting);
    EXAMPLE_REQUIRE(proven_job_system_init(heap, LOOPS + 1, 8, &threads) == PROVEN_OK, "three threads");
    for (int i = 0; i < LOOPS; ++i) {
        ex_worker_t *w = &g_workers[i];
        w->index = i;
        proven_http_event_server_config_t config = { .on = { .on_request = on_request }, .ctx = w };
        EXAMPLE_REQUIRE(proven_loop_create(heap, &w->loop) == PROVEN_OK &&
                        proven_http_event_server_create(w->loop, &config, &w->server) == PROVEN_OK &&
                        proven_job_group_submit(threads, &loops, run, w) == PROVEN_OK, "a loop on its own
thread, with a server");
    }

    // ---- one listener, and the thread that deals ----------------------------------
    proven_net_addr_t at;
    EXAMPLE_REQUIRE(proven_net_listen(proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0), 64, &g_listener,
&at) == PROVEN_OK &&
                    proven_job_group_submit(threads, &accepting, acceptor, NULL) == PROVEN_OK, "a listener and
its accepting thread");

    // ---- eight connections --------------------------------------------------------
    /* A client that keeps no connection open: eight requests are eight connections. */
    proven_http_client_config_t client_config = { .alloc = heap, .max_idle_connections = 0 };
    proven_http_client_t *client = NULL;
    EXAMPLE_REQUIRE(proven_http_client_create(&client_config, &client) == PROVEN_OK, "a client");
```

```
    char url[64];
    int n = snprintf(url, sizeof url, "http://127.0.0.1:%u/", (unsigned)at.port);
    int answers[LOOPS] = { 0 };
    for (int i = 0; i < 8; ++i) {
        proven_http_client_response_t response;
        proven_u8str_t body = { 0 };
        EXAMPLE_REQUIRE(proven_http_client_get(client, (proven_u8str_view_t){ (const proven_byte_t *)url,
(proven_size_t)n }, &response) == PROVEN_OK &&
                        proven_http_client_read_all(&response, heap, &body, 64) == PROVEN_OK, "a request is
answered");
        proven_u8str_view_t text = proven_u8str_as_view(&body);
        if (text.size == 6 && memcmp(text.ptr, "loop ", 5) == 0 && text.ptr[5] - '0' < LOOPS)
answers[text.ptr[5] - '0']++;
        proven_u8str_destroy(heap, &body);
        proven_http_client_finish(&response);
    }
    proven_http_client_destroy(client);
    EXAMPLE_REQUIRE(answers[0] == 4 && answers[1] == 4, "dealt in turn: four connections were served by each
loop");

    // ---- taking it apart, in this order ------------------------------------------
    /* The acceptor first, so that nothing more is posted. Then the loops. Then the servers. */
    atomic_store(&g_stop, true);
    proven_job_group_wait(threads, &accepting);
    (void)proven_net_listener_close(&g_listener);
    for (int i = 0; i < LOOPS; ++i) proven_loop_stop(g_workers[i].loop);
    proven_job_group_wait(threads, &loops);
    int served = 0;
    for (int i = 0; i < LOOPS; ++i) {
        /* Anything posted and not yet run is run here, before its server goes. */
        EXAMPLE_REQUIRE(proven_loop_poll(g_workers[i].loop, PROVEN_NET_DONT_WAIT) == PROVEN_OK, "a last
round");
        served += g_workers[i].served;
        proven_http_event_server_destroy(g_workers[i].server);
        proven_loop_destroy(g_workers[i].loop);
    }
    EXAMPLE_REQUIRE(served == 8, "eight requests were served in all");
    proven_job_system_close(threads);
    proven_job_system_destroy(threads);
    return EXAMPLE_OK();
}
```

14. What a connection holds, what was measured, and what is not here

What a connection holds. A connection with no request on it holds its struct and no buffer: requests are read into the loop's scratch buffer, and bytes are copied aside only when a request is incomplete. (A compressor exists only while a compressed response is being written.) On x86-64 Linux the registered tests measure 480 bytes of heap for an idle plain HTTP connection, failing above 512, and 976 bytes for an idle plain WebSocket connection, failing above 1 KiB. A TLS connection adds the engine's state, about 1.25 KiB while idle (Chapter 14, section 9). After a quiet second between requests the server gives back the expanded form of its record keys (wiped) and keeps the keys they were made from; the next request remakes them, which takes about a tenth of a microsecond per direction on a processor with AES instructions. An idle TLS connection quiet for a second measures 1,088 bytes in all, and the test fails at 2 KiB - the target for TLS, where a plain connection's is 1 KiB. **An HTTP/2 connection is another size**: 2,360 bytes idle after one small request, failing above 3 KiB, and growing with the header fields it is made to remember to about 12.8 KiB when both of its compression tables are full (Chapter 17, section 6). The numbers below are HTTP/1.1's; HTTP/2's, under the same load, are in Chapter 17, section 8.

What was measured. One run of a load program that is not part of the test suite, on one machine: a 16-thread x86-64 desktop processor at 3.5 GHz with 62 GB, Linux 6.12, GCC at `-O2`. The server and the client are separate processes on that machine, talking over the loopback interface, plain HTTP. Every connection makes one request and is then left open; after that a stated share of them make requests back to back, each waiting for its answer, for ten seconds. The handler answers two bytes. **Loopback is not a network** - there is no loss and no slow peer here - and a handler that does real work will be slower than one that answers "ok".

One loop:

Connections held	Making requests	Requests per second	Round trip: median, 99th percentile	Server's resident memory per connection
1,000	10	169,000	0.05 ms, 0.14 ms	479 B
50,000	10	175,000	0.04 ms, 0.11 ms	481 B
10,000	100	184,000	0.5 ms, 0.9 ms	486 B
50,000	500	171,000	2.8 ms, 5.2 ms	481 B
100,000	1,000	155,000	6.4 ms, 9.7 ms	480 B

What the table says: **connections that are doing nothing cost a busy one nothing** - ten connections are served as fast among fifty thousand as among a thousand - and they cost memory in proportion, about 480 bytes each in the server. One loop does a fixed amount of work per second, so when more connections ask at once each waits longer: the round trip grows with the number asking, not with the number held. No connection was lost in any row. Opening 100,000 took 96 seconds, against 1.7 for 50,000. That appears to be the client's system searching for a free port as its range runs out, not the server: the blocking server took as long or longer for the same step.

The system's own cost for a socket is separate and larger: over the same runs the kernel's slab memory grew by about 7.5 KiB per connection, for both of its ends together.

Several loops, with four client processes holding 12,500 connections each and 1% of them making requests:

Loops	Requests per second, all clients together	Round trip: median
1	167,000	2.9 ms
2	337,000	1.4 ms
4	514,000	1.0 ms

The blocking server of Chapter 11, for comparison, in the same program with its handler on its loop's thread: it held the same 100,000 connections and answered slightly faster - 186,000 requests per second at 50,000 held, against 171,000. What it costs is memory: 39 KiB allocated per connection, 10.9 KiB of it resident, against 480 bytes. And it is one thread whose handler waits; the reasons to choose this chapter's server are the ones of section 1, not raw speed for a trivial handler.

On Windows, until v0.80.0, the ceiling was low. There the loop waited with `WSAPoll`, which looks at every socket on every call. Both ends of the same program on one Windows 11 virtual machine, ten connections making requests:

Connections held	Requests per second	Round trip: median

100	40,000	0.2 ms
1,000	8,100	1.0 ms
2,000	2,700	3.2 ms
5,000	480	18 ms
10,000	150	66 ms

The client was on the same loop implementation, so each figure carries the cost twice. Asked for 20,000 connections, that machine opened 16,369 before its ports ran out. Since v0.81.0 the loop is built on an I/O completion port there (section 5), which is told only of the operations that finish and does not look at idle sockets. The same program on the same machine, two runs in a row (the two figures per cell; the client on the completion port as well):

Connections held	**Making requests**	**Requests per second**	**Round trip: median, 99th percentile**	**Server's private memory per connection**
100	10	42,000 / 46,000	0.18 / 0.16 ms, 0.8 / 0.4 ms	-
1,000	10	50,000 / 50,000	0.15 ms, 0.4 ms	1.2 KB
2,000	10	55,000 / 51,000	0.15 ms, 0.3 / 0.4 ms	1,065 B
5,000	10	48,000 / 36,000	0.16 / 0.26 ms, 0.4 ms	1,049 B
10,000	10	53,000 / 51,000	0.15 ms, 0.4 ms	1,040 B
10,000	100	47,000 / 48,000	2.0 / 1.9 ms, 3.9 / 4.2 ms	1,045 B
16,372 (20,000 asked)	9	53,000 / 54,000	0.13 ms, 0.3 ms	1,047 B

Ten connections are served as fast among sixteen thousand as among a thousand: at 10,000 held, about 350 times the requests of `WSAPoll` and a round trip four hundred times shorter. What the server's allocator holds is 536 bytes per idle connection in every row (the registered test's figure); the process's private memory, with the heap's own bookkeeping, is about 1 KB per connection. The machine is a virtual one on a shared host, so one row in a run can be a quarter slower than the other run (5,000 held); the ports still ran out at about 16,370. The blocking server of Chapter 11, whose selector is on the completion port too, measured 50,000 / 54,000 requests per second at 1,000 held and 53,000 / 52,000 at 10,000, a median of 0.15 ms, with 40 KB of private memory per connection.

What is not here.

- **A load test worth the name.** The figures above are one machine talking to itself: no network, no slow or hostile clients, no TLS under load, one handler that does nothing.
- Upgrades other than WebSocket on this server; a WebSocket client through a proxy or over HTTP/2.
- **In the event-driven client:** redirects, authentication, cookies, proxies, and name resolution (section 12 says what to do instead); pipelining, and any order among waiting requests but the order of starting.
- **TLS handshakes off the loop's thread.** Each costs the loop about 2 ms during which it serves nothing else (Chapter 14, section 9).

- **Sending a file without copying it**, HTTP/2. (Compressed responses are in Chapter 16, section 6: `proven_http_stream_compress` in the server - with `compress_zstd` in its configuration for Zstandard - and `decompress` - and `decompress_zstd` for Zstandard, `decompress_brotli` for Brotli - in the client's.)
- **Watching files, signals or child processes** with the loop. It watches sockets.

How this was tested. The registered tests drive the loop's timers and posts from one thread and from several, including cancellation from inside the callbacks they concern; drive every operation of section 5 on the system's selector and again on the poll kind - each answered once, never inside the call that started it, through cancellation, close with everything in flight, a send that fails, and three MiB to a slow reader; and run the server over loopback, plain and over TLS, through every row of the tables in sections 7 and 9: answers at once, from a timer and from another thread, uploads with pause and resume, a 16 MB download to a client that does not read until a write has been refused, pipelining, each refusal, each timeout, clients that disappear, and a server destroyed under an unanswered request - checking that `on_done` was called once for every `on_request`. The WebSocket is run against the blocking client of Chapter 12 and against raw sockets, through the rows of the tables of section 11 - all but `PROVEN_ERR_NOMEM`, and `PROVEN_ERR_RESET` from `proven_ws_event_accept` and from the send calls, which are not provoked. The client is run against this chapter's server on the same loop and against answers the test scripts itself, through the rows of the tables of section 12 - all but `PROVEN_ERR_UNREACHABLE`, `PROVEN_ERR_NOMEM`, an expired certificate, and three of the four time limits: only the response timeout is made to pass. All of them were run several hundred times under address and undefined-behaviour checking with the machine loaded, and under thread checking. Several loops are run as three servers on three threads behind one accepting thread, plain and over TLS, under thread checking. Timers further away than one turn of the loop's wheel (16.4 seconds) are tested with a clock the test moves. **Not done:** no run against a hostile peer beyond the cases listed, no run of a published WebSocket conformance suite against this server, and no load beyond the single run described above.

Chapter 16: Compression

Part III - Data structures. Prerequisites: Chapter 2 for the allocator a compressor is made with; Chapter 4 for the checksums. After this chapter you can compress and decompress a buffer or a stream of any length, read and write the `gzip` and `zlib` wrappers, read and write Zstandard, read Brotli, decide how much a stranger's data may expand to - and how much memory it may make you spend - before you refuse it, and say what was tested and how fast it is.

This chapter covers `deflate.h`, in section 7 `zstd.h`, and in section 8 `brotli.h`. All are pure computation - allocation through the allocator you give, nothing from the operating system - and are available in a freestanding build. Section 6 is the exception: it is about compressed HTTP responses, in the servers and clients of Chapters 11 and 15.

Table of contents

1. What DEFLATE does, in one page
2. A whole buffer
3. A stream
4. Levels, windows and what they cost
5. Data from somebody else
6. Over HTTP
7. Zstandard
8. Brotli
9. What is accepted, what is not here, and how this was tested

1. What DEFLATE does, in one page

DEFLATE (RFC 1951) is the compression inside `.gz` files, HTTP's `Content-Encoding: gzip`, PNG images, ZIP archives and WebSocket's `permessage-deflate`. It is thirty years old, it is not the best there is, and everything reads it.

It does two things, one after the other:

- **It replaces repeats by references.** Wherever the next few bytes have appeared before, within the last 32 KiB, it writes "the same as *distance* bytes ago, for *length* bytes" instead of the bytes. A log file, a web page or source code is mostly repeats.
- **It writes what is left in codes of different lengths**: short for what is common, long for what is rare.

So text shrinks to a third or a quarter, a table of numbers often to less, and **data that is already compressed or encrypted does not shrink at all** - it grows by a few bytes.

The compressed data needs something around it to be a file or a message. There are three choices, and **both ends must make the same one**:

Format	Around the data	Where it is used
`PROVEN_DEFLATE_GZIP`	A ten-byte header and, at the end, a CRC-32 of the data and its length (RFC 1952)	`.gz` files; HTTP's `gzip`
`PROVEN_DEFLATE_ZLIB`	A two-byte header and an Adler-32 of the data (RFC 1950)	PNG; HTTP's `deflate`
`PROVEN_DEFLATE_RAW`	Nothing	Inside another format that has its own framing: ZIP entries, `permessage-deflate`

The two with a checksum notice damage. Raw does not: a changed bit gives other data or an error, and nothing says which.

2. A whole buffer

```
proven_size_t proven_deflate_bound(proven_deflate_format_t format, proven_size_t input_size);
proven_err_t proven_deflate_all(const proven_deflate_options_t *options, proven_mem_view_t in,
                                proven_mem_mut_t out, proven_size_t *written);
proven_err_t proven_inflate_all(proven_allocator_t alloc, proven_deflate_format_t format, proven_mem_view_t
in,
                                proven_mem_mut_t out, proven_size_t *written, proven_size_t *consumed);
```

When the data is in memory and so is the room for the result, these two calls are all there is to it. Both write into a buffer of yours.

- **Compressing** needs a buffer of `proven_deflate_bound` bytes to be sure of fitting - the input's size plus a little, because data that does not compress grows slightly. With less, it may fail with `PROVEN_ERR_OUT_OF_BOUNDS`.
- **Decompressing** needs you to decide how large the result may be, because the compressed data does not get to. **The size of `out` is the limit**: a stream that would produce more is refused with `PROVEN_ERR_OUT_OF_BOUNDS`, however small it is itself. Section 5 says why there is no version of this that grows a buffer for you.

Compiled and run by the test suite:

```
#include <string.h>

/*
 * Compress a buffer, get it back, and refuse to be handed more than was agreed: the two
 * whole-buffer calls, and the one rule about decompressing what somebody else sent.
 */

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();

    /* Something that compresses: text repeats itself. */
    static proven_byte_t text[4000];
    static const char line[] = "GET /index.html HTTP/1.1 - 200 - text/html; charset=utf-8\n";
    for (proven_size_t i = 0; i < sizeof text; ++i) text[i] = (proven_byte_t)line[i % (sizeof line - 1)];

    /* Compressing never fails for want of room if the buffer is as large as the bound says. */
    static proven_byte_t packed[4200];
    proven_size_t packed_len = 0;
    proven_deflate_options_t options = { .alloc = heap, .format = PROVEN_DEFLATE_GZIP };
    EXAMPLE_REQUIRE(proven_deflate_bound(PROVEN_DEFLATE_GZIP, sizeof text) <= sizeof packed, "the bound for
4,000 bytes is a little more than 4,000");
    EXAMPLE_REQUIRE(proven_deflate_all(&options, (proven_mem_view_t){ text, sizeof text }, (proven_mem_mut_t)
{ packed, sizeof packed }, &packed_len) == PROVEN_OK,
                    "4,000 bytes of log lines compress");
    EXAMPLE_REQUIRE(packed_len < 200, "to less than 200: the same line, over and over");
    EXAMPLE_REQUIRE(packed[0] == 0x1f && packed[1] == 0x8b, "and what came out is a gzip member - a .gz file,
or an HTTP body with Content-Encoding: gzip");

    /* Decompressing: the output buffer is the limit. Here it is exactly as large as the data. */
    static proven_byte_t back[4000];
    proven_size_t back_len = 0, used = 0;
    EXAMPLE_REQUIRE(proven_inflate_all(heap, PROVEN_DEFLATE_GZIP, (proven_mem_view_t){ packed, packed_len },
(proven_mem_mut_t){ back, sizeof back }, &back_len, &used) == PROVEN_OK,
                    "it decompresses");
    EXAMPLE_REQUIRE(back_len == sizeof text && memcmp(back, text, sizeof text) == 0 && used == packed_len, "to
```

```
exactly what went in, using every byte of the member");

    /* The same member into a smaller buffer is refused. This is the whole defence against a
     * "decompression bomb": you say how much you are willing to receive, and that is all the
     * room there is. A few hundred bytes cannot make this program allocate a gigabyte. */
    static proven_byte_t small[1000];
    EXAMPLE_REQUIRE(proven_inflate_all(heap, PROVEN_DEFLATE_GZIP, (proven_mem_view_t){ packed, packed_len },
(proven_mem_mut_t){ small, sizeof small }, &back_len, NULL) == PROVEN_ERR_OUT_OF_BOUNDS,
                    "4,000 bytes do not fit in 1,000: PROVEN_ERR_OUT_OF_BOUNDS");

    /* Damage is noticed. gzip carries a CRC-32 of the data; one changed bit anywhere in the
     * compressed stream changes the data or breaks the format. */
    packed[packed_len / 2] ^= 0x04;
    EXAMPLE_REQUIRE(proven_inflate_all(heap, PROVEN_DEFLATE_GZIP, (proven_mem_view_t){ packed, packed_len },
(proven_mem_mut_t){ back, sizeof back }, &back_len, NULL) == PROVEN_ERR_INVALID_FORMAT,
                    "a changed bit: PROVEN_ERR_INVALID_FORMAT");
    packed[packed_len / 2] ^= 0x04;
    EXAMPLE_REQUIRE(proven_inflate_all(heap, PROVEN_DEFLATE_GZIP, (proven_mem_view_t){ packed, packed_len -
1 }, (proven_mem_mut_t){ back, sizeof back }, &back_len, NULL) == PROVEN_ERR_INVALID_FORMAT,
                    "a member cut short is not a member");

    /* The framing is a choice made at both ends. The same data as zlib is twelve bytes
     * smaller (a shorter header, a shorter checksum), and raw is smaller again - and has no
     * checksum at all, so damage to it can go unseen. */
    proven_size_t zlib_len = 0, raw_len = 0;
    static proven_byte_t other[4200];
    options.format = PROVEN_DEFLATE_ZLIB;
    EXAMPLE_REQUIRE(proven_deflate_all(&options, (proven_mem_view_t){ text, sizeof text }, (proven_mem_mut_t)
{ other, sizeof other }, &zlib_len) == PROVEN_OK && zlib_len == packed_len - 12,
                    "zlib framing: 12 bytes less than gzip");
    EXAMPLE_REQUIRE(proven_inflate_all(heap, PROVEN_DEFLATE_GZIP, (proven_mem_view_t){ other, zlib_len },
(proven_mem_mut_t){ back, sizeof back }, &back_len, NULL) == PROVEN_ERR_INVALID_FORMAT,
                    "and not gzip: a decompressor must be told which it is reading");
    options.format = PROVEN_DEFLATE_RAW;
    EXAMPLE_REQUIRE(proven_deflate_all(&options, (proven_mem_view_t){ text, sizeof text }, (proven_mem_mut_t)
{ other, sizeof other }, &raw_len) == PROVEN_OK && raw_len == zlib_len - 6,
                    "raw: no header and no checksum, 6 bytes less again");

    /* What does not compress grows a little - by the bound, at worst. */
    static proven_byte_t noise[4000], noisy[4200];
    proven_u32 x = 2463534242u;
    for (proven_size_t i = 0; i < sizeof noise; ++i) { x ^= x << 13; x ^= x >> 17; x ^= x << 5; noise[i] =
(proven_byte_t)x; }
    proven_size_t noisy_len = 0;
    options.format = PROVEN_DEFLATE_GZIP;
    EXAMPLE_REQUIRE(proven_deflate_all(&options, (proven_mem_view_t){ noise, sizeof noise },
(proven_mem_mut_t){ noisy, sizeof noisy }, &noisy_len) == PROVEN_OK &&
                    noisy_len > sizeof noise && noisy_len <= proven_deflate_bound(PROVEN_DEFLATE_GZIP, sizeof
noise),
                    "random bytes come out slightly larger, and never larger than the bound");

    return EXAMPLE_OK();
}
```

3. A stream

```
proven_err_t proven_deflate_create(const proven_deflate_options_t *options, proven_deflate_t **out);
proven_err_t proven_deflate(proven_deflate_t *z, proven_mem_view_t in, proven_size_t *consumed,
                            proven_mem_mut_t out, proven_size_t *produced, proven_deflate_flush_t flush, bool
*done);
```

```
void proven_deflate_reset(proven_deflate_t *z);
void proven_deflate_destroy(proven_deflate_t *z);

proven_err_t proven_inflate_create(proven_allocator_t alloc, proven_deflate_format_t format, proven_inflate_t
**out);
proven_err_t proven_inflate(proven_inflate_t *z, proven_mem_view_t in, proven_size_t *consumed,
                            proven_mem_mut_t out, proven_size_t *produced, bool *done);
void proven_inflate_reset(proven_inflate_t *z);
void proven_inflate_restart(proven_inflate_t *z);
void proven_inflate_destroy(proven_inflate_t *z);
```

When the data is a file larger than memory, a response still being produced, or a connection, both directions work the same way: an object that holds the state, and **one step function that takes some input and fills some output**.

The contract of a step is short, and worth reading once:

- It consumes as much of `in` as it can and produces as much into `out` as it can, and tells you how much of each. It never allocates and never writes past `out`. (Within `out`, the decompressor may write up to seven bytes beyond the ones it reports: pass space that is free.)
- `PROVEN_OK` **with nothing consumed and nothing produced is not an error.** It means the step needs more of one or the other: more input, or more room. Call it again when you have some. A stream that simply stops arriving looks exactly like this, which is why a caller that knows no more input is coming must treat it as a truncated stream.
- `*done` becomes true when the stream has ended and everything has been produced - also when the data exactly filled the room you gave. Input after the end is not consumed: `*consumed` tells you where the stream ended.

When compressing, `flush` **says how urgent the output is.**

`flush`	**Meaning**	**Use it**
`PROVEN_DEFLATE_FLUSH_NONE`	More is coming. Output appears when the compressor has a block's worth - possibly not for a long time	For everything except the two cases below
`PROVEN_DEFLATE_FLUSH_SYNC`	Everything given so far must be decodable now. The stream goes on	When somebody is waiting for what you have written: a message on a connection, a line of a live log. It costs a few bytes and some compression each time
`PROVEN_DEFLATE_FLUSH_FINISH`	That was the last input: end the stream	Once, at the end. Keep calling with it, and with more room, until `*done`

A sync flush ends on a byte boundary, and the last four bytes it produces are always `00 00 ff ff`. WebSocket's `permessage-deflate` sends each message as a sync flush with those four bytes left off, and the receiver puts them back (Chapter 12, section 9: the WebSocket drivers cut and put back those four bytes, and with context takeover keep one compressor and one decompressor for the whole connection). That is also why a raw stream need never end: the decompressor is fed piece after piece, `*done` never becomes true, and what it remembers of earlier pieces is what makes later ones small.

`proven_deflate_reset` and `proven_inflate_reset` start a new stream in the same object, which is cheaper than making another. After `*done`, a decompressor that should read a second gzip member following the first is reset and fed from where the first ended. `proven_inflate_restart` starts a new

stream too but keeps the last 32 KiB, so the new stream's matches may reach back into the old one: that is for `permessage-deflate` with context takeover, where a sender may end a message with a final block and its next message still refers to earlier ones.

Compiled and run by the test suite:

```
#include <string.h>

/*
 * Compression as a stream: data that arrives in pieces and leaves in pieces, through buffers
 * of whatever size you have. The same loop works for a file of any length, a response that
 * is still being produced, or a connection.
 */

/* Where the compressed bytes go. A real program writes them to a file or a socket. */
static proven_byte_t g_wire[8192];
static proven_size_t g_wire_len;

/* Give the compressor some input and take whatever output it has, 64 bytes at a time, until
 * it has taken all of the input - and, when `flush` asks for more than that, until it has
 * produced all of that too. */
static bool feed(proven_deflate_t *z, const char *text, proven_deflate_flush_t flush, bool *done) {
    proven_mem_view_t in = { (const proven_byte_t *)text, strlen(text) };
    for (;;) {
        proven_byte_t out[64];
        proven_size_t consumed = 0, produced = 0;
        if (proven_deflate(z, in, &consumed, (proven_mem_mut_t){ out, sizeof out }, &produced, flush, done) !=
PROVEN_OK) return false;
        if (g_wire_len + produced > sizeof g_wire) return false;
        memcpy(g_wire + g_wire_len, out, produced);
        g_wire_len += produced;
        in.ptr += consumed; in.size -= consumed;
        /* Nothing left to give, and it had less than a bufferful to say: it is up to date. */
        if (in.size == 0 && produced < sizeof out) return true;
    }
}

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();

    /* A compressor is made once. Zero in the options means the defaults: level 6, a 32 KiB
     * window. Level 1 is faster and compresses a little less; 9 is slower and a little more. */
    proven_deflate_options_t options = { .alloc = heap, .format = PROVEN_DEFLATE_ZLIB, .level = 6 };
    proven_deflate_t *z = NULL;
    EXAMPLE_REQUIRE(proven_deflate_create(&options, &z) == PROVEN_OK, "a compressor");

    /* Pieces go in as they come. With FLUSH_NONE the compressor keeps what it is given until
     * it has a block's worth - nothing may come out for a long time, and that is how it
     * compresses well. */
    bool done = false;
    EXAMPLE_REQUIRE(feed(z, "first piece of the stream; ", PROVEN_DEFLATE_FLUSH_NONE, &done), "a piece goes
in");
    const proven_size_t after_first = g_wire_len;
    EXAMPLE_REQUIRE(after_first <= 2 && !done, "and nothing but the two-byte header has come out yet");

    /* A sync flush says: the other side must be able to read everything up to here, now. It
     * costs a few bytes each time - use it when somebody is waiting, not after every write. */
    EXAMPLE_REQUIRE(feed(z, "second piece of the stream; ", PROVEN_DEFLATE_FLUSH_SYNC, &done) && g_wire_len >
after_first + 20 && !done,
                    "after a sync flush both pieces are on the wire");
```

```
    const proven_size_t at_flush = g_wire_len;

    /* FINISH ends the stream: the last block, and for zlib and gzip the checksum. */
    EXAMPLE_REQUIRE(feed(z, "third and last piece.", PROVEN_DEFLATE_FLUSH_FINISH, &done) && done, "the last
piece, and the stream is finished");

    /* The receiving side: a decompressor, fed 10 bytes at a time into a 16-byte buffer. */
    proven_inflate_t *u = NULL;
    EXAMPLE_REQUIRE(proven_inflate_create(heap, PROVEN_DEFLATE_ZLIB, &u) == PROVEN_OK, "a decompressor");
    char text[200];
    proven_size_t text_len = 0, at = 0, seen_at_flush = 0;
    bool ended = false;
    /* First only what had been written when the sync flush returned; then the rest. */
    for (proven_size_t limit = at_flush; !ended; limit = g_wire_len) {
        for (;;) {
            proven_byte_t out[16];
            proven_size_t consumed = 0, produced = 0;
            const proven_size_t give = limit - at < 10 ? limit - at : 10;
            if (proven_inflate(u, (proven_mem_view_t){ g_wire + at, give }, &consumed, (proven_mem_mut_t)
{ out, sizeof out }, &produced, &ended) != PROVEN_OK) break;
            if (text_len + produced > sizeof text) break;
            memcpy(text + text_len, out, produced);
            text_len += produced;
            at += consumed;
            /* PROVEN_OK with nothing consumed and nothing produced: it has used everything
             * it was offered and wants more. That is not an error - it is how a stream waits. */
            if (ended || (consumed == 0 && produced == 0)) break;
        }
        if (limit == at_flush) seen_at_flush = text_len;
        else break;
    }
    static const char whole[] = "first piece of the stream; second piece of the stream; third and last
piece.";
    EXAMPLE_REQUIRE(ended && text_len == sizeof whole - 1 && memcmp(text, whole, text_len) == 0, "the three
pieces come out as one text");
    EXAMPLE_REQUIRE(at == g_wire_len, "and the stream ended exactly where the compressor stopped writing");
    EXAMPLE_REQUIRE(seen_at_flush >= 55, "at the flush point the receiver already had both of the first two
pieces");

    /* Both objects are reused for the next stream rather than made again. */
    proven_deflate_reset(z);
    proven_inflate_reset(u);
    g_wire_len = 0;
    EXAMPLE_REQUIRE(feed(z, "another stream", PROVEN_DEFLATE_FLUSH_FINISH, &done) && done, "the same
compressor, a new stream");
    proven_byte_t again[32];
    proven_size_t consumed = 0, produced = 0;
    EXAMPLE_REQUIRE(proven_inflate(u, (proven_mem_view_t){ g_wire, g_wire_len }, &consumed, (proven_mem_mut_t)
{ again, sizeof again }, &produced, &ended) == PROVEN_OK &&
                    ended && produced == 14 && memcmp(again, "another stream", 14) == 0, "and the same
decompressor reads it");

    proven_deflate_destroy(z);
    proven_inflate_destroy(u);
    return EXAMPLE_OK();
}
```

4. Levels, windows and what they cost

A compressor is made from a `proven_deflate_options_t`. Zero-initialise it, set `alloc`, and set what you want changed:

Field	Meaning
`alloc`	Required. The compressor is one allocation, made here and freed by `proven_deflate_destroy`
`format`	The framing of section 1. Zero is `PROVEN_DEFLATE_RAW`
`level`	1 to 9: how hard to look for repeats. **0 is the default, 6.** -1 does not compress at all - the data is only wrapped
`window_bits`	How far back a repeat may be, as a power of two: 9 (512 bytes) to 15 (32 KiB). **0 is the default, 15**

The level trades time for size, and the trade is lopsided. On the development machine, compressing this library's own sources and manual (3.2 MiB of text):

Level	Compressed to	Compressing	Decompressing
1	29.4%	34 MiB/s	330 MiB/s
6 (default)	25.2%	16 MiB/s	375 MiB/s
9	25.0%	9 MiB/s	380 MiB/s

From 1 to 6 buys a seventh off the size for half the speed. From 6 to 9 buys almost nothing for nearly half the speed again. Decompression does not care what level was used.

zlib, the implementation everybody else uses, compresses the same data to the same sizes - within a quarter of a percent at levels 6 and 9, and it is a little larger at level 1 - and is about twice as fast at compressing, nearly three times at level 1. At decompressing the two are about the same: zlib 1.3.1, called from Python on the same data, gave between 190 and 310 MiB/s as the machine's load moved. (Before v0.42.0 this library's decompressor read a bit at a time and gave 75 to 90 MiB/s.) The compressor was written to be read; if its figures are too slow for a program, that program wants zlib.

The window is mostly a question of memory. A compressor holds twice its window and tables beside it; a decompressor always holds 32 KiB, because it cannot know what window the other side used:

	Memory
A decompressor	about 35 KiB
A compressor, `window_bits` 15 (default)	about 325 KiB
A compressor, `window_bits` 12	about 53 KiB
A compressor, `window_bits` 9	about 11 KiB

A smaller window compresses less well - repeats further apart than the window are not found - and makes no difference to speed worth planning around. Use it where there are many compressors at once (one per connection) or little memory.

5. Data from somebody else

Compressed data that arrives from outside the program is the dangerous case, for one reason:

It can be far larger than it looks. DEFLATE can say "the last byte again, 258 times" in two bytes, again and again. A hundred and seventeen bytes in this library's tests expand to 25,801; a

megabyte can become a gigabyte. A program that decompresses whatever it is sent into a buffer that grows as needed has given every sender the ability to exhaust its memory.

So nothing here grows. `proven_inflate` writes only into the space you pass it, and when that is full it stops and says so; `proven_inflate_all` takes the limit as the size of its output. Decide the largest result that makes sense for what you are reading - a configuration file, a request body, a message - and refuse anything beyond it. The decompressor's own memory is fixed at about 35 KiB whatever it is fed.

And it can simply be wrong, by accident or on purpose:

Returns	When
`PROVEN_ERR_INVALID_FORMAT`	The bytes are not a valid stream of the format the decompressor was made for; or - for gzip and zlib - the checksum at the end does not match the data; or, from `proven_inflate_all`, the input ended before the stream did
`PROVEN_ERR_UNSUPPORTED`	A zlib stream that needs a preset dictionary
`PROVEN_ERR_OUT_OF_BOUNDS`	From `proven_inflate_all`: the data does not fit in the buffer you gave

After an error a decompressor returns that error from every call until it is reset. Data it produced before the error is not to be trusted as complete: with gzip and zlib the checksum is only known to match at the very end.

Two more things worth knowing.

- **Compression can leak what it compresses.** If a secret and text chosen by an attacker are compressed together and the attacker can see the size of the result, the size says whether the chosen text repeated part of the secret, and the secret can be guessed a character at a time. This has broken HTTPS in practice (CRIME, BREACH). The rule is the application's to keep: do not compress a secret together with anything a stranger controls.
- **Raw DEFLATE has no checksum.** If the bytes may have been damaged on the way and the format around them does not check, use zlib or gzip.

6. Over HTTP

```
void proven_http_exchange_compress(proven_http_exchange_t *exchange);     /* the server of Chapter 11 */
void proven_http_stream_compress(proven_http_stream_t *stream);           /* the server of Chapter 15 */

bool proven_http_accepts_coding(const proven_http_header_t *headers, proven_size_t count, proven_http_coding_t
coding);
unsigned proven_http_accept_weight(const proven_http_header_t *headers, proven_size_t count,
proven_http_coding_t coding);
proven_http_coding_t proven_http_content_coding(const proven_http_header_t *headers, proven_size_t count);
```

HTTP calls this a *content coding*: the server compresses the body and says so in `Content-Encoding`, and it may only do that for a client whose `Accept-Encoding` allowed it. The four HTTP drivers of Chapter 11 and Chapter 15 do both halves - and **do neither until you ask**. A program that sets nothing sends the same bytes as before.

Unlike the rest of this chapter, this part needs sockets and is not in a freestanding build - except the three helper functions, which are plain header arithmetic in `http.h`.

A server compresses a response when its handler asks for that response. One call, before the response begins:

	What happens

The request allows gzip, and the response has a body of its own	Sent with `Content-Encoding: gzip` - or `zstd`, below
The request allows neither (no `Accept-Encoding`, `gzip;q=0`, only other codings)	Sent as it is
Status 204, 206 or 304; or your headers already have `Content-Encoding` or `Content-Range`	Sent as it is
A body sent in one piece that is under 256 bytes, or does not get smaller	Sent as it is
No memory for the compressor	Sent as it is
In every one of these cases	`Vary: Accept-Encoding` is added, unless your own `Vary` names it

`Vary` is there because the response now depends on a request header, and a cache in between must not hand the gzip to a client that cannot read it.

A server compresses in Zstandard too when its configuration says `compress_zstd`. A response asked to be compressed then goes as `zstd` when the request's `Accept-Encoding` weighs zstd at least as high as gzip, and as gzip otherwise, as before. curl, and the browsers that take zstd, send the two at one weight - `gzip, deflate, br, zstd` - so they get zstd; a client that writes `gzip;q=1, zstd;q=0.5` gets gzip, and one that does not name zstd gets what it got before. The tie goes to zstd because it is the cheaper of the two to make (section 7: six times the speed of gzip at level 1, at gzip 1's size or less). The level and the window are `compress_zstd_level` (1 to 9, zero is 1) and `compress_zstd_window_log` (10 to 23, zero is 17, 128 KiB); a body sent in one piece is compressed with tables sized to it whatever the window, a streamed one holds about 0.8 MiB at those defaults until it ends. Level 6 gives gzip 6's size or less at about three times its speed, for a server that wants gzip 6's size; its tables make a streamed response about 1.8 MiB. `proven_http_accept_weight` gives you the same weights, for a choice of your own.

A body sent in one piece is compressed first and sent with its new `Content-Length`. A body written in pieces is compressed as it is written and sent chunked, whatever length you announced - the length is still what you must write.

A client asks and decodes when its configuration says `decompress`. It then sends `Accept-Encoding: gzip` and decodes a body that comes back as `gzip` or `deflate`, on the way to you: `proven_http_client_read` fills your buffer with decoded bytes, and the event-driven client's `on_body` receives decoded pieces of at most 16 KiB.

With `decompress_zstd` as well, it asks for Zstandard too - `Accept-Encoding: zstd, gzip`

- and decodes a `zstd` body in the same way, with section 7's decoder and the 8 MiB window

limit that RFC 9659 sets for HTTP. It is a separate switch because of that window: a server may make a client hold up to 8 MiB for one response, where gzip needs 35 KiB. Without `decompress`, `decompress_zstd` is `PROVEN_ERR_INVALID_ARG`.

With `decompress_brotli`, it asks for Brotli too - `br` joins the list, `Accept-Encoding: zstd, br, gzip` with both switches - and decodes a `br` body with section 8's decoder, which takes every window RFC 7932 allows, up to 16 MiB; no HTTP specification sets a lower one, as RFC 9659 does for Zstandard. It is a switch of its own for the same reason as Zstandard's: a server decides how much of that a response costs you. Without `decompress` it is `PROVEN_ERR_INVALID_ARG`, and in a build with `PROVEN_NO_BROTLI` (section 8) `PROVEN_ERR_UNSUPPORTED`. The servers never send `br`: there is no Brotli encoder here.

Example: a page compressed, an account page not

```
#include <stdio.h>
#include <string.h>

/*
 * Compressed responses over HTTP: a server that compresses when its handler asks and the
 * client accepts, and a client that asks and decodes.
 *
 * Both are off until the program says otherwise. The server's part is one call in the
 * handler, per response; the client's part is one flag in its configuration.
 */

static proven_byte_t g_page[20000];        /* the page being served: text, so it compresses */

static void fill_page(void) {
    proven_size_t n = 0;
    for (unsigned k = 0; n < sizeof g_page; ++k) {
        char line[48];
        int len = snprintf(line, sizeof line, "row %u of a table that says much the same\n", k);
        for (int i = 0; i < len && n < sizeof g_page; ++i) g_page[n++] = (proven_byte_t)line[i];
    }
}

static void handle(void *ctx, proven_http_exchange_t *x) {
    proven_http_server_t **server = ctx;
    const proven_http_request_t *req = proven_http_exchange_request(x);
    proven_mem_view_t page = { g_page, sizeof g_page };

    if (proven_u8str_view_eq(req->target, PROVEN_LIT("/page"))) {
        /* One call, before the response begins. The server then does the rest: it reads the
         * request's Accept-Encoding, compresses if gzip is allowed, and writes
         * Content-Encoding, Vary and the new Content-Length. */
        proven_http_exchange_compress(x);
        proven_http_header_t type = { PROVEN_LIT("Content-Type"), PROVEN_LIT("text/plain") };
        (void)proven_http_exchange_respond(x, 200, &type, 1, page);

    } else if (proven_u8str_view_eq(req->target, PROVEN_LIT("/account"))) {
        /* Not asked for here, on purpose: this response would carry a secret next to text
         * the client chose, and the size of a compressed body gives the secret away a byte
         * at a time. Compression is asked for per response because only the handler knows. */
        (void)proven_http_exchange_respond(x, 200, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("token=... you
searched for: ...\n")));

    } else if (proven_u8str_view_eq(req->target, PROVEN_LIT("/accepts"))) {
        /* The same question the server asks, for a handler that wants to choose for itself -
         * between a file and its precompressed twin, say. */
        bool gzip = proven_http_accepts_coding(req->headers, req->header_count, PROVEN_HTTP_CODING_GZIP);
        (void)proven_http_exchange_respond(x, 200, NULL, 0, proven_mem_view_from_u8(gzip ?
PROVEN_LIT("gzip") : PROVEN_LIT("identity")));

    } else if (proven_u8str_view_eq(req->target, PROVEN_LIT("/log"))) {
        /* A compressed response written in pieces holds back what it is given. A flush hands
         * it over: the client reads "started" now, not when the work is done. */
        proven_http_exchange_compress(x);
        if (proven_http_exchange_begin(x, 200, NULL, 0, PROVEN_HTTP_LENGTH_UNKNOWN) == PROVEN_OK &&
            proven_http_exchange_write(x, proven_mem_view_from_u8(PROVEN_LIT("started\n"))) == PROVEN_OK &&
            proven_http_exchange_flush(x) == PROVEN_OK) {
            (void)proven_http_exchange_write(x, proven_mem_view_from_u8(PROVEN_LIT("finished\n")));
        }
```

```
    } else {
        (void)proven_http_exchange_respond(x, 200, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("bye\n")));
        proven_http_server_stop(*server);
    }
}

static void serve(void *arg) { (void)proven_http_server_run(*(proven_http_server_t **)arg); }

/* GET `path`; report the body's size as read, the Content-Length, and the coding named. */
static bool get(proven_http_client_t *client, proven_u16 port, const char *path,
                proven_size_t *read_size, proven_u64 *content_length, proven_http_coding_t *coding,
proven_u8str_t *body) {
    char url[96];
    int n = snprintf(url, sizeof url, "http://127.0.0.1:%u%s", (unsigned)port, path);
    proven_http_client_response_t resp;
    bool ok = proven_http_client_get(client, (proven_u8str_view_t){ (const proven_byte_t *)url,
(proven_size_t)n }, &resp) == PROVEN_OK && resp.status == 200;
    if (ok) {
        /* The headers are the server's, untouched: this is how to see what was sent. */
        *coding = proven_http_content_coding(resp.headers, resp.header_count);
        proven_u8str_view_t length = { 0 };
        *content_length = 0;
        if (proven_http_header_find(resp.headers, resp.header_count, PROVEN_LIT("Content-Length"), &length)) {
            for (proven_size_t i = 0; i < length.size; ++i) *content_length = *content_length * 10 +
(proven_u64)(length.ptr[i] - '0');
        }
        /* The limit is on what arrives here - decoded bytes, when the client decodes. */
        (void)proven_u8str_reset(body);
        ok = proven_http_client_read_all(&resp, proven_heap_allocator(), body, 1024 * 1024) == PROVEN_OK;
        *read_size = proven_u8str_as_view(body).size;
    }
    proven_http_client_finish(&resp);
    return ok;
}

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    fill_page();

    static proven_http_server_t *server;
    proven_http_server_config_t config = { .alloc = heap, .handler = handle, .handler_ctx = &server };
    config.compress_level = 6;              /* 1 is fastest, 9 smallest; zero means this */
    EXAMPLE_REQUIRE(proven_http_server_create(&config, &server) == PROVEN_OK, "a server");
    proven_net_addr_t at;
    proven_err_t err = proven_http_server_listen(server, proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0),
&at);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        proven_http_server_destroy(server);
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK, "it listens");
    proven_job_sys_t *threads = NULL;
    proven_job_group_t running;
    proven_job_group_init(&running);
    EXAMPLE_REQUIRE(proven_job_system_init(heap, 1, 4, &threads) == PROVEN_OK &&
                    proven_job_group_submit(threads, &running, serve, &server) == PROVEN_OK, "the server loop
is started");

    /* Two clients: one as it always was, one that asks for compressed responses. */
```

```
    proven_http_client_config_t plain_config = { .alloc = heap, .max_idle_connections = 2 };
    proven_http_client_config_t decoding_config = { .alloc = heap, .max_idle_connections = 2, .decompress =
true };
    proven_http_client_t *plain = NULL, *decoding = NULL;
    EXAMPLE_REQUIRE(proven_http_client_create(&plain_config, &plain) == PROVEN_OK &&
                    proven_http_client_create(&decoding_config, &decoding) == PROVEN_OK, "two clients");

    proven_u8str_t body = { 0 };
    proven_size_t read_size = 0;
    proven_u64 sent = 0;
    proven_http_coding_t coding = PROVEN_HTTP_CODING_IDENTITY;

    /* The client that sets nothing sends no Accept-Encoding, and nothing changes for it. */
    EXAMPLE_REQUIRE(get(plain, at.port, "/page", &read_size, &sent, &coding, &body), "the page, to a client
that did not ask");
    EXAMPLE_REQUIRE(coding == PROVEN_HTTP_CODING_IDENTITY && sent == sizeof g_page && read_size == sizeof
g_page, "arrives as it is");

    /* The one that asks gets the same bytes from a fraction of them. */
    EXAMPLE_REQUIRE(get(decoding, at.port, "/page", &read_size, &sent, &coding, &body), "the page, to a client
that asked");
    EXAMPLE_REQUIRE(coding == PROVEN_HTTP_CODING_GZIP && read_size == sizeof g_page &&
                    memcmp(proven_u8str_as_view(&body).ptr, g_page, sizeof g_page) == 0, "was sent as gzip and
read as the page");
    EXAMPLE_REQUIRE(sent < sizeof g_page / 5, "in less than a fifth of the bytes");
    printf("the page: %u bytes, sent as %u\n", (unsigned)sizeof g_page, (unsigned)sent);

    /* A response whose handler did not ask is not compressed, whoever asks for it. */
    EXAMPLE_REQUIRE(get(decoding, at.port, "/account", &read_size, &sent, &coding, &body) && coding ==
PROVEN_HTTP_CODING_IDENTITY, "the account page is sent as it is");

    EXAMPLE_REQUIRE(get(decoding, at.port, "/accepts", &read_size, &sent, &coding, &body) &&
                    proven_u8str_view_eq(proven_u8str_as_view(&body), PROVEN_LIT("gzip")), "the handler can
see that this client accepts gzip");
    EXAMPLE_REQUIRE(get(plain, at.port, "/accepts", &read_size, &sent, &coding, &body) &&
                    proven_u8str_view_eq(proven_u8str_as_view(&body), PROVEN_LIT("identity")), "and that the
other does not");

    /* A response flushed in the middle is still one gzip stream. */
    EXAMPLE_REQUIRE(get(decoding, at.port, "/log", &read_size, &sent, &coding, &body) && coding ==
PROVEN_HTTP_CODING_GZIP &&
                    proven_u8str_view_eq(proven_u8str_as_view(&body), PROVEN_LIT("started\nfinished\n")), "a
log sent in two parts, the first flushed, reads as one");

    EXAMPLE_REQUIRE(get(plain, at.port, "/quit", &read_size, &sent, &coding, &body), "stop");
    proven_job_group_wait(threads, &running);
    proven_u8str_destroy(heap, &body);
    proven_http_client_destroy(plain);
    proven_http_client_destroy(decoding);
    proven_http_server_destroy(server);
    proven_job_system_close(threads);
    proven_job_system_destroy(threads);
    return EXAMPLE_OK();
}
```

Example: the same on a loop

```
#include <stdio.h>
#include <string.h>

/*
```

```
 * The same on a loop: the event-driven server compressing a response that is written in
 * pieces, and the event-driven client decoding it as it arrives.
 *
 * Neither holds the body. The server compresses what it is given into its output queue, under
 * the queue's limit; the client hands on decoded pieces of at most 16 KiB.
 */

#define REPORT_BYTES ((proven_size_t)300000)

typedef struct {
    proven_loop_t *loop;
    proven_http_stream_t *stream;      /* the one response being written */
    proven_size_t written;
    proven_size_t received, pieces, largest;
    unsigned long sum_sent, sum_received;
    bool gzip, intact, done;
    proven_err_t why;
} ex_app_t;

/* The report is generated as it is written: there is no buffer holding all of it. */
static proven_byte_t report_byte(proven_size_t i) { return (proven_byte_t)("measured value \n"[i % 16]); }

// ---- the server ------------------------------------------------------------

/* Write until the server takes no more; on_writable calls this again when there is room. */
static void pump(ex_app_t *app) {
    proven_byte_t piece[4096];
    while (app->written < REPORT_BYTES) {
        proven_size_t n = REPORT_BYTES - app->written < sizeof piece ? REPORT_BYTES - app->written : sizeof
piece;
        for (proven_size_t i = 0; i < n; ++i) piece[i] = report_byte(app->written + i);
        proven_result_size_t took = proven_http_stream_write(app->stream, (proven_mem_view_t){ piece, n });
        if (took.err != PROVEN_OK) return;
        for (proven_size_t i = 0; i < took.value; ++i) app->sum_sent += piece[i];
        app->written += took.value;
        /* The first piece is handed over at once: the reader sees the report begin while
         * the rest is still being made. Without this it would wait in the compressor. */
        if (app->written == sizeof piece) (void)proven_http_stream_flush(app->stream);
        if (took.value < n) return;        /* the compressed output is at its limit: wait */
    }
    (void)proven_http_stream_end(app->stream);
}

static void on_request(void *ctx, proven_http_stream_t *stream, const proven_http_request_t *head) {
    (void)head;
    ex_app_t *app = ctx;
    app->stream = stream;
    /* Before the response begins. From here on the body is written exactly as it would be
     * without it: the length given is still the length to write, though the client is sent
     * chunks of gzip. What is written may wait in the compressor until more follows. */
    proven_http_stream_compress(stream);
    if (proven_http_stream_begin(stream, 200, NULL, 0, REPORT_BYTES) == PROVEN_OK) pump(app);
}
static void on_writable(void *ctx, proven_http_stream_t *stream) { (void)stream; pump(ctx); }

// ---- the client ------------------------------------------------------------

static void on_response(void *ctx, proven_http_event_request_t *request, const proven_http_response_t *head) {
    (void)request;
    ex_app_t *app = ctx;
```

```
    app->gzip = proven_http_content_coding(head->headers, head->header_count) == PROVEN_HTTP_CODING_GZIP;
}

/* Decoded pieces. A piece is a view that is good until this function returns. */
static void on_body(void *ctx, proven_http_event_request_t *request, proven_mem_view_t piece, bool last) {
    (void)request; (void)last;
    ex_app_t *app = ctx;
    for (proven_size_t i = 0; i < piece.size; ++i) {
        if (piece.ptr[i] != report_byte(app->received + i)) app->intact = false;
        app->sum_received += piece.ptr[i];
    }
    app->received += piece.size;
    app->pieces++;
    if (piece.size > app->largest) app->largest = piece.size;
}

static void on_done(void *ctx, proven_http_event_request_t *request, proven_err_t why) {
    (void)request;
    ex_app_t *app = ctx;
    app->why = why;
    app->done = true;
    proven_loop_stop(app->loop);
}

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    ex_app_t app = { .intact = true };
    EXAMPLE_REQUIRE(proven_loop_create(heap, &app.loop) == PROVEN_OK, "a loop");

    /* The window sets what each compressed response holds while it is being written: about
     * 53 KiB at 12 bits, 325 KiB at the full 15. With many at once, that is the number to choose. */
    proven_http_event_server_config_t server_config = { .on = { .on_request = on_request, .on_writable =
on_writable }, .ctx = &app, .compress_window_bits = 12 };
    proven_http_event_server_t *server = NULL;
    proven_net_addr_t at;
    EXAMPLE_REQUIRE(proven_http_event_server_create(app.loop, &server_config, &server) == PROVEN_OK, "a
server");
    proven_err_t err = proven_http_event_server_listen(server,
proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0), &at);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        proven_http_event_server_destroy(server);
        proven_loop_destroy(app.loop);
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK, "it listens");

    /* `decompress` asks for gzip and decodes it; `max_body_bytes` then also bounds the decoded
     * size, so a small body that unfolds into a huge one is cut off at the limit. */
    proven_http_event_client_config_t client_config = { .decompress = true, .max_body_bytes = 1024 * 1024 };
    proven_http_event_client_t *client = NULL;
    EXAMPLE_REQUIRE(proven_http_event_client_create(app.loop, &client_config, &client) == PROVEN_OK, "a client
that decodes");

    char url[64];
    int n = snprintf(url, sizeof url, "http://127.0.0.1:%u/report", (unsigned)at.port);
    proven_http_event_request_options_t options = {
        .url = { (const proven_byte_t *)url, (proven_size_t)n },
        .on = { on_response, on_body, NULL, on_done },
        .ctx = &app,
```

```
    };
    EXAMPLE_REQUIRE(proven_http_event_client_start(client, &options, NULL) == PROVEN_OK, "the request is
started");
    EXAMPLE_REQUIRE(proven_loop_run(app.loop) == PROVEN_OK && app.done, "the loop ran until it was done");

    EXAMPLE_REQUIRE(app.why == PROVEN_OK && app.gzip, "the response came as gzip");
    EXAMPLE_REQUIRE(app.received == REPORT_BYTES && app.intact && app.sum_received == app.sum_sent, "and was
delivered as the report that was written");
    EXAMPLE_REQUIRE(app.largest <= 16384 && app.pieces >= REPORT_BYTES / 16384, "in pieces of at most 16
KiB");
    printf("%u bytes written, compressed, sent, decoded and received\n", (unsigned)app.received);

    proven_http_event_client_destroy(client);
    proven_http_event_server_destroy(server);
    proven_loop_destroy(app.loop);
    return EXAMPLE_OK();
}
```

Reference

API	Intent	Return
`proven_http_exchange_compress(x)`	Ask that this response be compressed. Before `respond` or `begin`; afterwards it does nothing.	none.
`proven_http_stream_compress(stream)`	The same for the event-driven server: in `on_request` or later, before the response begins.	none.
`proven_http_exchange_flush(x)`, `proven_http_stream_flush(stream)`	Hand over what a compressed response holds back; the response goes on. For a response that is not being compressed: nothing, successfully.	`proven_err_t`: `INVALID_STATE` before the response begins or after it ends; otherwise as a write.
`compress_level` (both server configurations)	1 (fastest) to 9 (smallest); -1 stores without compressing. Zero is 6. Section 4 has the measurements.	`INVALID_ARG` from `create` outside that range.
`compress_window_bits` (both)	9 to 15: the memory a streamed response holds while it is written (section 4's table). Zero is 15.	the same.
`compress_zstd` (both server configurations)	Compress in `zstd` for a request that weighs it at least as high as gzip; gzip otherwise. False: gzip only.	-
`compress_zstd_level`, `compress_zstd_window_log` (both)	Section 7's encoder level (1 to 9, zero is 1) and window (10 to 23, zero is 17).	`INVALID_ARG` from `create` outside those ranges.
`decompress` (both client configurations)	Send `Accept-Encoding: gzip` and decode `gzip` and `deflate` bodies.	-

`decompress_zstd` (both client configurations)	With `decompress`: send `Accept-Encoding: zstd, gzip` and decode `zstd` bodies too.	`INVALID_ARG` from `create` without `decompress`.
`decompress_brotli` (both client configurations)	With `decompress`: ask for `br` too and decode `br` bodies (since v0.60.0).	`INVALID_ARG` from `create` without `decompress`; `UNSUPPORTED` in a build with `PROVEN_NO_BROTLI`.
`proven_http_accepts_coding(headers, count, coding)`	Whether a request's `Accept-Encoding` allows a coding: listed with a weight above zero, or covered by `*`.	`bool`. Without the field only identity is accepted.
`proven_http_accept_weight(headers, count, coding)`	How much: the coding's weight, else that of `*`, in thousandths.	0 to 1000; 0 exactly where `accepts_coding` says no. Without the field, 1000 for identity and 0 for the rest.
`proven_http_content_coding(headers, count)`	What `Content-Encoding` names.	`PROVEN_HTTP_CODING_IDENTITY`, `_GZIP` (also for `x-gzip`), `_DEFLATE`, `_ZSTD` (since v0.57.0; `_OTHER` before), `_BROTLI` for `br` (since v0.60.0; `_OTHER` before), or `_OTHER` for anything else and for more than one.

Cautions, and what goes wrong

- **Do not ask for a response that mixes a secret with text the client chose.** This is the leak of section 5 in its best-known form (BREACH): a page that carries a session token and also echoes a search term, compressed, tells an observer of the encrypted connection - through its length alone - whether the term matched part of the token. TLS does not help; it hides bytes, not sizes. This is why the server has no "compress everything" setting: the call is per response because the judgement is.
- **A streamed, compressed response does not arrive piece by piece unless you flush it.** What you write may sit in the compressor until more is written or the response ends. For an event stream (Chapter 10, section 13) or anything else a client must see as it happens, call `proven_http_exchange_flush` or `proven_http_stream_flush` after each piece: what was written so far then leaves in a form the client can read at once. A flush costs a few bytes and some of the compression, so flush where a reader is waiting and not after every write - or do not compress such a response at all.
- **A compressed response of a different size needs a different validator.** If you send an `ETag`, the compressed and the plain response are two representations; give them different tags, or a cache may match one against the other. The server does not rewrite yours.
- **The client's headers are the server's.** With `decompress`, `Content-Encoding: gzip` is still there and `Content-Length` still counts the compressed bytes - not the bytes you read. Count what you read, or look at `proven_http_content_coding` to know which it was.
- **The limit on what a server may send you is in decoded bytes.** `read_all`'s `max_bytes`, and the event-driven client's `max_body_bytes`, count what comes out of the decoder: two kilobytes that unfold into two megabytes are cut off at your limit, not at theirs. A plain `proven_http_client_read` is bounded by the buffer you give it, as always.
- **A damaged compressed body is** `PROVEN_ERR_INVALID_FORMAT`, never a short success: a stream that is cut off, fails its checksum, or is followed by bytes that are not another gzip member (or

Zstandard frame). Several gzip members, or Zstandard frames, in a row are read as one body; a Brotli stream has nothing after it.

- `deflate` **means two things in practice.** The standard says a zlib stream; some servers send raw DEFLATE. The client looks at the first two bytes and reads whichever it is. It never asks for `deflate` - only `gzip`, `zstd` with `decompress_zstd` and `br` with `decompress_brotli` - and the servers never send it.
- **Not decoded:** a `206 Partial Content` (a range of a compressed body is not a compressed body - and with `decompress` set, a request that carries `Range` is sent without `Accept-Encoding`); `zstd` without `decompress_zstd`; `br` without `decompress_brotli`; any other coding (`compress`); two codings applied one over the other. Such a body reaches you as it was sent, with its header.
- **Memory.** A response being decoded costs about 35 KiB (52 KiB in the event-driven client) while it is open - a `zstd` one, as much as its frame's window, up to 8 MiB, and about 400 KiB more, and a `br` one as much as its stream's window, up to 16 MiB, and its codes (section 8), each allocated as its output grows. A streamed response being compressed costs what section 4's table says for the window - 325 KiB by default - until it ends; one sent in one piece costs that only for the call, with a window no larger than the body. Ten thousand streamed responses at the default window are 3 GiB: lower `compress_window_bits`, or compress fewer. In `zstd` a streamed response holds about 0.8 MiB at the defaults (level 1, a 128 KiB window) - more than gzip's 325 KiB - and up to 2 MiB at level 3 with a 512 KiB window; one sent in one piece needs tables sized to the body, a few kilobytes for a short one.

7. Zstandard

```
#include "proven/zstd.h"

proven_err_t proven_zstd_decoder_create(proven_allocator_t alloc, proven_u64 window_max, proven_zstd_decoder_t
**out);
void proven_zstd_decoder_destroy(proven_zstd_decoder_t *z);
void proven_zstd_decoder_reset(proven_zstd_decoder_t *z);
proven_err_t proven_zstd_decode(proven_zstd_decoder_t *z, proven_mem_view_t in, proven_size_t *consumed,
                                proven_mem_mut_t out, proven_size_t *produced, bool *done);
proven_err_t proven_zstd_decode_all(proven_allocator_t alloc, proven_mem_view_t in, proven_mem_mut_t out,
proven_size_t *written);
```

Zstandard (RFC 8878) is a newer format than DEFLATE: `.zst` files, HTTP's `Content-Encoding: zstd` (RFC 9659), and some package formats. Its compressors range from faster than gzip's fastest to smaller than gzip's smallest; whatever was used, it decodes quickly. This library reads it, and since v0.58.0 writes it - at nine levels since v0.59.0 (below, "Writing Zstandard").

The decoder is used like section 3's decompressor - a state, and a step that takes what input there is and fills what output space there is - with two differences that the format makes necessary:

- **A step may allocate.** Zstandard reads parts of each block from its end, so a block (up to 128 KiB compressed) is held whole before it is decoded, and decoded whole next to the history that later blocks refer to. These buffers grow as a frame's output grows - a frame that says it holds 300 bytes costs 300 bytes of history, not 8 MiB - and are kept for the next frame. So `proven_zstd_decode` can return `PROVEN_ERR_NOMEM`.
- **Output can wait in the decoder**, by up to one block, while the space you give is too small for it.

The window is the memory, and the limit is yours. Every frame says how far back it may refer - its window - and a decoder must keep that much of what it produced. A frame may claim terabytes. `window_max` is the most you agree to: 0 means 8 MiB, which RFC 9659 sets for HTTP. A frame that needs more is `PROVEN_ERR_OUT_OF_BOUNDS` before anything is allocated for it.

	Memory
A decoder, made	about 20 KiB of tables

While decoding a frame	its window (at most your limit), plus up to 384 KiB for the block being decoded; through `proven_zstd_decode_all`, a frame that states its size and fits holds no window - it is decoded in place

A stream is a sequence of frames. `*done` becomes true where a frame ends, and that call stops there: what it consumed is that frame. Call again with the rest and the next frame begins - no reset is needed. Skippable frames (magic numbers `0x184D2A50` to `0x184D2A5F`) are read past, producing nothing. A frame that needs a dictionary is `PROVEN_ERR_UNSUPPORTED`; content sent over HTTP does not use one.

Example: a buffer, two frames, and a window too large

```
#include <string.h>

/*
 * Zstandard: a whole buffer, then a stream of two frames read piece by piece, and the window
 * limit that keeps a stranger's frame from deciding how much memory this program spends.
 */

/* 4,000 bytes of log lines, as libzstd wrote them at level 19 with a checksum: 77 bytes. */
static const proven_byte_t FRAME[] = {
    0x28, 0xb5, 0x2f, 0xfd, 0x64, 0xa0, 0x0e, 0xfd, 0x01, 0x00, 0x62, 0xc3,
    0x0c, 0x11, 0x90, 0x7d, 0x00, 0x04, 0x1c, 0xf9, 0xc4, 0x4b, 0x1e, 0x4d,
    0x80, 0xa0, 0x8f, 0x99, 0x3a, 0xe6, 0x0c, 0xe0, 0x84, 0xb9, 0x79, 0xda,
    0x52, 0xe7, 0x47, 0xe1, 0xa3, 0xb1, 0xbb, 0xdc, 0x1f, 0x9e, 0x10, 0xf3,
    0xe1, 0x8b, 0x28, 0xa3, 0x39, 0xc7, 0xde, 0xde, 0x4d, 0x45, 0x97, 0x11,
    0xd7, 0xf1, 0x4e, 0xad, 0x02, 0x00, 0xc6, 0xde, 0x95, 0x2f, 0x3b, 0x3b,
    0x9f, 0xe8, 0x22, 0xa1, 0x2a,
};

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    static const char line[] = "GET /index.html HTTP/1.1 - 200 - text/html; charset=utf-8\n";
    static proven_byte_t text[4000];
    for (proven_size_t i = 0; i < sizeof text; ++i) text[i] = (proven_byte_t)line[i % (sizeof line - 1)];

    /* A whole buffer: as with gzip, the output buffer is the limit. */
    static proven_byte_t out[8000];
    proven_size_t n = 0;
    EXAMPLE_REQUIRE(proven_zstd_decode_all(heap, (proven_mem_view_t){ FRAME, sizeof FRAME },
(proven_mem_mut_t){ out, sizeof out }, &n) == PROVEN_OK &&
                    n == sizeof text && memcmp(out, text, n) == 0, "77 bytes of Zstandard decode to the 4,000
bytes libzstd was given");
    EXAMPLE_REQUIRE(proven_zstd_decode_all(heap, (proven_mem_view_t){ FRAME, sizeof FRAME },
(proven_mem_mut_t){ out, 3999 }, &n) == PROVEN_ERR_OUT_OF_BOUNDS,
                    "and do not fit in 3,999: PROVEN_ERR_OUT_OF_BOUNDS");

    /* A stream: the same frame twice, given 10 bytes at a time with 500 bytes of room. Each
     * call takes what it can; `done` says a frame has just ended, and the next call simply
     * goes on with the frame that follows. */
    static proven_byte_t two[2 * sizeof FRAME];
    memcpy(two, FRAME, sizeof FRAME);
    memcpy(two + sizeof FRAME, FRAME, sizeof FRAME);
    proven_zstd_decoder_t *z = NULL;
    EXAMPLE_REQUIRE(proven_zstd_decoder_create(heap, 0, &z) == PROVEN_OK, "a decoder with the default window
limit, 8 MiB");
    proven_size_t in_at = 0, out_at = 0;
    int frames = 0;
    while (in_at < sizeof two || out_at < sizeof out) {
```

```
        proven_size_t give = sizeof two - in_at < 10 ? sizeof two - in_at : 10, room = sizeof out - out_at <
500 ? sizeof out - out_at : 500;
        proven_size_t used = 0, made = 0;
        bool done = false;
        EXAMPLE_REQUIRE(proven_zstd_decode(z, (proven_mem_view_t){ two + in_at, give }, &used,
(proven_mem_mut_t){ out + out_at, room }, &made, &done) == PROVEN_OK, "a step");
        in_at += used;
        out_at += made;
        if (done && used + made > 0) frames++;
        if (used == 0 && made == 0) break;              /* nothing more to give, or nowhere to put it */
    }
    EXAMPLE_REQUIRE(frames == 2 && out_at == 2 * sizeof text && memcmp(out + sizeof text, text, sizeof text)
== 0, "two frames, one after the other, 8,000 bytes");

    /* The window limit. Every frame says how far back it may refer, and a decoder must keep
     * that much; this hand-made header asks for 1 MiB. A decoder limited to 64 KiB refuses it
     * before allocating anything for it. reset makes the decoder usable again. */
    static const proven_byte_t wide[] = { 0x28, 0xb5, 0x2f, 0xfd, 0x00, (20 - 10) << 3, 0x19, 0x00, 0x00, 'a',
'b', 'c' };
    proven_zstd_decoder_t *small = NULL;
    proven_size_t used = 0, made = 0;
    bool done = false;
    EXAMPLE_REQUIRE(proven_zstd_decoder_create(heap, 64 * 1024, &small) == PROVEN_OK, "a decoder limited to 64
KiB");
    EXAMPLE_REQUIRE(proven_zstd_decode(small, (proven_mem_view_t){ wide, sizeof wide }, &used,
(proven_mem_mut_t){ out, sizeof out }, &made, &done) == PROVEN_ERR_OUT_OF_BOUNDS,
                    "a frame that needs 1 MiB of window: PROVEN_ERR_OUT_OF_BOUNDS");
    proven_zstd_decoder_reset(small);
    EXAMPLE_REQUIRE(proven_zstd_decode(small, (proven_mem_view_t){ FRAME, sizeof FRAME }, &used,
(proven_mem_mut_t){ out, sizeof out }, &made, &done) == PROVEN_OK &&
                    done && made == sizeof text, "after a reset, a frame within the limit decodes");
    EXAMPLE_REQUIRE(proven_zstd_decode(z, (proven_mem_view_t){ wide, sizeof wide }, &used, (proven_mem_mut_t)
{ out, sizeof out }, &made, &done) == PROVEN_OK &&
                    done && made == 3, "while the default limit takes it");

    proven_zstd_decoder_destroy(small);
    proven_zstd_decoder_destroy(z);
    return EXAMPLE_OK();
}
```

Reference

API	Intent	Return
`proven_zstd_decoder_create(alloc, window_max, &z)`	A decoder. `window_max` from 1 KiB to 2 GiB; 0 is 8 MiB.	`INVALID_ARG`, `NOMEM`.
`proven_zstd_decode(z, in, &consumed, out, &produced, &done)`	Take some of `in`, fill some of `out`; stop at a frame's end. Only the bytes reported produced are written.	`INVALID_FORMAT` (damaged, a checksum or stated size that does not match), `OUT_OF_BOUNDS` (window above the limit), `UNSUPPORTED` (a dictionary), `NOMEM`, `INVALID_ARG`; after an error, the same until a reset.
`proven_zstd_decoder_reset(z)`	Forget the stream in progress; keep the buffers.	none.
`proven_zstd_decoder_destroy(z)`	Free it. NULL is allowed.	none.

`proven_zstd_decode_all(alloc, in, out, &written)`	Every frame of `in` into `out`, with the default limit. `out`'s size is the limit on output. A frame that states its size and fits is decoded straight into `out`, holding no window of its own (since v0.86.0); bytes of `out` past `*written` may then be written.	`OUT_OF_BOUNDS` when it does not fit; `INVALID_FORMAT` also when `in` is empty or ends inside a frame.

Cautions, and what goes wrong

- **The window, not the size of the data, decides the memory.** A server - or anyone whose frames you decode - chooses it, up to your limit. A thousand responses at once, each with an 8 MiB window, are 8 GiB. Lower `window_max` where you can (a decoder for your own `.zst` files knows what window they use), and count decoders the way you count connections.
- **The checksum is optional in Zstandard.** A frame written without one can be damaged and still decode to wrong bytes; the decoder can only refuse what does not parse. Where it matters, check what you decoded yourself.
- **Not a gzip stream.** `proven_zstd_decode` reads Zstandard only. Over HTTP, the clients choose the decoder from `Content-Encoding`; for files, look at the first four bytes (`28 b5 2f fd`).

Nothing that libzstd refuses is accepted; some damage it reads through is refused. Where RFC 8878 leaves a case to the decoder it was settled as libzstd 1.5.7 settles it - Huffman codes up to 12 bits are read - or more strictly: every Huffman and sequence stream must be read to its first bit exactly, which libzstd's fast Huffman decoder does not check for four streams, and a compressed block of no bytes is refused, as RFC 8878 has it (libzstd's streaming decoder takes it as empty, its one-call decoder refuses it). Outside the registered tests, 80,232 truncated and changed copies of 92 streams libzstd made were given to both: none that libzstd refused was accepted here, none that both accepted came out different, and 1,985 that libzstd accepted were refused here: 1,948 for one of four Huffman streams not read exactly, 25 for bytes after the frame (which Node, the way libzstd was called, ignores), 8 for a compressed block of no bytes, and 4 for a block too short to hold its literal.

Speed. On the development machine, the C sources of this library (2.9 MiB) compressed by libzstd at levels 1, 3 and 19 decode through `proven_zstd_decode_all` at about 650, 650 and 750 MiB/s, and through a decoder given all of it in one call at about 590, 570 and 730 MiB/s; libzstd, in the same process, at about 1,020, 940 and 1,030 MiB/s - so this decoder runs at 0.6 to 0.7 of libzstd's speed (v0.86.0; about half before it). Before v0.86.0, 2.2 MiB of the same sources as gzip went through section 3's decompressor at about 335 MiB/s.

Writing Zstandard

```
proven_err_t proven_zstd_encoder_create(const proven_zstd_encoder_options_t *options, proven_zstd_encoder_t
**out);
void proven_zstd_encoder_destroy(proven_zstd_encoder_t *z);
void proven_zstd_encoder_reset(proven_zstd_encoder_t *z);
proven_err_t proven_zstd_encode(proven_zstd_encoder_t *z, proven_mem_view_t in, proven_size_t *consumed,
                                proven_mem_mut_t out, proven_size_t *produced, proven_zstd_flush_t flush, bool
*done);
proven_size_t proven_zstd_encode_bound(proven_size_t input_size);
proven_err_t proven_zstd_encode_all(const proven_zstd_encoder_options_t *options, proven_mem_view_t in,
proven_mem_mut_t out, proven_size_t *written);
```

The encoder is used like section 3's compressor: options (`alloc`, `level`, `window_log`, `checksum`), a state, and a step with a flush - `PROVEN_ZSTD_FLUSH_NONE` while more is coming, `_BLOCK` to make everything given so

far decodable while the frame goes on, `_FINISH` to end the frame. Its frames are read by any Zstandard decoder; they are not the bytes libzstd would have written for the same input.

Nine levels, as libzstd's first nine. Level 1 finds matches through one hash table, levels 2 and 3 through two (libzstd's "fast" and "double fast"); levels 4 to 9 keep every position in a hash chain and look at the first 4 to 32 of it, taking the best match found (levels 4 and 5) or first looking one byte on (6) or two (7 to 9) for a better one. Every block's literals get a Huffman code and its sequences FSE tables, each chosen for the block by what it would cost. Measured on the three files of section 4, on one core of the development machine, at the release flags:

	HTML page	**Markdown**	**C source**
this library, gzip level 1	26.25 % at 45 MiB/s	36.77 % at 33	27.24 % at 43
this library, gzip level 6	22.62 % at 25	32.89 % at 16	22.88 % at 21
this library, Zstandard level 1	24.93 % at 263	36.19 % at 191	26.71 % at 234
this library, Zstandard level 3	23.83 % at 193	34.58 % at 132	25.50 % at 172
this library, Zstandard level 6	21.66 % at 67	32.68 % at 49	22.79 % at 64
this library, Zstandard level 9	21.22 % at 37	32.11 % at 28	22.21 % at 36
libzstd 1.5.7, level 1	25.27 % at 312	36.68 % at 231	26.91 % at 273
libzstd 1.5.7, level 3	23.76 % at 232	34.42 % at 161	25.42 % at 213
libzstd 1.5.7, level 6	21.93 % at 74	32.65 % at 54	22.97 % at 67
libzstd 1.5.7, level 9	21.25 % at 48	32.12 % at 36	22.22 % at 43

Level 1 is between five and six times as fast as this library's gzip at level 1, and smaller; level 6 is no larger than gzip at level 6 and between 2.6 and 3.2 times as fast. Against libzstd, levels 1 to 3 and 5 to 8 run at 0.8 to 1.0 of the speed and level 9 at about 0.8, within a third of a point of the size. Level 6 looks at 12 candidates where libzstd's looks at 8, which keeps it at or below gzip 6 on all three files (libzstd 6 is slightly larger than gzip 6 on the C source). Level 4 is not libzstd's method at that level: smaller by about a point, at about half the speed. On inputs of megabytes libzstd's higher levels also use a larger window and tables, and are 0.3 to 0.5 points smaller there.

The table was measured before v0.89.0. Since then, on an x86 processor with BMI2 (shifts by a register without the flags), the encoder runs a second build of its block coder compiled for it - the same C, the same output, chosen when the processor is first asked: on four other inputs of 0.4 to 3.9 MB, levels 1 to 3 went from 0.76-0.88 of libzstd's speed to 0.82-0.96.

Memory. `proven_zstd_encode_all` compresses the caller's buffer where it is, with tables and block buffers sized to it: a few kilobytes for a short input. A stream keeps the window in its own buffer, and at most holds:

	Memory
Hash tables (zeroed when an encoder is made or reset)	64 KiB at level 1, 96 KiB at 2, 768 KiB at 3 and 4, 1 MiB at 5 to 9 (less when a whole buffer is short)
One block's work: its sequences, literals and output	about 500 KiB
The input buffer of a stream	the window and half a window, or the window and 128 KiB if that is more: 768 KiB at the default 512 KiB

The window (`window_log`, 10 to 23, zero is 19) is what a decoder must hold to read the frame, so it is the reader's memory too: 23 (8 MiB) is the most RFC 9659 asks an HTTP client to accept. A whole buffer no larger than the window states its own size as the window.

Example: written whole, in a stream with a flush, and the weights of a request

```
#include <string.h>

/*
 * Zstandard, written: a whole buffer, a stream that hands over what it has at a flush, and
 * the choice a server makes between zstd and gzip from a request's Accept-Encoding.
 */

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    static const char line[] = "GET /index.html HTTP/1.1 - 200 - text/html; charset=utf-8\n";
    static proven_byte_t text[40000];
    for (proven_size_t i = 0; i < sizeof text; ++i) text[i] = (proven_byte_t)line[i % (sizeof line - 1)];
    static proven_byte_t frame[48000], back[40000];

    /* A whole buffer. encode_bound is always enough; the frame states the input's size, so a
     * decoder needs no more window than that. */
    proven_zstd_encoder_options_t o = { .alloc = heap, .level = 1 };
    proven_size_t n = 0, got = 0;
    EXAMPLE_REQUIRE(proven_zstd_encode_bound(sizeof text) <= sizeof frame, "the bound for 40,000 bytes fits
the buffer");
    EXAMPLE_REQUIRE(proven_zstd_encode_all(&o, (proven_mem_view_t){ text, sizeof text }, (proven_mem_mut_t)
{ frame, sizeof frame }, &n) == PROVEN_OK && n < 200,
                    "40,000 bytes of repeated log lines compress to fewer than 200");
    EXAMPLE_REQUIRE(proven_zstd_decode_all(heap, (proven_mem_view_t){ frame, n }, (proven_mem_mut_t){ back,
sizeof back }, &got) == PROVEN_OK &&
                    got == sizeof text && memcmp(back, text, got) == 0, "and decode to every byte");

    /* A stream: 1,000 bytes at a time at level 6 into a 64 KiB window, with the checksum. A block flush
     * after the first 5,000 makes them decodable at once - what a server does before it
     * waits for more to send; FINISH ends the frame. */
    proven_zstd_encoder_t *z = NULL;
    o = (proven_zstd_encoder_options_t){ .alloc = heap, .level = 6, .window_log = 16, .checksum = true };
    EXAMPLE_REQUIRE(proven_zstd_encoder_create(&o, &z) == PROVEN_OK, "an encoder at level 6 - about gzip 6's
size - with a 64 KiB window");
    proven_size_t in_at = 0, out_at = 0;
    bool done = false;
    while (!done) {
        const proven_size_t give = sizeof text - in_at < 1000 ? sizeof text - in_at : 1000;
        const proven_zstd_flush_t flush = in_at + give == sizeof text ? PROVEN_ZSTD_FLUSH_FINISH
                                        : in_at + give == 5000 ? PROVEN_ZSTD_FLUSH_BLOCK :
PROVEN_ZSTD_FLUSH_NONE;
        proven_size_t used = 0, made = 0;
        EXAMPLE_REQUIRE(proven_zstd_encode(z, (proven_mem_view_t){ text + in_at, give }, &used,
(proven_mem_mut_t){ frame + out_at, sizeof frame - out_at }, &made, flush, &done) == PROVEN_OK,
                        "a step");
        in_at += used;
        out_at += made;
        if (flush == PROVEN_ZSTD_FLUSH_BLOCK) {
            /* Everything so far, decoded from what has come out so far: the frame is not over. */
            proven_zstd_decoder_t *d = NULL;
            proven_size_t dused = 0, dmade = 0;
            bool dend = true;
            EXAMPLE_REQUIRE(proven_zstd_decoder_create(heap, 0, &d) == PROVEN_OK &&
                            proven_zstd_decode(d, (proven_mem_view_t){ frame, out_at }, &dused,
```

```
(proven_mem_mut_t){ back, sizeof back }, &dmade, &dend) == PROVEN_OK &&
                            dmade == 5000 && !dend, "after the flush, a decoder gives back all 5,000 bytes and
waits for the rest");
            proven_zstd_decoder_destroy(d);
        }
    }
    EXAMPLE_REQUIRE(proven_zstd_decode_all(heap, (proven_mem_view_t){ frame, out_at }, (proven_mem_mut_t)
{ back, sizeof back }, &got) == PROVEN_OK &&
                    got == sizeof text && memcmp(back, text, got) == 0, "the finished frame decodes to the
40,000 bytes, checksum checked");
    proven_zstd_encoder_reset(z);                       /* the same encoder, a new frame */
    proven_size_t used = 0, made = 0;
    EXAMPLE_REQUIRE(proven_zstd_encode(z, (proven_mem_view_t){ text, 300 }, &used, (proven_mem_mut_t){ frame,
sizeof frame }, &made, PROVEN_ZSTD_FLUSH_FINISH, &done) == PROVEN_OK &&
                    done && used == 300, "after a reset, a second frame");
    proven_zstd_encoder_destroy(z);

    /* Which coding a request wants most: weights in thousandths. curl and browsers send zstd
     * and gzip at one weight; a server that offers zstd takes it then (compress_zstd). */
    const proven_http_header_t asks[] = { { PROVEN_LIT("Accept-Encoding"), PROVEN_LIT("gzip, deflate, br,
zstd") } };
    const proven_http_header_t prefers_gzip[] = { { PROVEN_LIT("Accept-Encoding"), PROVEN_LIT("gzip;q=1.0,
zstd;q=0.5") } };
    EXAMPLE_REQUIRE(proven_http_accept_weight(asks, 1, PROVEN_HTTP_CODING_ZSTD) == 1000 &&
proven_http_accept_weight(asks, 1, PROVEN_HTTP_CODING_GZIP) == 1000,
                    "a browser's list: zstd and gzip both 1000");
    EXAMPLE_REQUIRE(proven_http_accept_weight(prefers_gzip, 1, PROVEN_HTTP_CODING_ZSTD) == 500 &&
proven_http_accept_weight(prefers_gzip, 1, PROVEN_HTTP_CODING_GZIP) == 1000,
                    "a client that prefers gzip: zstd 500, gzip 1000");
    return EXAMPLE_OK();
}
```

Reference: the encoder

API	Intent	Return
`proven_zstd_encoder_create(&options, &z)`	An encoder: `level` 1 to 9 (zero is 3, libzstd's default), `window_log` 10 to 23 (zero is 19), `checksum`. Allocates its hash tables; its buffers come with input.	`INVALID_ARG`, `NOMEM`.
`proven_zstd_encode(z, in, &consumed, out, &produced, flush, &done)`	Take some of `in`, give some of `out`. With a flush, call again until all input is taken and nothing more comes (for FINISH, until `done`).	`INVALID_STATE` for input after FINISH (reset first); `NOMEM`, after which every call says so until a reset; `INVALID_ARG`.
`proven_zstd_encoder_reset(z)`	Start a new frame with the same options; the buffers are kept.	none.
`proven_zstd_encoder_destroy(z)`	Free it. NULL is allowed.	none.
`proven_zstd_encode_bound(n)`	The most a FINISH of `n` bytes, or `encode_all`, can write.	a size; the largest size when the bound does not fit.
`proven_zstd_encode_all(&options, in, out, &written)`	One frame of `in`, stating its size, in `out`.	`OUT_OF_BOUNDS` when `out` is too small.

Cautions when writing

A stream does not state its size, so a decoder must be ready for its whole window: choose `window_log` for the reader, not only for the ratio. The checksum is off unless asked for: without it a damaged frame can decode to wrong bytes. A block flush costs a block header and the tables for a short block - flush where a reader is waiting, not after every write. Compressing is no more secret than gzip: section 6's caution about secrets next to text the client chose holds for `zstd` as it does for gzip.

8. Brotli

```
#include "proven/brotli.h"

proven_err_t proven_brotli_decoder_create(proven_allocator_t alloc, proven_u64 window_max,
proven_brotli_decoder_t **out);
void proven_brotli_decoder_destroy(proven_brotli_decoder_t *b);
void proven_brotli_decoder_reset(proven_brotli_decoder_t *b);
proven_err_t proven_brotli_decode(proven_brotli_decoder_t *b, proven_mem_view_t in, proven_size_t *consumed,
                                  proven_mem_mut_t out, proven_size_t *produced, bool *done);
proven_err_t proven_brotli_decode_all(proven_allocator_t alloc, proven_mem_view_t in, proven_mem_mut_t out,
proven_size_t *written);
```

Brotli (RFC 7932) is HTTP's `Content-Encoding: br`. Servers mostly use it for files they compress once in advance at its highest quality, where it comes out smaller than gzip and Zstandard; compressing at that quality is slow, and decoding is fast whatever the quality. This library reads it, since v0.60.0, and does not write it: a server here sends gzip or Zstandard.

The decoder works as section 7's does - a state, and a step that takes what input there is and fills what output space there is, stopping anywhere: between two bits of a code if that is where your input ends, in the middle of a copy if that is where your space ends. Two things are the same as there:

- **A step may allocate.** The history that later bytes refer to grows with the stream's output, never past the window the stream states; and each meta-block (Brotli's unit, up to 16 MiB of output) brings its own prefix codes, built into a table that grows with how many there are. Both are kept for the next stream. So `proven_brotli_decode` can return `PROVEN_ERR_NOMEM`.
- **Output can wait in the decoder**, in that history, while the space you give is too small.

The window is the memory, and the limit is yours. A stream's first bits say how far back it may refer: 2^WBITS - 16 bytes, for WBITS from 10 to 24, so 16 MiB at most. `window_max` is the most you agree to: 0 means 16 MiB, which takes every stream RFC 7932 allows. A stream that needs more is `PROVEN_ERR_OUT_OF_BOUNDS` before anything is allocated for it. libbrotli's "large window" variant, which is not RFC 7932, is refused as malformed.

	Memory
A decoder, made	about 22 KiB
While decoding a stream	its history - as much of the window as its output has reached, 2^WBITS bytes at most, and 32 more - and its codes: tens of KiB for what encoders write; 1,343,024 bytes for the most a meta-block can have (256 codes of each of its three kinds, each as large as a code's table can be)

One stream, and nothing after it. Brotli streams do not follow each other the way gzip members and Zstandard frames do. `*done` becomes true where the stream ends, that call stops there - what it consumed is the stream - and later calls consume nothing more; reset to read another. `proven_brotli_decode_all` refuses bytes after the end, as do the HTTP clients.

The static dictionary. Brotli streams may copy words from a dictionary that every decoder carries: 122,784 bytes of words in many languages, and 121 ways of changing each (a space before, a capital first letter, a suffix such as `ing `). It is most of what this decoder adds to a program: with it, the library is 162,351 bytes larger (998,985 bytes of code and data against 836,634, at `-O2` on x86-64). A build with `-DPROVEN_NO_BROTLI` leaves the decoder and the dictionary out; `proven.h` then does not include `brotli.h`, and an HTTP client asked to decode Brotli is `PROVEN_ERR_UNSUPPORTED`.

Example: a buffer, a stream with something after it, and a window too large

```
#include <string.h>

/*
 * Brotli: a whole buffer, then the same stream read piece by piece with other bytes after it,
 * and the window limit that keeps a stranger's stream from deciding how much memory this
 * program spends.
 */

/* 4,000 bytes of log lines, as libbrotli wrote them at quality 11 with a 64 KiB window: 58 bytes. */
static const proven_byte_t STREAM[] = {
    0xe2, 0xf3, 0x01, 0xb1, 0xe3, 0x30, 0x76, 0xb3, 0x54, 0x22, 0x7f, 0x6e,
    0x31, 0x35, 0x23, 0x54, 0x7a, 0x83, 0xbc, 0xb8, 0x1c, 0x8d, 0x96, 0x8c,
    0x29, 0xa2, 0x82, 0x0d, 0x38, 0x60, 0x0f, 0x30, 0xc2, 0x45, 0x17, 0x5a,
    0x8c, 0xce, 0x18, 0x20, 0xa3, 0x8c, 0xc1, 0x41, 0x9a, 0xb7, 0x0a, 0x2d,
    0x23, 0xe3, 0x31, 0x41, 0x6b, 0xd0, 0x07, 0x39, 0x00, 0x40,
};

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    static const char line[] = "GET /index.html HTTP/1.1 - 200 - text/html; charset=utf-8\n";
    static proven_byte_t text[4000];
    for (proven_size_t i = 0; i < sizeof text; ++i) text[i] = (proven_byte_t)line[i % (sizeof line - 1)];

    /* A whole buffer: as with gzip, the output buffer is the limit. */
    static proven_byte_t out[8000];
    proven_size_t n = 0;
    EXAMPLE_REQUIRE(proven_brotli_decode_all(heap, (proven_mem_view_t){ STREAM, sizeof STREAM },
(proven_mem_mut_t){ out, sizeof out }, &n) == PROVEN_OK &&
                    n == sizeof text && memcmp(out, text, n) == 0, "58 bytes of Brotli decode to the 4,000
bytes libbrotli was given");
    EXAMPLE_REQUIRE(proven_brotli_decode_all(heap, (proven_mem_view_t){ STREAM, sizeof STREAM },
(proven_mem_mut_t){ out, 3999 }, &n) == PROVEN_ERR_OUT_OF_BOUNDS,
                    "and do not fit in 3,999: PROVEN_ERR_OUT_OF_BOUNDS");

    /* A stream with five other bytes after it, given 7 bytes at a time with 500 bytes of room.
     * `done` comes where the stream ends, and the bytes after it are not consumed: a Brotli
     * stream has no frames to follow it, so what they are is the caller's business. */
    static proven_byte_t more[sizeof STREAM + 5];
    memcpy(more, STREAM, sizeof STREAM);
    memcpy(more + sizeof STREAM, "after", 5);
    proven_brotli_decoder_t *b = NULL;
    EXAMPLE_REQUIRE(proven_brotli_decoder_create(heap, 0, &b) == PROVEN_OK, "a decoder with the default window
limit, 16 MiB");
    proven_size_t in_at = 0, out_at = 0;
    bool done = false;
    while (!done) {
        proven_size_t give = sizeof more - in_at < 7 ? sizeof more - in_at : 7, room = sizeof out - out_at <
500 ? sizeof out - out_at : 500;
        proven_size_t used = 0, made = 0;
        EXAMPLE_REQUIRE(proven_brotli_decode(b, (proven_mem_view_t){ more + in_at, give }, &used,
(proven_mem_mut_t){ out + out_at, room }, &made, &done) == PROVEN_OK, "a step");
```

```
        in_at += used;
        out_at += made;
        if (used == 0 && made == 0 && !done) break;     /* nothing more to give, or nowhere to put it */
    }
    EXAMPLE_REQUIRE(done && in_at == sizeof STREAM && out_at == sizeof text && memcmp(out, text, out_at) == 0,
                    "done after the 58 bytes of the stream, with the five after it left alone");

    /* The window limit. The first bits of a stream say how far back it may refer, and a
     * decoder must keep that much: this one, made with libbrotli's largest window, asks for
     * 16 MiB. A decoder limited to 64 KiB refuses it before allocating anything for it; reset
     * makes the decoder usable again. */
    static const proven_byte_t wide[] = { 0x0f, 0x01, 0x80, 0x61, 0x62, 0x63, 0x03 };
    proven_brotli_decoder_t *small = NULL;
    proven_size_t used = 0, made = 0;
    EXAMPLE_REQUIRE(proven_brotli_decoder_create(heap, 64 * 1024, &small) == PROVEN_OK, "a decoder limited to
64 KiB");
    EXAMPLE_REQUIRE(proven_brotli_decode(small, (proven_mem_view_t){ wide, sizeof wide }, &used,
(proven_mem_mut_t){ out, sizeof out }, &made, &done) == PROVEN_ERR_OUT_OF_BOUNDS,
                    "a stream with a 16 MiB window: PROVEN_ERR_OUT_OF_BOUNDS");
    proven_brotli_decoder_reset(small);
    EXAMPLE_REQUIRE(proven_brotli_decode(small, (proven_mem_view_t){ STREAM, sizeof STREAM }, &used,
(proven_mem_mut_t){ out, sizeof out }, &made, &done) == PROVEN_OK &&
                    done && made == sizeof text, "after a reset, a stream within the limit decodes");
    proven_brotli_decoder_reset(b);
    EXAMPLE_REQUIRE(proven_brotli_decode(b, (proven_mem_view_t){ wide, sizeof wide }, &used,
(proven_mem_mut_t){ out, sizeof out }, &made, &done) == PROVEN_OK &&
                    done && made == 3 && memcmp(out, "abc", 3) == 0, "while the default limit takes it - with
a few KiB of history, not 16 MiB");

    proven_brotli_decoder_destroy(small);
    proven_brotli_decoder_destroy(b);
    return EXAMPLE_OK();
}
```

Reference

API	Intent	Return
`proven_brotli_decoder_create(alloc, window_max, &b)`	A decoder. `window_max` from 1 KiB to 2 GiB; 0 is 16 MiB.	`INVALID_ARG`, `NOMEM`.
`proven_brotli_decode(b, in, &consumed, out, &produced, &done)`	Take some of `in`, fill some of `out`; stop at the stream's end. Only the bytes reported produced are written.	`INVALID_FORMAT` (not Brotli, or damaged), `OUT_OF_BOUNDS` (window above the limit), `NOMEM`, `INVALID_ARG`; after an error, the same until a reset.
`proven_brotli_decoder_reset(b)`	Forget the stream in progress; keep the buffers.	none.
`proven_brotli_decoder_destroy(b)`	Free it. NULL is allowed.	none.
`proven_brotli_decode_all(alloc, in, out, &written)`	The one stream of `in` into `out`, with the default limit. `out`'s size is the limit on output.	`OUT_OF_BOUNDS` when it does not fit; `INVALID_FORMAT` also when `in` is empty, ends inside the stream or goes on after it.

Cautions, and what goes wrong

- **The window decides the memory**, as in section 7: whoever wrote the stream chooses it, up to your limit. A thousand responses at once, each with a 16 MiB window and long enough to fill it,

are 16 GiB. Lower `window_max` where you know the streams (files you made with a known window), and count decoders the way you count connections.
- **There is no checksum.** Damage that still forms a valid stream decodes to other bytes; the decoder can only refuse what does not parse. Where it matters, check what you decoded.
- **Not recognisable by its first bytes.** Brotli has no magic number: a decoder given gzip, or text, fails somewhere inside rather than at once, and very short inputs of other kinds can happen to be valid streams. Over HTTP, `Content-Encoding` says what a body is; for files, keep the name (`.br`) or a header of your own.

Nothing that libbrotli refuses is accepted. Outside the registered tests, 1,437,490 truncated and changed copies of 176 streams were given to both this decoder and libbrotli 1.1.0's one-call decoder: none that libbrotli refused was accepted here, none that both accepted came out different, and 11,242 that it accepted were refused here - each a stream that ends before the input does, with the same output, whose remaining bytes libbrotli's one-call decoder ignores and `proven_brotli_decode_all` (like Python's binding of libbrotli) does not.

Speed. On the development machine, the C sources of this library (2.9 MiB) compressed by libbrotli at qualities 1, 5 and 11 decode at about 195, 265 and 290 MiB/s - 0.65 to 0.73 of libbrotli's own decoder in the same process (about 270, 410 and 435 MiB/s; v0.87.0, from 0.57 to 0.65 before); an uncompressed meta-block or a long run is copied at libbrotli's speed.

9. What is accepted, what is not here, and how this was tested

What the decompressor accepts is what zlib accepts. Where RFC 1951 leaves a corner open, each case was tried on zlib first and its verdict followed, because a real producer may emit what zlib tolerates. So: a block that uses a single code is accepted, as is a length written with the longest form of the code before the last; codes that describe more symbols than can exist, lengths and distances outside the format, a distance reaching back before the first byte, and a stream whose code tables do not add up are all refused. A gzip header may carry an extra field, a file name, a comment and its own checksum, all of which are read past (the checksum is checked); the name and the time are not reported.

What the compressor writes is valid DEFLATE that any decompressor reads. It is not, byte for byte, what zlib would have written for the same input, and it does not need to be.

Not here:

- **Preset dictionaries** (zlib's `FDICT`), in either direction.
- **ZIP archives, tar files**: those are containers. This is the codec inside them.
- **Deflate64**, and every other compression format: LZ4, bzip2, xz. Zstandard's slower levels (10 to 22 in libzstd: binary trees and optimal parsing), its dictionaries and its long-distance mode. Brotli compression; Brotli's large-window variant and shared dictionaries.
- **The gzip header's fields** - name, time, comment - cannot be set or read.
- **Several gzip members read as one stream** automatically: reset after each (section 3). (The HTTP clients of section 6 do this for a response body.)
- **Over HTTP:** compressed *request* bodies, in either direction; sending `br`; a call that pushes out what a streamed response has written so far; serving a precompressed file (a handler can do that itself: `proven_http_accepts_coding`, then its own `Content-Encoding`).

How this was tested. The registered test decompresses 93 streams that zlib 1.3.1 made - every level and strategy, small windows, flushes in the middle, all three formats - a byte at a time and in one call, and gets the data zlib was given; accepts ten hand-built streams that are unusual and that zlib accepts; refuses thirty hand-built streams that zlib refuses, one for each way a stream can be wrong; gives "more input, please" for every proper prefix of the short streams and an answer of some kind - never a write outside its buffer - for every single changed bit of them; and compresses

eleven kinds of data at every level and three window sizes and gets it back. Outside the registered tests: zlib decompressed everything this compressor produced for a larger corpus, at every level, window and format, with flushes and in small pieces (2,856 streams); this decompressor was compared with zlib on 5,838 more; and six million randomly damaged streams were decompressed under tools that report any read or write outside a buffer, with none reported and no stream that made the decoder do work without progress.

Section 6 was tested against bodies zlib made - gzip, zlib, raw DEFLATE, two members, every optional header field, and each way of being cut short or damaged - through both clients, whole and a byte a chunk, each checked against zlib's own verdict when the test's table was generated; and with both servers compressing at seven sizes, in one piece and streamed, plain and over TLS. Outside the registered tests, curl and Python read what both servers send, and both clients read what a Python server sends.

Section 7 was tested against libzstd 1.5.7, by way of Node: the registered test decodes 91 streams libzstd made from seven kinds of data - levels 1, 3, 9 and 19, with and without the checksum and the content size, matches reaching across blocks, a frame longer than its window - in one call, a byte at a time and in odd steps, and gets the data libzstd was given; accepts thirteen hand-built frames that are unusual and that libzstd accepts (among them every kind of repeat offset), and refuses twenty, with their reasons (damage, a dictionary, a window above 8 MiB, and the empty compressed block above); refuses every proper prefix of the short streams; gives the same verdict, whole and in steps, for every single changed bit of them; and shows the window limit refusing before any allocation, the memory held, and every failed allocation answered with `PROVEN_ERR_NOMEM`. Outside the registered tests: 130 more streams libzstd made - up to 2.2 MiB, several frames, flushed streams - decoded byte for byte in steps from 1 byte to 70,000; and about 70,000 truncations and changed bits of seven of them, under tools that report any read or write outside a buffer, with none reported. Over HTTP, the blocking, event-driven and HTTP/2 clients decoded what Node's servers sent in `zstd` (levels 1, 3 and 19, whole, flushed and in several frames, up to 3 MB), refused it cut short or followed by junk, and a client that did not ask was sent gzip; curl read a handler's own Zstandard from this library's server.

Section 8 was tested against libbrotli, by way of Python's binding (1.1.0): the registered test decodes 110 streams libbrotli made from ten kinds of data - qualities 0, 1, 5, 9 and 11, windows from 1 KiB to 16 MiB, text and generic modes - in one call, a byte at a time and in odd steps, and gets the data libbrotli was given; decodes 66 hand-built streams that libbrotli decodes, with the same result - metadata, uncompressed meta-blocks, every window size, dictionary words under all 121 transforms (capitals of two- and three-byte letters among them), every context mode, context maps with runs and move-to-front, every short distance code, block switches of each kind, every way of writing a prefix code - and refuses 25 that libbrotli refuses, each wrong in a different way; refuses every proper prefix of the short streams; gives the same verdict, whole and in steps, for every single changed bit of them (72,376); and shows the window limit refusing before any allocation, the memory held, and every failed allocation answered with `PROVEN_ERR_NOMEM`. The dictionary and its transforms were taken from the text of RFC 7932 and checked against the CRC-32s it gives and against libbrotli's own copy. Outside the registered tests: 358 more streams from Python's libbrotli and 144 from Node's (a copy of its own, 1.2.0) - this repository's files, the Korean manual, 2.2 MB of sources - decoded byte for byte in steps from 1 byte to 70,000, under the tools that report any read or write outside a buffer; the 1,437,490 damaged copies above; and the largest code table each alphabet can make, worked out and then built. Over HTTP, the blocking, event-driven and HTTP/2 clients decoded what Node's servers sent in `br` - qualities 1, 5 and 11, windows of 1 KiB to 16 MiB, whole and flushed, up to 3 MB - refused it cut short, followed by junk or twice, and a client that did not ask was sent gzip.

The encoder of section 7 was tested against its decoder and against libzstd. The registered test compresses eleven kinds of data (nothing, one byte, a run, numbered lines, noise with copies from far back, random bytes, more than 128 byte values, and more) at all nine levels and three windows, whole and - at a level of each kind - streamed with a block flush after each piece and output a byte at a time, and decodes every frame with this library's decoder, which checks each offset against the window the frame states; it checks that a stream's blocks are byte for byte those of the same input compressed whole at every level, the frame header, the bound over 8,000 inputs, flushes, the end and a reset, literals of one byte value in a compressed block, the memory held, and failed allocations, and pins the size and CRC of thirteen frames that libzstd decoded - five of them streamed after a reset with the buffer sliding. Outside the registered tests, libzstd decoded every frame the encoder made from a corpus - this repository's files, the Korean manual, a binary, 4 MB of sources, inputs of every size to 4,096 bytes and at every block and header boundary, literals of one byte, two and four values, periods that fill a block's sequences - at every level and at windows from 1 KiB to 8 MiB, whole and streamed in pieces from one byte, under the tools that report any read or write outside a buffer: 4,105 frames, every one byte for byte. Over HTTP, Node's libzstd and curl decoded what both servers sent in `zstd`, whole and streamed, at levels 1, 2, 3, 6 and 9 and windows of 1 KiB to 8 MiB, and the weights chose gzip where a client preferred it.

Not done: no coverage-guided fuzzing; no comparison with a second compressor's output besides zlib's; no external review. For Zstandard: no frames from other encoders than libzstd; no browser; no decoder but libzstd and this library's for the encoder's frames. For Brotli: no streams from encoders other than libbrotli's two copies; no browser.

Chapter 17: HTTP/2 - frames and header compression

Part V - Talking to the operating system. Prerequisites: Chapter 10 for what a request and a response are; Chapter 2 for the allocator the compression tables are made with. After this chapter you can say what HTTP/2 changes and what it does not, encode and decode header blocks, read and write frames, tell a well-formed request from one that must not be passed on, drive a connection - streams, windows, resets and goodbyes - over bytes you carry yourself, turn HTTP/2 - and HTTP/3 - on in the event-driven server of Chapter 15 without changing a handler, and say which of these this library does today.

This chapter covers `hpack.h` and `h2.h`. Both are pure computation, available in a freestanding build: HPACK allocates one table for an encoder and one for a decoder, the frame functions allocate nothing, and a connection allocates what its configuration bounds.

What is here, from the bottom up. Sections 2 to 4 are the codec: what travels on an HTTP/2 connection, read and written. Section 5 is the connection itself - which streams are open, how much may be sent, what to answer - as an object that is given bytes and gives bytes back. Sections 6 and 7 are that object at work: the event-driven server speaks HTTP/2 when asked to, beside HTTP/1.1 on the same port, and the event-driven client keeps an HTTP/2 connection for each origin that agrees to one. The blocking server and the blocking client of Chapter 11 speak HTTP/1.1 only: section 8 says exactly where the line is. Section 9 is the same server speaking HTTP/3, over QUIC on UDP, to the same handlers; section 10 is the client speaking it when a request asks.

Table of contents

1. What HTTP/2 changes, in one page
2. Header compression
3. Frames
4. A header list as a message
5. A connection
6. HTTP/2 in the event-driven server
7. HTTP/2 in the event-driven client
8. What is accepted, what is not here, and how this was tested
9. HTTP/3 in the event-driven server
10. HTTP/3 in the event-driven client

1. What HTTP/2 changes, in one page

HTTP/2 (RFC 9113) carries the same requests and responses as HTTP/1.1 - the same methods, status codes and header fields - in another shape. Three things are different:

- **Everything is a frame.** Nine bytes of header - a length, a type, flags, a stream number - and a payload. There is no text to parse and no line to end: a message is HEADERS frames and DATA frames.
- **Many exchanges share one connection.** Each request and its response are a *stream* with a number; frames of different streams are interleaved, so a slow response does not hold up the ones behind it. Each stream, and the connection as a whole, has a *window*: how many bytes of DATA the other side has said it will take.
- **Header fields are compressed**, by a scheme made for them (HPACK, RFC 7541), because the same fields are sent again with every request.

What HTTP/1.1 said in its first line is said by *pseudo-header fields*, whose names begin with a colon: `:method`, `:scheme`, `:authority` and `:path` for a request, `:status` for a response. There is no `Host` (it is `:authority`), no `Connection`, no `Transfer-Encoding`: framing does what they did.

A connection begins with 24 fixed bytes from the client (`PROVEN_H2_PREFACE`), then a SETTINGS frame from each side.

2. Header compression

```
proven_err_t proven_hpack_encoder_create(proven_allocator_t alloc, proven_size_t max_table_bytes,
proven_hpack_encoder_t **out);
proven_err_t proven_hpack_encode(proven_hpack_encoder_t *encoder, const proven_http_header_t *headers,
proven_size_t count,
                                 const bool *sensitive, proven_mem_mut_t out, proven_size_t *len);

proven_err_t proven_hpack_decoder_create(proven_allocator_t alloc, proven_size_t max_table_bytes,
proven_hpack_decoder_t **out);
proven_err_t proven_hpack_decode(proven_hpack_decoder_t *decoder, proven_mem_view_t block,
                                 proven_http_header_t *headers, proven_size_t header_cap,
                                 proven_mem_mut_t scratch, proven_size_t max_list_bytes, proven_size_t
*count);
```

HPACK sends a field in one of three ways: as a number that names an entry of a fixed table of 61 common fields (`:method: GET` is entry 2); as a number that names an entry of the *dynamic table*, which holds fields this connection sent before; or as a literal - which may be added to the dynamic table as it goes, and whose text may be Huffman-coded. A request that is 65 bytes the first time is 22 the second.

The two ends must keep the same table. Every block is decoded, in the order it was sent, by the one decoder of its connection and direction. Skip a block, or decode one twice, and every block after it means something else. That is why a decoder that meets a malformed block refuses everything afterwards, and why HTTP/2 ends the connection over it.

The table's size is yours to set, and it is a ceiling, not a cost: 4,096 bytes by default (`PROVEN_HPACK_DEFAULT_TABLE_BYTES`). The memory behind it grows as entries are added

- a few hundred bytes for a peer that sends a few short fields - to about one and a half

times the size when the table is full. An encoder made with 0 keeps no table and is stateless; a decoder made with 0 tells the peer to keep none.

Example: a request sent twice, a secret kept out, and a limit

```
#include <stdio.h>
#include <string.h>

/*
 * HPACK: how HTTP/2 sends header fields.
 *
 * An encoder turns a list of fields into a block of bytes; the decoder at the other end turns
 * the block back into the list. Both keep a table of fields that went before, and the second
 * time a field is sent it is one byte. The two tables must stay the same - which is why a
 * connection has exactly one encoder and one decoder in each direction, used in order.
 */

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    proven_hpack_encoder_t *encoder = NULL;
    proven_hpack_decoder_t *decoder = NULL;
    /* 4096 bytes of table is the protocol's default; each side may agree on another size. */
    EXAMPLE_REQUIRE(proven_hpack_encoder_create(heap, PROVEN_HPACK_DEFAULT_TABLE_BYTES, &encoder) == PROVEN_OK
&&
                    proven_hpack_decoder_create(heap, PROVEN_HPACK_DEFAULT_TABLE_BYTES, &decoder) ==
PROVEN_OK, "an encoder here, a decoder there");

    // ---- a request, twice ------------------------------------------------------------
```

```
    /* Names are lowercase in HTTP/2, and what HTTP/1 put in its first line is here as fields
     * whose names begin with a colon. */
    proven_http_header_t request[] = {
        { PROVEN_LIT(":method"), PROVEN_LIT("GET") },
        { PROVEN_LIT(":scheme"), PROVEN_LIT("https") },
        { PROVEN_LIT(":path"), PROVEN_LIT("/reports/2026/october") },
        { PROVEN_LIT(":authority"), PROVEN_LIT("www.example.com") },
        { PROVEN_LIT("user-agent"), PROVEN_LIT("example-client/1.0") },
        { PROVEN_LIT("authorization"), PROVEN_LIT("Bearer 5f2c1e9a7b3d") },
    };
    proven_byte_t block[512];
    EXAMPLE_REQUIRE(proven_hpack_encode_bound(request, 6) <= sizeof block, "the block will fit");
    proven_size_t first = 0;
    EXAMPLE_REQUIRE(proven_hpack_encode(encoder, request, 6, NULL, (proven_mem_mut_t){ block, sizeof block },
&first) == PROVEN_OK, "the request, encoded");

    /* The decoder gives the fields back as views: into the block, into the fixed table, or
     * into the scratch you lend it. */
    proven_http_header_t fields[16];
    proven_byte_t scratch[1024];
    proven_size_t count = 0;
    EXAMPLE_REQUIRE(proven_hpack_decode(decoder, (proven_mem_view_t){ block, first }, fields, 16,
(proven_mem_mut_t){ scratch, sizeof scratch }, 4096, &count) == PROVEN_OK && count == 6,
                    "and decoded");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(fields[2].name, PROVEN_LIT(":path")) &&
proven_u8str_view_eq(fields[2].value, PROVEN_LIT("/reports/2026/october")), "field for field");

    /* The same request again costs a fraction: both sides remember. */
    proven_size_t second = 0;
    EXAMPLE_REQUIRE(proven_hpack_encode(encoder, request, 6, NULL, (proven_mem_mut_t){ block, sizeof block },
&second) == PROVEN_OK, "the same request again");
    printf("the first time: %u bytes; the second: %u\n", (unsigned)first, (unsigned)second);
    EXAMPLE_REQUIRE(second < first / 2, "less than half");
    EXAMPLE_REQUIRE(proven_hpack_decode(decoder, (proven_mem_view_t){ block, second }, fields, 16,
(proven_mem_mut_t){ scratch, sizeof scratch }, 4096, &count) == PROVEN_OK && count == 6 &&
                    proven_u8str_view_eq(fields[4].value, PROVEN_LIT("example-client/1.0")), "and the decoder
still gives the whole list");

    // ---- what is not remembered ------------------------------------------------------
    /* A table is a compression context, and a secret in one can be guessed from the size of
     * later blocks. So the authorization value was never put in it - by either side: */
    proven_size_t entries = proven_hpack_decoder_table_count(decoder);
    bool secret_kept = false;
    for (proven_size_t i = 0; i < entries; ++i) {
        proven_u8str_view_t name, value;
        EXAMPLE_REQUIRE(proven_hpack_decoder_table_entry(decoder, i, &name, &value), "an entry of the table");
        if (proven_u8str_view_eq(name, PROVEN_LIT("authorization"))) secret_kept = true;
    }
    printf("the table holds %u entries, %u bytes by the protocol's count\n", (unsigned)entries,
(unsigned)proven_hpack_decoder_table_bytes(decoder));
    EXAMPLE_REQUIRE(entries == 3 && !secret_kept, "the path, the authority and the user agent - not the
secret");
    /* Any field can be given the same treatment. */
    proven_http_header_t more[] = { { PROVEN_LIT("x-api-key"), PROVEN_LIT("k-0123456789") } };
    bool sensitive[] = { true };
    bool marked[16] = { false };
    proven_size_t n = 0;
    /* The decoder can say which fields came marked - what something that passes them on needs. */
    EXAMPLE_REQUIRE(proven_hpack_encode(encoder, more, 1, sensitive, (proven_mem_mut_t){ block, sizeof
block }, &n) == PROVEN_OK &&
```

```
                    proven_hpack_decode_marked(decoder, (proven_mem_view_t){ block, n }, fields, marked, 16,
(proven_mem_mut_t){ scratch, sizeof scratch }, 4096, &count) == PROVEN_OK &&
                    proven_hpack_decoder_table_count(decoder) == 3 && marked[0], "a field marked sensitive
adds nothing to the table, and arrives marked");

    // ---- a limit, and what it is for ----------------------------------------------------
    /* A block is not the size of what it says. This one is five bytes and names the same
     * remembered field five times: */
    static const proven_byte_t repeat[] = { 0xbe, 0xbe, 0xbe, 0xbe, 0xbe };       /* "entry 62" - the newest
in the table */
    EXAMPLE_REQUIRE(proven_hpack_decode(decoder, (proven_mem_view_t){ repeat, sizeof repeat }, fields, 16,
(proven_mem_mut_t){ scratch, sizeof scratch }, 4096, &count) == PROVEN_OK && count == 5,
                    "five bytes, five fields");
    /* So the limit is on the decoded list - names, values, and 32 a field - not on the block.
     * Over it, the fields are withheld, but the decoder has read the block to its end and is
     * still usable: refuse the one request, keep the connection. */
    EXAMPLE_REQUIRE(proven_hpack_decode(decoder, (proven_mem_view_t){ repeat, sizeof repeat }, fields, 16,
(proven_mem_mut_t){ scratch, sizeof scratch }, 100, &count) == PROVEN_ERR_OUT_OF_BOUNDS && count == 0,
                    "a list limit of 100 bytes refuses it");
    EXAMPLE_REQUIRE(proven_hpack_decode(decoder, (proven_mem_view_t){ repeat, 1 }, fields, 16,
(proven_mem_mut_t){ scratch, sizeof scratch }, 100, &count) == PROVEN_OK && count == 1, "and the next block is
decoded as if nothing had happened");

    /* A block that is not HPACK at all is another matter: the tables can no longer be trusted
     * to match, and the decoder says so from then on. In HTTP/2 that ends the connection. */
    static const proven_byte_t nonsense[] = { 0x80 };                             /* "entry 0": there is none
*/
    EXAMPLE_REQUIRE(proven_hpack_decode(decoder, (proven_mem_view_t){ nonsense, 1 }, fields, 16,
(proven_mem_mut_t){ scratch, sizeof scratch }, 4096, &count) == PROVEN_ERR_INVALID_FORMAT &&
                    proven_hpack_decode(decoder, (proven_mem_view_t){ repeat, 1 }, fields, 16,
(proven_mem_mut_t){ scratch, sizeof scratch }, 4096, &count) == PROVEN_ERR_INVALID_FORMAT,
                    "a malformed block is refused, and so is everything after it");

    // ---- the table's size is negotiated -------------------------------------------------
    /* When the peer announces a smaller table, the encoder is told, and says at the start of
     * its next block that it followed. A decoder whose own limit was lowered insists on that. */
    proven_hpack_decoder_destroy(decoder);
    EXAMPLE_REQUIRE(proven_hpack_decoder_create(heap, PROVEN_HPACK_DEFAULT_TABLE_BYTES, &decoder) ==
PROVEN_OK, "a fresh decoder");
    proven_hpack_decoder_set_max_table_bytes(decoder, 0);          /* "this side will keep no table" ... */
    proven_hpack_encoder_set_peer_table_bytes(encoder, 0);         /* ... heard by the encoder */
    n = 0;
    EXAMPLE_REQUIRE(proven_hpack_encode(encoder, request, 6, NULL, (proven_mem_mut_t){ block, sizeof block },
&n) == PROVEN_OK && block[0] == 0x20, "the next block opens by saying: table size 0");
    EXAMPLE_REQUIRE(proven_hpack_decode(decoder, (proven_mem_view_t){ block, n }, fields, 16,
(proven_mem_mut_t){ scratch, sizeof scratch }, 4096, &count) == PROVEN_OK && count == 6 &&
                    proven_hpack_decoder_table_count(decoder) == 0, "and nothing is remembered on either
side");

    // ---- Huffman coding, by itself ------------------------------------------------------
    /* Literal strings are sent in a code that gives common characters fewer bits. */
    proven_u8str_view_t text = PROVEN_LIT("www.example.com");
    proven_byte_t coded[32], back[32];
    proven_size_t coded_len = 0, back_len = 0;
    EXAMPLE_REQUIRE(proven_hpack_huffman_size(text) == 12 && proven_hpack_huffman_encode(text,
(proven_mem_mut_t){ coded, sizeof coded }, &coded_len) == PROVEN_OK && coded_len == 12, "fifteen characters in
twelve bytes");
    EXAMPLE_REQUIRE(proven_hpack_huffman_decode((proven_mem_view_t){ coded, coded_len }, (proven_mem_mut_t)
{ back, sizeof back }, &back_len) == PROVEN_OK && back_len == 15 && memcmp(back, "www.example.com", 15) == 0,
"and back");
```

```
    proven_hpack_encoder_destroy(encoder);
    proven_hpack_decoder_destroy(decoder);
    return EXAMPLE_OK();
}
```

Reference

API	Intent	Return
`proven_hpack_encoder_create(alloc, max_table_bytes, &enc)`	An encoder that uses at most this much table (0: none).	`proven_err_t`: `INVALID_ARG`, `NOMEM`.
`proven_hpack_encode(enc, headers, count, sensitive, out, &len)`	Append one header block for these fields. `sensitive` is optional.	`proven_err_t`: `OUT_OF_BOUNDS` when `out` has less room than the bound - nothing written, nothing changed.
`proven_hpack_encode_bound(headers, count)`	Room that is always enough.	`proven_size_t`.
`proven_hpack_encoder_set_peer_table_bytes(enc, n)`	The peer's limit changed; announced in the next block.	none.
`proven_hpack_decoder_create(alloc, max_table_bytes, &dec)`	A decoder that allows the peer this much table.	`proven_err_t`.
`proven_hpack_decode(dec, block, headers, cap, scratch, max_list_bytes, &count)`	Decode one whole block.	`proven_err_t`: `OUT_OF_BOUNDS` - too large a list, too many fields, too small a scratch; the decoder is still good. `INVALID_FORMAT` - not HPACK; the decoder is finished.
`proven_hpack_decode_marked(dec, block, headers, never_indexed, cap, scratch, max_list_bytes, &count)`	The same, and `never_indexed[i]` says whether field `i` came in the form that must never be indexed.	as `proven_hpack_decode`.
`proven_hpack_decoder_set_max_table_bytes(dec, n)`	This side's limit changed; if lowered, the peer's next block must say it followed.	none.
`proven_hpack_decoder_table_count, _table_bytes, _table_entry`	Look at the decoder's table.	counts; `bool`.
`proven_hpack_huffman_size, _huffman_encode, _huffman_decode`	The string code alone.	size; `proven_err_t`.
`proven_hpack_encoder_destroy, proven_hpack_decoder_destroy`	Free one. Null is accepted.	none.

Cautions, and what goes wrong

- **A block is not the size of what it says.** Five bytes can name a large remembered field five times; five thousand can name it five thousand times. Always give `max_list_bytes`: it bounds the decoded list - names, values and 32 bytes a field, as the RFC counts - and `scratch` of that size is always enough.
- **Over the limit is not the end of the connection; malformed is.** `OUT_OF_BOUNDS` means the block was read to its end, the table is in step, and only this message is refused. `INVALID_FORMAT` means the tables may now differ, and the decoder repeats it for ever.

- **Views, three kinds.** A decoded name or value points into the block, into the library's fixed table, or into your scratch. All three must outlive your use of the fields; copy what you keep.
- **Secrets stay out of the table.** A table is a compression context, and what is in one can be guessed, byte by byte, from the size of later blocks that an attacker influences. The encoder never remembers the values of `authorization` and `proxy-authorization` and sends them marked so that no hop will; it does not remember `cookie` and `set-cookie`; and `sensitive` extends that to any field you name. Something that passes fields on - a proxy - learns which ones arrived marked from `proven_hpack_decode_marked`, and gives those marks to the encoder as its `sensitive`.
- **What is seldom the same twice stays out too.** `content-length`, `content-range`, `etag`, `age`, `if-none-match` and `if-modified-since` are sent by their name's index and not remembered: each new value would take a place in the table from a field that does repeat. (A response head whose `content-length` differs every time: 26 bytes and 430,000 a second with the value remembered, 13 bytes and 1,020,000 without, on one machine, encoded and decoded.)
- **Names are lowercase before they reach the encoder.** It encodes what it is given; the check that a list is fit to send is section 4's.
- **One encoder, one decoder, one direction, in order.** Not per request, not per stream.

3. Frames

```
proven_err_t proven_h2_frame_parse(proven_mem_view_t data, proven_h2_frame_t *out);
proven_err_t proven_h2_frame_check(const proven_h2_frame_t *frame, proven_u32 max_frame_size,
proven_h2_error_t *code, bool *stream_error);
proven_err_t proven_h2_frame_write(proven_mem_mut_t out, proven_size_t *len, const proven_h2_frame_t *frame);
```

A frame is nine bytes and a payload:

Field	Bits	Meaning
Length	24	Bytes of payload. Every endpoint accepts 16,384; more only if it said so
Type	8	One of ten; anything else is skipped
Flags	8	Meaning depends on the type
Stream	31 (and one reserved bit)	0 is the connection itself; a client opens odd streams

`proven_h2_frame_check` applies, before the payload is looked at, the rule each type has:

Type	Stream	Size	Carries
`DATA`	required	any	A piece of a body. May be padded. `END_STREAM` ends the sender's side
`HEADERS`	required	any	A header block, or its beginning. May be padded; may carry five bytes of priority, which are skipped
`CONTINUATION`	required	any	More of a header block, until `END_HEADERS`
`RST_STREAM`	required	4	An error code: this stream is over
`SETTINGS`	must be 0	a multiple of 6; 0 with `ACK`	Identifier and value pairs
`PING`	must be 0	8	Eight bytes to be sent back with `ACK`
`GOAWAY`	must be 0	at least 8	The last stream acted on, an error code, text for a human
`WINDOW_UPDATE`	either	4	How many more bytes may be sent
`PRIORITY`	required	5	Deprecated; checked and ignored

PUSH_PROMISE	required	any	Server push, which this library always declines

An error has one of two sizes. A *connection error* ends the connection: send GOAWAY with the code, then close. A *stream error* ends one stream: send RST_STREAM and go on. The check reports the code and which it is.

Example: a connection's first bytes, read as a server reads them

```
#include <stdio.h>
#include <string.h>

/*
 * HTTP/2 frames, and the header list inside them - read here from a buffer, as a server
 * reads the first bytes of a connection.
 *
 * Nothing in h2.h holds a connection together. It reads and writes frames, applies the rule
 * each frame type has, and checks that a decoded header list is a request. Which streams are
 * open and how much may be sent is the business of whatever uses these.
 */

/* What a client would send to open a connection and ask for one page. */
static proven_size_t client_bytes(proven_allocator_t heap, proven_byte_t *wire, proven_size_t cap) {
    proven_mem_mut_t out = { wire, cap };
    proven_size_t len = 0;
    memcpy(wire, PROVEN_H2_PREFACE, PROVEN_H2_PREFACE_SIZE);       /* 24 fixed bytes come first */
    len = PROVEN_H2_PREFACE_SIZE;

    /* Then the client's settings: here, "do not push" and a larger window. */
    proven_h2_setting_t settings[] = {
        { PROVEN_H2_SETTINGS_ENABLE_PUSH, 0 },
        { PROVEN_H2_SETTINGS_INITIAL_WINDOW_SIZE, 1048576 },
    };
    if (proven_h2_write_settings(out, &len, settings, 2) != PROVEN_OK) return 0;

    /* A request is a HEADERS frame on a new stream - a client's streams are odd numbers -
     * carrying an HPACK block. END_HEADERS: the block is whole. END_STREAM: no body follows. */
    proven_http_header_t request[] = {
        { PROVEN_LIT(":method"), PROVEN_LIT("GET") },
        { PROVEN_LIT(":scheme"), PROVEN_LIT("https") },
        { PROVEN_LIT(":path"), PROVEN_LIT("/") },
        { PROVEN_LIT(":authority"), PROVEN_LIT("www.example.com") },
        { PROVEN_LIT("accept"), PROVEN_LIT("text/html") },
    };
    proven_hpack_encoder_t *encoder = NULL;
    proven_byte_t block[256];
    proven_size_t block_len = 0;
    if (proven_hpack_encoder_create(heap, PROVEN_HPACK_DEFAULT_TABLE_BYTES, &encoder) != PROVEN_OK) return 0;
    proven_err_t err = proven_hpack_encode(encoder, request, 5, NULL, (proven_mem_mut_t){ block, sizeof
block }, &block_len);
    proven_hpack_encoder_destroy(encoder);
    if (err != PROVEN_OK) return 0;
    proven_h2_frame_t headers = { .length = (proven_u32)block_len, .type = PROVEN_H2_HEADERS,
                                  .flags = PROVEN_H2_FLAG_END_HEADERS | PROVEN_H2_FLAG_END_STREAM, .stream_id
= 1 };
    if (proven_h2_frame_write(out, &len, &headers) != PROVEN_OK) return 0;
    memcpy(wire + len, block, block_len);
    len += block_len;

    /* And, for the example's sake, one frame that breaks a rule: a PING must be eight bytes. */
```

```
    proven_h2_frame_t bad_ping = { .length = 4, .type = PROVEN_H2_PING };
    if (proven_h2_frame_write(out, &len, &bad_ping) != PROVEN_OK) return 0;
    memset(wire + len, 0, 4);
    return len + 4;
}

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    proven_byte_t wire[1024];
    proven_size_t size = client_bytes(heap, wire, sizeof wire);
    EXAMPLE_REQUIRE(size > 0, "the client's bytes");

    // ---- the server's side ------------------------------------------------------
    EXAMPLE_REQUIRE(size >= PROVEN_H2_PREFACE_SIZE && memcmp(wire, PROVEN_H2_PREFACE, PROVEN_H2_PREFACE_SIZE)
== 0, "the preface: this is HTTP/2");
    proven_size_t at = PROVEN_H2_PREFACE_SIZE;

    proven_hpack_decoder_t *decoder = NULL;
    EXAMPLE_REQUIRE(proven_hpack_decoder_create(heap, PROVEN_HPACK_DEFAULT_TABLE_BYTES, &decoder) ==
PROVEN_OK, "the connection's decoder");
    int frames = 0, requests = 0;
    proven_h2_error_t refused_with = PROVEN_H2_NO_ERROR;

    while (at < size) {
        /* Nine bytes of header say how long the frame is, what it is, and whose. */
        proven_h2_frame_t frame;
        proven_err_t err = proven_h2_frame_parse((proven_mem_view_t){ wire + at, size - at }, &frame);
        if (err == PROVEN_ERR_NEED_MORE) break;                    /* a real server would read more */
        EXAMPLE_REQUIRE(err == PROVEN_OK, "a frame header");

        /* Before the payload is looked at: is this frame allowed to be what it says? The size
         * limit is the one this side announced (the protocol's smallest, here). */
        proven_h2_error_t code;
        bool stream_error;
        if (proven_h2_frame_check(&frame, PROVEN_H2_MIN_MAX_FRAME_SIZE, &code, &stream_error) != PROVEN_OK) {
            /* A real server answers GOAWAY (or RST_STREAM when only the stream is at fault). */
            refused_with = code;
            EXAMPLE_REQUIRE(!stream_error, "a PING of the wrong size ends the connection");
            break;
        }
        proven_mem_view_t payload = { wire + at + PROVEN_H2_FRAME_HEADER_SIZE, frame.length };
        EXAMPLE_REQUIRE(PROVEN_H2_FRAME_HEADER_SIZE + frame.length <= size - at, "the whole frame is here");
        frames++;

        if (frame.type == PROVEN_H2_SETTINGS) {
            proven_size_t pos = 0;
            proven_h2_setting_t setting;
            while ((err = proven_h2_settings_next(payload, &pos, &setting, &code)) == PROVEN_OK) {
                if (setting.id == PROVEN_H2_SETTINGS_INITIAL_WINDOW_SIZE) printf("the client's window: %u
bytes a stream\n", (unsigned)setting.value);
                /* A setting this side does not know is simply not acted on. */
            }
            EXAMPLE_REQUIRE(err == PROVEN_ERR_EOF, "every setting was in range");
        } else if (frame.type == PROVEN_H2_HEADERS) {
            /* The header block is what is left when padding and the old priority fields are
             * taken away. (With END_HEADERS unset, CONTINUATION frames would carry the rest.) */
            proven_mem_view_t block;
            EXAMPLE_REQUIRE(proven_h2_frame_content(&frame, payload, &block) == PROVEN_OK && (frame.flags &
PROVEN_H2_FLAG_END_HEADERS) != 0, "a whole header block");
            proven_http_header_t fields[32];
```

```
            proven_byte_t scratch[2048];
            proven_size_t count = 0;
            EXAMPLE_REQUIRE(proven_hpack_decode(decoder, block, fields, 32, (proven_mem_mut_t){ scratch,
sizeof scratch }, 16384, &count) == PROVEN_OK, "decoded");

            /* A list of fields is not yet a request. This is where a name in capitals, a line
             * break in a value, a missing :path or a "connection" header are caught. */
            proven_h2_request_head_t head;
            EXAMPLE_REQUIRE(proven_h2_request_head(fields, count, &head) == PROVEN_OK, "a well-formed
request");
            printf("stream %u: %.*s %.*s for %.*s, %u more fields\n", (unsigned)frame.stream_id,
(int)head.method.size, (const char *)head.method.ptr,
                   (int)head.path.size, (const char *)head.path.ptr, (int)head.authority.size, (const char
*)head.authority.ptr, (unsigned)head.field_count);
            EXAMPLE_REQUIRE(proven_u8str_view_eq(head.path, PROVEN_LIT("/")) && head.field_count == 1 &&
proven_u8str_view_eq(head.fields[0].name, PROVEN_LIT("accept")), "its parts");
            requests++;
        }
        at += PROVEN_H2_FRAME_HEADER_SIZE + frame.length;
    }
    EXAMPLE_REQUIRE(frames == 2 && requests == 1 && refused_with == PROVEN_H2_FRAME_SIZE_ERROR, "two frames
taken, one request, and the third frame refused with FRAME_SIZE_ERROR");
    proven_hpack_decoder_destroy(decoder);

    // ---- what the server writes back ----------------------------------------------------
    proven_byte_t reply[256];
    proven_mem_mut_t out = { reply, sizeof reply };
    proven_size_t len = 0;
    static const proven_byte_t opaque[8] = { 'p', 'r', 'o', 'v', 'e', 'n', '!', '!' };
    EXAMPLE_REQUIRE(proven_h2_write_settings(out, &len, NULL, 0) == PROVEN_OK &&       /* its own settings:
all defaults */
                    proven_h2_write_settings_ack(out, &len) == PROVEN_OK &&            /* "yours are in force"
*/
                    proven_h2_write_window_update(out, &len, 0, 1 << 20) == PROVEN_OK &&   /* a megabyte more
may be sent on the connection */
                    proven_h2_write_ping(out, &len, opaque, false) == PROVEN_OK &&     /* are you there? */
                    proven_h2_write_rst_stream(out, &len, 3, PROVEN_H2_REFUSED_STREAM) == PROVEN_OK &&   /*
not that stream, not now */
                    proven_h2_write_goaway(out, &len, 1, PROVEN_H2_FRAME_SIZE_ERROR,
proven_mem_view_from_u8(PROVEN_LIT("a PING is eight bytes"))) == PROVEN_OK,
                    "six frames a server might send");

    /* Read back, as the client would. */
    at = 0;
    int seen = 0;
    while (at < len) {
        proven_h2_frame_t frame;
        proven_h2_error_t code;
        bool stream_error;
        EXAMPLE_REQUIRE(proven_h2_frame_parse((proven_mem_view_t){ reply + at, len - at }, &frame) ==
PROVEN_OK &&
                        proven_h2_frame_check(&frame, PROVEN_H2_MIN_MAX_FRAME_SIZE, &code, &stream_error) ==
PROVEN_OK, "each passes its own rule");
        proven_mem_view_t payload = { reply + at + PROVEN_H2_FRAME_HEADER_SIZE, frame.length };
        proven_u32 a = 0, b = 0;
        proven_mem_view_t debug;
        if (frame.type == PROVEN_H2_WINDOW_UPDATE) EXAMPLE_REQUIRE(proven_h2_read_window_update(payload, &a)
== PROVEN_OK && a == 1 << 20, "the increment");
        if (frame.type == PROVEN_H2_RST_STREAM) EXAMPLE_REQUIRE(proven_h2_read_rst_stream(payload, &a) ==
PROVEN_OK && a == PROVEN_H2_REFUSED_STREAM && frame.stream_id == 3, "the reset");
```

```
        if (frame.type == PROVEN_H2_GOAWAY) EXAMPLE_REQUIRE(proven_h2_read_goaway(payload, &a, &b, &debug) ==
PROVEN_OK && a == 1 && b == PROVEN_H2_FRAME_SIZE_ERROR && debug.size == 21, "the goodbye");
        seen++;
        at += PROVEN_H2_FRAME_HEADER_SIZE + frame.length;
    }
    EXAMPLE_REQUIRE(seen == 6, "six frames read back");

    // ---- responses and trailers ------------------------------------------------------------
    proven_http_header_t response[] = { { PROVEN_LIT(":status"), PROVEN_LIT("200") }, { PROVEN_LIT("content-
type"), PROVEN_LIT("text/html") } };
    proven_u16 status = 0;
    const proven_http_header_t *rest = NULL;
    proven_size_t rest_count = 0;
    EXAMPLE_REQUIRE(proven_h2_response_head(response, 2, &status, &rest, &rest_count) == PROVEN_OK && status
== 200 && rest_count == 1, "a response has one pseudo-header field: its status");
    proven_http_header_t smuggle[] = { { PROVEN_LIT(":status"), PROVEN_LIT("200") }, { PROVEN_LIT("x-note"),
PROVEN_LIT("fine\r\ntransfer-encoding: chunked") } };
    EXAMPLE_REQUIRE(proven_h2_response_head(smuggle, 2, &status, &rest, &rest_count) ==
PROVEN_ERR_INVALID_FORMAT, "a value with a line break in it is no response at all");
    proven_http_header_t trailers[] = { { PROVEN_LIT("x-checksum"), PROVEN_LIT("9f86d081") } };
    EXAMPLE_REQUIRE(proven_h2_trailers_check(trailers, 1) == PROVEN_OK, "trailers are ordinary fields");
    return EXAMPLE_OK();
}
```

Reference

API	Intent	Return
`proven_h2_frame_parse(data, &frame)`	Read nine bytes of frame header.	`proven_err_t`: `NEED_MORE` under nine bytes.
`proven_h2_frame_write(out, &len, &frame)`	Append a frame header.	`proven_err_t`: `INVALID_ARG` (length above 2^24-1, stream above 2^31-1), `OUT_OF_BOUNDS`.
`proven_h2_frame_check(&frame, max_frame_size, &code, &stream_error)`	Apply the table above.	`proven_err_t`: `INVALID_FORMAT` with the code to send and whether the stream or the connection ends.
`proven_h2_frame_content(&frame, payload, &content)`	The content of DATA or HEADERS, without padding and priority.	`proven_err_t`: `INVALID_FORMAT` for padding that does not fit (a connection error, `PROTOCOL_ERROR`).
`proven_h2_settings_next(payload, &pos, &setting, &code)`	Read the next setting.	`proven_err_t`: `EOF` at the end; `INVALID_FORMAT` with the code for a value out of range.
`proven_h2_write_settings, _settings_ack, _ping, _window_update, _rst_stream, _goaway`	Append one whole frame.	`proven_err_t`: `OUT_OF_BOUNDS` with nothing written.
`proven_h2_read_window_update, _rst_stream, _goaway`	Read the payload of a checked frame.	`proven_err_t`; a window increment of 0 is `INVALID_FORMAT`.

Cautions, and what goes wrong

- **Check before you read.** `proven_h2_frame_parse` cannot fail on nine bytes; whether they describe a frame that may exist is the check's answer. A frame longer than the size you announced is a connection error whatever it claims to be.

- **Flow control counts the whole payload.** `proven_h2_frame_content` tells you where the content of a padded DATA frame is; the window is charged for the padding too.
- **A header block may span frames**, and nothing else may come between them: HEADERS without `END_HEADERS` is followed by CONTINUATION frames of the same stream until one has it. Bound what you assemble - twice your header list limit is enough for any honest block - or a peer can make you hold as much as it cares to send.
- **Unknown is not wrong.** A frame type or a setting this library does not know passes the check, as the protocol requires; skip it.
- **Settings have ranges**, and a value outside one is a connection error: `ENABLE_PUSH` and `ENABLE_CONNECT_PROTOCOL` are 0 or 1, the initial window at most 2^31-1, the frame size from 16,384 to 2^24-1. `ENABLE_CONNECT_PROTOCOL`, once 1, may not become 0.

4. A header list as a message

```
proven_err_t proven_h2_request_head(const proven_http_header_t *headers, proven_size_t count,
proven_h2_request_head_t *out);
proven_err_t proven_h2_response_head(const proven_http_header_t *headers, proven_size_t count, proven_u16
*status,
                                     const proven_http_header_t **fields, proven_size_t *field_count);
proven_err_t proven_h2_trailers_check(const proven_http_header_t *headers, proven_size_t count);
```

A decoded header list is a list of byte strings. Whether it is a request is a separate question, and the answer matters most where HTTP/2 meets HTTP/1.1: a value with a line break in it, passed on as text, becomes two header lines - a request the receiving server never saw arrive. So a list is *malformed*, and must be refused rather than repaired or forwarded, when:

Rule	Why
A field name has an uppercase letter, a space, a control byte, a byte above `0x7e`, or a colon anywhere but first	Names are lowercase tokens; a colon starts a pseudo-header field and nothing else
A value has NUL, CR or LF, or begins or ends with a space or a tab	These are the delimiters of HTTP/1.1
A pseudo-header field comes after an ordinary one, appears twice, or is one nobody defined	They are the message's first line: once, and first
A request lacks `:method`, `:scheme` or `:path`; `:path` is empty for `http` or `https`; a request has `:status`	The first line must be whole. (`CONNECT` has `:method` and `:authority` only)
A response lacks `:status`, or it is not three digits	The same
`connection`, `proxy-connection`, `keep-alive`, `transfer-encoding` or `upgrade` is present; `te` says anything but `trailers`	These describe an HTTP/1.1 connection. Here they are an attempt to describe someone else's
Trailers contain any pseudo-header field	Trailers are ordinary fields

`proven_h2_request_head` applies them and hands back the parts: method, scheme, authority, path, and the ordinary fields that follow - and `protocol`, for an extended CONNECT (RFC 8441): `:protocol` with CONNECT only, and then scheme, authority and path all present. Whether one is allowed is the connection's to say: a server whose configuration has `enable_connect_protocol` says `SETTINGS_ENABLE_CONNECT_PROTOCOL` and takes them; any other refuses such a stream with `PROTOCOL_ERROR`, and ends its side of one it took with END_STREAM rather than a reset - a tunnel ends from both sides. A malformed message is a stream error of type `PROTOCOL_ERROR`: answer `400` if a response can still be sent, reset the stream if not.

5. A connection

```
proven_err_t proven_h2_conn_create(const proven_h2_config_t *config, proven_h2_conn_t **out);
void proven_h2_conn_destroy(proven_h2_conn_t *conn);

proven_err_t proven_h2_conn_feed(proven_h2_conn_t *conn, proven_mem_view_t in, proven_size_t *consumed,
proven_h2_event_t *event);
proven_mem_view_t proven_h2_conn_pending_output(const proven_h2_conn_t *conn);
void proven_h2_conn_output_sent(proven_h2_conn_t *conn, proven_size_t n);
```

`proven_h2_conn_t` is one HTTP/2 connection, for either end. It holds everything sections 2 to 4 left to you: the settings of both sides, the streams that are open, the windows in both directions, the two compression tables, and the answer the protocol requires to each frame. It does **no I/O and reads no clock**. The shape is the one the TLS engine of Chapter 14 has:

```
bytes the peer sent  -->  proven_h2_conn_feed  -->  one event: a request, a piece of a body, a reset, ...
your requests, responses and bodies  -->  proven_h2_conn_submit_* / _send_data
proven_h2_conn_pending_output  -->  bytes for the peer  -->  proven_h2_conn_output_sent
```

A socket, TLS, and every timeout are the caller's. That is what makes one object serve a blocking program, an event loop and a test with no network at all - the example below is two connections handing each other buffers, and section 6 is the same object behind a socket.

Events

`proven_h2_conn_feed` uses bytes of `in` until it has **one** event to report, and says how many it used. Call it again with the rest; `PROVEN_H2_EVENT_NONE` means everything was used.

Event	Means	Fields
`PROVEN_H2_EVENT_HEADERS`	A server: a request began, already checked by section 4's rules and taken apart. A client: a response head (`status` below 200 is an interim one; more follow)	`stream_id`, `request` or `status` and `fields`, `end_stream`, `user`
`PROVEN_H2_EVENT_DATA`	A piece of a body, as a view into `in`	`stream_id`, `data`, `end_stream`, `user`
`PROVEN_H2_EVENT_TRAILERS`	Trailer fields: the peer's side of the stream is over	`stream_id`, `fields`, `user`
`PROVEN_H2_EVENT_STREAM_RESET`	The stream is over before its end: the peer reset it, broke a rule on it, or said goodbye without acting on it	`stream_id`, `code`, `user`; on a client, `end_stream` when the response is nonetheless complete (below); `too_large` when this side reset it because the peer's header list was over `max_header_list_bytes`
`PROVEN_H2_EVENT_STREAM_WRITABLE`	A send that was cut short can go on	`stream_id`, `user`
`PROVEN_H2_EVENT_PONG`	The answer to `proven_h2_conn_ping`	`data`: the eight bytes
`PROVEN_H2_EVENT_GOAWAY`	The peer will take no new stream	`code`, `last_stream_id`, `data` (the text it sent with that: the first 256 bytes, for a log)
`PROVEN_H2_EVENT_CLOSED`	The connection is over, and every stream with it. Send what is pending, then close	`code`

Three things about this table are worth a second look.

A body is never copied. A DATA event points into the buffer you fed, so a body of any size passes through in pieces no larger than your reads. A frame that arrives over three reads is three events, and `end_stream` is on the last.

The peer's fault is an event, not an error. `proven_h2_conn_feed` returns `PROVEN_OK` when the peer breaks the protocol: the connection has queued a GOAWAY frame with the code, and reports `PROVEN_H2_EVENT_CLOSED`. A fault of one stream is `PROVEN_H2_EVENT_STREAM_RESET` for that stream and nothing else. The one error a feed returns for what the peer did is `PROVEN_ERR_INVALID_FORMAT`, on a server whose client did not begin with the 24-byte preface: **that is not HTTP/2**, nothing is said to it, and it is reported at the first byte that differs - so the bytes are still yours to hand to an HTTP/1.1 parser.

Some events need no input. A stream becomes writable when output is *sent*, not when something arrives. So after `proven_h2_conn_output_sent`, call `proven_h2_conn_feed` once more, with an empty view, and take what is due.

Sending

```
proven_err_t proven_h2_conn_submit_request(proven_h2_conn_t *conn, const proven_http_header_t *headers,
proven_size_t count,
                                           bool end_stream, proven_u32 *stream_id);
proven_err_t proven_h2_conn_submit_response(proven_h2_conn_t *conn, proven_u32 stream_id, const
proven_http_header_t *headers, proven_size_t count,
                                            bool end_stream);
proven_result_size_t proven_h2_conn_send_data(proven_h2_conn_t *conn, proven_u32 stream_id, proven_mem_view_t
data, bool end_stream);
proven_err_t proven_h2_conn_send_trailers(proven_h2_conn_t *conn, proven_u32 stream_id, const
proven_http_header_t *headers, proven_size_t count);
proven_err_t proven_h2_conn_reset(proven_h2_conn_t *conn, proven_u32 stream_id, proven_h2_error_t code);
```

A client opens a stream with a header list that begins with the four pseudo-header fields; a server answers a stream with one that begins with `:status`. Both are checked by the rules of section 4 before a byte is queued, so this library does not send what it would refuse.

`proven_h2_conn_send_data` **may take fewer bytes than you offer**, possibly none, and that is not an error. What it takes is the least of three things: the stream's window, the connection's window, and the room left under `max_buffered_output`. Keep the rest and offer it again when `PROVEN_H2_EVENT_STREAM_WRITABLE` names the stream. `end_stream` takes effect when all of what you offered was taken.

When several streams wait for the connection's window, they are told **in turn**. A stream that is told takes what the window has; when room comes back, the search for the next one to tell goes on from the stream after it. A long body therefore does not hold up the other streams of its connection. (Before v0.37.1 it could: the search began at the same stream every time.)

Flow control, receiving

```
void proven_h2_conn_consumed(proven_h2_conn_t *conn, proven_u32 stream_id, proven_size_t n);
```

The peer may send only as much as this side has said it will take: `initial_window` bytes a stream, and as many for the connection as a whole. **Room comes back when you say the bytes were dealt with**, with `proven_h2_conn_consumed` - not when they arrive. A handler that falls behind and stops calling it stops its peer, having buffered at most one window. That is HTTP/2's backpressure, and it works only if this call means what it says: call it when the bytes have gone where they were going, not when you were handed them.

What nobody consumes, the connection gives back by itself: padding, data for a stream that was reset, and whatever a stream had not consumed when it ended.

What a peer can make a connection hold or do

Every amount has a setting, and every setting a default:

Setting	Default	At the edge
`max_concurrent_streams`	100	A stream beyond it is refused (`REFUSED_STREAM`) and never reported
`initial_window`	65,535	DATA beyond a stream's window is `FLOW_CONTROL_ERROR` for the stream; beyond the connection's, for the connection
`max_frame_size`	16,384	A larger frame is `FRAME_SIZE_ERROR` for the connection
`max_header_list_bytes`	16 KiB	A larger list is `PROTOCOL_ERROR` for its stream; a header block of more than twice this ends the connection
`header_table_bytes`	4,096	The HPACK table in each direction; with `set_header_table` and 0, none
`max_buffered_output`	64 KiB	More body is not taken, and `submit` calls answer `PROVEN_ERR_BUSY`, until some is sent. Counted apart: the answers the protocol owes a peer that does not read them, beyond which the connection ends
`max_reset_debt`	100	Streams reset - by the peer, or for its fault - beyond the number that finished properly: one more ends the connection with `ENHANCE_YOUR_CALM`
`enable_connect_protocol`	false	A server: take extended CONNECT (RFC 8441) and say so in its first settings. The event-driven server sets it from `http2_websocket` (Chapter 15)

The last row is the answer to a client that opens streams and resets them at once, as fast as it can, to make a server start work it then throws away. No clock is involved: a reset adds one, a stream that ends as it should takes one off, and a peer that mostly finishes what it starts never comes near the limit.

A connection that has only opened holds one block of at most 400 bytes. One that has carried requests and is idle again holds that, a record of how its last streams ended, and the two compression tables, **which grow as they are used**: under 3 KiB after a hundred small requests, about 12.5 KiB when both are full at their default size of 4,096 bytes, and under 1,280 bytes with none. While streams are open it also holds 48 bytes for each, and the buffers for reading one header list: five times `max_header_list_bytes`, which is 80 KiB at the default. **That last one need not be each connection's**: give the configuration a `header_buffer` of `proven_h2_header_buffer_size` bytes and every connection driven by one thread can read into the same memory - what is read there is what an event reports, and an event is good only until the next call. The server of section 6 does this.

Settings changed after the opening

```
proven_err_t proven_h2_conn_set_max_streams(proven_h2_conn_t *conn, proven_u32 n);
proven_err_t proven_h2_conn_set_initial_window(proven_h2_conn_t *conn, proven_u32 n);
```

Two of the settings can be changed while the connection runs (v0.79.0): how many streams the peer may have open at once, and the window each stream starts with. A server under load lowers them and raises them again later. Each call queues a SETTINGS frame; RFC 9113 says a setting holds once the peer acknowledges it, and the connection keeps to that:

- **The values at creation are the ceilings.** What a connection may come to hold is sized by them when it is made, so a change can lower a number and raise it back, never above: `PROVEN_ERR_INVALID_ARG`.

- **Until the acknowledgement, the larger number holds.** A peer may open streams, or send body bytes, under the number it had before it read the change; those are taken. After the acknowledgement a stream over the new number is refused with `REFUSED_STREAM` (a client may send it again), and DATA beyond a stream's window is that stream's `FLOW_CONTROL_ERROR`.
- **Nothing open is cut.** Streams already open go on, however many there are. A window change moves every open stream's window by the difference (RFC 9113 section 6.9.2): a lowering can leave a stream with less than nothing, and it then waits until enough is consumed. A raise holds at once. The connection's own window is not a setting and does not change.
- **One change in flight.** A call made while one waits for its acknowledgement is kept, and the latest values go out together when it comes - one frame, however many calls.
- **On a client**, `max_streams` is also the most this side opens, and that holds at once.

The example below lowers both on a server and shows the client keeping to the new number.

Goodbye

```
void proven_h2_conn_shutdown(proven_h2_conn_t *conn, proven_h2_error_t code);
proven_err_t proven_h2_conn_ping(proven_h2_conn_t *conn, const proven_byte_t opaque[8]);
bool proven_h2_conn_is_closed(const proven_h2_conn_t *conn);
```

`proven_h2_conn_shutdown` with `PROVEN_H2_NO_ERROR` queues a GOAWAY frame and lets the open streams finish: no new stream is taken or begun, and `PROVEN_H2_EVENT_CLOSED` follows when the last has ended. The peer's GOAWAY is handled the same way, and a client is told which of its streams the server did not act on - `PROVEN_H2_EVENT_STREAM_RESET` with `PROVEN_H2_REFUSED_STREAM` - because those are safe to send again on another connection.

Example: both ends of a connection, with no socket between them

```
#include <stdio.h>
#include <string.h>

/*
 * An HTTP/2 connection, both ends of it, with no socket: what one end queues is handed to the
 * other as if it had crossed a wire.
 *
 * proven_h2_conn_t does no I/O. You feed it the bytes the peer sent and it reports one event
 * at a time; you ask it to send and it queues frames for you to collect. A real program puts
 * a socket (and TLS) where carry() is.
 */

typedef struct {
    proven_h2_conn_t *conn;
    const char *name;
    /* What this end has learned, for the checks at the bottom. */
    int requests, responses, trailers, resets, writable, pongs, goaways, closed;
    proven_size_t body_bytes;
    proven_u32 last_stream, reset_code, close_code;
    bool hold;                          /* do not say that DATA was consumed */
} ex_end_t;

/* Take every event the bytes in `in` hold. One call reports one event: call again with the
 * rest until there is no event and nothing left. */
static bool take_events(ex_end_t *e, proven_mem_view_t in) {
    proven_size_t pos = 0;
    for (;;) {
        proven_size_t used = 0;
        proven_h2_event_t ev;
        if (proven_h2_conn_feed(e->conn, (proven_mem_view_t){ in.ptr + pos, in.size - pos }, &used, &ev) !=
PROVEN_OK) return false;
        pos += used;
```

```
        switch (ev.kind) {
            case PROVEN_H2_EVENT_NONE:
                if (pos == in.size) return true;           /* everything was used: read more */
                break;
            case PROVEN_H2_EVENT_HEADERS:
                e->last_stream = ev.stream_id;
                if (ev.status == 0) {                      /* a server: a request, already checked and taken
apart */
                    e->requests++;
                    printf("%s: stream %u asks %.*s %.*s\n", e->name, (unsigned)ev.stream_id,
(int)ev.request.method.size, (const char *)ev.request.method.ptr,
                           (int)ev.request.path.size, (const char *)ev.request.path.ptr);
                } else {                                   /* a client: a response head */
                    e->responses++;
                    printf("%s: stream %u answers %u%s\n", e->name, (unsigned)ev.stream_id,
(unsigned)ev.status, ev.user ? " (the stream we marked)" : "");
                }
                break;
            case PROVEN_H2_EVENT_DATA:
                /* ev.data points into `in`: nothing was copied. Saying it was consumed is
                 * what lets the peer send more. */
                e->body_bytes += ev.data.size;
                if (!e->hold) proven_h2_conn_consumed(e->conn, ev.stream_id, ev.data.size);
                break;
            case PROVEN_H2_EVENT_TRAILERS: e->trailers++; break;
            case PROVEN_H2_EVENT_STREAM_RESET: e->resets++; e->reset_code = ev.code; break;
            case PROVEN_H2_EVENT_STREAM_WRITABLE: e->writable++; break;
            case PROVEN_H2_EVENT_PONG: e->pongs++; break;
            case PROVEN_H2_EVENT_GOAWAY: e->goaways++; break;
            case PROVEN_H2_EVENT_CLOSED: e->closed++; e->close_code = ev.code; return true;
        }
    }
}

/* The wire: what `from` has queued goes to `to`. Then `from` is asked once more, with
 * nothing, for what sending made due - a stream that may go on. */
static bool carry(ex_end_t *from, ex_end_t *to) {
    static proven_byte_t wire[200000];
    proven_mem_view_t out = proven_h2_conn_pending_output(from->conn);
    if (out.size > sizeof wire) return false;
    if (out.size > 0) memcpy(wire, out.ptr, out.size);
    proven_size_t n = out.size;
    proven_h2_conn_output_sent(from->conn, n);             /* a socket would say how many it took */
    if (!to->closed && !take_events(to, (proven_mem_view_t){ wire, n })) return false;
    return from->closed || take_events(from, (proven_mem_view_t){ wire, 0 });
}

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    ex_end_t client = { .name = "client" }, server = { .name = "server" };

    /* Zero means the default for every limit. `server` is the one thing that must be said. */
    proven_h2_config_t ccfg = { .alloc = heap };
    proven_h2_config_t scfg = { .alloc = heap, .server = true, .max_concurrent_streams = 10 };
    EXAMPLE_REQUIRE(proven_h2_conn_create(&ccfg, &client.conn) == PROVEN_OK && proven_h2_conn_create(&scfg,
&server.conn) == PROVEN_OK, "two ends");

    /* The opening is already queued: the client's preface and both ends' settings. */
    EXAMPLE_REQUIRE(carry(&client, &server) && carry(&server, &client) && carry(&client, &server), "settings
cross and are acknowledged");
```

```
    /* A client that waits for this before its first request never asks for more streams than the server
takes. */
    EXAMPLE_REQUIRE(proven_h2_conn_has_peer_settings(client.conn) &&
proven_h2_conn_has_peer_settings(server.conn), "each end has the other's settings");

    // ---- a request with a body, and its answer ------------------------------------------
    proven_http_header_t request[] = {
        { PROVEN_LIT(":method"), PROVEN_LIT("POST") }, { PROVEN_LIT(":scheme"), PROVEN_LIT("https") },
        { PROVEN_LIT(":authority"), PROVEN_LIT("www.example.com") }, { PROVEN_LIT(":path"), PROVEN_LIT("/
upload") },
        { PROVEN_LIT("content-type"), PROVEN_LIT("text/plain") },
    };
    proven_u32 stream = 0;
    int mark = 0;
    EXAMPLE_REQUIRE(proven_h2_conn_submit_request(client.conn, request, 5, false, &stream) == PROVEN_OK &&
stream == 1, "a client's first stream is 1");
    proven_h2_conn_stream_set_user(client.conn, stream, &mark);       /* events for this stream will carry it
*/
    EXAMPLE_REQUIRE(proven_h2_conn_stream_user(client.conn, stream) == &mark, "and it can be asked for");
    proven_result_size_t sent = proven_h2_conn_send_data(client.conn, stream,
proven_mem_view_from_u8(PROVEN_LIT("hello, ")), false);
    EXAMPLE_REQUIRE(sent.err == PROVEN_OK && sent.value == 7, "seven bytes of body");
    sent = proven_h2_conn_send_data(client.conn, stream, proven_mem_view_from_u8(PROVEN_LIT("world")),
true);     /* true: the end of the body */
    EXAMPLE_REQUIRE(sent.err == PROVEN_OK && sent.value == 5 && carry(&client, &server), "and five more, the
last");
    EXAMPLE_REQUIRE(server.requests == 1 && server.body_bytes == 12 &&
proven_h2_conn_stream_count(server.conn) == 1, "the server has the request and its twelve bytes");

    proven_http_header_t response[] = { { PROVEN_LIT(":status"), PROVEN_LIT("200") }, { PROVEN_LIT("content-
type"), PROVEN_LIT("text/plain") } };
    proven_http_header_t trailers[] = { { PROVEN_LIT("x-checksum"), PROVEN_LIT("9f86d081") } };
    EXAMPLE_REQUIRE(proven_h2_conn_submit_response(server.conn, stream, response, 2, false) == PROVEN_OK, "a
response head");
    sent = proven_h2_conn_send_data(server.conn, stream, proven_mem_view_from_u8(PROVEN_LIT("thanks")),
false);
    EXAMPLE_REQUIRE(sent.err == PROVEN_OK && sent.value == 6 && proven_h2_conn_send_trailers(server.conn,
stream, trailers, 1) == PROVEN_OK, "a body, and trailers to end it");
    EXAMPLE_REQUIRE(carry(&server, &client), "back to the client");
    EXAMPLE_REQUIRE(client.responses == 1 && client.body_bytes == 6 && client.trailers == 1, "the client has
all three parts");
    EXAMPLE_REQUIRE(proven_h2_conn_stream_count(client.conn) == 0 && proven_h2_conn_stream_count(server.conn)
== 0, "and the stream is over at both ends");

    // ---- flow control: a receiver that falls behind stops its sender ------------------------
    static proven_byte_t big[100000];
    server.hold = true;                                  /* the server's handler is busy: it consumes
nothing */
    server.body_bytes = 0;
    /* A field no hop may remember is marked for the one list that follows: here the last, as if it were a
key. */
    static const bool keep_out[5] = { false, false, false, false, true };
    proven_h2_conn_next_sensitive(client.conn, keep_out);
    EXAMPLE_REQUIRE(proven_h2_conn_submit_request(client.conn, request, 5, false, &stream) == PROVEN_OK &&
stream == 3 && carry(&client, &server), "a second request");
    sent = proven_h2_conn_send_data(client.conn, stream, (proven_mem_view_t){ big, sizeof big }, true);
    const proven_size_t taken = sent.value;
    printf("client: offered %u bytes, %u were taken\n", (unsigned)sizeof big, (unsigned)taken);
    /* The connection's window is 65,535 bytes by default, and the twelve bytes of the first
     * request have not been given back yet: windows are reopened in batches, not byte by byte. */
```

```
    EXAMPLE_REQUIRE(sent.err == PROVEN_OK && taken == 65535 - 12, "what the windows allow and no more: taking
less than was offered is not an error");
    EXAMPLE_REQUIRE(proven_h2_conn_send_room(client.conn, stream) == 0 && carry(&client, &server) &&
carry(&server, &client) && client.writable == 0, "nothing more may go until the server says so");
    proven_h2_conn_consumed(server.conn, stream, taken);   /* the handler caught up */
    server.hold = false;
    EXAMPLE_REQUIRE(carry(&server, &client) && client.writable == 1, "and now the client is told:
PROVEN_H2_EVENT_STREAM_WRITABLE");
    sent = proven_h2_conn_send_data(client.conn, stream, (proven_mem_view_t){ big + taken, sizeof big -
taken }, true);
    EXAMPLE_REQUIRE(sent.err == PROVEN_OK && sent.value == sizeof big - taken && carry(&client, &server) &&
server.body_bytes == sizeof big, "the rest follows");

    /* The server gives this one up instead of answering. */
    EXAMPLE_REQUIRE(proven_h2_conn_reset(server.conn, stream, PROVEN_H2_CANCEL) == PROVEN_OK && carry(&server,
&client), "a reset");
    EXAMPLE_REQUIRE(client.resets == 1 && client.reset_code == PROVEN_H2_CANCEL &&
proven_h2_conn_stream_count(client.conn) == 0, "which the client hears of, with the code");

    // ---- is anybody there? ------------------------------------------------------------
    static const proven_byte_t opaque[8] = { 'p', 'r', 'o', 'v', 'e', 'n', '!', '!' };
    EXAMPLE_REQUIRE(proven_h2_conn_ping(client.conn, opaque) == PROVEN_OK && carry(&client, &server) &&
carry(&server, &client) && client.pongs == 1, "a ping is answered by the connection itself");

    // ---- settings changed after the opening -------------------------------------------
    /* A server under load asks for fewer streams at once and smaller windows: one SETTINGS
     * frame, in force once the client acknowledges it. The numbers given at creation stay the
     * ceilings; streams already open are not touched. */
    EXAMPLE_REQUIRE(proven_h2_conn_set_max_streams(server.conn, 1) == PROVEN_OK &&
proven_h2_conn_set_initial_window(server.conn, 16384) == PROVEN_OK, "the server lowers both");
    EXAMPLE_REQUIRE(carry(&server, &client) && carry(&client, &server), "the change crosses, and is
acknowledged");
    EXAMPLE_REQUIRE(proven_h2_conn_submit_request(client.conn, request, 5, true, &stream) == PROVEN_OK &&
carry(&client, &server), "one request");
    proven_u32 second = 0;
    EXAMPLE_REQUIRE(proven_h2_conn_submit_request(client.conn, request, 5, true, &second) == PROVEN_ERR_BUSY,
"and not a second at the same time: the client holds itself to the new number");
    EXAMPLE_REQUIRE(proven_h2_conn_set_max_streams(server.conn, 11) == PROVEN_ERR_INVALID_ARG, "above the
number given at creation: refused");
    EXAMPLE_REQUIRE(proven_h2_conn_reset(server.conn, stream, PROVEN_H2_CANCEL) == PROVEN_OK && carry(&server,
&client), "the server lets that one go");

    // ---- a peer that breaks a rule ----------------------------------------------------
    /* A WINDOW_UPDATE that adds nothing is a protocol error. The connection says so to its
     * peer (GOAWAY), reports that it is over, and takes nothing more. */
    ex_end_t strict = { .name = "strict" };
    EXAMPLE_REQUIRE(proven_h2_conn_create(&scfg, &strict.conn) == PROVEN_OK, "another server");
    proven_byte_t bad[128];
    proven_size_t len = PROVEN_H2_PREFACE_SIZE;
    memcpy(bad, PROVEN_H2_PREFACE, PROVEN_H2_PREFACE_SIZE);
    EXAMPLE_REQUIRE(proven_h2_write_settings((proven_mem_mut_t){ bad, sizeof bad }, &len, NULL, 0) ==
PROVEN_OK, "a client that opens properly");
    static const proven_byte_t zero_increment[13] = { 0, 0, 4, PROVEN_H2_WINDOW_UPDATE, 0, 0, 0, 0, 0, 0, 0,
0, 0 };
    memcpy(bad + len, zero_increment, sizeof zero_increment);
    len += sizeof zero_increment;
    EXAMPLE_REQUIRE(take_events(&strict, (proven_mem_view_t){ bad, len }), "the feed itself succeeds: a peer's
fault is an event");
    printf("strict: closed with code %u\n", (unsigned)strict.close_code);
    EXAMPLE_REQUIRE(strict.closed == 1 && strict.close_code == PROVEN_H2_PROTOCOL_ERROR &&
```

```
proven_h2_conn_is_closed(strict.conn), "PROTOCOL_ERROR, and the connection is over");
    EXAMPLE_REQUIRE(proven_h2_conn_pending_output(strict.conn).size > 0, "with a GOAWAY still to be sent
before the socket is closed");
    proven_h2_conn_destroy(strict.conn);

    /* And bytes that are not HTTP/2 at all are an error of the call, at the first of them. */
    EXAMPLE_REQUIRE(proven_h2_conn_create(&scfg, &strict.conn) == PROVEN_OK, "one more");
    proven_size_t used = 0;
    proven_h2_event_t ev;
    EXAMPLE_REQUIRE(proven_h2_conn_feed(strict.conn, proven_mem_view_from_u8(PROVEN_LIT("GET /
HTTP/1.1\r\n\r\n")), &used, &ev) == PROVEN_ERR_INVALID_FORMAT && used == 0,
                    "HTTP/1.1 is PROVEN_ERR_INVALID_FORMAT with nothing taken: the bytes are yours to give to
something else");
    proven_h2_conn_destroy(strict.conn);

    // ---- goodbye ------------------------------------------------------------------
    proven_h2_conn_shutdown(server.conn, PROVEN_H2_NO_ERROR);          /* no new streams; open ones would
finish first */
    EXAMPLE_REQUIRE(carry(&server, &client) && carry(&client, &server), "the goodbye crosses");
    EXAMPLE_REQUIRE(client.goaways == 1 && client.closed == 1 && server.closed == 1 && client.close_code ==
PROVEN_H2_NO_ERROR, "with no stream open, both ends report the end at once");
    EXAMPLE_REQUIRE(proven_h2_conn_submit_request(client.conn, request, 5, true, &stream) ==
PROVEN_ERR_INVALID_STATE, "and nothing more is begun");

    proven_h2_conn_destroy(client.conn);
    proven_h2_conn_destroy(server.conn);
    return EXAMPLE_OK();
}
```

Reference

API	Intent	Return
`proven_h2_conn_create(&config, &conn)`	Make a connection; the opening is queued at once.	`proven_err_t`: `INVALID_ARG` (a setting out of range), `NOMEM`.
`proven_h2_header_buffer_size(max_header_list_bytes)`	How large `config.header_buffer` must be, if you supply one.	`proven_size_t`.
`proven_h2_conn_destroy(conn)`	Free it.	`void`.
`proven_h2_conn_feed(conn, in, &consumed, &event)`	Take bytes from the peer; report one event.	`proven_err_t`: `INVALID_FORMAT` (a server: not HTTP/2), `INVALID_STATE` after the end was reported, `NOMEM`.
`proven_h2_conn_pending_output(conn)`	Bytes to send.	`proven_mem_view_t`.
`proven_h2_conn_output_sent(conn, n)`	Say how many went out.	`void`.
`proven_h2_conn_submit_request(conn, headers, count, end_stream, &stream_id)`	A client: begin a request.	`proven_err_t`: `INVALID_ARG`, `BUSY` (stream limit, or output waiting), `INVALID_STATE` (after a goodbye), `OUT_OF_BOUNDS` (the peer accepts no list so large), `NOMEM`.
`proven_h2_conn_submit_response(conn, stream_id, headers, count, end_stream)`	A server: answer.	`proven_err_t`: `INVALID_ARG`, `NOT_FOUND`, `INVALID_STATE` (already answered), `BUSY`, `OUT_OF_BOUNDS`, `NOMEM`.

`proven_h2_conn_send_data(conn, stream_id, data, end_stream)`	Send body bytes: as many as may go.	`proven_result_size_t`: the bytes taken; `NOT_FOUND`, `INVALID_STATE`.
`proven_h2_conn_send_trailers(conn, stream_id, headers, count)`	End this side with trailer fields.	`proven_err_t`, as a response.
`proven_h2_conn_next_sensitive(conn, sensitive)`	Mark fields of the next header list given to the connection - one flag a field - as never to be indexed. The marks are for that one list, sent or refused.	`void`.
`proven_h2_conn_reset(conn, stream_id, code)`	Give up on a stream.	`proven_err_t`: `NOT_FOUND`.
`proven_h2_conn_consumed(conn, stream_id, n)`	Say that DATA was dealt with.	`void`.
`proven_h2_conn_ping(conn, opaque)`	Ask whether the peer is there.	`proven_err_t`: `INVALID_STATE` after the end.
`proven_h2_conn_set_max_streams(conn, n)`	Change how many streams the peer may have open at once: 0 to the value at creation. Held once the peer acknowledges; streams open are not touched.	`proven_err_t`: `INVALID_ARG` (above the ceiling), `INVALID_STATE` (after a goodbye), `NOMEM`.
`proven_h2_conn_set_initial_window(conn, n)`	Change each stream's window: 0 to the value at creation. Open streams move by the difference; a raise holds at once, a lowering once acknowledged.	`proven_err_t`, as above.
`proven_h2_conn_shutdown(conn, code)`	Say goodbye.	`void`.
`proven_h2_conn_is_closed(conn)`, `_stream_count(conn)`, `_send_room(conn, stream_id)`	Whether it is over; streams open; body bytes that may go now.	`bool`, `proven_size_t`.
`proven_h2_conn_stream_set_user(conn, stream_id, user)`, `_stream_user(conn, stream_id)`	A pointer of yours on an open stream.	`void`, `void *`.

Cautions, and what goes wrong

- **After every call, send what is pending.** The connection answers by itself - acknowledgements, window updates, resets, the last GOAWAY - and those answers are in `proven_h2_conn_pending_output` whether or not you asked for anything.
- **After sending, feed nothing.** `proven_h2_conn_feed` with an empty view collects what sending made due. A driver that waits for the socket instead will wait for a peer that is itself waiting.
- `proven_h2_conn_send_data` **returning 0 is not the end of the world, nor of the stream.** It is "not now". The exception is an empty `data` with `end_stream`: that always goes out, and 0 then means "ended".
- **Views die at the next call.** The fields of an event, and the pending output, are good until the next call that takes the connection. Copy what you keep.
- **The pointer comes with the event.** When a stream ends, `proven_h2_conn_stream_user` no longer finds it: the last event for the stream carries the pointer in `user`, and that is where to free what it points to.

- `PROVEN_H2_EVENT_CLOSED` **ends every stream without saying so one by one.** Walk your own list of what was open.
- **A response that ends before its request tells the client to stop.** The server's side sends `RST_STREAM` with `NO_ERROR`, as RFC 9113 provides, and the stream is over. A client sees `PROVEN_H2_EVENT_STREAM_RESET` with `PROVEN_H2_NO_ERROR` after the response: not a failure, and the event's `end_stream` is true. **Some servers reset with `NO_ERROR` in place of ending the stream**, after every byte the response's `content-length` declared; the length shows nothing is missing, and `end_stream` is true then too. One byte short, with no length (or a length that describes no body: HEAD, 204, 304), or with any other code, it is false: the response may be cut short.
- **A connection is for one thread at a time.**
- **No clock means no timeouts.** A peer that opens a connection and says nothing, or never opens a window, is the caller's to give up on.

6. HTTP/2 in the event-driven server

```
typedef struct {
    /* ... everything Chapter 15 describes ... */
    bool http2;
    proven_size_t max_concurrent_streams;
    proven_size_t http2_window_bytes;
} proven_http_event_server_config_t;

int proven_http_stream_protocol(const proven_http_stream_t *stream);
```

The event-driven server of Chapter 15 speaks HTTP/2 when its configuration says `http2`. **Nothing else changes for a handler**: the same four callbacks, the same `proven_http_stream_*` calls, the same request type. What was one exchange on a connection is now one stream of many, and the server does the translating:

A handler does	On HTTP/2 that is
reads `head->method`, `head->target`, the fields	the pseudo-header fields and the list, already checked by section 4's rules. `Host` is there, made from `:authority` when the client sent no `host` field; `version_minor` is 1
learns from `Content-Length` or `Transfer-Encoding` that a body follows	the same: HTTP/2 may send a body with neither, and such a request is given `Transfer-Encoding: chunked`, which means what it meant - a body whose length is not announced
`proven_http_stream_respond`, `_begin`, `_write`, `_end`	a response head, DATA frames as the client's windows and the output bound allow, and the end of the stream. Field names go out in lowercase; `Keep-Alive`, `Proxy-Connection` and `Upgrade` are not sent
is refused by `_write`, and waits for `on_writable`	exactly as on HTTP/1.1 - and for the same two reasons, though it cannot tell them apart: the server holds `max_buffered_output`, or the client is not making room
`proven_http_stream_pause`	no `on_body` until it resumes, as on HTTP/1.1. What had already arrived is kept - at most one window, because the client is not told it was consumed and can send no more - and delivered as one piece on resume

`proven_http_stream_compress`	unchanged: the compressor is above the framing
`proven_http_stream_abort`	the stream is reset; the connection and its other streams go on
gets `on_done(why)`	once for every `on_request`, as before: `PROVEN_OK` when the whole response was handed over, `PROVEN_ERR_RESET` when the client reset the stream or the connection ended, `PROVEN_ERR_TIMEOUT`, `PROVEN_ERR_OUT_OF_BOUNDS` for a body over the limit

`proven_http_stream_protocol` answers 1 or 2 (3 for HTTP/3, section 9), for a log line; a handler seldom has another reason to ask.

Which protocol a connection speaks

- **Over TLS, the handshake says** (ALPN). The TLS configuration must offer `h2` - and `http/1.1` after it, for clients that have nothing newer - when `http2` is on, and must not offer `h2` when it is off: the server refuses at creation a pair that could agree in the handshake to something it would then not speak. A connection that negotiated `h2` on TLS older than 1.2, or on a 1.2 suite without forward secrecy and authenticated encryption - possible only with the legacy options of Chapter 14 switched on - is ended with `INADEQUATE_SECURITY`, as RFC 9113 requires.
- **0-RTT, when the TLS configuration has** `early_data` (chapter 14, section 6; v0.84.0), as for HTTP/1.1 and HTTP/3: a request whose HEADERS arrived in the client's early data reaches its handler before the handshake is over, with `proven_http_stream_early` true, and may be answered at once; only GET, HEAD and OPTIONS without a body get that far - anything else that arrives early is answered 425 on its stream by the server.
- **In the clear, the first bytes say.** A client that knows it wants HTTP/2 begins with the 24-byte preface (curl's `--http2-prior-knowledge`); anything else is HTTP/1.1. The server decides at the first byte that differs, so `POST`, which shares the preface's first letter, is HTTP/1.1 at its second. `Upgrade: h2c` is not offered: RFC 9113 removed it.

Both protocols are served on one port, by one handler, at the same time.

Limits and time

Setting	For HTTP/2
`max_connections`	Connections, as before: an HTTP/2 connection is one, however many streams it carries
`max_concurrent_streams` (0: 100)	Requests in progress on one connection. A client is told the number and a stream beyond it is refused
`max_head_bytes`, `max_headers`	The size of a request's header list as HPACK counts it; more fields than `max_headers` is `431`
`max_body_bytes`	Per request: `413`, sent while the body is still arriving, which tells the client to stop
`max_buffered_output`	What the connection holds for a client that is not reading, **for all its streams together**
`http2_window_bytes` (0: 65,535, growing)	Request-body bytes a client may send on a stream, and on the connection, before the handler has taken them: what a paused request can make the connection hold. With 0

	the window starts there and grows while bodies are read - doubling while half of it is taken within two round trips, up to 16 MiB a stream and 24 MiB a connection (v0.75.0, as HTTP/3's since v0.71.0); a body not read stops the growth. A value set stays fixed.
`head_timeout_ms`	From the connection's first byte to its first request
`body_timeout_ms`	Silence from one request whose body is owed: that stream is reset, the connection and its other streams go on
`http2_header_table_bytes` (0: 4,096), `http2_no_header_table`	The header compression table in each direction: what an idle connection may come to hold, against how much a repeated field costs to send
`write_timeout_ms`	Output held with none of it accepted: the connection ends
`idle_timeout_ms`	No stream open: the server says goodbye (GOAWAY) and closes

```
proven_err_t proven_http_event_server_set_max_streams(proven_http_event_server_t *server, proven_size_t n);
proven_err_t proven_http_event_server_set_window(proven_http_event_server_t *server, proven_size_t n);
```

`max_concurrent_streams` can be changed while the server runs (v0.79.0), from the loop's thread: 1 to the configured value, which stays the ceiling. Each HTTP/2 connection says the new number to its client (section 5's `proven_h2_conn_set_max_streams`), and a connection made later opens with it. Requests already running go on; until a client acknowledges, it may still begin requests under the number before. HTTP/3 has its own rule (section 9). To take no new requests at all, shut the server down instead: 0 is `PROVEN_ERR_INVALID_ARG`.

`proven_http_event_server_set_window` (v0.82.0) does the same for request bodies: from the loop's thread, `n` becomes a ceiling on what a client may send ahead of the handler, on a stream and on a connection, HTTP/2 and HTTP/3, for the connections open and those to come. Room is given back only up to it - what the client may still send and what the server holds unread, together - and windows that grow (`http2_window_bytes` left 0) stop at it, growing again when it is raised. A configured window stays the most a protocol gives. What was granted stands: an HTTP/2 window said lower holds once the client acknowledges it, and HTTP/3 credit cannot be taken back, so a lower ceiling shows as what is outstanding is used up. Each HTTP/2 connection says `min(n, its window)` as its initial window (section 5's `proven_h2_conn_set_initial_window`). On an HTTP/3 connection open now a stream the client opens later may still use the window promised in the handshake; connections made later are promised at most `n`.

`proven_http_event_server_stop_listening` is the graceful way to end: each HTTP/2 connection is sent GOAWAY, finishes the streams it has, and closes. `proven_http_event_server_destroy` ends everything at once, and every open stream's handler gets `on_done(PROVEN_ERR_RESET)`.

Example: one port, one handler, both protocols

```
#include <stdio.h>
#include <string.h>

/*
 * One server, one port, both protocols: the event-driven HTTP server with `http2` on.
 *
```

```
 * The handler below is an ordinary one - the same callbacks, the same stream calls - and does
 * not know which protocol a request came by unless it asks. An HTTP/1.1 client and an HTTP/2
 * client then talk to it on the same port. The HTTP/2 client here is the connection engine of
 * this chapter's section 5 with a socket put through it, to show what one is made of: forty
 * lines. (The library's own, which keeps connections and shares them, is section 7's.)
 */

typedef struct { proven_loop_t *loop; int served, over_http2; } ex_app_t;

static void on_request(void *ctx, proven_http_stream_t *stream, const proven_http_request_t *head) {
    ex_app_t *app = ctx;
    /* The request looks the same either way: a method, a target, fields - Host among them,
     * which HTTP/2 carries as `:authority` and the server hands on under its usual name. */
    /* With `early_data` in the TLS configuration, a request may come as early data (chapter 14,
     * section 6) and may be a copy an attacker replayed. This answer is safe to give
     * twice; a handler whose request is not answers 425 and the client sends it again later. */
    if (proven_http_stream_early(stream) && head->method != PROVEN_HTTP_GET) {
        (void)proven_http_stream_respond(stream, 425, NULL, 0, (proven_mem_view_t){ 0 });
        return;
    }
    const int protocol = proven_http_stream_protocol(stream);
    char text[96];
    int n = snprintf(text, sizeof text, "%.*s over HTTP/%d\n", (int)head->target.size, (const char *)head-
>target.ptr, protocol);
    proven_http_header_t fields[] = { { PROVEN_LIT("Content-Type"), PROVEN_LIT("text/plain") } };
    if (proven_http_stream_respond(stream, 200, fields, 1, (proven_mem_view_t){ (const proven_byte_t *)text,
(proven_size_t)n }) != PROVEN_OK) return;
    if (protocol == 2) app->over_http2++;
}

static void on_done(void *ctx, proven_http_stream_t *stream, proven_err_t why) {
    (void)stream;
    if (why == PROVEN_OK) ((ex_app_t *)ctx)->served++;
}

static void run(void *arg) { (void)proven_loop_run(((ex_app_t *)arg)->loop); }

/* An HTTP/2 client in miniature: the engine, a socket, and a loop that moves bytes between
 * them until every stream it asked for has ended. Returns how many were answered 200. */
static int ask_three(proven_u16 port, char answers[3][64]) {
    proven_net_conn_t sock;
    if (proven_net_connect(proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, port),
proven_net_deadline_in(5000), &sock) != PROVEN_OK) return -1;

    /* The engine reads header lists into memory it allocates while streams are open - or into
     * memory of yours, which several connections on one thread may share. */
    static proven_u64 header_room[16384];
    proven_h2_config_t config = { .alloc = proven_heap_allocator(), .header_buffer = { (proven_byte_t
*)header_room, sizeof header_room } };
    proven_h2_conn_t *conn = NULL;
    if (proven_h2_header_buffer_size(0) > sizeof header_room || proven_h2_conn_create(&config, &conn) !=
PROVEN_OK) { (void)proven_net_close(&sock); return -1; }

    /* Three requests at once, on the one connection: streams 1, 3 and 5. */
    static const char *const paths[3] = { "/first", "/second", "/third" };
    for (int i = 0; i < 3; i++) {
        proven_http_header_t request[] = {
            { PROVEN_LIT(":method"), PROVEN_LIT("GET") }, { PROVEN_LIT(":scheme"), PROVEN_LIT("http") },
            { PROVEN_LIT(":authority"), PROVEN_LIT("localhost") }, { PROVEN_LIT(":path"),
proven_u8str_view_from_cstr(paths[i]) },
```

```
        };
        proven_u32 stream = 0;
        if (proven_h2_conn_submit_request(conn, request, 4, true, &stream) != PROVEN_OK)
{ proven_h2_conn_destroy(conn); (void)proven_net_close(&sock); return -1; }
    }

    int ended = 0, ok = 0;
    static proven_byte_t in[16384];
    while (ended < 3) {
        /* Whatever the engine has queued - the preface and settings, the requests, later its
         * acknowledgements - goes to the socket first. */
        proven_mem_view_t out = proven_h2_conn_pending_output(conn);
        if (out.size > 0) {
            if (proven_net_write_all(&sock, out, proven_net_deadline_in(5000)).err != PROVEN_OK) break;
            proven_h2_conn_output_sent(conn, out.size);
        }
        proven_result_size_t got = proven_net_read(&sock, (proven_mem_mut_t){ in, sizeof in },
proven_net_deadline_in(5000));
        if (got.err != PROVEN_OK) break;
        proven_size_t pos = 0;
        for (;;) {
            proven_size_t used = 0;
            proven_h2_event_t ev;
            if (proven_h2_conn_feed(conn, (proven_mem_view_t){ in + pos, got.value - pos }, &used, &ev) !=
PROVEN_OK) { ended = 3; break; }
            pos += used;
            const proven_u32 k = ev.stream_id ? (ev.stream_id - 1) / 2 : 0;     /* stream 1, 3, 5 -> 0, 1, 2
*/
            if (ev.kind == PROVEN_H2_EVENT_HEADERS && ev.status == 200 && k < 3) ok++;
            if (ev.kind == PROVEN_H2_EVENT_DATA && k < 3) {
                snprintf(answers[k], 64, "%.*s", (int)(ev.data.size < 63 ? ev.data.size : 63), (const char
*)ev.data.ptr);
                proven_h2_conn_consumed(conn, ev.stream_id, ev.data.size);
            }
            if ((ev.kind == PROVEN_H2_EVENT_DATA || ev.kind == PROVEN_H2_EVENT_HEADERS) && ev.end_stream)
ended++;
            if (ev.kind == PROVEN_H2_EVENT_STREAM_RESET || ev.kind == PROVEN_H2_EVENT_CLOSED) { ended = 3;
break; }
            if (ev.kind == PROVEN_H2_EVENT_NONE && pos == got.value) break;
        }
    }
    proven_h2_conn_destroy(conn);
    (void)proven_net_close(&sock);
    return ok;
}

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    ex_app_t app = { 0 };
    EXAMPLE_REQUIRE(proven_loop_create(heap, &app.loop) == PROVEN_OK, "a loop");

    /* `http2` is the whole of it. In the clear, a client that begins with the HTTP/2 preface
     * is HTTP/2 and every other is HTTP/1.1; over TLS the handshake decides (ALPN), and the
     * TLS configuration must then offer "h2". */
    proven_http_event_server_config_t config = { .on = { .on_request = on_request, .on_done = on_done }, .ctx
= &app, .http2 = true };
    proven_http_event_server_t *server = NULL;
    proven_net_addr_t at;
    EXAMPLE_REQUIRE(proven_http_event_server_create(app.loop, &config, &server) == PROVEN_OK &&
                    proven_http_event_server_listen(server, proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4,
```

```
0), &at) == PROVEN_OK, "a server that speaks both");
    /* Under load a server may ask for fewer requests at once on each connection, and raise the
     * number again later, up to the configured one (here the default, 100). From the loop's
     * thread, or before it runs, as here. */
    EXAMPLE_REQUIRE(proven_http_event_server_set_max_streams(server, 8) == PROVEN_OK &&
proven_http_event_server_set_max_streams(server, 0) == PROVEN_ERR_INVALID_ARG,
                    "eight requests at once from now on, and never none");
    /* And fewer bytes of request body held for a client ahead of the handler: 64 KiB per stream
     * and per connection at most, from now on - a ceiling on what is given, not taken back. */
    EXAMPLE_REQUIRE(proven_http_event_server_set_window(server, 64 * 1024) == PROVEN_OK, "64 KiB of body at
most ahead of the handler");
    proven_job_sys_t *threads = NULL;
    proven_job_group_t running;
    proven_job_group_init(&running);
    EXAMPLE_REQUIRE(proven_job_system_init(heap, 1, 4, &threads) == PROVEN_OK &&
                    proven_job_group_submit(threads, &running, run, &app) == PROVEN_OK, "the loop runs on
another thread");

    // ---- an HTTP/1.1 client ------------------------------------------------------------------
    char url[64];
    snprintf(url, sizeof url, "http://127.0.0.1:%u/hello", (unsigned)at.port);
    proven_http_client_config_t client_config = { .alloc = heap, .max_idle_connections = 1 };
    proven_http_client_t *client = NULL;
    proven_http_client_response_t resp;
    proven_u8str_t body = { 0 };
    EXAMPLE_REQUIRE(proven_http_client_create(&client_config, &client) == PROVEN_OK &&
                    proven_http_client_get(client, proven_u8str_view_from_cstr(url), &resp) == PROVEN_OK &&
resp.status == 200 &&
                    proven_http_client_read_all(&resp, heap, &body, 4096) == PROVEN_OK, "the blocking client
of chapter 11 is answered");
    printf("HTTP/1.1 client: %.*s", (int)proven_u8str_as_view(&body).size, (const char
*)proven_u8str_as_view(&body).ptr);
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&body), PROVEN_LIT("/hello over HTTP/1\n")),
"over HTTP/1.1");
    proven_u8str_destroy(heap, &body);
    proven_http_client_finish(&resp);
    proven_http_client_destroy(client);

    // ---- an HTTP/2 client, on the same port --------------------------------------------------
    char answers[3][64] = { "", "", "" };
    EXAMPLE_REQUIRE(ask_three(at.port, answers) == 3, "three requests on one HTTP/2 connection are each
answered");
    for (int i = 0; i < 3; i++) printf("HTTP/2 client:   %s", answers[i]);
    EXAMPLE_REQUIRE(strcmp(answers[0], "/first over HTTP/2\n") == 0 && strcmp(answers[1], "/second over
HTTP/2\n") == 0 && strcmp(answers[2], "/third over HTTP/2\n") == 0,
                    "by the same handler, which was told the protocol was 2");

    // ---- the end -----------------------------------------------------------------------------
    /* Stop the loop, then look at what the handler counted and take the server apart: nothing
     * else is touching either by then. */
    proven_loop_stop(app.loop);
    proven_job_group_wait(threads, &running);
    EXAMPLE_REQUIRE(app.served == 4 && app.over_http2 == 3, "four exchanges ended with on_done(PROVEN_OK),
three of them over HTTP/2");
    proven_http_event_server_destroy(server);
    proven_loop_destroy(app.loop);
    proven_job_system_close(threads);
    proven_job_system_destroy(threads);
    return EXAMPLE_OK();
}
```

Cautions, and what goes wrong

- **A handler that keeps its place across `on_writable` is safe, as it was.** `on_writable` is never called from inside a call the handler itself made - a write that called back into the same handler would send the same bytes twice. This is the server's promise for both protocols; on HTTP/2 it took a defect to find (section 8).
- `max_buffered_output` **is per connection, not per stream.** One slow client with a hundred streams holds what one HTTP/1.1 client would, not a hundred times it.
- **A client that does not read is not read from.** A response's head is always queued - the server has nowhere else to keep it - so what bounds a client that asks and never reads is that, while twice `max_buffered_output` waits for it, the server takes nothing more from that connection: further requests stay in the socket, as a pipelined request does on HTTP/1.1 while a response is owed. The most a connection then holds as output is twice `max_buffered_output` and one head for each stream that was open - at most `max_concurrent_streams` of them, each the size its handler made it. A client that uploads without reading its other responses therefore stops being read, and resumes when it reads.
- **An idle HTTP/2 connection is larger than an HTTP/1.1 one, by what it has been sent.** HTTP/1.1's is 480 bytes. HTTP/2's is 2,360 bytes after one small request, and grows with the header fields its client has made it remember, to about 12.8 KiB when both compression tables are full: ten thousand idle connections are between 22 and 128 MiB. `http2_header_table_bytes` lowers the ceiling and `http2_no_header_table` removes it (under 2 KiB, always; on Windows x64, whose loop socket holds an OVERLAPPED, 2,048 bytes), at the price of sending every field in full every time. The buffer for reading header lists - 80 KiB - is one for the whole server, not one each. A request in progress adds about 210 bytes: 160 in the server, 48 in the connection.
- **A handler that answers before the request has ended is done with it.** The server tells the client to stop sending (section 5), and what was already on its way is dropped unread - including a frame that would have been an error. If the body matters, answer when it has arrived.
- **WebSocket needs HTTP/1.1.** An upgrade is refused on an HTTP/2 stream (`PROVEN_ERR_UNSUPPORTED`): a browser that wants a WebSocket opens an HTTP/1.1 connection for it, which the same port accepts.
- **Trailers are not passed on**, in either direction, as they are not on HTTP/1.1: a request's trailers end its body and are dropped.

7. HTTP/2 in the event-driven client

```
typedef struct {
    /* ... everything Chapter 15 describes ... */
    bool http2;
    bool http2_cleartext;
    proven_u32 idle_timeout_ms;
    proven_size_t max_waiting_requests;
} proven_http_event_client_config_t;

int proven_http_event_request_protocol(const proven_http_event_request_t *request);
```

The event-driven client of Chapter 15 is, for HTTP/1.1, one connection to a request. With `http2` it **keeps a connection for each origin that agrees to HTTP/2 and puts every request to that origin on it**: a hundred requests to one server are one handshake, not a hundred. The callbacks and the `proven_http_event_request_*` calls are the same; so is what `start` promises - everything the options point to is copied or used before it returns, and `on_done` comes exactly once, never from inside `start`.

- **Over TLS**, `http2` and a TLS configuration that offers `h2` (and `http/1.1` after it): the handshake with each origin decides. An origin that chooses HTTP/1.1 is remembered as such and served one connection to a request, as before. (The last 64 such origins are remembered, at about 600 bytes

each; one that has been forgotten costs a handshake to learn again.) With `early_data` in the TLS configuration and a session made over `h2`, a new connection begins HTTP/2 inside its first flight: the preface, the settings and one request that is GET, HEAD or OPTIONS without a body go as early data (chapter 14, section 6; v0.84.0), the rest after the handshake. What the server refused goes again, as it was; a 425 sends the request once more on the same connection, now that it is up.

- **In the clear**, `http2_cleartext`: every `http` request of that client is HTTP/2 by prior knowledge. There is no way to ask a server in the clear, so say this only of servers you know; a client that talks to both kinds in the clear is two clients. Said of a server that does not speak it, it ends every request with `PROVEN_ERR_UNSUPPORTED`: the connection was made, and it ended - on bytes that were not HTTP/2, or with none at all - before the server's settings, which are the first sign that a server in the clear is an HTTP/2 server. Nothing is sent ahead of them, so no such request was acted on; but starting it again would end the same way, which is why the answer is not `PROVEN_ERR_AGAIN`. (It was, until v0.35.0.) A server that was only restarting at that moment gets the same answer: if you know that it speaks HTTP/2, start the request again.

A request may wait, where an HTTP/1.1 request never did: for the connection, while it is being made; for the server's first word, which says how many streams it allows - nothing is sent ahead of that, so nothing is refused for it; and for a free stream, when the server's limit is reached. A waiting request holds a copy of what it was started with. At most `max_waiting_requests` (1,000) wait for one connection; the next `start` is `PROVEN_ERR_BUSY`. And, with `max_connections` set and every connection in use, a request to an origin that has no connection waits for the client to have room for one - on no connection, as many again at most, and for `connect_timeout_ms` at most (Chapter 15).

What `on_done` says is what it said, with two more answers. `PROVEN_ERR_AGAIN`, for a request that an HTTP/2 server did not act on - it said goodbye first, or refused the stream, or the connection was lost before the request was sent. Such a request may be started again, whatever its method; the client does not do so by itself unless `retry_unsent` says how many times it may (below). `PROVEN_ERR_UNSUPPORTED`, for a request to a server in the clear that turned out not to speak HTTP/2 (`http2_cleartext`, above): not acted on either, and not worth starting again. `PROVEN_ERR_RESET` still means "it may have been acted on" - but a server that resets a stream with `NO_ERROR` once every byte of the response's declared `content-length` has arrived has answered in full, and that is `PROVEN_OK`, with the whole body (section 5 says which resets count). Over TLS there is no `PROVEN_ERR_UNSUPPORTED`: the handshake settles the protocol, and a server that agrees to HTTP/1.1 is spoken to in HTTP/1.1.

Setting	For HTTP/2
`connect_timeout_ms`	Making an origin's connection - the handshake, and the server's SETTINGS, which are what makes it an HTTP/2 connection - and every request waiting for it ends with it (`PROVEN_ERR_TIMEOUT`)
`response_timeout_ms`, `body_timeout_ms`	Each request's own: a request that is not answered ends, the connection and its other requests go on
`write_timeout_ms`	The connection's: output the server takes none of ends it
`idle_timeout_ms` (0: 60 s)	A connection with no request on it is kept this long for the next, then closed
`max_buffered_output`	For the connection's requests together
`decompress`	Unchanged; a paused request holds what arrived compressed, and decodes it on resume
`decompress_zstd`	The same for `zstd`: asked for in `accept-encoding`, decoded on the stream

decompress_brotli	The same for `br`

Example: three requests, one connection

```
#include <stdio.h>
#include <string.h>

/*
 * The event-driven client speaking HTTP/2: three requests, one connection.
 *
 * Client and server sit on one loop here, driven by hand, so the example needs no thread. The
 * client's callbacks are the ones an HTTP/1.1 program has; what differs is underneath - the
 * client keeps a connection for the origin and puts every request to it on that connection.
 */

typedef struct { int answered, over_http2; char text[3][32]; bool done[3]; } ex_seen_t;
typedef struct { ex_seen_t *seen; int which; } ex_call_t;

static void serve(void *ctx, proven_http_stream_t *stream, const proven_http_request_t *head) {
    (void)ctx;
    (void)proven_http_stream_respond(stream, 200, NULL, 0, (proven_mem_view_t){ head->target.ptr, head-
>target.size });     /* its own path, back */
}

static void on_response(void *ctx, proven_http_event_request_t *request, const proven_http_response_t *head) {
    ex_call_t *call = ctx;
    /* 2 here; 1 for a request that is still waiting for its connection, or whose server
     * turned out to speak only HTTP/1.1. */
    if (head->status == 200 && proven_http_event_request_protocol(request) == 2) call->seen->over_http2++;
}

static void on_body(void *ctx, proven_http_event_request_t *request, proven_mem_view_t piece, bool last) {
    (void)request; (void)last;
    ex_call_t *call = ctx;
    snprintf(call->seen->text[call->which], sizeof call->seen->text[0], "%.*s", (int)piece.size, (const char
*)piece.ptr);
}

static void on_done(void *ctx, proven_http_event_request_t *request, proven_err_t why) {
    (void)request;
    ex_call_t *call = ctx;
    call->seen->done[call->which] = true;
    if (why == PROVEN_OK) call->seen->answered++;
}

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    proven_loop_t *loop = NULL;
    EXAMPLE_REQUIRE(proven_loop_create(heap, &loop) == PROVEN_OK, "a loop");

    /* A server that speaks both protocols (section 6). */
    proven_http_event_server_config_t server_config = { .on = { .on_request = serve }, .http2 = true };
    proven_http_event_server_t *server = NULL;
    proven_net_addr_t at;
    EXAMPLE_REQUIRE(proven_http_event_server_create(loop, &server_config, &server) == PROVEN_OK &&
                    proven_http_event_server_listen(server, proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4,
0), &at) == PROVEN_OK, "a server");

    /* In the clear there is no handshake to ask in, so the client is told: every `http`
     * request of this client is HTTP/2. Over TLS it is `http2` instead, with a TLS
```

```
     * configuration that offers "h2", and the handshake decides for each origin. */
    proven_http_event_client_config_t client_config = { .alloc = heap, .http2_cleartext = true };
    proven_http_event_client_t *client = NULL;
    EXAMPLE_REQUIRE(proven_http_event_client_create(loop, &client_config, &client) == PROVEN_OK, "a client");

    ex_seen_t seen = { 0 };
    ex_call_t calls[3] = { { &seen, 0 }, { &seen, 1 }, { &seen, 2 } };
    static const char *const paths[3] = { "/first", "/second", "/third" };
    for (int i = 0; i < 3; i++) {
        char url[64];
        snprintf(url, sizeof url, "http://127.0.0.1:%u%s", (unsigned)at.port, paths[i]);
        proven_http_event_request_options_t request = {
            .url = proven_u8str_view_from_cstr(url),
            .on = { .on_response = on_response, .on_body = on_body, .on_done = on_done }, .ctx = &calls[i],
        };
        /* Started at once, before there is a connection: the first makes it, the others wait
         * for it, and all three go out together when the server has said what it allows. */
        EXAMPLE_REQUIRE(proven_http_event_client_start(client, &request, NULL) == PROVEN_OK, "a request is
started");
    }
    EXAMPLE_REQUIRE(proven_http_event_client_requests(client) == 3, "three are under way");

    proven_net_deadline_t until = proven_net_deadline_in(5000);
    while (!(seen.done[0] && seen.done[1] && seen.done[2]) && proven_time_monotonic_now() < until)
        EXAMPLE_REQUIRE(proven_loop_poll(loop, proven_net_deadline_in(10)) == PROVEN_OK, "a round of the
loop");

    for (int i = 0; i < 3; i++) printf("%s\n", seen.text[i]);
    EXAMPLE_REQUIRE(seen.answered == 3 && seen.over_http2 == 3, "all three were answered, over HTTP/2");
    EXAMPLE_REQUIRE(strcmp(seen.text[0], "/first") == 0 && strcmp(seen.text[1], "/second") == 0 &&
strcmp(seen.text[2], "/third") == 0, "each with its own answer");
    EXAMPLE_REQUIRE(proven_http_event_server_connections(server) == 1, "and the server saw one connection, not
three");

    proven_http_event_client_destroy(client);
    proven_http_event_server_destroy(server);
    proven_loop_destroy(loop);
    return EXAMPLE_OK();
}
```

Cautions, and what goes wrong

- **A request's object does not change under you.** One that was started while its origin was unknown, and whose origin turned out to speak HTTP/1.1, is still the pointer `start` gave you; `proven_http_event_request_protocol` then answers 1.
- **A request that waits can time out before it is sent**: `connect_timeout_ms` covers the connection it is waiting for. One that waits for a free stream has no clock of its own - it is the requests ahead of it that are being timed.
- **Pause holds at most one window**, as on the server: the client stops telling the server that it has consumed, and the server stops sending.
- **`PROVEN_ERR_AGAIN` is not retried for you, unless you ask.** `retry_unsent` is a number: how many times the client itself puts to the server again a request that would end that way - one behind a goodbye, one whose stream was refused, one waiting on a connection that was lost. It goes on the same connection when one stream was refused and the connection goes on, and otherwise on another for the same origin, which is made if there is none; requests that move together keep their order; you are told nothing of the try that failed, and `on_done` comes once, for the last. Zero, the default, is none: the caller decides. What the setting does not cover, and what it costs:

    - **Not a request whose body is written in pieces** - the client does not have the pieces to send again - **and not after** `on_response`: you have seen an answer begin.
    - **Not** `PROVEN_ERR_UNSUPPORTED`, which would end the same way again, **nor** `PROVEN_ERR_RESET`, which the server may have acted on, nor a time limit.
    - **At once, with no delay.** A server that refuses a stream because it is busy is asked again in the same moment; a small number is a few moments. A program that wants to wait between tries leaves the setting at zero and starts the request again itself.
    - **The time limits are each try's**: a request may take `connect_timeout_ms` or `response_timeout_ms` once for every try it has.
    - **A body held in memory stays in memory until the response begins**, not only until it has been sent.
    - **A try that fails for another reason ends the request with that reason.** If the new connection cannot be made, `on_done` says `PROVEN_ERR_REFUSED`, not `PROVEN_ERR_AGAIN`.
- **One connection is one point of failure.** When it ends, every request on it ends with it - `PROVEN_ERR_AGAIN` for those not yet sent, `PROVEN_ERR_RESET` for the others (and `PROVEN_ERR_UNSUPPORTED` for all of them when, in the clear, it never was HTTP/2).

8. What is accepted, what is not here, and how this was tested

What the decoder accepts is HPACK as RFC 7541 defines it. Any number of table size updates at the start of a block (the RFC promises at most two); integers of up to five continuation bytes; a literal's Huffman padding of up to seven one bits and nothing else.

What the encoder writes is valid HPACK that makes the common choices: a table reference where there is one, Huffman coding where it is shorter (not where it is the same length), new fields added to the table unless they are secrets or larger than it.

What a connection accepts is RFC 9113, with these choices where the RFC leaves one: the deprecated priority fields are ignored, except that a stream said to depend on itself is an error of that stream, as RFC 7540 had it; a frame for a stream that this side reset is dropped without comment for as long as the reset is among the last `max_concurrent_streams` endings it remembers; a header list over the limit is an error of its stream, not of the connection; `content-length` that disagrees with the body is an error of the stream; a response to `HEAD`, and one with status 204 or 304, may announce a length it does not carry.

Not here:

- **HTTP/2 in the blocking server and the blocking client** of Chapter 11: one thread to a connection suits one exchange at a time, which is HTTP/1.1.
- **A retry with a delay, or of anything but what the server did not act on**: the event-driven client reports such a request (`PROVEN_ERR_AGAIN`) and, when `retry_unsent` is set, puts it again at once, so many times. Waiting between tries, and trying again what may have been acted on, are its caller's.
- **Trailers in the event-driven server**, for either protocol.
- **Extended CONNECT** (RFC 8441, WebSocket over HTTP/2) and the `SETTINGS` it needs.
- **Server push**: `PUSH_PROMISE` is recognised so that it can be refused.
- **Priorities** (RFC 7540's scheme, which RFC 9113 deprecated): skipped.
- `Upgrade: h2c`, which RFC 9113 removed. (HTTP/3 is sections 9 and 10.)
- **The never-indexed mark on a decoded field**: a proxy would pass it on; nothing here does.

How this was tested. The static table and the Huffman code are not typed in: a private program reads them from the published text of RFC 7541, and stops unless there are 61 entries and 257 codes that are prefix-free and fill the code space exactly; the registered test checks the same of the table it is linked with. That test then replays all sixteen examples of the RFC's Appendix C through

the decoder, comparing the dynamic table with the RFC's after each; has the encoder reproduce, byte for byte, the examples whose choices it shares; refuses thirteen kinds of malformed block; holds each limit; and decodes every beginning and every single-bit change of the RFC's blocks. A second test tries each frame type's rule on both sides of its edge and thirty-one malformed requests. Outside the registered tests: this decoder read 47,142 blocks that fourteen other implementations produced for the same header lists (nghttp2, Go, Node, Python, Haskell and Swift encoders, including sequences in which the table size changes); Node's HTTP/2 server, which is nghttp2, decoded every block this encoder wrote for 1,290 requests on shared connections; and five million damaged blocks were decoded under tools that report any read or write outside a buffer, with none reported.

The connection has a registered test of its own. Two connections are wired together in memory and carry bodies of nine sizes through three window sizes, a byte at a time and in bulk, fifty streams at once, and a head of 40,000 bytes that leaves as three frames. A hundred and one scripted conversations each break one rule of RFC 9113, or stand just inside it, and the connection is held to the verdict the RFC names - ignored, an error of the stream with its code, an error of the connection with its code - once with the bytes in one piece and once a byte at a time. Each bound of section 5 is driven to its edge from both sides. A recorded conversation is replayed 12,214 times to a fresh connection in each role - cut short at every point near its start and at intervals after, or with one bit changed; and every allocation of a conversation is made to fail once. Outside the registered tests, through a private program that moves a socket through the connection: Node's HTTP/2 client and server (nghttp2) and curl exchanged requests, bodies up to 5 MB, heads of 20,000 bytes, trailers, interim responses, cancellations and settings changes with it in both roles, under twelve combinations of settings - 7,896 checks, none failed; and 5,000,230 random frames over 140,108 connections, with an application acting at random, broke none of the connection's invariants under the same tools.

The server of section 6 has its own registered test: the handlers of an HTTP/1.1 server, unchanged, answering a client made of this library's connection over real sockets - in the clear, over TLS 1.3 and over TLS 1.2 - with bodies both ways, sixty streams at once, compression, pause, resets from either side, a client that closes its sending side and is still answered, limits, each timeout, goodbye, and HTTP/1.1 clients on the same port throughout. Outside the registered tests, a private program serves both protocols on one port to Node's http2, http and https clients and to curl in five modes: 7,392 checks, none failed in the end. **One did at first**: two large responses side by side to Node over TLS stalled, because the server called `on_writable` from inside the handler's own write and the handler, re-entered, lost its place. This library's own client had not provoked it. The registered test now holds the handler to "never re-entered", with two streams racing.

h2spec, the conformance suite for HTTP/2 servers (version 2.6.0, 146 cases), was run against that server: 146 of 146 over TLS, and 145 of 146 in the clear. The one is "an invalid connection preface must end the connection": in the clear, bytes that are not the preface are not HTTP/2 here but HTTP/1.1, and are answered as such. Its first run found two things: a handler had no way to tell that a body with no announced length was coming (it now sees `Transfer-Encoding: chunked`), and a stream that depends on itself was not refused.

The client of section 7 has a registered test against the server of section 6 on one loop - in the clear and over TLS: forty requests on one connection, bodies both ways, compression, pause, abort, the server's limit on streams, each timeout, an idle connection let go, and an origin that turns out to speak only HTTP/1.1, whose waiting requests are served as HTTP/1.1 by the objects their caller already held. Outside the registered tests, a private program ran the client against Node's HTTP/2 servers over TLS and in the clear under four sets of server settings, and against an HTTPS server with no HTTP/2: 1,156 checks, none failed in the end. **Fifteen did at first**: against a server that

allows four streams, a burst of requests on a new connection went out before the server had said so, and the surplus came back refused. The client now sends nothing ahead of the server's settings.

What the registered tests did not find, once more. When the server was first made to hold its output for the length of a read (the table below is what that bought), a burst of whole-body responses filled the connection's output and the responses that began after them were reset for want of room. Every registered test passed. Node's client, in the private checks, had 945 of 7,392 fail. The hold now ends when a quarter of `max_buffered_output` is queued, a whole body leaves the room that heads need, and a registered test sends a hundred requests in one write to a server whose bound is smaller than their heads; the private checks were then run again in full - 7,896, 7,392 and 1,156 against Node and curl, h2spec as before - and none failed.

Speed, measured once, with no socket: two connections in memory on one core of a 3.5 GHz x86-64 processor, GCC `-O2`. A request head of twelve fields (299 bytes as literals) is Huffman-decoded 680,000 times a second, 204 MB/s; 385,000 requests a second are sent, decoded, answered with a head and read back, a hundred in flight; and bodies cross at 2.4 GB/s. These are the protocol's costs alone - a socket, TLS and a handler all cost more.

Speed over a socket, measured once: the load of Chapter 15, section 14, made again for HTTP/2 against the same server on the same machine - a load program that is not part of the test suite, client and server in two processes over the loopback interface, in the clear, one loop, a handler that answers two bytes, ten seconds a row. Every connection makes one request and is left open; then some of them keep a number of requests in flight. The HTTP/1.1 rows were made the same day against the same server process, for comparison.

Protocol	Connections held	Making requests	In flight on each	Requests per second	Round trip: median, 99th percentile
HTTP/1.1	1,000	10	1	176,000	0.04 ms, 0.12 ms
HTTP/2	1,000	10	1	153,000	0.05 ms, 0.16 ms
HTTP/2	1,000	10	10	821,000	0.1 ms, 0.2 ms
HTTP/2	1,000	10	100	1,368,000	0.6 ms, 1.4 ms
HTTP/1.1	10,000	100	1	188,000	0.5 ms, 0.9 ms
HTTP/2	10,000	100	1	165,000	0.6 ms, 0.9 ms
HTTP/2	10,000	100	10	820,000	1.2 ms, 2.0 ms

What the table says. **One request at a time, HTTP/2 is the slower by about an eighth**: each request is still one read and one write, and there is more to do for it between them. **With requests in flight together it is several times the faster**, because the server works through everything one read brought before it writes: a hundred requests that arrive together are answered in one write, and of the server's ten seconds the system's share falls from 7.3 to 0.8. The round trip grows with the number in flight, as it must: a hundred requests wait their turn on one loop. No connection and no request was lost in any row. In the server an idle HTTP/2 connection that had made its one request held 2,272 bytes (1,920 with `http2_no_header_table`), against 448 for HTTP/1.1, and the buffer for reading header lists, 82 KiB, is one for the whole server.

The library's own client (section 7), against the same server: 471,000 to 486,000 requests a second in three runs with a hundred in flight on its one connection - the two processes take turns there, the client's the longer one, the server busy about four seconds in ten - and 42,000 a second one at a time, 21 microseconds each at the median. (It was 403,000 with a hundred in flight until the

connection found a stream by its number instead of walking its table, and the client read a request's URL once instead of twice.) The same client over HTTP/1.1 did 22,500 a second while it made a connection for every request; since v0.36.0 it keeps them (Chapter 15, section 12) and does 129,000 a second with a hundred requests in flight on its six connections, and 44,000 one at a time on one. **Loopback is not a network**, and a handler that does real work will be slower than one that answers "ok".

Not done: no coverage-guided fuzzing; no measurement over a real network, with TLS, or with large bodies under load; no browser has talked to the server; no external review. A connection finds a stream in one step by a map from its number, which holds two slots for every place in the stream table; a stream that arrives to find its slot taken - by one open far longer than its neighbours - is found by walking the table, and a peer that arranges that on purpose makes the connection walk its table as it did before there was a map, and do nothing worse.

9. HTTP/3 in the event-driven server

```
typedef struct {
    /* ... everything Chapter 15 describes, and section 6's fields ... */
    bool http3;
    bool http3_retry;
    proven_size_t http3_window_bytes;
    proven_size_t http3_header_table_bytes;
    bool http3_no_header_table;
} proven_http_event_server_config_t;
```

HTTP/3 (RFC 9114) is HTTP over QUIC (RFC 9000): a transport on UDP with its own streams, flow control, loss recovery and congestion control, and TLS 1.3 inside it rather than under it. Its header compression is QPACK (RFC 9204), HPACK's successor. What it gives over HTTP/2 is that a lost packet holds up only its own stream, and that the transport and TLS handshake take one round trip together. All of it is this library's own code.

With `http3` on, the server of section 6 speaks it too. **Handlers still do not change**: section 6's table holds for HTTP/3 as written for HTTP/2, with QUIC's flow control where it says the client's windows, and `proven_http_stream_protocol` answers 3. What is different is underneath:

- **Each listen opens a UDP socket as well**, on the same address and port as its TCP listener; a listen whose UDP socket cannot be had fails, and the TCP listener is closed again. At port 0 a UDP port already in use under the number TCP was given is not a failure: another number is tried, up to 17 in all. HTTP/3 is over TLS only: the TLS configuration must offer `h3` (beside `h2` and `http/1.1`), and must not when `http3` is off. Over QUIC the server agrees `h3` alone, whatever else the list offers over TCP and in whatever order (RFC 9114 section 3.1); a QUIC client that does not offer it ends with `no_application_protocol`. Before version 0.73.2 the server took its own first choice that the client offered, so this library's client and server agreed `h2` over QUIC and spoke HTTP/3 under it.
- **How a client finds it.** HTTP/1.1 and HTTP/2 responses carry `Alt-Svc: h3=":PORT"` - the port the connection came in on - unless the handler wrote an `Alt-Svc` of its own, which is sent instead. A browser, or `curl --alt-svc`, tries HTTP/3 on its next request; `curl --http3-only` asks for it at once.
- **One socket, many connections.** QUIC connections are told apart by their connection IDs, not by sockets: a datagram goes to the connection whose ID it carries, a client's first packet makes one, and a short packet for an ID nobody holds is answered with a stateless reset.
- **What bounds a write** is QUIC's send buffer - 256 KiB a stream and 1 MiB a connection of data sent and not yet acknowledged, or, within what the client has granted, twice the congestion window, up to 16 MiB and 24 MiB - not `max_buffered_output`. A refused write is told `on_writable` as on the other two.
- **A paused request** stops being read, and QUIC's flow control stops the client at `http3_window_bytes`: by default 256 KiB a stream and 1 MiB a connection to begin with, grown - doubling while bodies

are read as fast as they come - up to 16 MiB and 24 MiB; a body that is not read stops the growth, so a paused one holds no more than the window it had (v0.71.0). The send buffers grow with what the path carries (twice the congestion window), within what the peer has granted, up to the same caps.

Setting	For HTTP/3
`max_connections`	Counts QUIC connections, with TCP ones: at the limit, a client's first packet is not answered
`max_concurrent_streams`	Request streams a client may open at once on one connection. Changed while the server runs (`proven_http_event_server_set_max_streams`, section 6): what a client was allowed already cannot be taken back (RFC 9000 section 4.6), so a lower number shows once enough requests have ended; a higher one is given at once
`max_head_bytes`, `max_headers`	The largest field section, as QPACK counts it, said to the client in SETTINGS; more fields than `max_headers` is `431`
`max_body_bytes`	Per request: `413`, as on HTTP/2
`http3_window_bytes` (0: 256 KiB a stream, 1 MiB a connection, growing up to 16 MiB and 24 MiB while bodies are read)	Request-body bytes a client may send before the handler has taken them; a value set is fixed; at most 2^31-1
`http3_header_table_bytes`, `http3_no_header_table` (0: 4,096)	The QPACK dynamic table a client's encoder may fill (since v0.95.0, RFC-0032): said in SETTINGS with 16 streams whose heads may wait for its inserts - such a stream is not read until they come, and one more than that ends the connection with QPACK_DECOMPRESSION_FAILED. The table's memory - about twice its size - is made when a client first sets a capacity. 32 to 1 MiB. Since v0.96.0 the server's encoder fills the client's table in turn, when the client allows one, up to the smaller of the two sizes: it inserts the fields it sends - not those marked sensitive, nor `authorization`, `proxy-authorization` or `cookie` values under 20 bytes, nor one whose entry is over a quarter of the table - and refers only to entries the client has acknowledged, so the client never waits. A client that never counts inserts (aioquic's) is sent the inserts and no reference. `http3_no_header_table`: no table either way, as before
`http3_retry`	A Retry for every client's first packet: see the cautions
`head_timeout_ms`	From a connection's first packet - the handshake included - to its first request

`body_timeout_ms`	Silence from one request whose body is owed: that stream is reset, the others go on
`write_timeout_ms`	Output that does not move: the connection is closed
`idle_timeout_ms`	No request open: the server says goodbye and closes. A connection on which nothing at all arrives - a client gone - is let go a second after the larger of this and `head_timeout_ms` (QUIC's idle timeout)

Goodbye is two GOAWAYs a round trip apart (RFC 9114 section 5.2): the first names the largest request a client may still send, so that one already on its way is answered and not refused; the second names the first request not acted on, and the connection closes with H3_NO_ERROR when what it took is answered. `proven_http_event_server_stop_listening` does this; the UDP socket stays open while a connection uses it, and listening again on the same port takes it up. `proven_http_event_server_shutdown` and `_destroy` end HTTP/3 connections as they end HTTP/2 ones.

Cautions, and what goes wrong

- **UDP reaches the server before any handshake.** Anyone can send a client's first packet from any address, and without `http3_retry` each one makes a connection - about 40 KiB while its handshake waits, measured with a thousand from addresses that never answered - until `head_timeout_ms` lets it go. Such an address is never sent more than three times what it sent (RFC 9000 section 8), so the server cannot be used to flood someone else, but a flood can hold `max_connections` of them. `http3_retry` keeps nothing for an address until it has shown that it receives, at the price of a round trip for every connection.
- **The key for Retry tokens and stateless resets is drawn when the server is made.** A restarted server cannot reset a client's old connection; that client waits out its idle timeout.
- **Timers fire at their time** (within about a millisecond on POSIX, at the system timer's interval on Windows; v0.70.0): a quiet connection's QUIC timer - a probe, a delayed acknowledgement - no longer waits for the next 16 ms boundary; a busy one does its work in each round.
- **How fast HTTP/3 is depends on the platform.** On Linux the datagrams of a round go to the kernel together - up to 45 of 1,452 bytes in one call (`UDP_SEGMENT`), and a receive may hand several back (`UDP_GRO`); on Windows the same, through USO and URO (`UDP_SEND_MSG_SIZE`, `UDP_RECV_MAX_COALESCED_SIZE`; v0.74.0); everywhere else each datagram is one system call. Each is encrypted and protected on its own either way. On the Windows 11 test VM, this server to this client in one process over loopback, 256 MiB: 2,545 Mbit/s over HTTP/3 against 1,021 one call a datagram, and 1,919 over HTTP/2 (x86-64; i686 1,807, 845 and 1,627). One 100 MiB download from this server to curl on Linux loopback ran at 369 MiB/s over HTTP/3, 382 over HTTP/2 and 499 over HTTP/1.1, all over TLS (v0.68.0; 219 at v0.67.0, one call a datagram; 174 at v0.65.0).
- **Datagrams start at 1,200 bytes and grow by probing** (RFC 9000 section 14.3), up to 1,452 where the path and the peer carry them: one probe after the handshake on a path that carries 1,452, none once found. A path that stops carrying the size is noticed by its losses or its probe timeouts, and the size goes back to 1,200 and is found again. Nothing is configured. Every datagram carries the don't-fragment bit and nothing is fragmented on this side (Linux `IP_PMTUDISC_PROBE`, Windows `IP_DONTFRAGMENT`; v0.69.0), so a probe that passes passed whole.
- **0-RTT, when the TLS configuration has** `early_data` (chapter 14, section 6; v0.73.0). A client that resumes may send its first requests in its first packet, and the server answers them in its first reply: one round trip instead of two. A request that arrives that way is handed to the handler before the handshake is complete, and `proven_http_stream_early` is true for it: **it may be a copy an attacker replayed**. Only GET, HEAD and OPTIONS without a body reach a handler early; the

server answers anything else that arrives early with 425 (Too Early) itself, and a handler that must not act on a copy answers 425 too - the client sends the request again once the handshake is over. Off by default.

- **A client's next connection skips the address check.** After each handshake the server gives the client a token (NEW_TOKEN, RFC 9000 section 8.1.3) bound to its IP address and good for a day; a connection that brings one back is validated at once - no limit of three times what it sent, no Retry under `http3_retry` - so a 0-RTT answer larger than about 3.6 KB is not held back a round trip. A token that does not check is taken as none.
- **Not over HTTP/3:** WebSocket (`proven_ws_event_accept` refuses the stream), trailers, server push, migration of this side's own (a client's new address is validated), QUIC versions other than 1. `proven_http_event_server_adopt` takes TCP connections only, so several loops cannot share one UDP port.
- **The event-driven client speaks HTTP/3 when a request asks** (section 10); it never follows this server's `Alt-Svc` on its own.

How this was tested. A registered test runs the server against a client made of this library's QUIC and HTTP/3 engines over UDP on loopback - every route of the HTTP/2 tests that is not WebSocket, the limits, each timeout, the goodbyes, Retry. Outside the registered tests: curl `--http3-only` (OpenSSL's QUIC) and aioquic's client against the server, every response checked byte for byte; h3spec 0.1.14, 76 of its 77 cases passed - the one not is a repeated unknown setting, which RFC 9114 lets a server take; a thousand client first packets from addresses that never answered, each sent at most three times what it sent, all let go after `head_timeout_ms`, and none kept with `http3_retry`.

10. HTTP/3 in the event-driven client

```
typedef struct {
    /* ... everything Chapter 15 describes, and section 7's fields ... */
    bool http3;
    proven_size_t http3_window_bytes;
    proven_size_t http3_header_table_bytes;
    bool http3_no_header_table;
    bool http3_fallback;
    proven_u32 http3_fallback_ms;
} proven_http_event_client_config_t;

typedef struct {
    /* ... */
    bool http3;            /* this request over HTTP/3 */
} proven_http_event_request_options_t;
```

HTTP/2 is chosen by the TLS handshake; HTTP/3 cannot be - it is another transport, on UDP, and there is nothing to ask before the first packet. So **the caller chooses**: a client with `http3` sends over HTTP/3 the requests whose options say `http3`, each with an `https` URL, and every other request as before. It never follows a server's `Alt-Svc` on its own. The callbacks and the request calls are the same for all three protocols; `proven_http_event_request_protocol` says 3, from the start.

What happens underneath is section 7's, on QUIC: one connection kept for each origin (scheme, host, port, address), on a UDP socket of its own, shared by its requests; a request beyond the server's limit on streams waits on it; an idle one is told goodbye after `idle_timeout_ms` and closed; `max_connections` counts it with the TCP ones; `retry_unsent` sends again a request the server refused unprocessed (H3_REQUEST_REJECTED, or behind its GOAWAY); `decompress` decodes as over the others. A write is bounded by QUIC's send buffer - 256 KiB a stream, 1 MiB a connection, or, within what the server has granted, twice the congestion window, up to 16 MiB and 24 MiB - and a paused response stops being read, so the server stops at the window it had; read as fast as it comes, the window grows to 16 MiB a stream and 24 MiB a connection unless `http3_window_bytes` fixes it

(v0.71.0). A server's encoder may fill a QPACK dynamic table for the client as the server's section above describes for its clients - `http3_header_table_bytes` and `http3_no_header_table`, the same defaults and limits (since v0.95.0) - and since v0.96.0 the client's encoder fills the server's table the same way.

Without `http3_fallback` there is no fallback. A request asked over HTTP/3 whose server does not answer over UDP ends with `PROVEN_ERR_TIMEOUT` when `connect_timeout_ms` has passed - an unconnected UDP socket hears no refusal - one whose server refuses `h3` with `PROVEN_ERR_UNSUPPORTED`, and one whose server is not trusted with `PROVEN_ERR_UNTRUSTED`. Over QUIC the client offers `h3` alone, whatever else its TLS configuration offers over TCP, and takes nothing else (since version 0.73.2). No TCP connection is tried: a caller that wants one starts the request again without `http3`.

With `http3_fallback` (since v0.94.0) the client does that itself, for the cases where TCP may well get through: a QUIC handshake that has not finished after `http3_fallback_ms` (0: 1 s, at most `connect_timeout_ms`), an address the system reports unreachable or a port it reports refused, a server that refuses `h3`. Every request waiting on that connection then starts again over TCP as it would have without `http3` - HTTP/2 where the client has `http2` and the server agrees, HTTP/1.1 otherwise. The caller keeps the object it was given: its calls and callbacks are the same, and `proven_http_event_request_protocol` now says 2 or 1. For five minutes after, requests to that origin asked over HTTP/3 go over TCP from the start (64 origins at most, the oldest forgotten first). An untrusted certificate is not a reason - TCP would show the same one

- and neither is a QUIC connection lost after it came up, which is `retry_unsent`'s matter. A

body to be written that was refused before the switch is asked for again (`on_writable`) once the request is over TCP. The TLS configuration goes as it is, `h3` among what it offers over TCP; a server does not choose that over TCP. Nothing here races QUIC and TCP from the start, and the client still never follows a server's `Alt-Svc`.

Cautions, and what goes wrong

- **Ask only servers known to speak it.** Nothing here learns which do; a server's `Alt-Svc` says so, and reading it is the caller's to do.
- **How fast it is depends on the platform**, as for the server: on Linux a round's datagrams go to the kernel together and a receive may hand several back; elsewhere one system call each. One 100 MiB download from this library's server to this client on Linux loopback ran at 393 MiB/s over HTTP/3 and 340 over HTTP/2 (v0.68.0; 233 over HTTP/3 at v0.67.0, 171 at v0.65.0). Datagrams grow from 1,200 to 1,452 bytes by probing, as the server's do.
- **A TLS session is kept per origin** (chapter 15; v0.72.0): a new connection resumes it, and keeps the server's NEW_TOKEN token for the next one, which then skips the address check.
- **0-RTT with `early_data` in the TLS configuration** (chapter 14, section 6; v0.73.0). On a new connection whose session allows it, requests that are GET, HEAD or OPTIONS without a body go in the first packet and are answered one round trip sooner; any other request waits for the handshake. A request refused as early data, or answered 425, is sent again once after the handshake and only its second answer is delivered. Off by default.
- **Not here:** connection migration of this side's own, WebSocket over HTTP/3.

How this was tested. A registered test runs this client against this library's server over UDP on loopback - every case of section 7's test that the server can answer, the client's refusals, no fallback, the fallback with `http3_fallback` (a server over TCP only, an origin remembered and forgotten, a body written after the switch, a server that refuses `h3`, an untrusted one), an untrusted server, the timeouts. Outside the registered tests: aioquic's server (downloads to 50 MB byte for byte, uploads, trailers, an interim response passed over, 400 requests) and two public servers, cloudflare-quic.com and www.google.com.

Chapter 6: Execution, Aliases, PAL, Freestanding, and Cross Builds

Part VI - Going further. Prerequisites: Parts II-V. After this chapter you can run work on more than one thread without the memory model surprising you, write a state machine that reads like a loop, and build for a target that has no operating system.

This chapter covers `coro.h`, `job.h`, `alias_xcv.h`, thread-safety and pointer provenance, PAL contracts, freestanding mode, and platform build notes. This is the hardest material in the manual, and it is last on purpose - nothing in Parts I-V depends on it.

Table of contents

1. Stackless coroutines
2. Job system
3. Threads, allocators, and pointer provenance
4. Alias layer
5. PAL contract
6. Freestanding subset
7. Cross compilation
8. Examples and misuse cases

1. Stackless coroutines

The coroutine macros implement stackless state machines using `__LINE__` labels inside a switch. A coroutine function should return `proven_i32`: `0` means yielded or waiting, `1` means done.

Structure

```
typedef struct {
proven_i32 state;
} proven_coro_t;
```

Macros

Macro	Intent
`PROVEN_CORO_INIT(c)`	Set state to zero before first use.
`PROVEN_CORO_BEGIN(c)`	Begin coroutine switch block.
`PROVEN_CORO_YIELD(c)`	Save resume point and return 0.
`PROVEN_CORO_AWAIT(c, cond)`	Return 0 until `cond` is true, then continue.
`PROVEN_CORO_END(c)`	Mark done and return 1.
`PROVEN_CORO_IS_DONE(c)`	Check whether state is -1.

How it expands (and the one rule that matters)

These macros are the classic Protothreads/Duff's-device trick. `BEGIN` opens a `switch (state)`, each `YIELD`/`AWAIT` records `__LINE__` as the resume label and `return`s, and `END` closes the switch. Conceptually:

```
proven_i32 f(Ctx *c) {
    switch (c->coro.state) {       /* PROVEN_CORO_BEGIN */
    case 0:
        /* ... code before the first yield ... */
        c->coro.state = 42; return 0; case 42:;   /* PROVEN_CORO_YIELD on line 42 */
        /* ... code after the yield ... */
    }
    c->coro.state = -1; return 1;  /* PROVEN_CORO_END */
}
```

The function **returns** at each yield and is **re-entered from the top**, jumping straight to the resume `case`. The consequence - the single rule to remember:

Local (stack) variables do NOT survive a yield. Anything whose value must persist across a `YIELD`/`AWAIT` must live in the coroutine's own struct (or be `static`), because the locals are re-created on every call.

Other constraints that follow from the expansion: two coroutine macros must not share a source line (they'd collide on `__LINE__`), and a `YIELD`/`AWAIT` must not sit inside a `switch` of your own (it would land in the wrong `case`). The coroutine has no stack, so it cannot yield from a helper function it calls - only from its own body.

Counter-example - a local across a yield

```
static proven_i32 bad(Ctx *c) {
    PROVEN_CORO_BEGIN(&c->coro);
    int i = 0;                      /* WRONG: a local */
    while (i < 3) {
        i += 1;
        PROVEN_CORO_YIELD(&c->coro); /* on re-entry `i` is re-initialized to 0 */
    }
    PROVEN_CORO_END(&c->coro);      /* loop never ends: infinite yields */
}
/* RIGHT: put `i` in Ctx (like `value` in the Counter example below). */
```

Example (a sketch: a coroutine is a whole function, so it cannot be shown as a statement fragment - the compiled version of this shape is the worked example in section 8, `manual/examples/ex_06_coro.c`):

```
typedef struct Counter {
proven_coro_t coro;
int value;
} Counter;

static proven_i32 counter_next(Counter *c) {
PROVEN_CORO_BEGIN(&c->coro);
c->value = 0;
while (c->value < 3) {
c->value += 1;
PROVEN_CORO_YIELD(&c->coro);
}
PROVEN_CORO_END(&c->coro);
}

Counter c = {0};
PROVEN_CORO_INIT(&c.coro);
while (!counter_next(&c)) {
use_value(c.value);
}
```

2. Job system

The job system owns worker threads and a bounded MPMC-style queue. Producers submit `routine(arg)` work items. Workers execute queued jobs until the system is closed and drained.

Structures

```
typedef struct {
void (*routine)(void *arg);
void *arg;
} proven_job_t;

typedef struct proven_job_sys proven_job_sys_t;
```

`proven_job_sys_t` is opaque.

Functions

API	Intent	Return
`proven_job_system_init(alloc, num_workers, max_queue_capacity, out_sys)`	Allocate and start a job system. Queue capacity must be a power of two.	`proven_err_t`.
`proven_job_system_close(sys)`	Stop accepting new jobs.	void.
`proven_job_system_destroy(sys)`	Close if needed, drain queue, join workers, free resources.	void.
`proven_job_submit(sys, routine, arg)`	Submit one job. Thread-safe with other submitters.	true if queued, false if full or closed.
`proven_job_execute_one(sys)`	Let the calling thread execute one available job.	true if a job ran.
`proven_job_submit_ex(sys, routine, arg)`	Submit, saying why a refusal happened.	`PROVEN_OK`; `PROVEN_ERR_AGAIN` when full (retry or help); `PROVEN_ERR_INVALID_STATE` when closed (stop).
`proven_job_group_init(&group)`	Start counting a batch of jobs.	void.
`proven_job_group_submit(sys, &group, routine, arg)`	Submit a job the group counts. A refused job is not counted.	as `submit_ex`.
`proven_job_group_pending(&group)`	Jobs of the group not finished yet.	`proven_size_t`.
`proven_job_group_wait(sys, &group)`	Return when every counted job has run; the caller runs queued jobs while it waits. What the jobs wrote is visible afterwards.	void.

```
/* Render 16 tiles on the pool and wait for all of them. render_tile is the
 * program's own job routine; the wait runs queued tiles itself while it waits. */
void render_tile(void *tile);
static int tiles[16];
proven_job_sys_t *pool = NULL;
if (proven_is_ok(proven_job_system_init(alloc, 4, 64, &pool))) {
    proven_job_group_t frame;
    proven_job_group_init(&frame);
    for (int i = 0; i < 16; ++i) {
        proven_err_t e = proven_job_group_submit(pool, &frame, render_tile, &tiles[i]);
        if (e == PROVEN_ERR_AGAIN) { (void)proven_job_execute_one(pool); --i; continue; }   /* full: help,
retry */
        if (e != PROVEN_OK) break;                                                          /* closed */
    }
    proven_job_group_wait(pool, &frame);
    proven_println("{} tiles still pending", PROVEN_ARG(proven_job_group_pending(&frame)));
    proven_job_system_close(pool);
    if (proven_job_submit_ex(pool, render_tile, &tiles[0]) == PROVEN_ERR_INVALID_STATE) {
        proven_println("closed: no more work is taken");   /* not "full - try again" */
    }
    proven_job_system_destroy(pool);
}
```

Important constraints:

- `proven_job_system_close()` may race with submitters. A submission admitted before close either commits or returns `false` before close returns; later submissions are rejected.
- `proven_job_system_destroy()` must not race with producer threads still calling `proven_job_submit()`.
- To use close as the stop signal, call close, join every producer, and only then destroy.
- Queue sequence counters assume they do not wrap beyond signed pointer-difference range during one job-system lifetime.

Concurrency model
The queue is a fixed-size ring buffer of `max_queue_capacity` slots (must be a power of two, so a slot index is `seq & (capacity - 1)`). Producers and consumers do not lock the whole queue; they coordinate with **atomic sequence counters**:

- A producer (`proven_job_submit`) atomically claims the next enqueue position. If claiming would overrun the slowest un-consumed slot, the queue is full and submit returns `false` rather than waiting for capacity or overwriting pending work. Posting the worker wake may briefly enter platform synchronization.
- A consumer (a worker thread, or your own thread via `proven_job_execute_one`) atomically claims the next dequeue position, reads the `routine`/`arg`, marks the slot reusable, and runs the routine outside the claim.

An idle worker blocks on a platform counting semaphore instead of repeatedly polling and yielding. A successful submit publishes its slot before posting one permit. A permit remains available when no worker is waiting, so a wake cannot be lost between an empty-queue observation and the wait. `close` first stops admission and waits for already-admitted submissions to finish, then posts one final permit per worker. If an external `proven_job_execute_one` consumes a job before a worker consumes its permit, the retained permit is merely stale: the worker performs an empty recheck and parks again, or exits after close.

Because submit and execute are MPMC-safe, *many* threads may submit and *many* may consume at once. What the library does **not** do for you: it does not synchronize the *data your jobs touch*. Two jobs that write the same variable still need their own locking/atomics - the queue only orders the handoff, not the work. For the thread-safety of the allocators those jobs use, the pointer-provenance hazards of sharing allocations, and the lock-free toolbox (CAS, ABA, tagged pointers, hazard pointers, epoch-based reclamation), see Chapter 2 section 7 "Allocator thread-safety & provenance".

Lifecycle state machine (drive it in this order):

```
init  --->  running  --(close)-->  closed  --(destroy: drain + join)-->  freed
```

- `init` reserves `num_workers` OS threads and the slot buffer up front.
- `close` flips a flag so every later `submit` returns `false`; already-queued and in-flight submissions settle before close returns, and every accepted job still runs.
- `destroy` closes if needed, then **blocks the calling thread until the queue is empty and every worker has joined**, then frees everything.

Memory visibility. A worker joining inside `destroy` is a synchronization point: every memory effect of every job is visible to the thread that called `destroy` *after* `destroy` *returns*. So the safe pattern for collecting results is "submit all -> close -> destroy -> read results". Reading a job's output from another thread *before* that join needs your own synchronization.

Counter-examples

```
/* WRONG: capacity must be a power of two. */
proven_job_system_init(alloc, 4, 100, &sys);   /* 100 is not 2^n */

/* WRONG: reading a result before the join makes it visible. */
int total = 0;
```

```
(void)proven_job_submit(sys, accumulate, &total);
proven_println("total = {}", PROVEN_ARG(total));  /* data race: job may still be running */
/* RIGHT: */
proven_job_system_close(sys);
proven_job_system_destroy(sys);                   /* joins workers -> writes now visible */
proven_println("total = {}", PROVEN_ARG(total));

/* WRONG: ignoring the submit result drops work silently when the ring is full. */
proven_job_submit(sys, work, arg);   /* [[nodiscard]]: a false return means NOT queued */
```

Example (a sketch, because the job routine has to be a function of its own; the compiled version, with the atomics a shared counter really needs, is the worked example in section 8, `manual/examples/ex_06_job.c`):

```
static void increment(void *arg) {
int *p = arg;
*p += 1;
}

proven_job_sys_t *sys = NULL;
proven_err_t e = proven_job_system_init(alloc, 2, 64, &sys);
if (!proven_is_ok(e)) return e;

int value = 0;
if (!proven_job_submit(sys, increment, &value)) {
proven_job_system_close(sys);
proven_job_system_destroy(sys);
return PROVEN_ERR_BUSY;
}

proven_job_system_close(sys);
proven_job_system_destroy(sys);
```

The lifecycle itself, with no routine in sight, is ordinary statement code:

```
proven_job_sys_t *sys = NULL;
proven_err_t e = proven_job_system_init(alloc, 2, 64, &sys);  /* capacity: power of two */
if (proven_is_ok(e)) {
    /* ... submit work here, from this thread or from producers you own ... */
    proven_job_system_close(sys);     /* every later submit now returns false */
    proven_job_system_destroy(sys);   /* drains the queue, joins the workers */
}
```

3. Threads, allocators, and pointer provenance

This section moved here from Chapter 2. It was the hardest material in the manual sitting in one of the first chapters a reader opens, and nothing in Chapters 1-5 depends on it. If you are sharing `proven` objects between threads, or building a lock-free structure on top of one, this is the section you need; if you are not, you can skip it entirely and nothing later will suffer.

The `proven_allocator_t` trait is just a `ctx` plus three function pointers; it contains **no synchronization of its own**. Whether concurrent use is safe depends entirely on the concrete allocator behind the trait, and on how you share the *objects* the allocator produces. This section states the guarantees, then explains the deeper pointer-provenance hazards and the lock-free concepts you would need if you build concurrent structures on top.

3.1 What is and isn't thread-safe

Allocator	Concurrent use of one instance	Why

`proven_heap_allocator()`	**Safe** for concurrent `alloc`/`realloc`/`free`	The trait is stateless (`ctx == NULL`); it forwards to the platform `aligned_alloc`/`posix_memalign`/`_aligned_malloc`+`free`, which are thread-safe since C11.
`proven_arena_t`	**Not safe**	`proven_arena_alloc_*` does a non-atomic read-modify-write of `arena->offset`.
`proven_pool_t`	**Not safe**	`proven_pool_*` pop/push the free-list (`bin`, `bin_len`) non-atomically.

Two rules follow:

1. **"Allocator safe" is not "object safe."** Even with the heap allocator, the containers built on it (`proven_u8str_t`, `proven_array_t`, `proven_map_t`, ...) add no internal locks. A `proven_u8str_t` that one thread appends to while another reads it is a data race regardless of how thread-safe the allocator is. Shared mutable objects always need *your* synchronization.
2. **Arena and pool must not be shared.** Concurrent `proven_arena_alloc` can tear `offset` and hand the *same bytes* to two threads or drop an allocation entirely; concurrent pool pop/push can hand out the same slot twice or underflow `bin_len`. A data race is undefined behavior on its own - see section 7.3 for why the consequences are worse than "a wrong number."

3.2 Pointer provenance in one paragraph

In C's abstract machine every pointer carries **provenance**: the identity of the storage instance it was derived from. Two rules matter here: (a) using a pointer outside the lifetime or bounds of its provenance object is undefined; and (b) the optimizer is allowed to assume that **pointers with different provenance do not alias**, and to reorder or elide memory accesses on that basis. Allocation, reallocation, and freeing all create and destroy provenance - which is exactly why they interact badly with unsynchronized sharing.

3.3 Where allocation + provenance bite under threads

- `realloc` **always relocates here.** The platform `realloc` is *allocate-new* + *copy* + *free-old* (an aligned block cannot be resized in place portably), so a successful grow **always** returns a new object with new provenance and ends the old one. Any retained copy of the old pointer - a cached element pointer, a borrowed `proven_mem_view_t`/`proven_u8str_view_t` - is now dangling. In a single thread you avoid this by not holding a view across a grow; under threads another thread can grow a shared container at *any* instant, leaving your view pointing into freed storage (rule (a): UB, plus a data race).
- **Address reuse / ABA.** `free` then `alloc` (especially the pool's free-list recycling) can return the *same address* for a *different* object with *fresh* provenance. A thread still holding the old pointer assumes the old provenance; by rule (b) the compiler may treat the two as non-aliasing even though the bytes coincide, and without a happens-before edge the new object's writes need not be visible. Same address, different provenance - still UB.
- `uintptr_t` **round-trips** + **torn reads.** The arena converts `backing.ptr` to an integer for alignment math. It is careful to derive the *result* pointer by offsetting the original `backing.ptr` (preserving its provenance) rather than fabricating a pointer from the integer - the provenance-correct technique. But if `offset` is read torn under a race, the computed pointer can land *outside* the backing object, i.e. an access with no valid provenance for that location. The race produces not just a wrong offset but a pointer with no right to point there.
- **Data race x provenance reasoning = miscompilation, not just wrong values.** Because the compiler applies single-threaded, provenance-based non-aliasing reasoning within each thread,

a racy allocator can let the optimizer "prove" non-aliasing that does not hold at runtime - yielding torn pointers, double allocations the compiler believes cannot overlap, or dropped stores. The failure mode is structural, not a flaky value.

3.4 The lock-free toolbox (concepts, if you build concurrency on top)

`proven` does **not** implement any lock-free allocator or safe-memory-reclamation scheme. If you build concurrent data structures over these allocators, you supply the following yourself (`<stdatomic.h>` is available - the job system in Chapter 6 uses it for its queue indices). These are the standard pieces and how they relate to the provenance hazards above.

- **CAS (compare-and-swap).** The atomic primitive behind almost all lock-free code: `atomic_compare_exchange_strong(&p, &expected, desired)` atomically sets `p = desired` only if `p == expected`, otherwise reports the current value. It lets one thread publish a change only if no other thread changed the word first.

```
text /* Treiber-stack style push (sketch): ex_node_t and n are illustrative, so this is a listing rather than a compiled block. */ _Atomic(ex_node_t *) head; ex_node_t *old = atomic_load(&head); do { n->next = old; } while (!atomic_compare_exchange_weak(&head, &old, n));
```

- **The ABA problem.** CAS compares *values*, not *history*. A lock-free pop reads `head == A`, plans to install `A->next`, then CASes. If, in between, other threads pop `A`, pop its successor, free them, and push `A` back (its address reused), the CAS still sees `head == A` and *succeeds* - installing a pointer to freed memory. The value matched (A->B->A) but the world changed. The pool's free-list is a textbook ABA candidate if naively made lock-free.
- **Tagged pointers / version counters.** Pack a monotonically increasing tag next to the pointer and CAS them together (a double-width CAS, or by stealing the low alignment bits / high bits). Every successful update bumps the tag, so an A->B->A sequence comes back with a *different* tag and the CAS fails - ABA detected. Costs/limits: bit-stealing needs guaranteed alignment and shrinks the usable address range; a full-width tag needs hardware double-word CAS (e.g. `cmpxchg16b`); the tag can in principle wrap. **Crucially, a tag fixes ABA *detection* on the shared word - it does not restore provenance.** The reused address is still a new object; the tag only stops you from *acting* on the stale view.
- **Hazard pointers (safe memory reclamation).** Each thread owns a few single-writer/multi-reader "hazard" slots. Before dereferencing a shared pointer it publishes that pointer into a slot and re-validates; a thread that wants to free an object first scans every hazard slot and **defers** the free (a retire list) if anyone is protecting it. This bounds memory and prevents use-after-free and reclamation-ABA, at the cost of a store + fence per protected access and a fixed number of simultaneously protected pointers per thread.
- **Epoch-based reclamation (EBR).** A global epoch counter; a thread "pins" the current epoch while touching shared structures, and retired memory is tagged with the epoch it was retired in. Memory retired in epoch *e* is freed only once every thread has been observed past *e* (a grace period of a couple of epochs). Cheaper per operation than hazard pointers (just a pinned flag), but a thread that stalls while pinned blocks all reclamation - unbounded memory growth. Variants: quiescent-state (QSBR) and interval-based reclamation.
- **How these tie back to provenance.** Reclamation schemes (hazard pointers, EBR) exist precisely to keep a freed object's *storage alive* - its provenance valid -- until no thread can still reference it. CAS + a tag keep the shared *word* consistent but say nothing about the lifetime of what it points to. That is why a correct lock-free stack typically needs **both**: a tag (for ABA on the head word) *and* an SMR scheme (for safe reclamation of popped nodes). A `proven` pool or arena gives you neither, so concurrency must be layered above them, not assumed.

3.5 Safe patterns with `proven`

- **Per-thread arena/pool.** Give each thread its own `proven_arena_t` / `proven_pool_t`. No sharing means no race and no cross-thread provenance - the simplest correct design, and usually the fastest.
- **Heap for cross-thread alloc/free, but synchronize the objects.** It is fine for thread A to allocate and thread B to free via `proven_heap_allocator()`; it is *not* fine for both to mutate the same `proven_array_t`/`proven_u8str_t` without a lock.
- **Hand off ownership with a happens-before edge.** Transfer a pointer to another thread through a mutex, atomic with release/acquire ordering, or a queue (like the job system) so that only one thread observes it at a time. This keeps each allocation's provenance confined to a single thread and makes the producer's writes visible to the consumer.
- **Do not pass borrowed views across threads** unless the owner is guaranteed not to grow/move/free for the whole duration of the borrow - and remember a grow here always relocates.
- **If you must share an arena/pool, wrap it** in your own mutex (or build a real lock-free allocator with the tools in section 7.4); the built-in ones assume a single owner at a time.

4. Alias layer

Why a second set of names exists

`proven_u8str_view_slice` is 26 characters. In a file that calls twenty such functions, the prefix is a third of the line, and the part that varies - the part you actually read - is squeezed into what is left.

Prefixes are not decoration in C. The language has one global namespace for functions, so a library that exports `slice` or `create` will eventually collide with something else the program links. The prefix is what makes a C library safe to combine with others, and it is why every serious C library has one.

`alias_xcv.h` is the compromise: a shorter `xcv_` spelling for every public symbol, in a header you opt into. Include it and `xcv_u8str_view_slice` works; do not, and the short names do not exist to collide with anything.

The canonical names remain the source of truth - for the ABI, for this manual, and for the tests. The alias layer is a spelling, not an API: nothing is exported under the short name that is not exported under the long one, and a completeness gate (`tests/test_docs_alias_completeness.c`) fails the build if a public function is ever added without its alias, because an alias layer with holes is worse than none - a caller who adopts it discovers the gaps one compile error at a time.

Wrong - expecting the aliases without including the header:

```
#include "proven.h"
xcv_u8str_view_t v;   /* wrong: proven.h does not pull in the alias layer */
```

Wrong - mixing the two spellings for the same idea in one file:

```
proven_result_u8str_t s = xcv_u8str_create(alloc, 32);   /* wrong: pick one and stay with it */
```

Both compile. Neither is a good idea: a reader now has to know both vocabularies to follow one function.

Include it after the canonical headers:

```
#include "proven.h"
#include "proven/alias_xcv.h"

xcv_allocator_t heap = xcv_heap_allocator();
(void)heap;
xcv_println("{}", XCV_ARG(123));
```

Alias design rules:

- Aliases are preprocessor conveniences, not a separate ABI.

- The canonical API remains the source of truth.
- Treat `alias_xcv.h` as a template if a project wants its own shorter local prefix.
- Keep alias tests synchronized when public API names are added or removed.

Alias macro families:

- Formatting: `XCV_ARG`, `XCV_ARG_CSTR`, `XCV_ARG_CSTR_N`, `XCV_FMT_IS_OK`.
- Containers: `XCV_ARRAY_*`, `XCV_RING_*`, `XCV_MAP_*`, `XCV_LIST_*`.
- Errors and arithmetic: `XCV_ERR_*`, `XCV_IS_OK`, `XCV_CKD_*`.
- Coroutines: `XCV_CORO_*`.
- Filesystem and sysio: `xcv_fs_*`, `xcv_print*`, `xcv_scan*` aliases.
- Types and allocators: `xcv_*_t` type aliases and allocator aliases.

For an exhaustive source-grounded alias table, see Chapter 7: Alias Index.

5. PAL contract

Why the syscalls are quarantined
Every library that touches an operating system has to decide where the OS-specific code goes. The usual answer is "wherever it is needed", with `#ifdef _WIN32` sprinkled through the source. That works, and it has two costs that compound: nobody can tell what the library actually requires from the OS, and porting means auditing every file.

This library puts all of it in one directory. **`platform/` is the only place that makes a syscall.** Everything in `src/proven/` is portable C that calls through the PAL, which means:

- **The requirement list is a file listing.** What this library needs from an operating system is exactly the set of `proven_sys_*` functions - nothing hidden, nothing discovered late.
- **Porting is bounded.** A new target reimplements `platform/`. Nothing in `src/proven/` changes, and the tests that exercise the portable half keep passing.
- **Freestanding is the same mechanism, not a special case.** Build without the hosted PAL files and the portable core is what remains. That is why the freestanding guide is a list of which files to leave out rather than a separate port.

The PAL is **internal**. `proven_sys_*` functions are not the public API and their signatures may change; the public wrappers in `fs.h`, `sysio.h`, `time.h` and `random.h` are what you call. The symbols gate knows this - it exempts the PAL from the "must be documented" requirement precisely because it is not a surface you are meant to program against.

Wrong - calling the PAL directly from application code:

```
proven_sys_fs_open(path, flags);   /* wrong: internal. Use proven_fs_open. */
```

You lose the error translation, the argument validation, and any guarantee that the call keeps working next release.

PAL areas:

- `proven_sys_mem`: heap allocation backend.
- `proven_sys_fs`: files, directories, paths, links, permissions, locks.
- `proven_sys_time`: the wall clock, the monotonic clock, sleep, time breakdown.
- `proven_sys_net`: sockets, readiness, name resolution.
- `proven_sys_env`: environment variables.
- `proven_sys_thread`: threads and synchronization for the job system.
- `proven_sys_io`: standard stream I/O.
- `proven_sys_math`: math helpers where needed.

Public application code should prefer high-level APIs:

- `proven_fs_*` for filesystem operations.

- `proven_sysio_*` and print/scan macros for console I/O.
- `proven_time_*` for time.
- `proven_job_*` for threaded jobs.

PAL internal exception: thread lifecycle code may need internal platform allocation to keep opaque OS metadata alive across entry points. That exception stays inside PAL code and should not leak into core container APIs.

6. Freestanding subset

`PROVEN_FREESTANDING` builds a reduced subset for OS-free or libc-minimal targets. The current freestanding build also defines:

```
-DPROVEN_FREESTANDING -DPROVEN_FMT_NO_FLOAT -DPROVEN_NO_U16STR -ffreestanding
```

Available modules in the current freestanding configuration:

- `types`
- `error`
- `memory`
- `align`
- `allocator`
- `arena`
- `pool`
- `buffer`
- `array`
- `list`
- `ring`
- `map`
- `algorithm`
- `u8str`
- `coro`
- `scan`
- `fmt` without floating-point formatting
- `panic`

Excluded or stubbed modules:

- `heap`: compiled, but `proven_heap_allocator()` returns an invalid zero allocator.
- `u16str`: excluded by `PROVEN_NO_U16STR` in the current freestanding build.
- `time`: partial. `src/proven/time.c` is compiled, so `proven_time_breakdown()` and the datetime formatters are available, but the clock backend (`platform/proven_sys_time.c`) is not - so `proven_time_now()`, `proven_time_now_datetime()`, and `proven_time_sleep()` have no implementation to link against.
- `fs`, `sysio`, `mmap`, `job`, `net`: hosted/PAL services excluded from the current freestanding subset. A hosted build can also leave `net` out with `-DPROVEN_NO_NET`.

See `manual-freestanding.md` for the exact source list and command examples.

7. Cross compilation

Why the build compiles for targets it cannot run

A library that claims to be portable is making a claim that decays silently. Code that assumed 64-bit pointers, or that `char` is signed, or that unaligned loads are fine, compiles perfectly on the machine it was written on and breaks on an ARM board six months later - and the commit that broke it looked harmless.

`./nob cross` compiles the library for every target in a matrix without running anything. The configured MinGW lanes additionally link a smoke executable. That sounds weak and is not: **most portability failures are compile-time failures.** A type that is not the width you assumed, a missing intrinsic, an alignment requirement the target actually enforces, a header that does not exist there - all of them fail the compiler, on this machine, in seconds, in the commit that introduced them.

What a compile-only check cannot catch is behaviour: endianness bugs, alignment faults that only happen at run time, and anything timing-dependent. Those need real hardware or an emulator, and the matrix does not pretend otherwise. It is the cheap half of portability testing, run on every build, rather than the expensive half run never.

Cross builds are also where the freestanding profile is exercised for real targets - Cortex-M, RISC-V and wasm32 among them - so the "no operating system" claim is checked by a compiler rather than by a paragraph in a guide.

Wrong - treating a green cross matrix as proof the library runs on that target:

```
/* wrong: ./nob cross compiles and links objects. It does not execute anything.
   Endianness, alignment faults and real timing still need the hardware. */
```

The build driver command is:

```
./nob cross -build-root build-out/proven_c_lib
```

Current target categories:

- Native GCC hosted.
- Native Clang hosted when available.
- Linux AArch64 hosted compile-only.
- Linux ARM hard-float hosted compile-only.
- Linux i686 hosted compile-only through `i686-linux-gnu-gcc` or `gcc -m32`.
- Windows x86_64 and i686 WinAPI compile-and-link smoke checks through MinGW.
- ARM Cortex-M freestanding compile-only.
- RISC-V ELF freestanding compile-only.

Rules:

- Missing optional compilers are skipped.
- Real compile errors fail the command.
- Cross builds are compile-only except for the configured MinGW compile-and-link smoke; none is runtime verification.
- Runtime behavior still needs a target runner, emulator, device, or OS environment.

8. Examples and misuse cases

Coroutine macros must not share one source line
Wrong:

```
PROVEN_CORO_YIELD(&c->coro); PROVEN_CORO_YIELD(&c->coro);
/* wrong: both use the same __LINE__ value */
```

Correct:

```
PROVEN_CORO_YIELD(&c->coro);
PROVEN_CORO_YIELD(&c->coro);
```

Coroutine local variables are ordinary locals
A stackless coroutine returns to the caller. Any local variable whose value must survive across yields should live in the coroutine state object, not as an automatic local inside the coroutine function.

Wrong:

```
static proven_i32 next(Task *t) {
int temporary = 0;
PROVEN_CORO_BEGIN(&t->coro);
temporary = 42;
PROVEN_CORO_YIELD(&t->coro);
use_int(temporary); /* wrong assumption: ordinary local lifetime/state is not persistent */
PROVEN_CORO_END(&t->coro);
}
```

Correct:

```
typedef struct Task {
proven_coro_t coro;
int temporary;
} Task;
```

Job destroy must not race with submitters

Wrong:

```
/* thread A */
proven_job_system_destroy(sys);

/* thread B at the same time */
proven_job_submit(sys, work, arg); /* wrong: external synchronization missing */
```

Correct:

```
proven_job_system_close(sys); /* may race with submit and makes it return false */
join_producer_threads();      /* no producer can access sys after this point */
proven_job_system_destroy(sys);
```

Job argument lifetime

Wrong:

```
void submit_bad(proven_job_sys_t *sys) {
int value = 10;
proven_job_submit(sys, work, &value);
} /* wrong: value may be dead before work runs */
```

Correct:

```
JobData *data = allocate_job_data();
proven_job_submit(sys, work_and_free_data, data);
```

Alias layer should not hide canonical docs

Wrong:

```
/* Document only xcv_map_t and forget proven_map_t. */
```

Correct: document canonical `proven_` API first, then mention aliases as optional local spelling.

Worked example: the bounded job system

Compiled and run by the test suite. Note the ordering the contract requires: submit, then close, then destroy. `proven_job_system_destroy` must not race with `proven_job_submit`.

```
#include <stdatomic.h>

/*
 * The job system: worker threads plus a bounded atomic MPMC queue. Idle workers
 * park on platform synchronization. It orders the *handoff* of work - it does
 * not synchronize the data the work touches. That is why the counter below is
 * an atomic and not a plain int: two jobs incrementing the same variable is a
 * data race unless the caller says otherwise.
```

```
 *
 * The lifecycle is a straight line, and it is not optional:
 *
 *     init -> submit... -> close -> destroy
 *
 * destroy must not race with submit. Nothing in the library enforces that; the
 * caller has to stop its producers first. Here there is only one producer - this
 * thread - so "stop the producers" means "finish the submit loop before closing".
 */

#define JOB_COUNT 64

static void increment(void *arg) {
    atomic_int *counter = arg;
    /* relaxed is enough: we only need the total to be right, not to order anything
     * against it. The join inside destroy is what publishes the result to us. */
    atomic_fetch_add_explicit(counter, 1, memory_order_relaxed);
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    proven_job_sys_t *sys = NULL;
    /* Queue capacity must be a power of two - the ring maps a sequence number to a
     * slot with a mask. Sized above JOB_COUNT so a submit cannot find the ring
     * full even if every worker is still starting up. */
    proven_err_t err = proven_job_system_init(alloc, 4, 128, &sys);
    EXAMPLE_REQUIRE(proven_is_ok(err), "starting a job system with 4 workers should succeed");
    if (!proven_is_ok(err)) return 1;

    /* Lives until after destroy: a job's arg must outlive the job, and jobs run
     * until destroy has drained the queue. */
    atomic_int counter = 0;

    proven_size_t submitted = 0;
    for (proven_size_t i = 0; i < JOB_COUNT; ++i) {
        /* submit returns false when the ring is full or the system is closed. It
         * never waits for queue capacity and never drops work silently; its wake
         * path may briefly enter platform synchronization. Ignoring the answer is
         * how you lose jobs, which is why it is [[nodiscard]]. */
        if (!proven_job_submit(sys, increment, &counter)) {
            /* A real caller would back off and retry, or run the job inline with
             * proven_job_execute_one. Here a full ring means the sizing above is
             * wrong, so say so rather than paper over it. */
            EXAMPLE_REQUIRE(false, "the queue was sized to hold every job");
            break;
        }
        ++submitted;
    }

    /* This thread is the only producer, and it is done submitting - so it is safe
     * to close. close makes every later submit fail; jobs already queued still run. */
    proven_job_system_close(sys);

    /* destroy blocks until the queue is empty and every worker has been joined.
     * That join is the synchronization point: after destroy returns, every memory
     * effect of every job is visible to this thread. Reading `counter` before this
     * line would be reading a value the workers are still writing. */
    proven_job_system_destroy(sys);
```

```
    int ran = atomic_load(&counter);
    EXAMPLE_REQUIRE(submitted == JOB_COUNT, "every job should have been accepted");
    EXAMPLE_REQUIRE(ran == (int)submitted, "every submitted job should have run exactly once");

    printf("submitted %zu jobs, %d ran\n", (size_t)submitted, ran);

    return EXAMPLE_OK();
}
```

Worked example: a stackless coroutine

Compiled and run by the test suite. The rule that catches everyone: locals do not survive a yield, so every piece of state the coroutine needs across a suspension has to live in its struct.

```
/*
 * A stackless coroutine is a switch statement in disguise: BEGIN opens a
 * switch on the saved state, each YIELD records __LINE__ as a resume label and
 * *returns*, and the next call re-enters the function from the top and jumps
 * straight back to that label.
 *
 * Everything that follows comes from that one fact:
 *
 *   - Locals do NOT survive a yield. The function returned; its stack frame is
 *     gone. Anything that must persist lives in the coroutine's own struct - which
 *     is what `value` and `remaining` are doing below.
 *   - Two coroutine macros must not share a source line (they would collide on
 *     __LINE__).
 *   - It cannot yield from a helper it calls: there is no stack to suspend.
 *
 * The payoff is that a suspended coroutine costs exactly its struct - four bytes
 * of state plus whatever you put next to it - and no thread, no stack, no context
 * switch.
 */

typedef struct {
    proven_coro_t coro;
    /* The generator's state. These would be `int i` locals in a normal loop; here
     * they have to be fields, or they would be reset to their initial values on
     * every resume and the loop would never end. */
    int value;
    int remaining;
} ex_squares_t;

/* A coroutine returns proven_i32: 0 = suspended (call me again), 1 = done. */
static proven_i32 squares_next(ex_squares_t *g) {
    PROVEN_CORO_BEGIN(&g->coro);

    g->remaining = 5;
    g->value = 1;

    while (g->remaining > 0) {
        g->value = g->value * g->value;
        PROVEN_CORO_YIELD(&g->coro);      /* the caller reads g->value here */
        g->value = g->value + 1;          /* resumes exactly on this line */
        g->remaining -= 1;
    }

    PROVEN_CORO_END(&g->coro);
}

int main(void) {
```

```
    proven_allocator_t alloc = proven_heap_allocator();

    /* The coroutine owns no memory, so there is nothing to destroy - but the values
     * it produces have to go somewhere, and that string does have an owner. */
    proven_result_u8str_t out = proven_u8str_create(alloc, 32);
    EXAMPLE_REQUIRE(proven_is_ok(out.err), "creating the output string should succeed");
    if (!proven_is_ok(out.err)) return 1;

    ex_squares_t gen = {0};
    PROVEN_CORO_INIT(&gen.coro);   /* unconditional, exactly once, before the first call */

    int produced = 0;
    int last = 0;

    /* Drive it to completion. squares_next returns 1 on the call that runs off the
     * end of the body - that call produces no value, so the loop body only runs
     * while it returned 0. */
    while (!squares_next(&gen)) {
        proven_fmt_result_t r = proven_u8str_append_fmt_grow(alloc, &out.value, "{} ",
                                                             PROVEN_ARG(gen.value));
        EXAMPLE_REQUIRE(PROVEN_FMT_IS_OK(r), "appending a generated value should succeed");
        last = gen.value;
        ++produced;
    }

    /* Done is sticky: the state is -1 and stays there. Calling it again would just
     * return 1 without re-running the body. */
    EXAMPLE_REQUIRE(PROVEN_CORO_IS_DONE(&gen.coro), "the generator should have finished");
    EXAMPLE_REQUIRE(squares_next(&gen) == 1, "a finished coroutine stays finished");

    /* 1, then (1+1)^2 = 4, then (4+1)^2 = 25, then 676, then 458329. */
    EXAMPLE_REQUIRE(produced == 5, "the generator yields once per iteration");
    EXAMPLE_REQUIRE(last == 458329, "the state carried across every yield");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&out.value),
                                         PROVEN_LIT("1 4 25 676 458329 ")),
                    "the generated sequence should be exactly this");

    printf("squares: %s\n", proven_u8str_as_cstr(&out.value));

    proven_u8str_destroy(alloc, &out.value);
    return EXAMPLE_OK();
}
```

Proven Freestanding Mode (v0.97.0)

Part VI - Going further. Prerequisites: Parts II-V, and Chapter 6. After this guide you can build the library for a target with no operating system and no libc, and you will know exactly which modules survive that and which do not.

This guide describes the current `PROVEN_FREESTANDING` configuration as implemented by `nob.c` and the public headers.

A note on the shape of this guide. Unlike the chapters, roughly half of it is procedure - flag lists, file listings, and two compile commands. Those sections are deliberately short: a compile command explained at length is a compile command nobody reads. The sections that carry a *decision* -- why this mode exists (section 0), what "excluded" really means (section 3), what your panic handler has to do (section 5), what you lose by dropping float (section 8), why hosted calls fail at link time (section 9), why the lifetime rules matter more here (section 10), and how the claims in this guide are verified (section 11) - are written out in full.

0. Why this mode exists, and why the whole library is shaped by it

Freestanding mode is for firmware, kernels, bootloaders, hypervisors, and anywhere else that has no operating system underneath it - no `malloc`, no `open`, no `printf`, often no `libc` at all.

The C standard has a name for this. A **hosted** implementation gives you the whole standard library and starts your program at `main`. A **freestanding** implementation guarantees only a handful of headers - `<stddef.h>`, `<stdint.h>`, `<limits.h>` and a few others - and is what a compiler targeting bare metal gives you. `-ffreestanding` tells the compiler to assume exactly that.

Most C libraries cannot be used here at all, and the reason is rarely a big one. It is a `malloc` buried three calls down, or a `printf` in an error path, or a global that is initialised by something that never runs. One hidden dependency on the host is enough to make a library unusable on a microcontroller.

This is the constraint that produced most of this library's design decisions, and it is worth seeing that connection, because it explains choices that otherwise look like taste:

Design decision	Read the earlier chapters as	And the reason is
The allocator is a parameter (Ch 2)	"so you can swap the strategy"	There is no `malloc` here. An arena over a `static` array is the only allocator, and code that took one as a parameter already works.
Errors are returned values (Ch 1)	"so you cannot ignore them"	There is no `errno` and nothing to unwind to. A return value is the only mechanism that exists.
Panics go through a hook (Ch 1 section 6)	"so you can override it"	There is no `abort()` and no `stderr`. The default traps because that is all it can portably do.
Syscalls live only in `platform/`	"separation of concerns"	It is the only directory that has to be replaced for a new target. Everything in `src/proven/` is portable by construction.

Views instead of C strings (Ch 3)	"so lengths cannot get lost"	`strlen` is libc. A view brings its length with it and needs nothing.

In other words: freestanding support is not a feature bolted onto the side. The rest of the manual has been describing a library built to survive here, and this guide is where you find out what survives and what does not.

What you give up
Everything that needs the operating system, and nothing else:

- **No heap.** `proven_heap_allocator` does not exist. You supply memory - a `static` array is usual - and put an arena or a pool over it.
- **No filesystem, no standard streams, no clock, no OS randomness.** `fs.h`, `sysio.h`, `mmap.h`, `time.h` and the OS entropy source are all hosted services. `proven_chacha_rng_t` still works if you seed it yourself, which is what `proven_random_set_source` is for.
- **No float formatting by default.** `PROVEN_FMT_NO_FLOAT` drops it, because the float path is the largest piece of code in the formatter and most firmware never prints a `double`. section 8 says how to turn it back on.
- **No UTF-16 strings.** `PROVEN_NO_U16STR` is on; there is no Windows API here to talk to.

What you keep is the whole portable core: errors, views, checked arithmetic, alignment, arenas, pools, buffers, strings, arrays, maps, lists, rings, sorting, searching, hashing, encoding, coroutines, and integer formatting and scanning.

What still bites you
The lifetime rules do not relax because the target got smaller - if anything they matter more, because there is no operating system to notice a mistake and no allocator that will refuse to reuse memory you still hold. section 10 is short and is the section people skip.

1. Build profile

The build driver uses these freestanding flags:

```
-std=c23
-ffreestanding
-DPROVEN_FREESTANDING
-DPROVEN_FMT_NO_FLOAT
-DPROVEN_NO_U16STR
```

The driver still probes `-std=c23` first and falls back to `-std=c2x` when the compiler uses the transitional spelling. The freestanding source contract stays C23-first even though the build wrapper accepts either flag spelling.

The hosted `./nob freestanding` command also links its local freestanding test executables statically on the build host. Cross freestanding targets in `./nob cross` use compile-only object checks.

Run the project freestanding check:

```
cc nob.c -o nob
./nob freestanding -build-root build-out/proven_c_lib
```

Run the cross-build matrix:

```
./nob cross -build-root build-out/proven_c_lib
```

2. Available source files

In the current freestanding build, `nob.c` compiles the portable source files except hosted-only modules.

Included core modules:

```
src/proven/memory.c
src/proven/arena.c
src/proven/pool.c
src/proven/buffer.c
src/proven/heap.c
src/proven/u8str.c
src/proven/array.c
src/proven/ring.c
src/proven/map.c
src/proven/algorithm.c
src/proven/hash.c
src/proven/hash_legacy.c
src/proven/hmac.c
src/proven/utf.c
src/proven/url.c
src/proven/http.c
src/proven/http_auth.c
src/proven/http_cookie.c
src/proven/sse.c
src/proven/ws.c
src/proven/encode.c
src/proven/random.c
src/proven/float_decimal.c
src/proven/float_parse.c
src/proven/float_format.c
src/proven/time.c
src/proven/fmt.c
src/proven/scan.c
src/proven/panic.c
platform/proven_sys_math.c
```

Excluded hosted modules:

```
src/proven/u16str.c
src/proven/fs.c
src/proven/stream.c
src/proven/sysio.c
src/proven/mmap.c
src/proven/job.c
platform/proven_sys_fs.c
platform/proven_sys_thread.c
platform/proven_sys_io.c
platform/proven_sys_env.c
platform/proven_sys_time.c
platform/proven_sys_random.c
platform/proven_sys_mem.c
```

Do not add excluded hosted modules to a bare-metal build unless you also provide a real platform backend for the target.

3. Module availability

How to read this table, and what "excluded" means

A module is excluded here for exactly one reason: it needs something only an operating system can provide. It is not that the code is untested on small targets or that somebody has not got round to it - `fs.h` needs a filesystem, `sysio.h` needs standard streams, `mmap.h` needs virtual memory, `job.h` needs threads. There is nothing to port.

The interesting rows are the two that are neither available nor excluded:

- `heap.h` **is a stub.** `proven_heap_allocator()` still exists and still compiles, and it returns an allocator whose function pointers are null - one that `proven_alloc_is_valid` reports as invalid. It does not silently allocate from somewhere else and it does not fail to link. That choice matters: code written against the allocator trait keeps compiling for a bare-metal target, and the place it breaks is the one line that asked for a heap, at run time, with a check you can write. section 4 shows what to pass instead.
- `time.h` **is limited.** The datetime formatting is pure arithmetic over a number you supply, so it compiles. What is missing is the number: reading a clock is a syscall, so `proven_time_now` has no backend here. Format timestamps you got from your own hardware timer.

Everything marked Available is the portable core, and "available" means the same tests that run on a hosted build run against it - the freestanding profile is built and checked by `./nob freestanding` on every release, not asserted in this table.

Module	Status in current freestanding profile	Notes
`types.h`, `error.h`, `align.h`, `memory.h`	Available	Requires fixed-width integer and `uintptr_t` support.
`allocator.h`	Available	Trait only. Caller supplies backing allocators.
`alloc_check.h`	Available	Pure bookkeeping over a caller-supplied record; reports through `proven_panic`.
`arena.h`	Available	Primary allocator for static memory regions.
`pool.h`	Available	Uses a caller-provided base allocator, often an arena.
`buffer.h`	Available	Fixed-capacity byte buffer.
`u8str.h`	Available	U8 strings and views.
`u16str.h`	Excluded	Current profile defines `PROVEN_NO_U16STR`.
`array.h`, `list.h`, `ring.h`, `map.h`	Available	No hidden OS dependency.
`algorithm.h`	Available	Sort/search helpers for arrays.
`hash.h`	Available	FNV-1a, SipHash-2-4, CRC-32, SHA-256 - byte-exact, no OS dependency. CRC-32 uses eight 256-entry tables (8 KiB of read-only data) by default; build the library with `-DPROVEN_CRC32_SMALL=1` to keep one 1 KiB table, about 5x slower on long inputs, same output.
`hash_legacy.h`	Available	SHA-1 and MD5, for formats that fix them (the WebSocket accept key, old checksum files). Same shape as SHA-256; no table larger than MD5's 64 constants.

`hmac.h`	Available	HMAC and HKDF over SHA-256, SHA-384 and SHA-512 (the last two are in `hash.h`). Pure arithmetic. `proven_mem_equal_ct` and `proven_mem_wipe`, which go with them, are in `memory.h`.
`deflate.h`	Available	DEFLATE, zlib and gzip, both ways. Pure arithmetic; one allocation per compressor (about 325 KiB at the default window, 11 KiB at the smallest) or decompressor (about 35 KiB), through the allocator you pass.
`zstd.h`	Available	Zstandard, both ways. Pure arithmetic. A decoder is about 16 KiB, and while a frame is decoded its window (at most the limit you set, 8 MiB by default) plus up to 384 KiB, grown as its output grows; an encoder its hash tables (64 KiB to 1 MiB by level), about 500 KiB for a block and, for a stream, an input buffer of one and a half windows - all through the allocator you pass. Decoder and encoder are separate objects: a program that only reads links no encoder.
`brotli.h`	Available	Brotli decompression. Pure arithmetic. A decoder is about 22 KiB, and while a stream is decoded its history (as much of the stream's window as its output has reached: 16 MiB at most) and its prefix codes (1.3 MiB at most), grown as needed through the allocator you pass. The 122,784-byte static dictionary is part of it; `-DPROVEN_NO_BROTLI` leaves both out.
`hpack.h`, `h2.h`	Available	HTTP/2's header compression, frames and connection. Pure arithmetic, no socket and no clock; HPACK allocates one

		table for an encoder and one for a decoder (each grows as it is used, to a ceiling of 4,096 bytes of entries by default), the frame functions nothing, and a connection (`proven_h2_conn_t`) what its configuration bounds, through the allocator it is given.
`cert.h`	Available, except `proven_cert_store_add_system`	X.509 certificates: reading, names, PEM, a store of trust anchors (through your allocator) and chain verification. You supply the time and the anchors; only reading the operating system's roots needs an operating system.
`tls.h`	The engine: available. The transport wrapper and `proven_tls_http_wrap`: hosted	TLS (1.3 and a cut-down 1.2) as a state machine with no I/O: you feed it the bytes that arrived and send the bytes it gives. A freestanding build must supply the random source and the clock in the config (`random`, `now`).
`url.h`, `http.h`	Available	URL parsing and percent-coding, and the HTTP/1.1 message codec. Pure text handling: no socket, no allocation. The transport is yours to supply - on a target with no sockets (wasm32, bare metal) the codec still parses and writes messages. `proven_http_date_format` takes a time you provide; the freestanding clock returns 0.
`http_auth.h`, `http_cookie.h`, `sse.h`	Available	Basic and Digest authentication values, a cookie jar, and a server-sent events parser. Text handling like the codec; the cookie jar allocates from the allocator you pass, and takes the current time as an argument rather than reading a clock. `proven_http_multipart_boundary` takes random bytes you

		supply, so it needs no entropy source.
`ws.h`	Available	The WebSocket codec: handshake values and checks, frame headers, masking, close payloads, and the message decoder. It uses SHA-1 and Base64 from the library and takes the key's and the masks' random bytes as arguments. A browser running a wasm32 module owns its own WebSocket; the codec's use there is a non-browser host, or the handshake arithmetic alone.
`encode.h`	Available	Hex and Base64 - pure computation, no OS.
`utf.h`	Available	Strict UTF-8 <-> UTF-16 transcoding - pure computation, no OS. `proven_utf8_append_to_u16str` is excluded with `PROVEN_NO_U16STR`; the rest works on raw `proven_u16` arrays.
`fmt.h`	Available without float	Current profile defines `PROVEN_FMT_NO_FLOAT`.
`scan.h`	Available	Scanner for memory views.
`float_parse.h`	Available	`proven_strtod`, `proven_parse_double_ascii` and `proven_parse_f64_ascii` all compile here: the decimal-to-binary64 engine is integer-only and needs no libc. The one difference is that the freestanding build does not set `errno` on overflow or underflow - the returned `proven_err_t` carries that instead, which is the value to check on a target with no `errno` at all. This is separate from `fmt.h`'s float **formatting**, which the profile does compile out.
`float_format.h`	Available without `fmt.h` integration	The binary64 formatter is integer-only and compiles, but the current profile defines `PROVEN_FMT_NO_FLOAT`, so {} will not

		render a float. Call the `float_format.h` entry points directly if you need digits on a target where you have decided the code size is worth it.
`time.h`	Limited	Core datetime formatting can compile; real PAL time is excluded.
`heap.h`	Stub	`proven_heap_allocator()` returns an invalid allocator.
`fs.h`, `stream.h`, `net.h`, `http_client.h`, `http_server.h`, `loop.h`, `http_event.h`, `ws_event.h`, `http_event_client.h`, `ws_conn.h`, `mmap.h`, `sysio.h`, `job.h`	Excluded	Require hosted PAL services.
`random.h`	Available	The generators and helpers are pure arithmetic. `proven_random_bytes` works here too, but only once you install an entropy source with `proven_random_set_source` - a board's TRNG, ring oscillator, or ADC noise floor. With none installed it returns **false** rather than falling back to a clock-seeded PRNG, which would look like success and be a security hole nothing reports. `proven_chacha_rng_seed_from_entropy` then turns the board's entropy into an endless cryptographic stream. `proven_random_u64` returns 0 with no source, silently; for a token or key use `proven_random_u64_checked`, which returns false.
`coro.h`	Available	Macro-only stackless coroutine support.
`panic.h`	Available	Override for target-specific trap/reset behavior.

4. Minimal static arena setup

There is no heap, so the memory comes from a static block and an arena is laid over it. The arena does not own that block: `alloc` bumps an offset, `free` is a no-op, and `reset` rewinds the whole thing at once.

This is the pattern the whole library was shaped to allow, and it is worth seeing why it works. Every function that can allocate takes a `proven_allocator_t` as a parameter, so code written for a

hosted build against `proven_heap_allocator()` runs here unchanged once you hand it an arena instead. Nothing in `u8str.h`, `array.h` or `map.h` knows or cares that the memory came from a `static` array in `.bss` rather than from `malloc`.

Three decisions to make when you size that block, none of which the library can make for you:

- **How big.** The arena cannot grow, so its size is a hard limit you are choosing at compile time. Too small and allocations start returning `PROVEN_ERR_NOMEM`; too large and you have spent RAM the rest of the firmware needed. This is the trade you are being asked to make explicitly rather than discovering it as heap fragmentation months later.
- **When to reset.** An arena's whole advantage is freeing everything at once. The natural points are a loop iteration, a received packet, a command - anywhere a batch of work has a clear end. Everything allocated during that batch dies at the reset, so **nothing may outlive it**.
- **Alignment of the backing array.** `alignas(max_align_t)` on the declaration, as below. Without it the array may start at an address that cannot hold a `double`, and the arena has nothing to fix that with.

Wrong - a view built in the arena that outlives the reset:

```
proven_u8str_view_t label = build_label(arena_alloc);   /* lives in the arena */
proven_arena_reset(&arena);
send(label);   /* wrong: those bytes are free space now, and the next allocation takes them */
```

On a hosted build this often survives long enough to look fine in testing. Here the next allocation gets the same bytes immediately, so it does not.

```
#include "proven/types.h"
#include "proven/memory.h"
#include "proven/arena.h"
#include "proven/array.h"
#include "proven/panic.h"

/* static storage: no OS, no heap, known at link time */
static alignas(PROVEN_MAX_ALIGN) proven_byte_t storage[4096];

proven_arena_t arena = proven_arena_create((proven_mem_mut_t){
    .ptr = storage,
    .size = sizeof storage,
});
proven_allocator_t arena_alloc = proven_arena_as_allocator(&arena);

proven_result_array_t r = PROVEN_ARRAY_INIT(arena_alloc, proven_i32, 16);
if (proven_is_ok(r.err)) {
    proven_array_t values = r.value;
    proven_err_t e = PROVEN_ARRAY_PUSH(&values, proven_i32, 42);
    (void)e;
    PROVEN_ARRAY_DESTROY(&values); /* arena free is a no-op */
}

proven_arena_destroy(&arena);      /* also a no-op: the caller owns `storage` */
```

Common mistake:

```
proven_allocator_t heap = proven_heap_allocator();
heap.alloc_fn(heap.ctx, 64, 8); /* wrong: heap is invalid in PROVEN_FREESTANDING */
```

Correct:

```
proven_allocator_t heap = proven_heap_allocator();
if (!proven_alloc_is_valid(heap)) {
    /* use an arena, pool, or target-provided allocator instead */
```

```
    proven_panic("no heap on this target");
}
```

5. Panic handler override

Why installing one is not optional here
On a hosted system a panic that traps is survivable: the process dies, the operating system cleans up, and something restarts it. On bare metal there is nothing underneath. A trap halts the core, and whatever the device was doing - holding a motor at speed, keeping a radio link, driving a heater - it is still doing when the CPU stops.

So the default handler is a placeholder for the one you must write, and what yours does is a product decision rather than a programming one. The usual shapes:

- **Put the hardware in a safe state, then halt.** Motors off, outputs to a known level, then spin or wait for a debugger. Correct for anything with a physical actuator.
- **Record and reset.** Write a reason code to a register or a reserved RAM area that survives reset, then trigger the watchdog. The next boot reports why the last one ended.
- **Halt loudly.** Blink an LED in a recognisable pattern. On a board with no console this is the entire diagnostic channel, and it is worth more than it sounds.

The rule from Chapter 1 section 6 applies with more force here: **the handler must not return.** If it does, `proven_arena_alloc_or_panic` proceeds with a block that was never allocated, and the failure moves from "the device stopped" to "the device is writing through a null pointer".

Wrong - a handler that logs and returns:

```
static void my_panic(const char *msg) {
    uart_write(msg);   /* wrong: this returns, and the caller uses a block it never got */
}
```

`proven_arena_alloc_or_panic()` and related panic paths call `proven_panic()`, which dispatches to the handler installed with `proven_set_panic_handler()`. The default handler traps.

The handler is a whole function, so this is a listing rather than a fragment:

```
#include "proven/panic.h"

static void my_panic(const char *msg) {
(void)msg;
/* optional: write msg to UART or crash log */
for (;;) {
/* or reset the MCU */
}
}

/* install once during startup; pass NULL to restore the default */
proven_set_panic_handler(my_panic);
```

A production panic handler should not return. If it returns, the allocation result that triggered the panic is not guaranteed to be valid.

6. Example Cortex-M compile command

```
arm-none-eabi-gcc \
-std=c23 \
-mcpu=cortex-m4 -mthumb \
-ffreestanding -nostdlib \
-DPROVEN_FREESTANDING \
-DPROVEN_FMT_NO_FLOAT \
-DPROVEN_NO_U16STR \
```

```
-Iinclude -Iplatform \
-c src/proven/memory.c -o memory.o
```

A real firmware build should compile the included freestanding modules listed above, compile your target startup code, and link with your linker script.

7. Example RISC-V ELF compile command

```
riscv64-elf-gcc \
-std=c23 \
-ffreestanding -nostdlib \
-DPROVEN_FREESTANDING \
-DPROVEN_FMT_NO_FLOAT \
-DPROVEN_NO_U16STR \
-Iinclude -Iplatform \
-c src/proven/arena.c -o arena.o
```

If your toolchain uses the `riscv64-unknown-elf-gcc` name, use that compiler instead. The project cross matrix checks both names when available.

8. Formatting in freestanding mode

What is left, and why that is usually enough
The formatter is portable computation - it builds bytes in a destination you supply - so almost all of it survives here. What is gone is one placeholder type: `double`.

That sounds like a big loss and rarely is. Correct float formatting means emitting the shortest decimal that reads back as the same value, on every input including subnormals, and doing that requires big-integer arithmetic and lookup tables. It is the largest single piece of code in the formatter. Most firmware formats integers, string views, characters and pointers - a sensor reading is a scaled integer, a status line is text - so the profile drops the float path by default and the binary is smaller for it.

What remains: `{}` for integers of every width, views, characters, booleans and pointers; the whole spec grammar (width, fill, alignment, base, sign); `PROVEN_ARG_OF` for your own types; and the scanner in full, including float *parsing*, which does not carry the same weight.

If you do need floats on a small target, the switch is `PROVEN_FMT_NO_FLOAT` and section 8a covers the capacity knob that goes with it. Measure the size change before deciding - on a part with 32 KB of flash it is not a rounding error.

Note the other consequence: **there is no `proven_println` here.** Formatting appends into a `proven_u8str_t` or a buffer, and getting those bytes to a UART or an RTT channel is your platform code, because a freestanding build has no standard output to assume.

Float formatting is disabled by `PROVEN_FMT_NO_FLOAT`. Integer and string-view formatting remain available.

Correct (`alloc` here is an arena allocator, as in section 4 - there is no heap):

```
proven_result_u8str_t r = proven_u8str_create(alloc, 32);
if (proven_is_ok(r.err)) {
    proven_fmt_result_t f = proven_u8str_append_fmt_grow(alloc, &r.value,
                                                         "value={}", PROVEN_ARG(123));
    (void)f;
    proven_u8str_destroy(alloc, &r.value);
}
```

Wrong:

```
proven_u8str_append_fmt_grow(alloc, &s, "{}", PROVEN_ARG(3.14));
/* wrong in the current freestanding profile: float args are excluded, so
   PROVEN_ARG has no _Generic association for double and this fails to compile */
```

8a. Tuning the float big-integer capacity

The exact decimal-to-binary64 fallback and the big-integer division helper use a fixed-size big integer whose capacity is set by `PROVEN_FLOAT_BIGINT_LIMBS` (default 160, in `proven/float_config.h`). This is the dominant factor in their stack usage: each big integer is `8 * PROVEN_FLOAT_BIGINT_LIMBS` bytes, and a division uses several plus 32-bit scratch.

Targets with tight stacks can lower it, for example:

```
-DPROVEN_FLOAT_BIGINT_LIMBS=48
```

The kept-significand cap (`PROVEN_FLOAT_MAX_SIGNIFICAND_DIGITS`) is derived from the capacity, so a smaller value never overflows: the parser still rounds correctly for inputs up to the derived number of significant digits and stays within one ULP for longer inputs. The Clinger and Eisel-Lemire fast paths do not use the big integer, so typical short inputs are unaffected. Keep the default (160) when exact rounding of pathological long decimals matters, since a binary64 rounding boundary can need up to 767 significant digits.

9. Avoid hosted APIs

Why this is a link error rather than a run-time surprise

The hosted modules are not compiled into a freestanding build at all. Calling one is therefore a **link error** - an undefined symbol, at build time, naming the function you should not have used.

That is the design working. The alternative, which many embedded libraries choose, is to provide stubs that return an error at run time; then a call to `proven_fs_open` on a microcontroller compiles, links, ships, and fails in the field on a path nobody exercised. Here it cannot get out of the build.

The practical consequence for your own code: if a module is shared between a hosted tool and firmware, the hosted-only calls need to be behind `#ifndef PROVEN_FREESTANDING`, and the link error tells you exactly which ones you missed.

Do not call these in the current freestanding profile:

```
proven_fs_open(...);
proven_println(...);
proven_env_get(...);
proven_mmap_create(...);
proven_job_system_init(...);
```

They require hosted PAL files that are intentionally excluded.

What your environment must provide

Exactly what any freestanding C program compiled by GCC or Clang needs, and nothing else: **`memcpy`, `memmove`, `memset` and `memcmp`**, which the compiler may call for copies and zeroing whatever the source says, and the compiler's own support library (`libgcc`: on Cortex-M the `__aeabi_*` division and soft-float helpers). Your newlib, picolibc or vendor HAL already has the four; if you have none, they are a few lines each. No `strlen`, no `malloc`, no stdio, no startup files.

No C library **headers** either. The library's freestanding sources include only the headers a freestanding compiler supplies itself (`<stddef.h>`, `<stdint.h>`, `<limits.h>` and their kind); where one calls `memcpy` it declares it. That was not true before v0.9.0: three files included `<string.h>` or `<math.h>`, which the embedded toolchains happen to ship and a bare Clang does not. The wasm32 target below is what found it, and what now keeps it found.

That is checked, not hoped for: `./nob cross` links every freestanding object of the library, for Cortex-M4 and RISC-V, with a program that supplies only those four functions, using `-nostdlib -nostartfiles -static -lgcc` (`tests/test_portability_freestanding_nocrt_link.c`). A static link fails on any unresolved symbol, so a clean link is the evidence. It is not run - there is no board here - so behaviour on the target is still yours to test.

wasm32 gets the same link with Clang and `wasm-ld`, which also refuses an undefined symbol. Its support library is compiler-rt's builtins rather than `libgcc`, and the one helper the library needs from it is `__multi3`, the 128-bit multiply. A build host need not have those builtins installed for wasm32, so the link program supplies that one function itself; your own wasm32 build links the builtins. wasm32 has no sockets and no filesystem here: it is the same freestanding subset as the other targets, not a hosted port.

10. Lifetime rules still apply

The section people skip, and why it costs more here
Freestanding does not relax any of the ownership rules in Chapter 0 section 5. If anything it sharpens them, for three reasons that all point the same way:

- **There is no allocator between you and the memory.** A dangling pointer into a heap block often survives for a while because the allocator has not handed that block out again. An arena hands the same bytes out on the very next allocation after a reset, so a stale view is overwritten immediately and deterministically.
- **There is nothing to catch you.** No MMU trap on a small target, no operating system to kill the process, no sanitizer in the field. A use-after-reset does not crash - it reads someone else's data and carries on, and the symptom appears somewhere unrelated.
- **The consequences are physical.** The wrong byte here can be a motor command or a radio packet.

The rules are the same ones you already know: a view is valid only while its owner is, an owned object is destroyed exactly once with the allocator that made it, and caller-owned state structs must not be copied. What changes is the margin for error, which is zero.

Freestanding does not relax container rules.

Wrong:

```
int *p = PROVEN_ARRAY_GET_MUT(&arr, int, 0);
PROVEN_ARRAY_PUSH(&arr, int, 9);
*p = 1; /* wrong: push may have moved the array */
```

Wrong:

```
proven_u8str_view_t key = make_stack_view();
PROVEN_MAP_SET_U8_BORROWED(&map, key, int, 1);
return; /* wrong if map survives after key bytes go out of scope */
```

11. Verification

Why this guide is checked rather than believed
Everything above is a claim about what compiles and links without an operating system, and claims like that rot quietly. One `#include <stdio.h>` added to a portable source file in an unrelated commit and the freestanding profile is broken - on a host build nothing would notice, because `stdio.h` is right there.

So the profile is built on every release rather than described. `./nob freestanding` compiles the portable core with `-ffreestanding` and the profile's defines, links the checks below **statically** on the

build host, and runs them. `./nob cross` then compiles the same profile for real embedded targets - Cortex-M, RISC-V and wasm32 among them - as a compile-only matrix.

The two checks are chosen for what they would catch:

- The **heap stub** check proves `proven_heap_allocator()` still exists and returns something `proven_alloc_is_valid` rejects. If somebody made it fail to link instead, trait-based code would stop compiling for bare metal - the failure section 3 explains this design avoids.
- The **compile check** builds a representative program against the profile, which catches a hosted header sneaking into a portable file - when the toolchain lacks that header. On the host and on the embedded toolchains `<string.h>` is always there, so it is the wasm32 target, which has no C library headers at all, that makes this check complete.

Neither runs on the actual hardware, and this guide does not claim otherwise: alignment faults, endianness and timing need a board. What is verified is the part that can be, on every build, rather than the whole thing on no build.

The project freestanding command builds and runs these local checks on the build host:

```
tests/test_portability_freestanding_heap_stub
tests/test_portability_compile_freestanding
tests/test_portability_compile_nofloat
tests/test_portability_compile_nou16str
tests/test_portability_freestanding
```

The cross command performs compile-only checks for available embedded compilers:

```
freestanding-arm-cortex-m4        arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb
freestanding-riscv64-elf          riscv64-elf-gcc
freestanding-riscv64-unknown-elf  riscv64-unknown-elf-gcc
freestanding-wasm32               clang --target=wasm32
```

Every target is reported as PASS, FAIL or SKIP. A missing toolchain is a SKIP - a gap in the evidence, not a pass; the freestanding targets may be skipped, the native host compilers and both Windows targets may not. Real compile or link failures fail the command. Runtime behavior still needs validation on the target or emulator.

Appendix A: `alias_xcv.h` Alias Index (Chapter 7)

This is a lookup table, not reading material. It is the one file in the manual with no prose to work through: 500-odd rows mapping every `xcv_` short name to its canonical `proven_` name. Skip it on a first read and come back when you meet an `xcv_` spelling you do not recognise, or when you are deciding whether to adopt the short names in your own code.

The file keeps its `manual-07` number because that number is a stable identifier used by links and by the build; its position in the reading order is "appendix", which is what this title says.

This appendix is generated from `include/proven/alias_xcv.h`. The alias layer is optional. The canonical `proven_` and `PROVEN_` names remain the source of truth for ABI, documentation, and tests.

Usage rule

Include aliases after the normal headers:

```
#include "proven.h"
#include "proven/alias_xcv.h"
```

Do not mix alias documentation with canonical API documentation. Use this index only as a spelling map.

Alias table

1333 aliases: 982 lowercase `xcv_` names and 351 uppercase `XCV_` names. The table is generated from `include/proven/alias_xcv.h`; `tests/test_docs_alias_completeness` fails the build if a public function has no alias. That gate compares the headers with each other, not with this appendix - the table had fallen 61 rows behind the header before it was regenerated, so regenerate it from the header rather than editing rows by hand.

There is deliberately no line-number column. It was wrong after every alias that got inserted above it, which is worse than having no column at all.

Alias macro	Expands to
XCV_ARG	PROVEN_ARG
XCV_ARG_CSTR	proven_arg_cstr
XCV_ARG_DATETIME	proven_arg_datetime
XCV_ARG_F64	proven_arg_f64
XCV_ARG_FN	PROVEN_ARG_FN(f)
XCV_ARG_I32	proven_arg_i32
XCV_ARG_I64	proven_arg_i64
XCV_ARG_NONE	proven_arg_none
XCV_ARG_PTR	proven_arg_ptr
XCV_ARG_STR_VIEW	proven_arg_str_view
XCV_ARG_U16	proven_arg_u16
XCV_ARG_U32	proven_arg_u32
XCV_ARG_U64	proven_arg_u64
XCV_ARRAY_DESTROY	PROVEN_ARRAY_DESTROY
XCV_ARRAY_GET	PROVEN_ARRAY_GET
XCV_ARRAY_GET_MUT	PROVEN_ARRAY_GET_MUT
XCV_ARRAY_INIT	PROVEN_ARRAY_INIT
XCV_ARRAY_IS_VALID	proven_array_is_valid
XCV_ARRAY_POP	PROVEN_ARRAY_POP

XCV_ARRAY_PUSH	PROVEN_ARRAY_PUSH
XCV_BROTLI_WINDOW_DEFAULT	PROVEN_BROTLI_WINDOW_DEFAULT
XCV_CERT_EKU_	PROVEN_CERT_EKU_
XCV_CERT_EKU_ANY	PROVEN_CERT_EKU_ANY
XCV_CERT_EKU_CLIENT_AUTH	PROVEN_CERT_EKU_CLIENT_AUTH
XCV_CERT_EKU_OTHER	PROVEN_CERT_EKU_OTHER
XCV_CERT_EKU_SERVER_AUTH	PROVEN_CERT_EKU_SERVER_AUTH
XCV_CERT_FAULT_ALGORITHM	PROVEN_CERT_FAULT_ALGORITHM
XCV_CERT_FAULT_BAD_SIGNATURE	PROVEN_CERT_FAULT_BAD_SIGNATURE
XCV_CERT_FAULT_CRITICAL_EXTENSION	PROVEN_CERT_FAULT_CRITICAL_EXTENSION
XCV_CERT_FAULT_EXPIRED	PROVEN_CERT_FAULT_EXPIRED
XCV_CERT_FAULT_MALFORMED	PROVEN_CERT_FAULT_MALFORMED
XCV_CERT_FAULT_NAME_CONSTRAINT	PROVEN_CERT_FAULT_NAME_CONSTRAINT
XCV_CERT_FAULT_NAME_MISMATCH	PROVEN_CERT_FAULT_NAME_MISMATCH
XCV_CERT_FAULT_NONE	PROVEN_CERT_FAULT_NONE
XCV_CERT_FAULT_NOT_A_CA	PROVEN_CERT_FAULT_NOT_A_CA
XCV_CERT_FAULT_NOT_YET_VALID	PROVEN_CERT_FAULT_NOT_YET_VALID
XCV_CERT_FAULT_NO_ISSUER	PROVEN_CERT_FAULT_NO_ISSUER
XCV_CERT_FAULT_PATH_LENGTH	PROVEN_CERT_FAULT_PATH_LENGTH
XCV_CERT_FAULT_PIN_MISMATCH	PROVEN_CERT_FAULT_PIN_MISMATCH
XCV_CERT_FAULT_TOO_DEEP	PROVEN_CERT_FAULT_TOO_DEEP
XCV_CERT_FAULT_USAGE	PROVEN_CERT_FAULT_USAGE
XCV_CERT_KEY_EC_P256	PROVEN_CERT_KEY_EC_P256
XCV_CERT_KEY_EC_P384	PROVEN_CERT_KEY_EC_P384
XCV_CERT_KEY_ED25519	PROVEN_CERT_KEY_ED25519
XCV_CERT_KEY_RSA	PROVEN_CERT_KEY_RSA
XCV_CERT_KEY_UNKNOWN	PROVEN_CERT_KEY_UNKNOWN
XCV_CERT_KU_	PROVEN_CERT_KU_
XCV_CERT_KU_CRL_SIGN	PROVEN_CERT_KU_CRL_SIGN
XCV_CERT_KU_DIGITAL_SIGNATURE	PROVEN_CERT_KU_DIGITAL_SIGNATURE
XCV_CERT_KU_KEY_AGREEMENT	PROVEN_CERT_KU_KEY_AGREEMENT
XCV_CERT_KU_KEY_CERT_SIGN	PROVEN_CERT_KU_KEY_CERT_SIGN
XCV_CERT_KU_KEY_ENCIPHERMENT	PROVEN_CERT_KU_KEY_ENCIPHERMENT
XCV_CERT_MAX_DEPTH	PROVEN_CERT_MAX_DEPTH
XCV_CERT_NAME_DNS	PROVEN_CERT_NAME_DNS
XCV_CERT_NAME_IP	PROVEN_CERT_NAME_IP
XCV_CERT_NAME_OTHER	PROVEN_CERT_NAME_OTHER
XCV_CERT_SIG_ECDSA	PROVEN_CERT_SIG_ECDSA
XCV_CERT_SIG_ED25519	PROVEN_CERT_SIG_ED25519
XCV_CERT_SIG_RSA_PKCS1	PROVEN_CERT_SIG_RSA_PKCS1
XCV_CERT_SIG_RSA_PSS	PROVEN_CERT_SIG_RSA_PSS
XCV_CERT_SIG_UNKNOWN	PROVEN_CERT_SIG_UNKNOWN
XCV_CERT_USE_CLIENT	PROVEN_CERT_USE_CLIENT
XCV_CERT_USE_SERVER	PROVEN_CERT_USE_SERVER
XCV_CKD_ADD	PROVEN_CKD_ADD

XCV_CKD_MUL	PROVEN_CKD_MUL
XCV_CKD_SUB	PROVEN_CKD_SUB
XCV_CONTAINER_OF	PROVEN_CONTAINER_OF
XCV_CORO_AWAIT	PROVEN_CORO_AWAIT
XCV_CORO_BEGIN	PROVEN_CORO_BEGIN
XCV_CORO_END	PROVEN_CORO_END
XCV_CORO_INIT	PROVEN_CORO_INIT
XCV_CORO_IS_DONE	PROVEN_CORO_IS_DONE
XCV_CORO_YIELD	PROVEN_CORO_YIELD
XCV_DEFAULT_ALIGNMENT	PROVEN_DEFAULT_ALIGNMENT
XCV_DEFLATE_FLUSH_FINISH	PROVEN_DEFLATE_FLUSH_FINISH
XCV_DEFLATE_FLUSH_NONE	PROVEN_DEFLATE_FLUSH_NONE
XCV_DEFLATE_FLUSH_SYNC	PROVEN_DEFLATE_FLUSH_SYNC
XCV_DEFLATE_GZIP	PROVEN_DEFLATE_GZIP
XCV_DEFLATE_RAW	PROVEN_DEFLATE_RAW
XCV_DEFLATE_ZLIB	PROVEN_DEFLATE_ZLIB
XCV_ERR_AGAIN	PROVEN_ERR_AGAIN
XCV_ERR_BUSY	PROVEN_ERR_BUSY
XCV_ERR_CANCELLED	PROVEN_ERR_CANCELLED
XCV_ERR_EOF	PROVEN_ERR_EOF
XCV_ERR_EXISTS	PROVEN_ERR_EXISTS
XCV_ERR_EXPIRED	PROVEN_ERR_EXPIRED
XCV_ERR_INVALID_ARG	PROVEN_ERR_INVALID_ARG
XCV_ERR_INVALID_ENCODING	PROVEN_ERR_INVALID_ENCODING
XCV_ERR_INVALID_FORMAT	PROVEN_ERR_INVALID_FORMAT
XCV_ERR_INVALID_STATE	PROVEN_ERR_INVALID_STATE
XCV_ERR_IO	PROVEN_ERR_IO
XCV_ERR_LAST	PROVEN_ERR_LAST
XCV_ERR_NAME_MISMATCH	PROVEN_ERR_NAME_MISMATCH
XCV_ERR_NEED_MORE	PROVEN_ERR_NEED_MORE
XCV_ERR_NOMEM	PROVEN_ERR_NOMEM
XCV_ERR_NOT_FOUND	PROVEN_ERR_NOT_FOUND
XCV_ERR_NOT_YET_VALID	PROVEN_ERR_NOT_YET_VALID
XCV_ERR_OUT_OF_BOUNDS	PROVEN_ERR_OUT_OF_BOUNDS
XCV_ERR_OVERFLOW	PROVEN_ERR_OVERFLOW
XCV_ERR_PERMISSION	PROVEN_ERR_PERMISSION
XCV_ERR_PROTOCOL	PROVEN_ERR_PROTOCOL
XCV_ERR_REFUSED	PROVEN_ERR_REFUSED
XCV_ERR_RESERVED_END	PROVEN_ERR_RESERVED_END
XCV_ERR_RESET	PROVEN_ERR_RESET
XCV_ERR_TIMEOUT	PROVEN_ERR_TIMEOUT
XCV_ERR_UNREACHABLE	PROVEN_ERR_UNREACHABLE
XCV_ERR_UNSUPPORTED	PROVEN_ERR_UNSUPPORTED
XCV_ERR_UNTRUSTED	PROVEN_ERR_UNTRUSTED
XCV_FS_APPEND	PROVEN_FS_APPEND

XCV_FS_CREATE	PROVEN_FS_CREATE
XCV_FS_LOCK_EXCLUSIVE	PROVEN_FS_LOCK_EXCLUSIVE
XCV_FS_LOCK_SHARED	PROVEN_FS_LOCK_SHARED
XCV_FS_LOCK_UNLOCK	PROVEN_FS_LOCK_UNLOCK
XCV_FS_PATH_SEP	PROVEN_FS_PATH_SEP
XCV_FS_PERM_DEFAULT	PROVEN_FS_PERM_DEFAULT
XCV_FS_PERM_GROUP_R	PROVEN_FS_PERM_GROUP_R
XCV_FS_PERM_GROUP_W	PROVEN_FS_PERM_GROUP_W
XCV_FS_PERM_GROUP_X	PROVEN_FS_PERM_GROUP_X
XCV_FS_PERM_OTHER_R	PROVEN_FS_PERM_OTHER_R
XCV_FS_PERM_OTHER_W	PROVEN_FS_PERM_OTHER_W
XCV_FS_PERM_OTHER_X	PROVEN_FS_PERM_OTHER_X
XCV_FS_PERM_OWNER_R	PROVEN_FS_PERM_OWNER_R
XCV_FS_PERM_OWNER_W	PROVEN_FS_PERM_OWNER_W
XCV_FS_PERM_OWNER_X	PROVEN_FS_PERM_OWNER_X
XCV_FS_READ	PROVEN_FS_READ
XCV_FS_TRUNC	PROVEN_FS_TRUNC
XCV_FS_TYPE_DIR	PROVEN_FS_TYPE_DIR
XCV_FS_TYPE_FILE	PROVEN_FS_TYPE_FILE
XCV_FS_TYPE_OTHER	PROVEN_FS_TYPE_OTHER
XCV_FS_WRITE	PROVEN_FS_WRITE
XCV_H2_CANCEL	PROVEN_H2_CANCEL
XCV_H2_COMPRESSION_ERROR	PROVEN_H2_COMPRESSION_ERROR
XCV_H2_CONNECT_ERROR	PROVEN_H2_CONNECT_ERROR
XCV_H2_CONTINUATION	PROVEN_H2_CONTINUATION
XCV_H2_DATA	PROVEN_H2_DATA
XCV_H2_DEFAULT_WINDOW	PROVEN_H2_DEFAULT_WINDOW
XCV_H2_ENHANCE_YOUR_CALM	PROVEN_H2_ENHANCE_YOUR_CALM
XCV_H2_EVENT_CLOSED	PROVEN_H2_EVENT_CLOSED
XCV_H2_EVENT_DATA	PROVEN_H2_EVENT_DATA
XCV_H2_EVENT_GOAWAY	PROVEN_H2_EVENT_GOAWAY
XCV_H2_EVENT_HEADERS	PROVEN_H2_EVENT_HEADERS
XCV_H2_EVENT_NONE	PROVEN_H2_EVENT_NONE
XCV_H2_EVENT_PONG	PROVEN_H2_EVENT_PONG
XCV_H2_EVENT_STREAM_RESET	PROVEN_H2_EVENT_STREAM_RESET
XCV_H2_EVENT_STREAM_WRITABLE	PROVEN_H2_EVENT_STREAM_WRITABLE
XCV_H2_EVENT_TRAILERS	PROVEN_H2_EVENT_TRAILERS
XCV_H2_FLAG_ACK	PROVEN_H2_FLAG_ACK
XCV_H2_FLAG_END_HEADERS	PROVEN_H2_FLAG_END_HEADERS
XCV_H2_FLAG_END_STREAM	PROVEN_H2_FLAG_END_STREAM
XCV_H2_FLAG_PADDED	PROVEN_H2_FLAG_PADDED
XCV_H2_FLAG_PRIORITY	PROVEN_H2_FLAG_PRIORITY
XCV_H2_FLOW_CONTROL_ERROR	PROVEN_H2_FLOW_CONTROL_ERROR
XCV_H2_FRAME_HEADER_SIZE	PROVEN_H2_FRAME_HEADER_SIZE
XCV_H2_FRAME_SIZE_ERROR	PROVEN_H2_FRAME_SIZE_ERROR

XCV_H2_GOAWAY	PROVEN_H2_GOAWAY
XCV_H2_HEADERS	PROVEN_H2_HEADERS
XCV_H2_HTTP_1_1_REQUIRED	PROVEN_H2_HTTP_1_1_REQUIRED
XCV_H2_INADEQUATE_SECURITY	PROVEN_H2_INADEQUATE_SECURITY
XCV_H2_INTERNAL_ERROR	PROVEN_H2_INTERNAL_ERROR
XCV_H2_MAX_MAX_FRAME_SIZE	PROVEN_H2_MAX_MAX_FRAME_SIZE
XCV_H2_MAX_WINDOW	PROVEN_H2_MAX_WINDOW
XCV_H2_MIN_MAX_FRAME_SIZE	PROVEN_H2_MIN_MAX_FRAME_SIZE
XCV_H2_NO_ERROR	PROVEN_H2_NO_ERROR
XCV_H2_PING	PROVEN_H2_PING
XCV_H2_PREFACE	PROVEN_H2_PREFACE
XCV_H2_PREFACE_SIZE	PROVEN_H2_PREFACE_SIZE
XCV_H2_PRIORITY	PROVEN_H2_PRIORITY
XCV_H2_PROTOCOL_ERROR	PROVEN_H2_PROTOCOL_ERROR
XCV_H2_PUSH_PROMISE	PROVEN_H2_PUSH_PROMISE
XCV_H2_REFUSED_STREAM	PROVEN_H2_REFUSED_STREAM
XCV_H2_RST_STREAM	PROVEN_H2_RST_STREAM
XCV_H2_SETTINGS	PROVEN_H2_SETTINGS
XCV_H2_SETTINGS_ENABLE_CONNECT_PROTOCOL	PROVEN_H2_SETTINGS_ENABLE_CONNECT_PROTOCOL
XCV_H2_SETTINGS_ENABLE_PUSH	PROVEN_H2_SETTINGS_ENABLE_PUSH
XCV_H2_SETTINGS_HEADER_TABLE_SIZE	PROVEN_H2_SETTINGS_HEADER_TABLE_SIZE
XCV_H2_SETTINGS_INITIAL_WINDOW_SIZE	PROVEN_H2_SETTINGS_INITIAL_WINDOW_SIZE
XCV_H2_SETTINGS_MAX_CONCURRENT_STREAMS	PROVEN_H2_SETTINGS_MAX_CONCURRENT_STREAMS
XCV_H2_SETTINGS_MAX_FRAME_SIZE	PROVEN_H2_SETTINGS_MAX_FRAME_SIZE
XCV_H2_SETTINGS_MAX_HEADER_LIST_SIZE	PROVEN_H2_SETTINGS_MAX_HEADER_LIST_SIZE
XCV_H2_SETTINGS_TIMEOUT	PROVEN_H2_SETTINGS_TIMEOUT
XCV_H2_STREAM_CLOSED	PROVEN_H2_STREAM_CLOSED
XCV_H2_WINDOW_UPDATE	PROVEN_H2_WINDOW_UPDATE
XCV_HMAC_MAX_SIZE	PROVEN_HMAC_MAX_SIZE
XCV_HMAC_SHA256	PROVEN_HMAC_SHA256
XCV_HMAC_SHA384	PROVEN_HMAC_SHA384
XCV_HMAC_SHA512	PROVEN_HMAC_SHA512
XCV_HPACK_DEFAULT_TABLE_BYTES	PROVEN_HPACK_DEFAULT_TABLE_BYTES
XCV_HPACK_ENTRY_OVERHEAD	PROVEN_HPACK_ENTRY_OVERHEAD
XCV_HTTP_BODY_CHUNKED	PROVEN_HTTP_BODY_CHUNKED
XCV_HTTP_BODY_LENGTH	PROVEN_HTTP_BODY_LENGTH
XCV_HTTP_BODY_NONE	PROVEN_HTTP_BODY_NONE
XCV_HTTP_BODY_UNTIL_CLOSE	PROVEN_HTTP_BODY_UNTIL_CLOSE
XCV_HTTP_BOUNDARY_SIZE	PROVEN_HTTP_BOUNDARY_SIZE
XCV_HTTP_CODING_BROTLI	PROVEN_HTTP_CODING_BROTLI
XCV_HTTP_CODING_DEFLATE	PROVEN_HTTP_CODING_DEFLATE
XCV_HTTP_CODING_GZIP	PROVEN_HTTP_CODING_GZIP
XCV_HTTP_CODING_IDENTITY	PROVEN_HTTP_CODING_IDENTITY
XCV_HTTP_CODING_OTHER	PROVEN_HTTP_CODING_OTHER
XCV_HTTP_CODING_ZSTD	PROVEN_HTTP_CODING_ZSTD

XCV_HTTP_CONNECT	PROVEN_HTTP_CONNECT
XCV_HTTP_COOKIE_MAX_SIZE	PROVEN_HTTP_COOKIE_MAX_SIZE
XCV_HTTP_DATE_SIZE	PROVEN_HTTP_DATE_SIZE
XCV_HTTP_DEFAULT_MAX_HEAD	PROVEN_HTTP_DEFAULT_MAX_HEAD
XCV_HTTP_DELETE	PROVEN_HTTP_DELETE
XCV_HTTP_DIGEST_MD5	PROVEN_HTTP_DIGEST_MD5
XCV_HTTP_DIGEST_MD5_SESS	PROVEN_HTTP_DIGEST_MD5_SESS
XCV_HTTP_DIGEST_SHA256	PROVEN_HTTP_DIGEST_SHA256
XCV_HTTP_DIGEST_SHA256_SESS	PROVEN_HTTP_DIGEST_SHA256_SESS
XCV_HTTP_EVENT_LENGTH_UNKNOWN	PROVEN_HTTP_EVENT_LENGTH_UNKNOWN
XCV_HTTP_GET	PROVEN_HTTP_GET
XCV_HTTP_HEAD	PROVEN_HTTP_HEAD
XCV_HTTP_LENGTH_UNKNOWN	PROVEN_HTTP_LENGTH_UNKNOWN
XCV_HTTP_MAX_CHUNK_LINE	PROVEN_HTTP_MAX_CHUNK_LINE
XCV_HTTP_MAX_TRAILER_BYTES	PROVEN_HTTP_MAX_TRAILER_BYTES
XCV_HTTP_METHOD_OTHER	PROVEN_HTTP_METHOD_OTHER
XCV_HTTP_OPTIONS	PROVEN_HTTP_OPTIONS
XCV_HTTP_PATCH	PROVEN_HTTP_PATCH
XCV_HTTP_POST	PROVEN_HTTP_POST
XCV_HTTP_PUT	PROVEN_HTTP_PUT
XCV_HTTP_RANGE_TO_END	PROVEN_HTTP_RANGE_TO_END
XCV_HTTP_TRACE	PROVEN_HTTP_TRACE
XCV_INDEX_NOT_FOUND	PROVEN_INDEX_NOT_FOUND
XCV_IS_OK	PROVEN_IS_OK
XCV_KEY_TYPE_INT	PROVEN_KEY_TYPE_INT
XCV_KEY_TYPE_U8_BORROWED	PROVEN_KEY_TYPE_U8_BORROWED
XCV_KEY_TYPE_U8_OWNED	PROVEN_KEY_TYPE_U8_OWNED
XCV_LIST_ENTRY	PROVEN_LIST_ENTRY
XCV_LIST_FOR_EACH	PROVEN_LIST_FOR_EACH
XCV_LIST_FOR_EACH_SAFE	PROVEN_LIST_FOR_EACH_SAFE
XCV_LIT	PROVEN_LIT
XCV_LOOP_BACKEND_AUTO	PROVEN_LOOP_BACKEND_AUTO
XCV_LOOP_BACKEND_IOCP	PROVEN_LOOP_BACKEND_IOCP
XCV_LOOP_BACKEND_SELECTOR	PROVEN_LOOP_BACKEND_SELECTOR
XCV_LOOP_BACKEND_URING	PROVEN_LOOP_BACKEND_URING
XCV_MAP_DESTROY	PROVEN_MAP_DESTROY
XCV_MAP_GET_INT	PROVEN_MAP_GET_INT
XCV_MAP_GET_MUT_INT	PROVEN_MAP_GET_MUT_INT
XCV_MAP_GET_MUT_U8_BORROWED	PROVEN_MAP_GET_MUT_U8_BORROWED
XCV_MAP_GET_U8_BORROWED	PROVEN_MAP_GET_U8_BORROWED
XCV_MAP_GET_U8_OWNED	PROVEN_MAP_GET_U8_OWNED
XCV_MAP_INIT_INT	PROVEN_MAP_INIT_INT
XCV_MAP_INIT_U8_BORROWED	PROVEN_MAP_INIT_U8_BORROWED
XCV_MAP_INIT_U8_OWNED	PROVEN_MAP_INIT_U8_OWNED
XCV_MAP_IS_VALID	proven_map_is_valid

XCV_MAP_REMOVE_INT	PROVEN_MAP_REMOVE_INT
XCV_MAP_REMOVE_U8_BORROWED	PROVEN_MAP_REMOVE_U8_BORROWED
XCV_MAP_REMOVE_U8_OWNED	PROVEN_MAP_REMOVE_U8_OWNED
XCV_MAP_SET_INT	PROVEN_MAP_SET_INT
XCV_MAP_SET_U8_BORROWED	PROVEN_MAP_SET_U8_BORROWED
XCV_MAP_SET_U8_OWNED	PROVEN_MAP_SET_U8_OWNED
XCV_MAP_SET_WITH_SCRATCH_INT	PROVEN_MAP_SET_WITH_SCRATCH_INT
XCV_MAP_SET_WITH_SCRATCH_U8_BORROWED	PROVEN_MAP_SET_WITH_SCRATCH_U8_BORROWED
XCV_MAX_ALIGN	PROVEN_MAX_ALIGN
XCV_MMAP_EXEC	PROVEN_MMAP_EXEC
XCV_MMAP_PRIVATE	PROVEN_MMAP_PRIVATE
XCV_MMAP_READ	PROVEN_MMAP_READ
XCV_MMAP_SHARED	PROVEN_MMAP_SHARED
XCV_MMAP_WRITE	PROVEN_MMAP_WRITE
XCV_NET_ADDR_TEXT_MAX	PROVEN_NET_ADDR_TEXT_MAX
XCV_NET_DONT_WAIT	PROVEN_NET_DONT_WAIT
XCV_NET_FAILED	PROVEN_NET_FAILED
XCV_NET_FAMILY_IPV4	PROVEN_NET_FAMILY_IPV4
XCV_NET_FAMILY_IPV6	PROVEN_NET_FAMILY_IPV6
XCV_NET_FAMILY_NONE	PROVEN_NET_FAMILY_NONE
XCV_NET_FAMILY_UNIX	PROVEN_NET_FAMILY_UNIX
XCV_NET_NO_DEADLINE	PROVEN_NET_NO_DEADLINE
XCV_NET_POLL_INLINE_MAX	PROVEN_NET_POLL_INLINE_MAX
XCV_NET_READABLE	PROVEN_NET_READABLE
XCV_NET_SELECTOR_EPOLL	PROVEN_NET_SELECTOR_EPOLL
XCV_NET_SELECTOR_KQUEUE	PROVEN_NET_SELECTOR_KQUEUE
XCV_NET_SELECTOR_IOCP	PROVEN_NET_SELECTOR_IOCP
XCV_NET_SELECTOR_POLL	PROVEN_NET_SELECTOR_POLL
XCV_NET_UNIX_PATH_MAX	PROVEN_NET_UNIX_PATH_MAX
XCV_NET_WRITABLE	PROVEN_NET_WRITABLE
XCV_OK	PROVEN_OK
XCV_RING_DESTROY	PROVEN_RING_DESTROY
XCV_RING_INIT	PROVEN_RING_INIT
XCV_RING_IS_VALID	proven_ring_is_valid
XCV_RING_POP	PROVEN_RING_POP
XCV_RING_PUSH	PROVEN_RING_PUSH
XCV_SCAN_ARG	PROVEN_SCAN_ARG(x)
XCV_SCAN_ARG_LONG	PROVEN_SCAN_ARG_LONG(ptr)
XCV_SCAN_ARG_TYPE_F64	PROVEN_SCAN_ARG_TYPE_F64
XCV_SCAN_ARG_TYPE_I32	PROVEN_SCAN_ARG_TYPE_I32
XCV_SCAN_ARG_TYPE_I64	PROVEN_SCAN_ARG_TYPE_I64
XCV_SCAN_ARG_TYPE_INT	PROVEN_SCAN_ARG_TYPE_INT
XCV_SCAN_ARG_TYPE_LLONG	PROVEN_SCAN_ARG_TYPE_LLONG
XCV_SCAN_ARG_TYPE_LONG	PROVEN_SCAN_ARG_TYPE_LONG
XCV_SCAN_ARG_TYPE_NONE	PROVEN_SCAN_ARG_TYPE_NONE

XCV_SCAN_ARG_TYPE_SHORT	PROVEN_SCAN_ARG_TYPE_SHORT
XCV_SCAN_ARG_TYPE_STR_VIEW	PROVEN_SCAN_ARG_TYPE_STR_VIEW
XCV_SCAN_ARG_TYPE_U32	PROVEN_SCAN_ARG_TYPE_U32
XCV_SCAN_ARG_TYPE_U64	PROVEN_SCAN_ARG_TYPE_U64
XCV_SCAN_ARG_TYPE_UINT	PROVEN_SCAN_ARG_TYPE_UINT
XCV_SCAN_ARG_TYPE_ULLONG	PROVEN_SCAN_ARG_TYPE_ULLONG
XCV_SCAN_ARG_TYPE_ULONG	PROVEN_SCAN_ARG_TYPE_ULONG
XCV_SCAN_ARG_TYPE_USHORT	PROVEN_SCAN_ARG_TYPE_USHORT
XCV_SCAN_ARG_ULONG	PROVEN_SCAN_ARG_ULONG(ptr)
XCV_SHA256_SIZE	PROVEN_SHA256_SIZE
XCV_SHA384_SIZE	PROVEN_SHA384_SIZE
XCV_SHA512_SIZE	PROVEN_SHA512_SIZE
XCV_TLS_CLIENT_AUTH_NONE	PROVEN_TLS_CLIENT_AUTH_NONE
XCV_TLS_CLIENT_AUTH_REQUEST	PROVEN_TLS_CLIENT_AUTH_REQUEST
XCV_TLS_CLIENT_AUTH_REQUIRE	PROVEN_TLS_CLIENT_AUTH_REQUIRE
XCV_TLS_LEGACY_CBC	PROVEN_TLS_LEGACY_CBC
XCV_TLS_LEGACY_DHE	PROVEN_TLS_LEGACY_DHE
XCV_TLS_LEGACY_NO_EXTENDED_MASTER_SECRET	PROVEN_TLS_LEGACY_NO_EXTENDED_MASTER_SECRET
XCV_TLS_LEGACY_RSA_KEY_EXCHANGE	PROVEN_TLS_LEGACY_RSA_KEY_EXCHANGE
XCV_TLS_SESSION_SIZE	PROVEN_TLS_SESSION_SIZE
XCV_TLS_VERIFY_CHAIN	PROVEN_TLS_VERIFY_CHAIN
XCV_TLS_VERIFY_PIN_ONLY	PROVEN_TLS_VERIFY_PIN_ONLY
XCV_TLS_VERSION_1_0	PROVEN_TLS_VERSION_1_0
XCV_TLS_VERSION_1_1	PROVEN_TLS_VERSION_1_1
XCV_TLS_VERSION_1_2	PROVEN_TLS_VERSION_1_2
XCV_TLS_VERSION_1_3	PROVEN_TLS_VERSION_1_3
XCV_U16_LIT	PROVEN_U16_LIT
XCV_VERSION_ENCODE	minor, patch) PROVEN_VERSION_ENCODE(major, minor, patch)
XCV_VERSION_MAJOR	PROVEN_VERSION_MAJOR
XCV_VERSION_MINOR	PROVEN_VERSION_MINOR
XCV_VERSION_NUM	PROVEN_VERSION_NUM
XCV_VERSION_PATCH	PROVEN_VERSION_PATCH
XCV_VERSION_STRING	PROVEN_VERSION_STRING
XCV_WS_ACCEPT_SIZE	PROVEN_WS_ACCEPT_SIZE
XCV_WS_BINARY	PROVEN_WS_BINARY
XCV_WS_CLOSE	PROVEN_WS_CLOSE
XCV_WS_CLOSE_ABNORMAL	PROVEN_WS_CLOSE_ABNORMAL
XCV_WS_CLOSE_GOING_AWAY	PROVEN_WS_CLOSE_GOING_AWAY
XCV_WS_CLOSE_INTERNAL_ERROR	PROVEN_WS_CLOSE_INTERNAL_ERROR
XCV_WS_CLOSE_INVALID_DATA	PROVEN_WS_CLOSE_INVALID_DATA
XCV_WS_CLOSE_NORMAL	PROVEN_WS_CLOSE_NORMAL
XCV_WS_CLOSE_NO_STATUS	PROVEN_WS_CLOSE_NO_STATUS
XCV_WS_CLOSE_POLICY	PROVEN_WS_CLOSE_POLICY
XCV_WS_CLOSE_PROTOCOL_ERROR	PROVEN_WS_CLOSE_PROTOCOL_ERROR
XCV_WS_CLOSE_TOO_BIG	PROVEN_WS_CLOSE_TOO_BIG

XCV_WS_CLOSE_UNSUPPORTED_DATA	PROVEN_WS_CLOSE_UNSUPPORTED_DATA
XCV_WS_CONTINUATION	PROVEN_WS_CONTINUATION
XCV_WS_DEFLATE_ANSWER_MAX	PROVEN_WS_DEFLATE_ANSWER_MAX
XCV_WS_DEFLATE_OFFER	PROVEN_WS_DEFLATE_OFFER
XCV_WS_DEFLATE_OFFER_PEER_NO_TAKEOVER	PROVEN_WS_DEFLATE_OFFER_PEER_NO_TAKEOVER
XCV_WS_DEFLATE_OFFER_TAKEOVER	PROVEN_WS_DEFLATE_OFFER_TAKEOVER
XCV_WS_EVENT_CLOSE	PROVEN_WS_EVENT_CLOSE
XCV_WS_EVENT_DATA	PROVEN_WS_EVENT_DATA
XCV_WS_EVENT_NONE	PROVEN_WS_EVENT_NONE
XCV_WS_EVENT_PING	PROVEN_WS_EVENT_PING
XCV_WS_EVENT_PONG	PROVEN_WS_EVENT_PONG
XCV_WS_KEY_SIZE	PROVEN_WS_KEY_SIZE
XCV_WS_MAX_CONTROL	PROVEN_WS_MAX_CONTROL
XCV_WS_MAX_FRAME_HEADER	PROVEN_WS_MAX_FRAME_HEADER
XCV_WS_PING	PROVEN_WS_PING
XCV_WS_PONG	PROVEN_WS_PONG
XCV_WS_TEXT	PROVEN_WS_TEXT
XCV_ZSTD_FLUSH_BLOCK	PROVEN_ZSTD_FLUSH_BLOCK
XCV_ZSTD_FLUSH_FINISH	PROVEN_ZSTD_FLUSH_FINISH
XCV_ZSTD_FLUSH_NONE	PROVEN_ZSTD_FLUSH_NONE
XCV_ZSTD_WINDOW_DEFAULT	PROVEN_ZSTD_WINDOW_DEFAULT
xcv_alloc_check_entry_t	proven_alloc_check_entry_t
xcv_alloc_check_live	proven_alloc_check_live
xcv_alloc_check_owns	proven_alloc_check_owns
xcv_alloc_check_t	proven_alloc_check_t
xcv_alloc_check_wrap	proven_alloc_check_wrap
xcv_alloc_checked	proven_alloc_checked
xcv_alloc_fn_t	proven_alloc_fn_t
xcv_alloc_is_valid	proven_alloc_is_valid
xcv_allocator_t	proven_allocator_t
xcv_arena_alloc	proven_arena_alloc
xcv_arena_alloc_aligned	proven_arena_alloc_aligned
xcv_arena_alloc_aligned_or_panic	proven_arena_alloc_aligned_or_panic
xcv_arena_alloc_or_panic	proven_arena_alloc_or_panic
xcv_arena_alloc_trait	proven_arena_alloc_trait
xcv_arena_as_allocator	proven_arena_as_allocator
xcv_arena_create	proven_arena_create
xcv_arena_destroy	proven_arena_destroy
xcv_arena_free_trait	proven_arena_free_trait
xcv_arena_realloc_aligned	proven_arena_realloc_aligned
xcv_arena_realloc_trait	proven_arena_realloc_trait
xcv_arena_reset	proven_arena_reset
xcv_arena_t	proven_arena_t
xcv_arg_bool	proven_arg_bool
xcv_arg_char	proven_arg_char

xcv_arg_cstr	proven_arg_cstr
xcv_arg_cstr_n	proven_arg_cstr_n
xcv_arg_custom	proven_arg_custom
xcv_arg_datetime	proven_arg_datetime
xcv_arg_f64	proven_arg_f64
xcv_arg_fn	proven_arg_fn
xcv_arg_i32	proven_arg_i32
xcv_arg_i64	proven_arg_i64
xcv_arg_identity	proven_arg_identity
xcv_arg_none	proven_arg_none
xcv_arg_ptr	proven_arg_ptr
xcv_arg_str_view	proven_arg_str_view
xcv_arg_t	proven_arg_t
xcv_arg_type_t	proven_arg_type_t
xcv_arg_u16	proven_arg_u16
xcv_arg_u32	proven_arg_u32
xcv_arg_u64	proven_arg_u64
xcv_arg_ucstr	proven_arg_ucstr
xcv_array_binary_search	proven_array_binary_search
xcv_array_clear	proven_array_clear
xcv_array_create	proven_array_create
xcv_array_destroy	proven_array_destroy
xcv_array_extend	proven_array_extend
xcv_array_get	proven_array_get
xcv_array_get_mut	proven_array_get_mut
xcv_array_insert	proven_array_insert
xcv_array_is_valid	proven_array_is_valid
xcv_array_linear_search	proven_array_linear_search
xcv_array_pop	proven_array_pop
xcv_array_push	proven_array_push
xcv_array_remove_at	proven_array_remove_at
xcv_array_reserve	proven_array_reserve
xcv_array_sort	proven_array_sort
xcv_array_swap_remove	proven_array_swap_remove
xcv_array_t	proven_array_t
xcv_array_truncate	proven_array_truncate
xcv_base64_decode	proven_base64_decode
xcv_base64_decoded_size	proven_base64_decoded_size
xcv_base64_encode	proven_base64_encode
xcv_base64_encoded_size	proven_base64_encoded_size
xcv_base64url_encode	proven_base64url_encode
xcv_brotli_decode	proven_brotli_decode
xcv_brotli_decode_all	proven_brotli_decode_all
xcv_brotli_decoder_create	proven_brotli_decoder_create
xcv_brotli_decoder_destroy	proven_brotli_decoder_destroy

xcv_brotli_decoder_reset	proven_brotli_decoder_reset
xcv_brotli_decoder_t	proven_brotli_decoder_t
xcv_buf_append	proven_buf_append
xcv_buf_create	proven_buf_create
xcv_buf_destroy	proven_buf_destroy
xcv_buf_t	proven_buf_t
xcv_byte_t	proven_byte_t
xcv_c_lib	proven_c_lib
xcv_cert_alt_name_next	proven_cert_alt_name_next
xcv_cert_fault_t	proven_cert_fault_t
xcv_cert_key_kind_t	proven_cert_key_kind_t
xcv_cert_key_sha256	proven_cert_key_sha256
xcv_cert_matches_host	proven_cert_matches_host
xcv_cert_name_kind_t	proven_cert_name_kind_t
xcv_cert_parse	proven_cert_parse
xcv_cert_sig_kind_t	proven_cert_sig_kind_t
xcv_cert_store	proven_cert_store
xcv_cert_store_add_der	proven_cert_store_add_der
xcv_cert_store_add_pem	proven_cert_store_add_pem
xcv_cert_store_add_system	proven_cert_store_add_system
xcv_cert_store_count	proven_cert_store_count
xcv_cert_store_create	proven_cert_store_create
xcv_cert_store_destroy	proven_cert_store_destroy
xcv_cert_store_t	proven_cert_store_t
xcv_cert_t	proven_cert_t
xcv_cert_use_t	proven_cert_use_t
xcv_cert_verify	proven_cert_verify
xcv_cert_verify_options_t	proven_cert_verify_options_t
xcv_cert_verify_result_t	proven_cert_verify_result_t
xcv_chacha_rng	proven_chacha_rng
xcv_chacha_rng_fill	proven_chacha_rng_fill
xcv_chacha_rng_next	proven_chacha_rng_next
xcv_chacha_rng_seed	proven_chacha_rng_seed
xcv_chacha_rng_seed_from_entropy	proven_chacha_rng_seed_from_entropy
xcv_compare_fn_t	proven_compare_fn_t
xcv_coro_t	proven_coro_t
xcv_crc32	proven_crc32
xcv_crc32_update	proven_crc32_update
xcv_cstr_len	proven_cstr_len
xcv_datetime_t	proven_datetime_t
xcv_deflate	proven_deflate
xcv_deflate_all	proven_deflate_all
xcv_deflate_bound	proven_deflate_bound
xcv_deflate_create	proven_deflate_create
xcv_deflate_destroy	proven_deflate_destroy

xcv_deflate_flush_t	proven_deflate_flush_t
xcv_deflate_format_t	proven_deflate_format_t
xcv_deflate_options_t	proven_deflate_options_t
xcv_deflate_reset	proven_deflate_reset
xcv_deflate_t	proven_deflate_t
xcv_entropy_fn	proven_entropy_fn
xcv_env_get	proven_env_get
xcv_eprint	proven_eprint
xcv_eprintln	proven_eprintln
xcv_err_t	proven_err_t
xcv_file_t	proven_file_t
xcv_float_format_f32_policy	proven_float_format_f32_policy
xcv_float_format_f64_policy	proven_float_format_f64_policy
xcv_float_format_options_fixed_default	proven_float_format_options_fixed_default
xcv_float_format_options_scientific	proven_float_format_options_scientific
xcv_float_format_options_shortest	proven_float_format_options_shortest
xcv_fmt_put	proven_fmt_put
xcv_fmt_to_writer_impl	proven_fmt_to_writer_impl
xcv_fprint	proven_fprint
xcv_free_fn_t	proven_free_fn_t
xcv_fs_chmod	proven_fs_chmod
xcv_fs_close	proven_fs_close
xcv_fs_copy	proven_fs_copy
xcv_fs_dir_close	proven_fs_dir_close
xcv_fs_dir_next	proven_fs_dir_next
xcv_fs_dir_open	proven_fs_dir_open
xcv_fs_entry_t	proven_fs_entry_t
xcv_fs_handle_t	proven_fs_handle_t
xcv_fs_is_absolute	proven_fs_is_absolute
xcv_fs_is_staging_name	proven_fs_is_staging_name
xcv_fs_link	proven_fs_link
xcv_fs_list	proven_fs_list
xcv_fs_list_destroy	proven_fs_list_destroy
xcv_fs_lock	proven_fs_lock
xcv_fs_lock_type_t	proven_fs_lock_type_t
xcv_fs_mkdir	proven_fs_mkdir
xcv_fs_mkdir_all	proven_fs_mkdir_all
xcv_fs_mode_t	proven_fs_mode_t
xcv_fs_open	proven_fs_open
xcv_fs_perms_t	proven_fs_perms_t
xcv_fs_pread	proven_fs_pread
xcv_fs_pwrite	proven_fs_pwrite
xcv_fs_read	proven_fs_read
xcv_fs_read_all	proven_fs_read_all
xcv_fs_read_all_bounded	proven_fs_read_all_bounded

xcv_fs_read_all_u8str	proven_fs_read_all_u8str
xcv_fs_remove	proven_fs_remove
xcv_fs_rename	proven_fs_rename
xcv_fs_rmdir	proven_fs_rmdir
xcv_fs_seek	proven_fs_seek
xcv_fs_size	proven_fs_size
xcv_fs_stat	proven_fs_stat
xcv_fs_stat_t	proven_fs_stat_t
xcv_fs_symlink	proven_fs_symlink
xcv_fs_sync	proven_fs_sync
xcv_fs_sync_dir	proven_fs_sync_dir
xcv_fs_tell	proven_fs_tell
xcv_fs_truncate	proven_fs_truncate
xcv_fs_type_t	proven_fs_type_t
xcv_fs_walk_close	proven_fs_walk_close
xcv_fs_walk_next	proven_fs_walk_next
xcv_fs_walk_open	proven_fs_walk_open
xcv_fs_write	proven_fs_write
xcv_fs_write_all	proven_fs_write_all
xcv_fs_write_file	proven_fs_write_file
xcv_fs_write_file_atomic	proven_fs_write_file_atomic
xcv_fs_write_file_durable	proven_fs_write_file_durable
xcv_h2_config_t	proven_h2_config_t
xcv_h2_conn_consumed	proven_h2_conn_consumed
xcv_h2_conn_create	proven_h2_conn_create
xcv_h2_conn_destroy	proven_h2_conn_destroy
xcv_h2_conn_feed	proven_h2_conn_feed
xcv_h2_conn_has_peer_settings	proven_h2_conn_has_peer_settings
xcv_h2_conn_is_closed	proven_h2_conn_is_closed
xcv_h2_conn_next_sensitive	proven_h2_conn_next_sensitive
xcv_h2_conn_output_sent	proven_h2_conn_output_sent
xcv_h2_conn_pending_output	proven_h2_conn_pending_output
xcv_h2_conn_ping	proven_h2_conn_ping
xcv_h2_conn_reset	proven_h2_conn_reset
xcv_h2_conn_send_data	proven_h2_conn_send_data
xcv_h2_conn_send_room	proven_h2_conn_send_room
xcv_h2_conn_send_trailers	proven_h2_conn_send_trailers
xcv_h2_conn_set_initial_window	proven_h2_conn_set_initial_window
xcv_h2_conn_set_max_streams	proven_h2_conn_set_max_streams
xcv_h2_conn_shutdown	proven_h2_conn_shutdown
xcv_h2_conn_stream_count	proven_h2_conn_stream_count
xcv_h2_conn_stream_set_user	proven_h2_conn_stream_set_user
xcv_h2_conn_stream_user	proven_h2_conn_stream_user
xcv_h2_conn_submit_request	proven_h2_conn_submit_request
xcv_h2_conn_submit_response	proven_h2_conn_submit_response

xcv_h2_conn_t	proven_h2_conn_t
xcv_h2_error_t	proven_h2_error_t
xcv_h2_event_kind_t	proven_h2_event_kind_t
xcv_h2_event_t	proven_h2_event_t
xcv_h2_frame_check	proven_h2_frame_check
xcv_h2_frame_content	proven_h2_frame_content
xcv_h2_frame_parse	proven_h2_frame_parse
xcv_h2_frame_t	proven_h2_frame_t
xcv_h2_frame_type_t	proven_h2_frame_type_t
xcv_h2_frame_write	proven_h2_frame_write
xcv_h2_header_buffer_size	proven_h2_header_buffer_size
xcv_h2_read_goaway	proven_h2_read_goaway
xcv_h2_read_rst_stream	proven_h2_read_rst_stream
xcv_h2_read_window_update	proven_h2_read_window_update
xcv_h2_request_head	proven_h2_request_head
xcv_h2_request_head_t	proven_h2_request_head_t
xcv_h2_response_head	proven_h2_response_head
xcv_h2_setting_id_t	proven_h2_setting_id_t
xcv_h2_setting_t	proven_h2_setting_t
xcv_h2_settings_next	proven_h2_settings_next
xcv_h2_trailers_check	proven_h2_trailers_check
xcv_h2_write_goaway	proven_h2_write_goaway
xcv_h2_write_ping	proven_h2_write_ping
xcv_h2_write_rst_stream	proven_h2_write_rst_stream
xcv_h2_write_settings	proven_h2_write_settings
xcv_h2_write_settings_ack	proven_h2_write_settings_ack
xcv_h2_write_window_update	proven_h2_write_window_update
xcv_hash_bytes	proven_hash_bytes
xcv_hash_keyed	proven_hash_keyed
xcv_heap_allocator	proven_heap_allocator
xcv_hex_decode	proven_hex_decode
xcv_hex_decoded_size	proven_hex_decoded_size
xcv_hex_encode	proven_hex_encode
xcv_hex_encoded_size	proven_hex_encoded_size
xcv_hkdf	proven_hkdf
xcv_hkdf_expand	proven_hkdf_expand
xcv_hkdf_extract	proven_hkdf_extract
xcv_hmac	proven_hmac
xcv_hmac_final	proven_hmac_final
xcv_hmac_hash_t	proven_hmac_hash_t
xcv_hmac_init	proven_hmac_init
xcv_hmac_size	proven_hmac_size
xcv_hmac_t	proven_hmac_t
xcv_hmac_update	proven_hmac_update
xcv_hpack_decode	proven_hpack_decode

xcv_hpack_decode_marked	proven_hpack_decode_marked
xcv_hpack_decoder	proven_hpack_decoder
xcv_hpack_decoder_create	proven_hpack_decoder_create
xcv_hpack_decoder_destroy	proven_hpack_decoder_destroy
xcv_hpack_decoder_set_max_table_bytes	proven_hpack_decoder_set_max_table_bytes
xcv_hpack_decoder_t	proven_hpack_decoder_t
xcv_hpack_decoder_table_bytes	proven_hpack_decoder_table_bytes
xcv_hpack_decoder_table_count	proven_hpack_decoder_table_count
xcv_hpack_decoder_table_entry	proven_hpack_decoder_table_entry
xcv_hpack_encode	proven_hpack_encode
xcv_hpack_encode_bound	proven_hpack_encode_bound
xcv_hpack_encoder	proven_hpack_encoder
xcv_hpack_encoder_create	proven_hpack_encoder_create
xcv_hpack_encoder_destroy	proven_hpack_encoder_destroy
xcv_hpack_encoder_set_peer_table_bytes	proven_hpack_encoder_set_peer_table_bytes
xcv_hpack_encoder_t	proven_hpack_encoder_t
xcv_hpack_huffman_decode	proven_hpack_huffman_decode
xcv_hpack_huffman_encode	proven_hpack_huffman_encode
xcv_hpack_huffman_size	proven_hpack_huffman_size
xcv_http_accept_weight	proven_http_accept_weight
xcv_http_accepts_coding	proven_http_accepts_coding
xcv_http_auth_offers	proven_http_auth_offers
xcv_http_basic_auth	proven_http_basic_auth
xcv_http_body_end	proven_http_body_end
xcv_http_body_feed	proven_http_body_feed
xcv_http_body_init	proven_http_body_init
xcv_http_body_kind_t	proven_http_body_kind_t
xcv_http_body_received	proven_http_body_received
xcv_http_body_t	proven_http_body_t
xcv_http_client_config_t	proven_http_client_config_t
xcv_http_client_create	proven_http_client_create
xcv_http_client_destroy	proven_http_client_destroy
xcv_http_client_finish	proven_http_client_finish
xcv_http_client_get	proven_http_client_get
xcv_http_client_read	proven_http_client_read
xcv_http_client_read_all	proven_http_client_read_all
xcv_http_client_request_t	proven_http_client_request_t
xcv_http_client_response_t	proven_http_client_response_t
xcv_http_client_send	proven_http_client_send
xcv_http_client_t	proven_http_client_t
xcv_http_client_upgrade	proven_http_client_upgrade
xcv_http_coding_t	proven_http_coding_t
xcv_http_content_coding	proven_http_content_coding
xcv_http_content_range_parse	proven_http_content_range_parse
xcv_http_cookie_jar_clear	proven_http_cookie_jar_clear

xcv_http_cookie_jar_count	proven_http_cookie_jar_count
xcv_http_cookie_jar_destroy	proven_http_cookie_jar_destroy
xcv_http_cookie_jar_header	proven_http_cookie_jar_header
xcv_http_cookie_jar_init	proven_http_cookie_jar_init
xcv_http_cookie_jar_store	proven_http_cookie_jar_store
xcv_http_cookie_jar_t	proven_http_cookie_jar_t
xcv_http_date_format	proven_http_date_format
xcv_http_date_parse	proven_http_date_parse
xcv_http_digest_algorithm_t	proven_http_digest_algorithm_t
xcv_http_digest_auth	proven_http_digest_auth
xcv_http_digest_challenge_parse	proven_http_digest_challenge_parse
xcv_http_digest_challenge_t	proven_http_digest_challenge_t
xcv_http_event_callbacks_t	proven_http_event_callbacks_t
xcv_http_event_client_callbacks_t	proven_http_event_client_callbacks_t
xcv_http_event_client_config_t	proven_http_event_client_config_t
xcv_http_event_client_create	proven_http_event_client_create
xcv_http_event_client_destroy	proven_http_event_client_destroy
xcv_http_event_client_requests	proven_http_event_client_requests
xcv_http_event_client_start	proven_http_event_client_start
xcv_http_event_client_t	proven_http_event_client_t
xcv_http_event_drained_fn	proven_http_event_drained_fn
xcv_http_event_request_abort	proven_http_event_request_abort
xcv_http_event_request_end	proven_http_event_request_end
xcv_http_event_request_options_t	proven_http_event_request_options_t
xcv_http_event_request_pause	proven_http_event_request_pause
xcv_http_event_request_protocol	proven_http_event_request_protocol
xcv_http_event_request_resume	proven_http_event_request_resume
xcv_http_event_request_set_user	proven_http_event_request_set_user
xcv_http_event_request_t	proven_http_event_request_t
xcv_http_event_request_user	proven_http_event_request_user
xcv_http_event_request_write	proven_http_event_request_write
xcv_http_event_server_adopt	proven_http_event_server_adopt
xcv_http_event_server_config_t	proven_http_event_server_config_t
xcv_http_event_server_connections	proven_http_event_server_connections
xcv_http_event_server_create	proven_http_event_server_create
xcv_http_event_server_destroy	proven_http_event_server_destroy
xcv_http_event_server_listen	proven_http_event_server_listen
xcv_http_event_server_set_max_streams	proven_http_event_server_set_max_streams
xcv_http_event_server_set_window	proven_http_event_server_set_window
xcv_http_event_server_shutdown	proven_http_event_server_shutdown
xcv_http_event_server_stop_listening	proven_http_event_server_stop_listening
xcv_http_event_server_t	proven_http_event_server_t
xcv_http_exchange_begin	proven_http_exchange_begin
xcv_http_exchange_compress	proven_http_exchange_compress
xcv_http_exchange_early	proven_http_exchange_early

xcv_http_exchange_end	proven_http_exchange_end
xcv_http_exchange_flush	proven_http_exchange_flush
xcv_http_exchange_peer	proven_http_exchange_peer
xcv_http_exchange_read	proven_http_exchange_read
xcv_http_exchange_request	proven_http_exchange_request
xcv_http_exchange_respond	proven_http_exchange_respond
xcv_http_exchange_t	proven_http_exchange_t
xcv_http_exchange_upgrade	proven_http_exchange_upgrade
xcv_http_exchange_write	proven_http_exchange_write
xcv_http_framing_t	proven_http_framing_t
xcv_http_handler_fn	proven_http_handler_fn
xcv_http_header_count	proven_http_header_count
xcv_http_header_find	proven_http_header_find
xcv_http_header_has_token	proven_http_header_has_token
xcv_http_header_t	proven_http_header_t
xcv_http_method_from_text	proven_http_method_from_text
xcv_http_method_t	proven_http_method_t
xcv_http_method_text	proven_http_method_text
xcv_http_multipart_boundary	proven_http_multipart_boundary
xcv_http_multipart_write_content_type	proven_http_multipart_write_content_type
xcv_http_multipart_write_end	proven_http_multipart_write_end
xcv_http_multipart_write_part	proven_http_multipart_write_part
xcv_http_multipart_write_part_end	proven_http_multipart_write_part_end
xcv_http_parse_request	proven_http_parse_request
xcv_http_parse_response	proven_http_parse_response
xcv_http_range_parse	proven_http_range_parse
xcv_http_reason_phrase	proven_http_reason_phrase
xcv_http_request_framing	proven_http_request_framing
xcv_http_request_keep_alive	proven_http_request_keep_alive
xcv_http_request_t	proven_http_request_t
xcv_http_response_framing	proven_http_response_framing
xcv_http_response_keep_alive	proven_http_response_keep_alive
xcv_http_response_t	proven_http_response_t
xcv_http_server_config_t	proven_http_server_config_t
xcv_http_server_connection_count	proven_http_server_connection_count
xcv_http_server_create	proven_http_server_create
xcv_http_server_destroy	proven_http_server_destroy
xcv_http_server_drained	proven_http_server_drained
xcv_http_server_listen	proven_http_server_listen
xcv_http_server_poll	proven_http_server_poll
xcv_http_server_run	proven_http_server_run
xcv_http_server_shutdown	proven_http_server_shutdown
xcv_http_server_shutting_down	proven_http_server_shutting_down
xcv_http_server_stop	proven_http_server_stop
xcv_http_server_t	proven_http_server_t

xcv_http_stream_abort	proven_http_stream_abort
xcv_http_stream_begin	proven_http_stream_begin
xcv_http_stream_buffered	proven_http_stream_buffered
xcv_http_stream_compress	proven_http_stream_compress
xcv_http_stream_early	proven_http_stream_early
xcv_http_stream_end	proven_http_stream_end
xcv_http_stream_flush	proven_http_stream_flush
xcv_http_stream_pause	proven_http_stream_pause
xcv_http_stream_peer	proven_http_stream_peer
xcv_http_stream_protocol	proven_http_stream_protocol
xcv_http_stream_respond	proven_http_stream_respond
xcv_http_stream_resume	proven_http_stream_resume
xcv_http_stream_set_user	proven_http_stream_set_user
xcv_http_stream_t	proven_http_stream_t
xcv_http_stream_user	proven_http_stream_user
xcv_http_stream_write	proven_http_stream_write
xcv_http_tls_wrap_fn	proven_http_tls_wrap_fn
xcv_http_write_chunk_begin	proven_http_write_chunk_begin
xcv_http_write_chunk_end	proven_http_write_chunk_end
xcv_http_write_content_range	proven_http_write_content_range
xcv_http_write_head_end	proven_http_write_head_end
xcv_http_write_header	proven_http_write_header
xcv_http_write_header_u64	proven_http_write_header_u64
xcv_http_write_last_chunk	proven_http_write_last_chunk
xcv_http_write_range	proven_http_write_range
xcv_http_write_request_line	proven_http_write_request_line
xcv_http_write_status_line	proven_http_write_status_line
xcv_i16	proven_i16
xcv_i32	proven_i32
xcv_i64	proven_i64
xcv_i8	proven_i8
xcv_inflate	proven_inflate
xcv_inflate_all	proven_inflate_all
xcv_inflate_create	proven_inflate_create
xcv_inflate_destroy	proven_inflate_destroy
xcv_inflate_reset	proven_inflate_reset
xcv_inflate_restart	proven_inflate_restart
xcv_inflate_t	proven_inflate_t
xcv_intptr_t	proven_intptr_t
xcv_is_ok	proven_is_ok
xcv_is_pow2	proven_is_pow2
xcv_job_execute_one	proven_job_execute_one
xcv_job_group_init	proven_job_group_init
xcv_job_group_pending	proven_job_group_pending
xcv_job_group_submit	proven_job_group_submit

xcv_job_group_t	proven_job_group_t
xcv_job_group_wait	proven_job_group_wait
xcv_job_submit	proven_job_submit
xcv_job_submit_ex	proven_job_submit_ex
xcv_job_sys	proven_job_sys
xcv_job_sys_t	proven_job_sys_t
xcv_job_system_close	proven_job_system_close
xcv_job_system_destroy	proven_job_system_destroy
xcv_job_system_init	proven_job_system_init
xcv_job_t	proven_job_t
xcv_key_type_t	proven_key_type_t
xcv_list_init	proven_list_init
xcv_list_insert_after	proven_list_insert_after
xcv_list_is_empty	proven_list_is_empty
xcv_list_node_t	proven_list_node_t
xcv_list_push_back	proven_list_push_back
xcv_list_remove	proven_list_remove
xcv_list_t	proven_list_t
xcv_loop_accept_cancel	proven_loop_accept_cancel
xcv_loop_accept_fn	proven_loop_accept_fn
xcv_loop_accept_t	proven_loop_accept_t
xcv_loop_accept	proven_loop_accept
xcv_loop_allocator	proven_loop_allocator
xcv_loop_attach	proven_loop_attach
xcv_loop_backend_t	proven_loop_backend_t
xcv_loop_backend	proven_loop_backend
xcv_loop_close	proven_loop_close
xcv_loop_connect	proven_loop_connect
xcv_loop_create_ex	proven_loop_create_ex
xcv_loop_create	proven_loop_create
xcv_loop_destroy	proven_loop_destroy
xcv_loop_fn	proven_loop_fn
xcv_loop_options_t	proven_loop_options_t
xcv_loop_poll	proven_loop_poll
xcv_loop_post	proven_loop_post
xcv_loop_recv_cancel	proven_loop_recv_cancel
xcv_loop_read	proven_loop_read
xcv_loop_recv	proven_loop_recv
xcv_loop_run	proven_loop_run
xcv_loop_scratch	proven_loop_scratch
xcv_loop_send_queued	proven_loop_send_queued
xcv_loop_send_wait	proven_loop_send_wait
xcv_loop_send	proven_loop_send
xcv_loop_sock_fns_t	proven_loop_sock_fns_t
xcv_loop_sock_t	proven_loop_sock_t

xcv_loop_stop	proven_loop_stop
xcv_loop_t	proven_loop_t
xcv_loop_timer_cancel	proven_loop_timer_cancel
xcv_loop_timer_is_set	proven_loop_timer_is_set
xcv_loop_timer_set	proven_loop_timer_set
xcv_loop_timer_t	proven_loop_timer_t
xcv_map_create	proven_map_create
xcv_map_create_trusted	proven_map_create_trusted
xcv_map_create_with_capacity	proven_map_create_with_capacity
xcv_map_destroy	proven_map_destroy
xcv_map_get	proven_map_get
xcv_map_get_mut	proven_map_get_mut
xcv_map_hash	proven_map_hash
xcv_map_is_valid	proven_map_is_valid
xcv_map_iter_init	proven_map_iter_init
xcv_map_iter_next	proven_map_iter_next
xcv_map_iter_t	proven_map_iter_t
xcv_map_key_t	proven_map_key_t
xcv_map_len	proven_map_len
xcv_map_remove	proven_map_remove
xcv_map_reserve	proven_map_reserve
xcv_map_set	proven_map_set
xcv_map_set_u8_owned	proven_map_set_u8_owned
xcv_map_set_with_scratch	proven_map_set_with_scratch
xcv_map_t	proven_map_t
xcv_md5	proven_md5
xcv_md5_final	proven_md5_final
xcv_md5_init	proven_md5_init
xcv_md5_to_hex	proven_md5_to_hex
xcv_md5_update	proven_md5_update
xcv_mem_align_up	proven_mem_align_up
xcv_mem_copy	proven_mem_copy
xcv_mem_equal_ct	proven_mem_equal_ct
xcv_mem_move	proven_mem_move
xcv_mem_mut_from_owned	proven_mem_mut_from_owned
xcv_mem_mut_slice_checked	proven_mem_mut_slice_checked
xcv_mem_mut_slice_unchecked	proven_mem_mut_slice_unchecked
xcv_mem_mut_t	proven_mem_mut_t
xcv_mem_t	proven_mem_t
xcv_mem_view_from_owned	proven_mem_view_from_owned
xcv_mem_view_from_u8	proven_mem_view_from_u8
xcv_mem_view_slice_checked	proven_mem_view_slice_checked
xcv_mem_view_slice_unchecked	proven_mem_view_slice_unchecked
xcv_mem_view_t	proven_mem_view_t
xcv_mem_wipe	proven_mem_wipe

xcv_memcmp	proven_memcmp
xcv_mmap_as_view	proven_mmap_as_view
xcv_mmap_create	proven_mmap_create
xcv_mmap_destroy	proven_mmap_destroy
xcv_mmap_flags_t	proven_mmap_flags_t
xcv_mmap_prot_t	proven_mmap_prot_t
xcv_mmap_sync	proven_mmap_sync
xcv_mmap_t	proven_mmap_t
xcv_net_accept	proven_net_accept
xcv_net_addr_any	proven_net_addr_any
xcv_net_addr_eq	proven_net_addr_eq
xcv_net_addr_format	proven_net_addr_format
xcv_net_addr_ipv4	proven_net_addr_ipv4
xcv_net_addr_ipv6	proven_net_addr_ipv6
xcv_net_addr_loopback	proven_net_addr_loopback
xcv_net_addr_parse	proven_net_addr_parse
xcv_net_addr_t	proven_net_addr_t
xcv_net_addr_unix	proven_net_addr_unix
xcv_net_close	proven_net_close
xcv_net_conn_handle	proven_net_conn_handle
xcv_net_conn_is_open	proven_net_conn_is_open
xcv_net_conn_local_addr	proven_net_conn_local_addr
xcv_net_conn_peer_addr	proven_net_conn_peer_addr
xcv_net_conn_set_nodelay	proven_net_conn_set_nodelay
xcv_net_conn_t	proven_net_conn_t
xcv_net_conn_transport	proven_net_conn_transport
xcv_net_connect	proven_net_connect
xcv_net_connect_finish	proven_net_connect_finish
xcv_net_connect_start	proven_net_connect_start
xcv_net_deadline_in	proven_net_deadline_in
xcv_net_deadline_t	proven_net_deadline_t
xcv_net_event_t	proven_net_event_t
xcv_net_family_t	proven_net_family_t
xcv_net_handle_t	proven_net_handle_t
xcv_net_listen	proven_net_listen
xcv_net_listener_addr	proven_net_listener_addr
xcv_net_listener_close	proven_net_listener_close
xcv_net_listener_handle	proven_net_listener_handle
xcv_net_listener_is_open	proven_net_listener_is_open
xcv_net_listener_t	proven_net_listener_t
xcv_net_pair	proven_net_pair
xcv_net_poll	proven_net_poll
xcv_net_poll_item_t	proven_net_poll_item_t
xcv_net_poll_scratch_size	proven_net_poll_scratch_size
xcv_net_poll_with	proven_net_poll_with

xcv_net_read	proven_net_read
xcv_net_ready_t	proven_net_ready_t
xcv_net_resolve	proven_net_resolve
xcv_net_selector_add	proven_net_selector_add
xcv_net_selector_count	proven_net_selector_count
xcv_net_selector_create	proven_net_selector_create
xcv_net_selector_create_poll	proven_net_selector_create_poll
xcv_net_selector_destroy	proven_net_selector_destroy
xcv_net_selector_kind	proven_net_selector_kind
xcv_net_selector_kind_t	proven_net_selector_kind_t
xcv_net_selector_modify	proven_net_selector_modify
xcv_net_selector_remove	proven_net_selector_remove
xcv_net_selector_t	proven_net_selector_t
xcv_net_selector_wait	proven_net_selector_wait
xcv_net_shutdown_write	proven_net_shutdown_write
xcv_net_udp_addr	proven_net_udp_addr
xcv_net_udp_close	proven_net_udp_close
xcv_net_udp_handle	proven_net_udp_handle
xcv_net_udp_is_open	proven_net_udp_is_open
xcv_net_udp_open	proven_net_udp_open
xcv_net_udp_recv_from	proven_net_udp_recv_from
xcv_net_udp_send_to	proven_net_udp_send_to
xcv_net_udp_t	proven_net_udp_t
xcv_net_waker_close	proven_net_waker_close
xcv_net_waker_drain	proven_net_waker_drain
xcv_net_waker_handle	proven_net_waker_handle
xcv_net_waker_open	proven_net_waker_open
xcv_net_waker_t	proven_net_waker_t
xcv_net_waker_wake	proven_net_waker_wake
xcv_net_write	proven_net_write
xcv_net_write_all	proven_net_write_all
xcv_panic	proven_panic
xcv_parse_double_ascii	proven_parse_double_ascii
xcv_parse_f64_ascii	proven_parse_f64_ascii
xcv_pem_next	proven_pem_next
xcv_pool_as_allocator	proven_pool_as_allocator
xcv_pool_destroy	proven_pool_destroy
xcv_pool_init	proven_pool_init
xcv_pool_t	proven_pool_t
xcv_print	proven_print
xcv_println	proven_println
xcv_ptrdiff_t	proven_ptrdiff_t
xcv_random_bytes	proven_random_bytes
xcv_random_set_source	proven_random_set_source
xcv_random_u64	proven_random_u64

xcv_random_u64_checked	proven_random_u64_checked
xcv_range_contains_ptr	proven_range_contains_ptr
xcv_reader_buffered	proven_reader_buffered
xcv_reader_from_file	proven_reader_from_file
xcv_reader_from_view	proven_reader_from_view
xcv_reader_is_valid	proven_reader_is_valid
xcv_reader_read	proven_reader_read
xcv_reader_read_line	proven_reader_read_line
xcv_realloc_fn_t	proven_realloc_fn_t
xcv_result_array_t	proven_result_array_t
xcv_result_buf_t	proven_result_buf_t
xcv_result_cstr_t	proven_result_cstr_t
xcv_result_f64_t	proven_result_f64_t
xcv_result_file_t	proven_result_file_t
xcv_result_i64_t	proven_result_i64_t
xcv_result_map_t	proven_result_map_t
xcv_result_mem_mut_t	proven_result_mem_mut_t
xcv_result_mem_view_t	proven_result_mem_view_t
xcv_result_mmap_t	proven_result_mmap_t
xcv_result_ring_t	proven_result_ring_t
xcv_result_size_t	proven_result_size_t
xcv_result_u16str_t	proven_result_u16str_t
xcv_result_u16str_view_t	proven_result_u16str_view_t
xcv_result_u64_t	proven_result_u64_t
xcv_result_u8str_t	proven_result_u8str_t
xcv_result_u8str_view_t	proven_result_u8str_view_t
xcv_ring_create	proven_ring_create
xcv_ring_destroy	proven_ring_destroy
xcv_ring_is_valid	proven_ring_is_valid
xcv_ring_pop	proven_ring_pop
xcv_ring_push	proven_ring_push
xcv_ring_t	proven_ring_t
xcv_rng_below	proven_rng_below
xcv_rng_f64	proven_rng_f64
xcv_rng_fill	proven_rng_fill
xcv_rng_is_valid	proven_rng_is_valid
xcv_rng_range	proven_rng_range
xcv_rng_shuffle	proven_rng_shuffle
xcv_rng_u64	proven_rng_u64
xcv_scan_arg_f64	proven_scan_arg_f64
xcv_scan_arg_i32	proven_scan_arg_i32
xcv_scan_arg_i64	proven_scan_arg_i64
xcv_scan_arg_identity	proven_scan_arg_identity
xcv_scan_arg_int	proven_scan_arg_int
xcv_scan_arg_llong	proven_scan_arg_llong

xcv_scan_arg_long	proven_scan_arg_long
xcv_scan_arg_none	proven_scan_arg_none
xcv_scan_arg_short	proven_scan_arg_short
xcv_scan_arg_str_view	proven_scan_arg_str_view
xcv_scan_arg_t	proven_scan_arg_t
xcv_scan_arg_type_t	proven_scan_arg_type_t
xcv_scan_arg_u32	proven_scan_arg_u32
xcv_scan_arg_u64	proven_scan_arg_u64
xcv_scan_arg_uint	proven_scan_arg_uint
xcv_scan_arg_ullong	proven_scan_arg_ullong
xcv_scan_arg_ulong	proven_scan_arg_ulong
xcv_scan_arg_ushort	proven_scan_arg_ushort
xcv_scan_f64	proven_scan_f64
xcv_scan_fmt	proven_scan_fmt
xcv_scan_fmt_cursor	proven_scan_fmt_cursor
xcv_scan_fmt_from_file	proven_scan_fmt_from_file
xcv_scan_fmt_from_stdin	proven_scan_fmt_from_stdin
xcv_scan_fmt_internal	proven_scan_fmt_internal
xcv_scan_fmt_internal_view	proven_scan_fmt_internal_view
xcv_scan_i64	proven_scan_i64
xcv_scan_i64_hex	proven_scan_i64_hex
xcv_scan_init	proven_scan_init
xcv_scan_skip_until	proven_scan_skip_until
xcv_scan_skip_until_number	proven_scan_skip_until_number
xcv_scan_skip_whitespace	proven_scan_skip_whitespace
xcv_scan_str	proven_scan_str
xcv_scan_t	proven_scan_t
xcv_scan_u64	proven_scan_u64
xcv_scan_u64_hex	proven_scan_u64_hex
xcv_set_panic_handler	proven_set_panic_handler
xcv_sha1	proven_sha1
xcv_sha1_final	proven_sha1_final
xcv_sha1_init	proven_sha1_init
xcv_sha1_to_hex	proven_sha1_to_hex
xcv_sha1_update	proven_sha1_update
xcv_sha256	proven_sha256
xcv_sha256_final	proven_sha256_final
xcv_sha256_init	proven_sha256_init
xcv_sha256_t	proven_sha256_t
xcv_sha256_to_hex	proven_sha256_to_hex
xcv_sha256_update	proven_sha256_update
xcv_sha384	proven_sha384
xcv_sha384_final	proven_sha384_final
xcv_sha384_init	proven_sha384_init
xcv_sha384_t	proven_sha384_t

xcv_sha384_update	proven_sha384_update
xcv_sha512	proven_sha512
xcv_sha512_final	proven_sha512_final
xcv_sha512_init	proven_sha512_init
xcv_sha512_t	proven_sha512_t
xcv_sha512_update	proven_sha512_update
xcv_size_t	proven_size_t
xcv_sse_event_t	proven_sse_event_t
xcv_sse_feed	proven_sse_feed
xcv_sse_init	proven_sse_init
xcv_sse_last_id	proven_sse_last_id
xcv_sse_t	proven_sse_t
xcv_strtod	proven_strtod
xcv_sysio_file_buffered	proven_sysio_file_buffered
xcv_sysio_lines_open	proven_sysio_lines_open
xcv_sysio_lines_t	proven_sysio_lines_t
xcv_sysio_out_t	proven_sysio_out_t
xcv_sysio_print_impl	proven_sysio_print_impl
xcv_sysio_read_line	proven_sysio_read_line
xcv_sysio_read_u16_line	proven_sysio_read_u16_line
xcv_sysio_scan_chunk_impl	proven_sysio_scan_chunk_impl
xcv_sysio_scanner_deinit	proven_sysio_scanner_deinit
xcv_sysio_scanner_init	proven_sysio_scanner_init
xcv_sysio_scanner_scan_impl	proven_sysio_scanner_scan_impl
xcv_sysio_std_t	proven_sysio_std_t
xcv_sysio_stderr	proven_sysio_stderr
xcv_sysio_stderr_writer	proven_sysio_stderr_writer
xcv_sysio_stdin	proven_sysio_stdin
xcv_sysio_stdin_lines	proven_sysio_stdin_lines
xcv_sysio_stdin_reader	proven_sysio_stdin_reader
xcv_sysio_stdin_u16_lines	proven_sysio_stdin_u16_lines
xcv_sysio_stdout	proven_sysio_stdout
xcv_sysio_stdout_buffered	proven_sysio_stdout_buffered
xcv_sysio_stdout_writer	proven_sysio_stdout_writer
xcv_sysio_u16_lines_open	proven_sysio_u16_lines_open
xcv_sysio_u16_lines_t	proven_sysio_u16_lines_t
xcv_text_encoding_t	proven_text_encoding_t
xcv_time_breakdown	proven_time_breakdown
xcv_time_locale_en	proven_time_locale_en
xcv_time_locale_t	proven_time_locale_t
xcv_time_monotonic_now	proven_time_monotonic_now
xcv_time_now	proven_time_now
xcv_time_now_datetime	proven_time_now_datetime
xcv_time_sleep	proven_time_sleep
xcv_time_t	proven_time_t

xcv_time_u16_fmt	proven_time_u16_fmt
xcv_time_u8_fmt	proven_time_u8_fmt
xcv_tls_alert_received	proven_tls_alert_received
xcv_tls_alert_sent	proven_tls_alert_sent
xcv_tls_alpn	proven_tls_alpn
xcv_tls_cipher_suite	proven_tls_cipher_suite
xcv_tls_client_auth_t	proven_tls_client_auth_t
xcv_tls_client_create	proven_tls_client_create
xcv_tls_close	proven_tls_close
xcv_tls_config	proven_tls_config
xcv_tls_config_create	proven_tls_config_create
xcv_tls_config_destroy	proven_tls_config_destroy
xcv_tls_config_t	proven_tls_config_t
xcv_tls_conn	proven_tls_conn
xcv_tls_conn_destroy	proven_tls_conn_destroy
xcv_tls_conn_t	proven_tls_conn_t
xcv_tls_feed	proven_tls_feed
xcv_tls_http_wrap	proven_tls_http_wrap
xcv_tls_is_established	proven_tls_is_established
xcv_tls_key_update	proven_tls_key_update
xcv_tls_now_fn	proven_tls_now_fn
xcv_tls_options_t	proven_tls_options_t
xcv_tls_output_sent	proven_tls_output_sent
xcv_tls_peer_certificate	proven_tls_peer_certificate
xcv_tls_peer_fault	proven_tls_peer_fault
xcv_tls_peer_key_sha256	proven_tls_peer_key_sha256
xcv_tls_pending_output	proven_tls_pending_output
xcv_tls_random_fn	proven_tls_random_fn
xcv_tls_read	proven_tls_read
xcv_tls_resumed	proven_tls_resumed
xcv_tls_server_create	proven_tls_server_create
xcv_tls_server_name	proven_tls_server_name
xcv_tls_session_t	proven_tls_session_t
xcv_tls_transport_client	proven_tls_transport_client
xcv_tls_transport_conn	proven_tls_transport_conn
xcv_tls_transport_server	proven_tls_transport_server
xcv_tls_verify_t	proven_tls_verify_t
xcv_tls_version	proven_tls_version
xcv_tls_write	proven_tls_write
xcv_transport_close	proven_transport_close
xcv_transport_is_valid	proven_transport_is_valid
xcv_transport_read	proven_transport_read
xcv_transport_reader	proven_transport_reader
xcv_transport_shutdown	proven_transport_shutdown
xcv_transport_stream_t	proven_transport_stream_t

xcv_transport_t	proven_transport_t
xcv_transport_write	proven_transport_write
xcv_transport_write_all	proven_transport_write_all
xcv_transport_writer	proven_transport_writer
xcv_u16	proven_u16
xcv_u16_reader_init	proven_u16_reader_init
xcv_u16_reader_read	proven_u16_reader_read
xcv_u16_reader_read_line	proven_u16_reader_read_line
xcv_u16_reader_t	proven_u16_reader_t
xcv_u16str_append	proven_u16str_append
xcv_u16str_append_grow	proven_u16str_append_grow
xcv_u16str_append_partial	proven_u16str_append_partial
xcv_u16str_as_ptr	proven_u16str_as_ptr
xcv_u16str_create	proven_u16str_create
xcv_u16str_create_from_view	proven_u16str_create_from_view
xcv_u16str_destroy	proven_u16str_destroy
xcv_u16str_len	proven_u16str_len
xcv_u16str_t	proven_u16str_t
xcv_u16str_view_t	proven_u16str_view_t
xcv_u32	proven_u32
xcv_u64	proven_u64
xcv_u8	proven_u8
xcv_u8str_append	proven_u8str_append
xcv_u8str_append_byte	proven_u8str_append_byte
xcv_u8str_append_fmt	proven_u8str_append_fmt
xcv_u8str_append_fmt_grow	proven_u8str_append_fmt_grow
xcv_u8str_append_fmt_trunc	proven_u8str_append_fmt_trunc
xcv_u8str_append_fmt_with_scratch	proven_u8str_append_fmt_with_scratch
xcv_u8str_append_grow	proven_u8str_append_grow
xcv_u8str_append_partial	proven_u8str_append_partial
xcv_u8str_as_cstr	proven_u8str_as_cstr
xcv_u8str_as_view	proven_u8str_as_view
xcv_u8str_borrow	proven_u8str_borrow
xcv_u8str_create	proven_u8str_create
xcv_u8str_create_from_view	proven_u8str_create_from_view
xcv_u8str_destroy	proven_u8str_destroy
xcv_u8str_fmt_internal	proven_u8str_fmt_internal
xcv_u8str_insert	proven_u8str_insert
xcv_u8str_insert_grow	proven_u8str_insert_grow
xcv_u8str_is_valid	proven_u8str_is_valid
xcv_u8str_mut_t	proven_u8str_mut_t
xcv_u8str_remove	proven_u8str_remove
xcv_u8str_replace_at	proven_u8str_replace_at
xcv_u8str_replace_at_grow	proven_u8str_replace_at_grow
xcv_u8str_replace_first	proven_u8str_replace_first

xcv_u8str_reserve	proven_u8str_reserve
xcv_u8str_reset	proven_u8str_reset
xcv_u8str_t	proven_u8str_t
xcv_u8str_view_cmp	proven_u8str_view_cmp
xcv_u8str_view_cmp_ptr	proven_u8str_view_cmp_ptr
xcv_u8str_view_contains	proven_u8str_view_contains
xcv_u8str_view_ends_with	proven_u8str_view_ends_with
xcv_u8str_view_eq	proven_u8str_view_eq
xcv_u8str_view_eq_ascii_ci	proven_u8str_view_eq_ascii_ci
xcv_u8str_view_find	proven_u8str_view_find
xcv_u8str_view_find_last	proven_u8str_view_find_last
xcv_u8str_view_from_bytes	proven_u8str_view_from_bytes
xcv_u8str_view_from_cstr	proven_u8str_view_from_cstr
xcv_u8str_view_is_well_formed	proven_u8str_view_is_well_formed
xcv_u8str_view_remove_prefix	proven_u8str_view_remove_prefix
xcv_u8str_view_remove_suffix	proven_u8str_view_remove_suffix
xcv_u8str_view_slice	proven_u8str_view_slice
xcv_u8str_view_split	proven_u8str_view_split
xcv_u8str_view_split_next	proven_u8str_view_split_next
xcv_u8str_view_split_t	proven_u8str_view_split_t
xcv_u8str_view_starts_with	proven_u8str_view_starts_with
xcv_u8str_view_starts_with_ascii_ci	proven_u8str_view_starts_with_ascii_ci
xcv_u8str_view_t	proven_u8str_view_t
xcv_u8str_view_to_cstr	proven_u8str_view_to_cstr
xcv_u8str_view_trim	proven_u8str_view_trim
xcv_u8str_view_trim_end	proven_u8str_view_trim_end
xcv_u8str_view_trim_start	proven_u8str_view_trim_start
xcv_uintptr_align_up	proven_uintptr_align_up
xcv_uintptr_t	proven_uintptr_t
xcv_url_default_port	proven_url_default_port
xcv_url_effective_port	proven_url_effective_port
xcv_url_encode_component	proven_url_encode_component
xcv_url_encode_path	proven_url_encode_path
xcv_url_encoded_size_max	proven_url_encoded_size_max
xcv_url_form_append	proven_url_form_append
xcv_url_form_decode	proven_url_form_decode
xcv_url_form_encode	proven_url_form_encode
xcv_url_parse	proven_url_parse
xcv_url_path_resolve	proven_url_path_resolve
xcv_url_percent_decode	proven_url_percent_decode
xcv_url_query_iter	proven_url_query_iter
xcv_url_query_iter_t	proven_url_query_iter_t
xcv_url_query_next	proven_url_query_next
xcv_url_resolve	proven_url_resolve
xcv_url_scheme_is	proven_url_scheme_is

xcv_url_split_target	proven_url_split_target
xcv_url_t	proven_url_t
xcv_utf16_append_to_u8str	proven_utf16_append_to_u8str
xcv_utf16_to_utf8	proven_utf16_to_utf8
xcv_utf16_to_utf8_partial	proven_utf16_to_utf8_partial
xcv_utf16_to_utf8_size	proven_utf16_to_utf8_size
xcv_utf8_append_to_u16str	proven_utf8_append_to_u16str
xcv_utf8_char_t	proven_utf8_char_t
xcv_utf8_decode_next	proven_utf8_decode_next
xcv_utf8_encode_one	proven_utf8_encode_one
xcv_utf8_validate	proven_utf8_validate
xcv_utf8_to_utf16	proven_utf8_to_utf16
xcv_utf8_to_utf16_partial	proven_utf8_to_utf16_partial
xcv_utf8_to_utf16_size	proven_utf8_to_utf16_size
xcv_utf_step_t	proven_utf_step_t
xcv_writer_buffered	proven_writer_buffered
xcv_writer_flush	proven_writer_flush
xcv_writer_from_buffer	proven_writer_from_buffer
xcv_writer_from_file	proven_writer_from_file
xcv_writer_from_u8str	proven_writer_from_u8str
xcv_writer_is_valid	proven_writer_is_valid
xcv_writer_write	proven_writer_write
xcv_writer_write_bom	proven_writer_write_bom
xcv_writer_write_partial	proven_writer_write_partial
xcv_writer_write_str	proven_writer_write_str
xcv_writer_write_u16	proven_writer_write_u16
xcv_ws_accept_key	proven_ws_accept_key
xcv_ws_check_request	proven_ws_check_request
xcv_ws_check_response	proven_ws_check_response
xcv_ws_check_response_deflate	proven_ws_check_response_deflate
xcv_ws_check_response_peer_no_takeover	proven_ws_check_response_peer_no_takeover
xcv_ws_check_response_takeover	proven_ws_check_response_takeover
xcv_ws_close_code_for	proven_ws_close_code_for
xcv_ws_close_code_is_valid	proven_ws_close_code_is_valid
xcv_ws_close_parse	proven_ws_close_parse
xcv_ws_close_write	proven_ws_close_write
xcv_ws_conn_accept	proven_ws_conn_accept
xcv_ws_conn_close	proven_ws_conn_close
xcv_ws_conn_close_code	proven_ws_conn_close_code
xcv_ws_conn_close_reason	proven_ws_conn_close_reason
xcv_ws_conn_compressed	proven_ws_conn_compressed
xcv_ws_conn_config_t	proven_ws_conn_config_t
xcv_ws_conn_connect	proven_ws_conn_connect
xcv_ws_conn_destroy	proven_ws_conn_destroy
xcv_ws_conn_open	proven_ws_conn_open

xcv_ws_conn_ping	proven_ws_conn_ping
xcv_ws_conn_pong_count	proven_ws_conn_pong_count
xcv_ws_conn_protocol	proven_ws_conn_protocol
xcv_ws_conn_receive	proven_ws_conn_receive
xcv_ws_conn_send_binary	proven_ws_conn_send_binary
xcv_ws_conn_send_part	proven_ws_conn_send_part
xcv_ws_conn_send_text	proven_ws_conn_send_text
xcv_ws_conn_set_compress	proven_ws_conn_set_compress
xcv_ws_conn_t	proven_ws_conn_t
xcv_ws_decoder_allow_compressed	proven_ws_decoder_allow_compressed
xcv_ws_decoder_feed	proven_ws_decoder_feed
xcv_ws_decoder_init	proven_ws_decoder_init
xcv_ws_decoder_t	proven_ws_decoder_t
xcv_ws_deflate_accept	proven_ws_deflate_accept
xcv_ws_deflate_accept_peer_no_takeover	proven_ws_deflate_accept_peer_no_takeover
xcv_ws_deflate_accept_takeover	proven_ws_deflate_accept_takeover
xcv_ws_deflate_terms_t	proven_ws_deflate_terms_t
xcv_ws_event_accept	proven_ws_event_accept
xcv_ws_event_callbacks_t	proven_ws_event_callbacks_t
xcv_ws_event_config_t	proven_ws_event_config_t
xcv_ws_event_connect	proven_ws_event_connect
xcv_ws_event_kind_t	proven_ws_event_kind_t
xcv_ws_event_t	proven_ws_event_t
xcv_ws_frame_parse	proven_ws_frame_parse
xcv_ws_frame_t	proven_ws_frame_t
xcv_ws_frame_write	proven_ws_frame_write
xcv_ws_make_key	proven_ws_make_key
xcv_ws_mask	proven_ws_mask
xcv_ws_message_t	proven_ws_message_t
xcv_ws_opcode_t	proven_ws_opcode_t
xcv_ws_check_origin	proven_ws_check_origin
xcv_ws_request_offers	proven_ws_request_offers
xcv_ws_stream_abort	proven_ws_stream_abort
xcv_ws_stream_buffered	proven_ws_stream_buffered
xcv_ws_stream_close	proven_ws_stream_close
xcv_ws_stream_compressed	proven_ws_stream_compressed
xcv_ws_stream_pause	proven_ws_stream_pause
xcv_ws_stream_peer	proven_ws_stream_peer
xcv_ws_stream_ping	proven_ws_stream_ping
xcv_ws_stream_resume	proven_ws_stream_resume
xcv_ws_stream_send	proven_ws_stream_send
xcv_ws_stream_send_piece	proven_ws_stream_send_piece
xcv_ws_stream_set_compress	proven_ws_stream_set_compress
xcv_ws_stream_set_user	proven_ws_stream_set_user
xcv_ws_stream_t	proven_ws_stream_t

xcv_ws_stream_user	proven_ws_stream_user
xcv_xoshiro256ss_next	proven_xoshiro256ss_next
xcv_xoshiro256ss_rng	proven_xoshiro256ss_rng
xcv_xoshiro256ss_seed	proven_xoshiro256ss_seed
xcv_zstd_decode	proven_zstd_decode
xcv_zstd_decode_all	proven_zstd_decode_all
xcv_zstd_decoder_create	proven_zstd_decoder_create
xcv_zstd_decoder_destroy	proven_zstd_decoder_destroy
xcv_zstd_decoder_reset	proven_zstd_decoder_reset
xcv_zstd_decoder_t	proven_zstd_decoder_t
xcv_zstd_encode	proven_zstd_encode
xcv_zstd_encode_all	proven_zstd_encode_all
xcv_zstd_encode_bound	proven_zstd_encode_bound
xcv_zstd_encoder_create	proven_zstd_encoder_create
xcv_zstd_encoder_destroy	proven_zstd_encoder_destroy
xcv_zstd_encoder_options_t	proven_zstd_encoder_options_t
xcv_zstd_encoder_reset	proven_zstd_encoder_reset
xcv_zstd_encoder_t	proven_zstd_encoder_t
xcv_zstd_flush_t	proven_zstd_flush_t